

数值分析（第 5 版）：Markdown 与 LaTeX 核对版

李庆扬、王能超、易大义。原扫描 322 页完整转录，经逐页 agent 核验和独立交叉复核。原书疑误保留，并附明确校注；来源页码指原 PDF，排版页码另计。

[章节索引](#)；[疑误与条件校注](#)。

原书 PDF 页 1；无印刷页码。

[查看原页](#)

普通高等教育“十三五”规划教材

普通高等院校数学精品教材

十三五

获教育部高等学校优秀教材二等奖

获全国优秀畅销书奖

数值分析

第 5 版

李庆扬 王能超 易大义 编

- 强调基本原理、基本理论，夯实基本素质
- 注重基本方法和技巧，提高应用能力
- 阐述严谨，脉络分明，深入浅出
- 反复锤炼，不断更新，长销 30 余年

华中科技大学出版社

<http://www.hustp.com>

版式记录 (agent 补充): 本页为封面, 无印刷页码。“十三五”也出现在封面徽标及背景装饰中; 完整图形与字样见原页。



普通高等教育“十三五”规划教材
普通高等院校数学精品教材

获教育部高等学校优秀教材二等奖
获全国优秀畅销书奖

数值分析 第5版

李庆扬 王能超 易大义 编

- 强调基本原理、基本理论，夯实基本素质
- 注重基本方法和技巧，提高应用能力
- 阐述严谨，脉络分明，深入浅出
- 反复锤炼，不断更新，长销30余年



华中科技大学出版社

<http://www.hustp.com>

原书 PDF 页 2；无印刷页码。

[查看原页](#)

普通高等教育“十三五”规划教材
普通高等院校数学精品教材

数值分析（第 5 版）

——数值算法分析与高效算法设计

李庆扬 王能超 易大义 编

华中科技大学出版社

中国·武汉

原书 PDF 页 3；无印刷页码。

[查看原页](#)

内容提要

本书是为理工科院校各专业普遍开设的“数值分析”课程而编写的教材。其上篇内容包括插值与逼近、数值积分与数值微分、常微分方程与线性方程组的数值解法、矩阵的特征值与特征向量计算等。每章附有习题并在书末给出部分答案。

本书下篇（高效算法设计）以讲座形式介绍快速算法、并行算法与加速算法方面的几个典型案例，力图普及推广超级计算方面的基础知识。全书阐述严谨，脉络分明，深入浅出，便于教学。

本书可作为理工科院校应用数学、力学、物理、计算机等专业的教材，也可供从事科学计算的科技工作者参考。

图书在版编目（CIP）数据

数值分析 / 李庆扬, 王能超, 易大义编. —5 版. —武汉: 华中科技大学出版社, 2018.4
ISBN 978-7-5680-3946-8

I. ①数... II. ①李... ②王... ③易... III. ①数值分析-高等学校-教材 IV. ① O241

中国版本图书馆 CIP 数据核字 (2018) 第 060527 号

数值分析（第 5 版）

Shuzhi Fenxi

李庆扬 王能超 易大义 编

项目	内容
策划编辑	王汉江
责任编辑	王汉江
封面设计	原色设计
责任校对	张会军
责任监印	周治超
出版发行	华中科技大学出版社（中国·武汉）
地址	武汉市东湖新技术开发区华工科技园
电话	(027)81321913
邮编	430223
录排	武汉市洪山区佳年华文印部
印刷	武汉华工鑫宏印务有限公司
开本	710 mm × 1000 mm 1/16
印张	20
字数	415 千字
版次	2018 年 4 月第 5 版第 1 次印刷
定价	39.80 元



华中出版

本书若有印装质量问题，请向出版社营销中心调换
全国免费服务热线：400-6679-118 竭诚为您服务
版权所有 侵权必究

原书 PDF 页 4; 无印刷页码。

[查看原页](#)

第 5 版前言

本书于 1981 年由华中科技大学出版社出版，至今已有 37 年。本书 1988 年获国家教委优秀教材二等奖，在国内为许多高校所选用。

今天，数值计算已进入超级计算的新时代，科技革命迅猛发展的新形势迫切要求普及推广高性能计算方面的新知识，鉴于这一认识本书推出第 5 版。

作为高效算法设计的关键技术，二分演化技术具有深邃的文化内涵，其设计思想新奇而玄妙，这方面内容可能尚未为人们所熟悉，笔者深信它处于算法设计学的前沿，因此选取快速算法设计、并行算法设计和加速算法设计方面的几个典型案例，汇集成讲座资料作为本书第 10 ~ 13 章，奉献给立志于从事高性能计算的读者参考。

本书中的第 10 ~ 13 章（讲座资料）由王能超撰写，错误与不当之处请读者不吝指正。

本书的再版，得到华中科技大学出版社的鼎力支持，在此表示衷心的感谢！

作者

2018 年 1 月

原书 PDF 页 5；无印刷页码。

[查看原页](#)

第 2 版前言

1980 年 7 月在大连召开的工科院校“应用数学专业教学学术会议”，根据教育部直属工科院校“应用数学专业教学计划”制定了“数值分析”课大纲，并决定由清华大学、华中工学院、浙江大学合编试用教材。本书就是根据这次会议的决定编写的。全书共分 9 章，第 1~3 章由李庆扬编写，第 4~6 章由王能超编写，第 7~9 章由易大义编写。教材初稿于 1980 年 12 月投交华中工学院出版社。

1981 年元月在杭州召开的工科院校计算数学第一次教材审稿会，对本书初稿进行了审查，1982 年元月在上海交通大学召开的第二次计算数学教材审稿会，又对本书第 1 版提出了修改意见。会议考虑到理工科院校各专业普遍开设“数值分析”课的情况，重新修订了大纲（72 学时）。本书第 2 版就是根据新大纲的要求修改的，它保持了第 1 版的主要内容及特点，但选材更注意基本要求，减少了部分内容，增加了部分习题答案。本书可作为理工科院校应用数学、力学、物理、计算机软件等专业大学生及其他专业研究生“数值分析”（或“计算方法”）课的教材，也可供学习“计算方法”的科技工作者参考。

我们对参加两次审稿会的同志表示衷心感谢，他们以认真负责的态度对本书提出了许多宝贵意见，对提高教材质量起了很大作用。

编者

1982 年 7 月

原书 PDF 页 6；印刷页码 1。

[查看原页](#)

目录

转录说明 (agent 补充): 以下为印刷目录；末列是目录所列的正文书页码。本页底部的目录页码另记在 YAML 中。

上篇 数值算法分析

编号	标题	书页码
第 1 章	绪论	1
1.1	数值分析研究的对象与特点	1
1.2	误差来源与误差分析的重要性	2
1.3	误差的基本概念	4
1.3.1	误差与误差限	4
1.3.2	相对误差与相对误差限	5
1.3.3	有效数字	6
1.3.4	数值运算的误差估计	7
1.4	数值运算中误差分析的方法与原则	9
1.4.1	要避免除数绝对值远远小于被除数绝对值的除法	9
1.4.2	要避免两相近数相减	10
1.4.3	要防止大数“吃掉”小数	11
1.4.4	注意简化计算步骤，减少运算次数	11
	小结	12
	习题	12
第 2 章	插值法	14
2.1	引言	14
2.2	Lagrange 插值	15
2.2.1	插值多项式的存在唯一性	15
2.2.2	线性插值与抛物插值	16
2.2.3	Lagrange 插值多项式	18
2.2.4	插值余项	19
2.3	逐次线性插值法	21
2.4	差商与 Newton 插值公式	23
2.4.1	差商及其性质	23
2.4.2	Newton 插值公式	24
2.5	差分与等距节点插值公式	26
2.5.1	差分及其性质	26
2.5.2	等距节点插值公式	28
2.6	Hermite 插值	29

原书 PDF 页 7; 印刷页码 2。

[查看原页](#)

目录 (续)

转录说明 (agent 补充): 以下为印刷目录; 末列是目录所列的正文书页码。本页底部的目录页码另记在 YAML 中。

编号	标题	书页码
2.7	分段低次插值	32
2.7.1	多项式插值的问题	32
2.7.2	分段线性插值	33
2.7.3	分段三次 Hermite 插值	34
2.8	三次样条插值	36
2.8.1	三次样条函数	36
2.8.2	三转角方程	37
2.8.3	三弯矩方程	39
2.8.4	计算步骤与例题	40
2.8.5	三次样条插值的收敛性	41
	小结	42
	习题	43
第 3 章	函数逼近与计算	45
3.1	引言与预备知识	45
3.1.1	问题的提出	45
3.1.2	Weierstrass 定理	46
3.1.3	连续函数空间 $C[a, b]$	47
3.2	最佳一致逼近多项式	47
3.2.1	最佳一致逼近多项式的存在性	47
3.2.2	Chebyshev 定理	48
3.2.3	最佳一次逼近多项式	50
3.3	最佳平方逼近	52
3.3.1	内积空间	52
3.3.2	函数的最佳平方逼近	54
3.4	正交多项式	57
3.4.1	正交化手续	57
3.4.2	Legendre 多项式	57
3.4.3	Chebyshev 多项式	60
3.4.4	其他常用的正交多项式	62
3.5	函数按正交多项式展开	63
3.6	曲线拟合的最小二乘法	65
3.6.1	一般的最小二乘逼近	65
3.6.2	用正交函数作最小二乘拟合	69
3.6.3	多元最小二乘拟合	71

编号	标题	书页码
3.7	Fourier 逼近与快速 Fourier 变换	71
3.7.1	最佳平方三角逼近与三角插值	71

原书 PDF 页 8；印刷页码 3。

[查看原页](#)

目录 (续)

转录说明 (agent 补充): 以下为印刷目录；末列是目录所列的正文书页码。本页底部的目录页码另记在 YAML 中。

编号	标题	书页码
3.7.2	快速 Fourier 变换	74
	小结	77
	习题	77
第 4 章	数值积分与数值微分	80
4.1	引言	80
4.1.1	数值求积的基本思想	80
4.1.2	代数精度的概念	81
4.1.3	插值型的求积公式	82
4.2	Newton-Cotes 公式	82
4.2.1	Cotes 系数	82
4.2.2	偶阶求积公式的代数精度	84
4.2.3	几种低阶求积公式的余项	85
4.2.4	复化求积法及其收敛性	86
4.3	Romberg 算法	88
4.3.1	梯形法的递推化	88
4.3.2	Romberg 公式	89
4.3.3	Richardson 外推加速法	91
4.3.4	梯形法的余项展开式	92
4.4	Gauss 公式	93
4.4.1	Gauss 点	94
4.4.2	Gauss-Legendre 公式	95
4.4.3	Gauss 公式的余项	96
4.4.4	Gauss 公式的稳定性	96
4.4.5	带权的 Gauss 公式	97
4.5	数值微分	99
4.5.1	中点方法	99
4.5.2	插值型的求导公式	100
4.5.3	实用的五点公式	102
4.5.4	样条求导	103
	小结	104
	习题	104
第 5 章	常微分方程数值解法	106
5.1	引言	106
5.2	Euler 方法	106

编号	标题	书页码
5.2.1	Euler 格式	106
5.2.2	后退的 Euler 格式	108

原书 PDF 页 9；印刷页码 4。

[查看原页](#)

目录 (续)

转录说明 (agent 补充): 以下为印刷目录；末列是目录所列的正文书页码。本页底部的目录页码另记在 YAML 中。

编号	标题	书页码
5.2.3	梯形格式	109
5.2.4	改进的 Euler 格式	110
5.2.5	Euler 两步格式	111
5.3	Runge-Kutta 方法	113
5.3.1	Taylor 级数法	113
5.3.2	Runge-Kutta 方法的基本思想	114
5.3.3	二阶 Runge-Kutta 方法	115
5.3.4	三阶 Runge-Kutta 方法	116
5.3.5	四阶 Runge-Kutta 方法	118
5.3.6	变步长的 Runge-Kutta 方法	119
5.4	单步法的收敛性和稳定性	120
5.4.1	单步法的收敛性	120
5.4.2	单步法的稳定性	122
5.5	线性多步法	124
5.5.1	基于数值积分的构造方法	124
5.5.2	Adams 显式格式	125
5.5.3	Adams 隐式格式	126
5.5.4	Adams 预测-校正系统	127
5.5.5	基于 Taylor 展开的构造方法	128
5.5.6	Milne 格式	130
5.5.7	Hamming 格式	131
5.6	方程组与高阶方程的情形	132
5.6.1	一阶方程组	132
5.6.2	化高阶方程组为一阶方程组	133
5.7	边值问题的数值解法	134
5.7.1	试射法	135
5.7.2	差分方程的建立	135
5.7.3	差分问题的可解性	137
5.7.4	差分方法的收敛性	138
	小结	140
	习题	140
第 6 章	方程求根	142
6.1	根的搜索	142
6.1.1	逐步搜索法	142

编号	标题	书页码
6.1.2	二分法	142
6.2	迭代法	144

原书 PDF 页 10；印刷页码 5。

[查看原页](#)

目录 (续)

转录说明 (agent 补充): 以下为印刷目录；末列是目录所列的正文书页码。本页底部的目录页码另记在 YAML 中。

编号	标题	书页码
6.2.1	迭代过程的收敛性	144
6.2.2	迭代公式的加工	147
6.3	Newton 法	149
6.3.1	Newton 公式	149
6.3.2	Newton 法的几何解释	150
6.3.3	Newton 法的局部收敛性	151
6.3.4	Newton 法应用举例	152
6.3.5	Newton 下山法	153
6.4	弦截法与抛物线法	154
6.4.1	弦截法	155
6.4.2	抛物线法	156
6.5	代数方程求根	158
6.5.1	多项式求值的秦九韶算法	158
6.5.2	代数方程的 Newton 法	159
6.5.3	劈因子法	160
	小结	162
	习题	162
第 7 章	解线性方程组的直接方法	164
7.1	引言	164
7.2	Gauss 消去法	164
7.2.1	消元手续	165
7.2.2	矩阵的三角分解	168
7.2.3	计算量	170
7.3	Gauss 主元素消去法	171
7.3.1	完全主元素消去法	172
7.3.2	列主元素消去法	173
7.3.3	Gauss-Jordan 消去法	175
7.4	Gauss 消去法的变形	178
7.4.1	直接三角分解法	178
7.4.2	平方根法	181
7.4.3	追赶法	184
7.5	向量和矩阵的范数	186
7.6	误差分析	192
7.6.1	矩阵的条件数	192

编号	标题	书页码
7.6.2	舍入误差	197
	小结	198

原书 PDF 页 11；印刷页码 6。

[查看原页](#)

目录 (续)

转录说明 (agent 补充): 以下为印刷目录；末列是目录所列的正文书页码。本页底部的目录页码另记在 YAML 中。

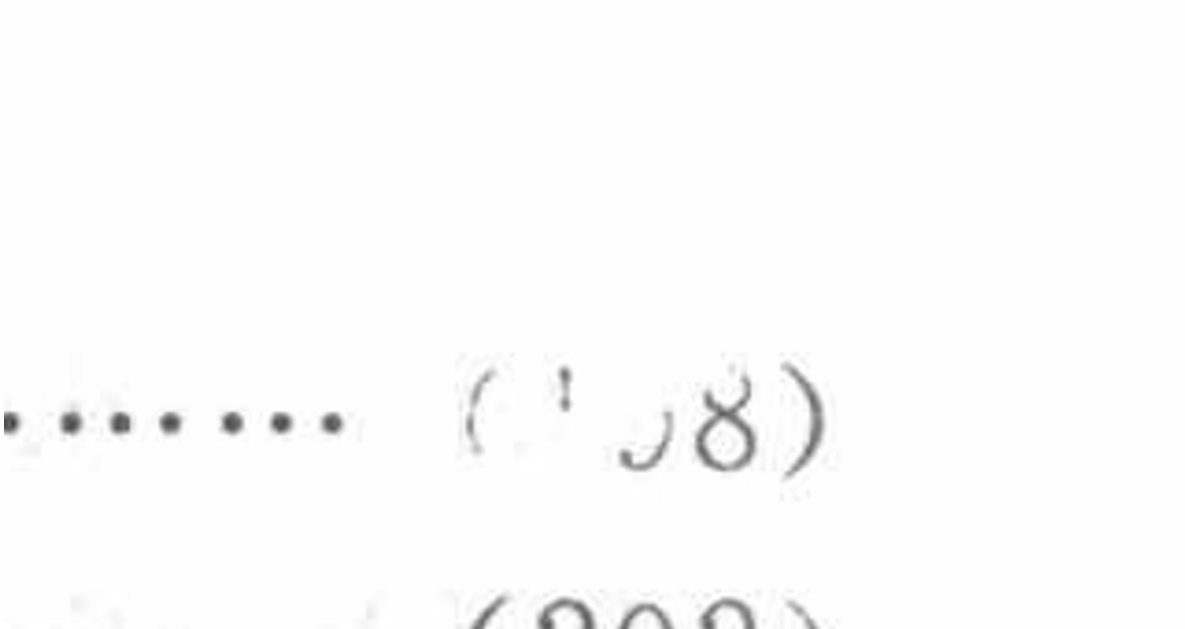
编号	标题	书页码
	习题	〔扫描残损；末位可辨为 8〕
第 8 章	解线性方程组的迭代法	202
8.1	引言	202
8.2	Jacobi 迭代法与 Gauss-Seidel 迭代法	204
8.2.1	Jacobi 迭代法	204
8.2.2	Gauss-Seidel 迭代法	205
8.3	迭代法的收敛性	206
8.4	解线性方程组的超松弛迭代法	213
	小结	217
	习题	217
第 9 章	矩阵的特征值与特征向量计算	220
9.1	引言	220
9.2	幂法及反幂法	222
9.2.1	幂法	222
9.2.2	加速方法	225
9.2.3	反幂法	227
9.3	Householder 方法	230
9.3.1	引言	230
9.3.2	用正交相似变换约化矩阵	232
9.4	QR 算法	237
9.4.1	引言	237
9.4.2	QR 算法	239
9.4.3	带原点位移的 QR 方法	242
	小结	246
	习题	246

下篇 高效算法设计

编号	标题	书页码
* 第 10 章	快速算法设计: 快速 Walsh 变换	248
10.1	美的 Walsh 函数	248
10.1.1	微积分的逼近法	248
10.1.2	Walsh 函数的复杂性	249

编号	标题	书页码
10.1.3	Walsh 分析的数学美	250
10.2	Walsh 函数代数化	251
10.2.1	时基上的二分集	251
10.2.2	Walsh 函数的矩阵表示	252

校注 (agent 补充): 第 7 章“习题”的目录页码前部残损, 末位可辨为 8; 不能从本页确认其完整原印数字。
正文起始处已回查为 PDF 第 212 页、书页 199, 该正文定位不冒充目录原印页码。



正文“习题”起始页与书页 199

原书 PDF 页 12；印刷页码 7。

[查看原页](#)

目录 (续)

转录说明 (agent 补充): 以下为印刷目录；末列是目录所列的正文书页码。本页底部的目录页码另记在 YAML 中。

编号	标题	书页码
10.3	Walsh 阵的二分演化	252
10.3.1	矩阵的对称性复制	253
10.3.2	Walsh 阵的演化生成	253
10.3.3	Walsh 阵的演化机制	254
10.3.4	Hadamard 阵的演化生成	255
10.4	快速变换 FWT	257
10.4.1	FWT 的设计思想	257
10.4.2	FWT 的演化机制	258
10.4.3	FWT 的计算流程	259
10.4.4	FWT 的算法实现	261
	小结	262
* 第 11 章	并行算法设计: 递推计算并行化	263
11.1	什么是并行计算	263
11.1.1	一则寓言故事	263
11.1.2	同步并行算法的设计策略	264
11.2	叠加计算	265
11.2.1	倍增技术	265
11.2.2	二分手续	267
11.2.3	数列求和的二分法	268
11.2.4	多项式求值的二分法	269
11.2.5	二分算法的效能分析	270
11.2.6	二分算法的基本特征	271
11.3	一阶线性递推	272
11.3.1	相关链的二分手续	272
11.3.2	算式的建立	273
11.3.3	二分算法的效能分析	275
11.4	三对角方程组	275
11.4.1	相关链的二分手续	276
11.4.2	算式的建立	277
	小结	279
* 第 12 章	加速算法设计: 重差加速技术	281
12.1	千古疑案	281
12.1.1	阿基米德的“穷竭法”	281
12.1.2	祖冲之“缀术”之谜	281

编号	标题	书页码
12.2	神来之笔	282

原书 PDF 页 13；印刷页码 8。

[查看原页](#)

目录 (续)

转录说明 (agent 补充): 以下为印刷目录；末列是目录所列的正文书页码。本页底部的目录页码另记在 YAML 中。

编号	标题	书页码
12.2.1	数学史上一篇千古奇文	282
12.2.2	“一飞冲天”的“刘徽神算”	283
12.3	奇光异彩	284
12.3.1	刘徽的新视野	285
12.3.2	偏差比中传出好“消息”	286
12.3.3	只要做一次“俯冲”	286
12.3.4	差之毫厘, 失之千里	287
12.3.5	“缀术”再剖析	288
12.3.6	平庸的新纪录	289
12.4	万能引擎	291
12.4.1	逼近加速的重差公设	292
12.4.2	重差加速法则	292
12.4.3	重差加速的逻辑推理	293
第 13 章	总览	294
13.1	算法重在设计	294
13.1.1	算法设计关系到科学计算的成败	294
13.1.2	算法设计追求简单与统一	295
13.2	直接法的缩减技术	295
13.2.1	数列求和的累加算法	295
13.2.2	缩减技术的设计机理	296
13.2.3	多项式求值的秦九韶算法	297
13.3	迭代法的校正技术	298
13.3.1	开方算法	298
13.3.2	校正技术的设计机理	299
13.4	迭代优化的超松弛技术	300
13.4.1	超松弛技术的设计机理	300
13.4.2	刘徽的“割圆术”	300
13.5	递推加速的二分技术	301
13.5.1	“结绳记数”的快速算法	301
13.5.2	二分技术的设计机理	302
	小结	303
	部分习题答案	305
	参考文献	308

原书 PDF 页 14；印刷页码 1。

[查看原页](#)

上篇数值算法分析

第 1 章绪论

1.1 数值分析研究的对象与特点

数值分析是研究各种数学问题求解的数值计算方法. 在电子计算机成为数值计算的主要工具以后, 人们迫切要求研究适合于计算机使用的数值计算方法. 为了更具体地说明数值分析的研究对象, 我们来考察用计算机解决科学计算问题时所经历的过程 (见图 1.1) .

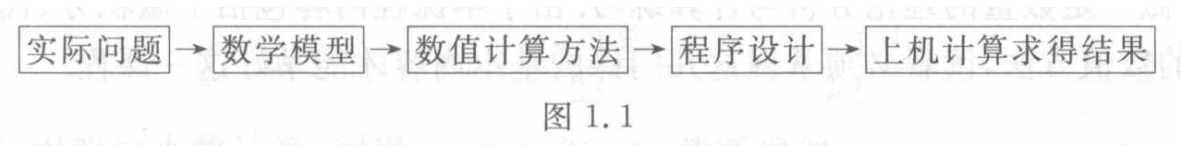


图 1.1。图中各矩形框依次以向右箭头相连：实际问题 → 数学模型 → 数值计算方法 → 程序设计 → 上机计算求得结果。

由实际问题的提出到上机计算求得结果, 整个过程都可看作应用数学的研究对象. 如果细分的话, 针对实际问题应用有关科学知识和数学理论建立数学模型这一过程, 通常作为应用数学的研究对象, 而根据数学模型提出求解的数值计算方法直到编出程序上机算出结果这一过程, 则是计算数学的研究对象, 也是数值分析的研究对象. 因此, 数值分析就是研究用计算机解决数学问题的数值方法及其理论, 它的内容包括函数的数值逼近、数值微分与数值积分、非线性方程数值解、数值线性代数、微分方程数值解等, 它们都是以数学问题为研究对象的. 因此, 数值分析是数学的一个分支, 只是它不像纯数学那样只研究数学本身的理论, 而是把理论与计算紧密结合起来, 着重研究数学问题的数值方法及其理论.

数值分析也称为计算方法, 但不应片面地将它理解为各种数值方法的简单罗列和堆积. 同数学分析一样, 它内容丰富, 研究方法深刻, 有自身理论体系的课程, 既有纯数学的高度抽象性与严密科学性的特点, 又有应用的广泛性与实际试验的高度技术性的特点, 是一门与计算机应用密切结合的、实用性很强的数学课程. 它与纯数学课程不同, 例如, 在考虑线性方程组数值解时, “线性代数” 中只介绍解存在的唯一性及有关理论和精确解法, 运用这些理论和方法, 无法在计算机上求解上百个未知数的方程组, 更不用说求解十几万个未知数的方程组了. 求解这类问题还应根据方程特点, 研究适合计算机使用的、满足精度要求的、计算省时间的有效算法及其相关的理

校注 (agent 补充): 本页末尾 “理” 字与下一页开头 “论” 字相接。

原书 PDF 页 15; 印刷页码 2。

[查看原页](#)

论; 在实现这些算法时往往还要根据计算机容量、字长、速度等指标, 研究具体求解步骤和程序设计技巧; 有的方法在理论上虽不够严格, 但通过实际计算、对比分析等手段, 只要能证明它们是行之有效的方法, 也应采用. 这些就是数值分析具有的特点, 概括起来有四点.

第一, 面向计算机, 要根据计算机特点提供实际可行的有效算法, 即算法只能包括加、减、乘、除运算和逻辑运算, 它们都是计算机能直接处理的.

第二, 有可靠的理论分析, 能任意逼近并达到精度要求, 对近似算法要保证收敛性和数值稳定性, 还要对误差进行分析. 这些都要建立在相应数学理论的基础上.

第三, 有好的计算复杂性. 时间复杂性好是指节省时间, 空间复杂性好是指节省存储量, 这也是建立算法要研究的问题, 它关系到算法能否在计算机上实现.

第四, 有数值实验. 任何一个算法, 除了从理论上要满足上述三点外, 还要通过数值实验证明它是行之有效的.

根据“数值分析”的特点, 学习时首先要注意掌握方法的基本原理和思想, 要注意方法处理的技巧及其与计算机的结合, 要重视误差分析、收敛性及稳定性的基本理论; 其次, 要通过例子, 学习使用各种数值方法解决实际计算问题; 最后, 为了掌握本课程的内容, 还应做一定数量的理论分析与计算练习. 由于本课程内容包括了微积分、代数、常微分方程的数值方法, 读者必须掌握这几门课的基本内容才能学好这一课程.

1.2 误差来源与误差分析的重要性

用计算机解决科学计算问题首先要建立数学模型, 它是对被描述的实际问题进行抽象、简化而得到的, 因而是近似的. 我们把数学模型与实际之间出现的这种误差称为模型误差. 只有实际问题提法正确, 建立数学模型时又抽象、简化得合理, 才能得到好的结果. 由于这种误差难以用数量表示, 通常都假定数学模型是合理的, 这种误差可忽略不计, 在“数值分析”中不予讨论. 在数学模型中往往还有一些根据观测得到的物理量, 如温度、长度、电压等, 这些参量显然也包含误差. 这种由观测产生的误差称为观测误差, 在“数值分析”中也不讨论这种误差. 数值分析只研究用数值方法求解数学模型产生的误差.

当数学模型不能得到精确解时, 通常要用数值方法求它的近似解, 其近似解与精确解之间的误差称为截断误差或方法误差. 例如, 当函数 $f(x)$ 用 Taylor 多项式

$$P_n(x) = f(0) + \frac{f'(0)}{1!}x + \frac{f''(0)}{2!}x^2 + \cdots + \frac{f^{(n)}(0)}{n!}x^n$$

近似代替时, 数值方法的截断误差是

$$R_n(x) = f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}x^{n+1}, \quad \xi \text{ 在 } x \text{ 与 } 0 \text{ 之间.}$$

原书 PDF 页 16；印刷页码 3。

[查看原页](#)

有了求解数学问题的计算公式以后，用计算机进行数值计算时，由于计算机的字长有限，原始数据在计算机上表示会产生误差，计算过程又可能产生新的误差，这种误差称为舍入误差，例如，用 3.14159 近似代替 π ，产生的误差

$$R = \pi - 3.14159 = 0.0000026 \dots$$

就是舍入误差. 在“数值分析”中除了研究数学问题的算法外，还要研究计算结果的误差是否满足精度要求，这就是误差估计问题. 本书主要讨论算法的截断误差与舍入误差，对舍入误差通常只作一些定性分析. 下面举例说明误差分析的重要性.

例 1.1 计算 $I_n = e^{-1} \int_0^1 x^n e^x dx (n = 0, 1, \dots)$ ，并估计误差.

解由分部积分可得计算 I_n 的递推公式

$$I_n = 1 - nI_{n-1} \quad (n = 1, 2, \dots), \quad (1.2.1)$$

$$I_0 = e^{-1} \int_0^1 e^x dx = 1 - e^{-1}.$$

若计算出 I_0 ，代入式 (1.2.1)，可逐次求出 $I_1, I_2, \dots$ 的值. 要算出 I_0 就要先计算 e^{-1} ，若用 Taylor 多项式展开部分和

$$e^{-1} \approx 1 + (-1) + \frac{(-1)^2}{2!} + \dots + \frac{(-1)^k}{k!},$$

并取 $k = 7$ ，用四位小数计算，则得 $e^{-1} \approx 0.3679$ ，截断误差

$$R_7 = |e^{-1} - 0.3679| \leq \frac{1}{8!} < \frac{1}{4} \times 10^{-4}.$$

计算过程中小数点后第五位的数字按四舍五入原则舍入，由此产生的舍入误差这里先不讨论. 当初值取为 $I_0 \approx 0.6321 = \tilde{I}_0$ 时，用式 (1.2.1) 递推的计算公式为方案 (A)

$$\begin{cases} \tilde{I}_0 = 0.6321, \\ \tilde{I}_n = 1 - n\tilde{I}_{n-1} \quad (n = 1, 2, \dots). \end{cases}$$

计算结果如表 1.1 的 $\tilde{I}_n$ 列所示. 用 $\tilde{I}_0$ 近似 I_0 产生的误差 $E_0 = I_0 - \tilde{I}_0$ 就是初值误差，它对后面计算结果是有影响的.

表 1.1

n	$\tilde{I}_n$ (用方案 (A) 计算)	I_n^* (用方案 (B) 计算)	n	$\tilde{I}_n$ (用方案 (A) 计算)	I_n^* (用方案 (B) 计算)
0	0.6321	0.6321	5	0.1480	0.1455
1	0.3679	0.3679	6	0.1120	0.1268
2	0.2642	0.2643	7	0.2160	0.1121

n	$\tilde{I}_n$ (用方案 (A) 计算)	I_n^* (用方案 (B) 计算)	n	$\tilde{I}_n$ (用方案 (A) 计算)	I_n^* (用方案 (B) 计算)
3	0.2074	0.2073	8	-0.728	0.1035
4	0.1704	0.1708	9	7.552	0.0684

原书 PDF 页 17；印刷页码 4。

[查看原页](#)

从表 1.1 可以看到, $\tilde{I}_8$ 出现负值, 这与一切 $I_n > 0$ 相矛盾. 实际上, 由积分估值得

$$\frac{e^{-1}}{n+1} = e^{-1} \left(\min_{0 \leq x \leq 1} e^x \right) \int_0^1 x^n dx < I_n < e^{-1} \left(\max_{0 \leq x \leq 1} e^x \right) \int_0^1 x^n dx = \frac{1}{n+1}. \quad (1.2.2)$$

因此, 当 n 较大时, 用 $\tilde{I}_n$ 近似 I_n 显然是不正确的. 这里, 计算公式与每步计算都是正确的, 那么, 是什么原因使计算结果错误呢? 主要就是初值 $\tilde{I}_0$ 有误差 $E_0 = I_0 - \tilde{I}_0$, 由此引起以后各步计算的误差 $E_n = I_n - \tilde{I}_n$ 满足关系 $E_n = -nE_{n-1}$ ($n = 1, 2, \dots$). 容易推得

$$E_n = (-1)^n n! E_0,$$

这说明 $\tilde{I}_0$ 有误差 E_0 , 则 $\tilde{I}_n$ 就是 E_0 的 $n!$ 倍误差. 例如, $n = 8$, 若 $|E_0| = \frac{1}{2} \times 10^{-4}$, 则 $|E_8| = 8! \times |E_0| > 2$. 这就说明 $\tilde{I}_8$ 完全不能近似 I_8 了. 我们现在换一种计算方案. 由式 (1.2.2) 取 $n = 9$, 得

$$\frac{e^{-1}}{10} < I_9 < \frac{1}{10},$$

粗略取 $I_9 \approx \frac{1}{2} \left(\frac{1}{10} + \frac{e^{-1}}{10} \right) = 0.0684 = I_9^*$, 然后将式 (1.2.1) 倒过来算, 即由 I_9^* 算出 $I_8^*, I_7^*, \dots, I_1^*$, 公式为方案 (B)

$$\begin{cases} I_9^* = 0.0684, \\ I_{n-1}^* = \frac{1}{n}(1 - I_n^*) \quad (n = 9, 8, \dots, 1). \end{cases}$$

计算结果如表 1.1 中的 I_n^* 列所示. 可以发现, I_0^* 与 I_0 的误差不超过 10^{-4} . 由于 $|E_0^*| = \frac{1}{n!}|E_n^*|$, E_0^* 比 E_n^* 缩小了 $n!$ 倍, 因此, 尽管 E_9^* 较大, 但由于误差逐步缩小, 故可用 I_n^* 近似 I_n . 反之, 当用方案 (A) 计算时, 尽管初值 $\tilde{I}_0$ 相当准确, 但由于误差传播是逐步扩大的, 因而计算结果不可靠. 此例说明, 在数值计算中如不注意误差分析, 用了类似于方案 (A) 的计算公式, 就会出现“差之毫厘, 失之千里”的错误结果. 尽管数值计算中估计误差比较困难, 但仍应重视计算过程中的误差分析.

1.3 误差的基本概念

1.3.1 误差与误差限 定义 1.1 设 x 为准确值, x^* 为 x 的一个近似值, 称 $e^* = x^* - x$ 为近似值的绝

校注 (agent 补充): 本页定义 1.1 末尾“绝”字与下一页“对误差”相接。

校注 (agent 补充, CH01-E001): 原书写“则 $\tilde{I}_n$ 就是 E_0 的 $n!$ 倍误差”, 此处照录. 由上式 $E_n = (-1)^n n! E_0$ 可知, 应理解为近似值 $\tilde{I}_n$ 的误差绝对值为初值误差绝对值的 $n!$ 倍; 疑为原书表述不严谨, 不是 OCR 错误。

原书 PDF 页 18; 印刷页码 5。

[查看原页](#)

对误差, 简称误差.

注意, 这样定义的误差 e^* 可正可负, 当绝对误差为正时近似值偏大, 叫做强近似值; 当绝对误差为负时近似值偏小, 叫做弱近似值.

通常, 我们不能算出准确值 x , 也不能算出误差 e^* 的准确值, 只能根据测量工具或计算情况估计出误差的绝对值不超过某正数 ε^* , 也就是误差绝对值的一个上界. ε^* 叫做近似值的误差限, 它总是正数. 例如, 用毫米刻度的米尺测量一长度 x (单位: mm, 下同), 读出和该长度接近的刻度 x^* , x^* 是 x 的近似值, 它的误差限是 0.5, 于是 $|x^* - x| \leq 0.5$; 如读出的长度为 765, 则有 $|765 - x| \leq 0.5$. 从该不等式仍不知道准确的 x 是多少, 但知道 $764.5 \leq x \leq 765.5$, 说明 x 在区间 $[764.5, 765.5]$ 上.

对于一般情形, $|x^* - x| \leq \varepsilon^*$, 即 $x^* - \varepsilon^* \leq x \leq x^* + \varepsilon^*$, 这个不等式有时也表示为

$$x = x^* \pm \varepsilon^*.$$

1.3.2 相对误差与相对误差限 误差限的大小还不能完全表示近似值的好坏. 例如, 有两个量 $x = 10 \pm 1, y = 1000 \pm 5$, 则

$$x^* = 10, \varepsilon_x^* = 1, y^* = 1000, \varepsilon_y^* = 5.$$

虽然 ε_y^* 比 ε_x^* 大 4 倍, 但 $\frac{\varepsilon_y^*}{y^*} = \frac{5}{1000} = 0.5\%$ 比 $\frac{\varepsilon_x^*}{x^*} = \frac{1}{10} = 10\%$ 要小得多, 这说明 y^* 近似 y 的程度比 x^* 近似 x 的程度要好得多. 所以, 除考虑误差的大小外, 还应考虑准确值 x 本身的大小. 近似值的误差 e^* 与准确值 x 的比值

$$\frac{e^*}{x} = \frac{x^* - x}{x}$$

称为近似值 x^* 的相对误差, 记作 e_r^* .

在实际计算中, 由于真值 x 总是不知道的, 通常取

$$e_r^* = \frac{e^*}{x^*} = \frac{x^* - x}{x^*}$$

作为 x^* 的相对误差, 条件是 $e_r^* = \frac{e^*}{x^*}$ 较小, 此时

$$\frac{e^*}{x} - \frac{e^*}{x^*} = \frac{e^*(x^* - x)}{x^*x} = \frac{(e^*)^2}{x^*(x^* - e^*)} = \frac{(e^*/x^*)^2}{1 - (e^*/x^*)}$$

是 e_r^* 的二次方项级, 故可忽略不计.

相对误差也可正可负, 它的绝对值上界叫做相对误差限, 记作 $\varepsilon_r^*, \varepsilon_r^* = \frac{\varepsilon^*}{|x^*|}$.

根据定义, $\frac{\varepsilon_x^*}{|x^*|} = 10\%$ 与 $\frac{\varepsilon_y^*}{|y^*|} = 0.5\%$ 分别为 x 与 y 的相对误差限, 可见 y^* 近似 y 的程度比 x^* 近似 x 的程度好.

原书 PDF 页 19; 印刷页码 6。

[查看原页](#)

1.3.3 有效数字 当准确值 x 有多位数时, 常常按四舍五入的原则得到 x 的前几位近似值 x^* . 例如

$$x = \pi = 3.14159265 \dots,$$

取前三位, $x_3^* = 3.14, \varepsilon_3^* \leq 0.002$; 取前五位, $x_5^* = 3.1416, \varepsilon_5^* \leq 0.000008$: 它们的误差都不超过末位数字的半个单位, 即

$$|\pi - 3.14| \leq \frac{1}{2} \times 10^{-2}, \quad |\pi - 3.1416| \leq \frac{1}{2} \times 10^{-4}.$$

若近似值 x^* 的误差限是某一位的半个单位, 该位到 x^* 的第一位非零数字共有 n 位, 就说 x^* 有 n 位有效数字. 如取 $x^* = 3.14$ 作 π 的近似值, x^* 就有三位有效数字; 取 $x^* = 3.1416$ 作 π 的近似值, x^* 就有五位有效数字. x^* 有 n 位有效数字可写成标准形式

$$x^* = \pm 10^m \times (a_1 + a_2 \times 10^{-1} + \dots + a_n \times 10^{-(n-1)}), \quad (1.3.1)$$

其中, a_1 是 1 到 9 中的一个数字; $a_2, \dots, a_n$ 是 0 到 9 中的一个数字; m 为整数, 且

$$|x - x^*| \leq \frac{1}{2} \times 10^{m-n+1}. \quad (1.3.2)$$

例 1.2 按四舍五入原则写出下列各数具有五位有效数字的近似数: 187.932 5, 0.037 855 51, 8.000 033, 2.718 281 8.

解按定义, 上述各数具有五位有效数字的近似数分别是 187.93, 0.037 856, 8.000 0, 2.718 3.

注意 $x = 8.000033$ 的五位有效数字近似数是 8.0000 而不是 8, 因为 8 只有一位有效数字.

例 1.3 重力常数 g , 如果以 m/s^2 为单位, $g \approx 9.80 \text{ m/s}^2$; 若以 km/s^2 为单位, $g \approx 0.00980 \text{ km/s}^2$, 它们都具有三位有效数字. 按第一种写法, 有

$$|g - 9.80| \leq \frac{1}{2} \times 10^{-2},$$

据式 (1.3.1), 这里 $m = 0, n = 3$; 按第二种写法, 有

$$|g - 0.00980| \leq \frac{1}{2} \times 10^{-5},$$

这里 $m = -3, n = 3$. 它们虽然写法不同, 但都具有三位有效数字. 至于绝对误差限, 由于单位不同, 结果也不同, $\varepsilon_1^* = \frac{1}{2} \times 10^{-2} \text{ m/s}^2, \varepsilon_2^* = \frac{1}{2} \times 10^{-5} \text{ km/s}^2$, 而相对误差都是

$$\varepsilon_r^* = 0.005/9.80 = 0.000005/0.00980.$$

注意相对误差与相对误差限是无量纲的, 而绝对误差与误差限是有量纲的. 例 1.3 说明有效位数与小数点后有多少位数无关. 然而, 从式 (1.3.2) 可得到具

校注 (agent 补充): 本页末尾“具”字与下一页“有 n 位有效数字”相接。

原书 PDF 页 20; 印刷页码 7。

[查看原页](#)

有 n 位有效数字的近似数 x^* , 其绝对误差限为

$$\varepsilon^* = \frac{1}{2} \times 10^{m-n+1},$$

在 m 相同的情况下, n 越大, 10^{m-n+1} 越小, 故有效位数越多, 绝对误差限越小. 关于有效数字与相对误差限的关系, 有如下定理.

定理 1.1 对于用式 (1.3.1) 表示的近似数 x^* , 若 x^* 具有 n 位有效数字, 则其相对误差限为

$$\varepsilon_r^* \leq \frac{1}{2a_1} \times 10^{-(n-1)};$$

反之, 若 x^* 的相对误差限 $\varepsilon_r^* \leq \frac{1}{2(a_1+1)} \times 10^{-(n-1)}$, 则 x^* 至少具有 n 位有效数字.

证明 由式 (1.3.1) 可得 $a_1 \times 10^m \leq |x^*| \leq (a_1 + 1) \times 10^m$, 当 x^* 有 n 位有效数字时, 有

$$\varepsilon_r^* = \frac{|x - x^*|}{|x^*|} \leq \frac{0.5 \times 10^{m-n+1}}{a_1 \times 10^m} = \frac{1}{2a_1} \times 10^{-n+1};$$

反之, 有

$$|x - x^*| = |x^*| \varepsilon_r^* \leq (a_1 + 1) \times 10^m \times \frac{1}{2(a_1 + 1)} \times 10^{-n+1} = 0.5 \times 10^{m-n+1},$$

故 x^* 有 n 位有效数字. 证毕.

定理 1.1 说明, 有效位数越多, 相对误差限越小.

例 1.4 要使 $\sqrt{20}$ 的近似值的相对误差限小于 0.1%, 要取几位有效数字?

解 由定理 1.1 知

$$\varepsilon_r^* \leq \frac{1}{2a_1} \times 10^{-n+1}.$$

由 $\sqrt{20} = 4.4\dots$ 知, $a_1 = 4$, 故只要取 $n = 4$, 就有

$$\varepsilon_r^* \leq 0.125 \times 10^{-3} < 10^{-3} = 0.1\%,$$

即只要对 $\sqrt{20}$ 的近似值取四位有效数字, 其相对误差限就小于 0.1%, 此时

$$\sqrt{20} \approx 4.472.$$

1.3.4 数值运算的误差估计 两个近似数 x_1^* 与 x_2^* , 其误差限分别为 $\varepsilon(x_1^*)$ 及 $\varepsilon(x_2^*)$, 它们进行加、减、乘、除运算得到的误差限分别为

$$\varepsilon(x_1^* \pm x_2^*) = \varepsilon(x_1^*) + \varepsilon(x_2^*),$$

$$\varepsilon(x_1^* x_2^*) \approx |x_1^*| \varepsilon(x_2^*) + |x_2^*| \varepsilon(x_1^*),$$

$$\varepsilon\left(\frac{x_1^*}{x_2^*}\right) \approx \frac{|x_1^*| \varepsilon(x_2^*) + |x_2^*| \varepsilon(x_1^*)}{|x_2^*|^2} \quad (x_2^* \neq 0).$$

更一般的情况是, 当自变量有误差时, 计算函数值也会产生误差, 其误差限可利

校注 (agent 补充): 本页末尾“可利”与下一页“用函数的 Taylor 展开式进行估计”相接。

原书 PDF 页 21; 印刷页码 8。

[查看原页](#)

用函数的 Taylor 展开式进行估计。设 $f(x)$ 是一元函数, x 的近似值为 x^* , 以 $f(x^*)$ 近似 $f(x)$, 其误差限记作 $\varepsilon(f(x^*))$, 可用 Taylor 展开

$$f(x) - f(x^*) = f'(x^*)(x - x^*) + \frac{f''(\xi)}{2}(x - x^*)^2,$$

ξ 介于 x, x^* 之间。取绝对值得

$$|f(x) - f(x^*)| \leq |f'(x^*)|\varepsilon(x^*) + \frac{|f''(\xi)|}{2}\varepsilon^2(x^*).$$

假定 $f'(x^*)$ 与 $f''(x^*)$ 的比值不太大, 可忽略 $\varepsilon(x^*)$ 的高阶项, 于是可得计算函数的误差限为

$$\varepsilon(f(x^*)) \approx |f'(x^*)|\varepsilon(x^*).$$

当 f 为多元函数时计算 $A = f(x_1, x_2, \dots, x_n)$, 如果 $x_1, x_2, \dots, x_n$ 的近似值为 $x_1^*, x_2^*, \dots, x_n^*$, 则 A 的近似值为 $A^* = f(x_1^*, x_2^*, \dots, x_n^*)$, 于是函数值 A^* 的误差 $e(A^*)$ 由 Taylor 展开, 得

$$\begin{aligned} e(A^*) &= A^* - A = f(x_1^*, x_2^*, \dots, x_n^*) - f(x_1, x_2, \dots, x_n) \\ &\approx \sum_{k=1}^n \left(\frac{\partial f(x_1^*, x_2^*, \dots, x_n^*)}{\partial x_k} \right) (x_k^* - x_k) = \sum_{k=1}^n \left(\frac{\partial f}{\partial x_k} \right)^* e_k^*, \end{aligned}$$

于是误差限为

$$\varepsilon(A^*) \approx \sum_{k=1}^n \left| \left(\frac{\partial f}{\partial x_k} \right)^* \right| \varepsilon(x_k^*); \quad (1.3.3)$$

而 A^* 的相对误差限为

$$\varepsilon_r^* = \varepsilon_r(A^*) = \frac{\varepsilon(A^*)}{|A^*|} \approx \sum_{k=1}^k \left| \left(\frac{\partial f}{\partial x_k} \right)^* \right| \frac{\varepsilon(x_k^*)}{|A^*|}. \quad (1.3.4)$$

例 1.5 已测得某场地长 l 的值为 $l^* = 110$ m, 宽 d 的值为 $d^* = 80$ m, 已知 $|l - l^*| \leq 0.2$ m, $|d - d^*| \leq 0.1$ m, 试求面积 $S = ld$ 的绝对误差限与相对误差限。

解 因 $S = ld$, $\frac{\partial S}{\partial l} = d$, $\frac{\partial S}{\partial d} = l$, 由式 (1.3.3) 知

$$\varepsilon(S^*) \approx \left| \left(\frac{\partial S}{\partial l} \right)^* \right| \varepsilon(l^*) + \left| \left(\frac{\partial S}{\partial d} \right)^* \right| \varepsilon(d^*),$$

其中

$$\left(\frac{\partial S}{\partial l} \right)^* = d^* = 80 \text{ m}, \quad \left(\frac{\partial S}{\partial d} \right)^* = l^* = 110 \text{ m},$$

而

$$\varepsilon(l^*) = 0.2 \text{ m}, \quad \varepsilon(d^*) = 0.1 \text{ m},$$

于是绝对误差限为

$$\varepsilon(S^*) \approx (80 \times 0.2 + 110 \times 0.1) \text{ m}^2 = 27 \text{ m}^2;$$

相对误差限为

$$\varepsilon_r(S^*) = \frac{\varepsilon(S^*)}{|S^*|} = \frac{\varepsilon(S^*)}{l^* d^*} \approx \frac{27}{8800} = 0.31\%.$$

校注 (agent 补充, CH01-E002): 式 (1.3.4) 原图的求和上限清楚印为 k , 本页保留 $\sum_{k=1}^k$ 。由函数有 n 个变量以及紧前式 (1.3.3) 的上限 n 判断, 疑为原书将 n 误排为 k 。OCR 候选写成 n , 不能据此静默改原书。

校注 (agent 补充, CH01-E003): 原书关于可忽略高阶项的条件写为 “ $f'(x^*)$ 与 $f''(x^*)$ 的比值不太大”, 此处照录。若意在使二阶项相对一阶项较小, 应控制 $|f''(\xi)|\varepsilon(x^*)/(2|f'(x^*)|)$ (在分母非零时); 原句疑有比值顺序或条件表述问题, 不是 OCR 错误。

原书 PDF 页 22; 印刷页码 9。

[查看原页](#)

1.4 数值运算中误差分析的方法与原则

数值运算中的误差分析是个很重要而复杂的问题, 上节讨论了不精确数据运算结果的误差限, 它只适用于简单情形。然而实际工程或科学计算问题往往要运算千万次, 而且每步运算都有误差, 但是每步都作误差分析是不可能的, 也是不科学的。这是因为, 误差积累有正有负, 绝对值有大有小, 都按最坏情况估计误差限得到的结果比实际误差大得多, 这种保守的误差估计不反映实际误差积累。考虑到误差分布的随机性, 有人用概率统计方法, 将数据和运算中的舍入误差视为适合某种分布的随机变量, 然后确定计算结果的误差分布, 这样得到的误差估计更接近实际, 这种方法称为概率分析法。

20 世纪 60 年代以后, 有人对舍入误差分析提出了一些新方法, 但都不是十分有效。目前解决这一问题的办法, 常常是针对不同类型问题逐个进行分析。由于定量分析常常是很困难的, 因此对误差积累问题进行定性分析就有重要意义, 这就要引入数值稳定性概念。运算过程舍入误差不增长的计算公式是数值稳定的, 否则是不稳定的。如在例 1.1 给出的两种计算方案中, 方案 (A) 由于初值有误差, 在计算过程中这一误差逐渐增大, 故是数值不稳定的; 而方案 (B) 虽然初值也有误差, 但计算过程误差不增长, 故是数值稳定的。研究一个计算公式是否稳定, 只要假定初始值有误差 ε_0 , 中间不再产生新误差, 考察由 ε_0 引起的误差积累是否增长, 如不增长就认为是稳定的, 否则是不稳定的。对于稳定的计算公式, 不具体估计舍入误差积累也可相信它是可用的, 误差限不会太大; 而不稳定的公式通常就不能使用, 如要使用, 其计算步数也只能很少, 并且注意对误差积累进行控制。在本课程中, 对各种计算过程都只研究它的稳定性, 而不具体估计舍入误差。这里只提出数值运算中应注意的若干原则, 它有助于鉴别计算结果的可靠性并防止误差危害现象的产生。

1.4.1 要避免除数绝对值远远小于被除数绝对值的除法 用绝对值小的数作除数, 舍入误差会增大, 如计算 $\frac{x}{y}$, 若 $0 < |y| \ll |x|$, 则可能对计算结果带来严重影响, 应尽量避免。

例 1.6 线性方程组 $\begin{cases} 0.00001x_1 + x_2 = 1 \\ 2x_1 + x_2 = 2 \end{cases}$ 的准确解为

$$x_1 = \frac{200000}{399999} = 0.50000125,$$

$$x_2 = \frac{199998}{199999} = 0.999995.$$

校注 (agent 补充, CH01-E004): 原书把例 1.6 的 x_1 写成 $200000/399999 = 0.50000125$, 此处照录。由原方程两式相减, $(2 - 0.00001)x_1 = 1$, 所以 $x_1 = 100000/199999 \approx 0.5000025000125$; 原书所列 $x_2 = 199998/199999$ 与后一结果一致。疑为原书 x_1 分式及小数有误, 不是 OCR 错误。

原书 PDF 页 23; 印刷页码 10。

[查看原页](#)

现在四位浮点十进制数（仿机器实际计算）下用消去法求解，上述方程组写成

$$\begin{cases} 10^{-4} \times 0.1000x_1 + 10^1 \times 0.1000x_2 = 10^1 \times 0.1000, \\ 10^1 \times 0.2000x_1 + 10^1 \times 0.1000x_2 = 10^1 \times 0.2000. \end{cases}$$

若用 $(10^{-4} \times 0.1000)/2$ 除以第一个方程然后减去第二个方程，则出现了用小数除以大数的现象，得

$$\begin{cases} 10^{-4} \times 0.1000x_1 + 10^1 \times 0.1000x_2 = 10^1 \times 0.1000, \\ 10^6 \times 0.2000x_2 = 10^6 \times 0.2000, \end{cases}$$

由此解出 $x_1 = 0, x_2 = 10^1 \times 0.1000 = 1$ ，显然严重失真。若反过来用第二个方程消去第一个方程中含 x_1 的项，则避免了大数被小数除的现象，得

$$\begin{cases} 10^6 \times 0.1000x_2 = 10^6 \times 0.1000, \\ 10^1 \times 0.2000x_1 + 10^1 \times 0.1000x_2 = 10^1 \times 0.2000, \end{cases}$$

由此求得相当好的近似解 $x_1 = 0.5000, x_2 = 10^1 \times 0.1000$ 。

1.4.2 要避免两相近数相减 在数值计算中，两相近数相减会导致有效数字严重损失。例如， $x = 532.65, y = 532.52$ 都有五位有效数字，但 $x - y = 0.13$ 只有两位有效数字。必须尽量避免出现这类运算，最好是改变计算方法，防止这种现象产生。现举例说明。

例 1.7 计算 $A = 10^7(1 - \cos 2^\circ)$ 。

解 由于 $\cos 2^\circ = 0.9994$ ，直接计算，得

$$A = 10^7(1 - \cos 2^\circ) = 10^7(1 - 0.9994) = 6 \times 10^3,$$

只有一位有效数字；若利用 $1 - \cos x = 2 \sin^2 \frac{x}{2}$ 计算，则有

$$A = 10^7(1 - \cos 2^\circ) = 2 \times (\sin 1^\circ)^2 \times 10^7 = 6.13 \times 10^3,$$

具有三位有效数字（这里 $\sin 1^\circ = 0.0175$ ）。

例 1.7 说明，可通过改变计算公式避免或减少有效数字的损失。类似地，如果 x_1 和 x_2 很接近，则

$$\lg x_1 - \lg x_2 = \lg \frac{x_1}{x_2}.$$

用右端算式，有效数字就不会损失。当 x 很大时，有

$$\sqrt{x+1} - \sqrt{x} = \frac{1}{\sqrt{x+1} + \sqrt{x}},$$

都用右端算式代替左端. 一般情况下, 当 $f(x) \approx f(x^*)$ 时, 可用 Taylor 展开

$$f(x) - f(x^*) = f'(x^*)(x - x^*) + \frac{f''(x^*)}{2}(x - x^*)^2 + \dots,$$

校注 (agent 补充): 本页末尾 Taylor 展开后的讨论续下页。

校注 (agent 补充, CH01-E005): 例 1.6 续文原书写“用小数除以大数”, 并在第二种消去结果的首行两侧印 10^6 , 此处均照录。按上下文, 第一种操作是将第一方程除以小量 $(10^{-4} \times 0.1000)/2$; 第二种操作直接以 $R_1 - 0.000005R_2$ 消去, 四位浮点结果应得到 $10^1 \times 0.1000x_2 = 10^1 \times 0.1000$ 。原印 10^6 的等式虽可通过两侧再同乘 10^5 得到, 但原文没有该缩放步骤; 上述措辞和指数均列为疑误, 未静默修改。

校注 (agent 补充, CH01-E006): 例 1.7 原文 “ 6.13×10^3 , 具有三位有效数字” 照录。独立数值复算 $10^7(1 - \cos 2^\circ) \approx 6091.729809$, 原列 6130 的误差约 38.270191, 超过三位有效数字所对应的 5。以本章有效数字定义衡量, 该“三位”判断存在问题; 这不影响使用半角公式减轻相消的思路。

原书 PDF 页 24; 印刷页码 11。

[查看原页](#)

取右端的有限项近似左端. 如果无法改变算式, 则采用增加有效位数进行运算; 在计算机上则采用双倍字长运算, 但这要增加机器计算时间和多占内存单元.

1.4.3 要防止大数“吃掉”小数 在数值运算中, 参加运算的数有时数量级相差很大, 而计算机位数有限, 如不注意运算次序就可能出现大数“吃掉”小数的现象, 影响计算结果的可靠性.

例 1.8 在五位十进制计算机上, 计算

$$A = 52\,492 + \sum_{i=1}^{1000} \delta_i,$$

其中 $0.1 \leq \delta_i \leq 0.9$.

解 把运算的数写成规格化形式, 有

$$A = 0.524\,92 \times 10^5 + \sum_{i=1}^{1000} \delta_i.$$

由于在计算机上计算时要对阶, 若取 $\delta_i = 0.9$, 对阶时 $\delta_i = 0.000\,009 \times 10^5$, 在五位的计算机中表示为机器数 0, 因此

$$A = 0.524\,92 \times 10^5 + 0.000\,009 \times 10^5 + \cdots + 0.000\,009 \times 10^5$$

$$\triangleq 0.524\,92 \times 10^5$$

(符号 $\triangleq$ 表示机器中相等),

结果显然不可靠, 这是由运算中出现了大数 52 492“吃掉”小数 δ_i 所造成的. 如果计算时先把数量级相同的 1 000 个 δ_i 相加, 最后再加 52 492, 就不会出现大数“吃”小数的现象, 这时有

$$0.1 \times 10^3 \leq \sum_{i=1}^{1000} \delta_i \leq 0.9 \times 10^3,$$

$$0.001 \times 10^5 + 0.524\,92 \times 10^5 \leq A \leq 0.009 \times 10^5 + 0.524\,92 \times 10^5,$$

$$52\,592 \leq A \leq 53\,392.$$

1.4.4 注意简化计算步骤，减少运算次数 同样一个计算问题，若能减少运算次数，不但可节省计算机的计算时间，还能减小舍入误差。这是数值计算必须遵从的原则，也是“数值分析”要研究的重要内容。

例 1.9 计算 x^{255} 的值。

解 如果逐个相乘要用 254 次乘法，但若写成

$$x^{255} = x \cdot x^2 \cdot x^4 \cdot x^8 \cdot x^{16} \cdot x^{32} \cdot x^{64} \cdot x^{128},$$

只要做 14 次乘法运算即可。又如计算多项式

$$P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

的值时，若直接计算 $a_k x^k$ 再逐项相加，一共需做

$$n + (n-1) + \cdots + 2 + 1 = \frac{n(n+1)}{2}$$

校注（agent 补充）：本页末尾乘法次数公式的说明续下页“次乘法 and n 次加法”。

原书 PDF 页 25; 印刷页码 12。

[查看原页](#)

次乘法和 n 次加法. 若采用秦九韶算法

$$\begin{cases} S_n = a_n, \\ S_k = xS_{k+1} + a_k \quad (k = n-1, n-2, \dots, 0), \\ P_n(x) = S_0, \end{cases} \quad (1.4.1)$$

只要做 n 次乘法和 n 次加法就可算出 $P_n(x)$ 的值.

在“数值分析”中, 这种节省计算次数的算法还有不少. 本书第 3 章介绍的 FFT 算法, 就是一个最成功的范例.

小结

误差问题是“数值分析”中重要而又困难的课题. 本章只介绍误差基本概念与分析误差的若干原则, 这对学习本课程是必需的. 但作为工程或科学计算的误差问题则要复杂得多, 人们往往根据不同问题分门别类进行研究. 这里不作详细介绍, 有兴趣的读者请参看有关文献, 例如文献 [1]、[2] 等.

习题

1. 设 $x > 0$, x 的相对误差为 δ , 求 $\ln x$ 的误差.
2. 设 x 的相对误差为 2%, 求 x^n 的相对误差.
3. 下列各数都是经过四舍五入得到的近似数, 即误差限不超过最后一位的半个单位, 试指出它们是几位有效数字:

$$x_1^* = 1.1021, x_2^* = 0.031, x_3^* = 385.6, x_4^* = 56.430, x_5^* = 7 \times 1.0.$$

4. 利用式 (1.3.3) 求下列近似值的误差限:
(1) $x_1^* + x_2^* + x_4^*$, (2) $x_1^* x_2^* x_3^*$, (3) x_2^*/x_4^* , 其中 $x_1^*, x_2^*, x_3^*, x_4^*$ 均为习题 3 所给的数.
5. 计算球体积要使相对误差限为 1%, 问度量半径为 R 时允许的相对误差限是多少.
6. 设 $Y_0 = 28$, 按递推公式

$$Y_n = Y_{n-1} - \frac{1}{100} \sqrt{783} \quad (n = 1, 2, \dots)$$

计算到 Y_{100} . 若取 $\sqrt{783} \approx 27.982$ (五位有效数字), 试问计算 Y_{100} 将有多大误差.

7. 求方程 $x^2 - 56x + 1 = 0$ 的两个根, 使它至少具有四位有效数字 ($\sqrt{783} \approx 27.982$).
8. 当 N 充分大时, 怎样求 $\int_N^{N+1} \frac{1}{1+x^2} dx$?
9. 正方形的边长大约为 100 cm, 应怎样测量才能使其面积误差不超过 1 cm²?
10. 设 $s = \frac{1}{2}gt^2$, 假定 g 是准确的, 而对 t 的测量有 ± 0.1 s 的误差, 证明: 当 t 增加时 s 的绝对误差增大, 而相对误差却减小.

11. 序列 $\{y_n\}$ 满足递推关系 $y_n = 10y_{n-1} - 1 (n = 1, 2, \dots)$, 若 $y_0 = \sqrt{2} \approx 1.41$ (三位有效数字),

校注 (agent 补充): 习题 3 第五个数的原图清楚写为 $x_5^* = 7 \times 1.0$, 未作简化或改写。习题 11 在本页仅给出条件, 题目后半句续下页。

原书 PDF 页 26; 印刷页码 13。

[查看原页](#)

计算到 y_{10} 时误差有多大? 这个计算过程稳定吗?

12. 计算 $(\sqrt{2}-1)^6$, 取 $\sqrt{2} \approx 1.4$, 利用下式计算, 哪一个得到的结果最好?

$$\frac{1}{(\sqrt{2}+1)^6}, \quad (3-2\sqrt{2})^3, \quad \frac{1}{(3+2\sqrt{2})^3}, \quad 99-70\sqrt{2}.$$

13. $f(x) = \ln(x - \sqrt{x^2 - 1})$, 求 $f(30)$ 的值. 求对数时误差有多大? 若改用另一等价公式

$$\ln(x - \sqrt{x^2 - 1}) = -\ln(x + \sqrt{x^2 + 1})$$

计算, 求对数时误差有多大?

14. 试用消元法解方程组 $\begin{cases} x_1 + 10^{10}x_2 = 10^{10}, \\ x_1 + x_2 = 2, \end{cases}$ 假定只用三位数计算, 结果是否可靠?

15. 已知三角形面积 $S = \frac{1}{2}ab \sin c$, 其中 c 为弧度, $0 < c < \frac{\pi}{2}$, 且测量 a, b, c 的误差分别为 $\Delta a, \Delta b, \Delta c$. 证明面积的误差 ΔS 满足

$$\left| \frac{\Delta S}{S} \right| \leq \left| \frac{\Delta a}{a} \right| + \left| \frac{\Delta b}{b} \right| + \left| \frac{\Delta c}{c} \right|.$$

校注 (agent 补充, CH01-E007): 习题 13 所谓“等价公式”的右端, 原图清楚印为 $-\ln(x + \sqrt{x^2 + 1})$, 此处保留加号。对于题中实值范围, $(x - \sqrt{x^2 - 1})(x + \sqrt{x^2 - 1}) = 1$, 故相应等价式的右端根号内应为 $x^2 - 1$ 。这是原书符号疑误, 不是 OCR 错误; 本页只转录题目, 未代做习题。

校注 (agent 补充, CH01-E008): 习题 15 把面积的有限误差直接写成上述不等式, 原文照录。该式可按一阶误差或微分估计理解; 若 Δ 表示有限误差, 乘积的高阶项不能一般地省略。例如 $\Delta a/a = \Delta b/b = 0.01$ 且 $\Delta c = 0$ 时, 精确相对面积变化为 $1.01^2 - 1 = 0.0201 > 0.02$ 。因此题目作为严格有限误差上界缺少限定。

原书 PDF 页 27；印刷页码 14。

原页：书页 14 / PDF 27

第 2 章插值法

2.1 引言

许多实际问题都要用函数 $y = f(x)$ 来表示某种内在规律的数量关系，其中相当一部分函数是通过实验或观测得到的。虽然 $f(x)$ 在某个区间 $[a, b]$ 上是存在的，有的还是连续的，但却只能给出 $[a, b]$ 上一系列点 x_i 的函数值 $y_i = f(x_i)$ ($i = 0, 1, \dots, n$)，这只是一张函数表。有的函数虽有解析表达式，但由于计算复杂，使用不方便，通常也造一个函数表，如大家熟悉的三角函数表、对数表、平方根和立方根表等。为了研究函数的变化规律，往往要求出不在表上的函数值。因此，我们希望根据给定的函数表构造一个既能反映函数 $f(x)$ 的特性、又便于计算的简单函数 $P(x)$ ，用 $P(x)$ 近似 $f(x)$ 。通常选一类较简单的函数如代数多项式或分段代数多项式作为 $P(x)$ ，并使 $P(x_i) = f(x_i)$ 对于 $i = 0, 1, \dots, n$ 成立，这样确定的 $P(x)$ 就是我们希望得到的插值函数。例如，在现代机械工业中用计算机程序控制加工机械零件，根据设计可给出零件外形曲线的某些型值点 (x_i, y_i) ($i = 0, 1, \dots, n$)，加工时为控制每步走刀方向及步数，就要算出零件外形曲线其他点的函数值，才能加工出外表光滑的零件，这就是求插值函数的问题。下面给出有关插值法的定义。

定义 2.1 设函数 $y = f(x)$ 在区间 $[a, b]$ 上有定义，且已知在点 $a \leq x_0 \leq x_1 < \dots < x_n \leq b$ 上的值 $y_0, y_1, \dots, y_n$ ，若存在一简单函数 $P(x)$ ，使

$$P(x_i) = y_i \quad (i = 0, 1, \dots, n) \quad (2.1.1)$$

成立，就称 $P(x)$ 为 $f(x)$ 的**插值函数**。点 $x_0, x_1, \dots, x_n$ 称为**插值节点**，包含插值节点的区间 $[a, b]$ 称为**插值区间**，求插值函数 $P(x)$ 的方法称为**插值法**。若 $P(x)$ 是次数不超过 n 的代数多项式，即

$$P(x) = a_0 + a_1x + \dots + a_nx^n, \quad (2.1.2)$$

其中 a_i 为实数，就称 $P(x)$ 为**插值多项式**，相应的插值法称为**多项式插值**；若 $P(x)$ 为分段的多项式，就称之为**分段插值**；若 $P(x)$ 为三角多项式，就称之为**三角插值**。本章只讨论多项式插值与分段插值。

从几何图形上看，插值法就是求曲线 $y = P(x)$ ，使其通过给定的 $n + 1$ 个点 (x_i, y_i) ， $i = 0, 1, \dots, n$ ，并用它近似已知曲线 $y = f(x)$ ，如图 2.1 所示。

插值法是一种古老的数学方法，它来自生产实践。早在一千多年前，我国科学家在研究历法中就应用了线性插值与二次插值，但它的基本理论和结果却是在微积分

本页末句续 PDF 28。

校注 (agent 补充): NA5E-E001

扫描原页中，定义 2.1 的前两个节点确实印为 $x_0 \leq x_1$ ；这里忠实保留。它与 2.2.1 证明中“ $i \neq j$ 时 $x_i \neq x_j$ ”的前提不一致。普通 Lagrange 插值存在唯一性要求节点互异，使用时应采用

$$a \leq x_0 < x_1 < \dots < x_n \leq b.$$

核验反例：当 $n = 1$ 、 $x_0 = x_1 = 0$ 、 $y_0 = y_1 = c$ 时，所有 $P(x) = c + tx$ （任意实数 t ）都满足节点条件，故不唯一。这里的严格不等号是有明确依据的校注，不能把它当作原书实际印字。

原书 PDF 页 28；印刷页码 15。

[原页：书页 15 / PDF 28](#)

页首接 PDF 27 的“微积分”。

产生以后才逐步完善的，其应用也日益增多。特别是在电子计算机广泛使用以后，由于航空、造船、精密机械加工等实际问题的需要，插值法在实践上和理论上显得更为重要，由此得到进一步发展，尤其是近几十年发展起来的样条 (spline) 插值，获得了更为广泛的应用。

本章主要研究如何求出插值函数（包括分段插值函数及样条插值函数），并讨论插值函数 $P(x)$ 的存在唯一性、收敛性及误差估计等。

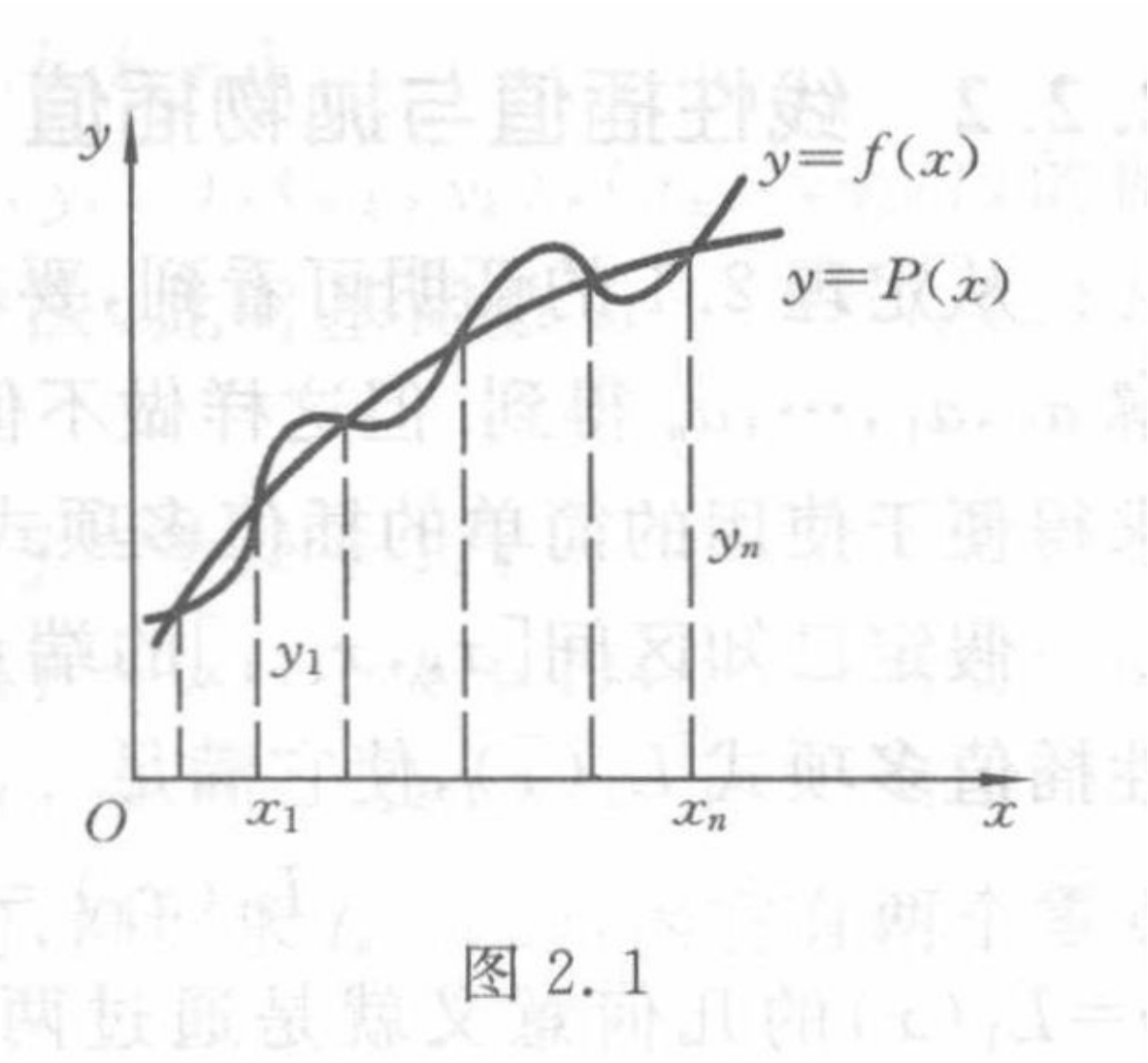


图 2.1

图 2.1 (原图): 曲线 $y = P(x)$ 与 $y = f(x)$ 通过共同的插值节点。全部文字标签: y 、 x 、 O 、 x_1 、 x_n 、 y_1 、 y_n 、 $y = f(x)$ 、 $y = P(x)$ 。

2.2 Lagrange 插值

2.2.1 插值多项式的存在唯一性

设 $P(x)$ 是形如式 (2.1.2) 的插值多项式, 用 H_n 代表所有次数不超过 n 的多项式集合, 于是 $P(x) \in H_n$ (符号 $\in$ 表示属于)。所谓插值多项式 $P(x)$ 存在唯一, 就是指在集合 H_n 中有且只有一个 $P(x)$ 满足式 (2.1.1)。由式 (2.1.1) 可得

$$\begin{cases} a_0 + a_1x_0 + \cdots + a_nx_0^n = y_0, \\ a_0 + a_1x_1 + \cdots + a_nx_1^n = y_1, \\ \vdots \\ a_0 + a_1x_n + \cdots + a_nx_n^n = y_n. \end{cases} \quad (2.2.1)$$

这是一个关于 $a_0, a_1, \dots, a_n$ 的 $n+1$ 元线性方程组。要证明插值多项式的存在唯一, 只要证明方程组 (2.2.1) 的解存在唯一, 也就是证明方程组 (2.2.1) 的系数行列式

$$V_n(x_0, x_1, \dots, x_n) = \begin{vmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{vmatrix} \quad (2.2.2)$$

不为零, 其中 $V_n(x_0, x_1, \dots, x_n)$ 称为 Vandermonde 行列式。利用行列式性质可得

$$V_n(x_0, x_1, \dots, x_n) = \prod_{i=1}^n \prod_{j=0}^{i-1} (x_i - x_j),$$

由于 $i \neq j$ 时 $x_i \neq x_j$, 上式所有因子 $x_i - x_j \neq 0$, 于是 $V_n(x_0, x_1, \dots, x_n) \neq 0$, 故方程组 (2.2.1) 存在唯一的一组解 $a_0, a_1, \dots, a_n$ 。以上论述可写成如下定理。

定理 2.1 满足条件式 (2.1.1) 的插值多项式 (2.1.2) 存在唯一。

原书 PDF 页 29; 印刷页码 16。

[原页: 书页 16 / PDF 29](#)

2.2.2 线性插值与抛物插值

从定理 2.1 的证明可看到, 要求插值多项式 $P(x)$, 可以通过求方程组 (2.2.1) 的解 $a_0, a_1, \dots, a_n$ 得到。但这样做不但计算复杂, 且难以得到 $P(x)$ 的简单表达式。为了求得便于使用的简单的插值多项式 $P(x)$, 先讨论 $n = 1$ 的情形。

假定已知区间 $[x_k, x_{k+1}]$ 的端点处的函数值 $y_k = f(x_k)$, $y_{k+1} = f(x_{k+1})$, 要求线性插值多项式 $L_1(x)$, 使它满足

$$L_1(x_k) = y_k, \quad L_1(x_{k+1}) = y_{k+1}.$$

$y = L_1(x)$ 的几何意义就是通过两点 (x_k, y_k) 和 (x_{k+1}, y_{k+1}) 的直线, 如图 2.2 所示。 $L_1(x)$ 的表达式可由几何意义直接给出, 即

$$L_1(x) = y_k + \frac{y_{k+1} - y_k}{x_{k+1} - x_k}(x - x_k) \quad (\text{点斜式}), \quad (2.2.3)$$

$$L_1(x) = \frac{x_{k+1} - x}{x_{k+1} - x_k}y_k + \frac{x - x_k}{x_{k+1} - x_k}y_{k+1} \quad (\text{两点式}). \quad (2.2.4)$$

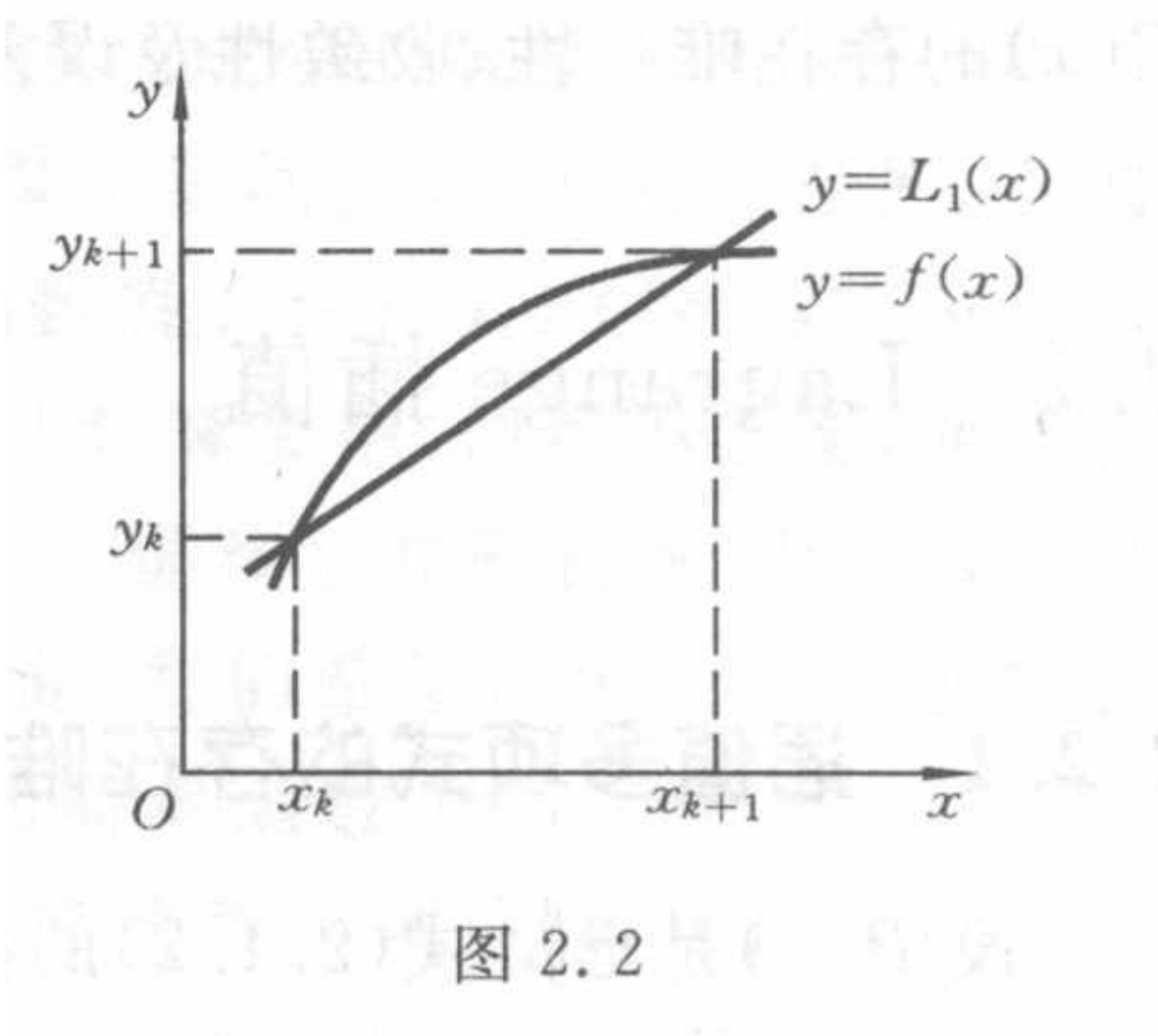


图 2.2

图 2.2 (原图): 直线 $y = L_1(x)$ 与曲线 $y = f(x)$ 在两节点相交。全部文字标签: y 、 x 、 O 、 x_k 、 x_{k+1} 、 y_k 、 y_{k+1} 、 $y = L_1(x)$ 、 $y = f(x)$ 。

由两点式方程看出, $L_1(x)$ 是由两个线性函数

$$l_k(x) = \frac{x - x_{k+1}}{x_k - x_{k+1}}, \quad l_{k+1}(x) = \frac{x - x_k}{x_{k+1} - x_k}$$

的线性组合得到的, 其系数分别为 y_k 及 y_{k+1} , 即

$$L_1(x) = y_k l_k(x) + y_{k+1} l_{k+1}(x). \quad (2.2.5)$$

显然, $l_k(x)$ 及 $l_{k+1}(x)$ 也是线性插值多项式, 在节点 x_k 及 x_{k+1} 上满足条件

$$l_k(x_k) = 1, \quad l_k(x_{k+1}) = 0, \quad l_{k+1}(x_k) = 0, \quad l_{k+1}(x_{k+1}) = 1.$$

称函数 $l_k(x)$ 及 $l_{k+1}(x)$ 为**一次插值基函数**或**线性插值基函数**, 它们的图形分别如图 2.3(a)、(b) 所示。

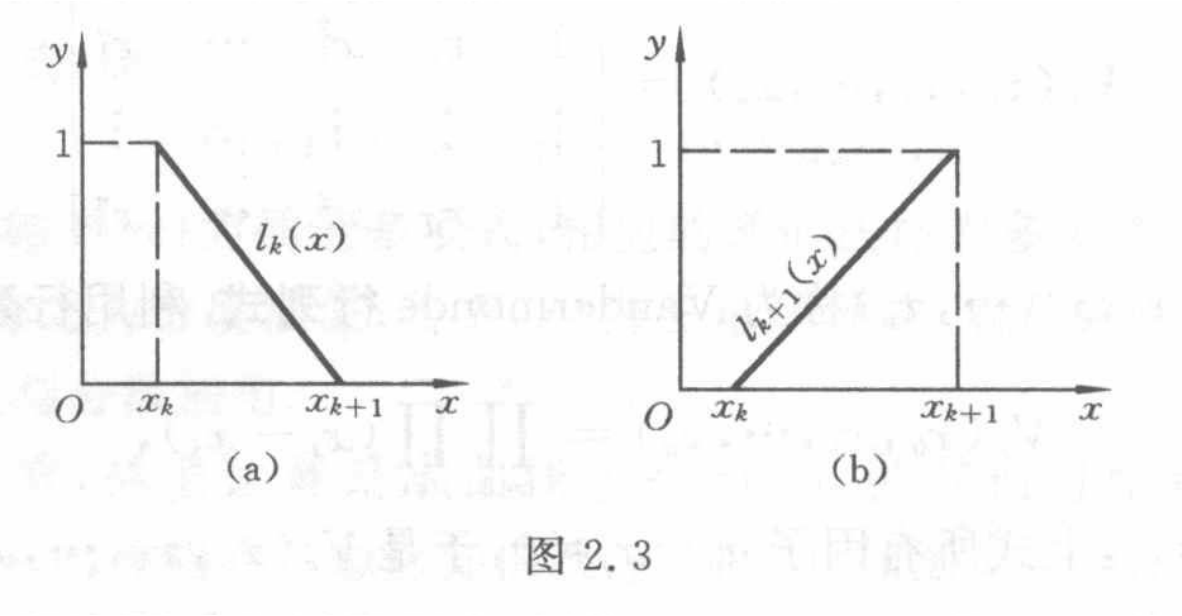


图 2.3

图 2.3 (原图): (a) $l_k(x)$ 在 x_k 处为 1, 在 x_{k+1} 处为 0; (b) $l_{k+1}(x)$ 在 x_k 处为 0, 在 x_{k+1} 处为 1。两图全部文字标签: y 、 x 、 O 、 1 、 x_k 、 x_{k+1} 、 $l_k(x)$ 、 $l_{k+1}(x)$ 、(a)、(b)。

下面讨论 $n = 2$ 的情况。假定插值节点为 x_{k-1}, x_k, x_{k+1} , 要求二次插值多项式 $L_2(x)$, 使它满足

页末插值条件续 PDF 30。

原书 PDF 页 30；印刷页码 17。

原页：书页 17 / PDF 30

页首承接 PDF 29 的二次插值条件；末式展开续 PDF 31。

$$L_2(x_j) = y_j \quad (j = k-1, k, k+1).$$

我们知道， $y = L_2(x)$ 在几何上就是通过三点 (x_{k-1}, y_{k-1}) 、 (x_k, y_k) 、 (x_{k+1}, y_{k+1}) 的抛物线。为了求出 $L_2(x)$ 的表达式，可采用基函数方法，此时基函数 $l_{k-1}(x)$ 、 $l_k(x)$ 及 $l_{k+1}(x)$ 是二次函数，且在节点上满足条件

$$\begin{cases} l_{k-1}(x_{k-1}) = 1, & l_{k-1}(x_j) = 0 \quad (j = k, k+1), \\ l_k(x_k) = 1, & l_k(x_j) = 0 \quad (j = k-1, k+1), \\ l_{k+1}(x_{k+1}) = 1, & l_{k+1}(x_j) = 0 \quad (j = k-1, k). \end{cases} \quad (2.2.6)$$

满足条件式 (2.2.6) 的插值基函数是很容易求出的，例如求 $l_{k-1}(x)$ ，因它有两个零点 x_k 及 x_{k+1} ，故可表示为

$$l_{k-1}(x) = A(x - x_k)(x - x_{k+1}),$$

其中 A 为待定系数，可由条件 $l_{k-1}(x_{k-1}) = 1$ 定出，即

$$A = \frac{1}{(x_{k-1} - x_k)(x_{k-1} - x_{k+1})},$$

故

$$l_{k-1}(x) = \frac{(x - x_k)(x - x_{k+1})}{(x_{k-1} - x_k)(x_{k-1} - x_{k+1})}.$$

同理，

$$l_k(x) = \frac{(x - x_{k-1})(x - x_{k+1})}{(x_k - x_{k-1})(x_k - x_{k+1})}, \quad l_{k+1}(x) = \frac{(x - x_{k-1})(x - x_k)}{(x_{k+1} - x_{k-1})(x_{k+1} - x_k)}.$$

函数 $l_{k-1}(x)$ 、 $l_k(x)$ 、 $l_{k+1}(x)$ 称为二次插值基函数或抛物插值基函数，它们在区间 $[x_{k-1}, x_{k+1}]$ 上的图形分别如图 2.4(a)、(b)、(c) 所示。

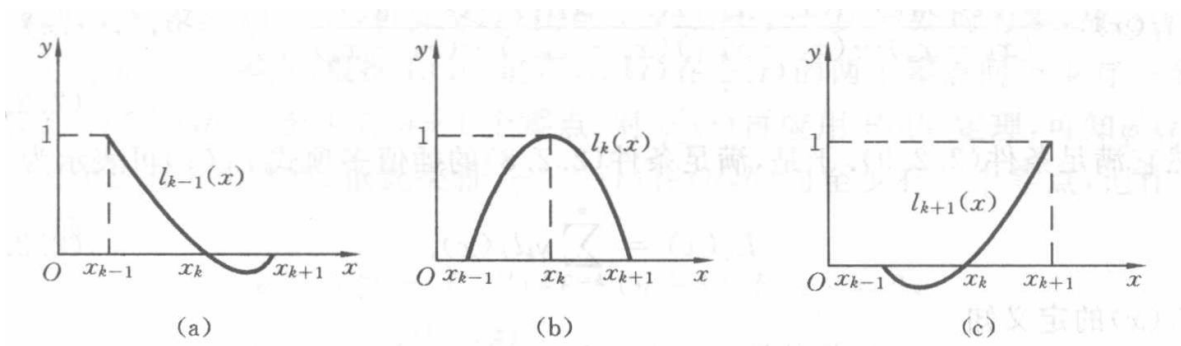


图 2.4

图 2.4 (原图): (a) $l_{k-1}(x)$, (b) $l_k(x)$, (c) $l_{k+1}(x)$, 分别在对应节点取 1、其他两个节点取 0; 左右基函数在部分区间为负。三图全部文字标签: y 、 x 、 O 、1、 x_{k-1} 、 x_k 、 x_{k+1} 、 $l_{k-1}(x)$ 、 $l_k(x)$ 、 $l_{k+1}(x)$ 、(a)、(b)、(c)。

利用二次插值基函数 $l_{k-1}(x), l_k(x), l_{k+1}(x)$, 立即可得到二次插值多项式

$$L_2(x) = y_{k-1}l_{k-1}(x) + y_k l_k(x) + y_{k+1}l_{k+1}(x), \quad (2.2.7)$$

显然, 它满足条件

$$L_2(x_j) = y_j \quad (j = k-1, k, k+1).$$

将上面求得的 $l_{k-1}(x), l_k(x), l_{k+1}(x)$ 代入式 (2.2.7), 得

原书 PDF 页 31; 印刷页码 18。

原页: 书页 18 / PDF 31

页首续 PDF 30 的式 (2.2.7) 展开; 页末公式续 PDF 32。

$$\begin{aligned} L_2(x) = & y_{k-1} \frac{(x-x_k)(x-x_{k+1})}{(x_{k-1}-x_k)(x_{k-1}-x_{k+1})} \\ & + y_k \frac{(x-x_{k-1})(x-x_{k+1})}{(x_k-x_{k-1})(x_k-x_{k+1})} \\ & + y_{k+1} \frac{(x-x_{k-1})(x-x_k)}{(x_{k+1}-x_{k-1})(x_{k+1}-x_k)}. \end{aligned}$$

2.2.3 Lagrange 插值多项式

上面对 $n=1$ 及 $n=2$ 的情况, 得到了一次与二次插值多项式 $L_1(x)$ 及 $L_2(x)$, 它们分别由式 (2.2.5) 和式 (2.2.7) 表示。这种用插值基函数表示的方法容易推广到一般情形。下面讨论通过 $n+1$ 个节点 $x_0 < x_1 < \cdots < x_n$ 的 n 次插值多项式 $L_n(x)$, 假定它满足条件

$$L_n(x_j) = y_j \quad (j = 0, 1, \dots, n). \quad (2.2.8)$$

为了构造 $L_n(x)$, 先定义 n 次插值基函数。

定义 2.2 若 n 次多项式 $l_j(x)$ ($j = 0, 1, \dots, n$) 在 $n+1$ 个节点 $x_0 < x_1 < \cdots < x_n$ 上满足条件

$$l_j(x_k) = \begin{cases} 1, & k = j, \\ 0, & k \neq j \end{cases} \quad (j, k = 0, 1, \dots, n), \quad (2.2.9)$$

就称这 $n+1$ 个 n 次多项式 $l_0(x), l_1(x), \dots, l_n(x)$ 为节点 $x_0, x_1, \dots, x_n$ 上的 n 次插值基函数。

$n=1$ 及 $n=2$ 时的情况前面已经讨论。用类似的推导方法, 可得到 n 次插值基函数为

$$l_k(x) = \frac{(x-x_0) \cdots (x-x_{k-1})(x-x_{k+1}) \cdots (x-x_n)}{(x_k-x_0) \cdots (x_k-x_{k-1})(x_k-x_{k+1}) \cdots (x_k-x_n)} \quad (k = 0, 1, \dots, n). \quad (2.2.10)$$

显然它满足条件 (2.2.9)。于是, 满足条件 (2.2.8) 的插值多项式 $L_n(x)$ 可表示为

$$L_n(x) = \sum_{k=0}^n y_k l_k(x). \quad (2.2.11)$$

由 $l_k(x)$ 的定义知

$$L_n(x_j) = \sum_{k=0}^n y_k l_k(x_j) = y_j \quad (j = 0, 1, \dots, n).$$

形如式 (2.2.11) 的插值多项式 $L_n(x)$ 称为 **Lagrange 插值多项式**, 而式 (2.2.5) 和式 (2.2.7) 是其 $n=1$ 和 $n=2$ 的特殊情形。若引入记号

$$\omega_{n+1}(x) = (x - x_0)(x - x_1) \cdots (x - x_n), \quad (2.2.12)$$

容易求得

$$\omega'_{n+1}(x_k) = (x_k - x_0) \cdots (x_k - x_{k-1})(x_k - x_{k+1}) \cdots (x_k - x_n),$$

于是式 (2.2.11) 可改写成

原书 PDF 页 32; 印刷页码 19。

原页: 书页 19 / PDF 32

页首接 PDF 31; 页末线性插值余项续 PDF 33。

$$L_n(x) = \sum_{k=0}^n y_k \frac{\omega_{n+1}(x)}{(x-x_k)\omega'_{n+1}(x_k)}. \quad (2.2.13)$$

注意, n 次插值多项式 $L_n(x)$ 通常是次数为 n 的多项式, 特殊情况次数可能小于 n 。例如, 通过三点 $(x_0, y_0), (x_1, y_1), (x_2, y_2)$ 的二次插值多项式 $L_2(x)$, 如果三点共线, 则 $y = L_2(x)$ 就是一条直线, 而不是抛物线, 这时 $L_2(x)$ 是一次式。

2.2.4 插值余项

若在 $[a, b]$ 上用 $L_n(x)$ 近似 $f(x)$, 则其截断误差为 $R_n(x) = f(x) - L_n(x)$, $R_n(x)$ 也称为插值多项式的余项或插值余项。关于插值余项估计有以下定理。

定理 2.2 设 $f^{(n)}(x)$ 在 $[a, b]$ 上连续, $f^{(n+1)}(x)$ 在 (a, b) 内存在, 节点 $a \leq x_0 < x_1 < \cdots < x_n \leq b$, $L_n(x)$ 是满足条件式 (2.2.8) 的插值多项式, 则对于任何 $x \in [a, b]$, 插值余项

$$R_n(x) = f(x) - L_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x). \quad (2.2.14)$$

这里, $\xi \in (a, b)$ 且依赖于 x , $\omega_{n+1}(x)$ 是式 (2.2.12) 所定义的。

证明由给定条件知, $R_n(x)$ 在节点 x_k ($k = 0, 1, \cdots, n$) 上为零, 即

$$R_n(x_k) = 0 \quad (k = 0, 1, \cdots, n),$$

于是

$$R_n(x) = K(x)(x-x_0)(x-x_1)\cdots(x-x_n) = K(x)\omega_{n+1}(x), \quad (2.2.15)$$

其中 $K(x)$ 是与 x 有关的待定函数。

现把 x 看成 $[a, b]$ 上一个固定点, 作函数

$$\varphi(t) = f(t) - L_n(t) - K(x)(t-x_0)(t-x_1)\cdots(t-x_n),$$

根据插值条件及余项定义, 可知 $\varphi(t)$ 在点 $x_0, x_1, \cdots, x_n$ 及 x 处均为零, 故 $\varphi(t)$ 在 $[a, b]$ 上有 $n+2$ 个零点, 根据 Rolle 定理, $\varphi'(t)$ 在 $\varphi(t)$ 的两个零点间至少有一个零点, 故 $\varphi'(t)$ 在 $[a, b]$ 上至少有 $n+1$ 个零点。对 $\varphi'(t)$ 再应用 Rolle 定理, 可知 $\varphi''(t)$ 在 $[a, b]$ 上至少有 n 个零点。依此类推, $\varphi^{(n+1)}(t)$ 在 (a, b) 内至少有一个零点, 记作 $\xi \in (a, b)$, 使

$$\varphi^{(n+1)}(\xi) = f^{(n+1)}(\xi) - (n+1)!K(x) = 0,$$

于是

$$K(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}, \quad \xi \in (a, b)$$

且依赖于 x 。

将上式代入式 (2.2.15)，就得到余项表达式 (2.2.14)。证毕。

应当指出，余项表达式只有在 $f(x)$ 的高阶导数存在时才能应用。 ξ 在 (a, b) 内的具体位置通常不可能给出，如果可以求出 $\max_{a \leq x \leq b} |f^{(n+1)}(x)| = M_{n+1}$ ，那么插值多项式 $L_n(x)$ 逼近 $f(x)$ 的截断误差限是

$$|R_n(x)| \leq \frac{M_{n+1}}{(n+1)!} |\omega_{n+1}(x)|. \quad (2.2.16)$$

当 $n = 1$ 时，线性插值余项为

原书 PDF 页 33; 印刷页码 20。

原页: 书页 20 / PDF 33

页首续 PDF 32 的线性插值余项; 末行误差估计的不等式续 PDF 34。

$$R_1(x) = \frac{1}{2}f''(\xi)\omega_2(x) = \frac{1}{2}f''(\xi)(x-x_0)(x-x_1), \quad \xi \in [x_0, x_1]; \quad (2.2.17)$$

当 $n=2$ 时, 抛物插值的余项为

$$R_2(x) = \frac{1}{6}f'''(\xi)(x-x_0)(x-x_1)(x-x_2), \quad \xi \in [x_0, x_2]. \quad (2.2.18)$$

例 2.1 已知 $\sin 0.32 = 0.314567$, $\sin 0.34 = 0.333487$, $\sin 0.36 = 0.352274$, 用线性插值及抛物插值计算 $\sin 0.3367$ 的值, 并估计截断误差。

解 由题意取 $x_0 = 0.32$, $y_0 = 0.314567$, $x_1 = 0.34$, $y_1 = 0.333487$, $x_2 = 0.36$, $y_2 = 0.352274$ 。

用线性插值计算, 取 $x_0 = 0.32$ 及 $x_1 = 0.34$, 由式 (2.2.3) 得

$$\begin{aligned} \sin 0.3367 &\approx L_1(0.3367) = y_0 + \frac{y_1 - y_0}{x_1 - x_0}(0.3367 - x_0) \\ &= 0.314567 + \frac{0.01892}{0.02} \times 0.0167 \\ &= 0.330365. \end{aligned}$$

其截断误差由式 (2.2.17) 得

$$|R_1(x)| \leq \frac{M_2}{2}|(x-x_0)(x-x_1)|,$$

其中

$$M_2 = \max_{x_0 \leq x \leq x_1} |f''(x)|.$$

因

$$f(x) = \sin x, \quad f''(x) = -\sin x,$$

可取

$$M_2 = \max_{x_0 \leq x \leq x_1} |\sin x| = \sin x_1 \leq 0.3335,$$

于是

$$\begin{aligned}
 |R_1(0.336\,7)| &= |\sin 0.336\,7 - L_1(0.336\,7)| \\
 &\leq \frac{1}{2} \times 0.333\,5 \times 0.016\,7 \times 0.003\,3 \\
 &\leq 0.92 \times 10^{-5}.
 \end{aligned}$$

若取 $x_1 = 0.34$, $x_2 = 0.36$ 为节点, 则线性插值为

$$\begin{aligned}
 \sin 0.336\,7 &\approx \tilde{L}_1(0.336\,7) = y_1 + \frac{y_2 - y_1}{x_2 - x_1}(0.336\,7 - x_1) \\
 &= 0.333\,487 + \frac{0.018\,787}{0.02} \times (-0.003\,3) \\
 &= 0.330\,387,
 \end{aligned}$$

其截断误差为

$$|\tilde{R}_1(x)| \leq \frac{M_2}{2} |(x - x_1)(x - x_2)|,$$

其中

$$M_2 = \max_{x_0 \leq x \leq x_2} |f''(x)| \leq 0.352\,3.$$

于是

$$|\tilde{R}_1(0.336\,7)| = |\sin 0.336\,7 - \tilde{L}_1(0.336\,7)|$$

原书 PDF 页 34; 印刷页码 21。

原页: 书页 21 / PDF 34

页首续 PDF 33 的 $|\widetilde{R}_1(0.3367)|$ 误差估计。

$$\begin{aligned} &\leq \frac{1}{2} \times 0.3523 \times 0.0033 \times 0.0233 \\ &\leq 1.36 \times 10^{-5}. \end{aligned}$$

用抛物插值计算 $\sin 0.3367$ 时, 由式 (2.2.7) 得

$$\begin{aligned} \sin 0.3367 &\approx y_0 \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)} + y_1 \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)} \\ &\quad + y_2 \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)} \\ &= L_2(0.3367) \\ &= 0.314567 \times \frac{0.7689 \times 10^{-4}}{0.0008} + 0.333487 \times \frac{3.89 \times 10^{-4}}{0.0004} \\ &\quad + 0.352274 \times \frac{-0.5511 \times 10^{-4}}{0.0008} \\ &= 0.330374. \end{aligned}$$

这个结果与六位有效数字的正弦函数表完全一样, 这说明查表时用二次插值精度已相当高了。其截断误差限由式 (2.2.18) 得

$$|R_2(x)| \leq \frac{M_3}{6} |(x-x_0)(x-x_1)(x-x_2)|,$$

其中

$$M_3 = \max_{x_0 \leq x \leq x_2} |f'''(x)| = \cos x_0 < 0.828,$$

于是

$$\begin{aligned} |R_2(0.3367)| &= |\sin 0.3367 - L_2(0.3367)| \\ &\leq \frac{1}{6} \times 0.828 \times 0.0167 \times 0.0033 \times 0.0233 \\ &\leq 0.178 \times 10^{-7}. \end{aligned}$$

2.3 逐次线性插值法

用 Lagrange 插值多项式 $L_n(x)$ 计算函数近似值时, 如需增加插值节点, 那么原来算出的数据均不能利用, 必须重新计算。为克服这个缺点, 通常可用逐次线性插值方法求得高次插值。如例 2.1 中 $L_2(0.3367)$ 是由式 (2.2.7) 计算的, 它也可由 $L_1(0.3367)$ 和 $\widetilde{L}_1(0.3367)$ 按类似线性插值的方法计算, 即

$$\begin{aligned}
L_2(0.3367) &= L_1(0.3367) + \frac{\tilde{L}_1(0.3367) - L_1(0.3367)}{0.36 - 0.32} \times (0.3367 - 0.32) \\
&= 0.330365 + \frac{0.000022}{0.04} \times 0.0167 \\
&= 0.330374.
\end{aligned}$$

校注 (agent 补充): NA5E-E002

上面的 0.828、 0.178×10^{-7} 和代入行的 3.89×10^{-4} 均经过原页放大核对, 确为扫描原书印字。**这几行不能直接作为正确的数值结果使用。**

1. 本例以弧度计算, $\cos 0.32 \approx 0.9492354180824409$, 所以 $\cos x_0 < 0.828$ 不成立。可用保守上界 $M_3 < 0.95$ 。
2. 即使照原书使用 0.828, 该乘积也等于 $1.77200694 \times 10^{-7}$, 大于原文所写的 $0.178 \times 10^{-7} = 1.78 \times 10^{-8}$ 。
3. 对正确的导数上界, 用原节点得到的截断误差上界为

$$|R_2(0.3367)| \leq \frac{0.95}{6} (0.0167)(0.0033)(0.0233) = 2.03309975 \times 10^{-7}.$$

这里的 R_2 指用精确节点函数值定义的插值截断误差。

4. 二次插值代入行的中间乘积应为 $0.0167 \times 0.0233 = 0.00038911 = 3.8911 \times 10^{-4}$ 。若严格按原书印出的 3.89×10^{-4} 计算整行, 会得到 0.3302826531125, 不能得到末行的 0.330374。保留精确乘积, 并使用题目给的有限精度表值, 可得

$$\begin{aligned}
L_1(0.3367) &= 0.3303652, \\
\tilde{L}_1(0.3367) &= 0.330387145, \\
L_2(0.3367) &= 0.3303743620375.
\end{aligned}$$

最后一项保留六位有效数字确实为 0.330374, 与 $\sin 0.3367 \approx 0.3303741915556280$ 的六位有效数字一致。

原书 PDF 页 35；印刷页码 22。

原页：书页 22 / PDF 35

本页末句续 PDF 36。

现令 $I_{i_1, i_2, \dots, i_n}(x)$ 表示函数 $f(x)$ 关于节点 $x_{i_1}, x_{i_2}, \dots, x_{i_n}$ 的 $n-1$ 次插值多项式, $I_{i_k}(x)$ 是零次多项式, 记 $I_{i_k}(x) = f(x_{i_k})$, $i_1, i_2, \dots, i_n$ 均为非负整数。一般情况, 两个 k 次插值多项式可通过线性插值得到 $k+1$ 次插值多项式

$$I_{0,1,\dots,k,l}(x) = I_{0,1,\dots,k}(x) + \frac{I_{0,1,\dots,k-1,l}(x) - I_{0,1,\dots,k}(x)}{x_l - x_k}(x - x_k). \quad (2.3.1)$$

这是关于节点 $x_0, \dots, x_k, x_l$ 的插值多项式。显然

$$I_{0,1,\dots,k,l}(x_i) = I_{0,1,\dots,k}(x_i) = f(x_i)$$

对于 $i = 0, 1, \dots, k-1$ 成立。当 $x = x_k$ 时, 有

$$I_{0,1,\dots,k,l}(x_k) = I_{0,1,\dots,k}(x_k) = f(x_k),$$

当 $x = x_l$ 时, 有

$$\begin{aligned} I_{0,1,\dots,k,l}(x_l) &= I_{0,1,\dots,k}(x_l) + \frac{f(x_l) - I_{0,1,\dots,k}(x_l)}{x_l - x_k}(x_l - x_k) \\ &= f(x_l). \end{aligned}$$

这就证明了式 (2.3.1) 的插值多项式满足插值条件, 称式 (2.3.1) 为 **Aitken 逐次线性插值公式**。当 $k = 0$ 时为线性插值。当 $k = 1$ 时插值节点为 x_0, x_1, x_l , 插值多项式为

$$I_{0,1,l}(x) = I_{0,1}(x) + \frac{I_{0,l}(x) - I_{0,1}(x)}{x_l - x_1}(x - x_1).$$

计算时可由 $k = 0$ 到 $k = n-1$ 逐次求得所需的插值多项式, 计算过程如表 2.1 所示。

表 2.1

表中“节点 / 函数值 / 1 次至 4 次 / 原表最右列”为 agent 添加的列名；原表无表头，以下保留全部单元格。

节点	函数值	1 次	2 次	3 次	4 次	原表最右列
x_0	$f(x_0) = I_0$					$x - x_0$
x_1	$f(x_1) = I_1$	$I_{0,1}$				$x - x_1$
x_2	$f(x_2) = I_2$	$I_{0,2}$	$I_{0,1,2}$			$x - x_2$
x_3	$f(x_3) = I_3$	$I_{0,3}$	$I_{0,1,3}$	$I_{0,1,2,3}$		$x - x_3$
x_4	$f(x_4) = I_4$	$I_{0,4}$	$I_{0,1,4}$	$I_{0,1,2,4}$	$I_{0,1,2,3,4}$	$x - x_4$

式 (2.3.1) 也可改为以下计算公式：

$$I_{0,1,\cdots,k+1}(x) = I_{0,1,\cdots,k}(x) + \frac{I_{1,2,\cdots,k+1}(x) - I_{0,1,\cdots,k}(x)}{x_{k+1} - x_0}(x - x_0),$$

(2.3.2)

并称它为 **Neville 算法**，其计算过程如表 2.2 所示。

表 2.2

表中列名为 agent 添加；原表以斜线联结 $I_{0,1}$ 、 $I_{0,1,2}$ 、 $I_{0,1,2,3}$ 、 $I_{0,1,2,3,4}$ 。最右列首行原印为 $x - x_0$ ，其后为 $x_i - x_0$ 。

节点	函数值	1 次	2 次	3 次	4 次	原表最右列
x_0	$f(x_0) = I_0$					$x - x_0$
x_1	$f(x_1) = I_1$	$I_{0,1}$				$x_1 - x_0$
x_2	$f(x_2) = I_2$	$I_{1,2}$	$I_{0,1,2}$			$x_2 - x_0$
x_3	$f(x_3) = I_3$	$I_{2,3}$	$I_{1,2,3}$	$I_{0,1,2,3}$		$x_3 - x_0$
x_4	$f(x_4) = I_4$	$I_{3,4}$	$I_{2,3,4}$	$I_{1,2,3,4}$	$I_{0,1,2,3,4}$	$x_4 - x_0$

从表 2.2 看到：每增加一个节点就计算一行，斜线上是 1 次到 4 次插值多项式的

转录核对说明（agent 补充）：表 2.2 首行最右格已据原图放大核为 $x - x_0$ ，纠正既有章节转录的 $x_0 - x_0$ ；这属于转录错误，不是原书校勘。

表 2.2

x_0	$f(x_0) = I_0$					$x - x_0$
x_1	$f(x_1) = I_1$	$I_{0,1}$				$x_1 - x_0$
x_2	$f(x_2) = I_2$	$I_{1,2}$	$I_{0,1,2}$			$x_2 - x_0$
x_3	$f(x_3) = I_3$	$I_{2,3}$	$I_{1,2,3}$	$I_{0,1,2,3}$		$x_3 - x_0$
x_4	$f(x_4) = I_4$	$I_{3,4}$	$I_{2,3,4}$	$I_{1,2,3,4}$	$I_{0,1,2,3,4}$	$x_4 - x_0$

原书 PDF 页 36；印刷页码 23。

原页：书页 23 / PDF 36

页首接 PDF 35 的“插值多项式的”。

值；如精度不满足要求，再增加一个节点，前面计算完全有效。这个算法适用于在计算机上计算，且具有自动选取节点并逐步比较精度的特点，程序也较简单。

例 2.2 已知 $f(x) = \operatorname{sh} x$ 的值在表 2.3 左端，用 Aitken 插值求 $\operatorname{sh} 0.23$ 的近似值。

解表 2.3 右端是各次插值的计算结果，由于 3 次插值的两个结果相同，因而不需再计算 4 次插值，故求得 $f(0.23) = 0.232\,034$ 。

表 2.3

原表表头为 x_i 、 $f(x_i)$ 、插值结果。左右数据在扫描中采用错行排布，以下分开排成可检索表；“1 次、2 次、3 次”为 agent 添加的列名。

左端节点数据：

x_i	$f(x_i)$
0.00	0.000 0
0.20	0.201 34
0.30	0.304 52
0.50	0.521 10
0.60	0.636 65

右端插值结果：

1 次	2 次	3 次
0.231 54		
0.233 465	0.232 118	
0.239 706	0.232 358	0.232 034
0.244 049	0.232 479	0.232 034

2.4 差商与 Newton 插值公式

2.4.1 差商及其性质

Lagrange 插值公式可以看作直线方程两点式的推广，如果从点斜式直线方程

$$P_1(x) = f_0 + \frac{f_1 - f_0}{x_1 - x_0}(x - x_0)$$

出发，将它推广到具有 $n + 1$ 个插值点 $(x_0, f_0), (x_1, f_1), \cdots, (x_n, f_n)$ 的情况，则可把插值多项式表示为

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0) \cdots (x - x_{n-1}), \quad (2.4.1)$$

其中 $a_0, a_1, \dots, a_n$ 为待定系数, 可由插值条件 $P_n(x_j) = f_j$ ($j = 0, 1, \dots, n$) 确定。

当 $x = x_0$ 时, $P_n(x_0) = a_0 = f_0$; 当 $x = x_1$ 时, $P_n(x_1) = a_0 + a_1(x - x_0) = f_1$, 推得

$$a_1 = \frac{f_1 - f_0}{x_1 - x_0};$$

当 $x = x_2$ 时, $P_n(x_2) = a_0 + a_1(x_2 - x_0) + a_2(x_2 - x_0)(x_2 - x_1) = f_2$,

推得

$$a_2 = \frac{\frac{f_2 - f_0}{x_2 - x_0} - \frac{f_1 - f_0}{x_1 - x_0}}{x_2 - x_1}.$$

依此递推, 可得到 $a_3, a_4, \dots, a_n$ 。为写出系数 a_k 的一般表达式, 先引进如下差商定义。

定义 2.3 称 $f[x_0, x_k] = \frac{f(x_k) - f(x_0)}{x_k - x_0}$ 为函数 $f(x)$ 关于点 x_0, x_k 的一阶差商, 称

定义 2.3 续 PDF 37。

校注 (agent 补充): 代入行的原书记号

上面“当 $x = x_1$ 时”一行, 扫描原书实际写成 $a_0 + a_1(x - x_0)$, 其中 x 没有下标 1, 本转录予以保留。在该行已经限定的 $x = x_1$ 条件下, 求系数时即代入 x_1 ; 不将这一记号静默改写成原文没有印出的 $a_1(x_1 - x_0)$ 。原页的 a_2 分式有整体分母 $x_2 - x_1$, OCR 漏掉了这一层分母, 本转录已恢复。

$$+ a_n(x - x_0) \cdots (x - x_{n-1}), \quad (2.4.1)$$

其中 $a_0, a_1, \dots, a_n$ 为待定系数, 可由插值条件 $P_n(x_j) = f_j$ ($j = 0, 1, \dots, n$) 确定。

当 $x = x_0$ 时, $P_n(x_0) = a_0 = f_0$; 当 $x = x_1$ 时, $P_n(x_1) = a_0 + a_1(x - x_0) = f_1$, 推得

$$a_1 = \frac{f_1 - f_0}{x_1 - x_0};$$

当 $x = x_2$ 时, $P_n(x_2) = a_0 + a_1(x_2 - x_0) + a_2(x_2 - x_0)(x_2 - x_1) = f_2$,

推得

$$a_2 = \frac{\frac{f_2 - f_0}{x_2 - x_0} - \frac{f_1 - f_0}{x_1 - x_0}}{x_2 - x_1}.$$

原书 PDF 页 37; 印刷页码 24。

原页: 书页 24 / PDF 37

页首承接 PDF 36 的定义 2.3; 页末递推式续 PDF 38。

$$f[x_0, x_1, x_k] = \frac{f[x_0, x_k] - f[x_0, x_1]}{x_k - x_1}$$

为 $f(x)$ 关于点 x_0, x_1, x_k 的二阶差商。一般地, 称

$$f[x_0, x_1, \dots, x_k] = \frac{f[x_0, x_1, \dots, x_{k-2}, x_k] - f[x_0, x_1, \dots, x_{k-1}]}{x_k - x_{k-1}} \quad (2.4.2)$$

为 $f(x)$ 的 k 阶差商。

差商有如下的基本性质:

(1) k 阶差商可表示为函数值 $f(x_0), f(x_1), \dots, f(x_k)$ 的线性组合, 即

$$f[x_0, x_1, \dots, x_k] = \sum_{j=0}^k \frac{f(x_j)}{(x_j - x_0) \cdots (x_j - x_{j-1})(x_j - x_{j+1}) \cdots (x_j - x_k)}. \quad (2.4.3)$$

这个性质可用归纳法证明。这个性质也表明差商与节点的排列次序无关, 称为差商的对称性, 即

$$f[x_0, x_1, \dots, x_k] = f[x_1, x_0, x_2, \dots, x_k] = \cdots = f[x_1, x_2, \dots, x_k, x_0].$$

(2) 由性质 (1) 及式 (2.4.2) 可得

$$f[x_0, x_1, \dots, x_k] = \frac{f[x_1, x_2, \dots, x_k] - f[x_0, x_1, \dots, x_{k-1}]}{x_k - x_0}. \quad (2.4.4)$$

(3) 若 $f(x)$ 在 $[a, b]$ 上存在 n 阶导数, 且节点 $x_0, x_1, \dots, x_n \in [a, b]$, 则 n 阶差商与导数关系为

$$f[x_0, x_1, \dots, x_n] = \frac{f^{(n)}(\xi)}{n!}, \quad \xi \in [a, b]. \quad (2.4.5)$$

这个公式可直接用 Rolle 定理证明。

差商的其他性质还可见本章习题。差商计算可列差商表如表 2.4 所示。

表 2.4

x_k	$f(x_k)$	一阶差商	二阶差商	三阶差商	四阶差商
x_0	$f(x_0)$				
x_1	$f(x_1)$	$f[x_0, x_1]$			
x_2	$f(x_2)$	$f[x_1, x_2]$	$f[x_0, x_1, x_2]$		
x_3	$f(x_3)$	$f[x_2, x_3]$	$f[x_1, x_2, x_3]$	$f[x_0, x_1, x_2, x_3]$	
x_4	$f(x_4)$	$f[x_3, x_4]$	$f[x_2, x_3, x_4]$	$f[x_1, x_2, x_3, x_4]$	$f[x_0, x_1, x_2, x_3, x_4]$

x_k	$f(x_k)$	一阶差商	二阶差商	三阶差商	四阶差商
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

2.4.2 Newton 插值公式

根据差商定义，把 x 看成 $[a, b]$ 上一点，可得

$$f(x) = f(x_0) + f[x, x_0](x - x_0),$$

原书 PDF 页 38; 印刷页码 25。

原页: 书页 25 / PDF 38

页首承接 PDF 37 的递推式; 本页例 2.3 的计算结果续 PDF 39。

$$\begin{aligned} f[x, x_0] &= f[x_0, x_1] + f[x, x_0, x_1](x - x_1), \\ &\vdots \\ f[x, x_0, \dots, x_{n-1}] &= f[x_0, x_1, \dots, x_n] + f[x, x_0, \dots, x_n](x - x_n). \end{aligned}$$

只要把后一式代入前一式, 就可得到

$$\begin{aligned} f(x) &= f(x_0) + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots \\ &\quad + f[x_0, x_1, \dots, x_n](x - x_0) \cdots (x - x_{n-1}) + f[x, x_0, \dots, x_n]\omega_{n+1}(x) \\ &= N_n(x) + R_n(x), \end{aligned}$$

其中

$$\begin{aligned} N_n(x) &= f(x_0) + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots \\ &\quad + f[x_0, x_1, \dots, x_n](x - x_0)(x - x_1) \cdots (x - x_{n-1}), \end{aligned} \quad (2.4.6)$$

$$R_n(x) = f(x) - N_n(x) = f[x, x_0, \dots, x_n]\omega_{n+1}(x). \quad (2.4.7)$$

$\omega_{n+1}(x)$ 是由式 (2.2.12) 定义的。由式 (2.4.6) 确定的多项式 $N_n(x)$ 显然满足插值条件, 且次数不超过 n , 它就是形如式 (2.4.1) 的多项式, 其系数为

$$a_k = f[x_0, x_1, \dots, x_k] \quad (k = 0, 1, \dots, n).$$

称 $N_n(x)$ 为 **Newton 差商插值多项式**。系数 a_k 就是差商表 2.4 中加横线的各阶差商, 它比 Lagrange 插值节省计算量, 且便于程序设计。

式 (2.4.7) 为插值余项, 由插值多项式的唯一性知, 它与式 (2.2.14) 是等价的, 事实上, 利用差商与导数关系式 (2.4.5), 可由式 (2.4.7) 推出式 (2.2.14)。但式 (2.4.7) 更有一般性, 它对于 f 是由离散点给出的情形或 f 导数不存在时均适用。

例 2.3 给出 $f(x)$ 的函数表 (见表 2.5 左边两列), 求 4 次 Newton 插值多项式, 并由此计算 $f(0.596)$ 的近似值。

表 2.5

x_k	$f(x_k)$	一阶差商	二阶差商	三阶差商	四阶差商	五阶差商
0.40	0.410 75					
0.55	0.578 15	<u>1.116 00</u>				
0.65	0.696 75	1.186 00	<u>0.280 00</u>			
0.80	0.888 11	1.275 73	0.358 93	<u>0.197 33</u>		
0.90	1.026 52	1.384 10	0.433 48	0.213 00	<u>0.031 34</u>	
1.05	1.253 82	1.515 33	0.524 93	0.228 63	0.031 26	-0.000 12

解首先根据给定函数表造出差商表（见表 2.5 的右边五列）。

从差商表看到，四阶差商近似常数。故取 4 次插值多项式 $N_4(x)$ 作近似即可。

$$\begin{aligned} N_4(x) = & 0.41075 + 1.116(x - 0.4) + 0.28(x - 0.4)(x - 0.55) \\ & + 0.19733(x - 0.4)(x - 0.55)(x - 0.65) \\ & + 0.03134(x - 0.4)(x - 0.55)(x - 0.65)(x - 0.8), \end{aligned}$$

校注 (agent 补充): CH02A-S01, 差商的左括号

本页页首递推式的最后一项，原书印为 $f(x, x_0, \dots, x_n]$ ，左右括号不配。本转录保留这一印字。根据定义 2.3、同页后续两式及差商记号，此处应理解为 $f[x, x_0, \dots, x_n]$ ；这是原书排印问题，不是 OCR 错误。CH02A-S01 是本 lane 的临时校注编号。

$$f[x, x_0, \dots, x_{n-1}] = f[x_0, x_1, \dots, x_n] + f(x, x_0, \dots, x_n](x - x_n).$$

校注 (agent 补充): CH02A-S03, 表 2.5 的中间舍入

上表按原书印字转录。若把左端两列所给小数作为精确输入，使用差商递推且保留精确分数，则第三个二阶差商为 $0.43346666\dots$ ，四阶差商分别为 $0.031238095\dots$ 、 $0.031428571\dots$ ，五阶差商为 $2/6825 = 0.000293040293\dots$ ；它们与表中 0.43348 、 0.03134 、 0.03126 、 -0.00012 不同。

逐列检查表明：表中 0.43348 可以由已舍入的一阶差商重算得到，而 0.35893 、 0.52493 更接近保留未舍入一阶差商的计算；原表的三至五阶差商则能从其已印出的前一列经舍入重算得到。因此，本例高阶差商对中间舍入敏感，不能把这张有限精度表当作精确差商表。此处记录的是原书数值计算的舍入一致性问题，不修改原表。

原书 PDF 页 39; 印刷页码 26。

[原页: 书页 26 / PDF 39](#)

页首为 PDF 38 例 2.3 的续算; 页末性质 1 的公式续 PDF 40。

于是

$$f(0.596) \approx N_4(0.596) = 0.631\,95.$$

截断误差

$$|R_4(x)| \approx |f[x_0, x_1, \dots, x_5]\omega_5(0.596)| \leq 3.63 \times 10^{-9}.$$

这说明截断误差很小, 可忽略不计。

2.5 差分与等距节点插值公式

上面讨论了节点任意分布的插值公式, 但实际应用时经常遇到等距节点的情形, 这时插值公式可以进一步简化, 计算也简单得多。为了得到等距节点的插值公式, 下面先介绍差分的概念。

2.5.1 差分及其性质

设函数 $y = f(x)$ 在等距节点 $x_k = x_0 + kh$ ($k = 0, 1, \dots, n$) 上的值 $f_k = f(x_k)$ 为已知, 这里 h 为常数, 称为**步长**。

定义 2.4 偏差

$$\Delta f_k = f_{k+1} - f_k, \quad (2.5.1)$$

$$\nabla f_k = f_k - f_{k-1}, \quad (2.5.2)$$

$$\delta f_k = f(x_k + h/2) - f(x_k - h/2) = f_{k+\frac{1}{2}} - f_{k-\frac{1}{2}} \quad (2.5.3)$$

分别称为 $f(x)$ 在 x_k 处以 h 为步长的**向前差分**、**向后差分**及**中心差分**。符号 Δ, ∇, δ 分别称为**向前差分算子**、**向后差分算子**及**中心差分算子**。

利用一阶差分可定义二阶差分为

$$\Delta^2 f_k = \Delta f_{k+1} - \Delta f_k = f_{k+2} - 2f_{k+1} + f_k.$$

一般地, 可定义 m 阶差分为

$$\Delta^m f_k = \Delta^{m-1} f_{k+1} - \Delta^{m-1} f_k, \quad \nabla^m f_k = \nabla^{m-1} f_k - \nabla^{m-1} f_{k-1}.$$

因中心差分 δf_k 用到 $f_{k+\frac{1}{2}}$ 及 $f_{k-\frac{1}{2}}$ 这两个值, 实际上不是函数表上的值。如果用函数表上的值, 一阶中心差分应写成

$$\delta f_{k+\frac{1}{2}} = f_{k+1} - f_k, \quad \delta f_{k-\frac{1}{2}} = f_k - f_{k-1};$$

二阶中心差分为 $\delta^2 f_k = \delta f_{k+\frac{1}{2}} - \delta f_{k-\frac{1}{2}}$; 等等。

除了已引入的差分算子外, 常用算子符号还有**不变算子** I 及**移位算子** E , 定义如下:

$$If_k = f_k, \quad Ef_k = f_{k+1},$$

于是, 由 $\Delta f_k = f_{k+1} - f_k = Ef_k - If_k = (E - I)f_k$, 可得 $\Delta = E - I$ 。

同理, 可得

$$\nabla = I - E^{-1}, \quad \delta = E^{\frac{1}{2}} - E^{-\frac{1}{2}}.$$

由差分定义并应用算子符号运算可得下列基本性质。

性质 1 各阶差分均可用函数值表示。例如

校注 (agent 补充): 例 2.3 的误差估计范围

页首 3.63×10^{-9} 使用原表 2.5 的五阶差商 -0.00012 及乘积 $\omega_5(0.596)$ 得到, 原文已使用近似号。原表高阶差商的舍入敏感性见 PDF 38 校注 CH02A-S03; 仅凭给定的有限节点表值, 不能对任意与这些节点相符的函数证明这一截断误差上界, 节点数据的舍入误差也未计入。以上是 agent 的适用范围说明, 原书结论仍在正文保留。

校注 (agent 补充): CH02A-S04, 例 2.3 的函数值结果

原书本页实际印为 $N_4(0.596) = 0.63195$, 本转录保留该数。按 PDF 38 原印的四次多项式逐项代入 $x = 0.596$, 得到

$$N_4(0.596) = 0.63191751982370304,$$

保留五位小数为 0.63192, 与原书的 0.63195 不符。这一差异并非末位舍入造成, 属于原书该计算结果的数值错误。

原书 PDF 页 40; 印刷页码 27。

原页: 书页 27 / PDF 40

页首承接 PDF 39 的性质 1。

$$\Delta^n f_k = (E - I)^n f_k = \sum_{j=0}^n (-1)^j \binom{n}{j} E^{n-j} f_k = \sum_{j=0}^n (-1)^j \binom{n}{j} f_{n+k-j}, \quad (2.5.4)$$

$$\nabla^n f_k = (I - E^{-1})^n f_k = \sum_{j=0}^n (-1)^{n-j} \binom{n}{j} E^{j-n} f_k = \sum_{j=0}^n (-1)^{n-j} \binom{n}{j} f_{k+j-n}, \quad (2.5.5)$$

其中 $\binom{n}{j} = \frac{n(n-1)\cdots(n-j+1)}{j!}$ 为二项式展开系数。

性质 2 可用各阶差分表示函数值, 例如, 可用向前差分表示 f_{n+k} , 即

$$f_{n+k} = E^n f_k = (I + \Delta)^n f_k = \sum_{j=0}^n \binom{n}{j} \Delta^j f_k. \quad (2.5.6)$$

性质 3 差商与差分有以下关系, 例如, 对于向前差分, 由定义

$$f[x_k, x_{k+1}] = \frac{f_{k+1} - f_k}{x_{k+1} - x_k} = \frac{\Delta f_k}{h},$$

$$f[x_k, x_{k+1}, x_{k+2}] = \frac{f[x_{k+1}, x_{k+2}] - f[x_k, x_{k+1}]}{x_{k+2} - x_k} = \frac{1}{2h^2} \Delta^2 f_k,$$

一般地有

$$f[x_k, x_{k+1}, \dots, x_{k+m}] = \frac{1}{m!} \frac{1}{h^m} \Delta^m f_k \quad (m = 1, 2, \dots, n). \quad (2.5.7)$$

同理, 对于向后差分, 有

$$f[x_k, x_{k-1}, \dots, x_{k-m}] = \frac{1}{m!} \frac{1}{h^m} \nabla^m f_k. \quad (2.5.8)$$

利用式 (2.5.7) 及式 (2.4.5) 又可得到

$$\Delta^n f_k = h^n f^{(n)}(\xi), \quad (2.5.9)$$

其中 $\xi \in (x_k, x_{k+n})$, 这是差分与导数的关系。差分的其他性质可参看习题 10 ~ 12。

计算差分可列差分表, 表 2.6 是向前差分表。

表 2.6

x_k	Δ	Δ^2	Δ^3	Δ^4
f_0	Δf_0	$\Delta^2 f_0$	$\Delta^3 f_0$	$\Delta^4 f_0$
f_1	Δf_1	$\Delta^2 f_1$	$\Delta^3 f_1$	$\vdots$
f_2	Δf_2	$\Delta^2 f_2$	$\vdots$	
f_3	Δf_3	$\vdots$		
f_4	$\vdots$			
$\vdots$				

校注 (agent 补充): CH02A-S02, 表 2.6 的首列表头

原书表 2.6 首列表头实际印为 x_k , 而其内容为 $f_0, f_1, \cdots$, 本转录均按原图保留。依各阶向前差分意义, 该列存放函数值, 表头应理解为 f_k ; 这是原书表头与内容不一致的问题。CH02A-S02 是本 lane 的临时校注编号。

表 2.6				
x_k	Δ	Δ^2	Δ^3	Δ^4
f_0	Δf_0	$\Delta^2 f_0$	$\Delta^3 f_0$	$\Delta^4 f_0$
f_1	Δf_1	$\Delta^2 f_1$	$\Delta^3 f_1$	$\vdots$
f_2	Δf_2	$\Delta^2 f_2$	$\vdots$	
f_3	Δf_3	$\vdots$		
f_4	$\vdots$			
$\vdots$				

原书 PDF 页 41；印刷页码 28。

原页：书页 28 / PDF 41

2.5.2 等距节点插值公式

将 Newton 差商插值多项式 (2.4.6) 中各阶差商用相应差分代替，就可得到各种形式的等距节点插值公式。这里只推导常用的前插公式与后插公式。

如果节点 $x_k = x_0 + kh$ ($k = 0, 1, \dots, n$)，要计算 x_0 附近点 x 的函数 $f(x)$ 的值，可令 $x = x_0 + th$, $0 \leq t \leq 1$ ，于是

$$\omega_{k+1}(x) = \prod_{j=0}^k (x - x_j) = t(t-1) \cdots (t-k) h^{k+1}.$$

将此式及式 (2.5.7) 代入式 (2.4.6)，得

$$N_n(x_0 + th) = f_0 + t\Delta f_0 + \frac{t(t-1)}{2!} \Delta^2 f_0 + \cdots + \frac{t(t-1) \cdots (t-n+1)}{n!} \Delta^n f_0. \quad (2.5.10)$$

式 (2.5.10) 称为 **Newton 前插公式**，其余项由式 (2.2.14) 得

$$R_n(x) = \frac{t(t-1) \cdots (t-n)}{(n+1)!} h^{n+1} f^{(n+1)}(\xi), \quad \xi \in (x_0, x_n). \quad (2.5.11)$$

如果要求表示函数在 x_n 附近的值 $f(x)$ ，此时应用 Newton 插值公式 (2.4.6)，插值点应按 $x_n, x_{n-1}, \dots, x_0$ 的次序排列，有

$$\begin{aligned} N_n(x) = & f(x_n) + f[x_n, x_{n-1}](x - x_n) + f[x_n, x_{n-1}, x_{n-2}](x - x_n)(x - x_{n-1}) \\ & + \cdots + f[x_n, x_{n-1}, \dots, x_0](x - x_n) \cdots (x - x_1). \end{aligned}$$

作变换 $x = x_n + th$ ($-1 \leq t \leq 0$)，并利用式 (2.5.8)，代入上式得

$$\begin{aligned} N_n(x_n + th) = & f_n + t\nabla f_n + \frac{t(t+1)}{2!} \nabla^2 f_n + \cdots \\ & + \frac{t(t+1) \cdots (t+n-1)}{n!} \nabla^n f_n. \end{aligned} \quad (2.5.12)$$

式 (2.5.12) 称为 **Newton 后插公式**，其余项

$$R_n(x) = f(x) - N_n(x_n + th) = \frac{t(t+1) \cdots (t+n) h^{n+1} f^{(n+1)}(\xi)}{(n+1)!}, \quad \xi \in (x_0, x_n).$$

利用 Newton 前插公式 (2.5.10) 计算函数值时，其系数就是 $f(x)$ 在 x_0 的各阶向前差分（见表 2.6）。若用 Newton 后插公式 (2.5.12) 求 $f(x)$ 的值，因 x 在 x_n 附近，则其系数为 $f(x)$ 在点 x_n 的各阶向后差分。

等距节点插值公式有不少实际应用，例如，很多工程设计计算都需要查各种函数表，用计算机计算时就必须解决计算机查表问题。如果把整个函数表存入内存，往往占用单元太多；如果用一个解析表达式近似该函数，又可能达不到精度要求。因此，采用存放大间隔函数表，并用插值公式计算函数近似值，是一种可行的方案。

例 2.4 在微电机设计计算中需要查磁化曲线表，通常给出的表（见表 2.7）是磁密 B 每间隔 100 高斯磁路每厘米长所需安匝数 at 的值，下面要解决 B 从 4000 至

例 2.4 的区间上限及解答续 PDF 42。

原书 PDF 页 42；印刷页码 29。

原页：书页 29 / PDF 42

页首接 PDF 41 例 2.4 的区间；页末 2.6 首句续 PDF 43。

11 000 区间的查表问题。

解为节省计算机存储单元，采用每 500 高斯存入一个 at 值。为了分析使用几阶插值公式合适，应先列出差分表。表 2.7 是以硅钢片的磁化曲线为例所造的差分表。

从差分表中看到三阶差分近似于 0，因此计算时只需用二阶差分，当 $4\,000 \leq B \leq 10\,500$ 时用 Newton 前插公式，当 $10\,500 < B \leq 11\,000$ 时用 Newton 后插公式。例如，求 $f(5\,200)$ 时取 $B_0 = 5\,000$ ， $f_0 = 1.58$ ， $\Delta f_0 = 0.11$ ， $\Delta^2 f_0 = 0.01$ ， $h = 500$ ， $B = 5\,200$ ， $t = 0.4$ ，于是由式 (2.5.10)，取 $n = 2$ ，得

$$f(5\,200) \approx 1.58 + 0.4 \times 0.11 + \frac{0.4 \times (-0.6)}{2} \times 0.01 \approx 1.62.$$

这个结果与直接查表得到的值相同，说明用此法在计算机上求 at 值是可行的。具体的查表程序作为练习由读者完成。

表 2.7

k	B_k	$at_k = f(B_k)$	Δf_k	$\Delta^2 f_k$	$\Delta^3 f_k$
0	4 000	1.38	0.10	0	0.01
1	4 500	1.48	0.10	0.01	0
2	5 000	1.58	0.11	0.01	0
3	5 500	1.69	0.12	0.01	0.02
4	6 000	1.81	0.13	0.03	−0.01
5	6 500	1.94	0.16	0.02	0.02
6	7 000	2.10	0.18	0.04	0
7	7 500	2.28	0.22	0.04	0
8	8 000	2.50	0.26	0.04	0.01
9	8 500	2.76	0.30	0.05	0.02
10	9 000	3.06	0.35	0.07	0.01
11	9 500	3.41	0.42	0.08	0.02
12	10 000	3.83	0.50	0.10	
13	10 500	4.33	0.60		
14	11 000	4.93			

2.6 Hermite 插值

不少实际问题不但要求在节点上函数值相等，而且还要求它的导数值相等，甚至

原书 PDF 页 43; 印刷页码 30。

[查看原页](#)

本页接上页 2.6 Hermite 插值正文。

要求高阶导数值也相等。满足这种要求的插值多项式就是 Hermite 插值多项式。下面只讨论函数值与导数值个数相等的情况。设在节点 $a \leq x_0 < x_1 < \cdots < x_n \leq b$ 上, $y_j = f(x_j), m_j = f'(x_j)$ ($j = 0, 1, \cdots, n$), 要求插值多项式 $H(x)$, 满足条件

$$H(x_j) = y_j, \quad H'(x_j) = m_j \quad (j = 0, 1, \cdots, n). \quad (2.6.1)$$

这里给出的 $2n + 2$ 个条件, 可唯一确定一个次数不超过 $2n + 1$ 的多项式

$$H_{2n+1}(x) = H(x),$$

其形式为

$$H_{2n+1}(x) = a_0 + a_1x + \cdots + a_{2n+1}x^{2n+1}.$$

如根据条件 (2.6.1) 来确定 $2n + 2$ 个系数 $a_0, a_1, \cdots, a_{2n+1}$, 显然非常复杂, 因此, 仍采用求 Lagrange 插值多项式的基函数方法。先求插值基函数 $\alpha_j(x)$ 及 $\beta_j(x)$ ($j = 0, 1, \cdots, n$), 共有 $2n + 2$ 个, 每一个基函数都是 $2n + 1$ 次多项式, 且满足条件

$$\begin{cases} \alpha_j(x_k) = \delta_{jk} = \begin{cases} 0, & j \neq k, \\ 1, & j = k, \end{cases} & \alpha'_j(x_k) = 0 \quad (j, k = 0, 1, \cdots, n), \\ \beta_j(x_k) = 0, & \beta'_j(x_k) = \delta_{jk}. \end{cases} \quad (2.6.2)$$

于是满足条件 (2.6.1) 的插值多项式 $H(x) = H_{2n+1}(x)$ 可写成用插值基函数表示的形式, 即

$$H_{2n+1}(x) = \sum_{j=0}^n [y_j \alpha_j(x) + m_j \beta_j(x)]. \quad (2.6.3)$$

由条件 (2.6.2), 显然有

$$H_{2n+1}(x_k) = y_k, \quad H'_{2n+1}(x_k) = m_k \quad (k = 0, 1, \cdots, n).$$

下面的问题就是求满足条件 (2.6.2) 的基函数 $\alpha_j(x)$ 及 $\beta_j(x)$ 。为此, 可利用 Lagrange 插值基函数 $l_j(x)$ 。令

$$\alpha_j(x) = (ax + b)l_j^2(x),$$

其中 $l_j(x)$ 是式 (2.2.10) 所表示的基函数。由条件式 (2.6.2), 有

$$\alpha_j(x_j) = (ax_j + b)l_j^2(x_j) = 1,$$

$$\alpha'_j(x_j) = l_j(x_j)[al_j(x_j) + 2(ax_j + b)l'_j(x_j)] = 0,$$

整理, 得

$$\begin{cases} ax_j + b = 1, \\ a + 2l'_j(x_j) = 0. \end{cases}$$

解得

$$a = -2l'_j(x_j), \quad b = 1 + 2x_jl'_j(x_j).$$

由于

$$l_j(x) = \frac{(x - x_0) \cdots (x - x_{j-1})(x - x_{j+1}) \cdots (x - x_n)}{(x_j - x_0) \cdots (x_j - x_{j-1})(x_j - x_{j+1}) \cdots (x_j - x_n)},$$

两端取对数再求导, 得

$$l'_j(x_j) = \sum_{\substack{k=0 \\ k \neq j}}^n \frac{1}{x_j - x_k},$$

续页: 基函数的结果见下一页。

原书 PDF 页 44; 印刷页码 31。

[查看原页](#)

本页接上页 2.6 Hermite 插值正文。

于是

$$\alpha_j(x) = \left[1 - 2(x - x_j) \sum_{\substack{k=0 \\ k \neq j}}^n \frac{1}{x_j - x_k} \right] l_j^2(x). \quad (2.6.4)$$

同理可得

$$\beta_j(x) = (x - x_j) l_j^2(x). \quad (2.6.5)$$

还可证明满足条件 (2.6.1) 的插值多项式是唯一的。用反证法, 假设 $H_{2n+1}(x)$ 及 $\overline{H}_{2n+1}(x)$ 均满足条件 (2.6.1), 于是

$$\varphi(x) = H_{2n+1}(x) - \overline{H}_{2n+1}(x)$$

在每个节点 x_k 上均有二重根, 即 $\varphi(x)$ 有 $2n+2$ 重根。但 $\varphi(x)$ 是不高于 $2n+1$ 次的多项式, 故 $\varphi(x) \equiv 0$ 。唯一性得证。

仿照 Lagrange 插值余项的证明方法, 若 $f(x)$ 在 (a, b) 内的 $2n+2$ 阶导数存在, 则其插值余项

$$R(x) = f(x) - H_{2n+1}(x) = \frac{f^{(2n+2)}(\xi)}{(2n+2)!} \omega_{n+1}^2(x), \quad (2.6.6)$$

其中 $\xi \in (a, b)$ 且与 x 有关。具体证明请读者自行完成。

作为带导数插值多项式 (2.6.3) 的重要特例是 $n=1$ 的情形。这时可取节点为 x_k 及 x_{k+1} , 插值多项式为 $H_3(x)$, 满足条件

$$\begin{cases} H_3(x_k) = y_k, & H_3(x_{k+1}) = y_{k+1}, \\ H'_3(x_k) = m_k, & H'_3(x_{k+1}) = m_{k+1}. \end{cases} \quad (2.6.7)$$

相应的插值基函数为 $\alpha_k(x), \alpha_{k+1}(x), \beta_k(x), \beta_{k+1}(x)$, 它们满足条件

$$\begin{aligned} \alpha_k(x_k) &= 1, & \alpha_k(x_{k+1}) &= 0, & \alpha'_k(x_k) &= \alpha'_k(x_{k+1}) = 0, \\ \alpha_{k+1}(x_k) &= 0, & \alpha_{k+1}(x_{k+1}) &= 1, \\ \alpha'_{k+1}(x_k) &= \alpha'_{k+1}(x_{k+1}) = 0, & \beta_k(x_k) &= \beta_k(x_{k+1}) = 0, \\ \beta'_k(x_k) &= 1, & \beta'_k(x_{k+1}) &= 0, & \beta_{k+1}(x_k) &= \beta_{k+1}(x_{k+1}) = 0, \\ \beta'_{k+1}(x_k) &= 0, & \beta'_{k+1}(x_{k+1}) &= 1. \end{aligned}$$

根据式 (2.6.4) 及式 (2.6.5) 的一般表达式, 可得

$$\begin{cases} \alpha_k(x) = \left(1 + 2 \frac{x - x_k}{x_{k+1} - x_k}\right) \left(\frac{x - x_{k+1}}{x_k - x_{k+1}}\right)^2, \\ \alpha_{k+1}(x) = \left(1 + 2 \frac{x - x_{k+1}}{x_k - x_{k+1}}\right) \left(\frac{x - x_k}{x_{k+1} - x_k}\right)^2; \end{cases} \quad (2.6.8)$$

$$\begin{cases} \beta_k(x) = (x - x_k) \left(\frac{x - x_{k+1}}{x_k - x_{k+1}}\right)^2, \\ \beta_{k+1}(x) = (x - x_{k+1}) \left(\frac{x - x_k}{x_{k+1} - x_k}\right)^2. \end{cases} \quad (2.6.9)$$

于是满足条件 (2.6.7) 的插值多项式是

$$H_3(x) = y_k \alpha_k(x) + y_{k+1} \alpha_{k+1}(x) + m_k \beta_k(x) + m_{k+1} \beta_{k+1}(x), \quad (2.6.10)$$

其余项 $R_3(x) = f(x) - H_3(x)$, 由式 (2.6.6) 得

续页: 余项公式见下一页。

原书 PDF 页 45; 印刷页码 32。

[查看原页](#)

本页接上页 2.6 Hermite 插值正文。

$$R_3(x) = \frac{1}{4!} f^{(4)}(\xi)(x-x_k)^2(x-x_{k+1})^2.$$

例 2.5 求满足 $P(x_j) = f(x_j)$ ($j = 0, 1, 2$) 及 $P'(x_1) = f'(x_1)$ 的插值多项式及其余项表达式。

解 由给定条件, 可确定次数不超过 3 的插值多项式。由于此多项式通过点 $(x_0, f(x_0))$, $(x_1, f(x_1))$ 及 $(x_2, f(x_2))$, 故其形式为

$$\begin{aligned} P(x) &= f(x_0) + f[x_0, x_1](x-x_0) + f[x_0, x_1, x_2](x-x_0)(x-x_1) \\ &\quad + A(x-x_0)(x-x_1)(x-x_2), \end{aligned}$$

其中 A 为待定常数, 可由条件 $P'(x_1) = f'(x_1)$ 确定, 通过计算可得

$$A = \frac{f'(x_1) - f[x_0, x_1] - (x_1 - x_0)f[x_0, x_1, x_2]}{(x_1 - x_0)(x_1 - x_2)}.$$

为了求出余项 $R(x) = f(x) - P(x)$ 的表达式, 可设

$$R(x) = f(x) - P(x) = K(x)(x-x_0)(x-x_1)^2(x-x_2),$$

其中 $K(x)$ 为待定函数。构造

$$\varphi(t) = f(t) - P(t) - K(x)(t-x_0)(t-x_1)^2(t-x_2).$$

显然 $\varphi(x_j) = 0$ ($j = 0, 1, 2$), 且 $\varphi'(x_1) = 0, \varphi(x) = 0$, 故 $\varphi(t)$ 在 (a, b) 内有五个零点 (重根算两个)。反复应用 Rolle 定理得, $\varphi^{(4)}(t)$ 在 (a, b) 内至少有一个零点 ξ , 故

$$\varphi^{(4)}(\xi) = f^{(4)}(\xi) - 4!K(x) = 0,$$

于是 $K(x) = f^{(4)}(\xi)/4!$, 余项表达式为

$$R(x) = f^{(4)}(\xi)(x-x_0)(x-x_1)^2(x-x_2)/4!, \quad (2.6.11)$$

其中 ξ 位于 x_0, x_1, x_2 和 x 所界定的范围内。

2.7 分段低次插值

2.7.1 多项式插值的问题

根据区间 $[a, b]$ 上给出的节点构造插值多项式 $L_n(x)$ 近似 $f(x)$ 时, 人们往往认为 $L_n(x)$ 的次数 n 越高逼近 $f(x)$ 的精度越好, 但实际上并非如此。这是因为, 对于任意的插值节点, 当 $n \rightarrow \infty$ 时, $L_n(x)$ 不一定收敛到 $f(x)$ 。20 世纪初 Runge 就给出了一个等距节点插值多项式 $L_n(x)$ 不收敛到 $f(x)$ 的例子: 考察函数 $f(x) = 1/(1+x^2)$, 它在 $[-5, 5]$ 上各阶导数均存在, 但在 $[-5, 5]$ 上取 $n+1$ 个等距节点

$$x_k = -5 + 10\frac{k}{n} \quad (k = 0, 1, \dots, n)$$

所构造的 Lagrange 插值多项式

续页: 该多项式和 Runge 现象见下一页。

原书 PDF 页 46; 印刷页码 33。

[查看原页](#)

本页接上页 2.7.1 正文。

$$L_n(x) = \sum_{j=0}^n \frac{1}{1+x_j^2} \frac{\omega_{n+1}(x)}{(x-x_j)\omega'_{n+1}(x_j)}$$

当 $n \rightarrow \infty$ 时只在 $|x| \leq 3.63$ 内收敛, 而在这区间外是发散的。

下面取 $n = 10$, 根据计算画出 $y = L_{10}(x)$ 及 $y = 1/(1+x^2)$ 在 $[-5, 5]$ 上的图形, 如图 2.5 所示。

从图 2.5 可看到, 在 $x = \pm 5$ 附近 $L_{10}(x)$ 与 $f(x) = 1/(1+x^2)$ 偏离很远, 例如, $L_{10}(4.8) = 1.80438, f(4.8) = 0.04160$ 。这说明用高次插值多项式 $L_n(x)$ 近似 $f(x)$ 效果并不好, 因而通常不用高次插值, 而用低次插值。从该例看到, 如果把 $y = 1/(1+x^2)$ 在节点 $x = 0, \pm 1, \pm 2, \pm 3, \pm 4, \pm 5$ 处用折线连起来, 显然比 $L_{10}(x)$ 逼近 $f(x)$ 好得多。这正是下面要讨论的分段低次插值的出发点。

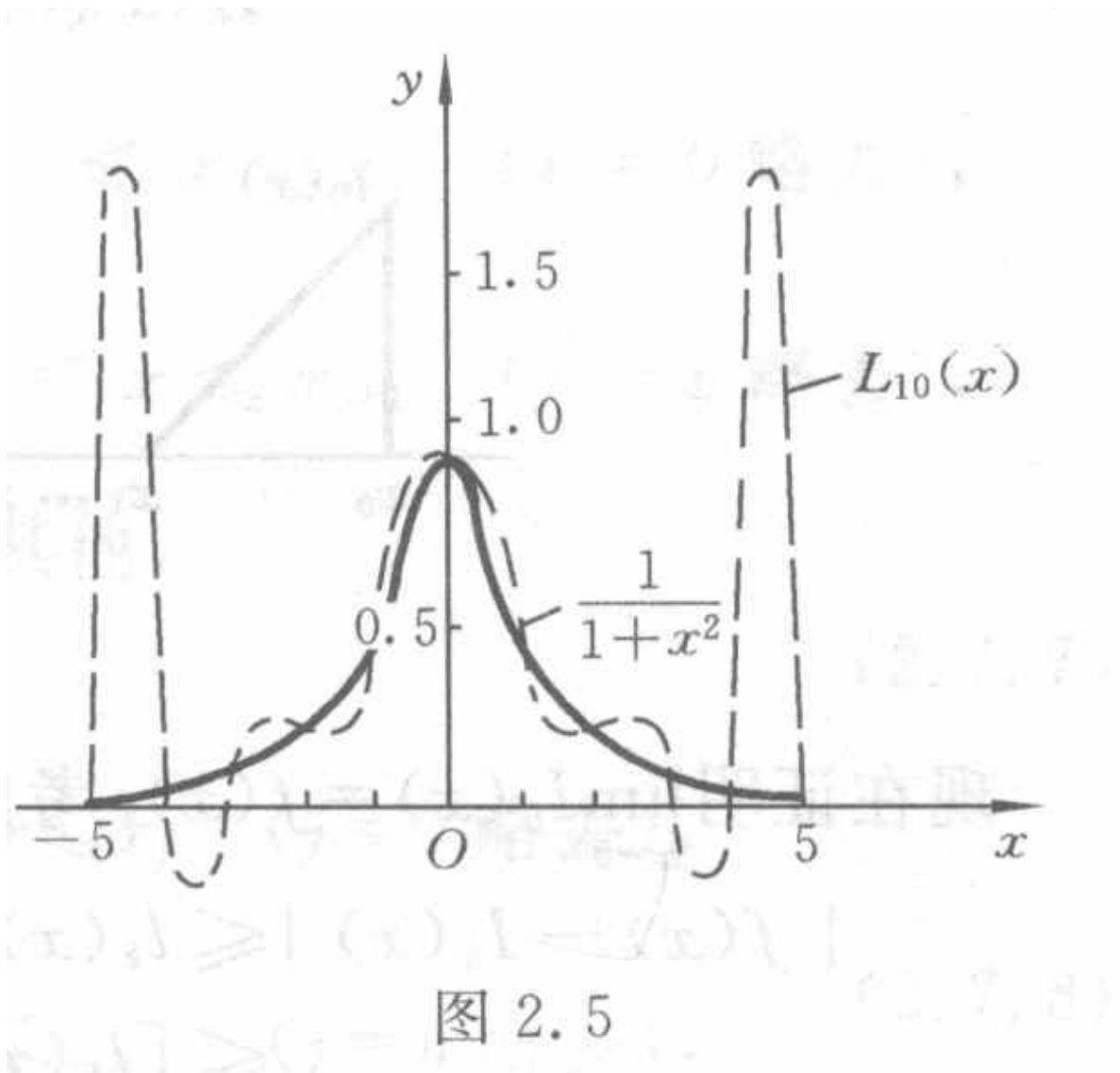


图 2.5

图 2.5 (原图): 横轴 x , 纵轴 y , 原点 O ; 横轴标有 $-5, 5$, 纵轴标有 $0.5, 1.0, 1.5$ 。两条曲线标签为 $L_{10}(x)$ 与 $\frac{1}{1+x^2}$ 。虚线插值曲线在两端附近出现明显振荡。

2.7.2 分段线性插值

所谓分段线性插值,就是将插值点用折线段连接起来逼近 $f(x)$ 。设已知节点 $a = x_0 < x_1 < \cdots < x_n = b$ 上的函数值 $f_0, f_1, \cdots, f_n$, 记 $h_k = x_{k+1} - x_k, h = \max_k h_k$ 。称 $I_h(x)$ 为分段线性插值函数, 如果满足:

- (1) 记 $I_h(x) \in C[a, b]$;
- (2) $I_h(x_k) = f_k$ ($k = 0, 1, \cdots, n$);
- (3) $I_h(x)$ 在每个区间 $[x_k, x_{k+1}]$ 上是线性函数,

则由定义可知, $I_h(x)$ 在每个小区间 $[x_k, x_{k+1}]$ 上可表示为

$$I_h(x) = \frac{x - x_{k+1}}{x_k - x_{k+1}} f_k + \frac{x - x_k}{x_{k+1} - x_k} f_{k+1} \quad (x_k \leq x \leq x_{k+1}). \quad (2.7.1)$$

若用插值基函数表示, 则 $I_h(x)$ 在整个区间 $[a, b]$ 上可表示为

$$I_h(x) = \sum_{j=0}^n f_j l_j(x), \quad (2.7.2)$$

其中基函数 $l_j(x)$ 满足条件 $l_j(x_k) = \delta_{jk}$ ($j, k = 0, 1, \cdots, n$), 其形式是

$$l_j(x) = \begin{cases} \frac{x - x_{j-1}}{x_j - x_{j-1}}, & x_{j-1} \leq x \leq x_j \quad (j=0 \text{ 略去}), \\ \frac{x_j - x_{j+1}}{x - x_{j+1}}, & x_j \leq x \leq x_{j+1} \quad (j=n \text{ 略去}), \\ 0, & x \in [a, b], \quad x \notin [x_{j-1}, x_{j+1}]. \end{cases} \quad (2.7.3)$$

分段线性插值基函数 $l_j(x)$ 只在 x_j 附近不为零, 在其他地方均为零, 这种性质称为**局部非零性质**, 如图 2.6 所示。当 $x \in [x_k, x_{k+1}]$ 时,

续页: 基函数恒等式和图 2.6 见下一页。

校注 (agent 补充; 原书疑误): 式 (2.7.3) 第二分支原扫描确为 $\frac{x_j - x_{j+1}}{x - x_{j+1}}$, 此处已原样保留, 参见[原式放大截图](#)。该式在 $x = x_{j+1}$ 处分母为零, 与本页分段线性、连续性及 $l_j(x_{j+1}) = 0$ 的条件矛盾。按这些条件, 第二分支应为 $\frac{x - x_{j+1}}{x_j - x_{j+1}}$ 。这属于原书疑误, 区别于 OCR 另将分母误识为 $x_j - x_{j+1}$ 。

原书 PDF 页 47；印刷页码 34。

[查看原页](#)

本页接上页 2.7.2 正文。

$$1 = \sum_{j=0}^n l_j(x) = l_k(x) + l_{k+1}(x), \quad f(x) = [l_k(x) + l_{k+1}(x)]f(x).$$

另一方面，这时

$$I_h(x) = f_k l_k(x) + f_{k+1} l_{k+1}(x).$$

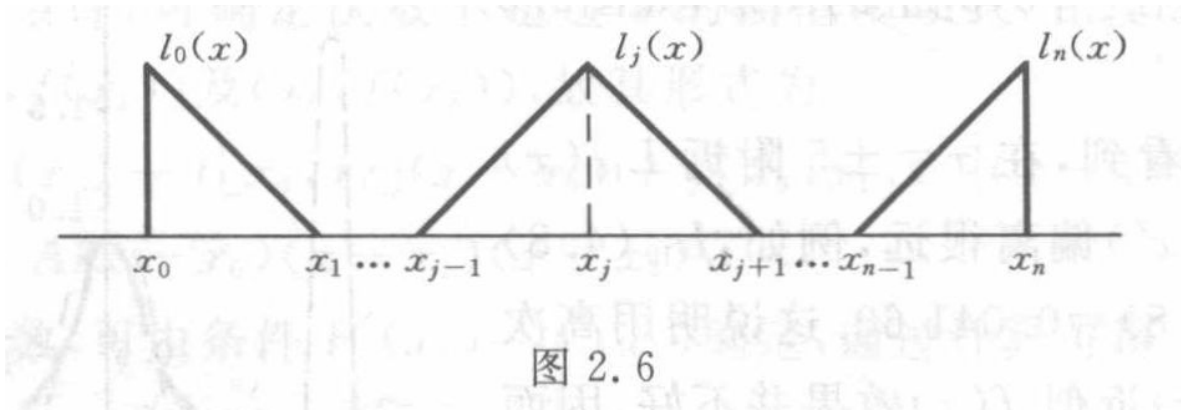


图 2.6

图 2.6（原图）：分段线性插值基函数的局部非零形状。左端 $l_0(x)$ 在 x_0 达到峰值并线性降到 x_1 ；内部 $l_j(x)$ 在 x_{j-1}, x_{j+1} 为零，在 x_j 达到峰值；右端 $l_n(x)$ 从 x_{n-1} 线性升到 x_n 。全部原图标签为 $l_0(x), l_j(x), l_n(x), x_0, x_1, \dots, x_{j-1}, x_j, x_{j+1}, \dots, x_{n-1}, x_n$ ；峰值未标数字。

现在证明 $\lim_{h \rightarrow 0} I_h(x) = f(x)$ 。考虑

$$\begin{aligned} |f(x) - I_h(x)| &\leq l_k(x)|f(x) - f_k| + l_{k+1}(x)|f(x) - f_{k+1}| \\ &\leq [l_k(x) + l_{k+1}(x)]\omega(h_k) \\ &= \omega(h_k) \leq \omega(h), \end{aligned} \quad (2.7.4)$$

这里 $\omega(h)$ 是函数 $f(x)$ 在区间 $[a, b]$ 上的连续模，即对于任意两点 $x', x'' \in [a, b]$ ，只要 $|x' - x''| \leq h$ ，就有

$$|f(x') - f(x'')| \leq \omega(h),$$

称 $\omega(h)$ 为 $f(x)$ 在 $[a, b]$ 上的连续模，当 $f(x) \in C[a, b]$ 时，就有 $\lim_{h \rightarrow 0} \omega(h) = 0$ 。

由前式可知，当 $x \in [a, b]$ 时有 $\max_{a \leq x \leq b} |f(x) - I_h(x)| \leq \omega(h)$ ，因此，只要 $f(x) \in C[a, b]$ ，就有 $\lim_{h \rightarrow 0} I_h(x) = f(x)$ 在 $[a, b]$ 上一致成立，故 $I_h(x)$ 在 $[a, b]$ 上一致收敛到 $f(x)$ 。

2.7.3 分段三次 Hermite 插值

分段线性插值函数 $I_h(x)$ 的导数是间断的，若在节点 x_k ($k = 0, 1, \dots, n$) 上除已知函数值 f_k 外还给出导数值 $f'_k = m_k$ ($k = 0, 1, \dots, n$)，就可构造一个导数连续的分段插值函数 $I_h(x)$ ，它满足如下条件：

- (1) $I_h(x) \in C^1[a, b]$ ($C^1[a, b]$ 代表区间 $[a, b]$ 上一阶导数连续的函数集合);
- (2) $I_h(x_k) = f_k, I'_h(x_k) = f'_k$ ($k = 0, 1, \dots, n$);
- (3) $I_h(x)$ 在每个小区间 $[x_k, x_{k+1}]$ 上是三次多项式。

根据两点三次插值多项式 (2.6.10) 可知, $I_h(x)$ 在区间 $[x_k, x_{k+1}]$ 上的表达式为

$$\begin{aligned}
 I_h(x) = & \left(\frac{x - x_{k+1}}{x_k - x_{k+1}} \right)^2 \left(1 + 2 \frac{x - x_k}{x_{k+1} - x_k} \right) f_k + \left(\frac{x - x_k}{x_{k+1} - x_k} \right)^2 \left(1 + 2 \frac{x - x_{k+1}}{x_k - x_{k+1}} \right) f_{k+1} \\
 & + \left(\frac{x - x_{k+1}}{x_k - x_{k+1}} \right)^2 (x - x_k) f'_k + \left(\frac{x - x_k}{x_{k+1} - x_k} \right)^2 (x - x_{k+1}) f'_{k+1}.
 \end{aligned} \tag{2.7.5}$$

若在整个区间 $[a, b]$ 上定义一组分段三次插值基函数 $\alpha_j(x)$ 及 $\beta_j(x)$ ($j = 0,$

续页: 上句的下标范围及公式在下一页续完。

原书 PDF 页 48; 印刷页码 35。

[查看原页](#)

本页接上页 2.7.3 正文; 开头续完下标范围。

$1, \dots, n$), 则 $I_h(x)$ 可表示为

$$I_h(x) = \sum_{j=0}^n [f_j \alpha_j(x) + f'_j \beta_j(x)], \quad (2.7.6)$$

其中 $\alpha_j(x), \beta_j(x)$ 依式 (2.6.8)、式 (2.6.9) 分别表示为

$$\alpha_j(x) = \begin{cases} \left(\frac{x - x_{j-1}}{x_j - x_{j-1}} \right)^2 \left(1 + 2 \frac{x - x_j}{x_{j-1} - x_j} \right), & x_{j-1} \leq x \leq x_j \quad (j=0 \text{ 略去}), \\ \left(\frac{x - x_{j+1}}{x_j - x_{j+1}} \right)^2 \left(1 + 2 \frac{x - x_j}{x_{j+1} - x_j} \right), & x_j \leq x \leq x_{j+1} \quad (j=n \text{ 略去}), \\ 0, & \text{其他,} \end{cases} \quad (2.7.7)$$

$$\beta_j(x) = \begin{cases} \left(\frac{x - x_{j-1}}{x_j - x_{j-1}} \right)^2 (x - x_j), & x_{j-1} \leq x \leq x_j \quad (j=0 \text{ 略去}), \\ \left(\frac{x - x_{j+1}}{x_j - x_{j+1}} \right)^2 (x - x_j), & x_j \leq x \leq x_{j+1} \quad (j=n \text{ 略去}), \\ 0, & \text{其他.} \end{cases} \quad (2.7.8)$$

由于 $\alpha_j(x), \beta_j(x)$ 的局部非零性质, 当 $x \in [x_k, x_{k+1}]$ 时, 只有 $\alpha_k(x), \alpha_{k+1}(x), \beta_k(x), \beta_{k+1}(x)$ 不为零, 于是式 (2.7.6) 的 $I_h(x)$ 可表示为

$$I_h(x) = f_k \alpha_k(x) + f_{k+1} \alpha_{k+1}(x) + f'_k \beta_k(x) + f'_{k+1} \beta_{k+1}(x) \quad (x_k \leq x \leq x_{k+1}). \quad (2.7.9)$$

为了研究 $I_h(x)$ 的收敛性, 由式 (2.7.7) 及式 (2.7.8) 直接得估计式

$$0 \leq \alpha_j(x) \leq 1, \quad (2.7.10)$$

$$\begin{cases} |\beta_k(x)| \leq \frac{4}{27} h_k, \\ |\beta_{k+1}(x)| \leq \frac{4}{27} h_k. \end{cases} \quad (2.7.11)$$

此外, 当 $f(x)$ 是分段三次多项式时, $f(x)$ 的插值多项式 $I_h(x)$ 就是它本身。例如, 当 $f(x) = 1$ 时就有 $\sum_{j=0}^n \alpha_j(x) = 1$ 。当 $x \in [x_k, x_{k+1}]$ 时, 得

$$\alpha_k(x) + \alpha_{k+1}(x) = 1. \quad (2.7.12)$$

由式 (2.7.9) ~ (2.7.12), 当 $x \in [x_k, x_{k+1}]$ 时还可得

$$\begin{aligned}
|f(x) - I_h(x)| &\leq \alpha_k(x)|f(x) - f_k| + \alpha_{k+1}(x)|f(x) - f_{k+1}| \\
&\quad + \frac{4}{27}h_k[|f'_k| + |f'_{k+1}|] \\
&\leq [\alpha_k(x) + \alpha_{k+1}(x)]\omega(h) + \frac{8}{27}h \max\{|f'_k|, |f'_{k+1}|\}, \\
|f(x) - I_h(x)| &\leq \omega(h) + \frac{8}{27}h \max_{0 \leq k \leq n} |f'_k|
\end{aligned} \tag{2.7.13}$$

对于 $x \in [a, b]$ 成立, 其中 $\omega(h)$ 是 $f(x)$ 在 $[a, b]$ 上的连续模。因此, 当 $f(x) \in C[a, b]$

续页: 收敛性论述在下一页续完。

原书 PDF 页 49; 印刷页码 36。

[查看原页](#)

本页接上页 2.7.3 正文。

时,

$$\lim_{h \rightarrow 0} I_h(x) = f(x).$$

这就证明了 $I_h(x)$ 在 $[a, b]$ 上一致收敛到 $f(x)$ 。

2.8 三次样条插值

上面讨论的分段低次插值函数都有一致收敛性, 但光滑性较差, 对于像高速飞机的机翼、船体放样等的型值线, 往往要求有二阶光滑度, 即有二阶连续导数。早期工程师制图时, 把富有弹性的细长木条 (所谓样条) 用压铁固定在样点上, 在其他地方让它自由弯曲, 然后画下长条的曲线, 称为样条曲线。它实际上是由分段三次曲线并接而成, 在连接点即样点上要求二阶导数连续, 从数学上加以概括就得到数学样条这一概念。下面讨论最常用的三次样条函数。

2.8.1 三次样条函数

定义 2.5 若函数 $S(x) \in C^2[a, b]$, 且在每个小区间 $[x_j, x_{j+1}]$ 上是三次多项式, 其中 $a = x_0 < x_1 < \cdots < x_n = b$ 是给定节点, 则称 $S(x)$ 是节点 $x_0, x_1, \dots, x_n$ 上的**三次样条函数**。若在节点 x_j 上给定函数值 $y_j = f(x_j)$ ($j = 0, 1, \dots, n$), 且

$$S(x_j) = y_j \quad (j = 0, 1, \dots, n) \quad (2.8.1)$$

成立, 则称 $S(x)$ 为**三次样条插值函数**。

从定义知, 要求出 $S(x)$, 在每个小区间 $[x_j, x_{j+1}]$ 上要确定 4 个待定系数, 共有 n 个小区间, 故应确定 $4n$ 个参数。根据 $S(x)$ 在 $[a, b]$ 上二阶导数连续, 在节点 x_j ($j = 1, 2, \dots, n-1$) 处应满足连续性条件

$$\begin{aligned} S(x_j - 0) &= S(x_j + 0), & S'(x_j - 0) &= S'(x_j + 0), \\ S''(x_j - 0) &= S''(x_j + 0), \end{aligned} \quad (2.8.2)$$

共有 $3n - 3$ 个条件, 再加上 $S(x)$ 满足插值条件 (2.8.1), 共有 $4n - 2$ 个条件, 因此还需要 2 个条件才能确定 $S(x)$ 。通常可在区间 $[a, b]$ 端点 $a = x_0, b = x_n$ 上各加一个条件 (称为**边界条件**), 边界条件可根据实际问题的要求给定。常见的有以下三种:

(1) 已知两端的一阶导数值, 即

$$\begin{cases} S'(x_0) = f'_0, \\ S'(x_n) = f'_n. \end{cases} \quad (2.8.3)$$

(2) 两端的二阶导数已知, 即

$$\begin{cases} S''(x_0) = f_0'', \\ S''(x_n) = f_n''. \end{cases} \quad (2.8.4)$$

特殊情况下的边界条件

续页：自然边界条件及第三类边界条件见下一页。

校注 (agent 补充; 原书论证条件疑漏): 本页开头承接上一页 “当 $f(x) \in C[a, b]$ 时” 即宣称分段三次 Hermite 插值一致收敛, 原文已保留。由 (2.7.13) 实际还需控制 $h \max_k |f'_k| \rightarrow 0$; 仅有连续性既不保证节点导数存在, 也不能控制随加密节点变化的导数值。比如加上 $f \in C^1[a, b]$ 就足以使节点导数一致有界, 从该估计推出结论。本注是对原文论证条件的补充, 不属于教材正文。

原书 PDF 页 50; 印刷页码 37。

[查看原页](#)

本页接上页 2.8.1 正文。

$$S''(x_0) = S''(x_n) = 0 \quad (2.8.4')$$

称为**自然边界条件**。

(3) 当 $f(x)$ 是以 $x_n - x_0$ 为周期的周期函数时, 则要求 $S(x)$ 也是周期函数, 这时边界条件应满足

$$\begin{cases} S(x_0 + 0) = S(x_n - 0), \\ S'(x_0 + 0) = S'(x_n - 0), \\ S''(x_0 + 0) = S''(x_n - 0), \end{cases} \quad (2.8.5)$$

而此時式 (2.8.1) 中 $y_0 = y_n$ 。这样确定的样条函数 $S(x)$ 称为**周期样条函数**。

2.8.2 三转角方程

现在构造满足条件(2.8.1)及加上相应边界条件的三次样条函数 $S(x)$ 的表达式。若假定 $S'(x)$ 在节点 x_j 处的值为 $S'(x_j) = m_j$ ($j = 0, 1, \dots, n$), 再由式 (2.8.1), 则由分段三次 Hermite 插值式 (2.7.6) 可得

$$S(x) = \sum_{j=0}^n [y_j \alpha_j(x) + m_j \beta_j(x)], \quad (2.8.6)$$

其中 $\alpha_j(x), \beta_j(x)$ 是插值基函数, 由式 (2.7.7)、式 (2.7.8) 表示。显然, 表达式 (2.8.6) 中 $S(x)$ 及 $S'(x)$ 在整个区间 $[a, b]$ 上连续, 且满足式 (2.8.1); 然而在式 (2.8.6) 中 m_j ($j = 0, 1, \dots, n$) 是未知的, 它可利用式 (2.8.2)

$$S''(x_j - 0) = S''(x_j + 0) \quad (j = 1, \dots, n-1)$$

及边界条件 (2.8.3) (也可以是其他边界条件) 来确定。为了求出 m_j , 下面考虑 $S(x)$ 在 $[x_j, x_{j+1}]$ 上的表达式

$$\begin{aligned} S(x) = & \frac{(x - x_{j+1})^2 [h_j + 2(x - x_j)]}{h_j^3} y_j + \frac{(x - x_j)^2 [h_j + 2(x_{j+1} - x)]}{h_j^3} y_{j+1} \\ & + \frac{(x - x_{j+1})^2 (x - x_j)}{h_j^2} m_j + \frac{(x - x_j)^2 (x - x_{j+1})}{h_j^2} m_{j+1}, \end{aligned} \quad (2.8.7)$$

这里 $h_j = x_{j+1} - x_j$ 。对 $S(x)$ 求二阶导数, 得

$$\begin{aligned} S''(x) = & \frac{6x - 2x_j - 4x_{j+1}}{h_j^2} m_j + \frac{6x - 4x_j - 2x_{j+1}}{h_j^2} m_{j+1} \\ & + \frac{6(x_j + x_{j+1} - 2x)}{h_j^3} (y_{j+1} - y_j), \end{aligned}$$

于是

$$S''(x_j + 0) = -\frac{4}{h_j}m_j - \frac{2}{h_j}m_{j+1} + \frac{6}{h_j^2}(y_{j+1} - y_j).$$

同理, 可得 $S''(x)$ 在区间 $[x_{j-1}, x_j]$ 上的表达式

$$\begin{aligned} S''(x) = & \frac{6x - 2x_{j-1} - 4x_j}{h_{j-1}^2}m_{j-1} + \frac{6x - 4x_{j-1} - 2x_j}{h_{j-1}^2}m_j \\ & + \frac{6(x_{j-1} + x_j - 2x)}{h_{j-1}^2}(y_j - y_{j-1}) \end{aligned}$$

续页: 左侧二阶导数在下一页继续。

校注 (agent 补充; 原书疑误): 本页末式第三项的分母在原扫描中为 h_{j-1}^2 , 上面原样保留, 参见[原式放大截图](#)。由同页前一个区间的 $S''(x)$ 公式换下标, 第三项分母应为 h_{j-1}^3 ; 这样代入 $x = x_j$ 才得到下一页的 $-6(y_j - y_{j-1})/h_{j-1}^2$ 。这是原书幂次疑误; OCR 还漏掉了多个 $j-1$ 下标, 已按图补回。

原书 PDF 页 51; 印刷页码 38。

[查看原页](#)

本页接上页 2.8.2 正文。

及

$$S''(x_j - 0) = \frac{2}{h_{j-1}}m_{j-1} + \frac{4}{h_{j-1}}m_j - \frac{6}{h_{j-1}^2}(y_j - y_{j-1}).$$

由条件 $S''(x_j + 0) = S''(x_j - 0)$ ($j = 1, 2, \dots, n-1$), 可得

$$\begin{aligned} & \frac{1}{h_{j-1}}m_{j-1} + 2\left(\frac{1}{h_{j-1}} + \frac{1}{h_j}\right)m_j + \frac{1}{h_j}m_{j+1} \\ &= 3\left(\frac{y_{j+1} - y_j}{h_j^2} + \frac{y_j - y_{j-1}}{h_{j-1}^2}\right) \quad (j = 1, 2, \dots, n-1), \end{aligned} \quad (2.8.8)$$

用 $\frac{1}{h_{j-1}} + \frac{1}{h_j}$ 除全式, 并注意 $y_j = f_j$, $\frac{y_{j+1} - y_j}{h_j} = f[x_j, x_{j+1}]$, 方程 (2.8.8) 可简化为

$$\lambda_j m_{j-1} + 2m_j + \mu_j m_{j+1} = g_j \quad (j = 1, 2, \dots, n-1), \quad (2.8.9)$$

其中

$$\lambda_j = \frac{h_j}{h_{j-1} + h_j}, \quad \mu_j = \frac{h_{j-1}}{h_{j-1} + h_j} \quad (j = 1, 2, \dots, n-1), \quad (2.8.10)$$

$$g_j = 3(\lambda_j f[x_{j-1}, x_j] + \mu_j f[x_j, x_{j+1}]) \quad (j = 1, 2, \dots, n-1). \quad (2.8.11)$$

方程 (2.8.9) 是关于未知数 $m_0, m_1, \dots, m_n$ 的 $n-1$ 个方程, 若加上已知条件 (2.8.3): $m_0 = f'_0$ 则 $m_n = f'_n$, 那么方程 (2.8.9) 为只含 $m_1, \dots, m_{n-1}$ 的 $n-1$ 个方程, 写成矩阵形式便是

$$\begin{pmatrix} 2 & \mu_1 & 0 & \cdots & \cdots & 0 \\ \lambda_2 & 2 & \mu_2 & \ddots & & \vdots \\ 0 & \lambda_3 & 2 & \mu_3 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & 0 \\ 0 & & \ddots & \lambda_{n-2} & 2 & \mu_{n-2} \\ 0 & \cdots & \cdots & 0 & \lambda_{n-1} & 2 \end{pmatrix} \begin{pmatrix} m_1 \\ m_2 \\ m_3 \\ \vdots \\ m_{n-2} \\ m_{n-1} \end{pmatrix} = \begin{pmatrix} g_1 - \lambda_1 f'_0 \\ g_2 \\ g_3 \\ \vdots \\ g_{n-2} \\ g_{n-1} - \mu_{n-1} f'_n \end{pmatrix}. \quad (2.8.12)$$

如果边界条件为式 (2.8.4), 则

$$\begin{cases} 2m_0 + m_1 = 3f[x_0, x_1] - \frac{h_0}{2}f''_0 = g_0, \\ m_{n-1} + 2m_n = 3f[x_{n-1}, x_n] + \frac{h_{n-1}}{2}f''_n = g_n. \end{cases} \quad (2.8.13)$$

如果边界条件为式 (2.8.4)' , 则

$$\begin{cases} 2m_0 + m_1 = 3f[x_0, x_1] = g_0, \\ m_{n-1} + 2m_n = 3f[x_{n-1}, x_n] = g_n. \end{cases} \quad (2.8.13')$$

于是, 式 (2.8.9) 与式 (2.8.13) 或式 (2.8.13)' 合并后用矩阵形式表示为

$$\begin{pmatrix} 2 & 1 & 0 & \cdots & \cdots & 0 \\ \lambda_1 & 2 & \mu_1 & \ddots & & \vdots \\ 0 & \lambda_2 & 2 & \mu_2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & 0 \\ \vdots & & \ddots & \lambda_{n-1} & 2 & \mu_{n-1} \\ 0 & \cdots & \cdots & 0 & 1 & 2 \end{pmatrix} \begin{pmatrix} m_0 \\ m_1 \\ m_2 \\ \vdots \\ m_{n-1} \\ m_n \end{pmatrix} = \begin{pmatrix} g_0 \\ g_1 \\ g_2 \\ \vdots \\ g_{n-1} \\ g_n \end{pmatrix}. \quad (2.8.14)$$

原书 PDF 页 52; 印刷页码 39。

[查看原页](#)

本页接上页 2.8.2 正文。

如果边界条件为周期性条件式 (2.8.5), 则

$$m_0 = m_n,$$

$$\frac{1}{h_0}m_1 + \frac{1}{h_{n-1}}m_{n-1} + 2\left(\frac{1}{h_0} + \frac{1}{h_{n-1}}\right)m_n = \frac{3}{h_0}f[x_0, x_1] + \frac{3}{h_{n-1}}f[x_{n-1}, x_n],$$

化简为

$$\mu_n m_1 + \lambda_n m_{n-1} + 2m_n = g_n,$$

$$\mu_n = \frac{h_{n-1}}{h_0 + h_{n-1}}, \quad \lambda_n = \frac{h_{n-1}}{h_0 + h_{n-1}},$$

$$g_n = 3(\mu_n f[x_0, x_1] + \lambda_n f[x_{n-1}, x_n]).$$

与式 (2.8.9) 合并, 用矩阵形式表示为

$$\begin{pmatrix} 2 & \mu_1 & 0 & \cdots & 0 \\ \lambda_2 & 2 & \mu_2 & \ddots & \vdots \\ 0 & \lambda_3 & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & 2 & \mu_{n-1} \\ 0 & \cdots & 0 & \lambda_n & 2 \end{pmatrix} \begin{pmatrix} m_1 \\ m_2 \\ \vdots \\ m_{n-1} \\ m_n \end{pmatrix} = \begin{pmatrix} g_1 \\ g_2 \\ \vdots \\ g_{n-1} \\ g_n \end{pmatrix}. \quad (2.8.15)$$

这里得到的方程组 (2.8.12)、(2.8.14) 及式 (2.8.15) 中, 每个方程都联系三个 m_j , m_j 在力学上解释为细梁在 x_j 截面处的转角, 故称之为**三转角方程**。这些方程系数矩阵对角元素均为 2, 非对角元素 $\mu_j + \lambda_j = 1$, 故系数矩阵具有强对角优势, 方程组 (2.8.12)、(2.8.14) 及 (2.8.15) 都有唯一解, 可用追赶法求得解 m_j ($j = 0, 1, \dots, n$), 从而得到 $S(x)$ 。关于线性方程组的解法见本书第 7 章。

2.8.3 三弯矩方程

三次样条插值函数 $S(x)$ 可以有多种表达方法, 有时用二阶导数值 $S''(x_j) = M_j$ ($j = 0, 1, \dots, n$) 表示使用起来更方便。 M_j 在力学上解释为细梁在 x_j 截面处的弯矩, 并且得到的弯矩与相邻两个弯矩有关, 故称之为**三弯矩方程**。

由于 $S(x)$ 在区间 $[x_j, x_{j+1}]$ 上是三次多项式, 故 $S''(x)$ 在 $[x_j, x_{j+1}]$ 上是线性函数, 可表示为

$$S''(x) = M_j \frac{x_{j+1} - x}{h_j} + M_{j+1} \frac{x - x_j}{h_j}. \quad (2.8.16)$$

对 $S''(x)$ 积分两次并利用 $S(x_j) = y_j$ 及 $S(x_{j+1}) = y_{j+1}$, 可定出积分常数, 于是

$$\begin{aligned}
S(x) = & M_j \frac{(x_{j+1} - x)^3}{6h_j} + M_{j+1} \frac{(x - x_j)^3}{6h_j} + \left(y_j - \frac{M_j h_j^2}{6} \right) \frac{x_{j+1} - x}{h_j} \\
& + \left(y_{j+1} - \frac{M_{j+1} h_j^2}{6} \right) \frac{x - x_j}{h_j} \quad (j = 0, 1, \dots, n-1).
\end{aligned} \tag{2.8.17}$$

对 $S(x)$ 求导, 得

$$S'(x) = -M_j \frac{(x_{j+1} - x)^2}{2h_j} + M_{j+1} \frac{(x - x_j)^2}{2h_j} + \frac{y_{j+1} - y_j}{h_j} - \frac{M_{j+1} - M_j}{6} h_j. \tag{2.8.18}$$

校注 (agent 补充; 原书疑误): 本页 λ_n 原式与 μ_n 同为 $h_{n-1}/(h_0 + h_{n-1})$, 已保留。由上一行未归一化方程, m_{n-1} 的系数应为 $\lambda_n = h_0/(h_0 + h_{n-1})$ 。此外, (2.8.15) 的右上角、左下角在原书均印为 0; 由 $m_0 = m_n$ 及本页的周期方程, 应分别为 λ_1 、 μ_n 。对应系统是有两个角元素的循环三对角系统, 不能不作处理便直接套用普通三对角追赶法。参见[原文](#)和[矩阵放大截图](#)。

原书 PDF 页 53; 印刷页码 40。

[查看原页](#)

本页接上页 2.8.3 正文。

由此可求得

$$S'(x_j + 0) = -\frac{h_j}{3}M_j - \frac{h_j}{6}M_{j+1} + \frac{y_{j+1} - y_j}{h_j}.$$

类似地, 可求出 $S(x)$ 在区间 $[x_{j-1}, x_j]$ 上的表达式, 从而得

$$S'(x_j - 0) = \frac{h_{j-1}}{6}M_{j-1} + \frac{h_{j-1}}{3}M_j + \frac{y_j - y_{j-1}}{h_{j-1}},$$

利用 $S'(x_j + 0) = S'(x_j - 0)$ 可得

$$\mu_j M_{j-1} + 2M_j + \lambda_j M_{j+1} = d_j \quad (j = 1, 2, \dots, n-1), \quad (2.8.19)$$

其中 μ_j, λ_j 由式 (2.8.10) 求得, 而

$$d_j = 6 \frac{f[x_j, x_{j+1}] - f[x_{j-1}, x_j]}{h_{j-1} + h_j} = 6f[x_{j-1}, x_j, x_{j+1}] \quad (j = 1, 2, \dots, n-1). \quad (2.8.20)$$

方程 (2.8.19) 与方程 (2.8.9) 完全类似, 只要加上式 (2.8.3) ~ (2.8.5) 的任一种边界条件就可得到三弯矩 M_j 的方程组。例如, 若边界条件为式 (2.8.3), 则端点方程为

$$2M_0 + M_1 = \frac{6}{h_0}(f[x_0, x_1] - f'_0), \quad M_{n-1} + 2M_n = \frac{6}{h_{n-1}}(f'_n - f[x_{n-1}, x_n]).$$

若边界条件为式 (2.8.4), 则端点方程为

$$M_0 = f''_0, \quad M_n = f''_n.$$

同样, 通过追赶法, 可求出三弯矩方程的解 M_j ($j = 0, 1, \dots, n$), 代入式 (2.8.17) 则得到三次插值样条函数 $S(x)$ 。

2.8.4 计算步骤与例题

样条函数, 特别是三次样条在实际中有广泛的应用, 在计算机上也容易实现。下面以方程 (2.8.12) 为例, 说明在计算机上求 $S(x)$ 的算法步骤:

- 步 1 输入初始数据 x_j, y_j ($j = 0, 1, \dots, n$) 及 f'_0, f'_n 和 n 。
- 步 2 j 从 0 到 $n-1$ 计算 $h_j = x_{j+1} - x_j$ 及 $f[x_j, x_{j+1}]$ 。
- 步 3 j 从 1 到 $n-1$ 由式 (2.8.10) 及式 (2.8.11) 计算 λ_j, μ_j, g_j 。
- 步 4 用追赶法 (公式见 7.4.3 节) 解方程 (2.8.12), 求出 m_j ($j = 1, 2, \dots, n-1$)。

步 5 计算 $S(x)$ 的系数或计算 $S(x)$ 在若干点上的值，并打印结果。

步 6 给定函数 $f(x) = \frac{1}{1+x^2}$, $-5 \leq x \leq 5$, 节点 $x_k = -5 + k$ ($k = 0, 1, \dots, 10$), 用三次样条插值求 $S_{10}(x)$ 。取 $S_{10}(x_k) = f(x_k)$ ($k = 0, 1, \dots, 10$), 有

$$S'_{10}(-5) = f'(-5), \quad S'_{10}(5) = f'(5).$$

利用上述步骤编制的程序计算 $S_{10}(x)$ 在表 2.8 所列各点的值，并与 $f(x)$ 及 2.7 节计算出的 $L_{10}(x)$ 比较，结果也如表 2.8 所示。由表 2.8 可看出，用三次样条插值得到的 $S_{10}(x)$ 能很好地逼近 $f(x)$ ，不会出现 Lagrange 插值 $L_{10}(x)$ 的“Runge”现象。

原书 PDF 页 54; 印刷页码 41。

[查看原页](#)

本页接上页 2.8.4 正文。

$f(x)$ 与 $L_{10}(x)$ 的图形如图 2.5 所示, 如画出 $S_{10}(x)$ 的曲线, 将与 $f(x)$ 很近似。

表 2.8

x	$\frac{1}{1+x^2}$	$S_{10}(x)$	$L_{10}(x)$	x	$\frac{1}{1+x^2}$	$S_{10}(x)$	$L_{10}(x)$
-5.0	0.038 46	0.038 46	0.038 46	-2.3	0.158 98	0.161 15	0.241 45
-4.8	0.041 60	0.037 58	1.804 38	-2.0	0.200 00	0.200 00	0.200 00
-4.5	0.047 06	0.042 48	1.578 72	-1.8	0.235 85	0.231 54	0.188 78
-4.3	0.051 31	0.048 42	0.888 08	-1.5	0.307 69	0.297 44	0.235 35
-4.0	0.058 82	0.058 82	0.058 82	-1.3	0.371 75	0.361 33	0.316 50
-3.8	0.064 77	0.065 56	-0.201 30	-1.0	0.500 00	0.500 00	0.500 00
-3.5	0.075 47	0.076 06	-0.226 20	-0.8	0.609 76	0.624 20	0.643 16
-3.3	0.084 10	0.084 26	-0.108 32	-0.5	0.800 00	0.820 51	0.843 40
-3.0	0.100 00	0.100 00	0.100 00	-0.3	0.917 43	0.927 54	0.940 90
-2.8	0.113 12	0.113 66	0.198 37	0	1.000 00	1.000 00	1.000 00
-2.5	0.137 93	0.139 71	0.253 76				

表格数值及原有小数分组空格均按扫描转录; [表 2.8 原图裁切](#)。

2.8.5 三次样条插值的收敛性

为了证明三次样条插值的收敛性, 需要用到向量与矩阵范数及有关结论, 可参看 7.5 节。

设 $A = (a_{ij})_n$ 为 $n \times n$ 矩阵, $x = (x_1, x_2, \dots, x_n)^T$ 为 n 维向量, 定义 x 及 A 的范数为

$$\begin{aligned} \|x\|_{\infty} &= \max_{1 \leq i \leq n} |x_i|, \\ \|A\|_{\infty} &= \sup_{x \neq 0} \frac{\|Ax\|_{\infty}}{\|x\|_{\infty}}. \end{aligned} \quad (2.8.21)$$

同样, 对于函数 $f(x) \in C[a, b]$, 也定义 f 的范数为

$$\|f\|_{\infty} = \sup_{a \leq x \leq b} |f(x)| = \max_{a \leq x \leq b} |f(x)|.$$

引理 若 $A = (a_{ij})_n$ 具有强对角占优, 即

$$\sum_{j \neq i} |a_{ij}| < |a_{ii}| \quad (i = 1, 2, \dots, n), \quad (2.8.22)$$

则 A^{-1} 存在, 且

$$\|A^{-1}\|_{\infty} \leq \left\{ \min \left(|a_{ij}| - \sum_{j \neq i} |a_{ij}| \right) \right\}^{-1}. \quad (2.8.23)$$

该引理的证明可参考定理 8.6 和第 8 章习题 20。

现在以自然边界条件 (2.8.4) ' 的三次样条插值函数 $S(x)$ 为例, 讨论其收敛性,

续页: 收敛性定理见下一页。

校注 (agent 补充; 原书疑误): 式 (2.8.21) 原图在 $\max$ 下用 i , 而被取最大值的分量印为 x_j ; 已原样保留, 标准表达需将哑指标统一。式 (2.8.23) 括号内第一项原图印为 $|a_{ij}|$; 结合 (2.8.22) 的对角占优量, 应为 $|a_{ii}|$, 并对行指标 i 取最小值。参见[范数公式原图裁切](#)。

校注 (agent 补充; 原书表值与条件不一致): 表 2.8 的 S_{10} 数值在此按原图保留。按上一页明定的节点、 $f(x) = 1/(1+x^2)$ 及 $S'_{10}(\pm 5) = f'(\pm 5)$, 精确有理数解三转角方程给出 $S_{10}(-4.8) \approx 0.04162183$ 、 $S_{10}(-4.5) \approx 0.04716801$, 与原表的 0.03758、0.04248 不同; 这不是 OCR 差异。复核结果见[独立数值检查](#)。本校注不以复算值替换原表。

原书 PDF 页 55; 印刷页码 42。

[查看原页](#)

本页接上页 2.8.5 正文。

此时方程为式 (2.8.14), 写成

$$Am = g, \quad (2.8.24)$$

其中

$$A = \begin{pmatrix} 2 & 1 & 0 & \cdots & 0 \\ \lambda_1 & 2 & \mu_1 & \ddots & \vdots \\ 0 & \lambda_2 & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & 2 & \mu_{n-1} \\ 0 & \cdots & 0 & \lambda_n & 2 \end{pmatrix}, \quad m = \begin{pmatrix} m_0 \\ m_2 \\ \vdots \\ m_{n-1} \\ m_n \end{pmatrix}, \quad g = \begin{pmatrix} g_0 \\ g_1 \\ \vdots \\ g_{n-1} \\ g_n \end{pmatrix}.$$

由于 $\mu_i + \lambda_i = 1$ ($i = 0, 1, \dots, n$), 且 $a_{ii} = 2$, 故由引理得

$$\|A^{-1}\| \leq 1. \quad (2.8.25)$$

定理 2.3 若 $f(x) \in C[a, b]$, $S(x)$ 是以 $a = x_0 < x_1 < \cdots < x_n = b$ 为节点, 满足条件式 (2.8.1) 及式 (2.8.4) ' 的三次样条插值函数, 令

$$h_j = x_{j+1} - x_j, \quad h = \max_{0 \leq j \leq n-1} h_j, \quad \delta = \min_{0 \leq j \leq n-1} h_j,$$

设 $h/\delta \leq c < \infty$, 则 $S(x)$ 在 $[a, b]$ 上一致收敛到 $f(x)$ 。

证明 由于 $S(x)$ 可用式 (2.8.6) 表示, 当 $x \in [a, b]$ 时, 利用式 (2.7.9) 可得到

$$\|f(x) - S(x)\|_\infty \leq \omega(h) + (8/27)h\|m\|_\infty, \quad (2.8.26)$$

由方程 (2.8.24), 并利用式 (2.8.25) 得

$$\|m\|_\infty \leq \|A^{-1}\|_\infty \|g\|_\infty \leq \|g\|_\infty^m, \quad (2.8.27)$$

根据式 (2.8.11) 及式 (2.8.13) ' 可估得

$$\|g\|_\infty \leq 3 \max_{0 \leq j \leq n-1} |f[x_j, x_{j+1}]]| \leq \frac{3}{\delta} \omega(h) \leq \frac{6}{\delta} \|f\|_\infty, \quad (2.8.28)$$

将式 (2.8.9) 及式 (2.8.28) 代入式 (2.8.26), 得

$$\|f(x) - S(x)\|_\infty \leq \left(1 + \frac{8}{9} \frac{h}{\delta}\right) \omega(h).$$

由于 $f(x) \in C[a, b]$, 当 $h \rightarrow 0$ 时, $\|f(x) - S(x)\|_\infty \rightarrow 0$, 故 $S(x)$ 在 $[a, b]$ 上一致收敛到 $f(x)$ 。证毕。

如果 $f(x) \in C^1[a, b]$, 则 $S'(x)$ 在 $[a, b]$ 上也一致收敛到 $f'(x)$ 。证明从略。

其他边界条件的三次样条插值函数收敛性证明与此类似, 读者可自行证明。

小结

插值理论是一个古老而实用的课题, 它是数值微积分、函数逼近、微分方程数值解等数值分析的基础。本章介绍的插值方法及差商与差分等概念是数值计算最基本的内容。Lagrange 插值多项式是数值积分与常微分方程数值解的重要工具, 理论上较重要。而分段多项式插值由于具有良好的稳定性与收敛性, 因此更便于应用, 特别是样条函数的理论与应用, 自 20 世纪 60 年代以来得到越来越多的重视。本章只介绍了三次样条函数, 它是实际应用中最重要样条函数之一。至于 B 样条和一般样条

续页: 小结在下一页续完。

校注 (agent 补充; 原书疑误): 本页 (2.8.24) 下面矩阵的末行次对角元印为 λ_n , 而自然边界的 (2.8.14) 对应位置为 1; 向量 m 的第二项印为 m_2 , 按方程顺序应为 m_1 。上述均保留原图, 参见[矩阵与向量原图裁切](#)。本页没有为自然边界另定义 λ_n , 不能沿用前页周期边界的同名系数。

校注 (agent 补充; 原书疑误): 式 (2.8.27) 最后一个范数原图上方确有多余的 m , 已按原式转录为 $\|g\|_\infty^m$; 由 (2.8.25) 应得 $\|m\|_\infty \leq \|g\|_\infty$, 无这个幂次。参见[原式放大裁图](#)。其后“将式 (2.8.9) 及式 (2.8.28) 代入”中的 (2.8.9) 应指此处的范数估计 (2.8.27); 原文编号已保留。

原书 PDF 页 56; 印刷页码 43。

[查看原页](#)

本页接上页小结。

函数我们都未涉及, 需要对样条函数作更进一步研究的读者可参看文献 [3]。

习题

1. 根据式 (2.2.2) 定义的 Vandermonde 行列式, 令

$$V_n(x) = V_n(x_0, x_1, \dots, x_{n-1}, x) = \begin{vmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^n \\ 1 & x & x^2 & \cdots & x^n \end{vmatrix},$$

证明 $V_n(x)$ 是 n 次多项式, 它的根是 $x_0, x_1, \dots, x_{n-1}$, 且

$$V_n(x) = V_{n-1}(x_0, x_1, \dots, x_{n-1})(x - x_0) \cdots (x - x_{n-1}).$$

2. 当 $x = 1, -1, 2$ 时, $f(x) = 0, -3, 4$, 求 $f(x)$ 的二次插值多项式。

3. 给出 $f(x) = \ln x$ 的数值表 (见表 2.9), 用线性插值及二次插值计算 $\ln 0.54$ 的近似值。

表 2.9

x	0.4	0.5	0.6	0.7	0.8
$\ln x$	-0.916 291	-0.693 147	-0.510 826	-0.357 765	-0.223 144

4. 给出 $\cos x, 0^\circ \leq x \leq 90^\circ$ 的函数表, 步长 $h = 1' = (1/60)^\circ$, 若函数表具有五位有效数字, 研究用线性插值求 $\cos x$ 近似值时的总误差限。

5. 设 $x_k = x_0 + kh, k = 0, 1, 2, 3$, 求 $\max_{x_0 \leq x \leq x_3} |l_2(x)|$ 。

6. 设 $x_j (j = 0, 1, \dots, n)$ 为互异节点, 求证:

$$(1) \sum_{j=0}^n x_j^k l_j(x) \equiv x^k \quad (k = 0, 1, \dots, n);$$

$$(2) \sum_{j=0}^n (x_j - x)^k l_j(x) \equiv 0 \quad (k = 1, 2, \dots, n).$$

7. 设 $f(x) \in C^2[a, b]$ 且 $f(a) = f(b) = 0$, 求证 $\max_{a \leq x \leq b} |f(x)| \leq \frac{1}{8}(b-a)^2 \max_{a \leq x \leq b} |f''(x)|$ 。

8. 在 $-4 \leq x \leq 4$ 上给出 $f(x) = e^x$ 的等距节点函数表, 若用二次插值求 e^x 的近似值, 要使截断误差不超过 10^{-6} , 使用函数表的步长 h 应取多少?

9. 若 $y_n = 2^n$, 求 $\Delta^4 y_n$ 及 $\delta^4 y_n$ 。

10. 如果 $f(x)$ 是 m 次多项式, 记 $\Delta f(x) = f(x+h) - f(x)$, 证明 $f(x)$ 的 k 阶差分 $\Delta^k f(x)$ ($0 \leq k \leq m$) 是 $m-k$ 次多项式, 并且 $\Delta^{m+l} f(x) = 0$ (l 为正整数)。
11. 证明 $\Delta(f_k g_k) = f_k \Delta g_k + g_{k+1} \Delta f_k$ 。
12.
$$\sum_{k=0}^{n-1} f_k \Delta g_k = f_n g_n - f_0 g_0 - \sum_{k=0}^{n-1} g_{k+1} \Delta f_k.$$
13. 证明 $\sum_{j=0}^{n-1} \Delta^2 y_j = \Delta y_n - \Delta y_0$ 。
14. 若 $f(x) = a_0 + a_1 x + \cdots + a_{n-1} x^{n-1} + a_n x^n$ 有 n 个不同实根 $x_1, x_2, \cdots, x_n$, 证明:

续页: 第 14 题结论在下一页。

校注 (agent 补充; 原书表值疑误): 表 2.9 在 $x = 0.7$ 列原扫描为 -0.357765 , 已保留, 参见[表 2.9 原图裁切](#)。实际 $\ln 0.7 \approx -0.356674944$, 六位小数为 -0.356675 , 故原表这一项疑有数字误排。

原书 PDF 页 57; 印刷页码 44。

[查看原页](#)

本页接上页习题 14 的结论。

$$\sum_{j=1}^n \frac{x_j^k}{f'(x_j)} = \begin{cases} 0, & 0 \leq k \leq n-2, \\ a_n^{-1}, & k = n-1. \end{cases}$$

15. 证明 n 阶差商有下列性质:

- (1) 若 $F(x) = cf(x)$, 则 $F[x_0, x_1, \dots, x_n] = cf[x_0, x_1, \dots, x_n]$;
 (2) 若 $F(x) = f(x) + g(x)$, 则 $F[x_0, x_1, \dots, x_n] = f[x_0, x_1, \dots, x_n] + g[x_0, x_1, \dots, x_n]$ 。

16. $f(x) = x^7 + x^4 + 3x + 1$, 求 $f[2^0, 2^1, \dots, 2^7]$ 及 $f[2^0, 2^1, \dots, 2^8]$ 。

17. 证明两点三次 Hermite 插值余项是

$$R_3(x) = f^{(4)}(\xi)(x - x_k)^2(x - x_{k+1})^2/4!, \quad \xi \in (x_k, x_{k+1}),$$

并由此求出分段三次 Hermite 插值的误差限。

18. 求一个次数不高于四次的多项式 $P(x)$, 使它满足 $P(0) = P(-k+1)$, 并由此求出分段三次 Hermite 插值的误差限。

19. 求一个次数不高于四次的多项式 $P(x)$, 使它满足 $P(0) = P'(0) = 0, P(1) = P'(1) = 1, P(2) = 1$ 。

20. 设 $f(x) \in C[a, b]$, 把 $[a, b]$ 分为 n 等份, 试构造一个台阶形的零次分段插值函数 $\varphi_n(x)$, 并证明当 $n \rightarrow \infty$ 时 $\varphi_n(x)$ 在 $[a, b]$ 上一致收敛到 $f(x)$ 。

21. 设 $f(x) = 1/(1+x^2)$, 在 $-5 \leq x \leq 5$ 上取 $n = 10$, 按等距节点求分段线性插值函数 $I_h(x)$, 计算各节点间中点处的 $I_h(x)$ 与 $f(x)$ 的值, 并估计误差。

22. 求 $f(x) = x^2$ 在 $[a, b]$ 上的分段线性插值函数 $I_h(x)$, 并估计误差。

23. 求 $f(x) = x^4$ 在 $[a, b]$ 上的分段 Hermite 插值, 并估计误差。

24. 给定数据如表 2.10 所示, 试求三次样条插值 $S(x)$, 并满足条件

(1) $S'(0.25) = 1.0000, S'(0.53) = 0.6868$;

(2) $S''(0.25) = S''(0.53) = 0$ 。

表 2.10

x_j	0.25	0.30	0.39	0.45	0.53
y_j	0.500 0	0.547 7	0.624 5	0.670 8	0.728 0

25. 若 $f(x) \in C^2[a, b]$, $S(x)$ 是三次样条函数, 证明

(1)

$$\begin{aligned} & \int_a^b [f''(x)]^2 dx - \int_a^b [S''(x)]^2 dx \\ &= \int_a^b [f''(x) - S''(x)]^2 dx + 2 \int_a^b S''(x)[f''(x) - S''(x)] dx; \end{aligned}$$

(2) 若 $f(x_i) = S(x_i)$ ($i = 0, 1, \dots, n$), 式中 x_i 为插值节点, 且 $a = x_0 < x_1 < \dots < x_n = b$, 则

$$\int_a^b S''(x)[f''(x) - S''(x)] dx = S''(b)[f'(b) - S'(b)] - S''(a)[f'(a) - S'(a)].$$

26. 编出计算三次样条函数 $S(x)$ 系数及其在插值节点中点的值的程序框图 ($S(x)$ 可用式 (2.8.7) 的表达式)。

校注 (agent 补充; 原书题目疑误): 第 18 题原扫描确为上述文字, 参见[第 18 题原图裁切](#)。题中 k 未定义, 所给多项式条件不完整, 末句又重复第 17 题的误差限要求, 无法从本页可靠恢复原意; 此处保留原文, 不补造条件。第 24 题的两组边界条件也按原书排列保留, 未将其合并解题。

原书 PDF 页 58; 印刷页码 45。

[查看原页](#)

第 3 章函数逼近与计算

3.1 引言与预备知识

3.1.1 问题的提出

在数值计算中经常遇到求函数值的问题. 用手工计算时常常通过函数表求得; 用计算机计算时, 若把函数表存入内存进行查表, 则占用单元太多, 不如直接用公式计算方便. 因此, 我们希望求出便于计算且计算量省的公式近似已知函数 $f(x)$. 例如, Taylor 展开式的部分和

$$P_n(x) = f(x_0) + \frac{f'(x_0)}{1!}(x - x_0) + \cdots + \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n \quad (3.1.1)$$

就是 $f(x)$ 的一种近似公式. 用它求 x_0 附近的函数值 $f(x)$ 误差较小, 当 $|x - x_0|$ 较大时误差就很大. 例如 $f(x) = e^x$ 在 $[-1, 1]$ 上用

$$P_4(x) = 1 + x + \frac{1}{2}x^2 + \frac{1}{6}x^3 + \frac{1}{24}x^4$$

近似 e^x , 其误差

$$R_4(x) = e^x - P_4(x) = \frac{1}{120}x^5e^\epsilon, \quad \epsilon \in (-1, 1),$$

于是

$$|R_4(x)| \leq \frac{e}{120}|x^5|, \quad \max_{-1 \leq x \leq 1} |R_4(x)| \leq \frac{e}{120} \approx 0.0226.$$

误差分布如图 3.1 所示, 它在整个区间上误差较大. 若在计算机上用这种方法计算 e^x , 如精度要求较高, 则需取很多项, 这样既费时又多占存储单元, 因此, 要求在给定精度下求计算次数最少的近似公式, 这就是函数逼近与计算要解决的问题. 这问题可叙述为: “对于函数类 A 中给定的函数 $f(x)$, 要求在另一类较简单的便于计算的函数类 B 中, 求函数 $P(x) \in B \subseteq A$, 使 $P(x)$ 与 $f(x)$ 之差在某种度量意义下最小.” 函数类 A 通常是区间 $[a, b]$ 上的连续函数, 记作 $C[a, b]$; 函数类 B 通常是代数多项式、分式有理函数或三角多项式. 而度量标准最常用的有两种, 一种是

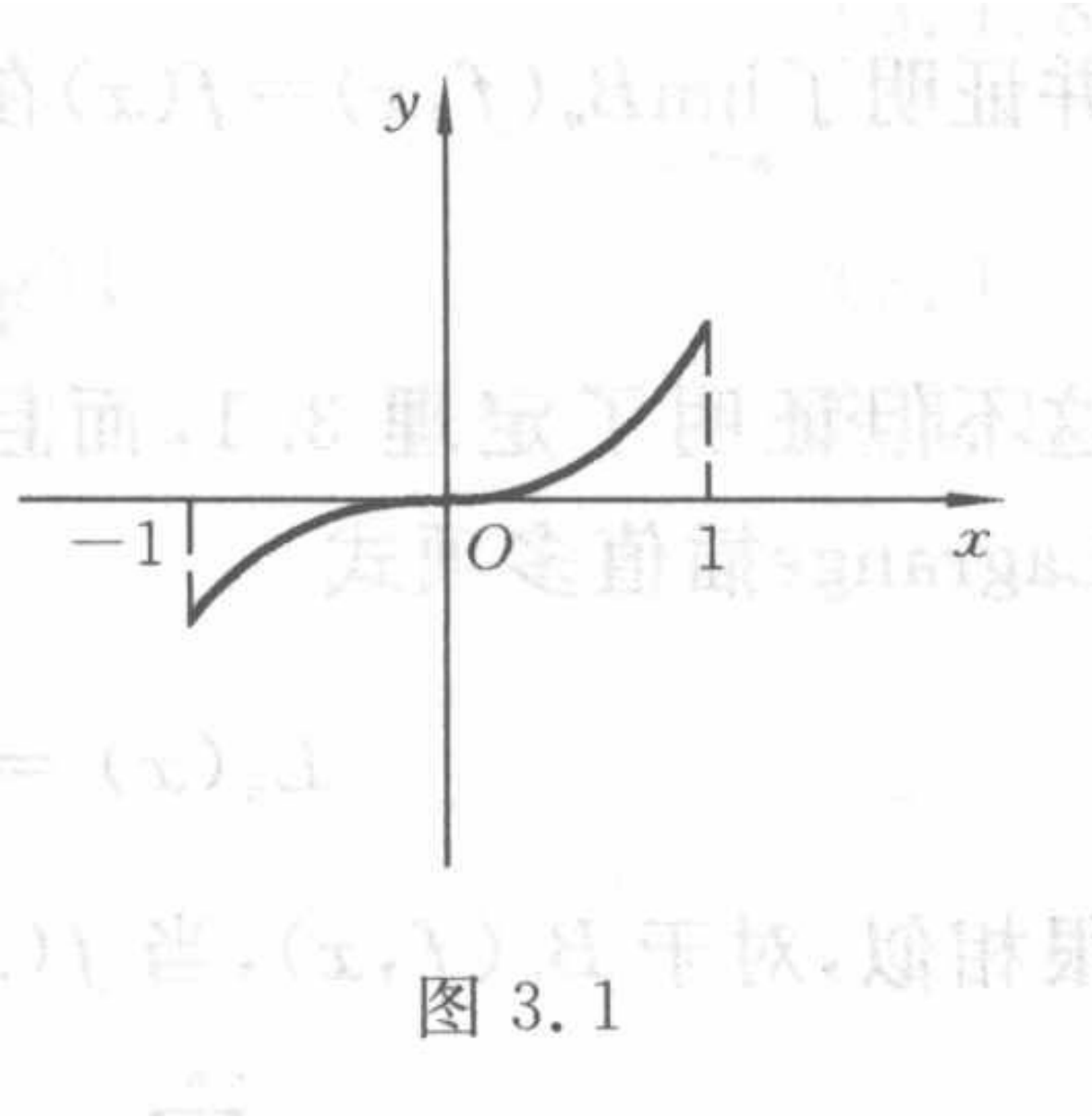


图 3.1（原图）。图中为上文误差分布曲线，横轴 x 、纵轴 y ；标签为 -1 、 1 、 O 。曲线在负半轴下方、正半轴上方，通过原点；在 $x = -1$ 和 $x = 1$ 处有竖向辅助线。

原书 PDF 页 59; 印刷页码 46。

[查看原页](#)

$$\|f(x) - P(x)\|_{\infty} = \max_{a \leq x \leq b} |f(x) - P(x)|, \quad (3.1.2)$$

在这种度量意义下的函数逼近称为一致逼近或均匀逼近; 另一种度量标准是

$$\|f(x) - P(x)\|_2 = \sqrt{\int_a^b [f(x) - P(x)]^2 dx}, \quad (3.1.3)$$

用这种度量的函数逼近称为均方逼近或平方逼近. 这里, 符号 $\|\cdot\|_{\infty}$ 及 $\|\cdot\|_2$ 是范数. 本章主要研究在这两种度量标准下用代数多项式 $P_n(x)$ 逼近 $f(x) \in C[a, b]$, 也就是用最佳一致逼近多项式与最佳平方逼近多项式逼近 $f(x) \in C[a, b]$.

3.1.2 Weierstrass 定理

用插值法或 Taylor 展开求 $f(x) \in C[a, b]$ 的逼近多项式, 在某些点上可能没有误差, 但在整个区间 $[a, b]$ 上误差可能很大, 2.7 节给出的 Runge 现象就说明了这一点. 因此, 用 $P_n(x)$ 一致逼近 $f(x)$, 首先要解决存在性问题, 即对于 $[a, b]$ 上的连续函数 $f(x)$, 是否存在多项式 $P_n(x)$ 一致收敛于 $f(x)$? Weierstrass 给出了下面定理.

定理 3.1 设 $f(x) \in C[a, b]$, 则对于任何 $\varepsilon > 0$, 总存在一个代数多项式 $P(x)$, 使

$$\|f(x) - P(x)\|_{\infty} < \varepsilon$$

在 $[a, b]$ 上一致成立.

这定理已在“数学分析”中证明过. 这里需要说明的是在许多证明方法中, Bernstein 在 1912 年给出的证明是一种构造性证明. 他根据函数整体逼近的特性造出 Bernstein 多项式

$$\begin{cases} B_n(f, x) = \sum_{k=0}^n f\left(\frac{k}{n}\right) P_k(x), \\ P_k(x) = \binom{n}{k} x^k (1-x)^{n-k}, \end{cases} \quad (3.1.4)$$

并证明了 $\lim_{n \rightarrow \infty} B_n(f, x) = f(x)$ 在 $[0, 1]$ 上一致成立; 若 $f(x)$ 在 $[0, 1]$ 上 m 阶导数连续, 则

$$\lim_{n \rightarrow \infty} B_n^{(m)}(f, x) = f^{(m)}(x).$$

这不但证明了定理 3.1, 而且由式 (3.1.4) 给出了 $f(x)$ 的一个逼近多项式. 它与 Lagrange 插值多项式

$$L_n(x) = \sum_{k=0}^n f(x_k) l_k(x), \quad \sum_{k=0}^n l_k(x_k) = 1$$

很相似, 对于 $B_n(f, x)$, 当 $f(x) = 1$ 时也有关系式

$$\sum_{k=0}^n P_k(x) = \sum_{k=0}^n \binom{n}{k} x^k (1-x)^{n-k} = 1. \quad (3.1.5)$$

这只要从恒等式

校注 (agent 补充, 原书疑误): 原页 Lagrange 插值式后的求和明确印为 $\sum_{k=0}^n l_k(x_k) = 1$, 此处按原文保留. 由基函数性质 $l_k(x_k) = 1$, 左端应为 $n+1$; 通常的恒等式应为 $\sum_{k=0}^n l_k(x) = 1$. 这不是 OCR 误识。

原书 PDF 页 60; 印刷页码 47。

[查看原页](#)

$$(x+y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k} \quad (3.1.6)$$

中令 $y = 1 - x$ 就可得到. 但这里当 $x \in [0, 1]$ 时还有 $P_k(x) \geq 0$, 于是

$$\sum_{k=0}^n |P_k(x)| = \sum_{k=0}^n P_k(x) = 1$$

是有界的, 因而只要 $|f(x)| \leq \delta$ 对于任意 $x \in [0, 1]$ 成立, 则

$$|B_n(f, x)| \leq \max_{0 \leq x \leq 1} |f(x)| \sum_{k=0}^n |P_k(x)| \leq \delta$$

有界, 故 $B_n(f, x)$ 是稳定的. 至于 Lagrange 插值多项式 $L_n(x)$, 由于 $\sum_{k=0}^n |l_k(x)|$ 无界, 因而不能保证高阶插值的稳定性与收敛性. 相比之下, 多项式 $B_n(f, x)$ 有良好的逼近性质, 但它收敛太慢, 比三次样条逼近效果差得多, 实际中很少使用.

3.1.3 连续函数空间 $C[a, b]$

区间 $[a, b]$ 上的所有实连续函数组成一个空间, 记作 $C[a, b]$. $f \in C[a, b]$ 的范数定义为

$$\|f\|_{\infty} = \max_{a \leq x \leq b} |f(x)|,$$

$\|\cdot\|_{\infty}$ 称为 ∞ -范数, 它满足范数 $\|\cdot\|$ 的三个性质: (1) $\|f\| \geq 0$, 当且仅当 $f \equiv 0$ 时才有 $\|f\| = 0$; (2) $\|af\| = |a|\|f\|$ 对于任意 $f \in C[a, b]$ 成立, a 为任意实数; (3) 对于任意 $f, g \in C[a, b]$, 有

$$\|f + g\| \leq \|f\| + \|g\| \quad (3.1.7)$$

式 (3.1.7) 称为三角不等式. 空间 $C[a, b]$ 可与向量空间类比, 函数 $f \in C[a, b]$ 可看成向量. 与向量空间类似, 当 $f, g \in C[a, b]$ 时, 定义 f 与 g 的距离为

$$D(f, g) = \|f - g\|_{\infty} \quad (3.1.8)$$

由式 (3.1.7) 可得到

$$D(f, g) \leq D(f, h) + D(h, g) \quad (3.1.9)$$

$$|\|f\|_{\infty} - \|g\|_{\infty}| \leq \|f - g\|_{\infty} \quad (3.1.10)$$

3.2 最佳一致逼近多项式

3.2.1 最佳一致逼近多项式的存在性

Chebyshev 从另一观点研究一致逼近问题. 他不让多项式次数 n 趋于无穷,

原书 PDF 页 61; 印刷页码 48。

[查看原页](#)

而是固定 n , 记次数不大于 n 的多项式集合为 H_n , 显然 $H_n \subseteq C[a, b]$. 又记 $H_n = \text{span}\{1, x, \dots, x^n\}$, 其中 $1, x, \dots, x^n$ 是 $[a, b]$ 上一组线性无关的函数组, 是 H_n 中的一组基. H_n 中的元素 $P_n(x)$ 可表示为

$$P_n(x) = a_0 + a_1x + \dots + a_nx^n,$$

其中 $a_0, a_1, \dots, a_n$ 为任意实数. 要在 H_n 中求 $P_n^*(x)$ 逼近 $f(x) \in C[a, b]$, 使其误差

$$\max_{a \leq x \leq b} |f(x) - P_n^*(x)| = \min_{P_n \in H_n} \max_{a \leq x \leq b} |f(x) - P_n(x)|,$$

这就是通常所谓最佳一致逼近或 Chebyshev 逼近问题. 为了说明这一概念, 先给出以下定义.

定义 3.1 $P_n(x) \in H_n, f(x) \in C[a, b]$, 称

$$\Delta(f, P_n) = \|f - P_n\|_\infty = \max_{a \leq x \leq b} |f(x) - P_n(x)| \quad (3.2.1)$$

为 $f(x)$ 与 $P_n(x)$ 在 $[a, b]$ 上的偏差.

显然, $\Delta(f, P_n) \geq 0$, $\Delta(f, P_n)$ 的全体组成一个集合, 记作 $\{\Delta(f, P_n)\}$, 它有下列下界 0. 若记集合的下确界为

$$E_n = \inf_{P_n \in H_n} \{\Delta(f, P_n)\} = \inf_{P_n \in H_n} \max_{a \leq x \leq b} |f(x) - P_n(x)| \quad (3.2.2)$$

则称 E_n 为 $f(x)$ 在 $[a, b]$ 上的最小偏差.

定义 3.2 假定 $f(x) \in C[a, b]$, 若存在

$$P_n^*(x) \in H_n, \Delta(f, P_n^*) = E_n \quad (3.2.3)$$

则称 $P_n^*(x)$ 是 $f(x)$ 在 $[a, b]$ 上的最佳一致逼近多项式或最小偏差逼近多项式, 简称最佳逼近多项式.

注意, 定义并未说明最佳逼近多项式是否存在, 但可证明下面的存在定理.

定理 3.2 若 $f(x) \in C[a, b]$, 则总存在 $P_n^*(x) \in H_n$, 使

$$\|f(x) - P_n^*(x)\|_\infty = E_n.$$

证明过程见文献 [4], 此处从略.

3.2.2 Chebyshev 定理

这一节要研究最佳逼近多项式的特性, 为此先引进偏差点定义.

定义 3.3 设 $f(x) \in C[a, b], P(x) \in H_n$, 若在 $x = x_0$ 上有

$$|P(x_0) - f(x_0)| = \max_{a \leq x \leq b} |P(x) - f(x)| = \mu,$$

则称 x_0 是 $P(x)$ 的偏差点.

若 $P(x_0) - f(x_0) = \mu$, 则称 x_0 为“正”偏差点.

若 $P(x_0) - f(x_0) = -\mu$, 则称 x_0 为“负”偏差点.

由于函数 $P(x) - f(x)$ 在 $[a, b]$ 上连续, 因此, 至少存在一个点 $x_0 \in [a, b]$, 使得

$$|P(x_0) - f(x_0)| = \mu,$$

也就是说, $P(x)$ 的偏差点总是存在的. 下面讨论最佳逼近多项式的偏差点性质.

原书 PDF 页 62; 印刷页码 49。

[查看原页](#)

定理 3.3 若 $P(x) \in H_n$ 是 $f(x) \in C[a, b]$ 的最佳逼近多项式, 则 $P(x)$ 同时存在正、负偏差点.

证明因 $P(x)$ 是 $f(x)$ 的最佳逼近多项式, 故 $\mu = E_n$. 由于 $P(x)$ 在 $[a, b]$ 上总有偏差点存在, 故可用反证法求证. 不妨假定只有正偏差点, 没有负偏差点, 于是, 对于一切 $x \in [a, b]$ 都有

$$P(x) - f(x) > -E_n.$$

因 $P(x) - f(x)$ 在 $[a, b]$ 上连续, 故有最小值大于 $-E_n$, 用 $-E_n + 2h$ 表示, 其中 $h > 0$. 于是, 对于一切 $x \in [a, b]$ 都有

$$-E_n + 2h \leq P(x) - f(x) \leq E_n,$$

$$-E_n + h \leq [P(x) - h] - f(x) \leq E_n - h,$$

即

$$|[P(x) - h] - f(x)| \leq E_n - h.$$

它表示多项式 $P(x) - h$ 与 $f(x)$ 的偏差小于 E_n , 与 E_n 是最小偏差的假定矛盾.

同样, 可证明只有负偏差点没有正偏差点也是不成立的. 证毕.

定理 3.3 的证明从几何上看是十分明显的. 如图 3.2 所示, 考察两曲线

$$y = f(x) + E_n \quad \text{与} \quad y = f(x) - E_n$$

在 $[a, b]$ 间形成带状区域, 曲线 $y = P(x)$ 在 $[a, b]$ 上就位于这带状区域之间. 定理 3.1 表明, $P(x)$ 的图形应当与这两条曲线至少各接触一次, 这是十分明显的. 若不与 $y = f(x) - E_n$ 接触, 则可把曲线 $y = P(x)$ 稍向下移动就得到位于曲线 $y = f(x)$ 较窄带状区域内的曲线 $y = P(x) - h$. 下面给出反映最佳逼近多项式特征的 Chebyshev 定理.

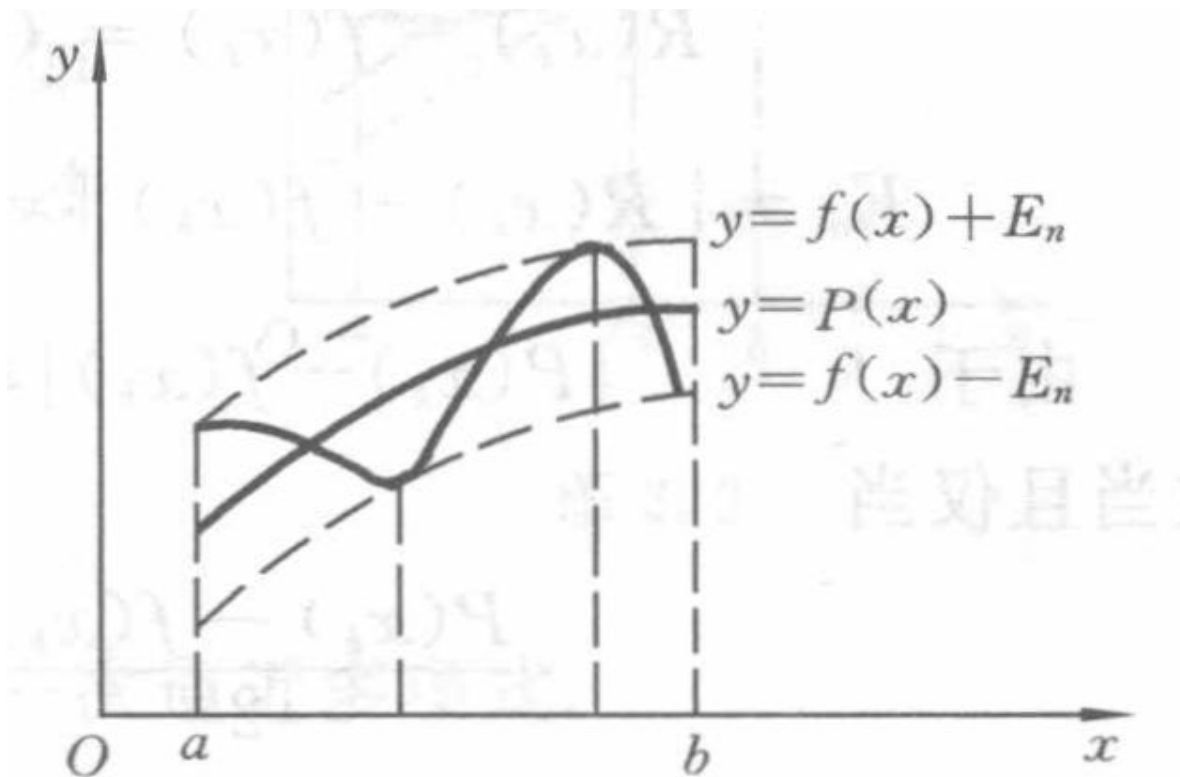


图 3.2

图 3.2 (原图)。带状区域及逼近曲线示意; 标签为 x 、 y 、 O 、 a 、 b 、 $y = f(x) + E_n$ 、 $y = P(x)$ 、 $y = f(x) - E_n$ 。图内还画有从曲线接触点向横轴延伸的竖向辅助线。

定理 3.4 $P(x) \in H_n$ 是 $f(x) \in C[a, b]$ 的最佳逼近多项式的充分必要条件是 $P(x)$ 在 $[a, b]$ 上至少有 $n+2$ 个轮流为“正”、“负”的偏差点, 即有 $n+2$ 个点 $a \leq x_1 < x_2 < \cdots < x_{n+2} \leq b$, 使

$$\begin{aligned} P(x_k) - f(x_k) &= (-1)^k \sigma \|P(x) - f(x)\|_{\infty}, \\ \sigma &= \pm 1, \quad k = 1, 2, \cdots, n+2, \end{aligned} \quad (3.2.4)$$

这样的点组称为 Chebyshev 交错点组。

证明只证充分性。假定在 $[a, b]$ 上有 $n+2$ 个点使式 (3.2.4) 成立, 要证明 $P(x)$ 是 $f(x)$ 在 $[a, b]$ 上的最佳逼近多项式。用反证法求证。若存在 $Q(x) \in H_n, Q(x) \neq P(x)$, 使

$$\|f(x) - Q(x)\|_{\infty} < \|f(x) - P(x)\|_{\infty}.$$

由于 $P(x) - Q(x) = [P(x) - f(x)] - [Q(x) - f(x)]$ 在点 $x_1, x_2, \cdots, x_{n+2}$ 上的符号与 $P(x_k) - f(x_k)$ ($k = 1, \cdots, n+2$) 一致, 故 $P(x) -$

校注 (agent 补充, 原书疑误): 图 3.2 旁正文印为“定理 3.1 表明”, 按原文保留。此段是在解释正、负偏差点同时存在, 应指本页定理 3.3; 定理 3.1 是 Weierstrass 定理。

原书 PDF 页 63; 印刷页码 50。

[查看原页](#)

$Q(x)$ 也在 $n+2$ 个点上轮流取符号 “+”、“-”。由连续函数性质, 它在 (a, b) 内有 $n+1$ 个零点, 但 $P(x) - Q(x) \neq 0$ 是不超过 n 次的多项式, 它的零点不超过 n . 这个矛盾说明假设不对, 故 $P(x)$ 就是所求最佳逼近多项式. 充分性得证.

必要性证明较繁, 但证明思想类似定理 3.3, 此处从略.

定理 3.4 说明, 用 $P(x)$ 逼近 $f(x)$ 的误差曲线 $y = P(x) - f(x)$ 是均匀分布的. 由定理 3.4 还可得出以下重要推论.

推论 1 若 $f(x) \in C[a, b]$, 则在 H_n 中存在唯一的最佳逼近多项式.

证明若 H_n 中有两个最佳逼近多项式 $P(x)$ 与 $Q(x)$, 则对于一切 $x \in [a, b]$, 都有

$$-E_n \leq P(x) - f(x) \leq E_n, \quad -E_n \leq Q(x) - f(x) \leq E_n,$$

于是

$$-E_n \leq \frac{P(x) + Q(x)}{2} - f(x) \leq E_n.$$

它表明 $R(x) = \frac{Q(x) + P(x)}{2}$ 也是 H_n 中的最佳逼近多项式, 因而 $R(x) - f(x)$ 的 $n+2$ 个点的交错点组 $\{x_k\}$ 满足

$$R(x_k) - f(x_k) = (-1)^k \sigma E_n \quad (k = 1, 2, \dots, n+2),$$

$$E_n = |R(x_k) - f(x_k)| = \left| \frac{P(x_k) - f(x_k)}{2} + \frac{Q(x_k) - f(x_k)}{2} \right|. \quad (3.2.5)$$

由于 $|P(x_k) - f(x_k)| \leq E_n$, $|Q(x_k) - f(x_k)| \leq E_n$, 故当且仅当

$$\frac{P(x_k) - f(x_k)}{2} = \frac{Q(x_k) - f(x_k)}{2} = \pm \frac{E_n}{2}$$

时式 (3.2.5) 才能成立, 于是

$$P(x_k) - f(x_k) = Q(x_k) - f(x_k).$$

这就得到 $P(x_k) = Q(x_k)$ ($k = 1, 2, \dots, n+2$), 从而表明 $P(x) - Q(x)$ 有 $n+2$ 个根. 这个矛盾说明 $Q(x) \equiv P(x)$, 推论 1 得证.

推论 2 若 $f(x) \in C[a, b]$, 则其最佳逼近多项式 $P_n^*(x) \in H_n$ 就是 $f(x)$ 的一个 Lagrange 插值多项式.

证明由定理 3.4 可知, $P_n^*(x) - f(x)$ 在 $[a, b]$ 上要么恒为零, 要么有 $n+2$ 个轮流取 “正”、“负” 的偏差点, 于是存在 $n+1$ 个点 $x_k \leq \bar{x}_k \leq x_{k+1}$ ($k = 1, 2, \dots, n+1$), 使 $P_n^*(\bar{x}_k) - f(\bar{x}_k) = 0$. 以 $\bar{x}_k$ 为插值节点的 Lagrange 插值多项式就是 $P_n^*(x)$.

3.2.3 最佳一次逼近多项式

定理 3.4 给出了最佳逼近多项式 $P(x)$ 的特性, 但要求出 $P(x)$ 却相当困难. 下面先讨论 $n = 1$ 的情形.

假定 $f(x) \in C^2[a, b]$, 且 $f''(x)$ 在 (a, b) 内不变号, 要求最佳一次逼近多项式

原书 PDF 页 64; 印刷页码 51。

[查看原页](#)

$P_1(x) = a_0 + a_1x$. 根据定理 3.4 可知, 至少有 3 个点 $a \leq x_1 < x_2 < x_3 \leq b$, 使 $P_1(x_k) - f(x_k) = (-1)^k \sigma \max_{a \leq x \leq b} |P_1(x) - f(x)|$ ($\sigma = \pm 1, k = 1, 2, 3$).

由于 $f''(x)$ 在 $[a, b]$ 上不变号, 故 $f'(x)$ 单调, $f'(x) - a_1$ 在 (a, b) 内只有一个零点, 记作 x_2 , 于是 $P_1'(x_2) - f'(x_2) = a_1 - f'(x_2) = 0$, 即 $f'(x_2) = a_1$.

另外两个偏差点必在区间端点, 即 $x_0 = a, x_1 = b$, 且满足

$$P_1(a) - f(a) = P_1(b) - f(b) = -[P_1(x_2) - f(x_2)].$$

由此得到

$$\begin{cases} a_0 + a_1a - f(a) = a_0 + a_1b - f(b), \\ a_0 + a_1a - f(a) = f(x_2) - (a_0 + a_1x_2). \end{cases} \quad (3.2.6)$$

解出

$$a_1 = \frac{f(b) - f(a)}{b - a} = f'(x_2), \quad (3.2.7)$$

代入式 (3.2.6), 得

$$a_0 = \frac{f(a) + f(x_2)}{2} - \frac{f(b) - f(a)}{b - a} \frac{a + x_2}{2}. \quad (3.2.8)$$

这就得到最佳一次逼近多项式 $P_1(x)$, 其几何意义如图 3.3 所示。直线 $y = P_1(x)$ 与弦 MN 平行, 且通过 MQ 的中点 D , 其方程为

$$y = \frac{1}{2}[f(a) + f(x_2)] + a_1 \left(x - \frac{a + x_2}{2} \right).$$

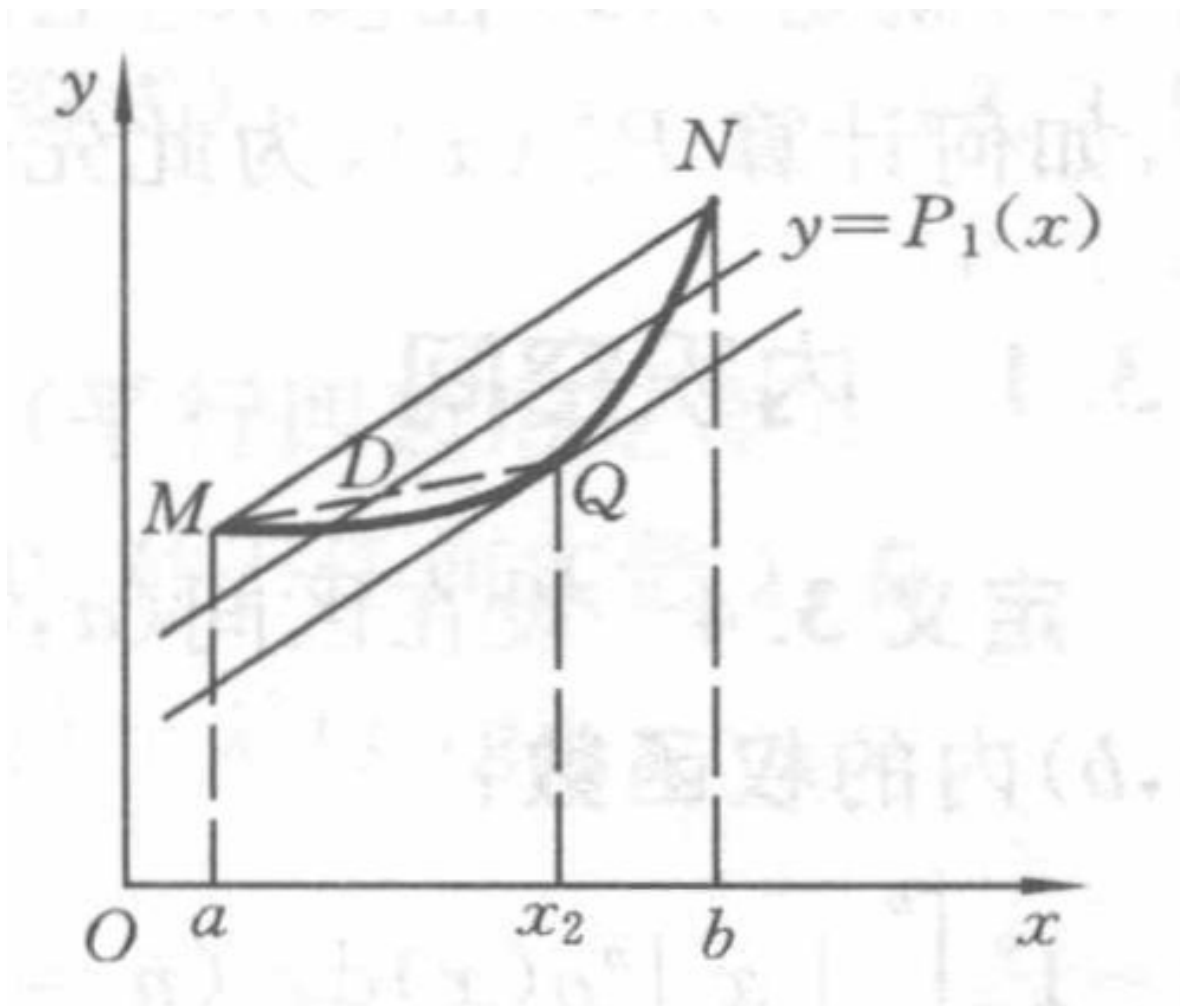


图 3.3

图 3.3 (原图)。最佳一次逼近的几何意义; 标签为 x 、 y 、 O 、 a 、 x_2 、 b 、 M 、 N 、 Q 、 D 、 $y = P_1(x)$ 。弦 MN 、逼近直线以及通过 Q 的平行线构成图中的三条斜线, D 为 MQ 的中点。

例 3.1 求 $f(x) = \sqrt{1+x^2}$ 在 $[0, 1]$ 上的最佳一次逼近多项式。

解由式 (3.2.7) 可算出

$$a_1 = \sqrt{2} - 1 \approx 0.414, \quad f'(x) = \frac{x}{\sqrt{1+x^2}},$$

故

$$\frac{x_2}{\sqrt{1+x_2^2}} = \sqrt{2} - 1,$$

解得

$$x_2 = \sqrt{\frac{\sqrt{2}-1}{2}} \approx 0.4551, \quad f(x_2) = \sqrt{1+x_2^2} \approx 1.0986.$$

由式 (3.2.8), 得

$$a_0 = \frac{1 + \sqrt{1 + x_2^2}}{2} - a_1 \frac{x_2}{2} \approx 0.955,$$

于是得 $\sqrt{1 + x^2}$ 的最佳一次逼近多项式为

$$P_1(x) = 0.955 + 0.414x,$$

故

$$\sqrt{1 + x^2} \approx 0.955 + 0.414x, \quad 0 \leq x \leq 1. \quad (3.2.9)$$

误差限为

$$\max_{0 \leq x \leq 1} |\sqrt{1 + x^2} - P_1(x)| \leq 0.045.$$

校注 (agent 补充, 原书疑误): 本页开头先把三个交错点记为 $x_1 < x_2 < x_3$, 随后原印 “ $x_0 = a, x_1 = b$ ”, 此处忠实保留。按前文编号应为 $x_1 = a, x_3 = b$ 。

校注 (agent 补充, 数值舍入): 原书对所写 $P_1(x) = 0.955 + 0.414x$ 给出误差限 0.045, 这里保留原文。若把这两个已舍入系数当作精确数, 则在 $x = 1$ 处误差为 $\sqrt{2} - 1.369 \approx 0.04521356 > 0.045$, 故所印误差限不适用于这组舍入后的系数。

原书 PDF 页 65; 印刷页码 52。

[查看原页](#)

在式 (3.2.9) 中若令 $x = \frac{b}{a} \leq 1$, 则可得一个求根式的公式

$$\sqrt{a^2 + b^2} \approx 0.955a + 0.414b.$$

3.3 最佳平方逼近

用均方误差最小作为度量标准, 研究函数 $f(x) \in C[a, b]$ 的逼近多项式, 就是这一节要讨论的最佳平方逼近问题. 若存在 $P_n^*(x) \in H_n$, 使

$$\|f - P_n^*\|_2 = \sqrt{\int_a^b [f(x) - P_n^*(x)]^2 dx} = \inf_{P \in H_n} \|f - P\|_2, \quad (3.3.1)$$

$P_n^*(x)$ 就是 $f(x)$ 在 $[a, b]$ 上的最佳平方逼近多项式. 这一节将研究 $P_n^*(x)$ 是否存在, 如何计算 $P_n^*(x)$, 为此先介绍一些有关内积空间的预备知识.

3.3.1 内积空间

定义 3.4 设在区间 (a, b) 内, 非负函数 $\rho(x)$ 满足以下条件, 就称 $\rho(x)$ 为区间 (a, b) 内的权函数:

1° $\int_a^b |x|^n \rho(x) dx (n = 0, 1, \dots)$ 存在;

2° 对于非负连续函数 $g(x)$, 若

$$\int_a^b g(x) \rho(x) dx = 0, \quad (3.3.2)$$

则在 (a, b) 内 $g(x) \equiv 0$.

定义 3.5 设 $f(x), g(x) \in C[a, b]$, $\rho(x)$ 是 $[a, b]$ 上的权函数, 积分

$$(f, g) = \int_a^b \rho(x) f(x) g(x) dx \quad (3.3.3)$$

称为函数 $f(x)$ 与 $g(x)$ 在 $[a, b]$ 上的内积.

容易验证这样定义的内积满足下列四条公理:

- (1) $(f, g) = (g, f)$;
- (2) $(cf, g) = c(f, g)$, c 为常数;
- (3) $(f_1 + f_2, g) = (f_1, g) + (f_2, g)$;
- (4) $(f, f) \geq 0$, 当且仅当 $f = 0$ 时 $(f, f) = 0$.

满足内积定义的函数空间称为内积空间, 因此, 连续函数空间 $C[a, b]$ 上定义了内积就形成一个内积空间. 这里定义的内积是 n 维 Euclid 空间 $\mathbf{R}^n$ 中两个向量内积定义的推广, 设

$$f = (f_1, f_2, \dots, f_n)^T, \quad g = (g_1, g_2, \dots, g_n)^T,$$

原书 PDF 页 66; 印刷页码 53。

[查看原页](#)

其内积定义为 $(f, g) = \sum_{k=1}^n f_k g_k$; 向量 $f \in \mathbf{R}^n$ 的模 (范数) 定义为

$$\|f\|_2 = \left(\sum_{k=1}^n f_k^2 \right)^{\frac{1}{2}},$$

将它推广到任何内积空间中就有下面的定义.

定义 3.6 $f(x) \in C[a, b]$, 量

$$\|f\|_2 = \sqrt{\int_a^b \rho(x) f^2(x) dx} = \sqrt{(f, f)} \quad (3.3.4)$$

称为 $f(x)$ 的 Euclid 范数.

它同样满足范数三条性质 (见 3.1 节), 头两条是显然的, 下面定理包含了第三条性质和其他重要结论.

定理 3.5 对于任何 $f, g \in C[a, b]$, 下列结论成立: 1°

$$|(f, g)| \leq \|f\|_2 \|g\|_2 \quad (\text{Cauchy-Schwarz 不等式}); \quad (3.3.5)$$

2°

$$\|f + g\|_2 \leq \|f\|_2 + \|g\|_2 \quad (\text{三角不等式}); \quad (3.3.6)$$

3° $\|f + g\|_2^2 + \|f - g\|_2^2 = 2(\|f\|_2^2 + \|g\|_2^2)$ (平行四边形定律).

证明若 $g = 0$, 则式 (3.3.5) 显然成立, 现考虑 $g \neq 0$. 对于任何实数 λ , 有

$$0 \leq (f + \lambda g, f + \lambda g) = (f, f) + 2\lambda(f, g) + \lambda^2(g, g).$$

现取 $\lambda = -\frac{(f, g)}{\|g\|_2^2}$, 代入上式得

$$\|f\|_2^2 - 2\frac{|(f, g)|^2}{\|g\|_2^2} + \frac{|(f, g)|^2}{\|g\|_2^2} \geq 0, \quad |(f, g)|^2 \leq \|f\|_2^2 \|g\|_2^2.$$

两边开平方即得式 (3.3.5).

利用式 (3.3.5) 考虑

$$\begin{aligned} \|f + g\|_2^2 &= (f + g, f + g) = (f, f) + 2(f, g) + (g, g) \\ &\leq \|f\|_2^2 + 2|(f, g)| + \|g\|_2^2 \\ &\leq \|f\|_2^2 + 2\|f\|_2 \|g\|_2 + \|g\|_2^2 \\ &= (\|f\|_2 + \|g\|_2)^2, \end{aligned}$$

两边开方则得式 (3.3.6).

平行四边形定律可直接计算得出, 可作为习题由读者完成. 证毕.

在 n 维空间中两个向量正交的定义也可推广到内积空间中.

定义 3.7 若 $f(x), g(x) \in C[a, b]$ 满足

$$(f, g) = \int_a^b \rho(x) f(x) g(x) dx = 0, \quad (3.3.7)$$

则称 f 与 g 在 $[a, b]$ 上带权 $\rho(x)$ 正交. 若函数族 $\varphi_0(x), \varphi_1(x), \dots, \varphi_n(x), \dots$ 满足关系

$$(\varphi_j, \varphi_k) = \int_a^b \rho(x) \varphi_j(x) \varphi_k(x) dx = \begin{cases} 0, & j \neq k, \\ A_k > 0, & j = k, \end{cases} \quad (3.3.8)$$

原书 PDF 页 67; 印刷页码 54。

[查看原页](#)

则称 $\{\varphi_k\}$ 是 $[a, b]$ 上带权 $\rho(x)$ 的正交函数族; 若 $A_k \equiv 1$, 则称 $\{\varphi_k\}$ 为标准正交函数族. 例如, 三角函数族

$$1, \cos x, \sin x, \cos 2x, \sin 2x, \dots$$

就是在区间 $[-\pi, \pi]$ 上的正交函数族 (权 $\rho(x) \equiv 1$), 其

$$(1, 1) = 2\pi,$$

$$(\sin nx, \sin mx) = \int_{-\pi}^{\pi} \sin nx \sin mx dx = \begin{cases} \pi, & m = n, \\ 0, & m \neq n \end{cases} (n, m = 1, 2, \dots),$$

$$(\cos nx, \cos mx) = \int_{-\pi}^{\pi} \cos nx \cos mx dx = \begin{cases} \pi, & m = n, \\ 0, & m \neq n \end{cases} (n, m = 1, 2, \dots),$$

$$(\cos nx, \sin mx) = \int_{-\pi}^{\pi} \cos nx \sin mx dx = 0 (n, m = 1, 2, \dots).$$

在 $\mathbf{R}^n$ 空间中, 任一向量都可用它的一组线性无关的基表示, 内积空间的任一元素 $f(x) \in C[a, b]$ 也同样可用线性无关的基表示, 此时相应地有如下定义.

定义 3.8 设 $\varphi_0(x), \varphi_1(x), \dots, \varphi_{n-1}(x)$ 在 $[a, b]$ 上连续, 如果

$$a_0 \varphi_0(x) + a_1 \varphi_1(x) + \dots + a_{n-1} \varphi_{n-1}(x) = 0$$

当且仅当 $a_0 = a_1 = \dots = a_{n-1} = 0$ 时成立, 则称 $\varphi_0, \varphi_1, \dots, \varphi_{n-1}$ 在 $[a, b]$ 上是线性无关的.

若函数族 $\{\varphi_k\} (k = 0, 1, \dots)$ 中的任何有限个 φ_k 线性无关, 则称 $\{\varphi_k\}$ 为线性无关函数族.

例如, $1, x, x^2, \dots, x^n, \dots$ 就是 $[a, b]$ 上的线性无关函数族. 若 $\varphi_0(x), \varphi_1(x), \dots, \varphi_{n-1}(x)$ 是 $[a, b]$ 上的线性无关函数, 且 $a_0, a_1, \dots, a_{n-1}$ 是任意实数, 则

$$S(x) = a_0 \varphi_0(x) + a_1 \varphi_1(x) + \dots + a_{n-1} \varphi_{n-1}(x)$$

的全体是 $C[a, b]$ 中的一个子集, 记作

$$\Phi = \text{span}\{\varphi_0, \varphi_1, \dots, \varphi_{n-1}\}. \quad (3.3.9)$$

下面给出判断函数族 $\{\varphi_k\} (k = 0, 1, \dots, n-1)$ 线性无关的充要条件.

定理 3.6 $\varphi_0(x), \varphi_1(x), \dots, \varphi_{n-1}(x)$ 在 $[a, b]$ 上线性无关的充分必要条件是它的 Cramer 行列式 $G_{n-1} \neq 0$, 其中

$$G_{n-1} = G(\varphi_0, \varphi_1, \dots, \varphi_{n-1}) = \begin{vmatrix} (\varphi_0, \varphi_0) & (\varphi_0, \varphi_1) & \cdots & (\varphi_0, \varphi_{n-1}) \\ (\varphi_1, \varphi_0) & (\varphi_1, \varphi_1) & \cdots & (\varphi_1, \varphi_{n-1}) \\ \vdots & \vdots & & \vdots \\ (\varphi_{n-1}, \varphi_0) & (\varphi_{n-1}, \varphi_1) & \cdots & (\varphi_{n-1}, \varphi_{n-1}) \end{vmatrix}. \quad (3.3.10)$$

定理的证明由读者自行完成.

3.3.2 函数的最佳平方逼近

下面研究在区间 $[a, b]$ 上一般的最佳平方逼近问题. 对 $f(x) \in C[a, b]$ 及 $C[a, b]$

校注 (agent 补充, 原书术语疑误): 原页定理 3.6 写作“Cramer 行列式”, 按原文保留. 式 (3.3.10) 是由内积组成的 Gram 行列式, 通常称“Gram (格拉姆) 行列式”。

原书 PDF 页 68; 印刷页码 55。

[查看原页](#)

中的一个子集 $\Phi = \text{span}\{\varphi_0, \varphi_1, \dots, \varphi_n\}$, 若存在 $S^*(x) \in \Phi$, 使

$$\|f - S^*\|_2^2 = \inf_{S \in \varphi} \|f - S\|_2^2 = \inf_{S \in \varphi} \int_a^b \rho(x)[f(x) - S(x)]^2 dx, \quad (3.3.11)$$

则称 $S^*(x)$ 是 $f(x)$ 在子集 $\Phi \subseteq C[a, b]$ 中的最佳平方逼近函数. 为了求 $S^*(x)$, 由式 (3.3.11) 可知, 该问题等价于求多元函数

$$I(a_0, a_1, \dots, a_n) = \int_a^b \rho(x) \left[\sum_{j=0}^n a_j \varphi_j(x) - f(x) \right]^2 dx \quad (3.3.12)$$

的最小值. 由于 $I(a_0, a_1, \dots, a_n)$ 是关于 $a_0, a_1, \dots, a_n$ 的二次函数, 利用多元函数极值的必要条件

$$\begin{aligned} \frac{\partial I}{\partial a_k} &= 0 \quad (k = 0, 1, \dots, n), \\ \frac{\partial I}{\partial a_k} &= 2 \int_a^b \rho(x) \left[\sum_{j=0}^n a_j \varphi_j(x) - f(x) \right] \varphi_k(x) dx = 0 \quad (k = 0, 1, \dots, n), \end{aligned}$$

于是有

$$\sum_{j=0}^n (\varphi_k, \varphi_j) a_j = (f, \varphi_k) \quad (k = 0, 1, \dots, n). \quad (3.3.13)$$

这是关于 $a_0, a_1, \dots, a_n$ 的线性方程组, 称为法方程. 由于 $\varphi_0, \varphi_1, \dots, \varphi_n$ 线性无关, 故系数行列式 $G(\varphi_0, \varphi_1, \dots, \varphi_n) \neq 0$, 于是方程组 (3.3.13) 有唯一解 $a_k = a_k^* (k = 0, 1, \dots, n)$, 从而得到

$$S^*(x) = a_0^* \varphi_0(x) + \dots + a_n^* \varphi_n(x).$$

下面证明 $S^*(x)$ 满足式 (3.3.11), 即对任何 $S(x) \in \Phi$, 有

$$\int_a^b \rho(x)[f(x) - S^*(x)]^2 dx \leq \int_a^b \rho(x)[f(x) - S(x)]^2 dx. \quad (3.3.14)$$

为此只要考虑

$$\begin{aligned} D &= \int_a^b \rho(x)[f(x) - S(x)]^2 dx - \int_a^b \rho(x)[f(x) - S^*(x)]^2 dx \\ &= \int_a^b \rho(x)[S(x) - S^*(x)]^2 dx + 2 \int_a^b \rho(x)[S^*(x) - S(x)][f(x) - S^*(x)] dx. \end{aligned}$$

由于 $S^*(x)$ 的系数 a_k^* 是方程 (3.3.13) 的解, 故

$$\int_a^b \rho(x)[f(x) - S^*(x)] \varphi_k(x) dx = 0 \quad (k = 0, 1, \dots, n),$$

从而上式第二个积分为零, 于是

$$D = \int_a^b \rho(x)[S(x) - S^*(x)]^2 dx \geq 0,$$

故式 (3.3.14) 成立. 这就证明了 $S^*(x)$ 是 $f(x)$ 在 Φ 中的最佳平方逼近函数. 若令 $\delta = f(x) - S^*(x)$, 则平方误差为

$$\|\delta\|_2^2 = (f - S^*, f - S^*) = (f, f) - (S^*, f) = \|f\|_2^2 - \sum_{k=0}^n a_k^* (\varphi_k, f). \quad (3.3.15)$$

校注 (agent 补充, 原书符号不一致): 式 (3.3.11) 两处下确界下标原印 $S \in \varphi$ (小写), 本页前文子集定义为 Φ (大写). 此处按原图保留小写, 所指应为前文的 Φ .

原书 PDF 页 69; 印刷页码 56。

[查看原页](#)

取 $\varphi_k(x) = x^k, \rho(x) \equiv 1, f(x) \in C[0, 1]$, 即要在 H_n 中求 n 次最佳平方逼近多项式

$$S^*(x) = a_0^* + a_1^*x + \cdots + a_n^*x^n,$$

此时

$$(\varphi_j, \varphi_k) = \int_0^1 x^{k+j} dx = \frac{1}{k+j+1},$$

$$(f, \varphi_k) = \int_0^1 f(x)x^k dx \equiv d_k.$$

若用 H 表示行列式 $G_n = G(1, x, x^2, \cdots, x^n)$ 对应的矩阵, 则

$$H = \begin{pmatrix} 1 & 1/2 & \cdots & 1/(n+1) \\ 1/2 & 1/3 & \cdots & 1/(n+2) \\ \vdots & \vdots & & \vdots \\ 1/(n+1) & 1/(n+2) & \cdots & 1/(2n+1) \end{pmatrix}, \quad (3.3.16)$$

H 称为 Hilbert 矩阵. 记

$$a = (a_0, a_1, \cdots, a_n)^T, \quad d = (d_0, d_1, \cdots, d_n)^T,$$

其中

$$d_k = (f, x^k) (k = 0, 1, \cdots, n), \quad (3.3.17)$$

则方程

$$Ha = d$$

的解 $a_k = a_k^* (k = 0, 1, \cdots, n)$ 即为所求.

例 3.2 设 $f(x) = \sqrt{1+x^2}$, 求 $[0, 1]$ 上的一次最佳平方逼近多项式.

解利用式 (3.3.17), 得

$$d_0 = \int_0^1 \sqrt{1+x^2} dx = \frac{1}{2} \ln(1+\sqrt{2}) + \frac{\sqrt{2}}{2} \approx 1.147,$$

$$d_1 = \int_0^1 x\sqrt{1+x^2} dx = \frac{1}{3}(1+x^2)^{\frac{3}{2}} \Big|_0^1 = \frac{2\sqrt{2}-1}{3} \approx 0.609,$$

由方程组

$$\begin{bmatrix} 1 & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{3} \end{bmatrix} \begin{pmatrix} a_0 \\ a_1 \end{pmatrix} = \begin{pmatrix} 1.147 \\ 0.609 \end{pmatrix},$$

解出

$$a_0 = 0.934, \quad a_1 = 0.426, \quad S_1^*(x) = 0.934 + 0.426x.$$

平方误差

$$\|\delta\|_2^2 = (f, f) - (S_1^*, f)$$

$$= \int_0^1 (1+x^2)dx - 0.426d_1 - 0.934d_0 = 0.0026,$$

最大误差

$$\|\delta\|_\infty = \max_{0 \leq x \leq 1} |\sqrt{1+x^2} - S_1^*(x)| \approx 0.066.$$

用 $\{1, x, x^2, \dots, x^n\}$ 作基求最佳平方逼近多项式, 当 n 较大时, 系数矩阵式 (3.3.16) 是高度病态的 (病态矩阵概念见第 7 章), 求法方程 (3.3.13) 的解时, 舍入误差很大, 这时, 用正交多项式作基才能求得最小平方逼近多项式 (见 3.5 节).

校注 (agent 补充, 数值舍入): 原书例 3.2 的系数与平方误差 0.0026 均按原文保留。该数值来自把 d_0, d_1 分别近似为 1.147, 0.609 后代入简化式; 对于印出的函数 $S_1^*(x) = 0.934 + 0.426x$, 直接计算 $\int_0^1 [\sqrt{1+x^2} - S_1^*(x)]^2 dx \approx 0.0007136324$, 与 0.0026 不同。舍入后的系数不再严格满足精确内积对应的法方程, 不能把简化式的数值视为实际平方误差。

原书 PDF 页 70; 印刷页码 57。

[查看原页](#)

3.4 正交多项式

3.4.1 正交化手续

定义 3.9 设 $g_n(x)$ 是首项系数 $a_n \neq 0$ 的 n 次多项式, 如果多项式序列 $g_0(x), g_1(x), \dots$ 满足

$$(g_j, g_k) = \int_a^b \rho(x) g_j(x) g_k(x) dx = \begin{cases} 0, & j \neq k, \\ A_k > 0, & j = k \end{cases} \quad (j, k = 0, 1, \dots),$$

则称多项式序列 $g_0(x), g_1(x), \dots$ 在 $[a, b]$ 上带权 $\rho(x)$ 正交, 并称 $g_n(x)$ 是 $[a, b]$ 上带权 $\rho(x)$ 的 n 次正交多项式.

一般来说, 当权函数 $\rho(x)$ 及区间 $[a, b]$ 给定以后, 可以由线性无关的一组基 $\{1, x, x^2, \dots, x^n, \dots\}$ 并利用正交化方法构造出正交多项式

$$g_0(x) = 1, \quad g_n(x) = x^n - \sum_{k=0}^{n-1} \frac{(x^n, g_k)}{(g_k, g_k)} \cdot g_k(x), \quad k = 1, 2, \dots.$$

这样构造的正交多项式有以下性质: (1) $g_n(x)$ 是最高项系数为 1 的 n 次多项式. (2) 任一 n 次多项式 $P_n(x) \in H_n$ 均可表示为 $g_0(x), g_1(x), \dots, g_n(x)$ 的线性组合. (3) 当 $n \neq m$ 时, $(g_n, g_m) = 0$ 且 $g_n(x)$ 与任一次数小于 n 的多项式正交. (4) 有递推关系

$$g_{n+1}(x) = (x - \alpha_n)g_n(x) - \beta_n g_{n-1}(x), \quad n = 0, 1, \dots,$$

其中

$$\begin{aligned} g_0(x) &= 1, \quad g_{n-1}(x) = 0, \\ \alpha_n &= \frac{(xg_n, g_n)}{(g_n, g_n)}, \quad n = 0, 1, \dots, \\ \beta_n &= \frac{(g_n, g_n)}{(g_{n-1}, g_{n-1})}, \quad n = 1, 2, \dots. \end{aligned}$$

这里 $(xg_n, g_n) = \int_a^b xg_n^2(x)\rho(x)dx$. (5) 设 $g_0(x), g_1(x), \dots$ 是在 $[a, b]$ 上带权 $\rho(x)$ 的正交多项式序列, 则 $g_n(x)$ ($n \geq 1$) 的 n 个根都是单重实根, 且都在区间 (a, b) 内. 以上性质的证明可参看相关文献. 下面主要给出几类常见的而又十分重要的正交多项式.

3.4.2 Legendre 多项式

当区间为 $[-1, 1]$ 、权函数 $\rho(x) \equiv 1$ 时, 由 $\{1, x, x^2, \dots, x^n, \dots\}$ 正交化得到的多

校注 (agent 补充, 原书索引疑误): 本页正交化构造式末尾原印 “ $k = 1, 2, \dots$ ”, 而 k 已是求和哑指标, 外部序号应为 $n = 1, 2, \dots$; 转录保留原式。

校注 (agent 补充, 原书初值疑误): 递推初值原印 $g_{n-1}(x) = 0$, 按原文保留。若对一般 n 成立, 会与 $g_0(x) = 1$ 冲突; 递推通常应以 $g_{-1}(x) = 0$ 开始。

原书 PDF 页 71; 印刷页码 58。

[查看原页](#)

项式就称为 Legendre 多项式, 并用 $P_0(x), P_1(x), \dots, P_n(x), \dots$ 表示. 这是 Legendre 于 1785 年引进的. 1814 年 Rodrigul 给出了简单的表达式

$$P_0(x) = 1, \quad P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} \{(x^2 - 1)^n\} \quad (n = 1, 2, \dots). \quad (3.4.1)$$

由于 $(x^2 - 1)^n$ 是 $2n$ 次多项式, 求 n 阶导数后得

$$P_n(x) = \frac{1}{2^n n!} (2n)(2n-1) \cdots (n+1)x^n + a_{n-1}x^{n-1} + \cdots + a_0,$$

于是得首项 x^n 的系数 $a_n = \frac{(2n)!}{2^n (n!)^2}$. 显然最高项系数为 1 的 Legendre 多项式为

$$\tilde{P}_n(x) = \frac{n!}{(2n)!} \frac{d^n}{dx^n} [(x^2 - 1)^n]. \quad (3.4.2)$$

Legendre 多项式有以下几个重要性质.

性质 1 正交性

$$\int_{-1}^1 P_n(x) P_m(x) dx = \begin{cases} 0, & m \neq n, \\ \frac{2}{2n+1}, & m = n. \end{cases} \quad (3.4.3)$$

证明令 $\varphi(x) = (x^2 - 1)^n$, 则

$$\varphi^{(k)}(\pm 1) = 0 \quad (k = 0, 1, \dots, n-1).$$

设 $Q(x)$ 是区间 $[-1, 1]$ 上 n 阶连续可微的函数, 由分部积分知

$$\begin{aligned} \int_{-1}^1 P_n(x) Q(x) dx &= \frac{1}{2^n n!} \int_{-1}^1 Q(x) \varphi^{(n)}(x) dx = -\frac{1}{2^n n!} \int_{-1}^1 Q'(x) \varphi^{(n-1)}(x) dx \\ &= \cdots = \frac{(-1)^n}{2^n n!} \int_{-1}^1 Q^{(n)}(x) \varphi(x) dx. \end{aligned}$$

下面分两种情况讨论.

(1) 若 $Q(x)$ 是次数小于 n 的多项式, 则 $Q^{(n)}(x) \equiv 0$, 故当 $m \neq n$ 时, 有

$$\int_{-1}^1 P_m(x) P_n(x) dx = 0.$$

(2) 若

$$Q(x) = P_n(x) = \frac{1}{2^n n!} \varphi^{(n)}(x) = \frac{(2n)!}{2^n (n!)^2} x^n + \cdots,$$

$$Q^{(n)}(x) = P_n^{(n)}(x) = \frac{(2n)!}{2^n n!},$$

则

$$\int_{-1}^1 P_n^2(x) dx = \frac{(-1)^n (2n)!}{2^{2n} (n!)^2} \int_{-1}^1 (x^2 - 1)^n dx = \frac{(2n)!}{2^{2n} (n!)^2} \int_{-1}^1 (1 - x^2)^n dx.$$

由于

$$\int_0^1 (1 - x^2)^n dx = \int_0^{\frac{\pi}{2}} \cos^{2n+1} t dt = \frac{2 \cdot 4 \cdots 2n}{1 \cdot 3 \cdots (2n+1)},$$

故当 $m = n$ 时, 有

$$\int_{-1}^1 P_n^2(x) dx = \frac{2}{2n+1}.$$

证毕.

校注 (agent 补充, 原书系数疑误): 由 $(x^2 - 1)^n$ 求导后写出的 $P_n(x)$ 首项系数中, 原分母明确印为 $2^2 n!$, 此处按原文保留。依据式 (3.4.1), 该处应为 $2^n n!$; 这也与紧随其后的 $a_n = (2n)!/[2^n (n!)^2]$ 一致。

原书 PDF 页 72; 印刷页码 59。

[查看原页](#)

性质 2 奇偶性

$$P_n(-x) = (-1)^n P_n(x). \quad (3.4.4)$$

由于 $\varphi(x) = (x^2 - 1)^n$ 是偶次多项式, 经过偶次求导仍为偶次多项式, 经过奇次求导则为奇次多项式, 故 n 为偶数时 $P_n(x)$ 为偶函数, n 为奇数时 $P_n(x)$ 为奇函数, 于是式 (3.4.4) 成立.

性质 3 递推关系考虑 $n+1$ 次多项式 $xP_n(x)$, 它可表示为

$$xP_n(x) = a_0P_0(x) + a_1P_1(x) + \cdots + a_{n+1}P_{n+1}(x),$$

两边乘以 $P_k(x)$, 并从 -1 到 1 积分, 得

$$\int_{-1}^1 xP_n(x)P_k(x)dx = a_k \int_{-1}^1 P_k^2(x)dx.$$

当 $k \leq n-2$ 时, $xP_k(x)$ 次数不大于 $n-1$, 上式左端积分为零, 故得 $a_k = 0$. 当 $k = n$ 时 $xP_n^2(x)$ 为奇函数, 左端积分仍为零, 故 $a_n = 0$. 于是

$$xP_n(x) = a_{n-1}P_{n-1}(x) + a_{n+1}P_{n+1}(x),$$

其中

$$a_{n-1} = \frac{2n-1}{2} \int_{-1}^1 xP_n(x)P_{n-1}(x)dx = \frac{2n-1}{2} \frac{2n}{4n^2-1} = \frac{n}{2n+1},$$

$$a_{n+1} = \frac{2n+3}{2} \int_{-1}^1 xP_n(x)P_{n+1}(x)dx = \frac{2n+3}{2} \frac{2(n+1)}{(2n+1)(2n+3)} = \frac{n+1}{2n+1},$$

从而得到以下的递推公式:

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x) \quad (n = 1, 2, \cdots). \quad (3.4.5)$$

由 $P_0(x) = 1$, $P_1(x) = x$, 利用式 (3.4.5) 就可推出

$$P_2(x) = (3x^2 - 1)/2,$$

$$P_3(x) = (5x^3 - 3x)/2,$$

$$P_4(x) = (35x^4 - 30x^2 + 3)/8,$$

$$P_5(x) = (63x^5 - 70x^3 + 15x)/8,$$

$$P_6(x) = (231x^6 - 315x^4 + 105x^2 - 5)/16,$$

⋮

图 3.4 给出了 $P_0(x), P_1(x), P_2(x), P_3(x)$ 的图形.

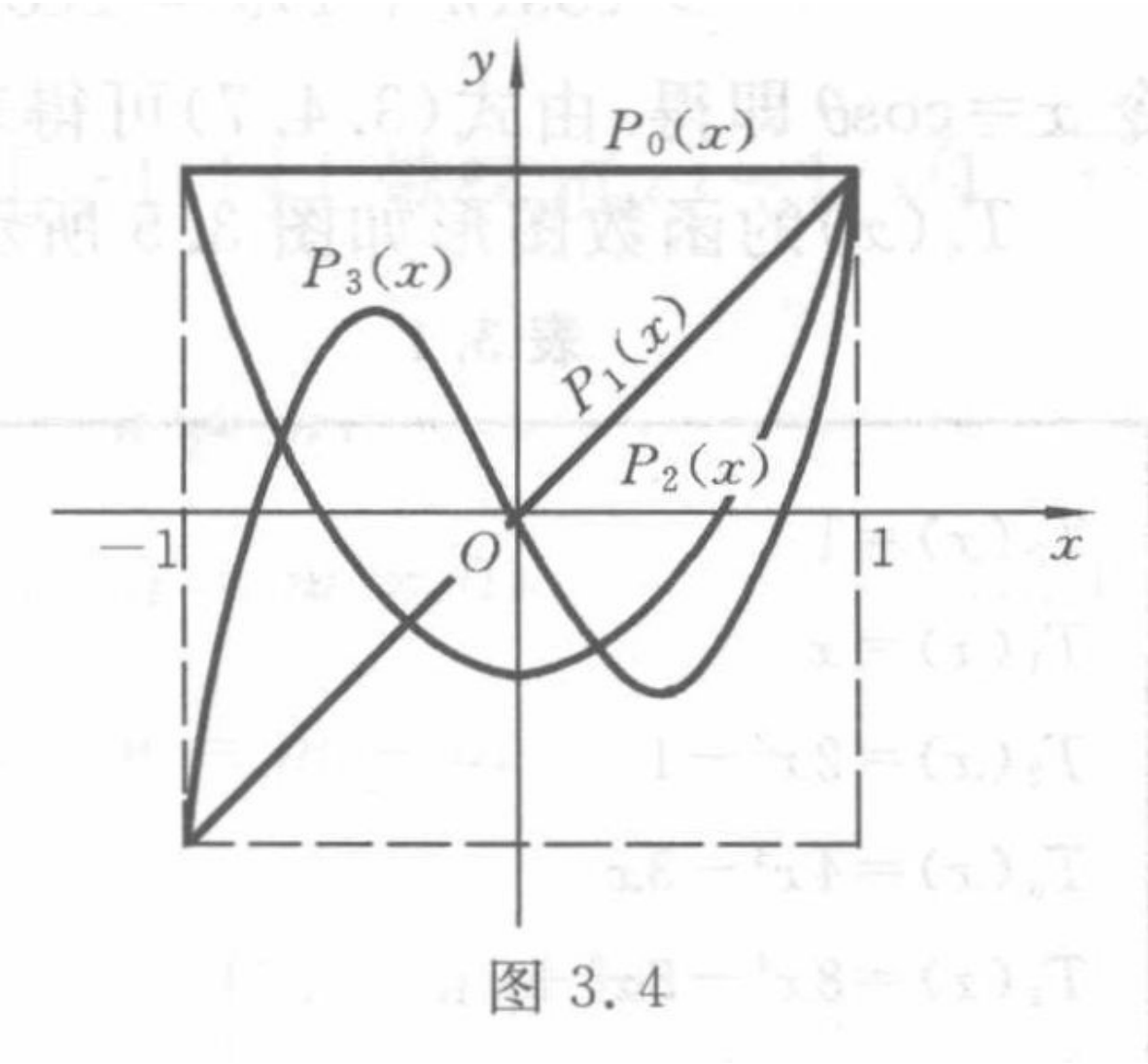


图 3.4

图 3.4 (原图)。Legendre 多项式 $P_0(x)$ 、 $P_1(x)$ 、 $P_2(x)$ 、 $P_3(x)$ 的图形。全部标签： x 、 y 、 O 、 -1 、 1 、 $P_0(x)$ 、 $P_1(x)$ 、 $P_2(x)$ 、 $P_3(x)$ ；图中包含区间端点处的竖向及下边界虚线。

性质 4 在所有最高项系数为 1 的 n 次多项式中，Legendre 多项式 $\tilde{P}_n(x)$ 在 $[-1, 1]$ 上与零的平方误差最小。

设 $Q_n(x)$ 是任意一个最高项系数为 1 的 n 次多项式，它可表示为

原书 PDF 页 73; 印刷页码 60。

[查看原页](#)

$$Q_n(x) = \tilde{P}_n(x) + \sum_{k=0}^{n-1} a_k \tilde{P}_k(x),$$

于是

$$(Q_n, Q_n) = \int_{-1}^1 Q_n^2(x) dx = (\tilde{P}_n, \tilde{P}_n) + \sum_{k=0}^{n-1} a_k^2 (\tilde{P}_k, \tilde{P}_k) \geq (\tilde{P}_n, \tilde{P}_n)$$

当且仅当 $a_0 = a_1 = \cdots = a_{n-1} = 0$ 时等号才成立, 即当 $Q_n(x) \equiv \tilde{P}_n(x)$ 时平方误差最小.

性质 5 $P_n(x)$ 在区间 $(-1, 1)$ 内有 n 个不同的实零点.

3.4.3 Chebyshev 多项式

当权函数 $\rho(x) = \frac{1}{\sqrt{1-x^2}}$, 区间为 $[-1, 1]$ 时, 由序列 $\{1, x, x^2, \cdots, x^n, \cdots\}$ 正交化得到的正交多项式就是 Chebyshev 多项式, 它可表示为

$$T_n(x) = \cos(n \arccos x), \quad |x| \leq 1. \quad (3.4.6)$$

若令 $x = \cos \theta$, 则

$$T_n(x) = \cos n\theta, \quad 0 \leq \theta \leq \pi.$$

Chebyshev 多项式有很多重要性质.

性质 1 递推关系

$$\begin{aligned} T_0(x) &= 1, \quad T_1(x) = x, \\ T_{n+1}(x) &= 2xT_n(x) - T_{n-1}(x) \quad (n = 1, 2, \cdots). \end{aligned} \quad (3.4.7)$$

这只要由三角恒等式

$$\cos(n+1)\theta = 2\cos\theta\cos n\theta - \cos(n-1)\theta \quad (n \geq 1)$$

令 $x = \cos \theta$ 即得. 由式 (3.4.7) 可得表 3.1.

$T_n(x)$ 的函数图形如图 3.5 所示.

表 3.1

多项式	表达式
$T_0(x)$	1
$T_1(x)$	x
$T_2(x)$	$2x^2 - 1$
$T_3(x)$	$4x^3 - 3x$
$T_4(x)$	$8x^4 - 8x^2 + 1$
$T_5(x)$	$16x^5 - 20x^3 + 5x$
$T_6(x)$	$32x^6 - 48x^4 + 18x^2 - 1$
$T_7(x)$	$64x^7 - 112x^5 + 56x^3 - 7x$
$T_8(x)$	$128x^8 - 256x^6 + 160x^4 - 32x^2 + 1$

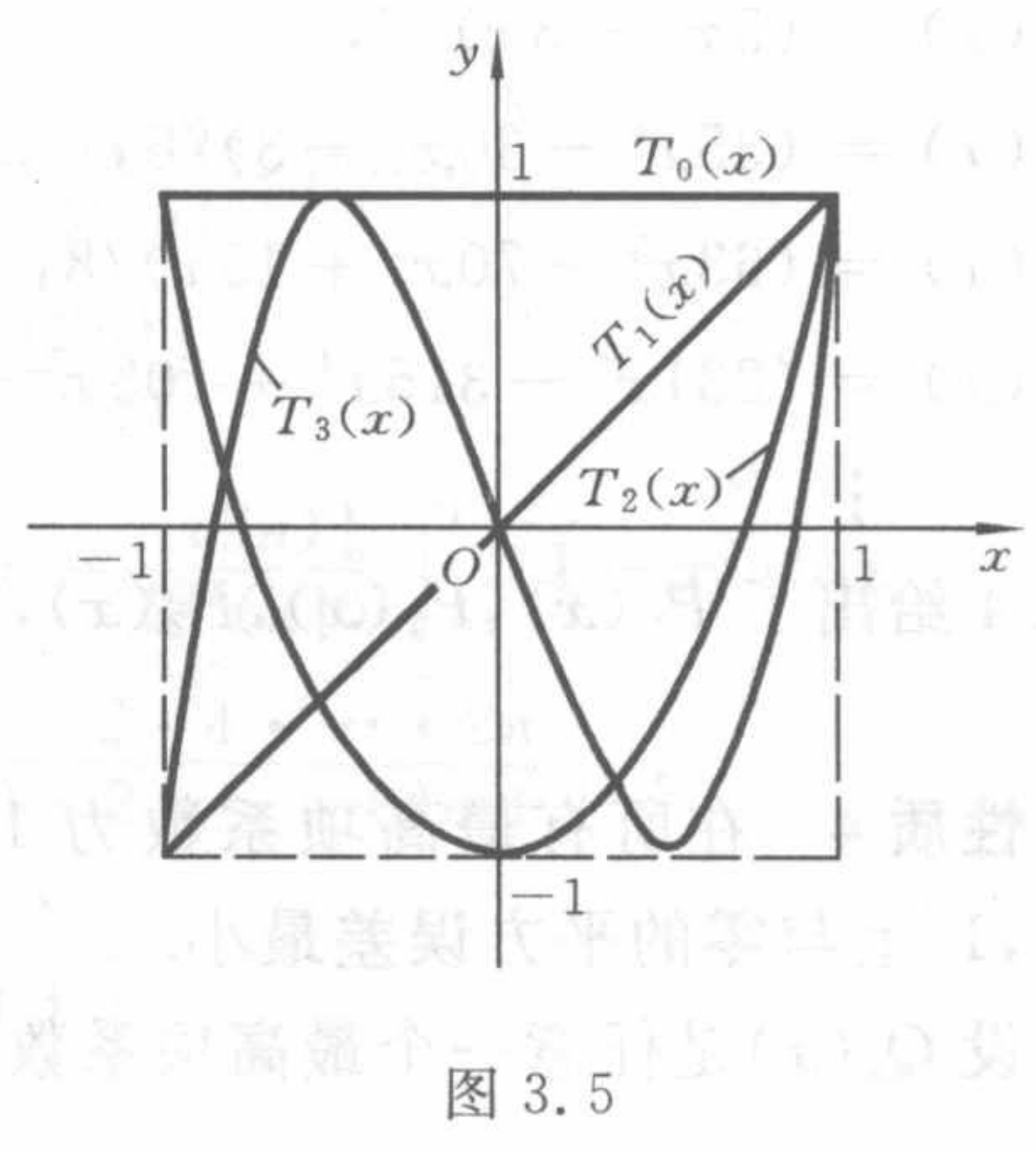


图 3.5 (原图)。Chebyshev 多项式 $T_0(x)$ 、 $T_1(x)$ 、 $T_2(x)$ 、 $T_3(x)$ 的图形。全部标签： x 、 y 、 O 、横轴的 -1 、 1 、纵轴的 1 、 -1 、 $T_0(x)$ 、 $T_1(x)$ 、 $T_2(x)$ 、 $T_3(x)$ ；在 $x = \pm 1$ 处画有辅助线。

原书 PDF 页 74; 印刷页码 61。

[查看原页](#)

由递推关系式 (3.4.7) 还可得到 $T_n(x)$ 的最高项系数是 $2^{n-1} (n \geq 1)$.

性质 2 $T_n(x)$ 对零的偏差最小. 它可写成如下定理.

定理 3.7 在区间 $[-1, 1]$ 上所有最高项系数为 1 的一切 n 次多项式中, $\omega_n(x) = \frac{1}{2^{n-1}} T_n(x)$ 与零的偏差最小, 其偏差为 $\frac{1}{2^{n-1}}$.

证明由于 $\omega_n(x) = \frac{1}{2^{n-1}} T_n(x) = x^n - P_{n-1}^*(x)$,

$$\max_{-1 \leq x \leq 1} |\omega_n(x)| = \frac{1}{2^{n-1}} \max_{-1 \leq x \leq 1} |T_n(x)| = \frac{1}{2^{n-1}},$$

且点 $x_k = \cos \frac{k}{n} \pi (k = 0, 1, \dots, n)$ 是 $T_n(x)$ 的 Chebyshev 交错点组, 由定理 3.4 可知, 在区间 $[-1, 1]$ 上, x^n 在 H_{n-1} 中最佳逼近多项式为 $P_{n-1}^*(x)$, 即 $\omega_n(x)$ 是与零的偏差最小的多项式. 证毕.

例 3.3 求 $f(x) = 2x^3 + x^2 + 2x - 1$ 在 $[-1, 1]$ 上的最佳二次逼近多项式.

解由题意, 所求最佳逼近多项式 $P_2^*(x)$ 应满足

$$\max_{-1 \leq x \leq 1} |f(x) - P_2^*(x)| = \min.$$

由定理 3.6 可知, 当

$$f(x) - P_2^*(x) = \frac{1}{2} T_3(x) = 2x^3 - \frac{3}{2}x$$

时, 与零偏差最小, 故

$$P_2^*(x) = f(x) - \frac{1}{2} T_3(x) = x^2 + \frac{7}{2}x - 1$$

就是 $f(x)$ 在 $[-1, 1]$ 上的最佳二次逼近多项式.

性质 3 Chebyshev 多项式 $\{T_k(x)\}$ 在区间 $[-1, 1]$ 上带权 $\rho(x) = 1/\sqrt{1-x^2}$ 正交, 且

$$\int_{-1}^1 \frac{T_n(x) T_m(x) dx}{\sqrt{1-x^2}} = \begin{cases} 0, & n \neq m, \\ \frac{\pi}{2}, & n = m \neq 0, \\ \pi, & n = m = 0. \end{cases} \quad (3.4.8)$$

事实上, 令 $x = \cos \theta$, 则 $dx = -\sin \theta d\theta$, 于是

$$\int_{-1}^1 \frac{T_n(x) T_m(x) dx}{\sqrt{1-x^2}} = \int_0^\pi \cos n\theta \cos m\theta d\theta = \begin{cases} 0, & n \neq m, \\ \frac{\pi}{2}, & n = m \neq 0, \\ \pi, & n = m = 0. \end{cases}$$

性质 4 $T_{2k}(x)$ 只含 x 的偶次幂, $T_{2k+1}(x)$ 只含 x 的奇次幂.

此性质由递推关系直接得到.

校注 (agent 补充, 原书交叉引用疑误): 例 3.3 中原文写“由定理 3.6 可知”, 照录。此处使用的是本页定理 3.7 的最小偏差性质; 定理 3.6 是线性无关与内积行列式的判别条件。

原书 PDF 页 75; 印刷页码 62。

[查看原页](#)

性质 5 $T_n(x)$ 在区间 $[-1, 1]$ 上有 n 个零点 $x_k = \cos \frac{2k-1}{2n}\pi, k = 1, \dots, n$. 此外, 实际计算中时常要求 x^n 用 $T_0, T_1, \dots, T_n$ 的线性组合表示, 其公式为

$$x^n = 2^{1-n} \sum_{k=0}^{\lfloor \frac{n}{2} \rfloor} \binom{n}{k} T_{n-2k}(x). \quad (3.4.9)$$

这里规定 $T_0 = 1/2$. $n = 1 \sim 8$ 的结果如表 3.2 所示.

表 3.2

幂函数	Chebyshev 多项式表示
1	T_0
x	T_1
x^2	$\frac{1}{2}(T_0 + T_2)$
x^3	$\frac{1}{4}(3T_1 + T_3)$
x^4	$\frac{1}{8}(3T_0 + 4T_2 + T_4)$
x^5	$\frac{1}{16}(10T_1 + 5T_3 + T_5)$
x^6	$\frac{1}{32}(10T_0 + 15T_2 + 6T_4 + T_6)$
x^7	$\frac{1}{64}(35T_1 + 21T_3 + 7T_5 + T_7)$
x^8	$\frac{1}{128}(35T_0 + 56T_2 + 28T_4 + 8T_6 + T_8)$

3.4.4 其他常用的正交多项式

一般地说, 如果区间 $[a, b]$ 及权函数 $\rho(x)$ 不同, 则得到的正交多项式也不同. 除上述两种最重要的正交多项式外, 下面再给出三种较常用的正交多项式.

1. 第二类 Chebyshev 多项式 在区间 $[-1, 1]$ 上带权 $\rho(x) = \sqrt{1-x^2}$ 的正交多项式称为第二类 Chebyshev 多项式, 其表达式为

$$U_n(x) = \frac{\sin[(n+1)\arccos x]}{\sqrt{1-x^2}}. \quad (3.4.10)$$

由 $x = \cos \theta$ 可得

$$\begin{aligned} \int_{-1}^1 U_n(x)U_m(x)\sqrt{1-x^2}dx &= \int_0^\pi \sin(n+1)\theta \sin(m+1)\theta d\theta \\ &= \begin{cases} 0, & m \neq n, \\ \frac{\pi}{2}, & m = n, \end{cases} \end{aligned}$$

校注 (agent 补充, 记号约定): 式 (3.4.9) 后原文规定 $T_0 = 1/2$, 而表 3.2 首行原印 $1 = T_0$; 两处均忠实保留. 式 (3.4.9) 的求和使用临时的半权常数项约定, 表 3.2 则与前文通常定义 $T_0 = 1$ 一致. 使用时须区分这两种约定.

原书 PDF 页 76; 印刷页码 63。

[查看原页](#)

即 $\{U_n(x)\}$ 是区间 $[-1, 1]$ 上带权 $\rho(x) = \sqrt{1-x^2}$ 的正交多项式族。还可得到递推关系式

$$U_0(x) = 1, \quad U_1(x) = 2x,$$

$$U_{n+1}(x) = 2xU_n(x) - U_{n-1}(x) \quad (n = 1, 2, \dots).$$

2. Laguerre 多项式 在区间 $[0, \infty)$ 上带权 $\rho(x) = e^{-x}$ 的正交多项式称为 Laguerre 多项式, 其表达式为

$$L_n(x) = e^n \frac{d^n}{dx^n} (x^n e^{-x}). \quad (3.4.11)$$

它也具有正交性质

$$\int_0^\infty e^{-x} L_n(x) L_m(x) dx = \begin{cases} 0, & m \neq n, \\ (n!)^2, & m = n \end{cases}$$

和递推关系

$$L_0(x) = 1, \quad L_1(x) = 1 - x,$$

$$L_{n+1}(x) = (1 + 2n - x)L_n(x) - n^2 L_{n-1}(x) \quad (n = 1, 2, \dots).$$

3. Hermite 多项式 在区间 $(-\infty, \infty)$ 上带权 $\rho(x) = e^{-x^2}$ 的正交多项式称为 Hermite 多项式, 其表达式为

$$H_n(x) = (-1)^n e^{x^2} \frac{d^n}{dx^n} (e^{-x^2}). \quad (3.4.12)$$

它满足正交关系

$$\int_{-\infty}^{+\infty} e^{-x^2} H_m(x) H_n(x) dx = \begin{cases} 0, & m \neq n, \\ 2^n n! \sqrt{\pi}, & m = n, \end{cases}$$

并有递推关系

$$H_0(x) = 1, \quad H_1(x) = 2x,$$

$$H_{n+1}(x) = 2xH_n(x) - 2nH_{n-1}(x) \quad (n = 1, 2, \dots).$$

3.5 函数按正交多项式展开

设 $f(x) \in C[a, b]$, 用正交多项式 $\{g_0(x), g_1(x), \dots, g_n(x)\}$ 作基, 求最佳平方逼近多项式

$$S_n(x) = a_0 g_0(x) + a_1 g_1(x) + \dots + a_n g_n(x). \quad (3.5.1)$$

由 $\{g_k(x)\}$ 的正交性及式 (3.3.13), 可求得系数

$$a_k = \frac{(f, g_k)}{(g_k, g_k)} \quad (k = 0, 1, \dots, n), \quad (3.5.2)$$

于是, $f(x)$ 的最佳平方逼近多项式为

校注 (agent 补充): 式 (3.4.11) 的原扫描明确印为 e^n , 已忠实保留; 这是疑似原书错误, 不是 OCR 误读。取 $n = 0$, 原式给出 $L_0(x) = e^{-x}$, 与紧接着印出的 $L_0(x) = 1$ 矛盾。相容的 Rodrigues 表达式前因子应为 e^x 。见 [原式放大裁切](#)。

原书 PDF 页 77; 印刷页码 64。

[查看原页](#)

$$S_n(x) = \sum_{k=0}^n \frac{(f, g_k)}{(g_k, g_k)} g_k(x). \quad (3.5.3)$$

由式 (3.3.15) 可得均方误差为

$$\|\delta_n\|_2 = \|f - S_n\|_2 = \sqrt{\|f\|_2^2 - \sum_{k=0}^n \frac{(f, g_k)}{(g_k, g_k)} (f, g_k)}. \quad (3.5.4)$$

若 $f(x)$ 在 $[a, b]$ 上按正交多项式 $\{g_k(x)\}$ 展开, 系数 a_k ($k = 0, 1, \dots$) 按式 (3.5.2) 计算, 这样可得到 $f(x)$ 的展开式

$$f(x) \sim \sum_{k=0}^{\infty} a_k g_k(x). \quad (3.5.5)$$

式 (3.5.5) 右端级数称为广义 Fourier 级数, 系数 a_k 称为广义 Fourier 系数, 它是 Fourier 级数的直接推广。从以上讨论可知, 任何 $f(x) \in C[a, b]$ 均可展开成广义 Fourier 级数, 其部分和 $S_n(x)$ 是 $f(x)$ 的最佳平方逼近, 系数 a_k 与 n 无关, 因此, 当 n 增加时, 只要多计算增加的系数 a_k 。级数 (3.5.5) 可能不一致收敛于 $f(x)$, 但在 $f(x)$ 满足一定条件下也可一致收敛到 $f(x)$ 。

下面考虑函数 $f(x) \in C[-1, 1]$ 按 Legendre 多项式 $\{P_0(x), P_1(x), \dots\}$ 展开的最佳平方逼近多项式 $S_n^*(x)$ 。由式 (3.5.1) 和式 (3.5.2) 可得

$$S_n^*(x) = a_0^* P_0(x) + a_1^* P_1(x) + \dots + a_n^* P_n(x), \quad (3.5.6)$$

其中

$$a_k^* = \frac{(f, P_k)}{(P_k, P_k)} = \frac{2k+1}{2} \int_{-1}^1 f(x) P_k(x) dx. \quad (3.5.7)$$

根据式 (3.5.4), 平方误差为

$$\|\delta_k\|_2^2 = \int_{-1}^1 f^2(x) dx - \sum_{k=0}^n \frac{2}{2k+1} (a_k^*)^2. \quad (3.5.8)$$

例 3.4 求 $f(x) = e^x$ 在 $[-1, 1]$ 上的三次最佳平方逼近多项式。

解 先计算 (f, P_k) ($k = 0, 1, 2, 3$), 即

$$(f, P_0) = \int_{-1}^1 e^x dx = e - \frac{1}{e} \approx 2.3504,$$

$$(f, P_1) = \int_{-1}^1 x e^x dx = 2e^{-1} \approx 0.7358,$$

$$(f, P_2) = \int_{-1}^1 \left(\frac{3}{2}x^2 - \frac{1}{2} \right) e^x dx = e - \frac{7}{e} \approx 0.1431,$$

$$(f, P_3) = \int_{-1}^1 \left(\frac{5}{2}x^3 - \frac{3}{2}x \right) e^x dx = 37\frac{1}{e} - 5e \approx 0.02013.$$

由式 (3.5.7) 得

$$a_0^* = (f, P_0)/2 = 1.1752, \quad a_1^* = 3(f, P_1)/2 = 1.1036,$$

$$a_2^* = 5(f, P_2)/2 = 0.3578, \quad a_3^* = 7(f, P_3)/2 = 0.07046,$$

代入式 (3.5.6), 得

校注 (agent 补充): 式 (3.5.8) 左端原书印为 $\|\delta_k\|_2^2$, 已保留。其右端求和上限为 n , 与前式 (3.5.4) 一致时左端应记为 $\|\delta_n\|_2^2$, 属疑似原书下标错误。见 [原式放大裁切](#)。

原书 PDF 页 78; 印刷页码 65。

[查看原页](#)

$$S_3^*(x) = 0.9963 + 0.9979x + 0.5367x^2 + 0.1761x^3.$$

均方误差为

$$\|\delta_n\|_2 = \|e^x - S_3^*(x)\|_2 = \sqrt{\int_{-1}^1 e^{2x} dx - \sum_{k=0}^3 \frac{2}{2k+1} (a_k^*)^2} \leq 0.0084,$$

最大误差为

$$\|\delta_n\|_\infty = \|e^x - S_3(x)\|_\infty \leq 0.0112.$$

如果 $f(x) \in C[a, b]$, 求 $f(x)$ 在区间 $[a, b]$ 上的最佳平方逼近多项式。作变换

$$x = \frac{b-a}{2}t + \frac{b+a}{2} \quad (-1 \leq t \leq 1),$$

于是

$$F(t) = f\left(\frac{b-a}{2}t + \frac{b+a}{2}\right).$$

在 $[-1, 1]$ 上可用 Legendre 多项式作最佳平方逼近多项式 $S_n^*(t)$, 从而得到 $f(x)$ 在区间 $[a, b]$ 上的最佳平方逼近多项式 $S_n^*\left(\frac{1}{b-a}(2x-a-b)\right)$ 。

由于 Legendre 多项式 $\{P_k(x)\}$ 是在区间 $[-1, 1]$ 上由 $\{1, x, x^2, \dots, x^k, \dots\}$ 正交化得到的, 因此利用函数的 Legendre 展开部分和得到最佳平方逼近多项式与由

$$S^*(x) = a_0 + a_1x + \dots + a_nx^n$$

直接通过解法方程得到的 H_n 中的最佳平方逼近多项式是一致的, 只是当 n 较大时求法方程会出现病态方程, 计算误差较大, 不能使用。而用 Legendre 展开不用解线性方程组, 不存在病态问题, 计算公式使用起来也较方便, 因此通常都用此法求最佳平方逼近多项式。

3.6 曲线拟合的最小二乘法

3.6.1 一般的最小二乘逼近

在科学实验的统计方法研究中, 往往要从一组实验数据 (x_i, y_i) ($i = 0, 1, \dots, m$) 中寻找自变量 x 与因变量 y 之间的函数关系 $y = F(x)$ 。由于观测数据往往不准确, 因此不要求 $y = F(x)$ 经过所有点 (x_i, y_i) , 而只要求在给定点 x_i 上误差 $\delta_i = F(x_i) - y_i$ ($i = 0, 1, \dots, m$) 按某种标准最小。若记 $\delta = (\delta_0, \delta_1, \dots, \delta_m)^T$, 就是要求向量 δ 的范数 $\|\delta\|$ 最小。如果用最大范数, 计算上困难较大, 通常就采用 Euclid 范数 $\|\delta\|_2$ 作为误差度量的标准。关于最小二乘法的一般提法是: 对于给定的一组数据 (x_i, y_i) ($i = 0, 1, \dots, m$), 要求在函数空间 $\Phi = \text{span}\{\varphi_0, \varphi_1, \dots, \varphi_n\}$ 中找一个函数 $y = S^*(x)$, 使误差平方和

校注 (agent 补充): 本页最大误差式原书写 $S_3(x)$, 而同例的最佳平方逼近多项式记作 $S_3^*(x)$; 转录保留原来的无星号记法。

原书 PDF 页 79; 印刷页码 66。

[查看原页](#)

$$\|\delta\|_2^2 = \sum_{i=0}^m \delta_i^2 = \sum_{i=0}^m [S^*(x_i) - y_i]^2 = \min_{S(x) \in \varphi} \sum_{i=1}^m [S(x_i) - y_i]^2, \quad (3.6.1)$$

这里

$$S(x) = a_0\varphi_0(x) + a_1\varphi_1(x) + \cdots + a_n\varphi_n(x) \quad (n < m). \quad (3.6.2)$$

这就是一般的最小二乘逼近, 用几何语言说, 就称为曲线拟合的最小二乘法。

用最小二乘法求拟合曲线时, 首先要确定 $S(x)$ 的形式。这不是单纯数学问题, 还与所研究问题的运动规律及所得观测数据 (x_i, y_i) 有关; 通常要从问题的运动规律及给定数据描图来确定 $S(x_i)$ 的形式, 并通过实际计算选出较好的结果——这点将从下面的例题得到说明。 $S(x)$ 的一般表达式为式 (3.6.2) 所示的线性形式。若 $\varphi_k(x)$ 是 k 次多项式, $S(x)$ 就是 n 次多项式。为了使问题的提法更有一般性, 通常把最小二乘法中 $\|\delta\|_2^2$ 都考虑为加权平方和

$$\|\delta\|_2^2 = \sum_{i=0}^m \omega(x_i) [S(x_i) - f(x_i)]^2. \quad (3.6.3)$$

这里 $\omega(x) \geq 0$ 是 $[a, b]$ 上的权函数, 它表示不同点 $(x_i, f(x_i))$ 处的数据比重不同, 例如, $\omega(x_i)$ 可表示在点 $(x_i, f(x_i))$ 处重复观测的次数。用最小二乘法求拟合曲线的问题, 就是在形如式 (3.6.2) 的 $S(x)$ 中求一函数 $y = S^*(x)$, 使式 (3.6.3) 取得最小。它转化为求多元函数

$$I(a_0, a_1, \dots, a_n) = \sum_{i=0}^m \omega(x_i) \left[\sum_{j=0}^n a_j \varphi_j(x_i) - f(x_i) \right]^2 \quad (3.6.4)$$

的极小点 $(a_0^*, a_1^*, \dots, a_n^*)$ 问题。这与 3.3 节讨论的问题完全类似。由求多元函数极值的必要条件, 有

$$\frac{\partial I}{\partial a_k} = 2 \sum_{i=0}^m \omega(x_i) \left[\sum_{j=0}^n a_j \varphi_j(x_i) - f(x_i) \right] \varphi_k(x_i) = 0 \quad (k = 0, 1, \dots, n).$$

若记

$$(\varphi_j, \varphi_k) = \sum_{i=0}^m \omega(x_i) \varphi_j(x_i) \varphi_k(x_i), \quad (3.6.5)$$

则

$$(f, \varphi_k) = \sum_{i=0}^m \omega(x_i) f(x_i) \varphi_k(x_i) \equiv d_k \quad (k = 0, 1, \dots, n),$$

可改写为

$$\sum_{j=0}^n (\varphi_k, \varphi_j) a_j = d_k \quad (k = 0, 1, \dots, n). \quad (3.6.6)$$

此方程称为法方程。它也可写成矩阵形式

$$Ga = d,$$

其中

$$a = (a_0, a_1, \dots, a_n)^T, \quad d = (d_0, d_1, \dots, d_n)^T,$$

$$G = \begin{pmatrix} (\varphi_0, \varphi_0) & (\varphi_0, \varphi_1) & \cdots & (\varphi_0, \varphi_n) \\ (\varphi_1, \varphi_0) & (\varphi_1, \varphi_1) & \cdots & (\varphi_1, \varphi_n) \\ \vdots & \vdots & & \vdots \\ (\varphi_n, \varphi_0) & (\varphi_n, \varphi_1) & \cdots & (\varphi_n, \varphi_n) \end{pmatrix}, \quad (3.6.7)$$

校注 (agent 补充): 式 (3.6.1) 右端原书求和下限明确为 $i = 1$, 其左侧两个求和及本节数据下标均从 $i = 0$ 开始; 原式已保留, 右端下限疑应为 $i = 0$ 。最小值的下标原书印为 $S(x) \in \varphi$ (小写), 而前页函数空间记为 Φ , 此处亦保留原印字形。见 [原式放大裁切](#)。

原书 PDF 页 80; 印刷页码 67。

[查看原页](#)

由于 $\varphi_0, \varphi_1, \dots, \varphi_n$ 线性无关, 故 $|G| \neq 0$, 方程组 (3.6.6) 存在唯一的解

$$a_k = a_k^* \quad (k = 0, 1, \dots, n),$$

从而得到函数 $f(x)$ 的最小二乘解为

$$S^*(x) = a_0^* \varphi_0(x) + a_1^* \varphi_1(x) + \dots + a_n^* \varphi_n(x).$$

可以证明, 这样得到的 $S^*(x)$ 对于任何形如式 (3.6.2) 的 $S(x)$, 都有

$$\sum_{i=0}^m \omega(x_i) [S^*(x_i) - f(x_i)]^2 \leq \sum_{i=0}^m \omega(x_i) [S(x_i) - f(x_i)]^2,$$

故 $S^*(x)$ 确是所求最小二乘解。它的证明与式 (3.3.14) 相似, 读者可自己完成。

例 3.5 已知一组实验数据如表 3.3 所示, 求它的拟合曲线。

表 3.3

x_i	1	2	3	4	5
f_i	4	4.5	6	8	8.5
ω_i	2	1	3	1	1

解 在坐标纸上标出所给数据, 如图 3.6 所示。从图 3.6 可看到, 各点分布在一条直线附近, 故可选择线性函数作拟合曲线。令 $S_1(x) = a_0 + a_1 x$, 这里 $m = 4, n = 1, \varphi_0(x) = 1, \varphi_1(x) = x$, 故

$$(\varphi_0, \varphi_0) = \sum_{i=0}^4 \omega_i = 8, \quad (\varphi_0, \varphi_1) = (\varphi_1, \varphi_0) = \sum_{i=0}^4 \omega_i x_i = 22,$$

$$(\varphi_1, \varphi_1) = \sum_{i=0}^4 \omega_i x_i^2 = 74, \quad (\varphi_0, f) = \sum_{i=0}^4 \omega_i f_i = 47,$$

$$(\varphi_1, f) = \sum_{i=0}^4 \omega_i x_i f_i = 145.5.$$

由式 (3.6.6) 得方程组

$$\begin{cases} 8a_0 + 22a_1 = 47, \\ 22a_0 + 74a_1 = 145.5, \end{cases}$$

解得

$a_0 = 2.77, \quad a_1 = 1.13.$

于是所求拟合曲线为

$S_1^*(x) = 2.77 + 1.13x.$

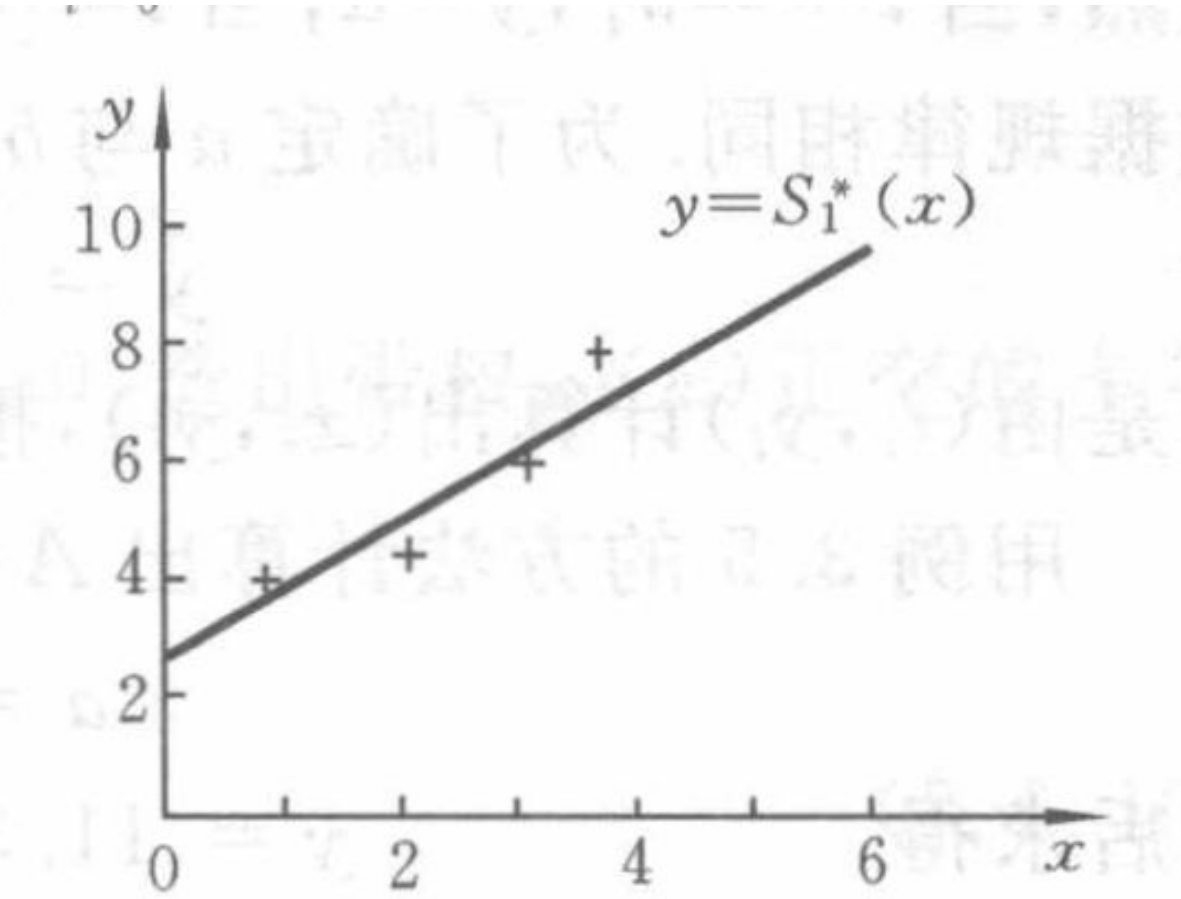


图 3.6

图 3.6 (原图裁切)。图中以十字标记数据点, 并画出拟合直线 $y = S_1^*(x)$ 。图中文字与符号: $x, y, 0$; 横轴标值 2, 4, 6, 纵轴标值 2, 4, 6, 8, 10; 曲线标签 $y = S_1^*(x)$ 。

例 3.6 在某化学反应过程中, 根据实验所得生成物的质量分数与时间的关系如表 3.4 所示, 求质量分数 y 与时间 t 的拟合曲线 $y = F(t)$ 。

表 3.4

t/min	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$y/\times 10^{-3}$	4.00	6.40	8.00	8.80	9.22	9.50	9.70	9.86	10.00	10.20	10.32	10.42	10.50	10.55	10.58	10.60

解将所给数据标在坐标纸上,如图 3.7 所示。可以看到,质量分数开始时增加较快,后来逐渐减慢,到一定时间就基本稳定在一个水平上,即当 $t \rightarrow \infty$ 时, y 趋于某

校注 (agent 补充): 最小二乘不等式右端的 $\omega(x_i)$ 后原书印有上标星号,原式已保留。该星号在权函数定义中没有定义,疑为多余印字;与式 (3.6.3) 的一致写法应为 $\omega(x_i)[S(x_i) - f(x_i)]^2$ 。见 [原式放大裁切](#)。

原书 PDF 页 81; 印刷页码 68。

[查看原页](#)

个数, 故 $y = F(t)$ 有一水平渐近线。另外, $t = 0$ 时, 反应未开始, 质量分数为零。根据这些特点, 可设想 $y = F(t)$ 是双曲线型 $1/y = a + b/t$, 即 $y = t/(at + b)$ 。它与给定数据的规律大致符合。

为了确定 a, b , 令 $\bar{y} = 1/y, x = 1/t$, 于是可用 x 的线性函数 $S_1(x) = a + bx$ 拟合数据 $(x_i, \bar{y}_i)$ ($i = 1, 2, \dots, 16$), $x_i, \bar{y}_i$ 由原始数据 (t_i, y_i) 根据变换计算出来。与例 3.5 方法相同, 解方程组

$$\begin{cases} 16a + 3.38073b = 1.8372 \times 10^3, \\ 3.38073a + 1.58435b = 0.52886 \times 10^3, \end{cases}$$

得

$$a = 80.6621, \quad b = 161.6822.$$

从而得到

$$y = t/(80.6621t + 161.6822) = F^{(1)}(t),$$

其误差为

$$\delta_i^{(1)} = y_i - F^{(1)}(t_i) \quad (i = 1, 2, \dots, 16).$$

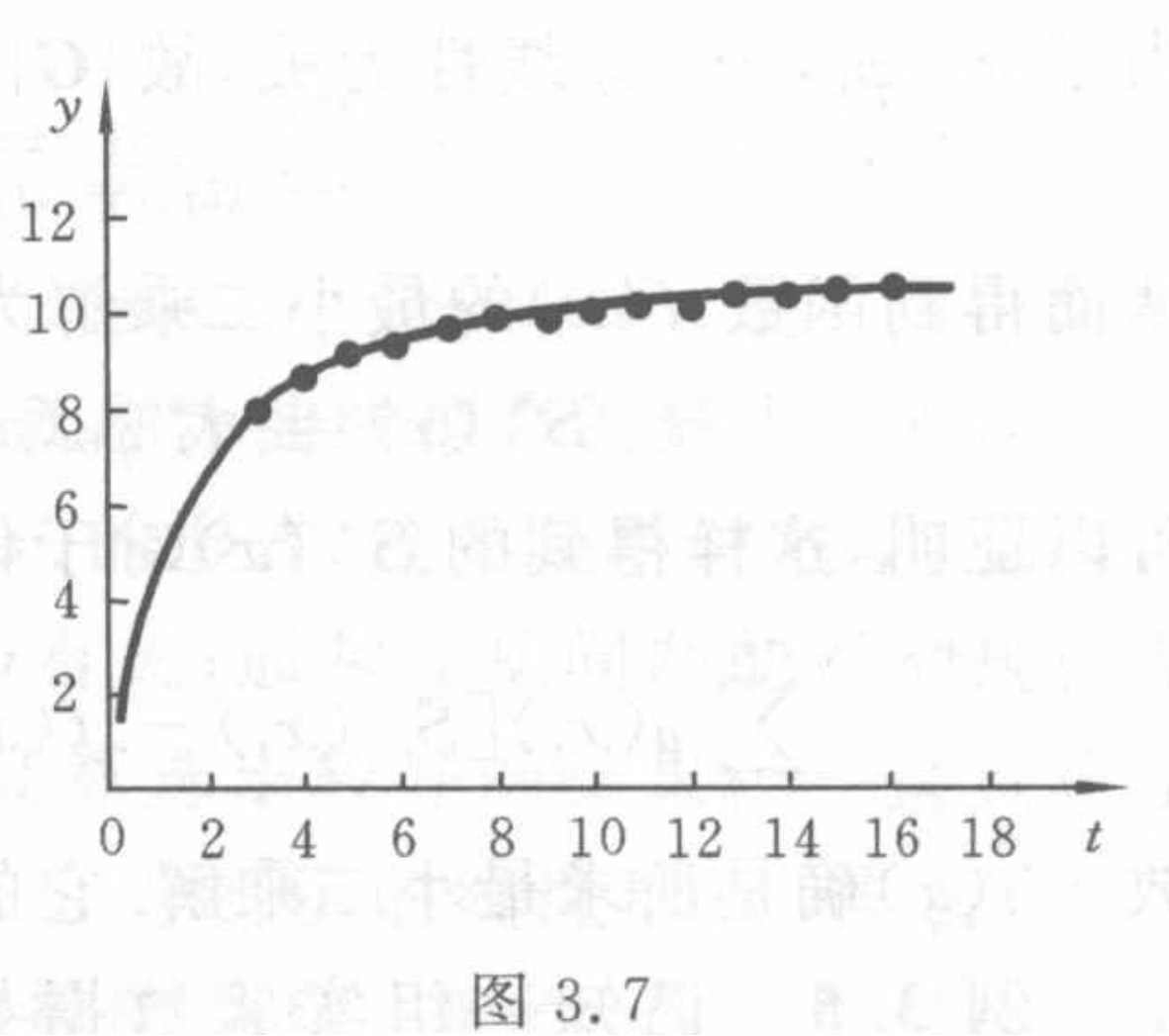


图 3.7 (原图裁切)。实验数据点及逐渐趋于水平的拟合曲线。图中文字与符号：纵轴 y ，横轴 t ，原点 0 ；横轴标值 $2, 4, 6, 8, 10, 12, 14, 16, 18$ ，纵轴标值 $2, 4, 6, 8, 10, 12$ 。

由图 3.7，符合给定数据的函数还可选为指数形式。此时可令拟合曲线形如

$$y = ae^{b/t}.$$

显然，当 $t \rightarrow \infty$ 时， $y \rightarrow a$ ；当 $t \rightarrow 0$ 时，若 $b < 0$ ，则 $y \rightarrow 0$ ，且 t 增加时 y 增加。这些与给出数据规律相同。为了确定 a 与 b ，对上式两端取对数，得 $\ln y = \ln a + b/t$ 。令

$$\hat{y} = \ln y, \quad A = \ln a, \quad x = 1/t,$$

于是由 (t_i, y_i) 计算出 $(x_i, \hat{y}_i)$ ，拟合数据 (x_i, y_i) 的曲线仍为 $S_1(x) = A + bx$ 。

用例 3.5 的方法计算出 $A = -4.48072, b = -1.0567$ ，从而

$$a = e^A = 11.3253 \times 10^{-3},$$

最后求得

$$y = 11.3253 \times 10^{-3} e^{-1.0567t} = F^{(2)}(t),$$

误差为

$$\delta_i^{(2)} = y_i - F^{(2)}(t_i) \quad (i = 1, 2, \dots, 16).$$

怎样比较这两个数学模型的好坏呢？只要分别计算各点误差，从中挑选误差较小的模型即可。本例经过计算可得

$$\|\delta^{(1)}\|_{\infty} = \max_i |\delta_i^{(1)}| = 0.568 \times 10^{-3},$$

$$\|\delta^{(2)}\|_{\infty} = \max_i |\delta_i^{(2)}| = 0.277 \times 10^{-3},$$

而均方误差为

$$\|\delta^{(1)}\|_2 = \sqrt{\sum_{i=1}^m (\delta_i^{(1)})^2} = 1.19 \times 10^{-3},$$

$$\|\delta^{(2)}\|_2 = \sqrt{\sum_{i=1}^m (\delta_i^{(2)})^2} = 0.34 \times 10^{-3},$$

校注 (agent 补充)：本页最终 $F^{(2)}(t)$ 的原书指数明确印为 $-1.0567t$ ，已保留。由上文设定 $y = ae^{b/t}$ 及 $b = -1.0567$ ，相容表达式应为 $e^{-1.0567/t}$ ；前者在 $t \rightarrow \infty$ 时趋于零，与本页给出的模型性质和数据不相容，属疑似原书漏印除号。见 [原式放大裁切](#)。上文“拟合数据 (x_i, y_i) ”亦按原印保留，其语境对应的是变换后的 $(x_i, \hat{y}_i)$ 。

原书 PDF 页 82; 印刷页码 69。

[查看原页](#)

由此可知, $\|\delta^{(2)}\|_2$ 及 $\|\delta^{(2)}\|_\infty$ 都比较小, 所以用 $y = F^{(2)}(t)$ 作拟合曲线比较好。

从本例看到, 拟合曲线的数学模型并不是一开始就能选得好的, 往往要通过分析确定若干模型后, 再经过实际计算, 才能选到较好的模型。本例的指数模型就比双曲线模型好。

现在很多计算机配有自动选择数学模型的程序, 其方法与本例相同。程序中因变量与自变量变换的函数类型较多, 通过计算比较误差找到拟合得较好的曲线, 最后输出曲线图形及数学表达式。

3.6.2 用正交函数作最小二乘拟合

用最小二乘法得到的法方程组 (3.6.6), 其系数矩阵 G 是病态的, 但如果 $\varphi_0(x), \varphi_1(x), \dots, \varphi_n(x)$ 是关于点集 $\{x_i\}$ ($i = 0, 1, \dots, m$) 带权 $\omega(x_i)$ ($i = 0, 1, \dots, m$) 正交的函数族, 即

$$(\varphi_j, \varphi_k) = \sum_{i=0}^m \omega(x_i) \varphi_j(x_i) \varphi_k(x_i) = \begin{cases} 0, & j \neq k, \\ A_k > 0, & j = k, \end{cases} \quad (3.6.8)$$

则方程组 (3.6.6) 的解为

$$a_k^* = \frac{(f, \varphi_k)}{(\varphi_k, \varphi_k)} = \frac{\sum_{i=0}^m \omega(x_i) f(x_i) \varphi_k(x_i)}{\sum_{i=0}^m \omega(x_i) \varphi_k^2(x_i)} \quad (k = 0, 1, \dots, n), \quad (3.6.9)$$

且平方误差为

$$\|\delta\|_2^2 = \|f\|_2^2 - \sum_{k=0}^n A_k (a_k^*)^2.$$

现在根据给定节点 $x_0, x_1, \dots, x_m$ 及权函数 $\omega(x) > 0$, 造出带权 $\omega(x)$ 正交的多项式 $\{P_n(x)\}$ 。注意 $n \leq m$, 用递推公式表示 $P_k(x)$, 即

$$\begin{cases} P_0(x) = 1, \\ P_1(x) = (x - \alpha_1)P_0(x), \\ P_{k+1}(x) = (x - \alpha_{k+1})P_k(x) - \beta_k P_{k-1}(x) \quad (k = 1, 2, \dots, n-1). \end{cases} \quad (3.6.10)$$

这里 $P_k(x)$ 是首项系数为 1 的 k 次多项式。根据 $P_k(x)$ 的正交性, 得

$$\left\{ \begin{array}{l} a_{k+1} = \frac{\sum_{i=0}^m \omega(x_i) x_i P_k^2(x_i)}{\sum_{i=0}^m \omega(x_i) P_k^2(x_i)} = \frac{(x P_k(x), P_k(x))}{(P_k(x), P_k(x))} = \frac{(x P_k, P_k)}{(P_k, P_k)}, \\ \beta_k = \frac{\sum_{i=0}^m \omega(x_i) P_k^2(x_i)}{\sum_{i=0}^m \omega(x_i) P_{k-1}^2(x_i)} = \frac{(P_k, P_k)}{(P_{k-1}, P_{k-1})} \end{array} \right. \quad (k = 0, 1, \dots, n-1). \quad (3.6.11)$$

校注 (agent 补充): 式 (3.6.11) 第一行原书印为拉丁字母 a_{k+1} , 式 (3.6.10) 则为希腊字母 α_{k+1} , 已分别保留。该式末的共同范围原书为 $k = 0, 1, \dots, n-1$; 若直接用于第二行 β_k , $k = 0$ 会涉及未定义的 P_{-1} 。依式 (3.6.10), 实际递推使用的是 $\beta_1, \dots, \beta_{n-1}$ 。见 [原式裁切](#)。

原书 PDF 页 83; 印刷页码 70。

[查看原页](#)

下面用归纳法证明这样给出的 $\{P_k(x)\}$ 是正交的。由式 (3.6.10) 第二式及式 (3.6.11) 中 a_1 的表达式, 有

$$\begin{aligned}(P_l, P_1) &= (P_0, xP_0) - \alpha_1(P_0, P_0) \\ &= (P_0, xP_0) - \frac{(xP_0, P_0)}{(P_0, P_0)}(P_0, P_0) = 0.\end{aligned}$$

现假定 $(P_1, P_s) = 0$ ($l \neq s$) 对于 $s = 0, 1, \dots, l-1$ 及 $l = 0, 1, \dots, k$ ($k < n$) 均成立, 要证 $(P_{k+1}, P_s) = 0$ 对于 $s = 0, 1, \dots, k$ 均成立。由式 (3.6.10), 有

$$\begin{aligned}(P_{k+1}, P_s) &= ((x - \alpha_{k+1})P_k, P_s) - \beta_k(P_{k-1}, P_s) \\ &= (xP_k, P_s) - \alpha_{k+1}(P_k, P_s) - \beta_k(P_{k-1}, P_s),\end{aligned}\tag{3.6.12}$$

由归纳法假定, 当 $0 \leq s \leq k-2$ 时

$$(P_k, P_s) = 0, \quad (P_{k-1}, P_s) = 0.$$

另外, $xP_s(x)$ 是首项系数为 1 的 $s+1$ 次多项式, 它可用 $P_0, P_1, \dots, P_{s+1}$ 的线性组合表示, 而 $s+1 \leq k-1$, 故由归纳法假定又有

$$(xP_k, P_s) \equiv (P_k, xP_s) = 0,$$

于是由式 (3.6.12), 当 $s \leq k-2$ 时, $(P_{k+1}, P_s) = 0$ 。

再看

$$(P_{k+1}, P_{k-1}) = (xP_k, P_{k-1}) - \alpha_{k+1}(P_k, P_{k-1}) - \beta_k(P_{k-1}, P_{k-1}),\tag{3.6.13}$$

由假定有

$$(P_k, P_{k-1}) = 0,$$

$$(xP_k, P_{k-1}) = (P_k, xP_{k-1}) = \left(P_k, P_k + \sum_{j=0}^{k-1} C_j P_j \right) = (P_k, P_k).$$

利用式 (3.6.11) 的 β_k 表达式及以上结果, 得

$$(P_{k+1}, P_{k-1}) = (xP_k, P_{k-1}) - \beta_k(P_{k-1}, P_{k-1}) = (P_k, P_k) - (P_k, P_k) = 0.$$

最后, 由式 (3.6.11) 有

$$\begin{aligned}
(P_{k+1}, P_k) &= (xP_k, P_k) - \alpha_{k+1}(P_k, P_k) - \beta_k(P_k, P_{k-1}) \\
&= (xP_k, P_k) - \frac{(xP_k, P_k)}{(P_k, P_k)}(P_k, P_k) = 0.
\end{aligned}$$

至此已证明了由式 (3.6.10) 及式 (3.6.11) 确定的多项式, $\{P_k(x)\}$ ($k = 0, 1, \dots, n; n \leq m$) 组成一个关于点集 $\{x_i\}$ 的正交系。

用正交多项式 $\{P_k(x)\}$ 的线性组合作最小二乘曲线拟合, 只要在根据式 (3.6.10) 及式 (3.6.11) 逐步求 $P_k(x)$ 的同时, 相应计算出系数

$$a_k = \frac{(f, P_k)}{(P_k, P_k)} = \frac{\sum_{i=0}^m \omega(x_i) f(x_i) P_k(x_i)}{\sum_{i=0}^m \omega(x_i) P_k^2(x_i)} \quad (k = 0, 1, \dots, n),$$

并逐步把 $\alpha_k^* P_k(x)$ 累加到 $F(x)$ 中去, 最后就可得到所求的拟合曲线

$$y = F(x) = \alpha_0^* P_0(x) + \alpha_1^* P_1(x) + \dots + \alpha_n^* P_n(x),$$

校注 (agent 补充): 本页第一式左端原扫描为 (P_l, P_1) (第一下标为字母 l), 依右端计算应对应 (P_0, P_1) 。下一段归纳假设原印为 $(P_1, P_s) = 0$, 括号内条件却为 $l \neq s$, 疑应为 $(P_l, P_s) = 0$ 。均忠实保留并见 [原文裁切](#)。页尾系数公式用 a_k , 其后累加与拟合曲线改用 α_k^* , 也按原印保留; 此处应是同一组拟合系数的记号不一致, 不应与递推系数 α_k 混用。见 [原文裁切](#)。

原书 PDF 页 84; 印刷页码 71。

[查看原页](#)

这里 n 可事先给定或在计算过程中根据误差确定。

用这种方法编程序不用解方程组, 只用递推公式, 并且当逼近次数增加一次时, 只要把程序中循环数加 1, 其余不用改变。这是目前用多项式作曲线拟合最好的计算方法, 有通用的语言程序供用户使用。

3.6.3 多元最小二乘拟合

上面介绍的最小二乘法的有关概念与方法可推广到多元函数, 例如, 已知多元函数

$$y = f(x_1, x_2, \dots, x_l)$$

的一组测量数据 $(x_{1i}, x_{2i}, \dots, x_{li}, y_i)$ ($i = 1, 2, \dots, m$), 以及一组权系数 $\omega_i > 0$ ($i = 1, 2, \dots, m$), 要求函数

$$S_n(x_1, x_2, \dots, x_l) = \sum_{k=1}^n a_k \varphi_k(x_1, x_2, \dots, x_l), \quad n \leq m,$$

使得

$$F(a_0, a_1, \dots, a_n) = \sum_{i=1}^m \omega_i [y_i - S_n(x_{1i}, x_{2i}, \dots, x_{li})]^2$$

最小, 这与式 (3.6.4) 的极值问题完全一样, 系数 $a_0, a_2, \dots, a_n$ 同样满足法方程组 (3.6.6), 只是这里

$$(\varphi_k, \varphi_j) = \sum_{i=1}^m \omega_i \varphi_k(x_{1i}, x_{2i}, \dots, x_{li}) \varphi_j(x_{1i}, x_{2i}, \dots, x_{li}).$$

求解法方程组 (3.6.6) 就可得到 a_k ($k = 0, 1, \dots, n$), 从而得到 $S_n(x_1, x_2, \dots, x_l)$ 。称 $S_n(x_1, x_2, \dots, x_l)$ 为函数 $f(x_1, x_2, \dots, x_l)$ 的最小二乘拟合。

3.7 Fourier 逼近与快速 Fourier 变换

当 $f(x)$ 是周期函数时, 显然用三角多项式比用代数多项式逼近更合适, 本节主要讨论用三角多项式作最小平方逼近及快速 Fourier 变换 (Fast Fourier Transform), 简称 FFT 算法。

3.7.1 最佳平方三角逼近与三角插值

设 $f(x)$ 是以 2π 为周期的平方可积函数, 用三角多项式

$$S_n(x) = \frac{1}{2}a_0 + a_1 \cos x + b_1 \sin x + \dots + a_n \cos nx + b_n \sin nx \quad (3.7.1)$$

作最佳平方逼近函数。由于三角函数族 $1, \cos x, \sin x, \dots, \cos kx, \sin kx, \dots$ 在 $[0, 2\pi]$ 上是正交函数族, 于是 $f(x)$ 在 $[0, 2\pi]$ 上的最小平方三角逼近多项式 $S_n(x)$ 的系数是

校注 (agent 补充): 3.6.3 原书的 S_n 求和从 $k = 1$ 开始, 但后续使用 $F(a_0, a_1, \dots, a_n)$ 和 a_k ($k = 0, \dots, n$); 段中系数列表印为 $a_0, a_2, \dots, a_n$; 内积公式右端第二因子的原印下标为 i , 而左端为 j , 相容写法应使用 φ_j 。这些原书记号不一致均已保留, 见 [原文裁切](#)。

原书 PDF 页 85; 印刷页码 72。

[查看原页](#)

$$\begin{cases} a_k = \frac{1}{\pi} \int_0^{2\pi} f(x) \cos kx \, dx & (k = 0, 1, \dots, n), \\ b_k = \frac{1}{\pi} \int_0^{2\pi} f(x) \sin kx \, dx & (k = 1, 2, \dots, n), \end{cases} \quad (3.7.2)$$

a_k, b_k 称为 Fourier 系数, 函数 $f(x)$ 按 Fourier 系数展开得到的级数

$$\frac{1}{2}a_0 + \sum_{k=1}^{\infty} (a_k \cos kx + b_k \sin kx) \quad (3.7.3)$$

就称为 Fourier 级数, 只要 $f'(x)$ 在 $[0, 2\pi]$ 上分段连续, 则级数 (3.7.3) 一致收敛到 $f(x)$ 。

当 $f(x)$ 只在给定的离散点集 $\left\{x_j = \frac{2\pi}{N}j, j = 0, 1, \dots, N-1\right\}$ 上已知时, 则可类似得到在离散点集上的正交性与相应的离散 Fourier 系数。为方便起见, 下面只给出奇数个点的情形。令

$$x_j = \frac{2\pi j}{2m+1} \quad (j = 0, 1, \dots, 2m),$$

可以证明, 对于任何 $0 \leq k, l \leq m$, 成立

$$\begin{cases} \sum_{j=0}^{2m} \sin lx_j \sin kx_j = \begin{cases} 0, & l \neq k, \, l = k = 0, \\ \frac{2m+1}{2}, & l = k \neq 0, \end{cases} \\ \sum_{j=0}^{2m} \cos lx_j \cos kx_j = \begin{cases} 0, & l \neq k, \\ \frac{2m+1}{2}, & l = k \neq 0, \\ 2m+1, & l = k = 0, \end{cases} \\ \sum_{j=0}^{2m} \cos lx_j \sin kx_j = 0, \quad 0 \leq k, j \leq m. \end{cases}$$

这就表明, 函数族 $\{1, \cos x, \sin x, \dots, \cos mx, \sin mx\}$ 在点集 $\left\{x_j = \frac{2\pi j}{2m+1}\right\}$ 上正交。若令 $f_j = f(x_j)$ ($j = 0, 1, \dots, 2m$), 则 $f(x)$ 的最小二乘三角逼近为

$$S_n(x) = \frac{1}{2}a_0 + \sum_{k=1}^n (a_k \cos kx + b_k \sin kx), \quad n < m,$$

其中

$$\begin{cases} a_k = \frac{2}{2m+1} \sum_{j=0}^{2m} f_j \cos \frac{2\pi jk}{2m+1} & (k = 0, 1, \dots, n), \\ b_k = \frac{2}{2m+1} \sum_{j=0}^{2m} f_j \sin \frac{2\pi jk}{2m+1} & (k = 1, \dots, n). \end{cases} \quad (3.7.4)$$

当 $n = m$ 时, 可证明 $S_m(x_j) = f_j$ ($j = 0, 1, \dots, 2m$), 于是

$$S_m(x) = \frac{1}{2}a_0 + \sum_{k=1}^m (a_k \cos kx + b_k \sin kx)$$

校注 (agent 补充): 离散正交性公式的最后一行, 原书条件为 $0 \leq k, j \leq m$, 已保留。这里 j 是求和指标, 函数频率为 k, l ; 与该组公式前的条件一致时应写 $0 \leq k, l \leq m$ 。

原书 PDF 页 86; 印刷页码 73。

[查看原页](#)

就是三角插值多项式, 系数仍由式 (3.7.4) 表示。

更一般情形, 假定 $f(x)$ 是以 2π 为周期的复函数, 给定在 N 个等分点 $x_j = \frac{2\pi}{N}j$ ($j = 0, 1, \dots, N-1$) 上的值 $f_j = f\left(\frac{2\pi}{N}j\right)$, 由于

$$e^{ijx} = \cos(jx) + i\sin(jx) \quad (j = 0, 1, \dots, N-1; i = \sqrt{-1}),$$

函数族 $\{1, e^{ix}, \dots, e^{i(N-1)x}\}$ 在区间 $[0, 2\pi]$ 上是正交的。将函数 e^{ijx} 在等距点集 $x_k = \frac{2\pi}{N}k$ ($k = 0, 1, \dots, N-1$) 上的值 e^{ijx_k} 组成的向量记作

$$\varphi_j = \left(1, e^{ij\frac{2\pi}{N}}, \dots, e^{ij\frac{2\pi}{N}(N-1)}\right)^T.$$

当 $j = 0, 1, \dots, N-1$ 时, N 个复向量 $\varphi_0, \varphi_1, \dots, \varphi_{N-1}$ 具有下面所定义的正交性:

$$(\varphi_l, \varphi_s) = \sum_{k=0}^{N-1} e^{il\frac{2\pi}{N}k} e^{-is\frac{2\pi}{N}k} = \sum_{k=0}^{N-1} e^{i(l-s)\frac{2\pi}{N}k} = \begin{cases} 0, & l \neq s, \\ N, & l = s. \end{cases} \quad (3.7.5)$$

事实上, 令 $r = e^{i(l-s)\frac{2\pi}{N}}$, 若 $0 \leq l, s \leq N-1$, 则有

$$0 \leq l \leq N-1, \quad -(N-1) \leq -s \leq 0,$$

于是

$$-(N-1) \leq l-s \leq N-1, \quad -1 < -\frac{N-1}{N} \leq \frac{l-s}{N} \leq \frac{N-1}{N} < 1.$$

若 $l-s \neq 0$, 则 $r \neq 1$, 从而 $r^N = e^{i(l-s)2\pi} = 1$, 于是

$$(\varphi_l, \varphi_s) = \sum_{k=0}^{N-1} r^k = \frac{1-r^N}{1-r} = 0.$$

若 $l=s$, 则 $r=1$, 于是 $(\varphi_s, \varphi_s) = \sum_{k=0}^{N-1} r^k = N$ 。这就证明了式 (3.7.5) 成立, 即 $\varphi_0, \varphi_1, \dots, \varphi_{N-1}$ 是正交的。

因此, $f(x)$ 在 N 个点 $\left\{x_j = \frac{2\pi}{N}j, j = 0, 1, \dots, N-1\right\}$ 上的最小二乘 Fourier 逼近为

$$S(x) = \sum_{k=0}^{n-1} C_k e^{ikx}, \quad n \leq N, \quad (3.7.6)$$

其中

$$C_k = \frac{1}{N} \sum_{j=0}^{N-1} f_j e^{-ikj\frac{2\pi}{N}} \quad (k = 0, 1, \dots, N-1). \quad (3.7.7)$$

在式 (3.7.6) 中, 若 $n = N$, 则 $S(x)$ 为 $f(x)$ 在点 x_j ($j = 0, 1, \dots, N-1$) 上的插值函数, 即 $S(x_j) = f(x_j)$, 于是由式 (3.7.6) 得

$$f_j = \sum_{k=0}^{N-1} C_k e^{ik \frac{2\pi}{N} j} \quad (j = 0, 1, \dots, N-1). \quad (3.7.8)$$

式 (3.7.7) 表示了由 $\{f_j\}$ 求 $\{C_k\}$ 的过程, 称为 $f(x)$ 的离散 Fourier 变换, 简称

原书 PDF 页 87; 印刷页码 74。

[查看原页](#)

DFT; 而式 (3.7.8) 是由 $\{C_k\}$ 求 $\{f_j\}$ 的过程, 称为反变换。它们是使用计算机进行 Fourier 分析的主要方法, 在数字信号处理、全息技术、光谱和声谱分析等很多领域都有广泛应用。

3.7.2 快速 Fourier 变换

不论是按式 (3.7.7) 由 $\{f_k\}$ 求 $\{C_j\}$, 按式 (3.7.8) 由 $\{C_j\}$ 求 $\{f_k\}$, 还是由式 (3.7.4) 计算 Fourier 逼近系数 a_k, b_k , 都可归结为计算

$$C_j = \sum_{k=0}^{N-1} x_k w^{kj} \quad (j = 0, 1, \dots, N-1), \quad (3.7.9)$$

其中 $w = \exp(-i2\pi/N)$ (正变换) 或 $w = \exp(i2\pi/N)$ (反变换); $\{x_k\}$ ($k = 0, 1, \dots, N-1$) 是已知复数序列。如直接用式 (3.7.9) 计算 C_j , 需要 N 次复数乘法和 N 次复数加法 (称为 N 个操作), 计算全部 C_j 共要 N^2 个操作。当 N 较大且处理数据很多时, 就是用高速的电子计算机, 很多实际问题仍然无法计算, 直到 20 世纪 60 年代中期产生了快速 Fourier 变换, 即 FFT 算法, 大大提高了运算速度, 才使 Fourier 变换得以广泛应用。FFT 算法的思想就是尽量减少乘法次数, 例如, 计算 $ab + ac = a(b + c)$, 用左端计算时做两次乘法, 用右端计算时只做一次乘法。用式 (3.7.9) 计算全部 C_j , 表面看要做 N^2 次乘法, 实际上, 在所有 $\exp(i2\pi kj/N)$, $j, k = 0, 1, \dots, N-1$ 中, 只有 N 个不同的值 $w^0, w^1, \dots, w^{N-1}$, 特别当 $N = 2^p$ 时, 只有 $N/2$ 个不同的值。因此, 可把同一个 w^r 对应的 x_k 相加后再乘 w^r , 这样就能大量减少乘法次数。为了具体推导 FFT 算法, 先给出定义:

设正整数 m 除以 N 后得商 q 及余数 r , 则 $m = qN + r$, r 称为 m 的 N 同余数。由于 $w = \exp(i2\pi/N)$, $w^N = e^{i2\pi} = 1$, 故有 $w^m = (w^N)^q w^r = w^r$ 。

因此计算 w^m 时可用 w 的 N 同余数 r 代替 m , 从而推出 FFT 算法。下面以 $N = 2^3$ 为例说明 FFT 的计算方法。由于 $0 \leq k, j \leq N-1 = 2^3 - 1 = 7$, 则式 (3.7.9) 的和是

$$C_j = \sum_{k=0}^7 x_k w^{jk} \quad (j = 0, 1, \dots, 7). \quad (3.7.10)$$

将 k, j 用二进制表示为

$$k = k_2 2^2 + k_1 2 + k_0 2^0 = (k_2 k_1 k_0),$$

$$j = j_2 2^2 + j_1 2 + j_0 2^0 = (j_2 j_1 j_0),$$

其中 k_r, j_r ($r = 0, 1, 2$) 只能取 0 或 1, 例如, $6 = 2^2 + 2^1 + 0 \times 2^0 = (110)$ 。根据 k, j 表示法, 有 $C_j = C(j_2 j_1 j_0)$, $x_k = x(k_2 k_1 k_0)$ 。式 (3.7.10) 可表示为

$$\begin{aligned} C(j_2 j_1 j_0) &= \sum_{k_0=0}^1 \sum_{k_1=0}^1 \sum_{k_2=0}^1 x(k_2 k_1 k_0) w^{(k_2 k_1 k_0)(j_2 2^2 + j_1 2 + j_0 2^0)} \\ &= \sum_{k_0=0}^1 \left\{ \sum_{k_1=0}^1 \left[\sum_{k_2=0}^1 x(k_2 k_1 k_0) w^{j_0(k_2 k_1 k_0)} \right] w^{j_1(k_1 k_0 0)} \right\} (w^{j_2(k_0 0 0)}). \end{aligned} \quad (3.7.11)$$

校注(agent 补充): 原书“特别当 $N = 2^p$ 时, 只有 $N/2$ 个不同的值”已保留。对原文列出的全部 $\exp(i2\pi kj/N)$ 而言, 取 $j = 1$ 已有 N 个不同值; 若利用 $w^{r+N/2} = -w^r$, 才可用 $N/2$ 个值及符号表示它们。因此该处“不同的值”疑漏了“除符号外”一类限定。原书“可用 w 的 N 同余数 r 代替 m ”也按原文保留, 根据前段定义, 此处应是指指数 m 的余数。

原书 PDF 页 88; 印刷页码 75。

[查看原页](#)

若引入记号

$$\begin{cases} A_0(k_2 k_1 k_0) = x(k_2 k_1 k_0), \\ A_1(k_1 k_0 j_0) = \sum_{k_2=0}^1 A_0(k_2 k_1 k_0) w^{j_0(k_2 k_1 k_0)}, \\ A_2(k_0 j_1 j_0) = \sum_{k_1=0}^1 A_1(k_1 k_0 j_0) w^{j_1(k_1 k_0 0)}, \\ A_3(j_2 j_1 j_0) = \sum_{k_0=0}^1 A_2(k_0 j_1 j_0) w^{j_2(k_0 0 0)}, \end{cases} \quad (3.7.12)$$

则式 (3.7.11) 变成

$$C(j_2 j_1 j_0) = A_3(j_2 j_1 j_0).$$

它说明利用 N 同余数可把计算 C_j 分为 p 步, 用式 (3.7.12) 计算, 每计算一个 A_q 只用 2 次复数乘法, 计算一个 C_j 用 $2p$ 次复数乘法, 计算全部 C_j 共用 $2pN$ 次复数乘法。若注意 $w^{j_0 2^{p-1}} = w^{j_0 N/2} = (-1)^{j_0}$, 式 (3.7.12) 还可进一步简化为

$$\begin{aligned} A_1(k_1 k_0 j_0) &= \sum_{k_2=0}^1 A_0(k_2 k_1 k_0) w^{j_0(k_2 k_1 k_0)} \\ &= A_0(0k_1 k_0) w^{j_0(0k_1 k_0)} + A_0(1k_1 k_0) w^{j_0 2^2} w^{j_0(0k_1 k_0)} \\ &= [A_0(0k_1 k_0) + (-1)^{j_0} A_0(1k_1 k_0)] w^{j_0(0k_1 k_0)}, \end{aligned}$$

$$A_1(k_1 k_0 0) = A_0(0k_1 k_0) + A_0(1k_1 k_0),$$

$$A_1(k_1 k_0 1) = [A_0(0k_1 k_0) - A_0(1k_1 k_0)] w^{(0k_1 k_0)}.$$

将表达式中的二进制表示还原为十进制表示: $k = (0k_1 k_0) = k_1 2^1 + k_0 2^0$, 即 $k = 0, 1, 2, 3$, 得

$$\begin{cases} A_1(2k) = A_0(k) + A_0(k + 2^2), \\ A_1(2k + 1) = [A_0(k) - A_0(k + 2^2)] w^k \end{cases} \quad (k = 0, 1, 2, 3). \quad (3.7.13)$$

同样, 式 (3.7.12) 中的 A_2 可简化为

$$A_2(k_0 j_1 j_0) = [A_1(0k_0 j_0) + (-1)^{j_1} A_1(1k_0 j_0)] w^{j_1(0k_0 0)},$$

即

$$A_2(k_0 0 j_0) = A_1(0k_0 j_0) + A_1(1k_0 j_0),$$

$$A_2(k_0 1 j_0) = [A_1(0 k_0 j_0) - A_1(1 k_0 j_0)] w^{(0 k_0 0)}.$$

将表达式中的二进制表示还原为十进制表示, 得

$$\begin{cases} A_2(k 2^2 + j) = A_1(2k + j) + A_1(2k + j + 2^2), & k = 0, 1, \\ A_2(k 2^2 + j + 2) = [A_1(2k + j) - A_1(2k + j + 2^2)] w^{2k}, & j = 0, 1. \end{cases} \quad (3.7.14)$$

同样, 式 (3.7.12) 中 A_3 可简化为

$$A_3(j_2 j_1 j_0) = A_2(0 j_1 j_0) + (-1)^{j_2} A_2(1 j_1 j_0),$$

即

$$A_3(0 j_1 j_0) = A_2(0 j_1 j_0) + A_2(1 j_1 j_0),$$

$$A_3(1 j_1 j_0) = A_2(0 j_1 j_0) - A_2(1 j_1 j_0).$$

原书 PDF 页 89; 印刷页码 76。

[查看原页](#)

将表达式中的二进制表示还原为十进制表示, 得

$$\begin{cases} A_3(j) = A_2(j) + A_2(j + 2^2), \\ A_3(j + 2^2) = A_2(j) - A_2(j + 2^2) \end{cases} \quad (j = 0, 1, 2, 3). \quad (3.7.15)$$

根据式 (3.7.13)、式 (3.7.14)、式 (3.7.15), 由 $A_0(k) = x(k) = x_k$ ($k = 0, 1, \dots, 7$) 逐次计算到 $A_3(j) = C_j$ ($j = 0, 1, \dots, 7$), 结果如表 3.5 所示。

表 3.5

单元码号	(0) 000	(1) 001	(2) 010	(3) 011	(4) 100	(5) 101	(6) 110	(7) 111
					$w^0 = 1$	w^1	w^2	w^3
$x_k =$ $A_0(k)$	$A_0(0)$	$A_0(1)$	$A_0(2)$	$A_0(3)$	$A_0(4)$	$A_0(5)$	$A_0(6)$	$A_0(7)$
A_1	$A_0(0) +$ $A_0(4)$	$[A_0(0) -$ $A_0(4)]w^0$	$A_0(1) +$ $A_0(5)$	$[A_0(1) -$ $A_0(5)]w^1$	$A_0(2) +$ $A_0(6)$	$[A_0(2) -$ $A_0(6)]w^2$	$A_0(3) +$ $A_0(7)$	$[A_0(3) -$ $A_0(7)]w^3$
A_2	$A_1(0) +$ $A_1(4)$	$A_1(0) +$ $A_1(5)$	$[A_1(0) -$ $A_1(4)]w^0$	$[A_1(1) -$ $A_1(5)]w^0$	$A_1(2) +$ $A_1(6)$	$A_1(3) +$ $A_1(7)$	$[A_1(2) -$ $A_1(6)]w^2$	$[A_1(3) -$ $A_1(7)]w^2$
$C_i =$ $A_3(j)$	$(0) + (4)$	$(1) + (5)$	$(2) + (6)$	$(3) + (7)$	$(0) - (4)$	$(1) - (5)$	$(2) - (6)$	$(3) - (7)$

表 3.5 原图裁切。

上面推导的 $N = 2^3$ 的计算公式可类似地推广到 $N = 2^p$ 的情形。根据式 (3.7.13)、式 (3.7.14)、式 (3.7.15), 一般情况的 FFT 计算公式如下:

$$\begin{cases} A_q(k2^q + j) = A_{q-1}(k2^{q-1} + j) + A_{q-1}(k2^{q-1} + j + 2^{p-1}), \\ A_q(k2^q + j + 2^{q-1}) = [A_{q-1}(k2^{q-1} + j) - A_{q-1}(k2^{q-1} + j + 2^{p-1})]w^{k2^{q-1}}, \end{cases} \quad (3.7.16)$$

其中 $q = 1, 2, \dots, p$; $k = 0, 1, \dots, 2^{p-q} - 1$; $j = 0, 1, \dots, 2^{q-1} - 1$ 。 A_q 括号内的数代表它的位置, 在计算机中代表存放数的地址。一组 A_q 占用 N 个复数单元, 计算时需给出两组单元, 从 $A_0(m)$ ($m = 0, 1, \dots, N - 1$) 出发, q 由 1 到 p 算到 $A_p(j) = C_j$ ($j = 0, 1, \dots, N - 1$), 即为所求。计算过程中只要按地址号存放 A_q , 最后得到的 $A_p(j)$ 就是所求离散频谱的次序 (注意, 目前一些计算机程序计算结果地址是逆序排列, 还要增加倒地址的一步才是这里介绍的结果)。这个计算公式不仅具有不倒地址的优点, 而且计算只有两重循环, 外循环 q 由 1 计算到 p , 内循环 k 由 0 计算到 $2^{p-q} - 1$, j 由 0 计算到 $2^{q-1} - 1$, 更重要的是整个计算过程计算量小。由公式可看到, 算一个 A_q 共做 $2^{p-q}2^{q/1} = N/2$ 次复数乘法; 而最后一步计算 A_p 时, 由于 $w^{k2^{p-1}} = (w^{N/2})^k = (-1)^k = (-1)^0 = 1$ (注意, $q = p$ 时 $2^{p-q} - 1 = 0$, 故 $k = 0$), 因此, 总共要算 $(p-1)N/2$ 次复数乘法, 它比直接用式 (3.7.9) 计算需 N^2 次乘法快得多, 计算量之比是 $N : (p-1)/2$ 。当 $N = 2^{10}$ 时二者之比是 $1024 : 4.5 \approx 230 : 1$ 。它比一般 FFT 的计算量 pN 次乘法

校注 (agent 补充): 表 3.5 的 A_2 行、单元 (1) 原书为 $A_1(0) + A_1(5)$, 按式 (3.7.14) 代入 $k = 0, j = 1$ 应为 $A_1(1) + A_1(5)$; 表末行标签原书为 $C_i = A_3(j)$, 与正文 C_j 的记号不一致。两处均按原印保留。运算量段落原印为 $2^{p-q}2^{q/1} = N/2$, 放大可辨指数中为斜杠, 依前文循环次数应为 $2^{p-q}2^{q-1} = N/2$ 。见 [运算量原式裁切](#)。

原书 PDF 页 90; 印刷页码 77。

[查看原页](#)

也快一倍。式 (3.7.16) 的计算公式称为改进的 FFT 算法, 下面给出这一算法的程序步骤:

步 1 给出数组 $A_1(N), A_2(N)$ 及 $w(N/2)$ 。

步 2 将已知的记录复数数组 $\{x_k\}$ 输入到单元 $A_1(k)$ (k 从 0 到 $N-1$) 中。

步 3 计算 $w^m = \exp\left(-i\frac{2\pi}{N}m\right)$ 或 $w^m = \exp\left(i\frac{2\pi}{N}m\right)$, 存放在单元 $w(m)$ (m 从 0 到 $(N/2)-1$) 中。

步 4 q 循环从 1 到 p , 若 q 为奇数做步 5, 否则做步 6。

步 5 k 循环从 0 到 $2^{p-q}-1$, j 循环从 0 到 $2^{q-1}-1$, 计算

$$A_2(k2^q + j) = A_1(k2^{q-1} + j) + A_1(k2^{q-1} + j + 2^{p-1}),$$

$$A_2(k2^q + j + 2^{q-1}) = [A_1(k2^{q-1} + j) - A_1(k2^{q-1} + j + 2^{q-1})]w(k2^{q-1}),$$

转步 7。

步 6 k 循环从 0 到 $2^{p-q}-1$, j 循环从 0 到 2^{q-1} , 计算

$$A_1(k2^q + j) = A_2(k2^{q-1} + j) + A_2(k2^{q-1} + j + 2^{p-1}),$$

$$A_1(k2^q + j + 2^{q-1}) = [A_2(k2^{q-1} + j) - A_2(k2^{q-1} + j + 2^{q-1})]w(k2^{q-1}),$$

k, j 循环结束, 做下一步。

步 7 若 $q = p$ 转步 8, 否则 $q+1 \rightarrow q$ 转步 4。

步 8 q 循环结束, 若 $p =$ 偶数, 将 $A_1(j) \rightarrow A_2(j)$, 则 $C_j = A_2(j)$ ($j = 0, 1, \dots, N-1$) 即为所求。

小结

函数逼近是数学中一个很重要的课题, 本章只介绍了函数逼近最基本的概念及常用的正交多项式, 并从函数计算观点介绍了用多项式逼近连续函数的主要方法。这些都是计算数学的基础, 需要详细了解的读者可参看文献 [4]。本章对函数逼近与计算的另一重要课题——有理逼近没有涉及, 这方面内容可参看文献 [5]。

本章介绍的曲线拟合的最小二乘法和离散 Fourier 变换都可从最佳平方逼近得到, 它们在实际中都有广泛的应用, 本章着重介绍了在计算机上有效和节省时间的算法, 而不要求理论上的详细讨论。特别是 FFT 算法, 不但应用中意义重大, 而且也是计算数学中节省运算次数的一个典范。本章介绍的改进 FFT 算法比目前使用的 FFT 算法计算次数又减少了, 程序也较简单, 这一算法的推导与应用可参看文献 [6]。

习题

1. (1) 利用区间变换推出区间为 $[a, b]$ 的 Bernstein 多项式。

校注 (agent 补充): 步 5、步 6 的差式中, 第二个读取地址的偏移量原书均印为 2^{q-1} , 与式 (3.7.16) 的 2^{p-1} 不一致; 正文保留原式。步 6 的 j 循环上界原印为 2^{q-1} , 相较式 (3.7.16) 与步 5 少了 “-1”。按该上界执行会多计算一个 j , 在 $q = p$ 时也会越出 N 个数组单元范围。见 [原程序步骤裁切](#)。

原书 PDF 页 91; 印刷页码 78。

[查看原页](#)

(2) 对 $f(x) = \sin x$ 在 $[0, \pi/2]$ 上求一次和三次 Bernstein 多项式, 画出图形, 并与相应的 Taylor 级数部分和误差作比较。

2. 求证:

(1) 当 $m \leq f(x) \leq M$ 时, $m \leq B_n(f, x) \leq M$; (2) 当 $f(x) = x$ 时, $B_n(f, x) = x$ 。

3. 在次数不超过 6 的多项式中, 求 $f(x) = \sin 4x$ 在 $[0, 2\pi]$ 上的最佳一致逼近多项式。

4. 假设 $f(x)$ 在 $[a, b]$ 上连续, 求 $f(x)$ 的零次最佳一致逼近多项式。

5. 选取常数 a , 使 $\max_{0 \leq x \leq 1} |x^3 - ax|$ 达到极小, 又问这个解是否唯一。

6. 求 $f(x) = \sin x$ 在 $[0, \pi/2]$ 上的最佳一次逼近多项式, 并估计误差。

7. 求 $f(x) = e^x$ 在 $[0, 1]$ 上的最佳一次逼近多项式。

8. 如何选取 r , 使 $p(x) = x^2 + r$ 在 $[-1, 1]$ 上与零偏差最小? r 是否唯一?

9. 设 $f(x) = x^4 + 3x^3 - 1$, 在 $[0, 1]$ 上求三次最佳逼近多项式。

10. 令 $T_n(x) = T_n(2x - 1)$, $x \in [0, 1]$, 求 $T_0^*(x), T_1^*(x), T_2^*(x), T_3^*(x)$ 。

11. 试证 $\{T_n^*(x)\}$ 是在 $[0, 1]$ 上带权 $\rho = \frac{1}{\sqrt{x-x^2}}$ 的正交多项式。

12. $f(x)$ 是 $[-a, a]$ 上的连续奇 (偶) 函数, 证明: 不管 n 是奇数或偶数, $f(x)$ 的最佳逼近多项式 $F_n^*(x) \in H_n$ 也是奇 (偶) 函数。

13. 求 a, b , 使 $\int_0^{\pi/2} [ax + b - \sin x]^2 dx$ 为最小, 并与题 1 及题 6 的一次逼近多项式误差作比较。

14. $f(x), g(x) \in C^1[a, b]$, 定义

$$(1) (f, g) = \int_a^b f'(x)g'(x) dx,$$

$$(2) (f, g) = \int_a^b f'(x)g'(x) dx + f(a)g(a),$$

它们是否构成内积?

15. 用 Schwarz 不等式 (3.4.5) 估计 $\int_0^1 \frac{x^6}{1+x} dx$ 的上界, 并用积分中值定理估计同一积分的上、下界, 并比较其结果。

16. 选择 a , 使积分 $\int_{-1}^1 (x - ax^2)^2 dx$, $\int_{-1}^1 |x - ax^2| dx$ 取得最小值。

17. 设 $\varphi_1 = \text{span}\{1, x\}$, $\varphi_2 = \text{span}\{x^{100}, x^{101}\}$, 分别在 φ_1, φ_2 上求一元素, 使其为 $x^2 \in C[0, 1]$ 的最佳平方逼近, 并比较其结果。

18. $f(x) = |x|$ 在 $[-1, 1]$ 上, 求在 $\varphi_1 = \text{span}\{1, x^2, x^4\}$ 上的最佳平方逼近。

19. $u_n(x) = \frac{\sin[(n+1)\arccos x]}{\sqrt{1-x^2}}$ 是第二类 Chebyshev 多项式, 证明它有递推关系

$$u_{n+1}(x) = 2xu_n(x) - u_{n-1}(x).$$

20. 将 $f(x) = \sin \frac{1}{2}x$ 在 $[-1, 1]$ 上按 Legendre 多项式及 Chebyshev 多项式展开, 求三次最佳平方逼近多项式并画出误差图形, 再计算均方误差。
21. 把 $f(x) = \arccos x$ 在 $[-1, 1]$ 上展开成 Chebyshev 级数。
22. 用最小二乘法求一个形如 $y = a + bx^2$ 的经验公式, 使它与表 3.6 所示数据相拟合, 并求均方误差。

校注 (agent 补充): 题 10 的定义左端原书为 $T_n(x)$, 后面要求的是 $T_0^*, T_1^*, T_2^*, T_3^*$, 且题 11 使用 T_n^* ; 定义左端疑漏星号, 应为 $T_n^*(x) = T_n(2x - 1)$ 。题 15 的 Schwarz 不等式引用原书印为 (3.4.5), 本章 Schwarz 不等式的编号为 (3.3.5), 疑为交叉引用误印。两处均保留原文, 见 [题 10 原图裁切](#)、[题 15 原图裁切](#)。

原书 PDF 页 92；印刷页码 79。

[查看原页](#)

表 3.6

x_i	19	25	31	38	44
y_i	19.0	32.3	49.0	73.3	97.8

23. 观察物体的直线运动，得出时间 t 与距离 s 的关系如表 3.7 所示，求运动方程。

表 3.7

t/s	0	0.9	1.9	3.0	3.9	5.0
s/m	0	10	30	50	80	110

24. 在某化学反应里，根据实验所得分解物的质量分数 y 与时间 t 的关系如表 3.8 所示，用最小二乘拟合求 $y = F(t)$ 。

表 3.8

t/min	0	5	10	15	20	25
$y/\times 10^{-4}$	0	1.27	2.16	2.86	3.44	3.87
t/min	30	35	40	45	50	55
$y/\times 10^{-4}$	4.15	4.37	4.51	4.58	4.62	4.64

25. 编出用正交多项式作最小二乘拟合的程序框图。
26. 编出改进 FFT 算法的程序框图。
27. 已知 $\{x_k\} = (4, 3, 2, 1, 0, 1, 2, 3)$ ，用改进 FFT 算法求 $\{x_k\}$ 离散频谱 $\{C_k\}$ ($k = 0, 1, \cdots, 7$)。

原书 PDF 页 93; 印刷页码 80。

[原页图像](#)

第 4 章数值积分与数值微分

4.1 引言

4.1.1 数值求积的基本思想

有很多实际问题常常需要计算积分才能求解。有些数值方法，如微分方程和积分方程的求解，也都和积分计算有联系。

依据人们所熟知的微积分基本定理，对于积分 $I = \int_a^b f(x) dx$ ，只要找到被积函数 $f(x)$ 的原函数 $F(x)$ ，便有下列 Newton-Leibniz 公式：

$$\int_a^b f(x) dx = F(b) - F(a).$$

但实际使用这种求积方法往往有困难，因为大量的被积函数，诸如 $\frac{\sin x}{x}$ ， $\sin x^2$ 等等，找不到用初等函数表示的原函数；另外，当 $f(x)$ 是由测量或数值计算给出的一张数据表时，Newton-Leibniz 公式也不能直接运用。因此有必要研究积分的数值计算问题。

积分中值定理告诉我们，在积分区间 (a, b) 内存在一点 ξ ，有下式成立：

$$\int_a^b f(x) dx = (b - a)f(\xi).$$

就是说，底为 $b - a$ 而高为 $f(\xi)$ 的矩形面积恰等于所求曲边梯形的面积 I （见图 4.1）。问题在于点 ξ 的具体位置一般是不知道的，因而难以准确算出 $f(\xi)$ 的值。我们将 $f(\xi)$ 称为区间 $[a, b]$ 上的平均高度。这样，只要对平均高度 $f(\xi)$ 提供一种算法，相应地便获得一种数值求积方法。

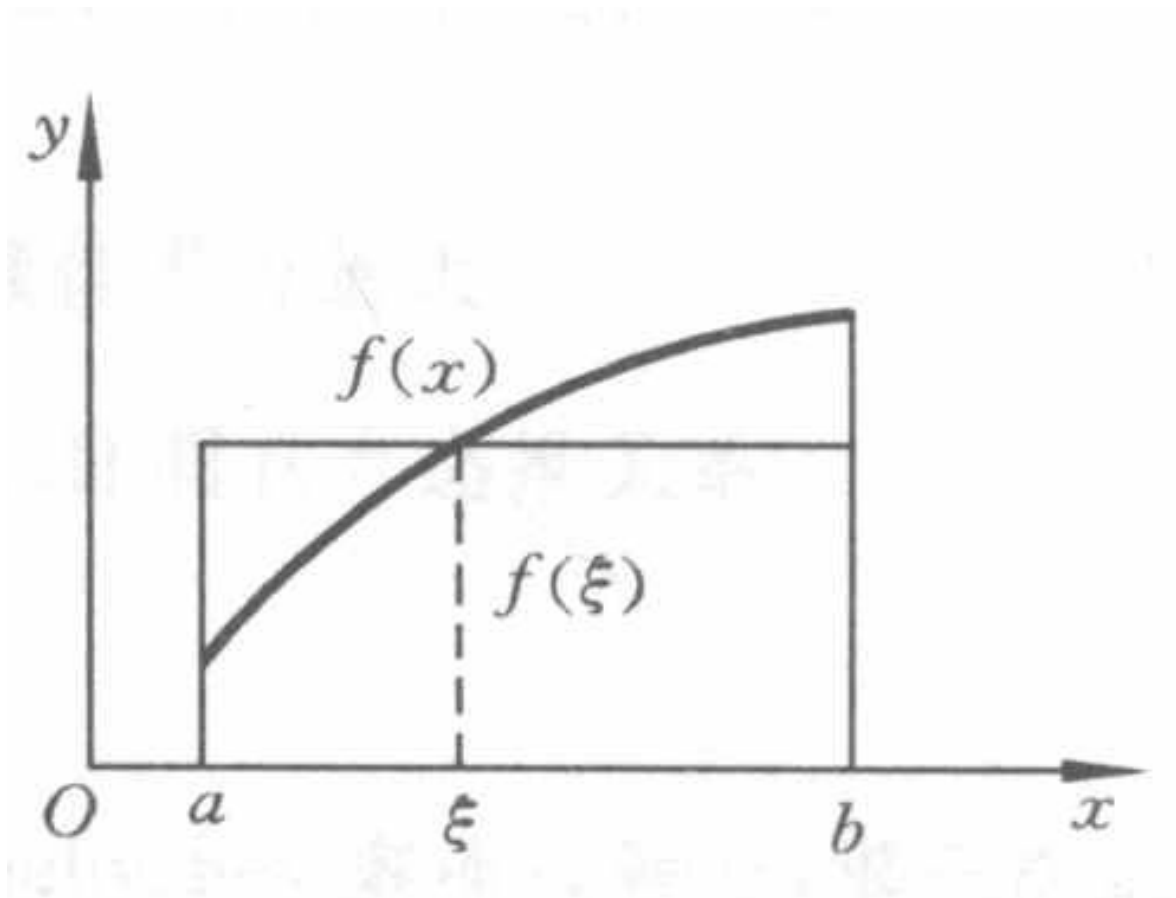


图 4.1

图 4.1 (原图)。图意说明 (agent 补充): 曲线 $f(x)$ 下方从 a 到 b 的曲边梯形, 与高为 $f(\xi)$ 的矩形面积相等。图中符号标签: $x, y, O, a, \xi, b, f(x), f(\xi)$ 。

如果用两端点的“高度” $f(a)$ 与 $f(b)$ 取算术平均作为平均高度 $f(\xi)$ 的近似值, 这样导出的求积公式

$$T = \frac{b-a}{2}[f(a) + f(b)] \quad (4.1.1)$$

便是我们所熟悉的梯形公式 (几何意义参见图 4.2)。如果改用区间中点 $c = \frac{a+b}{2}$ 的

原书 PDF 页 94；印刷页码 81。

[原页图像](#)

“高度” $f(c)$ 近似地取代平均高度 $f(\xi)$ ，则又可导出所谓中矩形公式（以下简称矩形公式）

$$R = (b - a)f\left(\frac{a + b}{2}\right). \quad (4.1.2)$$

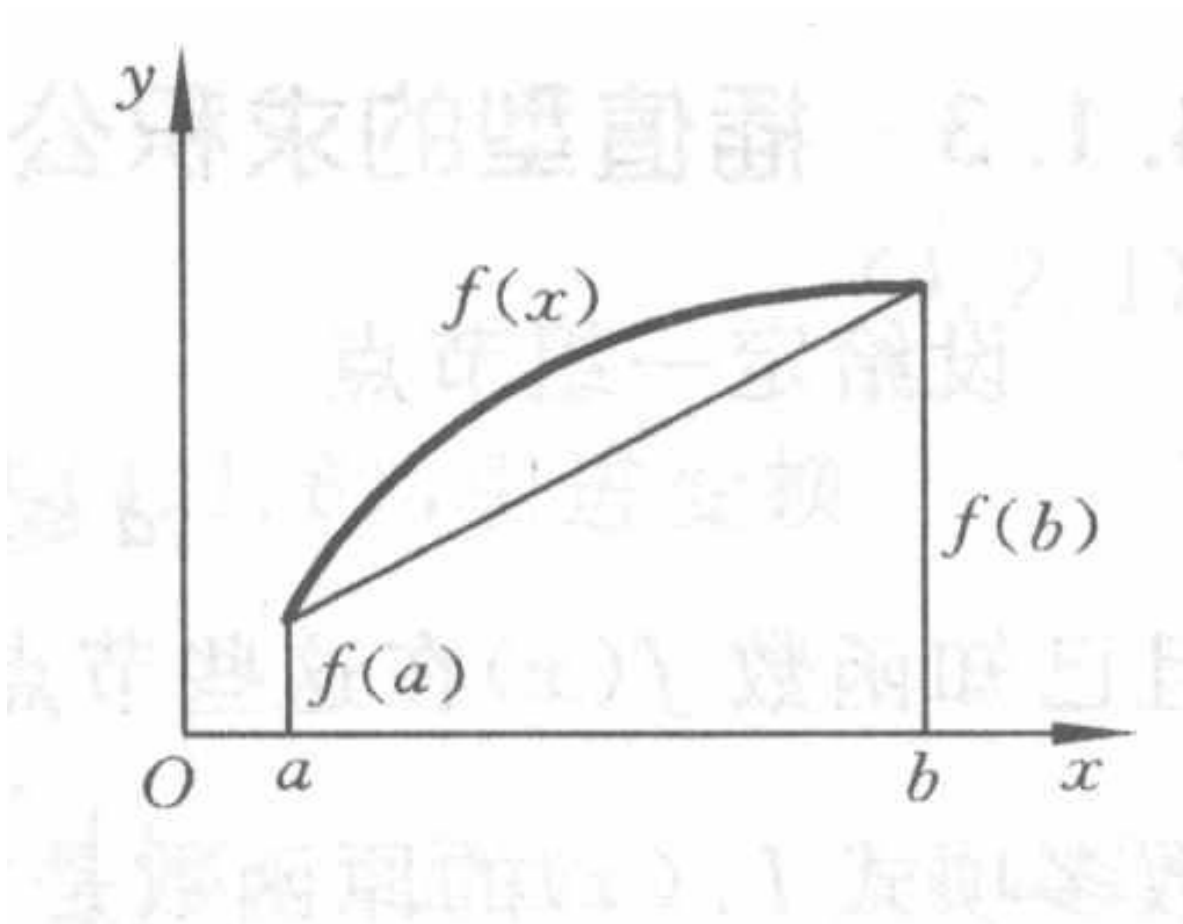


图 4.2

图 4.2（原图）。图意说明（agent 补充）：用连接曲线 $f(x)$ 在 a, b 两端点的线段形成的梯形近似曲边梯形。图中符号标签： $x, y, O, a, b, f(x), f(a), f(b)$ 。

更一般地，可以在区间 $[a, b]$ 上适当选取某些节点 x_k ，然后用 $f(x_k)$ 加权平均得到平均高度 $f(\xi)$ 的近似值，这样构造出的求积公式具有下列形式：

$$\int_a^b f(x) dx \approx \sum_{k=0}^n A_k f(x_k). \quad (4.1.3)$$

其中 x_k 称为求积节点； A_k 称为求积系数，亦称伴随节点 x_k 的权。权 A_k 仅仅与节点 x_k 的选取有关，而不依赖于被积函数 $f(x)$ 的具体形式。

这类数值积分方法通常称为机械求积，其特点是将积分求值问题归结为函数值的计算，这就避开了 Newton-Leibniz 公式需要寻求原函数的困难。

4.1.2 代数精度的概念

数值求积方法是近似方法，为保证精度，自然希望求积公式对于“尽可能多”的函数都能准确地成立，这就提出了所谓代数精度的概念。

定义 4.1 如果某个求积公式对于次数不大于 m 的多项式均能准确地成立，但对于 $m+1$ 次多项式就不一定准确，则称该求积公式具有 m 次代数精度。

不难验证，梯形公式 (4.1.1) 和矩形公式 (4.1.2) 均具有一次代数精度。

一般地，欲使求积公式 (4.1.3) 具有 m 次代数精度，只要令它对于 $f(x) = 1, x, x^2, \dots, x^m$ 都能准确地成立，这就要求

$$\begin{cases} \sum A_k = b - a, \\ \sum A_k x_k = \frac{1}{2}(b^2 - a^2), \\ \vdots \\ \sum A_k x_k^m = \frac{1}{m+1}(b^{m+1} - a^{m+1}). \end{cases} \quad (4.1.4)$$

为简洁起见，这里省略了符号 $\sum_{k=0}^n$ 中的上、下限。

如果事先选定求积节点 x_k ，譬如，以区间 $[a, b]$ 的等距分点作为节点，这时取 $m = n$ 求解方程组 (4.1.4) 即可确定求积系数 A_k ，而使求积公式 (4.1.3) 至少具有 n 次代数精度。4.2 节将介绍这样一类求积公式，梯形公式作为其中的一个特例。

构造出形如式 (4.1.3) 的求积公式的问题，原则上是个确定参数 x_k 和 A_k 的代数问题。

原书 PDF 页 95; 印刷页码 82。

[原页图像](#)

4.1.3 插值型的求积公式

设给定一组节点

$$a \leq x_0 < x_1 < x_2 < \cdots < x_n \leq b$$

且已知函数 $f(x)$ 在这些节点上的值, 作插值函数 $L_n(x)$ (参见式 (2.2.11))。由于代数多项式 $L_n(x)$ 的原函数是容易求出的, 取 $I_n = \int_a^b L_n(x) dx$ 作为积分 $I = \int_a^b f(x) dx$ 的近似值, 这样构造出的求积公式

$$I_n = \sum_{k=0}^n A_k f(x_k) \quad (4.1.5)$$

是插值型的, 其中求积系数 A_k 通过插值基函数 $l_k(x)$ 积分

$$A_k = \int_a^b l_k(x) dx \quad (4.1.6)$$

得出。

由插值余项定理 (定理 2.2) 即知, 对于插值型的求积公式 (4.1.5), 其余项

$$R[f] = I - I_n = \int_a^b \frac{f^{n+1}(\xi)}{(n+1)!} w(x) dx, \quad (4.1.7)$$

其中 ξ 与变量 x 有关, $w(x) = (x - x_0)(x - x_1) \cdots (x - x_n)$ 。

如果求积公式 (4.1.5) 是插值型的, 按式 (4.1.7), 对于次数不大于 n 的多项式 $f(x)$, 其余项 $R[f]$ 等于零, 因而这时求积公式至少具有 n 次代数精度。

反之, 如果求积公式 (4.1.5) 至少具有 n 次代数精度, 则它必定是插值型的。事实上, 这时式 (4.1.5) 对于插值基函数 $l_k(x)$ 应准确成立, 即有

$$\int_a^b l_k(x) dx = \sum_{j=0}^n A_j l_k(x_j).$$

注意到 $l_k(x_j) = \delta_{kj}$, 上式右端实际上即等于 A_k , 因而式 (4.1.6) 成立。

综上所述, 得到如下结论。

定理 4.1 形如式 (4.1.5) 的求积公式至少有 n 次代数精度的充分必要条件是, 它是插值型的。

校注 (agent 补充): 式 (4.1.7) 原扫描将导数阶数印作 $f^{n+1}(\xi)$, 未带括号; 此处按原式保留。由其引用的插值余项定理及上下文, 这里表示 $n+1$ 阶导数。

4.2 Newton-Cotes 公式

4.2.1 Cotes 系数

设将积分区间 $[a, b]$ 划分为 n 等份, 步长 $h = \frac{b-a}{n}$, 选取等距节点 $x_k = a + kh$ 构

原书 PDF 页 96；印刷页码 83。

[原页图像](#)

造出的插值型求积公式

$$I_n = (b-a) \sum_{k=0}^n C_k^{(n)} f(x_k) \quad (4.2.1)$$

称为 Newton-Cotes 公式，其中 $C_k^{(n)}$ 称为 Cotes 系数。按式 (4.1.6)，引进变换

$$x = a + th,$$

则有

$$C_k^{(n)} = \frac{h}{b-a} \int_0^n \prod_{\substack{j=0 \\ j \neq k}}^n \frac{t-j}{k-j} dt = \frac{(-1)^{n-k}}{nk!(n-k)!} \int_0^n \prod_{\substack{j=0 \\ j \neq k}}^n (t-j) dt. \quad (4.2.2)$$

由于是多项式的积分，Cotes 系数的计算不会遇到实质性的困难。当 $n=1$ 时，

$$C_0^{(1)} = C_1^{(1)} = \frac{1}{2},$$

这时求积公式是我们所熟悉的梯形公式 (4.1.1)。

当 $n=2$ 时，按式 (4.2.2)，Cotes 系数为

$$C_0^{(2)} = \frac{1}{4} \int_0^2 (t-1)(t-2) dt = \frac{1}{6},$$

$$C_1^{(2)} = -\frac{1}{2} \int_0^2 t(t-2) dt = \frac{4}{6}, \quad C_2^{(2)} = \frac{1}{4} \int_0^2 t(t-1) dt = \frac{1}{6}.$$

相应的求积公式是下列 Simpson 公式：

$$S = \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right]. \quad (4.2.3)$$

而 $n=4$ 的 Newton-Cotes 公式则特别称为 Cotes 公式，其形式为

$$C = \frac{b-a}{90} [7f(x_0) + 32f(x_1) + 12f(x_2) + 32f(x_3) + 7f(x_4)]. \quad (4.2.4)$$

这里 $x_k = a + kh$, $h = \frac{b-a}{4}$ 。

表 4.1 列出 Cotes 系数表开头的一部分。

表 4.1

n	$C_0^{(n)}$	$C_1^{(n)}$	$C_2^{(n)}$	$C_3^{(n)}$	$C_4^{(n)}$	$C_5^{(n)}$
1	$\frac{1}{2}$	$\frac{1}{2}$				
2	$\frac{1}{6}$	$\frac{2}{3}$	$\frac{1}{6}$			
3	$\frac{1}{8}$	$\frac{3}{8}$	$\frac{3}{8}$	$\frac{1}{8}$		
4	$\frac{7}{90}$	$\frac{16}{45}$	$\frac{2}{15}$	$\frac{16}{45}$	$\frac{7}{90}$	
5	$\frac{19}{288}$	$\frac{25}{96}$	$\frac{25}{144}$	$\frac{25}{144}$	$\frac{25}{96}$	$\frac{19}{288}$

原书 PDF 页 97; 印刷页码 84。

[原页图像](#)

表 4.1 (续表)

n	$C_0^{(n)}$	$C_1^{(n)}$	$C_2^{(n)}$	$C_3^{(n)}$	$C_4^{(n)}$	$C_5^{(n)}$	$C_6^{(n)}$	$C_7^{(n)}$	$C_8^{(n)}$
6	$\frac{41}{840}$	$\frac{9}{35}$	$\frac{9}{280}$	$\frac{34}{105}$	$\frac{9}{280}$	$\frac{9}{35}$	$\frac{41}{840}$		
7	$\frac{751}{17280}$	$\frac{3577}{17280}$	$\frac{1323}{17280}$	$\frac{2989}{17280}$	$\frac{2989}{17280}$	$\frac{1323}{17280}$	$\frac{3577}{17280}$	$\frac{751}{17280}$	
8	$\frac{989}{28350}$	$\frac{5888}{28350}$	$\frac{-928}{28350}$	$\frac{10496}{28350}$	$\frac{-4540}{28350}$	$\frac{10496}{28350}$	$\frac{-928}{28350}$	$\frac{5888}{28350}$	$\frac{989}{28350}$

可以看到, 当 $n \geq 8$ 时, Cotes 系数有正有负, 这时稳定性得不到保证。因此, 实际计算中不用高阶的 Newton-Cotes 公式。

校注 (agent 补充, CH04A-E01): 上述 “当 $n \geq 8$ 时” 若理解为对所有这类整数 n 成立, 则存在原书表述错误。保留原文。由式 (4.2.2) 直接积分, $n = 9$ 时系数为

$$\left(\frac{2857}{89600}, \frac{15741}{89600}, \frac{27}{2240}, \frac{1209}{5600}, \frac{2889}{44800}, \frac{2889}{44800}, \frac{1209}{5600}, \frac{27}{2240}, \frac{15741}{89600}, \frac{2857}{89600} \right),$$

全部为正; $n = 8$ 出现负系数这一事实仍成立。上述反例为 agent 按原公式进行的符号积分核验, 不属于原书正文。

4.2.2 偶阶求积公式的代数精度

作为插值型的求积公式, n 阶的 Newton-Cotes 公式至少具有 n 次的代数精度 (见定理 4.1)。实际的代数精度能否进一步提高呢?

先看 Simpson 公式 (4.2.3), 它是二阶 Newton-Cotes 公式, 因此至少具有二次代数精度。进一步用 $f(x) = x^3$ 进行检验, 按 Simpson 公式计算, 得

$$S = \frac{b-a}{6} \left[a^3 + 4 \left(\frac{a+b}{2} \right)^3 + b^3 \right].$$

另一方面直接求积得 $I = \int_a^b x^3 dx = \frac{b^4 - a^4}{4}$ 。这时有 $S = I$, 即 Simpson 公式对次数不超过三次的多项式均能准确成立。又容易验证它对于 $f(x) = x^4$ 通常是不准确的, 因此, Simpson 公式实际上具有三次代数精度。

一般地, 可以证明下述论断。

定理 4.2 当阶 n 为偶数时, Newton-Cotes 公式 (4.2.1) 至少有 $n+1$ 次代数精度。

证明 只要验证, 当 n 为偶数时, Newton-Cotes 公式对 $f(x) = x^{n+1}$ 的余项为零。

按余项公式 (4.1.7), 由于这里 $f^{(n+1)}(x) = (n+1)!$, 从而有

$$R[f] = \int_a^b \prod_{j=0}^n (x - x_j) dx.$$

引进变换 $x = a + th$, 并注意到 $x_j = a + jh$, 有

$$R[f] = h^{n+2} \int_0^n \prod_{j=0}^n (t - j) \, dt.$$

若 n 为偶数, 则 $\frac{n}{2}$ 为整数, 再令 $t = u + \frac{n}{2}$, 进一步有

$$R[f] = h^{n+2} \int_{-n/2}^{n/2} \prod_{j=0}^n \left(u + \frac{n}{2} - j\right) \, du,$$

原书 PDF 页 98; 印刷页码 85。

[原页图像](#)

据此可以断定 $R[f] = 0$, 因为被积函数

$$H(u) = \prod_{j=0}^n \left(u + \frac{n}{2} - j\right) = \prod_{j=-n/2}^{n/2} (u - j)$$

是个奇函数。证毕。

4.2.3 几种低阶求积公式的余项

首先考察梯形公式, 按余项公式 (4.1.7), 梯形公式 (4.1.1) 的余项为

$$R_T = I - T = \int_a^b \frac{f''(\xi)}{2} (x-a)(x-b) dx.$$

这里积分的核函数 $(x-a)(x-b)$ 在区间 $[a, b]$ 上保号 (非正), 应用积分中值定理, 在 (a, b) 内存在一点 η , 使

$$R_T = \frac{f''(\eta)}{2} \int_a^b (x-a)(x-b) dx = -\frac{f''(\eta)}{12} (b-a)^3, \quad \eta \in (a, b). \quad (4.2.5)$$

再研究 Simpson 公式 (4.2.3) 的余项 $R_S = I - S$ 。为此构造次数不大于三的多项式 $H(x)$, 使之满足

$$H(a) = f(a), \quad H(b) = f(b), \quad H(c) = f(c), \quad H'(c) = f'(c), \quad (4.2.6)$$

这里 $c = \frac{a+b}{2}$ 。由于 Simpson 公式具有三次代数精度, 它对于这样构造出的三次式 $H(x)$ 是准确的:

$$\int_a^b H(x) dx = \frac{b-a}{6} [H(a) + 4H(c) + H(b)].$$

而由插值条件 (4.2.6) 知, 上式右端实际上等于按 Simpson 公式 (4.2.3) 求得的积分值 S , 因此积分余项

$$R_S = I - S = \int_a^b [f(x) - H(x)] dx.$$

不难证明, 对于满足条件 (4.2.6) 的多项式 $H(x)$, 其插值余项

$$f(x) - H(x) = \frac{f^{(4)}(\xi)}{4!} (x-a)(x-c)^2(x-b),$$

故有

$$R_S = \int_a^b \frac{f^{(4)}(\xi)}{4!} (x-a)(x-c)^2(x-b) dx.$$

这时积分核 $(x-a)(x-c)^2(x-b)$ 在 $[a, b]$ 上保号 (非正), 用积分中值定理有

$$R_S = \frac{f^{(4)}(\eta)}{4!} \int_a^b (x-a)(x-c)^2(x-b) dx = -\frac{b-a}{180} \left(\frac{b-a}{2}\right)^4 f^{(4)}(\eta). \quad (4.2.7)$$

关于 Cotes 公式 (4.2.4) 的积分余项, 这里不再具体推导, 仅列出结果如下:

$$R_C = I - C = -\frac{2(b-a)}{945} \left(\frac{b-a}{4}\right)^6 f^{(6)}(\eta). \quad (4.2.8)$$

原书 PDF 页 99; 印刷页码 86。

[原页图像](#)

4.2.4 复化求积法及其收敛性

前已指出, 在使用 Newton-Cotes 公式时, 提高阶的途径并不总能取得满意的效果。为了改善求积的精度, 通常采用复化求积法。

设将积分区间 $[a, b]$ 划分为 n 等份, 步长 $h = \frac{b-a}{n}$, 分点为 $x_k = a + kh$, $k = 0, 1, \dots, n$ 。所谓复化求积法, 就是先用低阶的 Newton-Cotes 公式求得每个子区间 $[x_k, x_{k+1}]$ 上的积分值 I_k , 然后再求和, 用 $\sum_{k=0}^{n-1} I_k$ 作为所求积分 I 的近似值。

复化梯形公式的形式是

$$T_n = \sum_{k=0}^{n-1} \frac{h}{2} [f(x_k) + f(x_{k+1})] = \frac{h}{2} \left[f(a) + 2 \sum_{k=1}^{n-1} f(x_k) + f(b) \right], \quad (4.2.9)$$

依式 (4.2.5), 其积分余项

$$I - T_n = \sum_{k=0}^{n-1} \left[-\frac{h^3}{12} f''(\eta_k) \right] = -\frac{b-a}{12} h^2 f''(\eta). \quad (4.2.10)$$

此外, 记子区间 $[x_k, x_{k+1}]$ 的中点为 $x_{k+\frac{1}{2}}$, 则复化 Simpson 公式为

$$\begin{aligned} S_n &= \sum_{k=0}^{n-1} \frac{h}{6} [f(x_k) + 4f(x_{k+\frac{1}{2}}) + f(x_{k+1})] \\ &= \frac{h}{6} \left[f(a) + 4 \sum_{k=0}^{n-1} f(x_{k+\frac{1}{2}}) + 2 \sum_{k=1}^{n-1} f(x_k) + f(b) \right]. \end{aligned} \quad (4.2.11)$$

如果将每个子区间 $[x_k, x_{k+1}]$ 划分为 4 等分, 内分点依次记作 $x_{k+\frac{1}{4}}$, $x_{k+\frac{1}{2}}$, $x_{k+\frac{3}{4}}$, 则复化 Cotes 公式具有形式

$$\begin{aligned} C_n &= \frac{h}{90} \left[7f(a) + 32 \sum_{k=0}^{n-1} f(x_{k+\frac{1}{4}}) + 12 \sum_{k=0}^{n-1} f(x_{k+\frac{1}{2}}) \right. \\ &\quad \left. + 32 \sum_{k=0}^{n-1} f(x_{k+\frac{3}{4}}) + 14 \sum_{k=1}^{n-1} f(x_k) + 7f(b) \right]. \end{aligned} \quad (4.2.12)$$

依据式 (4.2.7) 与式 (4.2.8), 容易得到复化 Simpson 公式和复化 Cotes 公式的余项, 它们分别是

$$I - S_n = -\frac{b-a}{180} \left(\frac{h}{2} \right)^4 f^{(4)}(\eta), \quad \eta \in [a, b], \quad (4.2.13)$$

$$I - C_n = -\frac{2(b-a)}{945} \left(\frac{h}{4} \right)^6 f^{(6)}(\eta), \quad \eta \in [a, b]. \quad (4.2.14)$$

其他 Newton-Cotes 公式亦可用类似的手续加以复化。显然, 复化的 Newton-Cotes 公式仍然是形如式 (4.1.3) 的机械求积公式。

例 4.1 对于函数 $f(x) = \frac{\sin x}{x}$, 试利用表 4.2 计算积分 $I = \int_0^1 \frac{\sin x}{x} dx$ 。

解 将积分区间 $[0, 1]$ 划分为 8 等份, 应用复化梯形法求得 $T_8 = 0.945\,690\,9$; 而

原书 PDF 页 100; 印刷页码 87。

[原页图像](#)

如果将 $[0, 1]$ 划分为 4 等份, 应用复化 Simpson 法有 $S_4 = 0.946\,083\,2$ 。

比较上面两个结果 T_8 与 S_4 , 它们都需要提供 9 个点上的函数值, 计算量基本相同¹, 然而精度却差别很大, 同积分的准确值² $I = 0.946\,083\,1$ 比较, 复化梯形法的结果 $T_8 = 0.945\,690\,9$ 只有两位有效数字, 而复化 Simpson 法的结果 $S_4 = 0.946\,083\,2$ 却有六位有效数字。

校注 (agent 补充, CH04A-E02): 原文 “只有两位有效数字” 已保留。按本书 1.3.3 的式 (1.3.2), 以本页所列数值比较, $|I - T_8| = 0.0003922 < 0.0005$, 而大于 0.00005 ; 因此 T_8 应具有三位有效数字。此为数值判断疑误, 与原文转录状态分开记录。

容易证明, 复化的梯形法、Simpson 法和 Cotes 法当步长 $h \rightarrow 0$ 时均收敛到所求的积分值 I 。现在考察当 h 很小时误差的渐近性态。

表 4.2

x	$f(x)$
0	1
1/8	0.997 397 8
1/4	0.989 615 8
3/8	0.976 726 7
1/2	0.958 851 0
5/8	0.936 155 6
3/4	0.908 851 6
7/8	0.877 192 5
1	0.841 470 9

先研究梯形法, 按余项公式 (4.2.10), 有

$$\frac{I - T_n}{h^2} = -\frac{1}{12} \sum_{k=0}^{n-1} h f''(\eta_k) \rightarrow -\frac{1}{12} \int_a^b f''(x) dx,$$

从而当 $h \rightarrow 0$ 时有下列渐近关系式:

$$\frac{I - T_n}{h^2} \rightarrow -\frac{1}{12} [f'(b) - f'(a)].$$

类似地, 对于复化的 Simpson 法和 Cotes 法分别有

$$\frac{I - S_n}{h^4} \rightarrow -\frac{1}{180 \times 2^4} [f'''(b) - f'''(a)],$$

$$\frac{I - C_n}{h^6} \rightarrow -\frac{2}{945 \times 4^6} [f^{(5)}(b) - f^{(5)}(a)].$$

¹使用求积公式时, 计算的工作量主要耗费在函数求值上。

²所谓积分的 “准确值”, 是指每一位数字都是有效数字的积分值。

定义 4.2 如果一种复化求积公式 I_n 当 $h \rightarrow 0$ 时成立渐近关系式

$$\frac{I - I_n}{h^p} \rightarrow C \quad (C \neq 0, \text{定数}),$$

则称求积公式 I_n 是 p 阶收敛的。

在这种意义下, 复化的梯形法、Simpson 法和 Cotes 法分别具有二阶、四阶和六阶收敛精度。而当 h 很小时, 对于复化的梯形法、Simpson 法和 Cotes 法分别有下列误差估计式:

$$I - T_n \approx -\frac{h^2}{12}[f'(b) - f'(a)], \quad (4.2.15)$$

$$I - S_n \approx -\frac{1}{180} \left(\frac{h}{2}\right)^4 [f'''(b) - f'''(a)], \quad (4.2.16)$$

$$I - C_n \approx -\frac{2}{945} \left(\frac{h}{4}\right)^6 [f^{(5)}(b) - f^{(5)}(a)]. \quad (4.2.17)$$

由此可见, 若将步长 h 减半 (即等份数 n 加倍), 则梯形法、Simpson 法与 Cotes 法的误差分别减至原有误差的 $1/4$ 、 $1/16$ 与 $1/64$ 。

原书 PDF 页 101; 印刷页码 88。

[原页图像](#)

4.3 Romberg 算法

4.3.1 梯形法的递推化

上一节介绍的复化求积方法对提高精度是行之有效的, 但在使用求积公式之前必须给出合适的步长。步长太大, 精度难以保证; 步长太小, 又会导致计算量的增加。而事先给出一个恰当的步长往往是困难的。

实际计算中常常采用变步长的计算方案, 即在步长逐次分半 (即步长二分) 的过程中, 反复利用复化求积公式进行计算, 直至所求得的积分值满足精度要求为止。

下面, 我们在变步长的过程中探讨梯形法的计算规律。

设将求积区间 $[a, b]$ 分成 n 等份, 则一共有 $n+1$ 个分点, 按梯形公式 (4.2.9) 计算积分值 T_n , 需要提供 $n+1$ 个函数值。如果将求积区间再二分一次, 则分点增至 $2n+1$ 个。将二分前后两个积分值联系起来加以考察, 注意到每个子区间 $[x_k, x_{k+1}]$ 经过二分只增加了一个分点 $x_{k+\frac{1}{2}} = \frac{1}{2}(x_k + x_{k+1})$, 用复化梯形公式求得该子区间上的积分值为

$$\frac{h}{4}[f(x_k) + 2f(x_{k+\frac{1}{2}}) + f(x_{k+1})].$$

注意, 这里 $h = \frac{b-a}{n}$ 代表二分前的步长。将每个子区间上的积分值相加, 得

$$T_{2n} = \frac{h}{4} \sum_{k=0}^{n-1} [f(x_k) + f(x_{k+1})] + \frac{h}{2} \sum_{k=0}^{n-1} f(x_{k+\frac{1}{2}}),$$

从而利用式 (4.2.9) 可导出下列递推公式:

$$T_{2n} = \frac{1}{2}T_n + \frac{h}{2} \sum_{k=0}^{n-1} f(x_{k+\frac{1}{2}}). \quad (4.3.1)$$

例 4.2 计算积分值 $I = \int_0^1 \frac{\sin x}{x} dx$ 。

解 先对整个区间 $[0, 1]$ 使用梯形公式。对于函数 $f(x) = \frac{\sin x}{x}$, 它在 $x=0$ 的值定义为 $f(0) = 1$, 而 $f(1) = 0.8414709$, 据梯形公式计算得

$$T_1 = \frac{1}{2}[f(0) + f(1)] = 0.9207355.$$

然后将区间二等分, 再求出中点的函数值 $f(1/2) = 0.9588510$, 从而利用递推公式 (4.3.1), 有

原书 PDF 页 102；印刷页码 89。

[原页图像](#)

$$T_2 = \frac{1}{2}T_1 + \frac{1}{2}f\left(\frac{1}{2}\right) = 0.939\,793\,3.$$

进一步二分求积区间，并计算新分点上的函数值，得

$$f(1/4) = 0.989\,615\,8, \quad f(3/4) = 0.908\,851\,6.$$

再利用式 (4.3.1)，有

$$T_4 = \frac{1}{2}T_2 + \frac{1}{4}\left[f\left(\frac{1}{4}\right) + f\left(\frac{3}{4}\right)\right] = 0.944\,513\,5.$$

这样不断二分下去，计算结果如表 4.3 所示（表中 k 代表二分次数，区间等份数 $n = 2^k$ ）。

表 4.3

k	1	2	3	4	5
T_n	0.939 793 3	0.944 513 5	9.945 690 9	0.945 985 0	0.946 059 6

k	6	7	8	9	10
T_n	0.946 076 9	0.946 081 5	0.946 082 7	0.946 083 0	0.946 083 1

积分 I 的准确值为 0.946 083 1，用变步长方法二分 9 次得到了这个结果。

校注 (agent 补充, CH04A-E03)：表 4.3 的 $k = 3$ 单元格原扫描确实印为 9.945 690 9，这里按原表保留。由例 4.1 的 T_8 和本页下段的 T_8 可知首位应为 0，即 0.945 690 9，这是原书排印错误。

校注 (agent 补充, CH04A-E04)：本页说明 k 为二分次数，表中 $k = 9$ 为 0.946 083 0， $k = 10$ 才列出 0.946 083 1。故表后“二分 9 次得到了这个结果”与本页表格、次数定义不一致；原句已保留。

校注(agent 补充, CH04A-E07):表 4.3 的 $k = 5$ 单元格原扫描印为 0.946 059 6,这里按原表保留.对本例 $f(x) = \sin x/x$ ($f(0) = 1$) 在 $[0, 1]$ 上取 $n = 32$ 独立高精度计算复化梯形值，得 $T_{32} = 0.946\,058\,560\,962\,768\,067\,\dots$ ，保留七位小数应为 0.946 058 6；原表数值有误。

4.3.2 Romberg 公式

梯形法的算法简单，但精度较差，收敛的速度缓慢。如何提高收敛速度以节省计算量，自然是人们极为关心的课题。

根据梯形法的误差公式 (4.2.15)，积分值 T_n 的截断误差大致与 h^2 成正比，因此当步长二分后，截断误差将减至原有误差的 1/4，即有

$$\frac{I - T_{2n}}{I - T_n} \approx \frac{1}{4}.$$

将上式移项整理, 可得

$$I - T_{2n} \approx \frac{1}{3}(T_{2n} - T_n). \quad (4.3.2)$$

由此可见, 只要二分前后的两个积分值 T_n 与 T_{2n} 相当接近, 就可以保证计算结果 T_{2n} 的误差很小。这种直接用计算结果来估计误差的方法通常称为误差的事后估计法。

按式 (4.3.2), 积分近似值 T_{2n} 的误差大致等于 $\frac{1}{3}(T_{2n} - T_n)$, 因此, 如果用这个误差值作为 T_{2n} 的一种补偿, 可以期望, 所得到的

$$\bar{T} = T_{2n} + \frac{1}{3}(T_{2n} - T_n) = \frac{4}{3}T_{2n} - \frac{1}{3}T_n \quad (4.3.3)$$

可能是更好的结果。

再考察例 4.2, 所求得的两个梯形值 $T_4 = 0.9445135$ 和 $T_8 = 0.9456909$ 的精

原书 PDF 页 103; 印刷页码 90。

[原页图像](#)

度都很差（与准确值 $I = 0.946\,083\,1$ 比较，只有两三位有效数字），但如果将它们按式（4.3.3）作线性组合，则新的近似值

$$\bar{T} = \frac{4}{3}T_8 - \frac{1}{3}T_4 = 0.946\,083\,3$$

却有六位有效数字。

按公式（4.3.3）组合得到的近似值 $\bar{T}$ ，其实质究竟是什么呢？直接验证易知

$$S_n = \frac{4}{3}T_{2n} - \frac{1}{3}T_n, \quad (4.3.4)$$

这就是说，用梯形法二分前后的两个积分值 T_n 与 T_{2n} ，按式（4.3.3）作线性组合，结果得到 Simpson 法的积分值 S_n 。

再考察 Simpson 法，按误差公式（4.2.16），其截断误差大致与 h^4 成正比，因此，若将步长折半，则误差将减至原有误差的 $1/16$ ，即有

$$\frac{I - S_{2n}}{I - S_n} \approx \frac{1}{16}, \quad I \approx \frac{16}{15}S_{2n} - \frac{1}{15}S_n.$$

不难直接验证，上式右端的值其实等于 C_n ，就是说，用 Simpson 法二分前后的两个积分值 S_n 与 S_{2n} ，按式（4.3.3）作线性组合，结果得到 Cotes 法的积分值 C_n ，即

$$C_n = \frac{16}{15}S_{2n} - \frac{1}{15}S_n. \quad (4.3.5)$$

重复同样的手续，依据 Cotes 法的误差公式（4.2.17）可进一步导出下列 Romberg 公式：

$$R_n = \frac{64}{63}C_{2n} - \frac{1}{63}C_n. \quad (4.3.6)$$

在变步长的过程中运用式（4.3.4）、式（4.3.5）和式（4.3.6），就能将粗糙的梯形值 T_n 逐步加工成精度较高的 Simpson 值 S_n 、Cotes 值 C_n 和 Romberg 值 R_n 。

例 4.3 用加速公式（4.3.4）、（4.3.5）和（4.3.6）加工例 4.2 得到的梯形值，计算结果如表 4.4（ k 代表二分次数）所示。

表 4.4

k	T_2^k	S_2^{k-1}	C_2^{k-2}	R_2^{k-3}
0	0.920 735 5			
1	0.939 793 3	0.946 145 9		
2	0.944 513 5	0.946 086 9	0.946 083 0	
3	0.945 690 9	0.946 083 3	0.946 083 1	0.946 083 1

我们看到，这里利用二分 3 次的结果（它们的精度都很差，只有两三位是有效数字），通过三次加速求得 $R_1 = 0.946\,083\,1$ ，这个结果的每一位数字都是有效数字，可见加速效果是十分显著的。

校注 (agent 补充): 表 4.4 的上、下标按原扫描字位保留为 T_2^k 、 S_2^{k-1} 、 C_2^{k-2} 、 R_2^{k-3} 。由本页 “ k 代表二次数” 及公式 (4.3.4) 至 (4.3.6), 各列数据对应的分段数实际上为 2^k 、 2^{k-1} 、 2^{k-2} 、 2^{k-3} , 即通常记作 T_{2^k} 、 $S_{2^{k-1}}$ 、 $C_{2^{k-2}}$ 、 $R_{2^{k-3}}$ 。另本页由 Simpson 值组合为 Cotes 值的文字仍引用 “式 (4.3.3)”, 已按原文保留; 该处实际所列权为 $16/15$ 和 $-1/15$, 见式 (4.3.5)。

原书 PDF 页 104; 印刷页码 91。

[原页图像](#)

4.3.3 Richardson 外推加速法

上述加速过程还可以再继续下去, 其理论依据是梯形法的余项可展开成下列级数形式。

定理 4.3 设 $f(x) \in C^\infty[a, b]$, 则成立

$$T(h) = I + a_1 h^2 + a_2 h^4 + a_3 h^6 + \cdots + a_k h^{2k} + \cdots, \quad (4.3.7)$$

其中系数 a_k ($k = 1, 2, \dots$) 与 h 无关。

这一余项公式的推导将在 4.3.4 节给出。

按式 (4.3.7), 有

$$T\left(\frac{h}{2}\right) = I + \frac{\alpha_1}{4} h^2 + \frac{\alpha_2}{16} h^4 + \frac{\hat{\alpha}_3}{64} h^6 + \cdots. \quad (4.3.8)$$

注意, 这里的 $\hat{\alpha}_k$ 与将要出现的 β_k, γ_k 等均为与 h 无关的系数。

设将式 (4.3.7) 与式 (4.3.8) 按以下方式作线性组合:

$$T_1(h) = \frac{4}{3} T\left(\frac{h}{2}\right) - \frac{1}{3} T(h), \quad (4.3.9)$$

则可以从余项展开式中消去误差的主要部分 h^2 项, 从而得到

$$T_1(h) = I + \beta_1 h^4 + \beta_2 h^6 + \beta_3 h^8 + \cdots. \quad (4.3.10)$$

比较式 (4.3.9) 与式 (4.3.4) 知, 这样构造出的 $\{T_1(h)\}$ 其实就是 Simpson 值序列。

又根据式 (4.3.10), 有

$$T_1\left(\frac{h}{2}\right) = I + \frac{\beta_1}{16} h^4 + \hat{\beta}_2 h^6 + \beta_3 h^8 + \cdots,$$

若令

$$T_2(h) = \frac{16}{15} T_1\left(\frac{h}{2}\right) - \frac{1}{15} T_1(h),$$

则又可进一步从余项展开式中消去 h^4 项, 从而有

$$T_2(h) = I + \gamma_1 h^6 + \gamma_2 h^8 + \cdots.$$

这样构造出的 $\{T_2(h)\}$ 其实就是 Cotes 值序列。

如此继续下去，每加速一次，误差的量级便提高二阶。一般地，将 $T_0(h) = T(h)$ 按公式

$$T_m(h) = \frac{4^m}{4^m - 1} T_{m-1}\left(\frac{h}{2}\right) - \frac{1}{4^m - 1} T_{m-1}(h) \quad (4.3.11)$$

经过 m ($m = 1, 2, \dots$) 次加速后，余项便取下列形式：

$$T_m(h) = I + \delta_1 h^{2(m+1)} + \delta_2 h^{2(m+2)} + \dots \quad (4.3.12)$$

上述处理方法通常称为 Richardson 外推加速方法。

设以 $T_0^{(k)}$ 表示二分 k 次后求得的梯形值，且以 $T_m^{(k)}$ 表示序列 $\{T_0^{(k)}\}$ 的 m 次加速值，则依递推公式 (4.3.11)，即

$$T_m^{(k)} = \frac{4^m}{4^m - 1} T_{m-1}^{(k+1)} - \frac{1}{4^m - 1} T_{m-1}^{(k)} \quad (k = 1, 2, \dots) \quad (4.3.13)$$

校注 (agent 补充)：式 (4.3.8) 的系数按原扫描保留为 α_1 、 α_2 、 $\hat{\alpha}_3$ ，相邻正文为 $\hat{\alpha}_k$ ； $T_1(h/2)$ 的展开式按原扫描保留 $\hat{\beta}_2$ 与未带帽的 β_3 。这些字形已放大核对，未用常见写法替换原符号。

校注 (agent 补充，CH04A-E05)：式 (4.3.13) 原列 $k = 1, 2, \dots$ ，已照录；但下一页的三角表及算法需要计算 $T_m^{(0)}$ ，因而实现同一递推式时也须允许 $k = 0$ 。这是原书递推指标范围的缺漏。

校注 (agent 补充，CH04A-E06)：仅有 $f \in C^\infty[a, b]$ 并不能保证式 (4.3.7) 的无穷级数收敛且等于 $T(h)$ 。在外推分析中应将其理解为 $h \rightarrow 0$ 时的渐近展开，即对所需有限阶截断并保留余项；本转录保留原书等式和条件。下一页式 (4.3.14) 的无穷 Taylor 等式也有同样的光滑性与收敛性区别。

原书 PDF 页 105; 印刷页码 92。

[原页图像](#)

可以逐行构造出下列三角形数表—— T 数表:

$$\begin{array}{ccccccc} & & & & & & T_0^{(0)} \\ & & & & & & T_0^{(1)} & T_1^{(0)} \\ & & & & & T_0^{(2)} & T_1^{(1)} & T_2^{(0)} \\ & & & T_0^{(3)} & T_1^{(2)} & T_2^{(1)} & T_3^{(0)} \\ & & \vdots & \vdots & \vdots & \vdots & \ddots \end{array}$$

可以证明, 如果 $f(x)$ 充分光滑, 那么 T 数表每一列的元素及对角线元素均收敛到所求的积分值 I , 即

$$\lim_{k \rightarrow \infty} T_m^{(k)} = I \quad (m \text{ 固定}), \quad \lim_{m \rightarrow \infty} T_m^{(0)} = I.$$

电子计算机上的所谓 Romberg 算法, 就是在二分过程中逐步形成 T 数表的具体方法, 其步骤如下:

步 1 准备初值。计算 $T_0^{(0)} = \frac{b-a}{2}[f(a) + f(b)]$ 且令 $1 \rightarrow k$ (k 记录二分的次数)。

步 2 求梯形值。按递推公式 (4.3.1) 计算梯形值 $T_0^{(k)}$ 。

步 3 求加速值。按加速公式 (4.3.13) 逐个求出 T 数表第 $k+1$ 行其余各元素 $T_j^{(k-j)}$ ($j = 1, 2, \dots, k$)。

步 4 精度控制。对于指定精度 ε , 若 $|T_k^{(0)} - T_{k-1}^{(0)}| < \varepsilon$, 则终止计算, 并取 $T_k^{(0)}$ 作为所求的结果; 否则令 $k+1 \rightarrow k$ (意即二分一次), 转步 2 继续计算。

4.3.4 梯形法的余项展开式

前述外推方法的基础是梯形法的余项展开式 (4.3.7), 现在运用 Taylor 展开方法推导这个公式。

将 $f(x)$ 在子区间 $[x_k, x_{k+1}]$ 的中点 $x_{k+\frac{1}{2}} = \frac{1}{2}(x_k + x_{k+1})$ 展开, 有

$$\begin{aligned} f(x) = & f_{k+\frac{1}{2}} + (x - x_{k+\frac{1}{2}})f'_{k+\frac{1}{2}} + \frac{(x - x_{k+\frac{1}{2}})^2}{2!}f''_{k+\frac{1}{2}} + \frac{(x - x_{k+\frac{1}{2}})^3}{3!}f'''_{k+\frac{1}{2}} \\ & + \frac{(x - x_{k+\frac{1}{2}})^4}{4!}f^{(4)}_{k+\frac{1}{2}} + \frac{(x - x_{k+\frac{1}{2}})^5}{5!}f^{(5)}_{k+\frac{1}{2}} + \frac{(x - x_{k+\frac{1}{2}})^6}{6!}f^{(6)}_{k+\frac{1}{2}} + \dots, \end{aligned} \quad (4.3.14)$$

其中 $f_{k+\frac{1}{2}}^{(j)}$ 是 $f^{(j)}(x_{k+\frac{1}{2}})$ 的缩写。据此写出 $f(x_k)$ 和 $f(x_{k+1})$ 的展开式, 易知

$$\frac{h}{2}[f(x_k) + f(x_{k+1})] = hf_{k+\frac{1}{2}} + \frac{h}{2!}\left(\frac{h}{2}\right)^2 f''_{k+\frac{1}{2}} + \frac{h}{4!}\left(\frac{h}{2}\right)^4 f^{(4)}_{k+\frac{1}{2}} + \frac{h}{6!}\left(\frac{h}{2}\right)^6 f^{(6)}_{k+\frac{1}{2}} + \dots,$$

求和得

$$\begin{aligned} T(h) &= \frac{h}{2} \sum [f(x_k) + f(x_{k+1})] \\ &= h \sum f_{k+\frac{1}{2}} + \frac{h^3}{2! \times 2^2} \sum f''_{k+\frac{1}{2}} + \frac{h^5}{4! \times 2^4} \sum f^{(4)}_{k+\frac{1}{2}} + \frac{h^7}{6! \times 2^6} \sum f^{(6)}_{k+\frac{1}{2}} + \dots. \end{aligned} \quad (4.3.15)$$

原书 PDF 页 106; 印刷页码 93。

[原页图像](#)

4.3.4 梯形法的余项展开式 (续)

为简洁起见, 这里省略了符号 $\sum_{k=0}^{n-1}$ 中的上、下限。

另一方面, 将式 (4.3.14) 在子区间 $[x_k, x_{k+1}]$ 上求积分, 有

$$\int_{x_k}^{x_{k+1}} f(x) dx = hf_{k+\frac{1}{2}} + \frac{2}{3!} \left(\frac{h}{2}\right)^3 f''_{k+\frac{1}{2}} + \frac{2}{5!} \left(\frac{h}{2}\right)^5 f^{(4)}_{k+\frac{1}{2}} + \frac{2}{7!} \left(\frac{h}{2}\right)^7 f^{(6)}_{k+\frac{1}{2}} + \cdots,$$

然后再关于 k 从 0 到 $n-1$ 求和, 得

$$\begin{aligned} I &= \int_a^b f(x) dx \\ &= h \sum f_{k+\frac{1}{2}} + \frac{h^3}{3! \times 2^2} \sum f''_{k+\frac{1}{2}} + \frac{h^5}{5! \times 2^4} \sum f^{(4)}_{k+\frac{1}{2}} + \frac{h^7}{7! \times 2^6} \sum f^{(6)}_{k+\frac{1}{2}} + \cdots. \end{aligned} \quad (4.3.16)$$

利用式 (4.3.16) 从式 (4.3.15) 中消去项 $h \sum f_{k+\frac{1}{2}}$, 得

$$T(h) = I + \frac{h^2}{2! \times 6} \sum f''_{k+\frac{1}{2}} + \frac{h^5}{4! \times 20} \sum f^{(4)}_{k+\frac{1}{2}} + \frac{3h^7}{6! \times 224} \sum f^{(6)}_{k+\frac{1}{2}} + \cdots. \quad (4.3.17)$$

又对 $f''(x)$ 应用式 (4.3.16), 并注意到

$$\int_a^b f''(x) dx = f'(b) - f'(a),$$

有

$$h \sum f''_{k+\frac{1}{2}} = f'(b) - f'(a) - \frac{h^3}{3! \times 2^2} \sum f^{(4)}_{k+\frac{1}{2}} - \frac{h^5}{5! \times 2^4} \sum f^{(6)}_{k+\frac{1}{2}} + \cdots,$$

代入式 (4.3.17), 整理得

$$T(h) = I + \frac{h^2}{2! \times 6} [f'(b) - f'(a)] - \frac{h^5}{4! \times 30} \sum f^{(4)}_{k+\frac{1}{2}} - \frac{h^7}{6! \times 56} \sum f^{(6)}_{k+\frac{1}{2}} + \cdots.$$

再对 $f^{(4)}(x)$ 应用式 (4.3.16), 有

$$h \sum f^{(4)}_{k+\frac{1}{2}} = f'''(b) - f'''(a) - \frac{h^3}{3! \times 2^2} \sum f^{(6)}_{k+\frac{1}{2}} + \cdots,$$

从而可进一步得

$$\begin{aligned}
 T(h) = I &+ \frac{h^2}{2! \times 6} [f'(b) - f'(a)] - \frac{h^4}{4! \times 30} [f'''(b) - f'''(a)] \\
 &+ \frac{h^7}{6! \times 42} \sum f_{k+\frac{1}{2}}^{(6)} - \dots.
 \end{aligned}
 \tag{4.3.18}$$

反复施行上述手续，即可得到形如式 (4.3.7) 的余项公式。

最后再强调一下，应用 Romberg 方法时，一定要注意余项展开式 (4.3.7) 成立这个前提，不然就得出错误的结果。

4.4 Gauss 公式

形如式 (4.1.3) 的机械求积公式

(本句及公式续下页。)

校注 (agent 补充, CH04B-E001): 原扫描式 (4.3.17) 首个分子明确印为 h^2 , 本转录原样保留; 该处疑为原书排印错误。由本页式 (4.3.16) 及中点展开的梯形项相减, f'' 项系数应为 $h^3/12 = h^3/(2! \times 6)$, 才能在代入紧接着的 $h \sum f''$ 关系后得到原页下一式的 $h^2[f'(b) - f'(a)]/12$ 。这不是 OCR 改写; [局部原图证据](#)。

原书 PDF 页 107; 印刷页码 94。

[原页图像](#)

4.4 Gauss 公式 (续)

$$\int_a^b f(x) dx \approx \sum_{k=0}^n A_k f(x_k) \quad (4.4.1)$$

中含有 $2n+2$ 个待定参数 x_k, A_k ($k=0, 1, \dots, n$), 适当选择这些参数, 有可能使求积公式具有 $2n+1$ 次代数精度。这类求积公式称为 Gauss 公式。

4.4.1 Gauss 点

Gauss 公式的求积节点称为 Gauss 点。

定义 4.3 如果求积公式 (4.4.1) 具有 $2n+1$ 次代数精度, 则称其节点 x_k ($k=0, 1, \dots, n$) 是 Gauss 点。

下面从分析 Gauss 点的特性着手研究 Gauss 公式的构造问题。

定理 4.4 对于插值型求积公式 (4.4.1), 其节点 x_k ($k=0, 1, \dots, n$) 是 Gauss 点的充分必要条件, 是以这些点为零点的多项式 $\omega(x) = \prod_{k=0}^n (x - x_k)$ 与任意次数不超过 n 的多项式 $P(x)$ 均正交, 即

$$\int_a^b P(x)\omega(x) dx = 0. \quad (4.4.2)$$

证明 先证必要性。设 $P(x)$ 是任意次数不超过 n 的多项式, 则 $P(x)\omega(x)$ 的次数不超过 $2n+1$ 。因此, 如果 $x_0, x_1, \dots, x_n$ 是 Gauss 点, 则求积公式 (4.4.1) 对于 $P(x)\omega(x)$ 能准确成立, 即有

$$\int_a^b P(x)\omega(x) dx = \sum_{k=0}^n A_k P(x_k)\omega(x_k).$$

但 $\omega(x_k) = 0$ ($k=0, 1, \dots, n$), 故式 (4.4.2) 成立。

再证充分性。对于任意给定的次数不超过 $2n+1$ 的多项式 $f(x)$, 用 $\omega(x)$ 除 $f(x)$, 记商为 $P(x)$, 余式为 $Q(x)$, $P(x)$ 与 $Q(x)$ 都是次数不超过 n 的多项式:

$$f(x) = P(x)\omega(x) + Q(x).$$

而利用式 (4.4.2), 得

$$\int_a^b f(x) dx = \int_a^b Q(x) dx. \quad (4.4.3)$$

由于所给求积公式 (4.4.1) 是插值型的, 它对于 $Q(x)$ 能准确成立, 即

$$\int_a^b Q(x) \, dx = \sum_{k=0}^n A_k Q(x_k).$$

再注意到 $\omega(x_k) = 0$, 知 $Q(x_k) = f(x_k)$, 从而有

$$\int_a^b Q(x) \, dx = \sum_{k=0}^n A_k f(x_k),$$

于是由式 (4.4.3) 知

$$\int_a^b f(x) \, dx = \sum_{k=0}^n A_k f(x_k).$$

(证明续下页。)

原书 PDF 页 108; 印刷页码 95。

[原页图像](#)

4.4.1 Gauss 点 (续)

可见求积公式 (4.4.1) 对于一切次数不超过 $2n+1$ 的多项式均能准确成立, 因此 x_k ($k=0, 1, \dots, n$) 是 Gauss 点。证毕。

4.4.2 Gauss-Legendre 公式

不失一般性, 可取 $a=-1, b=1$ 而考察区间 $[-1, 1]$ 上的 Gauss 公式①

$$\int_{-1}^1 f(x) dx \approx \sum_{k=0}^n A_k f(x_k). \quad (4.4.4)$$

我们知道, Legendre 多项式 (见式 (3.4.1)) 是区间 $[-1, 1]$ 上的正交多项式, 因此, Legendre 多项式 $P_{n+1}(x)$ 的零点就是求积公式 (4.4.4) 的 Gauss 点。形如式 (4.4.4) 的 Gauss 公式特别地称为 Gauss-Legendre 公式。

若取 $P_1(x) = x$ 的零点 $x_0 = 0$ 作节点构造求积公式

$$\int_{-1}^1 f(x) dx \approx A_0 f(0),$$

令它对 $f(x) = 1$ 准确成立, 即可定出 $A_0 = 2$ 。这样构造出的一点 Gauss-Legendre 公式是中矩形公式。

再取 $P_2(x) = \frac{1}{2}(3x^2 - 1)$ 的两个零点 $\pm \frac{1}{\sqrt{3}}$ 构造求积公式:

$$\int_{-1}^1 f(x) dx \approx A_0 f\left(-\frac{1}{\sqrt{3}}\right) + A_1 f\left(\frac{1}{\sqrt{3}}\right),$$

令它对于 $f(x) = 1, x$ 都准确成立, 有

$$\begin{cases} A_0 + A_1 = 2, \\ A_0 \left(-\frac{1}{\sqrt{3}}\right) + A_1 \left(\frac{1}{\sqrt{3}}\right) = 0. \end{cases}$$

由此解出 $A_0 = A_1 = 1$, 从而得到两点 Gauss-Legendre 公式

$$\int_{-1}^1 f(x) dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right).$$

三点 Gauss-Legendre 公式的形式是

$$\begin{aligned} \int_{-1}^1 f(x) dx \\ \approx \frac{5}{9} f\left(-\frac{\sqrt{15}}{5}\right) + \frac{8}{9} f(0) + \frac{5}{9} f\left(\frac{\sqrt{15}}{5}\right). \end{aligned}$$

表 4.5

n	x_k	A_k
0	0.000 000 0	2.000 000 0
1	$\pm 0.577\,350\,3$	1.000 000 0
2	$\pm 0.774\,596\,7$	0.555 555 6
2	0.000 000 0	0.888 888 9
3	$\pm 0.861\,136\,3$	0.347 854 8
3	$\pm 0.339\,881\,0$	0.652 145 2
4	$\pm 0.906\,179\,3$	0.236 926 9
4	$\pm 0.538\,469\,3$	0.478 628 7
4	0.000 000 0	0.568 888 9

表 4.5 列出了 Gauss-Legendre 公式 (4.4.4) 的节点

(本句续下页。)

①对于任意求积区间 $[a, b]$, 通过变换 $x = \frac{b-a}{2}t + \frac{a+b}{2}$ 可以化到区间 $[-1, 1]$ 上, 这时

$$\int_a^b f(x) \mathrm{d} x = \frac{b-a}{2} \int_{-1}^1 f\left(\frac{b-a}{2} t + \frac{a+b}{2}\right) \mathrm{d} t.$$

校注 (agent 补充, CH04B-E002): 表 4.5 的 $n=3$ 第二对节点在原扫描中印为 $\pm 0.339\,881\,0$, 本表原样保留; 这是疑似原书数值排印错误。 $P_4(x) = (35x^4 - 30x^2 + 3)/8$ 的较小正零点为 $\sqrt{(15 - 2\sqrt{30})}/35 \approx 0.339\,981\,0436$, 保留七位小数应为 $0.339\,981\,0$ 。原表权重 $0.652\,145\,2$ 未改动。[表 4.5 原扫描局部](#)。

原书 PDF 页 109; 印刷页码 96。

[原页图像](#)

4.4.2 Gauss-Legendre 公式 (续)

和系数。

4.4.3 Gauss 公式的余项

定理 4.5 对于 Gauss 公式 (4.4.1), 其余项

$$R(x) = \int_a^b f(x) dx - \sum_{k=0}^n A_k f(x_k) = \frac{f^{(2n+2)}(\xi)}{(2n+2)!} \int_a^b \omega^2(x) dx, \quad (4.4.5)$$

这里 $\omega(x) = (x - x_0)(x - x_1) \cdots (x - x_n)$ 。

证明 以 $x_0, x_1, \dots, x_n$ 为节点构造次数不大于 $2n+1$ 的多项式 $H(x)$, 使其满足条件

$$H(x_i) = f(x_i), \quad H'(x_i) = f'(x_i) \quad (i = 0, 1, \dots, n).$$

我们知道, 这样的 $H(x)$ 称为 Hermite 插值多项式 (见式 (2.6.3)、(2.6.6))。由于 Gauss 公式具有 $2n+1$ 次代数精度, 它对于 $H(x)$ 能准确成立, 即

$$\int_a^b H(x) dx = \sum_{k=0}^n A_k H(x_k) = \sum_{k=0}^n A_k f(x_k),$$

故有

$$\begin{aligned} R(x) &= \int_a^b f(x) dx - \sum_{k=0}^n A_k f(x_k) = \int_a^b f(x) dx - \int_a^b H(x) dx \\ &= \int_a^b \frac{f^{(2n+2)}(\eta)}{(2n+2)!} \omega^2(x) dx. \end{aligned}$$

再考虑到该函数 $\omega^2(x)$ 在 $[a, b]$ 上保号, 应用积分中值定理得式 (4.4.5)。证毕。

4.4.4 Gauss 公式的稳定性

对比 Newton-Cotes 公式, Gauss 公式不但是高精度的, 而且是数值稳定的。Gauss 公式的稳定性之所以能够得到保证, 是由于它的求积系数具有非负性。

定理 4.6 Gauss 公式 (4.4.1) 的求积系数 A_k ($k = 0, 1, \dots, n$) 全是正的。

证明 考察

$$l_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^n \frac{x - x_j}{x_k - x_j},$$

它们是 n 次多项式，因而 $l_k^2(x)$ 是 $2n$ 次多项式，故 Gauss 公式 (4.4.1) 对于它能准确成立，即有

$$\int_a^b l_k^2(x) dx = \sum_{i=0}^n A_i l_k^2(x_i).$$

注意到 $l_k(x_i) = \delta_{ki}$ ，上式右端实际上即等于 A_k ，从而有 $A_k = \int_a^b l_k^2(x) dx > 0$ 。证毕。

现在讨论求积过程的稳定性问题。在用求积公式 $I_n = \sum_{k=0}^n A_k f(x_k)$ 进行实际计算时，通常不一定能提供准确的数据 $f_k = f(x_k)$ ，而只是给出含有误差（譬如由于计

（本句续下页。）

原书 PDF 页 110; 印刷页码 97。

[原页图像](#)

4.4.4 Gauss 公式的稳定性 (续)

算机字长的限制引进的舍入误差) 的数据 f_k^* , 故实际求得的积分值为

$$I_n^* = \sum_{k=0}^n A_k f_k^*.$$

人们自然关心, 数据误差对积分值的影响能否加以控制呢?

由于 Gauss 公式的求积系数具有非负性, 故

$$|I_n^* - I_n| \leq \sum_{k=0}^n A_k |f_k^* - f_k| \leq \left(\sum_{k=0}^n A_k \right) \max_{0 \leq k \leq n} |f_k^* - f_k|.$$

再利用式 (4.1.4) 的第一式, 有

$$|I_n^* - I_n| \leq (b-a) \max_{0 \leq k \leq n} |f_k^* - f_k|,$$

由此即可断定 Gauss 公式是稳定的。

4.4.5 带权的 Gauss 公式

考察积分 $I = \int_a^b \rho(x)f(x) dx$, 这里 $\rho(x) \geq 0$ 称为权函数, 当 $\rho(x) \equiv 1$ 时即为普通的积分。

可以仿照处理普通积分的方法讨论带权的积分。考察求积公式

$$\int_a^b \rho(x)f(x) dx \approx \sum_{k=0}^n A_k f(x_k),$$

如果它对于任意次数不超过 $2n+1$ 的多项式均能准确地成立, 则称之为 Gauss 型的。上述 Gauss 公式的求积节点 x_k 仍称为 Gauss 点。同样地, x_k ($k=0, 1, \dots, n$) 是 Gauss 点的充要条件为, $\omega(x) = \prod_{k=0}^n (x - x_k)$ 是区间 $[a, b]$ 上关于权函数 $\rho(x)$ 的正交多项式。

若 $a = -1, b = 1$, 且取权函数 $\rho(x) = \frac{1}{\sqrt{1-x^2}}$, 则所建立的 Gauss 公式为

$$\int_{-1}^1 \frac{f(x)}{\sqrt{1-x^2}} dx \approx \sum_{k=0}^n A_k f(x_k). \quad (4.4.6)$$

式 (4.4.6) 称为 Gauss-Chebyshev 公式。由于区间 $[-1, 1]$ 上关于权函数 $\frac{1}{\sqrt{1-x^2}}$ 的正交多项式是 Chebyshev 多项式 (见 3.4.2 节), 因此求积公式 (4.4.6) 的 Gauss 点是 $n+1$ 次 Chebyshev 多项式的零点, 即为

$$x_k = \cos\left(\frac{2k+1}{2n+2}\pi\right) \quad (k = 0, 1, \dots, n).$$

值得指出的是，运用正交多项式的零点构造 Gauss 求积公式，这种方法只是针对某些特殊的权函数才有效。构造 Gauss 公式的一般方法则是 4.1.2 节介绍过的待定系数法。

例如，设要构造下列形式的 Gauss 公式：

(公式续下页。)

原书 PDF 页 111; 印刷页码 98。

[原页图像](#)

4.4.5 带权的 Gauss 公式 (续)

$$\int_0^1 \sqrt{x} f(x) dx \approx A_0 f(x_0) + A_1 f(x_1), \quad (4.4.7)$$

令它对于 $f(x) = 1, x, x^2, x^3$ 准确成立, 得

$$\begin{cases} A_0 + A_1 = \frac{2}{3}, \\ x_0 A_0 + x_1 A_1 = \frac{2}{5}, \\ x_0^2 A_0 + x_1^2 A_1 = \frac{2}{7}, \\ x_0^3 A_0 + x_1^3 A_1 = \frac{2}{9}. \end{cases} \quad (4.4.8)$$

由于

$$x_0 A_0 + x_1 A_1 = x_0 (A_0 + A_1) + (x_1 - x_0) A_1,$$

利用式 (4.4.8) 的第一式, 可将第二式化为

$$\frac{2}{3} x_0 + (x_1 - x_0) A_1 = \frac{2}{5}.$$

同样地, 利用第二式化第三式, 利用第三式化第四式, 分别得

$$\frac{2}{5} x_0 + (x_1 - x_0) x_1 A_1 = \frac{2}{7}, \quad \frac{2}{7} x_0 + (x_1 - x_0) x_1^2 A_1 = \frac{2}{9}.$$

从上面三个式子中消去 $(x_1 - x_0) A_1$, 有

$$\begin{cases} \frac{2}{5} x_0 + \left(\frac{2}{5} - \frac{2}{3} x_0 \right) x_1 = \frac{2}{7}, \\ \frac{2}{7} x_0 + \left(\frac{2}{7} - \frac{2}{5} x_0 \right) x_1^2 = \frac{2}{9}, \end{cases}$$

即

$$\begin{cases} \frac{2}{5} (x_0 + x_1) - \frac{2}{3} x_0 x_1 = \frac{2}{7}, \\ \frac{2}{7} (x_0 + x_1) - \frac{2}{5} x_0 x_1 = \frac{2}{9}. \end{cases}$$

由此解出

$$x_0 x_1 = \frac{5}{21}, \quad x_0 + x_1 = \frac{10}{9},$$

从而求出

$$\begin{aligned} x_0 &= 0.821\,162, & x_1 &= 0.289\,949, \\ A_0 &= 0.389\,111, & A_1 &= 0.277\,556. \end{aligned}$$

于是形如式 (4.4.7) 的 Gauss 公式是

$$\int_0^1 \sqrt{x} f(x) \, dx \approx 0.389\,111 f(0.821\,162) + 0.277\,556 f(0.289\,949).$$

校注 (agent 补充, CH04B-E003): 原扫描在“从上面三个式子中消去”后的第二行印为 $\frac{2}{7}x_0 + \left(\frac{2}{7} - \frac{2}{5}x_0\right)x_1^2 = \frac{2}{9}$, 本转录原样保留。疑误是括号后 x_1^2 的指数: 前一行有 $(x_1 - x_0)x_1 A_1 = \frac{2}{7} - \frac{2}{5}x_0$, 代入再上一组的第四式应只再乘 x_1 , 故此处应为 $\left(\frac{2}{7} - \frac{2}{5}x_0\right)x_1$, 才与紧接着“即”后的第二行相符。这是原书疑似排印错误, 非 OCR 错误。[局部原扫描](#)。

原书 PDF 页 112; 印刷页码 99。

[原页图像](#)

4.5 数值微分

4.5.1 中点方法

按照数学分析的定义, 导数 $f'(a)$ 是差商 $\frac{f(a+h)-f(a)}{h}$ 当 $h \rightarrow 0$ 时的极限。如果精度要求不高, 我们可以简单地取差商作为导数的近似值, 这样便建立起一种数值微分方法, 即

$$f'(a) \approx \frac{f(a+h)-f(a)}{h}.$$

类似地, 亦可用向后差商作近似运算, 即

$$f'(a) \approx \frac{f(a)-f(a-h)}{h}$$

或用中心差商作近似计算, 即

$$f'(a) \approx \frac{f(a+h)-f(a-h)}{2h}.$$

后一种数值微分方法称中点方法, 它其实是前两种方法的算术平均。

在图 4.3 上, 上述三种导数的近似值分别表示弦线 AB 、 AC 和 BC 的斜率, 比较这三种弦线与切线 AT (其斜率等于导数值 $f'(a)$) 平行的程度, 从图形上可以明显地看出, 其中以 BC 的斜率更接近于切线 AT 的斜率。因此就精度而言, 以中点方法更为可取。

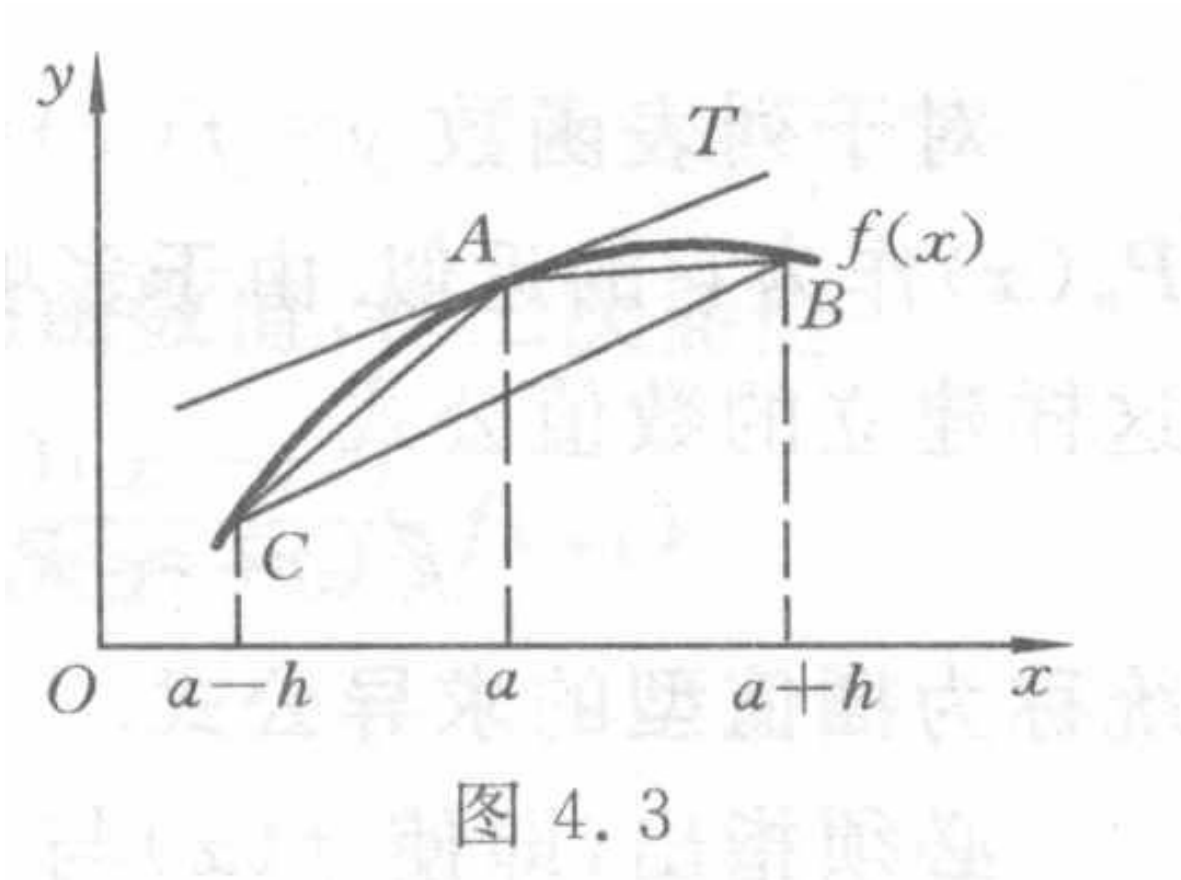


图 4.3

图 4.3。原图标签：横轴 x 、纵轴 y 、原点 O ；横轴位置 $a-h$ 、 a 、 $a+h$ ，对应曲线上的点 C 、 A 、 B ；曲线标记 $f(x)$ ，过 A 的切线标记 T 。图中画出弦线 AB 、 AC 、 BC 与切线 AT 。（此段为图中标签和图意的文字转录；原图未给出具体函数或数值。）

上述三种数值微分方法有个共同点，它们都是将导数的计算归结为计算 f 在若干节点上的函数值。这类数值微分方法称为机械求导方法。

要利用中点公式

$$G(h) = \frac{f(a+h) - f(a-h)}{2h}$$

计算导数 $f'(a)$ 的近似值，首先必须选取合适的步长，为此需要进行误差分析。分别将 $f(a \pm h)$ 在 $x = a$ 处作 Taylor 展开，有

$$f(a \pm h) = f(a) \pm hf'(a) + \frac{h^2}{2!}f''(a) \pm \frac{h^3}{3!}f'''(a) + \frac{h^4}{4!}f^{(4)}(a) \pm \frac{h^5}{5!}f^{(5)}(a) + \cdots,$$

代入上式得

$$G(h) = f'(a) + \frac{h^2}{3!}f'''(a) + \frac{h^4}{5!}f^{(5)}(a) + \cdots.$$

原书 PDF 页 113；印刷页码 100。

[原页图像](#)

4.5.1 中点方法（续）

由此得知，从截断误差的角度看，步长越小，计算结果越准确。

再考察舍入误差，按中点公式计算。当 h 很小时，因 $f(a + h)$ 与 $f(a - h)$ 很接近，直接相减会造成有效数字的严重损失（参见 1.4 节）。因此，从舍入误差的角度来看，步长是不宜太小的。

例如，用中点公式求 $f(x) = \sqrt{x}$ 在 $x = 2$ 处的一阶导数，有

$$G(h) = \frac{\sqrt{2 + h} - \sqrt{2 - h}}{2h}.$$

设取四位数字计算，结果如表 4.6（导数的准确值 $f'(2) = 0.353\,553$ ）所示。

表 4.6

h	$G(h)$	h	$G(h)$	h	$G(h)$
1	0.366 0	0.05	0.353 0	0.001	0.350 0
0.5	0.356 4	0.01	0.350 0	0.000 5	0.300 0
0.1	0.353 5	0.005	0.350 0	0.000 1	0.300 0

在表 4.6 中， $h = 0.1$ 的逼近效果最好，如果进一步缩小步长，则逼近的效果会越来越差。

4.5.2 插值型的求导公式

对于列表函数 $y = f(x)$ （见表 4.7），运用插值原理，可以建立插值多项式 $y = P_n(x)$ 作为它的近似。由于多项式的求导比较容易，取 $P'_n(x)$ 的值作为 $f'(x)$ 的近似值，这样建立的数值公式

$$f'(x) \approx P'_n(x) \tag{4.5.1}$$

统称为插值型的求导公式。

表 4.7

x	x_0	x_1	x_2	$\cdots$	x_n
y	y_0	y_1	y_2	$\cdots$	y_n

必须指出，即使 $f(x)$ 与 $P_n(x)$ 的值相差不多，导数的近似值 $P'_n(x)$ 与导数的真值 $f'(x)$ 仍然可能差别很大，因而在使用求导公式（4.5.1）时应特别注意误差的分析。

依据插值余项定理，求导公式（4.5.1）的余项为

$$f'(x) - P'_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega'_{n+1}(x) + \frac{\omega_{n+1}(x)}{(n+1)!} \frac{d}{dx} f^{(n+1)}(\xi),$$

其中

$$\omega_{n+1}(x) = \prod_{k=0}^n (x - x_k).$$

在此余项公式中, 由于 ξ 是 x 的未知函数, 无法对其第二项 $\frac{\omega_{n+1}(x)}{(n+1)!} \frac{d}{dx} f^{(n+1)}(\xi)$ 作出进一步的说明, 因此, 对于随意给出的点 x , 误差 $f'(x) - P'_n(x)$ 是无法预估的。但是, 如果限定求某个节点 x_k 上的导数值, 那么上面的第二项因 $\omega_{n+1}(x_k) = 0$ 而变为

(本句续下页。)

校注 (agent 补充, CH04B-E004): 原页“其中”后的节点多项式印为 $\prod_{k=0}^n (x - x_i)$, 乘积指标 k 与因子下标 i 不一致, 已按原扫描保留。结合本页表 4.7 和下文 $\omega_{n+1}(x_k) = 0$, 应将因子写作 $(x - x_k)$, 或统一将乘积指标改为 i ; 这是原书疑似下标排印错误, 非 OCR 错误。[局部原扫描](#)。

原书 PDF 页 114; 印刷页码 101。

[原页图像](#)

4.5.2 插值型的求导公式 (续)

零, 这时有余项公式

$$f'(x_k) - P'_n(x_k) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega'_{n+1}(x_k). \quad (4.5.2)$$

下面仅仅考察节点处的导数值。为简化讨论, 假定所给的节点是等距的。

1. 两点公式 设已给出两个节点 x_0, x_1 上面的函数值 $f(x_0), f(x_1)$, 作线性插值公式

$$P_1(x) = \frac{x - x_1}{x_0 - x_1} f(x_0) + \frac{x - x_0}{x_1 - x_0} f(x_1).$$

对上式两端求导, 记 $x_1 - x_0 = h$, 有

$$P'_1(x) = \frac{1}{h} [-f(x_0) + f(x_1)],$$

于是有下列求导公式:

$$P'_1(x_0) = \frac{1}{h} [f(x_1) - f(x_0)], \quad P'_1(x_1) = \frac{1}{h} [f(x_1) - f(x_0)].$$

而利用余项公式 (4.5.2) 知, 带余项的两点公式是

$$f'(x_0) = \frac{1}{h} [f(x_1) - f(x_0)] - \frac{h}{2} f''(\xi),$$

$$f'(x_1) = \frac{1}{h} [f(x_1) - f(x_0)] + \frac{h}{2} f''(\xi).$$

2. 三点公式 设已给出三个节点 $x_0, x_1 = x_0 + h, x_2 = x_0 + 2h$ 上的函数值, 作二次插值

$$\begin{aligned} P_2(x) &= \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} f(x_0) + \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} f(x_1) \\ &\quad + \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} f(x_2), \end{aligned}$$

令 $x = x_0 + th$, 上式可表示为

$$P_2(x_0 + th) = \frac{1}{2}(t-1)(t-2)f(x_0) - t(t-2)f(x_1) + \frac{1}{2}t(t-1)f(x_2),$$

两端对 t 求导, 有

$$P_2'(x_0 + th) = \frac{1}{2h}[(2t - 3)f(x_0) - (4t - 4)f(x_1) + (2t - 1)f(x_2)]. \quad (4.5.3)$$

这里撇号表示对变量 x 求导数。上式分别取 $t = 0, 1, 2$, 得到以下三种三点公式:

$$P_2'(x_0) = \frac{1}{2h}[-3f(x_0) + 4f(x_1) - f(x_2)],$$

$$P_2'(x_1) = \frac{1}{2h}[-f(x_0) + f(x_2)],$$

(第三个公式续下页。)

原书 PDF 页 115; 印刷页码 102。

[原页图像](#)

4.5.2 插值型的求导公式 (续)

2. 三点公式 (续)

$$P_2'(x_2) = \frac{1}{2h}[f(x_0) - 4f(x_1) + 3f(x_2)],$$

而带余项的三点求导公式为

$$\begin{aligned} f'(x_0) &= \frac{1}{2h}[-3f(x_0) + 4f(x_1) - f(x_2)] + \frac{h^2}{3}f'''(\xi), \\ f'(x_1) &= \frac{1}{2h}[-f(x_0) + f(x_2)] - \frac{h^2}{6}f'''(\xi), \\ f'(x_2) &= \frac{1}{2h}[f(x_0) - 4f(x_1) + 3f(x_2)] + \frac{h^2}{3}f'''(\xi), \end{aligned} \quad (4.5.4)$$

其中式 (4.5.4) 是我们所熟悉的中点公式。在三点公式中, 它由于少用了一个函数值 $f(x_1)$ 而引人注目。

用插值多项式 $P_n(x)$ 作为 $f(x)$ 的近似函数, 还可以建立高阶数值微分公式

$$f^{(k)}(x) \approx P_n^{(k)}(x), \quad k = 1, 2, \dots.$$

例如, 将式 (4.5.3) 再对 t 求导一次, 有

$$P_2''(x_0 + th) = \frac{1}{h^2}[f(x_0) - 2f(x_1) + f(x_2)],$$

于是有

$$P_2''(x_1) = \frac{1}{h^2}[f(x_1 - h) - 2f(x_1) + f(x_1 + h)],$$

而带余项的二阶三点公式为

$$f''(x_1) = \frac{1}{h^2}[f(x_1 - h) - 2f(x_1) + f(x_1 + h)] - \frac{h^2}{12}f^{(4)}(\xi). \quad (4.5.5)$$

4.5.3 实用的五点公式

设已给出五个节点 $x_i = x_0 + ih$, $i = 0, 1, 2, 3, 4$ 上的函数值, 重复同样的手续, 不难导出下列五点公式:

$$m_0 = \frac{1}{12h}[-25f(x_0) + 48f(x_1) - 36f(x_2) + 16f(x_3) - 3f(x_4)],$$

$$m_1 = \frac{1}{12h}[-3f(x_0) - 10f(x_1) + 18f(x_2) - 6f(x_3) + f(x_4)],$$

$$m_2 = \frac{1}{12h}[f(x_0) - 8f(x_1) + 8f(x_3) - f(x_4)],$$

$$m_3 = \frac{1}{12h}[-f(x_0) + 6f(x_1) - 18f(x_2) + 10f(x_3) + 3f(x_4)],$$

$$m_4 = \frac{1}{12h}[3f(x_0) - 16f(x_1) + 36f(x_2) - 48f(x_3) + 25f(x_4)].$$

其中 m_i 代表一阶导数 $f'(x_i)$ 的近似值, 读者不难导出这些求导公式的余项。

再记 M_i 表示二阶导数 $f''(x_i)$ 的近似值, 二阶五点公式如下:

(公式续下页。)

原书 PDF 页 116; 印刷页码 103。

[原页图像](#)

4.5.3 实用的五点公式 (续)

$$M_0 = \frac{1}{12h^2}[35f(x_0) - 104f(x_1) + 114f(x_2) - 56f(x_3) + 11f(x_4)],$$

$$M_1 = \frac{1}{12h^2}[11f(x_0) - 20f(x_1) + 6f(x_2) + 4f(x_3) - f(x_4)],$$

$$M_2 = \frac{1}{12h^2}[-f(x_0) + 16f(x_1) - 30f(x_2) + 16f(x_3) - f(x_4)],$$

$$M_3 = \frac{1}{12h^2}[-f(x_0) + 4f(x_1) + 6f(x_2) - 20f(x_3) + 11f(x_4)],$$

$$M_4 = \frac{1}{12h^2}[11f(x_0) - 56f(x_1) + 114f(x_2) - 104f(x_3) + 35f(x_4)].$$

对于给定的一张数据表, 用五点公式求节点上的导数值往往可以获得满意的结果。五个相邻节点的选择原则, 一般是在所考察的节点的两侧各取两个邻近的节点, 如果一侧的节点数不足两个 (即一侧只有一个节点或没有节点), 则用另一侧的节点补足。

例 4.4 利用 $f(x) = \sqrt{x}$ 的一张数据表, 按五点公式求节点上的导数值 m_i 与 M_i , 并与导数的准确值比较, 如表 4.8 所示。

表 4.8

x_i	$f(x_i)$	m_i	$f'(x_i)$	$M_i / \times 10^3$	$f''(x_i) / \times 10^3$
100	10.000 000	0.050 000	0.050 000	-0.247 58	-0.250 00
101	10.049 875	0.049 751	0.049 752	-0.245 91	-0.246 30
102	10.099 504	0.049 507	0.049 507	-0.241 91	-0.242 68
103	10.148 891	0.049 267	0.049 266	-0.239 58	-0.239 16
104	10.198 039	0.049 029	0.049 029	-0.236 91	-0.235 72
105	10.246 950	0.048 795	0.048 795	-0.236 66	-0.232 36

4.5.4 样条求导

我们知道, 样条函数 $S(x)$ 作为 $f(x)$ 的近似函数, 不但彼此的函数值很接近, 导数值也很接近。例如, 对于三次样条 $S_3(x)$, 有

$$|f^{(a)}(x) - S_3^{(a)}(x)| = O(h^{4-a}) \quad (a = 0, 1, 2, 3),$$

因此, 用样条函数建立数值微分公式是很自然的, 即

$$f^{(a)}(x) \approx S_3^{(a)}(x) \quad (a = 0, 1, 2, 3). \quad (4.5.6)$$

与前述插值型微分公式 (4.5.1) 不同, 样条微分公式 (4.5.6) 可以用来计算插值范围内任何一点 x (不仅是节点 x_i) 上的导数值。对于等距分划

$$\pi : a = x_0 < x_1 < x_2 < \cdots < x_n = b, \quad x_{k+1} - x_k = h,$$

三次样条 $S_3(x)$ 在节点上的导数值 $S_3'(x_k) = m_k$ 满足下列连续性方程组

(方程组续下页。)

校注 (agent 补充, 原表记法说明): 表 4.8 后两列表头按原扫描保留为 $M_i / \times 10^3$ 与 $f''(x_i) / \times 10^3$, 未改写原表头。表内数值对应导数乘 10^3 后的数值, 例如 $f''(100) = -1/(4 \cdot 100^{3/2}) = -0.00025$, 表中列为 -0.25000 ; 此说明仅解释该页缩放记法。[表 4.8 原扫描局部](#)。

原书 PDF 页 117; 印刷页码 104。

[原页图像](#)

4.5.4 样条求导 (续)

$$m_{k-1} + 4m_k + m_{k+1} = 3(y_{k+1} - y_{k-1})/h, \quad k = 1, 2, \dots, n-1. \quad (4.5.7)$$

设已给出端点处的一阶导数值 $m_0 = y'_0, m_n = y'_n$, 则求解方程组 (4.5.7) 得出的 m_k 即可作为导数 $f'(x_k)$ 的近似值。

小结

本章介绍积分和微分的数值计算方法。我们知道, 积分和微分是两种分析运算, 它们都是用极限来定义的。数值积分和数值微分则归结为函数值的四则运算, 从而使计算过程可以在计算机上完成。

处理数值积分和数值微分的基本方法是逼近法: 设法构造某个简单函数 $P(x)$ 近似 $f(x)$, 然后对 $P(x)$ 求积 (求导) 得到 $f(x)$ 的积分 (导数) 的近似值。本章基于插值原理推导了数值积分和数值微分的基本公式。

插值求积公式分 Newton-Cotes 公式与 Gauss 公式两类。前者取等距节点, 算法简单而容易编制程序。Gauss 公式的精度高, 但节点没有规律。运用带权的 Gauss 公式, 能把复杂的积分化简, 还可以直接计算奇异积分。构造 Gauss 公式会碰到选取求积节点 (所谓 Gauss 点) 的困难。

梯形法是众所周知的一种简单的求积方法。可以看到, 将二分过程中的梯形值适当进行加权平均, 会获得高精度的积分值。运用这种外推加速技术的 Romberg 算法, 在电子计算机上有着广泛的应用。

限于篇幅, 本章略去了数值积分的一些重要内容, 如奇异积分、振荡函数的积分等, 这些内容在文献 [7] 中有较为详尽的论述。关于高维积分的数值计算, 有兴趣的读者可参看文献 [8]。

习题

1. 确定下列求积公式中的待定参数, 使其代数精度尽量高, 并指明所构造出的求积公式所具有的代数精度:

(1)

$$\int_{-h}^h f(x) dx \approx A_{-1}f(-h) + A_0f(0) + A_1f(h);$$

(2)

$$\int_{-2h}^{2h} f(x) dx \approx A_{-1}f(-h) + A_0f(0) + A_1f(h);$$

(3)

$$\int_{-1}^1 f(x) dx \approx [f(-1) + 2f(x_1) + 3f(x_2)]/3;$$

(4)

$$\int_0^h f(x) \, dx \approx h[f(0) + f(h)]/1 + ah^2[f'(0) - f'(h)].$$

2. 分别用梯形公式和 Simpson 公式计算下列积分:

(1)

$$\int_0^1 \frac{x}{4+x^2} \, dx, \quad n=8;$$

(2)

$$\int_0^1 \frac{(1-e^{-x})^{\frac{1}{2}}}{x} \, dx, \quad n=10;$$

(第 2 题其余小题续下页。)

校注 (agent 补充, CH04B-E005): 第 1 题 (4) 首项的分母在原扫描中明确印为 “1”, 故保留 $h[f(0)+f(h)]/1$ 。该处疑为原书排印错误: 仅作排印一致性检查, 当 $f \equiv 1$ 时, 积分为 h , 而所印右端为 $2h$, 并且导数项为零; 疑应将分母印为 “2”。此处不求解题中参数或代数精度。[习题原扫描局部](#)。

原书 PDF 页 118; 印刷页码 105。

[原页图像](#)

习题 (续)

2. (续)

(3)

$$\int_1^9 \sqrt{x} \, dx, \quad n = 4;$$

(4)

$$\int_0^{\pi/6} \sqrt{4 - \sin^2 \varphi} \, d\varphi, \quad n = 6.$$

3. 直接验证 Cotes 公式 (4.2.4) 具有五次代数精度。

4. 用 Simpson 公式求积分 $\int_0^1 e^{-x} \, dx$ 并估计误差。

5. 推导下列三种矩形求积公式:

$$\int_a^b f(x) \, dx = (b-a)f(a) + \frac{f'(\eta)}{2}(b-a)^2,$$

$$\int_a^b f(x) \, dx = (b-a)f(b) - \frac{f'(\eta)}{2}(b-a)^2,$$

$$\int_a^b f(x) \, dx = (b-a)f\left(\frac{a+b}{2}\right) + \frac{f''(\eta)}{24}(b-a)^3.$$

6. 证明梯形公式 (4.2.9) 和 Simpson 公式 (4.2.11) 当 $n \rightarrow \infty$ 时收敛到积分 $\int_a^b f(x) \, dx$ 。

7. 用复化梯形公式求积分 $\int_a^b f(x) \, dx$ 。要将积分区间 $[a, b]$ 分成多少等份, 才能保证误差不超过 ε (不计舍入误差)?

8. 用 Romberg 方法计算积分 $\frac{2}{\sqrt{\pi}} \int_0^1 e^{-x} \, dx$, 要求误差不超过 10^{-5} 。

9. 卫星轨道是一个椭圆, 椭圆周长的计算公式是

$$s = a \int_0^{\pi/2} \sqrt{1 - \left(\frac{c}{a}\right)^2 \sin^2 \theta} \, d\theta,$$

其中 a 是椭圆的长半轴, c 是地球中心与轨道中心(椭圆中心)的距离。记 h 为近地点距离, H 为远地点距离, $R = 6371 \, \text{km}$ 为地球半径, 则 $a = (2R + H + h)/2$, $c = (H - h)/2$ 。

我国第一颗人造卫星近地点距离 $h = 439\text{ km}$ ，远地点距离 $2\,384\text{ km}$ ，试求卫星轨道的周长。

10. 证明等式

$$n \sin \frac{\pi}{n} = \pi - \frac{\pi^3}{3!n^2} + \frac{\pi^5}{5!n^4} - \cdots,$$

试依据 $n \sin \frac{\pi}{n}$ ($n = 3, 6, 12$) 的值，用外推算法求 π 的近似值。

11. 用下列方法计算积分 $\int_1^3 \frac{\mathrm{d}y}{y}$ ，并比较结果。

- (1) Romberg 方法；
- (2) 三点及五点 Gauss 公式；
- (3) 将积分区间分为四等份，用复化两点 Gauss 公式。

12. 用三点公式和五点公式求 $f(x) = \frac{1}{(1+x)^2}$ 在 $x = 1.0, 1.1$ 和 1.2 处的导数值，并估计误差。 $f(x)$ 的值由表 4.9 给出。

表 4.9

x	1.0	1.1	1.2	1.3	1.4
$f(x)$	0.250 0	0.226 8	0.206 6	0.189 0	0.173 6

校注 (agent 补充, CH04B-E006): 第 9 题的原扫描明确写 $s = a \int_0^{\pi/2} \cdots \mathrm{d}\theta$, 本转录保留原式。若 s 表示文中的椭圆“周长”, 该式疑漏系数 4: 所印表达式对应四分之一周长; 以 $c = 0$ 的圆作一致性检查, 原式给出 $\pi a/2$, 而圆周长为 $2\pi a$ 。仅记录原书公式疑误, 不代做本题数值计算。[局部原扫描](#)。

原书 PDF 页 119; 印刷页码 106。

[原页](#)

第 5 章常微分方程数值解法

5.1 引言

科学技术中常常需要求解常微分方程的定解问题。这类问题最简单的形式，是本章将要着重考察的一阶方程的初值问题

$$y' = f(x, y), \quad (5.1.1)$$

$$y(x_0) = y_0. \quad (5.1.2)$$

我们知道，只要函数 $f(x, y)$ 适当光滑——譬如关于 y 满足 Lipschitz 条件

$$|f(x, y) - f(x, \bar{y})| \leq L|y - \bar{y}|,$$

理论上就可以保证初值问题 (5.1.1)、(5.1.2) 的解 $y = y(x)$ 存在并且唯一。

虽然求解常微分方程有各种各样的解析方法，但解析方法只能用来求解一些特殊类型的方程，实际问题中归结出来的微分方程主要靠数值解法求解。

所谓**数值解法**，就是寻求解 $y(x)$ 在一系列离散节点

$$x_1 < x_2 < \cdots < x_n < x_{n+1} < \cdots$$

上的近似值 $y_1, y_2, \cdots, y_n, y_{n+1}, \cdots$ 。相邻两个节点的间距 $h = x_{n+1} - x_n$ 称为**步长**。今后如不特别说明，总是假定 h 为定数，这时节点为 $x_n = x_0 + nh$, $n = 0, 1, 2, \cdots$ 。

初值问题 (5.1.1)、(5.1.2) 的数值解法有个基本特点，它们都采取“步进式”，即求解过程顺着节点排列的次序一步一步地向推进。描述这类算法，只要给出用已知信息 $y_n, y_{n-1}, y_{n-2}, \cdots$ 计算 y_{n+1} 的递推公式即可。这种计算公式称为**差分格式**。

5.2 Euler 方法

5.2.1 Euler 格式

我们知道，在 Oxy 平面上，微分方程 (5.1.1) 的解 $y = y(x)$ 称为它的**积分曲线**。积分曲线上一点 (x, y) 的切线斜率等于函数 $f(x, y)$ 的值。如果按函数 $f(x, y)$ 在 Oxy 平面上建立一个方向场，那么，积分曲线上每一点的切线方向均与方向场在该点的方向一致。

基于上述几何解释，我们从初始点 $P_0(x_0, y_0)$ 出发，先依方向场在该点的方向推

原书 PDF 页 120; 印刷页码 107。

原页

进到 $x = x_1$ 上一点 P_1 , 然后再从 P_1 依方向场的方向推进到 $x = x_2$ 上一点 P_2 , 循此前进作出一条折线 $P_0P_1P_2\cdots$ (见图 5.1)。

一般地, 设已作出该折线的极点 P_n , 过 $P_n(x_n, y_n)$ 依方向场的方向再推进到 $P_{n+1}(x_{n+1}, y_{n+1})$, 显然两个极点 P_n, P_{n+1} 的坐标有下列关系:

$$\frac{y_{n+1} - y_n}{x_{n+1} - x_n} = f(x_n, y_n),$$

即

$$y_{n+1} = y_n + hf(x_n, y_n). \quad (5.2.1)$$

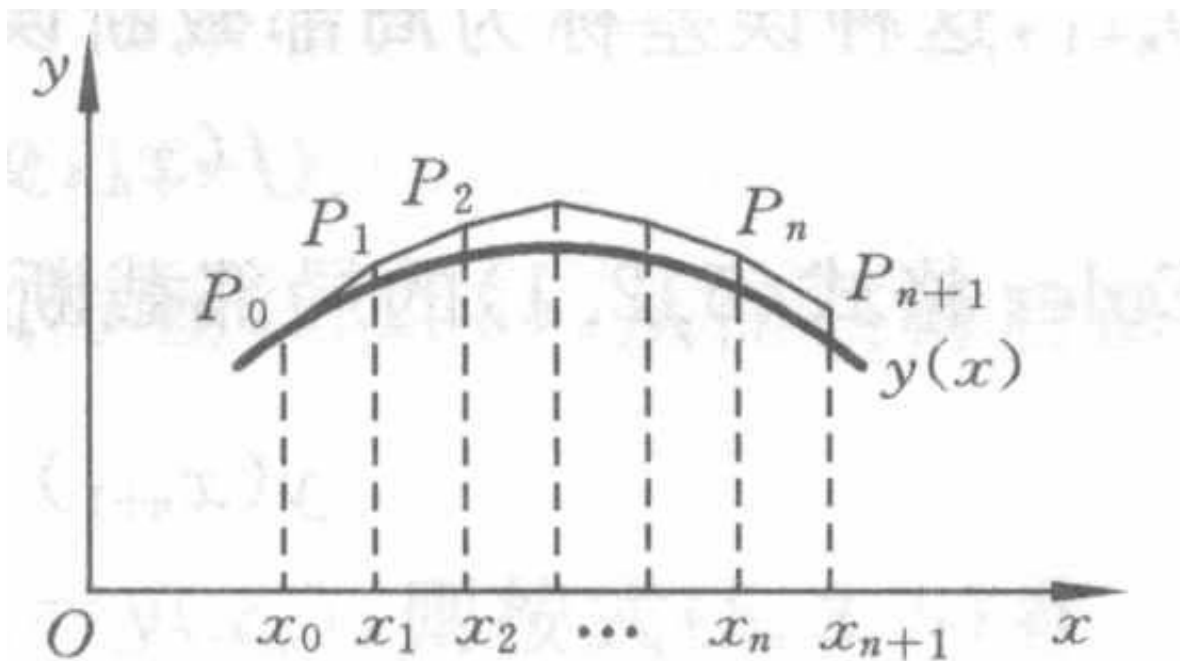


图 5.1

图 5.1 (原图): Euler 折线 $P_0P_1P_2\cdots P_nP_{n+1}$ 与积分曲线 $y(x)$ 。原图符号标签: $O, x, y, x_0, x_1, x_2, \cdots, x_n, x_{n+1}, P_0, P_1, P_2, P_n, P_{n+1}, y(x)$ 。

这就是著名的 **Euler 格式**。若初值 y_0 已知, 则依格式 (5.2.1) 可逐步算出

$$y_1 = y_0 + hf(x_0, y_0), \quad y_2 = y_1 + hf(x_1, y_1), \quad \cdots$$

例 5.1 求解初值问题

$$\begin{cases} y' = y - 2x/y & (0 < x < 1), \\ y(0) = 1. \end{cases} \quad (5.2.2)$$

解为便于比较, 本章将用多种数值方法求解上述初值问题。这里先用 Euler 方法, Euler 格式的具体形式为

$$y_{n+1} = y_n + h \left(y_n - \frac{2x_n}{y_n} \right).$$

取步长 $h = 0.1$, 计算结果如表 5.1 所示。

表 5.1

x_n	y_n	$y(x_n)$	x_n	y_n	$y(x_n)$
0.1	1.1000	1.0954	0.6	1.5090	1.4832
0.2	1.1918	1.1832	0.7	1.5803	1.5492
0.3	1.2774	1.2649	0.8	1.6498	1.6125
0.4	1.3582	1.3416	0.9	1.7178	1.6733
0.5	1.4351	1.4142	1.0	1.7848	1.7321

初值问题 (5.2.2) 有解 $y = \sqrt{1+2x}$, 按这个解析式子算出的准确值 $y(x_n)$ 同近似值 y_n 一起列在表 5.1 中, 二者相比较可以看出, Euler 方法的精度很差。

还可以通过几何直观来考察 Euler 方法的精度。假设 $y_n = y(x_n)$, 即顶点 P_n 落在积分曲线 $y = y(x)$ 上, 那么, 按 Euler 方法作出的折线 $P_n P_{n+1}$ 便是 $y = y(x)$ 过点 P_n 的切线 (见图 5.2)。从图形上看, 这样定出的顶点 P_{n+1} 显著地偏离了原来的积分曲线, 可见 Euler 方法是相当粗糙的。

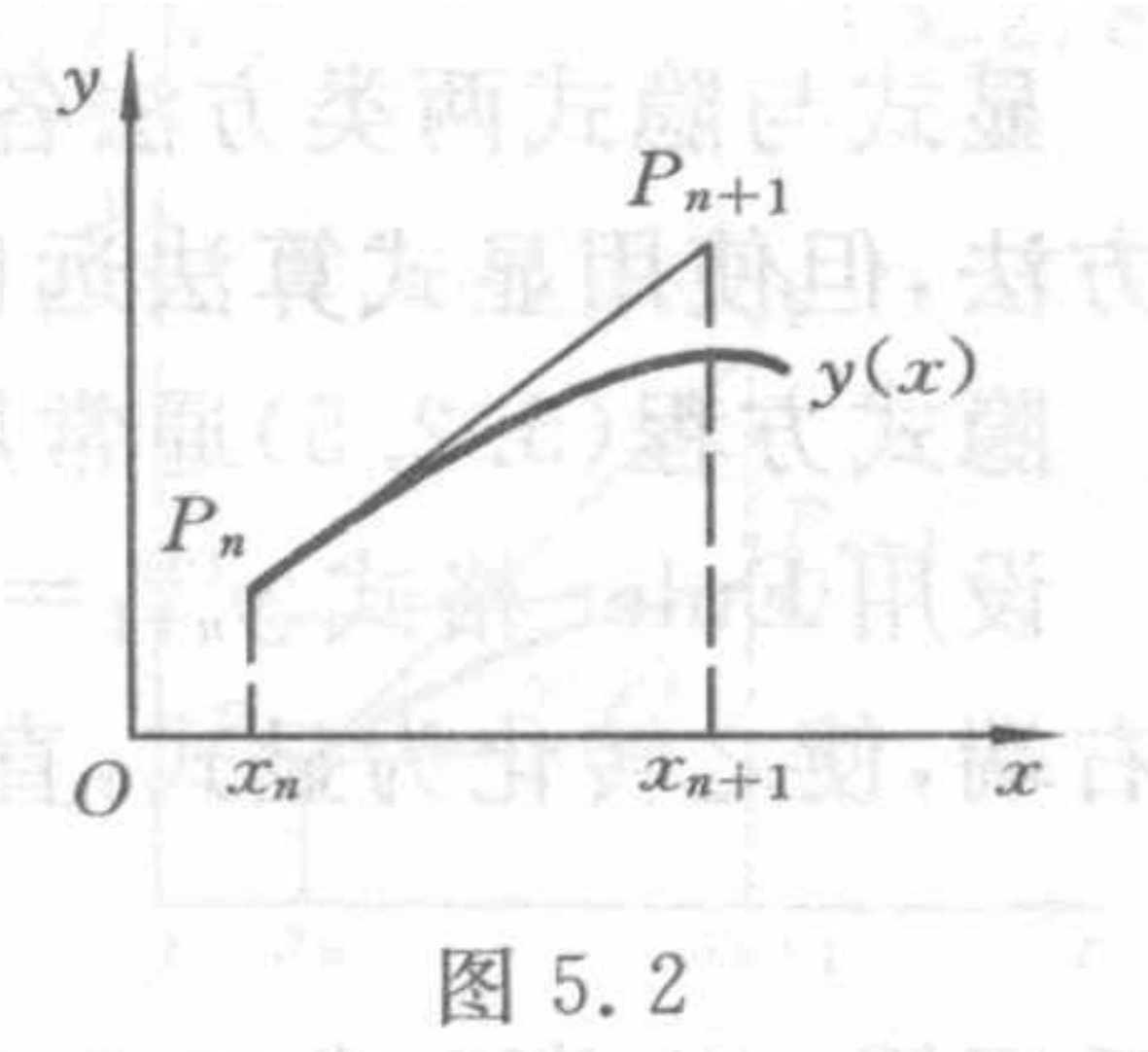


图 5. 2

图 5.2 (原图): 从积分曲线上 P_n 沿切线得到 P_{n+1} , 与曲线 $y(x)$ 在 x_{n+1} 的值有偏差。原图符号标签: $O, x, y, x_n, x_{n+1}, P_n, P_{n+1}, y(x)$ 。

人们常以 Taylor 展开为工具来分析计算公式的

原书 PDF 页 121; 印刷页码 108。

[原页](#)

精度。为简化分析, 假定 y_n 是准确的, 即在 $y_n = y(x_n)$ 的前提下估计误差 $y(x_{n+1}) - y_{n+1}$, 这种误差称为**局部截断误差**。注意到

$$f(x_n, y_n) = f(x_n, y(x_n)) = y'(x_n),$$

Euler 格式 (5.2.1) 的局部截断误差显然为

$$y(x_{n+1}) - y_{n+1} = \frac{h^2}{2} y''(\xi) \approx \frac{h^2}{2} y''(x_n). \quad (5.2.3)$$

5.2.2 后退的 Euler 格式

方程 $y' = f(x, y)$ 中含有导数项 $y'(x)$, 这是微分方程的本质特征, 也正是它难以求解的症结所在。数值解法的关键在于设法消除其导数项, 这项手续称为**离散化**。由于差分是微分的近似运算, 实现离散化的基本途径之一是直接用差商替代导数。譬如, 若在点 x_n 列出方程

$$y'(x_n) = f(x_n, y(x_n)), \quad (5.2.4)$$

并用差商 $\frac{y(x_{n+1}) - y(x_n)}{h}$ 近似替代其中的导数 $y'(x_n)$, 结果有

$$y(x_{n+1}) \approx y(x_n) + hf(x_n, y(x_n)).$$

设 $y(x_n)$ 的近似值 y_n 已知, 用它代入上式右端进行计算, 并取计算结果 y_{n+1} 作为 $y(x_{n+1})$ 的近似值, 这就是 Euler 格式。

对于在点 x_{n+1} 列出的方程 (5.1.1), 有

$$y'(x_{n+1}) = f(x_{n+1}, y(x_{n+1})),$$

若用向后差商 $\frac{y(x_{n+1}) - y(x_n)}{h}$ 替代导数 $y'(x_{n+1})$, 则可将上式离散化得

$$\frac{y_{n+1} - y_n}{h} = f(x_{n+1}, y_{n+1}),$$

即

$$y_{n+1} = y_n + hf(x_{n+1}, y_{n+1}), \quad (5.2.5)$$

此为**后退的 Euler 格式**。

后退的 Euler 格式与 Euler 格式有着本质的区别, 后者是关于 y_{n+1} 的一个直接的计算格式, 这类格式是**显式的**; 而格式 (5.2.5) 的右端含有未知的 y_{n+1} , 它实际上是关于 y_{n+1} 的一个函数方程, 这类格式是**隐式的**。

显式与隐式两类方法各有特点。考虑到数值稳定性等因素，人们有时需要选用隐式方法，但使用显式算法远比隐式方便。

隐式方程 (5.2.5) 通常用迭代法求解，而迭代过程的实质是逐步显式化。

设用 Euler 格式 $y_{n+1}^{(0)} = y_n + hf(x_n, y_n)$ 给出迭代初值 $y_{n+1}^{(0)}$ ，用它代入式 (5.2.5) 的右端，使之转化为显式，直接计算得

$$y_{n+1}^{(1)} = y_n + hf(x_{n+1}, y_{n+1}^{(0)}),$$

然后再用 $y_{n+1}^{(1)}$ 代入式 (5.2.5) 的右端，又有

原书 PDF 页 122; 印刷页码 109。

[原页](#)

$$y_{n+1}^{(2)} = y_n + hf(x_{n+1}, y_{n+1}^{(1)}).$$

如此反复进行迭代, 得

$$y_{n+1}^{(k+1)} = y_n + hf(x_{n+1}, y_{n+1}^{(k)}) \quad (k = 0, 1, \dots).$$

如果迭代过程收敛, 则极限值 $y_{n+1} = \lim_{k \rightarrow \infty} y_{n+1}^{(k)}$ 必满足隐式方程 (5.2.5), 从而获得后退的 Euler 方法的解。

再考察后退的 Euler 格式的局部截断误差。假设 $y_n = y(x_n)$, 则按式 (5.2.5) 有

$$y_{n+1} = y(x_n) + hf(x_{n+1}, y_{n+1}), \quad (5.2.6)$$

由于

$$f(x_{n+1}, y_{n+1}) = f(x_{n+1}, y(x_{n+1})) + f_y(x_{n+1}, \eta)[y_{n+1} - y(x_{n+1})],$$

其中 η 介于 y_{n+1} 与 $y(x_{n+1})$ 之间。又

$$f(x_{n+1}, y(x_{n+1})) = y'(x_{n+1}) = y'(x_n) + hy''(x_n) + \dots,$$

代入式 (5.2.6) 有

$$y_{n+1} = hf_y(x_{n+1}, \eta)[y_{n+1} - y(x_{n+1})] + y(x_n) + hy'(x_n) + h^2 y''(x_n) + \dots,$$

将它与 Taylor 展开式

$$y(x_{n+1}) = y(x_n) + hy'(x_n) + \frac{h^2}{2} y''(x_n) + \dots$$

相减, 得

$$y(x_{n+1}) - y_{n+1} = hf_y(x_{n+1}, \eta)[y(x_{n+1}) - y_{n+1}] - \frac{h^2}{2} y''(x_n) + \dots.$$

再注意到

$$\frac{1}{1 - hf_y(x_{n+1}, \eta)} = 1 + hf_y(x_{n+1}, \eta) + \dots,$$

最后整理, 得

$$y(x_{n+1}) - y_{n+1} \approx -\frac{h^2}{2} y''(x_n). \quad (5.2.7)$$

5.2.3 梯形格式

比较 Euler 格式与后退的 Euler 格式的误差公式 (5.2.3)、(5.2.7)，可以看到，如果将这两种方法进行算术平均，即可消除误差的主要部分 $\pm \frac{h^2}{2} y''$ 而获得更高的精度。这种平均化方法通常称为**梯形方法**，其计算格式为

$$y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, y_{n+1})]. \quad (5.2.8)$$

梯形方法的这种平均化思想亦可借助于几何直观来说明，仍设顶点 $P_n(x_n, y_n)$ 落在积分曲线 $y = y(x)$ 上（见图 5.3），用 Euler 方法求解时，过点 P_n 以斜率 $f(x_n, y_n)$ 引折线交 $x = x_{n+1}$ 得出顶点 A ，而用后退的 Euler 方法求解时，则以点 $Q(x_{n+1}, y_{n+1})$ 的斜率 $f(x_{n+1}, y_{n+1})$ 从顶点 P_n 引折线交 $x = x_{n+1}$ 得另一顶点 B 。A 和 B 两点均偏离

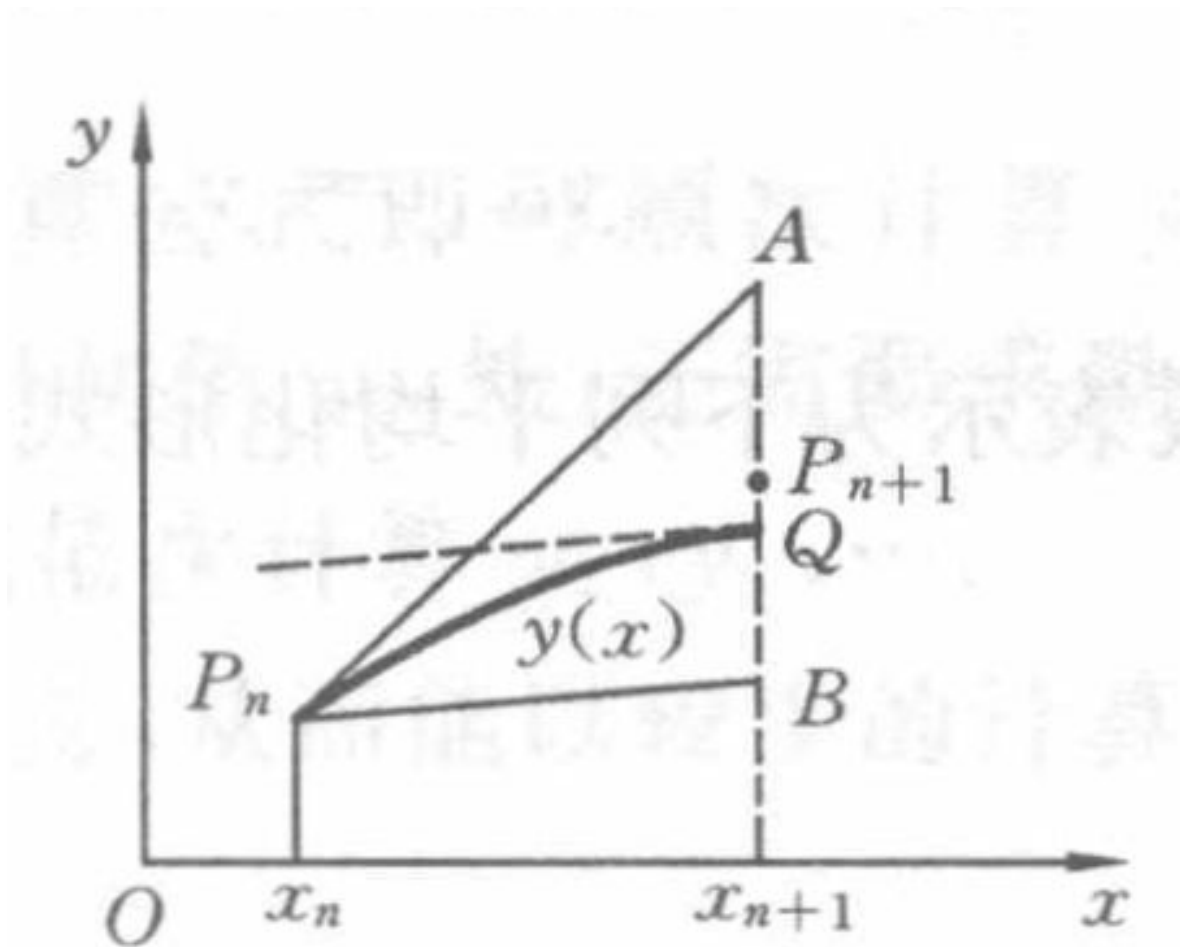


图 5.3

图 5.3 (原图): P_n 出发的两条线分别交 $x = x_{n+1}$ 于 A, B ，取平均所得点为 P_{n+1} ；积分曲线标为 $y(x)$ 。原图全部标签： $O, x, y, x_n, x_{n+1}, P_n, A, B, P_{n+1}, Q, y(x)$ 。

原书 PDF 页 123; 印刷页码 110。

原页

点 Q 比较远, 然而从图形上可以明显地看出, AB 的中点 P_{n+1} 相当接近点 Q , 可见梯形方法确实改善了精度 (这里的解释不够严密, 然而令人信服的)。

梯形方法是隐式的, 可用迭代法求解。同后退的 Euler 方法一样, 仍用 Euler 方法提供迭代初值, 则梯形法的迭代公式为

$$\begin{cases} y_{n+1}^{(0)} = y_n + hf(x_n, y_n), \\ y_{n+1}^{(k+1)} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, y_{n+1}^{(k)})] \end{cases} \quad (k = 0, 1, 2, \dots). \quad (5.2.9)$$

为了分析迭代过程的收敛性, 将式 (5.2.8) 与式 (5.2.9) 相减, 得

$$y_{n+1} - y_{n+1}^{(k+1)} = \frac{h}{2}[f(x_{n+1}, y_{n+1}) - f(x_{n+1}, y_{n+1}^{(k)})],$$

于是有

$$|y_{n+1} - y_{n+1}^{(k+1)}| \leq \frac{hL}{2}|y_{n+1} - y_{n+1}^{(k)}|,$$

其中 L 为 $f(x, y)$ 关于 y 的 Lipschitz 常数。如果选取 h 充分小, 使得 $\frac{hL}{2} < 1$, 则当 $k \rightarrow \infty$ 时有 $y_{n+1}^{(k)} \rightarrow y_{n+1}$, 这说明迭代过程 (5.2.9) 是收敛的 (关于迭代法的收敛性, 可参见 6.2 节)。

5.2.4 改进的 Euler 格式

我们看到, 梯形方法虽然提高了精度, 但其算法复杂, 在应用迭代公式 (5.2.9) 进行实际计算时, 每迭代一次, 都要重新计算函数 f 的值, 而迭代又要反复进行若干次, 计算量很大, 而且往往难以预测。为了控制计算量, 通常希望只迭代一两次就转入下一步计算, 从而简化算法。

具体地说, 先用 Euler 格式求得一个初步的近似值 $\bar{y}_{n+1}$, 称之为**预测值**。预测值 $\bar{y}_{n+1}$ 的精度可能很差, 再用梯形公式 (5.2.8) 将它校正一次, 即按式 (5.2.9) 迭代一次, 得 y_{n+1} , 这个结果称为**校正值**。而这样建立的预测-校正系统通常称为**改进的 Euler 格式**:

预测

$$\bar{y}_{n+1} = y_n + hf(x_n, y_n), \quad (5.2.10)$$

校正

$$y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, \bar{y}_{n+1})]. \quad (5.2.11)$$

这一计算格式亦可表示为

$$y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_n + h, y_n + hf(x_n, y_n))], \quad (5.2.12)$$

或表示为下列平均化形式：

$$\begin{cases} y_p = y_n + hf(x_n, y_n), \\ y_c = y_n + hf(x_{n+1}, y_p), \\ y_{n+1} = \frac{1}{2}(y_p + y_c). \end{cases}$$

原书 PDF 页 124; 印刷页码 111。

[原页](#)

例 5.2 用改进的 Euler 方法求解初值问题 (5.2.2)。

解改进的 Euler 格式为

$$\begin{cases} y_p = y_n + h \left(y_n - \frac{2x_n}{y_n} \right), \\ y_c = y_n + h \left(y_p - \frac{2x_{n+1}}{y_p} \right), \\ y_{n+1} = \frac{1}{2}(y_p + y_c). \end{cases}$$

仍取 $h = 0.1$, 计算结果如表 5.2 所示。与例 5.1 Euler 方法的计算结果比较, 改进的 Euler 方法明显地改善了精度。

表 5.2

x_n	y_n	$y(x_n)$	x_n	y_n	$y(x_n)$
0.1	1.0959	1.0954	0.6	1.4860	1.4832
0.2	1.1841	1.1832	0.7	1.5525	1.5492
0.3	1.2662	1.2649	0.8	1.6153	1.6125
0.4	1.3434	1.3416	0.9	1.6782	1.6733
0.5	1.4164	1.4142	1.0	1.7379	1.7321

5.2.5 Euler 两步格式

再考察改进的 Euler 格式 (5.2.10)、(5.2.11), 可以看到, 其预测公式 (Euler 格式) 的精度差, 与校正公式 (梯形格式) 不匹配。现在构造能与梯形方法在精度上相匹配的显式方法, 为此改用中心差商 $\frac{y(x_{n+1}) - y(x_{n-1}))}{2h}$ 替代方程 (5.2.4) 左端的导数项 $y'(x_n)$, 这时离散化得到所谓 **Euler 两步格式**

$$y_{n+1} = y_{n-1} + 2hf(x_n, y_n). \quad (5.2.13)$$

前面介绍过的数值方法, 无论是 Euler 方法、后退的 Euler 方法, 还是改进的 Euler 方法, 它们都是**单步法**, 其特点是在计算 y_{n+1} 时只用到前面一步的信息 y_n ; 然而, Euler 两步格式除了含 y_n 外, 还显含更前面一步的信息 y_{n+1} ——即调用了前面两步的信息, **Euler 两步法**因此而得名。

单步法的优点是“自开始”, 只要给出初值 y_0 , 依计算公式即可顺次计算 $y_1, y_2, \dots$ 。然而两步法不是自开始的, 在实际计算时, 除了给出初值 y_0 外, 还要求助于其他单步法再提供一个**开始值** y_1 , 然后才能启动计算公式依次计算 $y_2, y_3, \dots$ 。

两步法的优点是, 由于它调用了两个节点上的已知信息, 从而能以较少的计算量获得较高的精度。

现在用 Euler 两步格式与梯形格式相匹配, 得到如下预测-校正系统:

校注 (agent 补充, 原书疑误): 表 5.2 在 $x_n = 0.8$ 的 y_n 原印为 1.6153, 已按原图保留。按例 5.2 所列递推式从 $y_0 = 1$ 、 $h = 0.1$ 计算, 得 $y_7 = 1.5525140913\dots$ 、 $y_8 = 1.6164747827\dots$, 保留四位小数应为 1.6165。这是原书数值疑误, 非 OCR 修正; 其余各行按同一计算与表中四位小数相符。

校注 (agent 补充, 原书下标错误): 本页“两步格式”说明中的“更前面一步的信息 y_{n+1} ”原图确实印为 $n+1$; 按紧邻的式 (5.2.13), 此处应指 y_{n-1} 。正文保留原印下标。

原书 PDF 页 125; 印刷页码 112。

[原页](#)

预测

$$\bar{y}_{n+1} = y_{n-1} + 2hf(x_n, y_n), \quad (5.2.14)$$

校正

$$y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, \bar{y}_{n+1})]. \quad (5.2.15)$$

与改进的 Euler 格式 (5.2.10)、(5.2.11) 比较, 系统 (5.2.14)、(5.2.15) 的一个突出的特点是, 它的预测公式与校正公式具有同等精度。下面将会看到, 据此能比较方便地估计出截断误差, 并且基于这种估计, 可以提供一种提高精度的简易方法。

截断误差的分析仍用 Taylor 方法。假设预测公式 (5.2.14) 中的 y_n 和 y_{n-1} 都是准确的, 即 $y_n = y(x_n)$, $y_{n-1} = y(x_{n-1})$, 则容易验证其局部截断误差

$$y(x_{n+1}) - \bar{y}_{n+1} \approx \frac{h^3}{3}y'''(x_n). \quad (5.2.16)$$

此外, 在分析校正公式 (5.2.15) 的误差时, 假定其中的预测值 $\bar{y}_{n+1}$ 是准确的, 即 $\bar{y}_{n+1} = y(x_{n+1})$, 这时有

$$y(x_{n+1}) - y_{n+1} \approx -\frac{h^3}{12}y'''(x_n). \quad (5.2.17)$$

比较式 (5.2.16) 和式 (5.2.17) 可发现, 校正值的误差 $y(x_{n+1}) - y_{n+1}$ 大约只有预测值的误差 $y(x_{n+1}) - \bar{y}_{n+1}$ 的 1/4 (注意符号相反), 即有

$$\frac{y(x_{n+1}) - y_{n+1}}{y(x_{n+1}) - \bar{y}_{n+1}} \approx -\frac{1}{4}.$$

由此可导出下列事后估计式:

$$\begin{cases} y(x_{n+1}) - \bar{y}_{n+1} \approx -\frac{4}{5}(\bar{y}_{n+1} - y_{n+1}), \\ y(x_{n+1}) - y_{n+1} \approx \frac{1}{5}(\bar{y}_{n+1} - y_{n+1}). \end{cases} \quad (5.2.18)$$

可以期望, 利用这样估计出的误差作为计算结果的一种补偿, 有可能使精度得到改善。

设以 p_n 和 c_n 分别表示第 n 步的预测值和校正值, 按估计式 (5.2.18), $p_{n+1} - \frac{4}{5}(p_{n+1} - c_{n+1})$ 和 $c_{n+1} + \frac{1}{5}(p_{n+1} - c_{n+1})$ 分别可以取作 p_{n+1} 和 c_{n+1} 的改进值。在校正值 c_{n+1} 尚未求出之前, 可用上一步的偏差值 $p_n - c_n$ 替代 $p_{n+1} - c_{n+1}$ 来改进预测值 p_{n+1} , 这样设计的计算方案共有六个环节:

预测

$$p_{n+1} = y_{n-1} + 2hy'_n,$$

改进

$$m_{n+1} = p_{n+1} - \frac{4}{5}(p_n - c_n),$$

计算

$$m'_{n+1} = f(x_{n+1}, m_{n+1}),$$

校正

$$c_{n+1} = y_n + \frac{h}{2}(m'_{n+1} + y'_n),$$

改进

$$y_{n+1} = c_{n+1} + \frac{1}{5}(p_{n+1} - c_{n+1}),$$

原书 PDF 页 126; 印刷页码 113。

[原页](#)

计算

$$y'_{n+1} = f(x_{n+1}, y_{n+1}).$$

运用上述方案计算 y_{n+1} 时, 要用到先一步的信息 $y_n, y'_n, p_n - c_n$ 和更前一步的信息 y_{n-1} , 因此在启动计算之前必须给出开始值 y_1 和 $p_1 - c_1$, y_1 可用其他单步法 (例如改进的 Euler 方法) 来计算, $p_1 - c_1$ 则一般令它等于零。(计算的实践表明, 这种简单的处理方法通常可以获得令人满意的效果。)

5.3 Runge-Kutta 方法

本节介绍一类高精度的单步法——Runge-Kutta 方法。这类方法与下述 Taylor 级数法有着紧密的联系。

5.3.1 Taylor 级数法

设初值问题 (5.1.1)、(5.1.2) 有解 $y = y(x)$, 用 Taylor 展开, 有

$$y(x_{n+1}) = y(x_n) + hy'(x_n) + \frac{h^2}{2!}y''(x_n) + \frac{h^3}{3!}y'''(x_n) + \cdots, \quad (5.3.1)$$

其中 $y(x)$ 的各阶导数依据所给方程 (5.1.1) 可以用函数 f 来表达。下面引进函数序列 $f^{(j)}(x, y)$ 来描述求导过程, 即

$$y' = f \equiv f^{(0)}, \quad y'' = \frac{\partial f^{(0)}}{\partial x} + f \frac{\partial f^{(0)}}{\partial y} \equiv f^{(1)}, \quad y''' = \frac{\partial f^{(1)}}{\partial x} + f \frac{\partial f^{(1)}}{\partial y} \equiv f^{(2)}, \quad \dots$$

一般地

$$y^{(j)} = \frac{\partial f^{(j-2)}}{\partial x} + f \frac{\partial f^{(j-2)}}{\partial y} \equiv f^{(j-1)}, \quad j = 2, 3, \dots,$$

而具体写出则有

$$\begin{cases} y' = f, \\ y'' = \frac{\partial f}{\partial x} + f \frac{\partial f}{\partial y}, \\ y''' = \frac{\partial^2 f}{\partial x^2} + 2f \frac{\partial^2 f}{\partial x \partial y} + f^2 \frac{\partial^2 f}{\partial y^2} + \frac{\partial f}{\partial y} \left(\frac{\partial f}{\partial x} + f \frac{\partial f}{\partial y} \right), \\ \vdots \end{cases} \quad (5.3.2)$$

在展开式 (5.3.1) 的右端截取若干项, 并且在 (x_n, y_n) 按式 (5.3.2) 计算系数 $y^{(j)}(x_n)$ 的近似值 $y_n^{(j)}$, 结果导出下列 **Taylor 格式**

$$y_{n+1} = y_n + hy'_n + \frac{h^2}{2!}y''_n + \cdots + \frac{h^p}{p!}y_n^{(p)}, \quad (5.3.3)$$

其中一阶 Taylor 格式 ($p = 1$)

$$y_{n+1} = y_n + hy'_n$$

就是 Euler 格式，而提高 Taylor 格式的阶 p 即可提高计算结果的精度。显然， p 阶

原书 PDF 页 127; 印刷页码 114。

[原页](#)

Taylor 格式 (5.3.3) 的局部截断误差为

$$y(x_{n+1}) - y_{n+1} = \frac{h^{p+1}}{(p+1)!} y^{(p+1)}(\xi), \quad x_n < \xi < x_{n+1},$$

因此, 它按下述定义具有 p 阶精度。

定义 5.1 如果一种方法的局部截断误差为 $O(h^{p+1})$, 则称该方法具有 p 阶精度。

例 5.3 用 Taylor 级数法求解例 5.1 的初值问题 (5.2.2)。

解 直接求导知

$$\begin{aligned} y' &= y - \frac{2x}{y}, \\ y'' &= y' - \frac{2}{y^2}(y - xy'), \\ y''' &= y'' + \frac{2}{y^2}(xy'' + 2y') - \frac{4x(y')^2}{y^3}, \\ y^{(4)} &= y''' + \frac{2}{y^2}(xy''' + 3y'') - \frac{12y'}{y^3}(xy'' + y') + \frac{12x(y')^3}{y^4}. \end{aligned}$$

据此用四阶 Taylor 格式即可获得令人满意的结果 (仍取步长 $= 0.1$ 计算, 结果如表 5.3 所示)。

表 5.3

x_n	y_n	$y(x_n)$
0.1	1.0954	1.0954
0.2	1.1832	1.1832
0.3	1.2649	1.2649

应当指出, Taylor 格式 (5.3.3) 从形式上看似简单, 但具体构造这种格式往往是相当困难的, 因为它需要按式 (5.3.2) 提供导数值 $y_n^{(j)}$ 。当阶数提高时, 求导过程式 (5.3.2) 可能很复杂, 因此 Taylor 级数法通常不直接使用, 但可以用它来启发思路。

5.3.2 Runge-Kutta 方法的基本思想

Runge-Kutta 方法实质上是间接地使用 Taylor 级数法的一种方法。

考察差商 $\frac{y(x_{n+1}) - y(x_n)}{h}$, 根据微分中值定理, 存在 $0 < \theta < 1$, 使得

$$\frac{y(x_{n+1}) - y(x_n)}{h} = y'(x_n + \theta h),$$

于是, 利用所给方程 $y' = f(x, y)$ 得到

$$y(x_{n+1}) = y(x_n) + hf(x_n + \theta h, y(x_n + \theta h)). \quad (5.3.4)$$

设 $K^* = f(x_n + \theta h, y(x_n + \theta h))$, 称 K^* 为区间 $[x_n, x_{n+1}]$ 上的**平均斜率**。由此可见, 只要对平均斜率提供一种算法, 那么由式 (5.3.4) 便相应地导出一种计算格式。

Euler 格式 (5.2.1) 简单地取点 x_n 的斜率值 $K_1 = f(x_n, y_n)$ 作为平均斜率 K^* ,

原书 PDF 页 128; 印刷页码 115。

[原页](#)

精度自然很低。

再考察改进的 Euler 格式 (5.2.10)、(5.2.11), 它可改写成下列平均化的形式:

$$\begin{cases} y_{n+1} = y_n + \frac{h}{2}(K_1 + K_2), \\ K_1 = f(x_n, y_n), \\ K_2 = f(x_{n+1}, y_n + hK_1). \end{cases}$$

可见, 改进的 Euler 格式可以这样理解: 它用 x_n 与 x_{n+1} 两个点的斜率值 K_1 与 K_2 取算术平均作为平均斜率 K^* , 而 x_{n+1} 处的斜率值 K_2 则通过已知信息 y_n 来预测。

这个处理过程启示我们, 如果设法在 (x_n, x_{n+1}) 内多预测几个点的斜率值, 然后将它们加权平均作为平均斜率 K^* , 则有可能构造出具有更高精度的计算格式。这就是 Runge-Kutta 方法的基本思想。

5.3.3 二阶 Runge-Kutta 方法

首先推广改进的 Euler 方法, 随意考察区间 (x_n, x_{n+1}) 内一点

$$x_{n+p} = x_n + ph, \quad 0 < p \leq 1,$$

希望用 x_n 和 x_{n+p} 两个点的斜率值 K_1 和 K_2 线性组合得到平均斜率 K^* , 即令

$$y_{n+1} = y_n + h(\lambda_1 K_1 + \lambda_2 K_2),$$

其中 λ_1, λ_2 为待定系数。同改进的 Euler 格式一样, 这里仍取 $K_1 = f(x_n, y_n)$, 问题在于该怎样预测 x_{n+p} 处的斜率值 K_2 。

仿照改进的 Euler 格式, 先用 Euler 格式提供 $y(x_{n+p})$ 的预测值, 即

$$y_{n+p} = y_n + phK_1,$$

然后再用预测值 y_{n+p} 通过计算 f 产生斜率值

$$K_2 = f(x_{n+p}, y_{n+p}).$$

这样设计出的计算格式具有如下形式:

$$\begin{cases} y_{n+1} = y_n + h(\lambda_1 K_1 + \lambda_2 K_2), \\ K_1 = f(x_n, y_n), \\ K_2 = f(x_{n+p}, y_n + phK_1). \end{cases} \quad (5.3.5)$$

式 (5.3.5) 含有三个待定系数 λ_1, λ_2 和 p , 希望适当选取这些系数的值, 使得格式具有二阶精度。

根据式 (5.3.3), 二阶 Taylor 格式为

$$y_{n+1} = y_n + hy'_n + \frac{h^2}{2}y''_n,$$

或表示为

$$y_{n+1} = y_n + hf_n + \frac{h^2}{2}(f_x + f \cdot f_y)_n,$$

其中 f_n 和 $(f_x + f \cdot f_y)_n$ 的下标 n 均表示在 (x_n, y_n) 取值。另一方面, 有

原书 PDF 页 129; 印刷页码 116。

[原页](#)

$$K_1 = f_n, \quad K_2 = f(x_{n+p}, y_n + phK_1) = f_n + ph(f_x + f \cdot f_y)_n + \cdots,$$

代入式 (5.3.5), 得

$$y_{n+1} = y_n + (\lambda_1 + \lambda_2)hf_n + \lambda_2ph^2(f_x + f \cdot f_y)_n + \cdots.$$

由此可见, 欲使格式 (5.3.5) 具有二阶精度, 只要成立

$$\begin{cases} \lambda_1 + \lambda_2 = 1, \\ \lambda_2 p = \frac{1}{2}. \end{cases} \quad (5.3.6)$$

满足条件 (5.3.6) 的一族格式 (5.3.5) 统称为二阶 **Runge-Kutta 格式**。

除了改进的 Euler 格式外, 另一种特殊的二阶 Runge-Kutta 格式是所谓**变形的 Euler 格式**, 其形式如下:

$$\begin{cases} y_{n+1} = y_n + hK_2, \\ K_1 = f(x_n, y_n), \\ K_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_1\right). \end{cases}$$

表面上看, 变形的 Euler 格式 $y_{n+1} = y_n + hK_2$ 仅显含一个斜率值 K_2 , 但 K_2 是通过 K_1 计算出来的, 因此每完成一步仍然需要两次计算函数 f 的值, 工作量和改进的 Euler 格式相同。

总之, 二阶 Runge-Kutta 方法用多算一次函数值 f 的办法, 避开了二阶 Taylor 级数法所要求的 f 导数值的计算。在这种意义上可以说, Runge-Kutta 方法实质上是 Taylor 方法的变形。

5.3.4 三阶 Runge-Kutta 方法

为了进一步提高精度, 设除 x_n, x_{n+p} 外再考察一点 $x_{n+q} = x_n + qh$, $p \leq q \leq 1$, 并用三个点 x_n, x_{n+p}, x_{n+q} 的斜率值 K_1, K_2, K_3 线性组合得到平均斜率 K^* , 这时计算格式为

$$y_{n+1} = y_n + h(\lambda_1 K_1 + \lambda_2 K_2 + \lambda_3 K_3).$$

其中 K_1, K_2 仍用格式 (5.3.5) 所取的形式。

为了预测点 x_{n+p} 处的斜率值 K_3 , 在区间 (x_n, x_{n+q}) 内有两个斜率值 K_1 和 K_2 可以利用。用 K_1 和 K_2 线性组合给出区间 $[x_n, x_{n+q}]$ 上的平均斜率, 从而得到 $y(x_{n+q})$ 的预测值 $y_{n+q} = y_n + qh(rK_1 + sK_2)$, 于是, 再通过计算函数值 f 得到

$$K_3 = f(x_{n+q}, y_{n+q}) = f(x_n + qh, y_n + qh(rK_1 + sK_2)).$$

这样设计出的计算格式具有如下形式：

$$\begin{cases} y_{n+1} = y_n + h(\lambda_1 K_1 + \lambda_2 K_2 + \lambda_3 K_3), \\ K_1 = f(x_n, y_n), \\ K_2 = f(x_n + ph, y_n + phK_1), \\ K_3 = f(x_n + qh, y_n + qh(rK_1 + sK_2)). \end{cases} \quad (5.3.7)$$

校注 (agent 补充, 原书下标错误): 5.3.4 首段“再考察一点 $x_{n+p} = x_n + qh$ ”及后段“为了预测点 x_{n+p} 处的斜率值 K_3 ”，原图两处均印为 $n+p$ ，已保留。按同页对 $K_3 = f(x_{n+q}, y_{n+q})$ 的定义及式 (5.3.7)，这两处所指节点均应为 x_{n+q} 。已放大原印字确认，非 OCR 误识。

原书 PDF 页 130; 印刷页码 117。

[原页](#)

希望适当选择系数 $\lambda_1, \lambda_2, \lambda_3$ 和 p, q, r, s , 能使上述格式具有三阶精度。

为便于进行数学演算, 引进算子

$$D = \frac{\partial}{\partial x} + f \frac{\partial}{\partial y}, \quad D^2 = \frac{\partial^2}{\partial x^2} + 2f \frac{\partial^2}{\partial x \partial y} + f^2 \frac{\partial^2}{\partial y^2},$$

则根据式 (5.3.2), 有

$$y' = f, \quad y'' = Df, \quad y''' = D^2 f + \frac{\partial f}{\partial y} Df,$$

于是三阶 Taylor 格式

$$y_{n+1} = y_n + hy'_n + \frac{h^2}{2} y''_n + \frac{h^3}{6} y'''_n$$

可表示为

$$y_{n+1} = y_n + hf_n + \frac{h^2}{2} Df_n + \frac{h^3}{6} (D^2 f + f_y Df)_n, \quad (5.3.8)$$

其中 f_n, Df_n 等的下标 n 均表示在 (x_n, y_n) 取值。

另一方面, 再将式 (5.3.7) Taylor 展开, 其中

$$K_1 = f_n,$$

$$K_2 = f(x_n + ph, y_n + phK_1) = f_n + phDf_n + \frac{(ph)^2}{2} D^2 f_n + \cdots,$$

$$K_3 = f(x_n + qh, y_n + qh(rK_1 + sK_2)) = f_n + qh\bar{D}f_n + \frac{(qh)^2}{2} \bar{D}^2 f_n + \cdots.$$

这里, 算子 $\bar{D} = \frac{\partial}{\partial x} + (rK_1 + sK_2) \frac{\partial}{\partial y}$ 。

若取

$$r + s = 1, \quad (5.3.9)$$

则易知

$$\bar{D} = D + hpsDf_n \frac{\partial}{\partial y} + \cdots, \quad (5.3.10)$$

$$\overline{D}^2 = D^2 + \cdots,$$

因此

$$K_3 = f_n + qhDf_n + \frac{h^2}{2}(q^2D^2f + 2pqsDf \cdot f_y)_n + \cdots.$$

将以上 K_1, K_2, K_3 的展开式一起代入式 (5.3.7), 再令

$$\lambda_1 + \lambda_2 + \lambda_3 = 1, \quad (5.3.11)$$

则有

$$\begin{aligned} y_{n+1} = & y_n + hf_n + h^2(\lambda_2p + \lambda_3q)Df_n \\ & + h^3 \left(\frac{1}{2}\lambda_2p^2D^2f + \frac{1}{2}\lambda_3q^2D^2f + \lambda_3pqs f_y Df \right)_n + \cdots, \end{aligned}$$

于是为了保证它能与三阶 Taylor 格式 (5.3.8) 具有同等精度, 只要满足下列方程组:

$$\begin{cases} \lambda_2p + \lambda_3q = \frac{1}{2}, \\ \lambda_2p^2 + \lambda_3q^2 = \frac{1}{3}, \\ \lambda_3pqs = \frac{1}{6}. \end{cases} \quad (5.3.12)$$

原书 PDF 页 131; 印刷页码 118。

[原页](#)

满足条件 (5.3.9)、(5.3.11)、(5.3.12) 的一族格式 (5.3.7) 统称三阶 **Runge-Kutta 格式**。下列 Kutta 格式是其中的一个特例:

$$\begin{cases} y_{n+1} = y_n + \frac{h}{6}(K_1 + 4K_2 + K_3), \\ K_1 = f(x_n, y_n), \\ K_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_1\right), \\ K_3 = f(x_n + h, y_n - hK_1 + 2hK_2). \end{cases}$$

5.3.5 四阶 Runge-Kutta 方法

继续上述过程, 经过较复杂的数学演算, 可以导出各种四阶 Runge-Kutta 格式, 下列**经典格式**是其中常用的一个:

$$\begin{cases} y_{n+1} = y_n + \frac{h}{6}(K_1 + 2K_2 + 2K_3 + K_4), \\ K_1 = f(x_n, y_n), \\ K_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_1\right), \\ K_3 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_2\right), \\ K_4 = f(x_n + h, y_n + hK_3). \end{cases} \quad (5.3.13)$$

四阶 Runge-Kutta 方法的每一步需要四次计算函数值 f , 可以证明其截断误差为 $O(h^5)$ 。其证明极其烦琐, 这里从略。

例 5.4 设取步长 $h = 0.2$, 从 $x = 0$ 直到 $x = 1$ 用四阶 Runge-Kutta 方法求解初值问题 (5.2.2)。

解这里, 经典的四阶 Runge-Kutta 格式 (5.3.13) 具有如下形式:

$$\begin{cases} y_{n+1} = y_n + \frac{h}{6}(K_1 + 2K_2 + 2K_3 + K_4), \\ K_1 = y_n - \frac{2x_n}{y_n}, \\ K_2 = y_n + \frac{h}{2}K_1 - \frac{2x_n + h}{y_n + \frac{h}{2}K_1}, \\ K_3 = y_n + \frac{h}{2}K_2 - \frac{2x_n + h}{y_n + \frac{h}{2}K_2}, \\ K_4 = y_n + hK_3 - \frac{2(x_n + h)}{y_n + hK_3}. \end{cases}$$

表 5.4 列出计算结果 y_n , 其中 $y(x_n)$ 仍表示准确解。

原书 PDF 页 132; 印刷页码 119。

[原页](#)

比较例 5.4 和例 5.2 的计算结果, 显然以四阶 Runge-Kutta 方法的精度为高。注意, 虽然四阶 Runge-Kutta 方法的计算量 (每一步要四次计算函数 f) 比改进的 Euler 方法 (它是一种二阶 Runge-Kutta 方法, 每一步只要两次计算函数 f) 大一倍, 但由于这里放大了步长 ($h = 0.2$), 造出表 5.4 和表 5.2 所耗费的计算量几乎相同。这个例子又一次显示了选择算法的重要意义。

表 5.4

x_n	y_n	$y(x_n)$
0.2	1.1832	1.1832
0.4	1.3417	1.3416
0.6	1.4833	1.4832
0.8	1.6125	1.6125
1.0	1.7321	1.7321

然而值得指出的是, Runge-Kutta 方法的推导基于 Taylor 展开方法, 因而它要求所求的解具有较好的光滑性质。反之, 如果解的光滑性差, 那么, 使用四阶 Runge-Kutta 方法可能反而不如使用改进的 Euler 方法求得的数值解的精度高。实际计算时, 应当针对问题的具体特点选择合适的算法。

5.3.6 变步长的 Runge-Kutta 方法

单从每一步看, 步长越小, 截断误差就越小; 但随着步长的缩小, 在一定求解范围内所要完成的步数就增加了。步数的增加不但引起计算量的增大, 而且可能导致舍入误差的严重积累。因此同积分的数值计算一样, 微分方程的数值解法也有个选择步长的问题。

在选择步长时, 需要考虑两个问题:

- (1) 怎样衡量和检验计算结果的精度?
- (2) 如何依据所获得的精度处理步长?

考察经典的四阶 Runge-Kutta 格式 (5.3.13)。从节点 x_n 出发, 先以 h 为步长求出一个近似值, 作为 $y_{n+1}^{(h)}$, 由于格式的局部截断误差为 $O(h^5)$, 故有

$$y(x_{n+1}) - y_{n+1}^{(h)} \approx Ch^5; \quad (5.3.14)$$

然后将步长折半, 即取 $\frac{h}{2}$ 为步长从 x_n 跨两步到 x_{n+1} , 再求得一个近似值 $y_{n+1}^{(h/2)}$, 每跨一步的截断误差是 $C\left(\frac{h}{2}\right)^5$, 因此有

$$y(x_{n+1}) - y_{n+1}^{(h/2)} = 2C\left(\frac{h}{2}\right)^5; \quad (5.3.15)$$

比较式 (5.3.14) 和式 (5.3.15) 可以看到, 步长折半后, 误差大约减小到 $\frac{1}{16}$, 即有

$$\frac{y(x_{n+1}) - y_{n+1}^{(h/2)}}{y(x_{n+1}) - y_{n+1}^{(h)}} \approx \frac{1}{16}.$$

由此易得下列事后估计式

$$y(x_{n+1}) - y_{n+1}^{(h/2)} \approx \frac{1}{15}(y_{n+1}^{(h/2)} - y_{n+1}^{(h)}).$$

原书 PDF 页 133; 印刷页码 120。

[原页](#)

这样, 可以通过检查步长折半前后两次计算结果的偏差

$$\Delta = |y_{n+1}^{(h/2)} - y_{n+1}^{(h)}|$$

来判定所选的步长是否合适。具体地说, 将分以下两种情况处理:

- (1) 对于给定的精度 ε , 如果 $\Delta > \varepsilon$, 应反复将步长折半进行计算, 直至 $\Delta < \varepsilon$ 为止, 这时取最终得到的 $y_{n+1}^{(h/2)}$ 作为结果;
- (2) 如果 $\Delta < \varepsilon$, 则反复将步长加倍, 直至 $\Delta > \varepsilon$ 为止, 这时再将步长折半一次, 就得到所要的结果。

这种通过加倍或折半处理步长的方法称为**变步长方法**。表面上看, 为了选择步长, 每一步的计算量增加了, 但从总体考虑往往是合算的。

5.4 单步法的收敛性和稳定性

5.4.1 单步法的收敛性

我们看到, 数值解法的基本思想是, 通过某种离散化手段, 将微分方程转化为差分方程 (代数方程) 来求解。这种转化是否合理, 还要看差分问题的解 y_n 当 $h \rightarrow 0$ 时是否会收敛到微分方程的准确解 $y(x_n)$ 。需要注意的是, 如果只考虑 $h \rightarrow 0$, 那么节点 $x_n = x_0 + nh$ 对于固定的 n 将趋于 x_0 , 这时讨论收敛性是没有意义的。

定义 5.2 若一种数值方法对于任意固定的 $x_n = x_0 + nh$, 当 $h \rightarrow 0$ (同时 $n \rightarrow \infty$) 时有 $y_n \rightarrow y(x_n)$, 则称该方法是**收敛的**。

先就简单的初值问题

$$\begin{cases} y' = \lambda y, \\ y(0) = y_0 \end{cases} \quad (5.4.1)$$

考察 Euler 方法的收敛性。方程 $y' = \lambda y$ 的 Euler 格式是

$$y_{n+1} = (1 + \lambda h)y_n, \quad (5.4.2)$$

它显然有解

$$y_n = y_0(1 + \lambda h)^n.$$

由于这里 $x_0 = 0, x_n = nh$, 有

$$y_n = y_0 \left[(1 + \lambda h)^{\frac{1}{\lambda h}} \right]^{\lambda x_n}.$$

再注意到当 $h \rightarrow 0$ 时, 有

$$(1 + \lambda h)^{\frac{1}{\lambda h}} \rightarrow e,$$

差分方程的解 y_n 当 $h \rightarrow 0$ 时确定收敛到原微分方程的准确解

$$y(x_n) = y_0 e^{\lambda x_n}.$$

下面进一步考察一般的单步法。

所谓单步法，就是在计算 y_{n+1} 时只用到它前一步的信息 y_n 。Taylor 级数法、

原书 PDF 页 134; 印刷页码 121。

[原页](#)

Runge-Kutta 方法等都是单步法的例子。显式单步法的共同特征是, 它们都是将 y_n 加上某种形式的增量得出 y_{n+1} 的, 其计算公式形如

$$y_{n+1} = y_n + h\varphi(x_n, y_n, h), \quad (5.4.3)$$

其中 $\varphi(x, y, h)$ 称为**增量函数**。

不同的单步法对应于不同的增量函数。不难就已知的几种单步法写出增量函数 $\varphi(x, y, h)$ 的具体形式, 譬如, 对于 Euler 格式 (5.2.1), 有 $\varphi = f(x, y)$, 而对于改进的 Euler 格式 (5.2.12), 有

$$\varphi = \frac{1}{2}[f(x, y) + f(x + h, y + hf(x, y))]. \quad (5.4.4)$$

关于单步法有下述收敛性定理。

定理 5.1 假设单步法 (5.4.3) 具有 p 阶精度, 且增量函数 $\varphi(x, y, h)$ 关于 y 满足 Lipschitz 条件

$$|\varphi(x, y, h) - \varphi(x, \bar{y}, h)| \leq L_\varphi(y - \bar{y}), \quad (5.4.5)$$

又设初值 y_0 是准确的, 即 $y_0 = y(x_0)$, 则其**整体截断误差**

$$y(x_n) - y_n = O(h^p). \quad (5.4.6)$$

证明 设以 $\bar{y}_{n+1}$ 表示取 $y_n = y(x_n)$ 用格式 (5.4.3) 求得的结果, 即

$$\bar{y}_{n+1} = y(x_n) + h\varphi(x_n, y(x_n), h), \quad (5.4.7)$$

其局部截断误差为 $y(x_{n+1}) - \bar{y}_{n+1}$, 由于所给方法具有 p 阶精度, 按定义 5.1, 存在定数 C , 使

$$|y(x_{n+1}) - \bar{y}_{n+1}| \leq Ch^{p+1}.$$

又由式 (5.4.7) 与式 (5.4.3), 得

$$|\bar{y}_{n+1} - y_{n+1}| \leq |y(x_n) - y_n| + h|\varphi(x_n, y(x_n), h) - \varphi(x_n, y_n, h)|.$$

利用假设条件 (5.4.5), 有

$$|\bar{y}_{n+1} - y_{n+1}| \leq (1 + hL_\varphi)|y(x_n) - y_n|,$$

从而有

$$\begin{aligned} |y(x_{n+1}) - y_{n+1}| &\leq |\bar{y}_{n+1} - y_{n+1}| + |y(x_{n+1}) - \bar{y}_{n+1}| \\ &\leq (1 + hL_\varphi)|y(x_n) - y_n| + Ch^{p+1}, \end{aligned}$$

即对于整体截断误差 $e_n = y(x_n) - y_n$ 成立下列递推关系式:

$$|e_{n+1}| \leq (1 + hL_\varphi)|e_n| + Ch^{p+1}. \quad (5.4.8)$$

据此不等式反复递推, 可得

$$|e_n| \leq (1 + hL_\varphi)^n |e_0| + \frac{Ch^q}{L_\varphi} [(1 + hL_\varphi)^n - 1]. \quad (5.4.9)$$

再注意到当 $x_n - x_0 = nh \leq T$ 时³

校注 (agent 补充, 原书公式错误): 式 (5.4.5) 原图右端为 $L_\varphi(y - \bar{y})$, 没有绝对值符号, 已原样保留。标准 Lipschitz 条件及随后对非负误差的估计需要右端为 $L_\varphi|y - \bar{y}|$; 当 $y < \bar{y}$ 且 $L_\varphi > 0$ 时, 原印右端为负, 不能作为绝对差的上界。

校注 (agent 补充, 原书指数错误): 式 (5.4.9) 原印为 Ch^q/L_φ , 已放大确认 q 并保留。按式 (5.4.8) 的等比求和, $Ch^{p+1} \sum_{j=0}^{n-1} (1 + hL_\varphi)^j = \frac{Ch^p}{L_\varphi} [(1 + hL_\varphi)^n - 1]$, 故此处指数应为 p ; 后页式 (5.4.10) 也用 p 。

³对于任意实数 x , 有 $1 + x \leq e^x$, 而当 $x \geq -1$ 时, 成立 $0 \leq (1 + x)^n \leq e^{nx}$ 。

原书 PDF 页 135; 印刷页码 122。

[原页](#)

$$(1 + hL_\varphi)^n \leq (e^{hL_\varphi})^n \leq e^{nL_\varphi},$$

最终得估计式

$$|e_n| \leq |e_0|e^{TL_\varphi} + \frac{Ch^p}{L_\varphi}(e^{TL_\varphi} - 1). \quad (5.4.10)$$

由此可以断定, 如果初值是准确的, 即 $e_0 = 0$, 则式 (5.4.6) 成立。证毕。

依据这一定理, 判断单步法 (5.4.3) 的收敛性, 归结为验证增量函数 φ 能否满足 Lipschitz 条件 (5.4.5)。

对 Euler 方法, 由于其增量函数 φ 就是 f , 故当 f 满足 Lipschitz 条件时它是收敛的。

再考察改进的 Euler 方法, 其增量函数已由式 (5.4.4) 给出, 这时有

$$\begin{aligned} & |\varphi(x, y, h) - \varphi(x, \bar{y}, h)| \\ & \leq \frac{1}{2} [|f(x, y) - f(x, \bar{y})| + |f(x + h, y + hf(x, y)) \\ & \quad - f(x + h, \bar{y} + hf(x, \bar{y}))|]. \end{aligned}$$

假定 f 关于 y 满足 Lipschitz 条件, 记 Lipschitz 常数为 L , 则由上式推得

$$|\varphi(x, y, h) - \varphi(x, \bar{y}, h)| \leq L \left(1 + \frac{h}{2}L\right) |y - \bar{y}|.$$

设定 $h \leq h_0$ (h_0 为定数), 上式表明 φ 关于 y 的 Lipschitz 常数

$$L_\varphi = L \left(1 + \frac{h_0}{2}L\right),$$

因此, 改进的 Euler 方法也是收敛的。

类似地, 不难验证其他 Runge-Kutta 方法的收敛性。

5.4.2 单步法的稳定性

前面关于收敛性的讨论有个前提, 必须假定数值方法本身的计算是准确的。实际情形并不是这样, 差分方程的求解还会有计算误差, 譬如由于数字舍入而引起的小扰动。这类小扰动在传播过程中会不会恶性增长, 以致“淹没”了差分方程的“真解”呢? 这就是差分方法的稳定性问题。在实际计算时, 希望某一步产生的扰动值在后面的计算中能够被控制, 甚至是逐步衰减的。

定义 5.3 若一种数值方法在节点值 y_n 上产生大小为 δ 的扰动, 在以后各节点值 y_m ($m > n$) 上产生的偏差均不超过 δ , 则称该方法是稳定的。

稳定性问题比较复杂, 为简化讨论, 仅考察下列模型方程 $y' = \lambda y$ 。为保证微分方程本身的稳定性, 假定 $\lambda < 0$ 。

先研究 Euler 方法的稳定性。模型方程 $y' = \lambda y$ 的 Euler 格式为

$$y_{n+1} = (1 + h\lambda)y_n. \quad (5.4.11)$$

设在节点值 y_n 上有一扰动值 ε_n ，它的传播使节点值 y_{n+1} 产生大小为 ε_{n+1} 的扰动值，

校注 (agent 补充, 原书指数错误): 页首原印为 $(1 + hL_\varphi)^n \leq (e^{nL_\varphi})^n \leq e^{hL_\varphi}$, 放大确认后按原式保留。由前页 $nh \leq T$ 及 $1 + u \leq e^u$, 所需估计应为 $(1 + hL_\varphi)^n \leq (e^{hL_\varphi})^n = e^{nhL_\varphi} \leq e^{TL_\varphi}$, 与本页式 (5.4.10) 一致。

原书 PDF 页 136; 印刷页码 123。

[原页](#)

假设用 $y_n^* = y_n + \varepsilon_n$ 按 Euler 格式得出 $y_{n+1}^* = y_{n+1} + \varepsilon_{n+1}$ 的计算过程不再有新的误差, 则扰动值满足

$$\varepsilon_{n+1} = (1 + h\lambda)\varepsilon_n.$$

可见扰动值满足原来的差分方程 (5.4.11)。这样, 如果差分方程的解是不增长的, 即有

$$|y_{n+1}| \leq |y_n|,$$

则它就是稳定的。这一论断对于下面将要研究的其他方法同样适用。

显然, 为要保证差分方程 (5.4.11) 的解是不增长的, 只要选取 h 充分小, 使

$$|1 + h\lambda| \leq 1. \quad (5.4.12)$$

这说明 Euler 方法是**条件稳定的**, 其稳定性条件为 $h \leq -2/\lambda$ 。记 $\tau = -1/\lambda$, 则稳定性条件 (5.4.12) 可表示为

$$h \leq 2\tau. \quad (5.4.13)$$

若自变量 x 表示时间, 则 τ 是一个具有时间量纲的量, 工程上习惯地称它为**时间常数**。时间常数 τ 可用来刻画原方程的解 $y(x)$ 的衰减速度。事实上, 在时间 τ 内, 解 $y = y(0)e^{\lambda x}$ 衰减到它的 e^{-1} 倍。因此, τ 越小, 解 $y(x)$ 衰减得越快。

Euler 方法的稳定性条件 $h \leq 2\tau$ 表明, 时间常数 τ 越小, 稳定性对步长 h 的限制越苛刻。

再考察后退的 Euler 方法。对于模型方程 $y' = \lambda y$, 其后退的 Euler 格式为

$$y_{n+1} = y_n + h\lambda y_{n+1},$$

解出 y_{n+1} , 有 $y_{n+1} = \frac{1}{1 - h\lambda} y_n$ 。由于 $\lambda < 0$, 这时恒成立

$$\left| \frac{1}{1 - h\lambda} \right| \leq 1,$$

从而有 $|y_{n+1}| \leq |y_n|$, 因而后退的 Euler 方法**恒稳定** (或称**无条件稳定**)。

例 5.5 考察初值问题

$$\begin{cases} y' = -100y, \\ y(0) = 1, \end{cases}$$

其准确解 $y = e^{-100x}$ 是一个按指数曲线衰减得很快的函数, 如图 5.4 所示。

对于所给方程 $y' = -100y$, 其时间常数 $\tau = 0.01$, 因此, 按式 (5.4.13), 为保证 Euler 方法稳定, 步长 h 应使 τ 不超过 $2\tau = 0.02$ 。若取步长 $h = 0.025$, 则 Euler 格式的具体形式为

$$y_{n+1} = -1.5y_n,$$

计算结果列于表 5.5 的第二列。可以看到，Euler 方法的解 y_n （图 5.4 中用符号“■”标出）

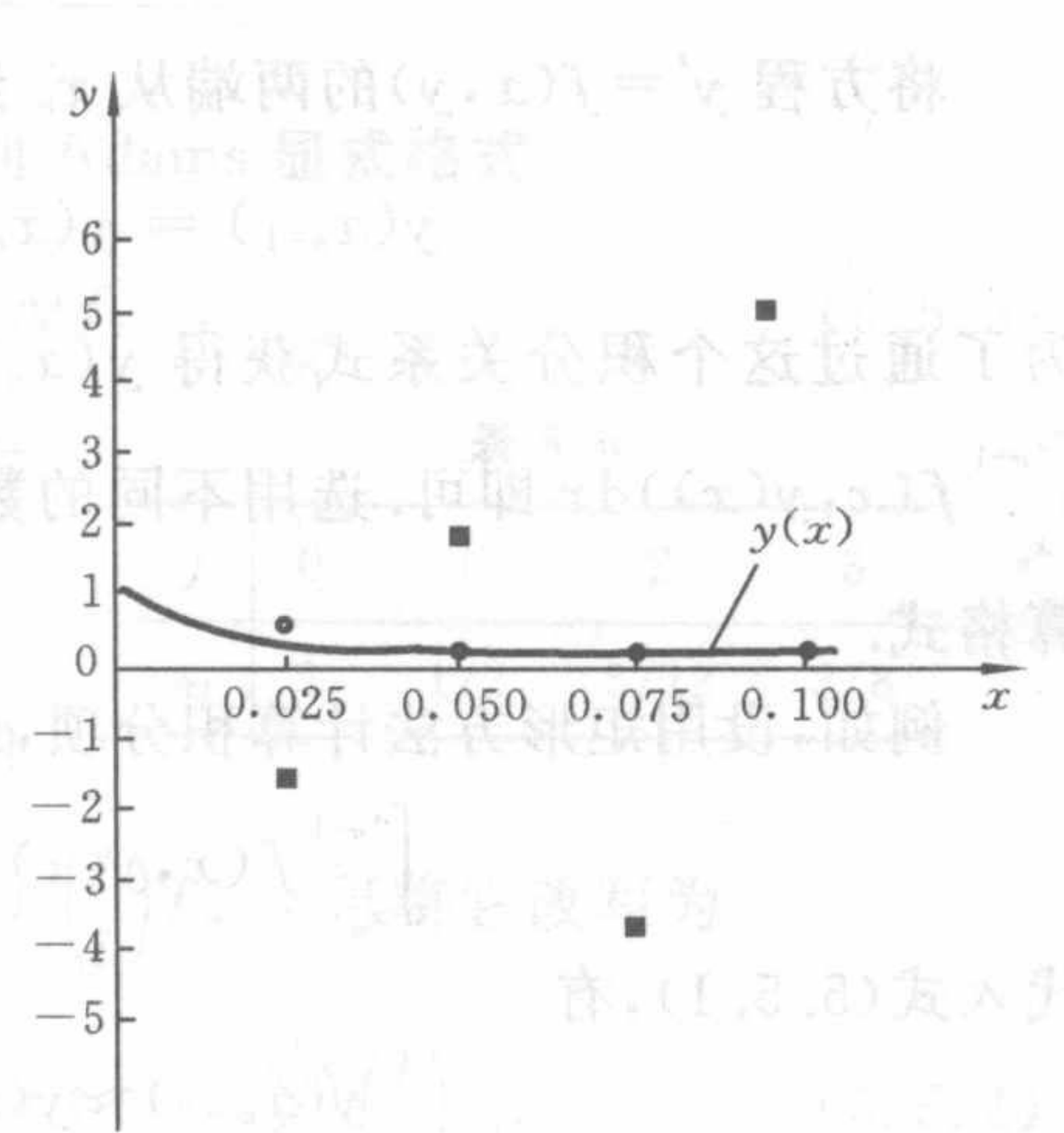


图 5.4

图 5.4（原图）：坐标系中画出衰减曲线 $y(x)$ 、正负交替且幅度增大的方形离散点，以及靠近横轴的圆形离散点。原图全部文字与刻度标签： $x, y, y(x)$ ；横轴 0.025, 0.050, 0.075, 0.100；纵轴 -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6。点的说明随例 5.5 跨页续述。

校注（agent 补充，原书文字/符号错误）：例 5.5 原印“步长 h 应使 τ 不超过 $2\tau = 0.02$ ”，已经放大确认并保留。式 (5.4.13) 约束的是步长 h ，应为“步长 h 应不超过 $2\tau = 0.02$ ”；原句中的“使 τ ”使被约束量错置。

原书 PDF 页 137；印刷页码 124。

[原页](#)

(接上页) 在准确值 $y(x_n)$ 的上下波动，计算过程明显地不稳定。

表 5.5

节点	Euler 方法	后退的 Euler 方法
0.025	−1.5	0.2857
0.050	2.25	0.0816
0.075	−3.375	0.0233
0.100	5.0625	0.0067

再考察后退的 Euler 方法，取 $h = 0.025$ 时计算格式为

$$y_{n+1} = \frac{1}{3.5}y_n.$$

计算结果列于表 5.5 的第三列（图 5.4 中用符号 “●” 标出），这时计算过程是稳定的。

5.5 线性多步法

在逐步推进的求解过程中，计算 y_{n+1} 之前事实上已经求出了一系列的近似值 $y_n, y_{n-1}, y_{n-2}, \cdots$ ，如果充分利用前面多步的信息来预测 y_{n+1} ，则可以期望会获得较高的精度。这就是构造所谓线性多步法的基本思想。

构造多步法有多种途径，下面介绍其中两种：基于数值积分的构造方法和基于 Taylor 展开的构造方法。

5.5.1 基于数值积分的构造方法

将方程 $y' = f(x, y)$ 的两端从 x_n 到 x_{n+1} 求积分，得

$$y(x_{n+1}) = y(x_n) + \int_{x_n}^{x_{n+1}} f(x, y(x)) \, dx. \tag{5.5.1}$$

为了通过这个积分关系式获得 $y(x_{n+1})$ 的近似值，只要近似地算出其中的积分项 $\int_{x_n}^{x_{n+1}} f(x, y(x)) \, dx$ 即可。选用不同的数值方法计算这个积分项，就会导出不同的计算格式。

例如，设用矩形方法计算积分项，即

$$\int_{x_n}^{x_{n+1}} f(x, y(x)) \, dx \approx hf(x_n, y(x_n)),$$

代入式 (5.5.1)，有

$$y(x_{n+1}) \approx y(x_n) + hf(x_n, y(x_n)),$$

据此离散化即可导出 Euler 格式。

原书 PDF 页 138; 印刷页码 125。

[原页](#)

为了改善精度, 可以改用梯形方法计算积分项, 即

$$\int_{x_n}^{x_{n+1}} f(x, y(x)) \mathrm{d}x \approx \frac{h}{2} [f(x_n, y(x_n)) + f(x_{n+1}, y(x_{n+1}))],$$

再代入式 (5.5.1), 有

$$y(x_{n+1}) \approx y(x_n) + \frac{h}{2} [f(x_n, y(x_n)) + f(x_{n+1}, y(x_{n+1}))],$$

据此离散化即可导出格式 (5.2.8), **梯形法**因此而得名。

我们知道, 基于插值原理可以建立一系列的数值积分方法, 运用这些方法可以导出求解微分方程的一系列的计算格式。一般地, 设已构造出 $f(x, y(x))$ 的插值多项式 $P_r(x)$, 那么, 计算 $\int_{x_n}^{x_{n+1}} P_r(x) \mathrm{d}x$ 作为 $\int_{x_n}^{x_{n+1}} f(x, y(x)) \mathrm{d}x$ 的近似值, 即可将式 (5.5.1) 离散化得到下列计算公式:

$$y_{n+1} = y_n + \int_{x_n}^{x_{n+1}} P_r(x) \mathrm{d}x. \quad (5.5.2)$$

5.5.2 Adams 显式格式

运用插值方法的关键, 在于选取合适的插值节点。记 $f_k = f(x_k, y_k)$, 先用 $r+1$ 个数据点 $(x_n, f_n), (x_{n-1}, f_{n-1}), \dots, (x_{n-r}, f_{n-r})$ 构造插值多项式 $P_r(x)$, 注意到这里插值节点 $x_n, x_{n-1}, \dots, x_{n-r}$ 等距, 运用 Newton 后插公式 (见 2.4.2 节) 可写出

$$P_r(x_n + th) = \sum_{j=0}^r (-1)^j \binom{-t}{j} \Delta^j f_{n-j},$$

其中 $t = \frac{x - x_n}{h}$, Δ^j 表示 j 阶向前差分, 而

$$\binom{s}{j} = \frac{s(s-1)\cdots(s-j+1)}{j!}.$$

将 $P_r(x)$ 的上述表达式代入式 (5.5.2), 即得下列 **Adams 显式格式**

$$y_{n+1} = y_n + h \sum_{j=0}^r \alpha_j \Delta^j f_{n-j}, \quad (5.5.3)$$

其中 α_j 为不依赖于 n 和 r 的系数, $\alpha_j = (-1)^j \int_0^1 \binom{-t}{j} \mathrm{d}t$ 。它的前几个值如表 5.6 所示。

表 5.6

j	0	1	2	3
α_j	1	1/2	5/12	3/8

实际计算时, 将格式 (5.5.3) 中的差分展开往往是方便的, 利用差分展开式 $\Delta^j f_{n-j} = \sum_{i=0}^r (-1)^i \binom{j}{i} f_{n-j}$, 可将它改写为

$$y_{n+1} = y_n + h \sum_{i=0}^r \beta_{ri} f_{n-i}, \quad \beta_{ri} = (-1)^i \sum_{j=i}^r \binom{j}{i} \alpha_j. \quad (5.5.4)$$

校注 (agent 补充): 原图中上述未编号“差分展开式”的求和上限印作 r , 求和项末尾印作 f_{n-j} , 这里均保留原文。末尾下标疑为原书排印错误, 通常应为 f_{n-i} ; 例如 $j = r = 1$ 时, 原印刷式右边为零, 而左边为 $f_n - f_{n-1}$ 。按通常写法求和上限为 j ; 若约定 $\binom{j}{i} = 0$ ($i > j$), 则上限写 r 也可。此疑误已对照原页局部图, 并非 OCR 替换。

原书 PDF 页 139; 印刷页码 126。

[原页](#)

这里, 系数 β_{ri} 与 r 的定值有关, 其具体数值如表 5.7 所示。式 (5.5.4) 是含有参数 r 的一族格式, $r+1$ 为格式的步数。特别地, 一步显式 Adams 格式 ($r=0$) 即为 Euler 格式。两步显式 Adams 格式 ($r=1$) 为

$$y_{n+1} = y_n + \frac{h}{2}(3f_n - f_{n-1}), \quad (5.5.5)$$

表 5.7

i	0	1	2	3
β_{0i}	1			
β_{1i}	3/2	-1/2		
β_{2i}	23/12	-16/12	5/12	
β_{3i}	55/24	-59/24	37/24	-9/24

而四步显式 Adams 格式 ($r=3$) 则为

$$y_{n+1} = y_n + \frac{h}{24}(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}). \quad (5.5.6)$$

5.5.3 Adams 隐式格式

上述 Adams 显式方法选 $x_n, x_{n-1}, \dots, x_{n-r}$ 作为插值节点, 这时, 用插值函数 $P_r(x)$ 在求积区间 $[x_n, x_{n+1}]$ 上逼近 $f(x, y(x))$, 实际上是个外推过程, 效果不够理想。为了改善逼近效果, 可以变外推为内插, 改用 $x_{n+1}, x_n, \dots, x_{n-r+1}$ 为插值节点; 而通过数据点 $(x_{n+1}, f_{n+1}), (x_n, f_n), \dots, (x_{n-r+1}, f_{n-r+1})$ 插出函数 $P_r(x)$, 然后重复前一段的推导过程。对应于式 (5.5.3), 这里有下列 **Adams 隐式格式**:

$$y_{n+1} = y_n + h \sum_{j=0}^r \alpha_j^* \Delta^j f_{n-j+1}, \quad \alpha_j^* = (-1)^j \int_{-1}^0 \binom{-t}{j} dt. \quad (5.5.7)$$

它的前几个值如表 5.8 所示。

表 5.8

j	0	1	2	3
α_j^*	1	-1/2	-1/12	-1/24

表 5.9

i	0	1	2	3
β_{0i}^*	1			
β_{1i}^*	1/2	1/2		

i	0	1	2	3
β_{2i}^*	5/12	8/12	-1/12	
β_{3i}^*	9/24	19/24	-5/24	1/24

将差分展开，式 (5.5.7) 可改写为

$$y_{n+1} = y_n + h \sum_{i=0}^r \beta_{ri}^* f_{n-i+1}, \quad \beta_{ri}^* = (-1)^i \sum_{j=i}^r \binom{j}{i} \alpha_j^*.$$

(5.5.8)

表 5.9 提供了系数 β_{ri}^* 的部分数值。式 (5.5.8) 是含有参数 r 的一族格式， $r+1$ 为格式的步数。特别地，一步隐式 Adams 格式 ($r=0$) 为后退的 Euler 格式。两步隐式 Adams 格式 ($r=1$) 为梯形格式，而四步隐式 Adams 格式 ($r=3$) 则为

$$y_{n+1} = y_n + \frac{h}{24}(9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}).$$

(5.5.9)

原书 PDF 页 140; 印刷页码 127。

[原页](#)

5.5.4 Adams 预测-校正系统

可以同前述 Euler 方法一样分析 Adams 方法的误差。譬如, 对于格式 (5.5.6), 假设 $y_{n-k} = y(x_{n-k})$ ($k = 0, 1, 2, 3$), 这时

$$f_{n-k} = f(x_{n-k}, y_{n-k}) = f(x_{n-k}, y(x_{n-k})) = y'(x_{n-k}) \quad (k = 0, 1, 2, 3),$$

代入式 (5.5.6), 有

$$y_{n+1} = y(x_n) + \frac{h}{24}[55y'(x_n) - 59y'(x_{n-1}) + 37y'(x_{n-2}) - 9y'(x_{n-3})].$$

将上式右端各项在点 x_n 展开, 得

$$\begin{aligned} y_{n+1} &= y(x_n) + hy'(x_n) + \frac{h^2}{2}y''(x_n) + \frac{h^3}{6}y'''(x_n) \\ &\quad + \frac{h^4}{24}y^{(4)}(x_n) - \frac{49}{144}h^5y^{(5)}(x_n) + \cdots. \end{aligned}$$

另一方面, 对于准确解 $y(x_{n+1})$, 有

$$y(x_{n+1}) = y(x_n) + hy'(x_n) + \frac{h^2}{2}y''(x_n) + \frac{h^3}{6}y'''(x_n) + \frac{h^4}{24}y^{(4)}(x_n) + \frac{h^5}{120}y^{(5)}(x_n) + \cdots,$$

于是显式格式 (5.5.6) 的局部截断误差为

$$y(x_{n+1}) - y_{n+1} \approx \frac{251}{720}h^5y^{(5)}(x_n). \quad (5.5.10)$$

类似地可以导出隐式格式 (5.5.9) 的局部截断误差为

$$y(x_{n+1}) - y_{n+1} \approx -\frac{19}{720}h^5y^{(5)}(x_n). \quad (5.5.11)$$

注意, 显式格式 (5.5.6) 与隐式格式 (5.5.9) 都具有四阶精度, 这两种格式可匹配成下列 Adams **预测-校正系统**:

预测

$$\bar{y}_{n+1} = y_n + \frac{h}{24}(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}), \quad (5.5.12)$$

$$\bar{f}_{n+1} = f(x_{n+1}, \bar{y}_{n+1});$$

校正

$$y_{n+1} = y_n + \frac{h}{24}(9\bar{f}_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}), \quad (5.5.13)$$

$$f_{n+1} = f(x_{n+1}, y_{n+1}).$$

这种预测-校正方法是四步法，用它在计算 y_{n+1} 时，不但要用到前一步的信息 y_n, f_n ，而且要用到更前三步的信息 $f_{n-1}, f_{n-2}, f_{n-3}$ ，因此它不是自开始的。实际计算时，必须借助某种单步法，如四阶 Taylor 格式（见 5.3.1 节）或四阶 Runge-Kutta 格式（见 5.3.5 节）为它提供开始值 y_1, y_2, y_3 。

例 5.6 用 Adams 预测-校正系统 (5.5.12)、(5.5.13) 求解初值问题 (5.2.2)。

解 取步长 $h = 0.1$ 计算。用例 5.3 按四阶 Taylor 格式求得的结果 y_1, y_2, y_3 作为开始值，然后启动格式 (5.5.12)、(5.5.13) 进行计算。计算结果如表 5.10 所示。表

原书 PDF 页 141; 印刷页码 128。

[原页](#)

(接上页) 中 $\bar{y}_n$ 和 y_n 分别为预测值和校正值, 同时列出了准确值 $y(x_n)$ 以比较计算结果的精度。

依据误差公式 (5.5.10) 和式 (5.5.11), 对于系统 (5.5.12)、(5.5.13) 中的预测值 $\bar{y}_{n+1}$ 和校正值 y_{n+1} , 分别有

$$y(x_{n+1}) - \bar{y}_{n+1} \approx \frac{251}{720} h^5 y^{(5)}(x_n),$$

$$y(x_{n+1}) - y_{n+1} \approx -\frac{19}{720} h^5 y^{(5)}(x_n).$$

于是有下列事后估计式:

$$y(x_{n+1}) - \bar{y}_{n+1} \approx -\frac{251}{720} (\bar{y}_{n+1} - y_{n+1}),$$

$$y(x_{n+1}) - y_{n+1} \approx \frac{19}{720} (\bar{y}_{n+1} - y_{n+1}).$$

表 5.10

x_n	$\bar{y}_n$	y_n	$y(x_n)$
0		1	1
0.1		1.0954	1.0954
0.2		1.1832	1.1832
0.3		1.2649	1.2649
0.4	1.3415	1.3416	1.3416
0.5	1.4141	1.4142	1.4142
0.6	1.4832	1.4832	1.4832
0.7	1.5491	1.5492	1.5492
0.8	1.6125	1.6125	1.6125
0.9	1.6734	1.6734	1.6733
1.0	1.7321	1.7321	1.7231

利用这一结果, 可将 Adams 预测-校正系统 (5.5.12)、(5.5.13) 进一步加工成下列计算方案:

预测

$$p_{n+1} = y_n + \frac{h}{24} (55y'_n - 59y'_{n-1} + 37y'_{n-2} - 9y'_{n-3}),$$

改进

$$m_{n+1} = p_{n+1} + \frac{251}{720} (c_n - p_n),$$

计算

$$m_{n+1} = f(x_{n+1}, m_{n+1}),$$

校正

$$c_{n+1} = y_n + \frac{h}{24}(9m'_{n+1} + 19y'_n - 5y'_{n-1} + y'_{n-2}),$$

改进

$$y_{n+1} = c_{n+1} - \frac{19}{720}(c_{n+1} - p_{n+1}),$$

计算

$$y'_{n+1} = f(x_{n+1}, y_{n+1}).$$

关于这种计算方案，读者可参看 5.2 节末的有关论述。

5.5.5 基于 Taylor 展开的构造方法

我们看到，基于数值积分可以构造出一系列求解常微分方程的计算格式，然而下面将要介绍的 Taylor 展开方法更具有—般性，各种线性多步格式（包括前述 Adams 格式）均可通过这种方法构造出来。

一般的线性多步格式具有下列形式：

$$y_{n+1} = \alpha_0 y_n + \alpha_1 y_{n-1} + \cdots + \alpha_r y_{n-r} + h(\beta_{-1} y'_{n+1} + \beta_0 y'_n + \beta_1 y'_{n-1} + \cdots + \beta_r y'_{n-r}),$$

缩记作

校注（agent 补充，ch05b-E002）：本页两条“事后估计式”及计算方案两条“改进”式的分母均印为 720，已照录。由本页前两条局部误差式相减， $y_{n+1} - \bar{y}_{n+1} \approx (270/720)h^5 y^{(5)}(x_n)$ ，消去 $h^5 y^{(5)}(x_n)$ 后，事后估计的系数应为 251/270 和 19/270；相应改进式中的分母也应为 270。这是原书疑误，非 OCR 错。

校注（agent 补充，ch05b-E003）：计算方案第一条“计算”式左端印为 m_{n+1} ，未带撇号；已照录。按下一条校正式使用的 m'_{n+1} 及导数定义，此处疑漏印撇号，应为 $m'_{n+1} = f(x_{n+1}, m_{n+1})$ 。

校注（agent 补充，ch05b-E004）：表 5.10 末行准确值原印为 1.7231，已照录。已另行目视核对 PDF 120（书 107 页）式（5.2.2）后所列解析解 $y = \sqrt{1+2x}$ 及表 5.1； $y(1) = \sqrt{3} \approx 1.7321$ ，因此本格疑有数字对调。

原书 PDF 页 142; 印刷页码 129。

[原页](#)

$$y_{n+1} = \sum_{k=0}^r \alpha_k y_{n-k} + h \sum_{k=-1}^r \beta_k y'_{n-k}, \quad (5.5.14)$$

其中 $y'_{n-k} = f(x_{n-k}, y_{n-k})$ 。注意到 $y'_{n+1} = f(x_{n+1}, y_{n+1})$ 中含有未知的 y_{n+1} ，则当 $\beta_{-1} = 0$ 时，格式 (5.5.14) 是显式的， $\beta_{-1} \neq 0$ 时则是隐式的。

设 $y_{n-k} = y(x_{n-k}), y'_{n-k} = y'(x_{n-k})$ ，由 Taylor 展开，有

$$\begin{aligned} y_{n-k} &= \sum_{j=0}^p \frac{(-kh)^j}{j!} y_n^{(j)} + \frac{(-kh)^{p+1}}{(p+1)!} y_n^{(p+1)} + \cdots, \\ y'_{n-k} &= \sum_{j=1}^p \frac{(-kh)^{j-1}}{(j-1)!} y_n^{(j)} + \frac{(-kh)^p}{p!} y_n^{(p+1)} + \cdots. \end{aligned}$$

代入式 (5.5.14)，整理得

$$\begin{aligned} y_{n+1} &= \left(\sum_{k=0}^r \alpha_k \right) y_n + \sum_{j=1}^p \frac{h^j}{j!} \left[\sum_{k=1}^r (-k)^j \alpha_k + j \sum_{k=-1}^r (-k)^{j-1} \beta_k \right] y_n^{(j)} \\ &\quad + \frac{h^{p+1}}{(p+1)!} \left[\sum_{k=1}^r (-k)^{p+1} \alpha_k + (p+1) \sum_{k=-1}^r (-k)^p \beta_k \right] y_n^{(p+1)} + \cdots. \end{aligned} \quad (5.5.15)$$

于是，欲使格式 (5.5.14) 成为 p 阶精度的，即局部截断误差为 $O(h^{p+1})$ ，只要令展开式 (5.5.15) 与 $y(x_{n+1})$ 的 Taylor 展开式

$$y(x_{n+1}) = \sum_{j=0}^p \frac{h^j}{j!} y_n^{(j)} + \frac{h^{p+1}}{(p+1)!} y_n^{(p+1)} + \cdots \quad (5.5.16)$$

能符合到 h^p 项，为此要求成立

$$\begin{cases} \sum_{k=0}^r \alpha_k = 1, \\ \sum_{k=1}^r (-k)^j \alpha_k + j \sum_{k=-1}^r (-k)^{j-1} \beta_k = 1 \quad (j = 1, 2, \dots, p). \end{cases} \quad (5.5.17)$$

如果格式 (5.5.14) 的系数满足这组条件，则将式 (5.5.16) 与式 (5.5.15) 相减，即得局部截断误差

$$\begin{aligned} &y(x_{n+1}) - y_{n+1} \\ &= \frac{h^{p+1}}{(p+1)!} \left[1 - \sum_{k=1}^r (-k)^{p+1} \alpha_k - (p+1) \sum_{k=-1}^r (-k)^p \beta_k \right] y_n^{(p+1)} + \cdots. \end{aligned} \quad (5.5.18)$$

下面进一步具体地考察四步方法。先研究下列形式的四步显式格式：

$$y_{n+1} = \alpha_0 y_n + \alpha_1 y_{n-1} + \alpha_2 y_{n-2} + h(\beta_0 y'_n + \beta_1 y'_{n-1} + \beta_2 y'_{n-2} + \beta_3 y'_{n-3}), \quad (5.5.19)$$

欲使这类格式成为四阶的，按式 (5.5.17)，其系数应当满足条件

$$\begin{cases} \alpha_0 + \alpha_1 + \alpha_2 = 1, \\ -\alpha_1 - 2\alpha_2 + \beta_0 + \beta_1 + \beta_2 + \beta_3 = 1, \\ \alpha_1 + 4\alpha_2 - 2\beta_1 - 4\beta_2 - 6\beta_3 = 1, \\ -\alpha_1 - 8\alpha_2 + 3\beta_1 + 12\beta_2 + 27\beta_3 = 1, \\ \alpha_1 + 16\alpha_2 - 4\beta_1 - 32\beta_2 - 108\beta_3 = 1. \end{cases} \quad (5.5.20)$$

原书 PDF 页 143; 印刷页码 130。

[原页](#)

这里有七个待定系数 $\alpha_0, \alpha_1, \alpha_2, \beta_0, \beta_1, \beta_2, \beta_3$, 然而它们所要适合的条件只有五个, 因此有两个自由度。设令 $\alpha_1 = \alpha_2 = 0$, 求解式 (5.5.20) 得到

$$\alpha_0 = 1, \quad \beta_0 = \frac{55}{24}, \quad \beta_1 = -\frac{59}{24}, \quad \beta_2 = \frac{37}{24}, \quad \beta_3 = -\frac{9}{24}.$$

这时式 (5.5.19) 为四阶 Adams 格式 (5.5.6)。另外, 用定出的系数值具体地代入式 (5.5.18), 即可导出误差公式 (5.5.10)。

为了提供与式 (5.5.19) 相匹配的隐式方法, 舍弃其中的 y'_{n-3} 而代之以 y'_{n+1} , 有

$$y_{n+1} = \alpha_0 y_{n+1} + \alpha_1 y_{n-1} + \alpha_2 y_{n-2} + h(\beta_{-1} y'_{n+1} + \beta_0 y'_n + \beta_1 y'_{n-1} + \beta_2 y'_{n-2}). \quad (5.5.21)$$

为使这类格式具有四阶精度, 按式 (5.5.17), 系数应当满足条件

$$\begin{cases} \alpha_0 + \alpha_1 + \alpha_2 = 1, \\ -\alpha_1 - 2\alpha_2 + \beta_{-1} + \beta_0 + \beta_1 + \beta_2 = 1, \\ \alpha_1 + 4\alpha_2 + 2\beta_{-1} - 2\beta_1 - 4\beta_2 = 1, \\ -\alpha_1 - 8\alpha_2 + 3\beta_{-1} + 3\beta_1 + 12\beta_2 = 1, \\ \alpha_1 + 16\alpha_2 + 4\beta_{-1} - 4\beta_1 - 32\beta_2 = 1. \end{cases} \quad (5.5.22)$$

特别地, 取 $\alpha_1 = \alpha_2 = 0$ 即得四阶 Adams 格式 (5.5.9), 通过这一途径同时可以导出误差公式 (5.5.11)。

5.5.6 Milne 格式

与格式 (5.5.19) 不同的另一类格式是用 y_{n-3} 而不用 y'_{n-3} , 其形式为

$$y_{n+1} = \alpha_0 y_n + \alpha_1 y_{n-1} + \alpha_2 y_{n-2} + \alpha_3 y_{n-3} + h(\beta_0 y'_n + \beta_1 y'_{n-1} + \beta_2 y'_{n-2}). \quad (5.5.23)$$

类似于前面的讨论, 运用 Taylor 展开方法不难导出形如式 (5.5.23) 的一类四阶方法。这类方法中包含有著名的 Milne 格式

$$y_{n+1} = y_{n-3} + \frac{4h}{3}(2y'_n - y'_{n-1} + 2y'_{n-2}), \quad (5.5.24)$$

其局部截断误差为

$$y(x_{n+1}) - y_{n+1} \approx \frac{14}{45} h^5 y_n^{(5)}. \quad (5.5.25)$$

Milne 格式也可以通过数值积分的途径推导出来。事实上, 从积分恒等式

$$y(x_{n+1}) = y(x_{n-3}) + \int_{x_{n-3}}^{x_{n+1}} y'(x) dx \quad (5.5.26)$$

着手, 用数据点 $(x_n, y'_n), (x_{n-1}, y'_{n-1}), (x_{n-2}, y'_{n-2})$ 插出二次式 $P_2(x)$, 然后用 $P_2(x)$ 替代 $y'(x)$ 计算式 (5.5.26) 中的积分项, 即可得到上述格式 (5.5.24)。

再在形如式 (5.5.21) 的四阶隐式方法中适当挑选一个与 Milne 格式相匹配。在式 (5.5.22) 中令 $\alpha_1 = 1, \alpha_2 = 0$, 可定出下列四阶格式:

校注 (agent 补充, ch05b-E005): 式 (5.5.21) 右端开头在原图中印作 $\alpha_0 y_{n+1} \alpha_1 y_{n-1}$ (“+1” 下沉至下标位置, 且两项之间无正常加号), 上式按可见字形保留。由本页 “舍弃 y'_{n-3} 而代之以 y'_{n+1} ” 及式 (5.5.19)、(5.5.22), 此处疑为排印错误, 预期应是 $\alpha_0 y_n + \alpha_1 y_{n-1}$ 。局部原图如下, 仅用于保存这一排印疑误的可审计证据。

式 (5.5.21) 右端开头在原图中印作 $\alpha_0 y_{n+1} \alpha_1 y_{n-1}$ (“+1” 下沉至下标位置, 且两项之间无正常加号), 上式按可见字形保留。

$$y_{n+1} = \alpha_0 y_{n+1} \alpha_1 y_{n-1} + \alpha_2 y_{n-2} + h(\beta_{-1} y'_{n+1} + \beta_0 y'_n + \beta_1 y'_{n-1} + \beta_2 y'_{n-2}). \quad (5.5.24)$$

式 (5.5.21) 右端开头在原图中印作 $\alpha_0 y_{n+1} \alpha_1 y_{n-1}$ (“+1” 下沉至下标位置, 且两项之间无正常加号), 上式按可见字形保留。

原书 PDF 页 144; 印刷页码 131。

[原页](#)

$$y_{n+1} = y_{n-1} + \frac{h}{3}(y'_{n+1} + 4y'_n + y'_{n-1}). \quad (5.5.27)$$

上述格式通常称为 Simpson 格式, 用数值积分的 Simpson 方法处理积分恒等式

$$y(x_{n+1}) = y(x_{n-1}) + \int_{x_{n-1}}^{x_{n+1}} y'(x) dx$$

即可导出这一格式。

用 Simpson 格式 (5.5.27) 与 Milne 格式 (5.5.24) 匹配, 构成下列预测-校正系统:

预测

$$\bar{y}_{n+1} = y_{n-3} + \frac{4h}{3}(2y'_n - y'_{n-1} + 2y'_{n-2}), \quad \bar{y}'_{n+1} = f(x_{n+1}, \bar{y}_{n+1});$$

校正

$$y_{n+1} = y_{n-1} + \frac{h}{3}(\bar{y}'_{n+1} + 4y'_n + y'_{n-1}), \quad y'_{n+1} = f(x_{n+1}, y_{n+1}).$$

5.5.7 Hamming 格式

上述预测-校正系统的缺点是稳定性差。为了改善稳定性, 重新挑选校正公式。为此考察式 (5.5.21) 中不显含 y'_{n-2} 的一类格式

$$y_{n+1} = \alpha_0 y_n + \alpha_1 y_{n-1} + \alpha_2 y_{n-2} + h(\beta_{-1} y'_{n+1} + \beta_0 y'_n + \beta_1 y'_{n-1}), \quad (5.5.28)$$

按条件 (5.5.22), 形如式 (5.5.28) 的四阶格式含有一个自由度——譬如可取 α_1 为独立参数。若取 $\alpha_1 = 1$ 即得 Simpson 格式 (5.5.27)。

Hamming 用 α_1 的不同数值进行试验, 发现当 $\alpha_1 = 0$ 时格式的稳定性能较好, 即

$$y_{n+1} = \frac{1}{8}(9y_n - y_{n-2}) + \frac{3h}{8}(y'_{n+1} + 2y'_n - y'_{n-1}), \quad (5.5.29)$$

其局部截断误差为

$$y(x_{n+1}) - y_{n+1} \approx -\frac{h^5}{40} y_n^{(5)}. \quad (5.5.30)$$

Hamming 格式 (5.5.29) 不能用数值积分方法推导出来。

用 Milne 格式 (5.5.24) 与 Hamming 格式 (5.5.29) 相匹配, 并利用误差公式 (5.5.25)、(5.5.30) 改进计算结果, 即可建立下列预测-校正系统:

预测

$$p_{n+1} = y_{n-3} + \frac{4h}{3}(2y'_n - y'_{n-1} + 2y'_{n-2}),$$

改进

$$m_{n+1} = p_{n+1} - \frac{112}{121}(p_n - c_n),$$

计算

$$m'_{n+1} = f(x_{n+1}, m_{n+1}),$$

校正

$$c_{n+1} = \frac{1}{8}(9y_n - y_{n-2}) + \frac{3h}{8}(m'_{n+1} + 2y'_n - y'_{n-1}),$$

改进

$$y_{n+1} = c_{n+1} + \frac{9}{121}(p_{n+1} - c_{n+1}),$$

原书 PDF 页 145; 印刷页码 132。

[原页](#)

计算

$$y'_{n+1} = f(x_{n+1}, y_{n+1}).$$

5.6 方程组与高阶方程的情形

5.6.1 一阶方程组

前面研究了单个方程 $y' = f$ 的数值解法, 只要把 y 和 f 理解为向量, 那么, 所提供的各种计算格式即可应用到一阶方程组的情形。

考察一阶方程组

$$y'_i = f_i(x, y_1, y_2, \dots, y_N) \quad (i = 1, 2, \dots, N)$$

的初值问题, 初始条件为

$$y_i(x_0) = y_i^0 \quad (i = 1, 2, \dots, N).$$

若采用向量的记号, 记

$$y = (y_1, y_2, \dots, y_N)^T, \quad y^0 = (y_1^0, y_2^0, \dots, y_N^0)^T,$$

$$f = (f_1, f_2, \dots, f_N)^T,$$

则上述方程组的初值问题可表示为 $\begin{cases} y' = f(x, y), \\ y(x_0) = y_0. \end{cases}$ 求解这一初值问题的四阶 Runge-Kutta 格式为

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

其中

$$k_1 = f(x_n, y_n), \quad k_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right),$$

$$k_3 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right), \quad k_4 = f(x_n + h, y_n + hk_3);$$

或表示为

$$y_{i,n+1} = y_{in} + \frac{h}{6}(K_{i1} + 2K_{i2} + 2K_{i3} + K_{i4}) \quad (i = 1, 2, \dots, N),$$

其中

$$\begin{aligned} K_{i1} &= f_i(x_n, y_{1n}, y_{2n}, \dots, y_{Nn}), \\ K_{i2} &= f_i\left(x_n + \frac{h}{2}, y_{1n} + \frac{h}{2}K_{11}, y_{2n} + \frac{h}{2}K_{21}, \dots, y_{Nn} + \frac{h}{2}K_{N1}\right), \\ K_{i3} &= f_i\left(x_n + \frac{h}{2}, y_{1n} + \frac{h}{2}K_{12}, y_{2n} + \frac{h}{2}K_{22}, \dots, y_{Nn} + \frac{h}{2}K_{N2}\right), \\ K_{i4} &= f_i(x_n + h, y_{1n} + hK_{13}, y_{2n} + hK_{23}, \dots, y_{Nn} + hK_{N3}). \end{aligned}$$

这里 y_{in} 是第 i 个因变量 $y_i(x)$ 在节点 $x_n = x_0 + nh$ 的近似值。

为了帮助理解这一格式的计算过程，再考察以下两个方程的特殊情形：

原书 PDF 页 146; 印刷页码 133。

[原页](#)

$$\begin{cases} y' = f(x, y, z), & y(x_0) = y_0, \\ z' = g(x, y, z), & z(x_0) = z_0, \end{cases}$$

这时四阶 Runge-Kutta 格式具有如下形式:

$$\begin{cases} y_{n+1} = y_n + \frac{h}{6}(K_1 + 2K_2 + 2K_3 + K_4), \\ z_{n+1} = z_n + \frac{h}{6}(L_1 + 2L_2 + 2L_3 + L_4), \end{cases} \quad (5.6.1)$$

其中

$$\begin{cases} K_1 = f(x_n, y_n, z_n), \\ K_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_1, z_n + \frac{h}{2}L_1\right), \\ K_3 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_2, z_n + \frac{h}{2}L_2\right), \\ K_4 = f(x_n + h, y_n + hK_3, z_n + hL_3), \\ L_1 = g(x_n, y_n, z_n), \\ L_2 = g\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_1, z_n + \frac{h}{2}L_1\right), \\ L_3 = g\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_2, z_n + \frac{h}{2}L_2\right), \\ L_4 = g(x_n + h, y_n + hK_3, z_n + hL_3). \end{cases} \quad (5.6.2)$$

这是一步法, 利用节点 x_n 上的值 y_n, z_n , 由式 (5.6.2) 顺序计算 $K_1, L_1, K_2, L_2, K_3, L_3, K_4, L_4$, 然后代入式 (5.6.1) 即可求得节点 x_{n+1} 上的 y_{n+1}, z_{n+1} 。

5.6.2 化高阶方程组为一阶方程组

关于高阶微分方程 (或方程组) 的初值问题, 原则上总可以归结为一阶方程组来求解。例如, 考察下列 m 阶微分方程:

$$y^{(m)} = f(x, y, y', \dots, y^{(m-1)}), \quad (5.6.3)$$

初始条件为

$$y(x_0) = y_0, \quad y'(x_0) = y'_0, \quad \dots, \quad y^{(m-1)}(x_0) = y_0^{(m-1)}. \quad (5.6.4)$$

只要引进新的变量 $y_1 = y, y_2 = y', \dots, y_m = y^{(m-1)}$, 即可将 m 阶方程 (5.6.3) 化为如下的一阶方程组:

$$\begin{cases} y_1' = y_2, \\ y_2' = y_3, \\ \vdots \\ y_{m-1}' = y_m, \\ y_m' = f(x, y_1, y_2, \cdots, y_m); \end{cases}$$

(5.6.5)

初始条件 (5.6.4) 则相应地化为

原书 PDF 页 147; 印刷页码 134。

[原页](#)

$$y_1(x_0) = y_0, \quad y_2(x_0) = y'_0, \quad \cdots, \quad y_m(x_0) = y_0^{(m-1)}. \quad (5.6.6)$$

不难证明, 初值问题 (5.6.3)、(5.6.4) 和 (5.6.5)、(5.6.6) 是彼此等价的。

特别地, 对于下列二阶方程的初值问题:

$$\begin{cases} y'' = f(x, y, y'), \\ y(x_0) = y_0, \quad y'(x_0) = y'_0. \end{cases}$$

引进新的变量 $z = y'$, 即可化为下列一阶方程组的初值问题:

$$\begin{cases} y' = z, \quad y(x_0) = y_0, \\ z' = f(x, y, z), \quad z(x_0) = y'_0. \end{cases}$$

针对这个问题应用四阶 Runge-Kutta 格式 (5.6.1), 有

$$\begin{cases} y_{n+1} = y_n + \frac{h}{6}(K_1 + 2K_2 + 2K_3 + K_4), \\ z_{n+1} = z_n + \frac{h}{6}(L_1 + 2L_2 + 2L_3 + L_4). \end{cases}$$

按式 (5.6.2), 有

$$K_1 = z_n, \quad L_1 = f(x_n, y_n, z_n);$$

$$K_2 = z_n + \frac{h}{2}L_1, \quad L_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_1, z_n + \frac{h}{2}L_1\right);$$

$$K_3 = z_n + \frac{h}{2}L_2, \quad L_3 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_2, z_n + \frac{h}{2}L_2\right);$$

$$K_4 = z_n + hL_3, \quad L_4 = f(x_n + h, y_n + hK_3, z_n + hL_3).$$

如果消去 K_1, K_2, K_3, K_4 , 则上述格式可表示为

$$\begin{cases} y_{n+1} = y_n + hz_n + \frac{h^2}{6}(L_1 + L_2 + L_3), \\ z_{n+1} = z_n + \frac{h}{6}(L_1 + 2L_2 + 2L_3 + L_4), \end{cases}$$

这里

$$L_1 = f(x_n, y_n, z_n),$$

$$L_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}z_n, z_n + \frac{h}{2}L_1\right),$$

$$L_3 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}z_n + \frac{h^2}{4}L_1, z_n + \frac{h}{2}L_2\right),$$

$$L_4 = f\left(x_n + h, y_n + hz_n + \frac{h^2}{2}L_2, z_n + hL_3\right).$$

5.7 边值问题的数值解法

在具体求解微分方程时，必须附加某种定解条件。微分方程和定解条件一起组成定

原书 PDF 页 148; 印刷页码 135。

[原页](#)

(接上页) 解问题。对于高阶微分方程, 定解条件通常有两种给法: 一种是给出了积分曲线在初始时刻的性态, 这类条件称为**初始条件**, 相应的定解问题称为**初值问题**; 另一种是给出了积分曲线首末两端的性态, 这类条件则称为**边界条件**, 相应的定解问题称为**边值问题**。

以二阶方程

$$y'' = f(x, y, y') \quad (5.7.1)$$

为例, 其初始条件的给法是

$$y(a) = \alpha, \quad y'(a) = m. \quad (5.7.2)$$

前面已经研究过这种初值问题的数值解法。

设在区间 $a < x < b$ 上求解方程 (5.7.1), 则边界条件可以给为

$$y(a) = \alpha, \quad y(b) = \beta. \quad (5.7.3)$$

5.7.1 试射法

可以用所谓“试射法”来求解边值问题 (5.7.1)、(5.7.3)。这种方法的基本思想是设法将边值问题转化为初值问题来求解, 即依据边界条件 (5.7.3) 寻求与它等价的初始条件 (5.7.2), 也就是说, 反复调整初始时刻的斜率值 $y'(a) = m$, 使初值问题 (5.7.1)、(5.7.2) 的积分曲线 $y = y(x)$ 能“命中” $y(b) = \beta$ (见图 5.5)。

试射法的计算过程很简单。设凭经验能够提供 m 的两个预测值 m_1, m_2 , 分别按这两个斜率值“试射”——求解相应的初值问题 (5.7.1)、(5.7.2), 从而获得 $y(b)$ 的两个结果 β_1, β_2 。如果 β_1 与 β_2 均不满足预定的精度, 就用线性插值方法校正 m_1, m_2 得新的斜率值

$$m_3 = m_1 + \frac{m_2 - m_1}{\beta_2 - \beta_1}(\beta - \beta_1),$$

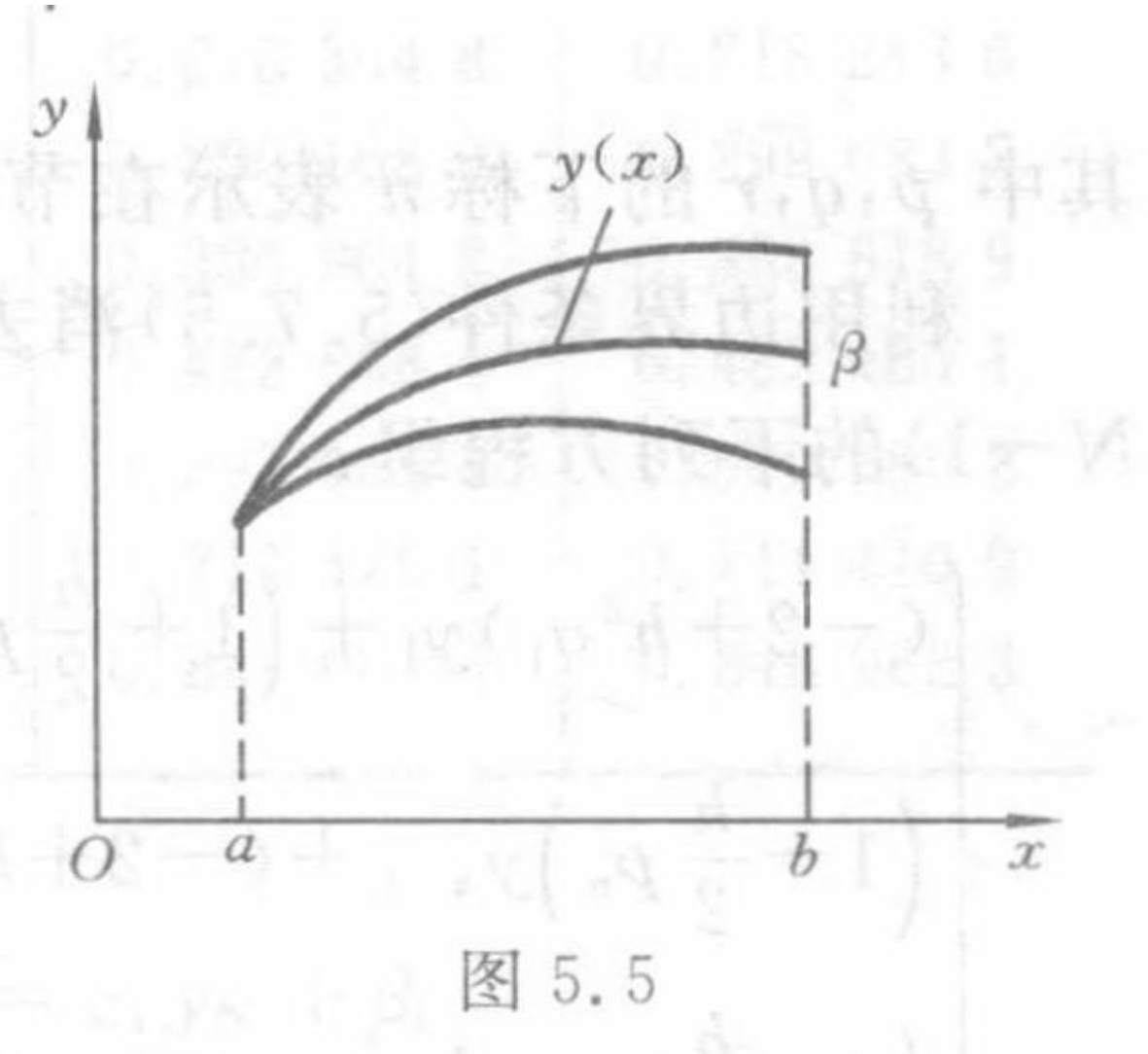


图 5.5

图 5.5（原图）符号与图意：坐标轴 x, y ，原点 O ，横轴端点 a, b ，目标右端值 β ，积分曲线标记 $y(x)$ 。三条曲线从 $x = a$ 的同一左端点出发，在 $x = b$ 处到达不同高度； $y(x)$ 指向达到目标 β 的中间曲线。端点位置以竖虚线标出；图中未给出具体函数或数值。

然后再按斜率值 m_3 试射，求解相应的初值问题 (5.7.1)、(5.7.2)，又得新的结果 $y(b) = \beta_3$ 。继续这一过程，直到计算结果 $y(b)$ 与 β 相当符合为止。

上述试射法过分依赖于经验，局限性大。下面介绍求解边值问题的差分方法。

5.7.2 差分方程的建立

应用差分方法的关键，在于恰当地选取差商逼近微分方程中的导数。我们知道，逼近一阶导数 $y'(x)$ 可用向前差商 $\frac{y(x+h)-y(x)}{h}$ ，亦可用向后差商 $\frac{y(x)-y(x-h)}{h}$ 或中心差商 $\frac{y(x+h)-y(x-h)}{2h}$ 。中心差商是向前差商与向后差商的算术平均。为逼近二阶导数 $y''(x)$ ，一般用二阶差商——向前差商的向后差商（即向后差商的向前差商）

原书 PDF 页 149; 印刷页码 136。

[原页](#)

$$y''(x) \approx \frac{\frac{y(x+h)-y(x)}{h} - \frac{y(x)-y(x-h)}{h}}{h} = \frac{y(x+h) - 2y(x) + y(x-h)}{h^2}.$$

设将积分区间 $[a, b]$ 划分为 N 等份, 步长 $h = \frac{b-a}{N}$, 节点 $x_n = x_0 + nh, n = 0, 1, \dots, N$ 。用差商替代相应的导数, 可将边值问题 (5.7.1)、(5.7.3) 离散化得下列计算公式:

$$\frac{y_{n+1} - 2y_n + y_{n-1}}{h^2} = f\left(x_n, y_n, \frac{y_{n+1} - y_{n-1}}{2h}\right) \quad (n = 1, 2, \dots, N-1), \quad (5.7.4)$$

$$y_0 = \alpha, \quad y_N = \beta. \quad (5.7.5)$$

如果函数 f 是非线性的, 那么所归结出的差分方程也是非线性的, 这时实际求解比较困难。

如果所给方程 (5.7.1) 是如下形式的线性方程:

$$y'' + p(x)y' + q(x)y = r(x), \quad (5.7.6)$$

则差分方程 (5.7.4) 相应的形式为

$$\frac{y_{n+1} - 2y_n + y_{n-1}}{h^2} + p_n \frac{y_{n+1} - y_{n-1}}{2h} + q_n y_n = r_n \quad (n = 1, \dots, N-1), \quad (5.7.7)$$

其中 p, q, r 的下标 n 表示在节点 x_n 取值。

利用边界条件 (5.7.5) 消去式 (5.7.7) 中的 y_0 和 y_N , 整理得到关于 y_n ($1 \leq n \leq N-1$) 的下列方程组:

$$\begin{cases} (-2 + h^2 q_1) y_1 + \left(1 + \frac{h}{2} p_1\right) y_2 = h^2 r_1 - \left(1 - \frac{h}{2} p_1\right) \alpha, \\ \left(1 - \frac{h}{2} p_n\right) y_{n-1} + (-2 + h^2 q_n) y_n + \left(1 + \frac{h}{2} p_n\right) y_{n+1} = h^2 r_n \quad (2 \leq n \leq N-2), \\ \left(1 - \frac{h}{2} p_{N-1}\right) y_{N-2} + (-2 + h^2 q_{N-1}) y_{N-1} = h^2 r_{N-1} - \left(1 + \frac{h}{2} p_{N-1}\right) \beta. \end{cases} \quad (5.7.8)$$

这样归结出的方程组是所谓**三对角型**的, 即

$$\begin{pmatrix} -2 + h^2 q_1 & 1 + \frac{h}{2} p_1 & & & \\ 1 - \frac{h}{2} p_2 & -2 + h^2 q_2 & 1 + \frac{h}{2} p_2 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 - \frac{h}{2} p_{N-2} & -2 + h^2 q_{N-2} & 1 + \frac{h}{2} p_{N-2} \\ & & & 1 - \frac{h}{2} p_{N-1} & -2 + h^2 q_{N-1} \end{pmatrix},$$

原书 PDF 页 150；印刷页码 137。

原页

因为它的系数矩阵仅在主对角线及其相邻的两条对角线上有非零元素。求解这种三对角型方程组，用所谓**追赶法**特别有效。在 7.4.3 节将介绍追赶法。

例 5.7 用差分方法解边值问题
$$\begin{cases} y'' - y = x, & 0 < x < 1, \\ y(0) = 0, & y(1) = 1. \end{cases}$$

解取步长 $h = 0.1$ ，节点 $x_n = \frac{n}{10}$ ($n = 0, 1, 2, \cdots, 10$)，差分方程 (5.7.8) 的形式是

$$\begin{pmatrix} -2-10^{-2} & 1 & & & & & & & & \\ & 1 & -2-10^{-2} & 1 & & & & & & \\ & & \ddots & \ddots & \ddots & & & & & \\ & & & 1 & -2-10^{-2} & 1 & & & & \\ & & & & 1 & -2-10^{-2} & & & & \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_8 \\ y_9 \end{pmatrix} = \begin{pmatrix} 0.1 \times 10^{-2} \\ 0.2 \times 10^{-2} \\ \vdots \\ 0.8 \times 10^{-2} \\ -1 + 0.9 \times 10^{-2} \end{pmatrix}.$$

表 5.11 列出了差分方法的计算结果，表中最末一列是按解的表达式

$$y = \frac{2(e^x - e^{-x})}{e - e^{-1}} - x$$

算出的准确值。

表 5.11

x_n	y_n	$y(x_n)$
0.1	0.0704894	0.0704673
0.2	0.1426836	0.1426409
0.3	0.2183048	0.2182436
0.4	0.2991089	0.2990332
0.5	0.3869042	0.3868189
0.6	0.4835684	0.4834801
0.7	0.5910684	0.5909852
0.8	0.7114791	0.7114109
0.9	0.8470045	0.8469633

附带指出，在应用上，有时边界条件按以下方式给出：

当 $x = a$ 时， $y' = \alpha_0 y + \beta_0$ ；当 $x = b$ 时， $y' = \alpha_1 y + \beta_1$ 。

这里 $\alpha_0, \beta_0, \alpha_1, \beta_1$ 均为已知常数，这时，边界条件中所包含的微商也要替换成相应的差商

$$\frac{y_1 - y_0}{h} = \alpha_0 y_0 + \beta_0, \quad \frac{y_N - y_{N-1}}{h} = \alpha_1 y_N + \beta_1.$$

它们和差分方程 (5.7.7) 一起，仍然构成包含 $N + 1$ 个未知数的线性方程组。

5.7.3 差分问题的可解性

我们知道, 通过自变量的适当变换可以消除线性方程 (5.7.6) 中的一阶导数项。下面仅就缺一阶导数项的方程来讨论这一问题。考察边值问题

$$\begin{cases} y'' - q(x)y = r(x), \\ y(a) = \alpha, \quad y(b) = \beta. \end{cases} \quad (5.7.9)$$

这里假定 $q(x) \geq 0$, 其对应的差分问题是

$$\begin{cases} \frac{y_{n+1} - 2y_n + y_{n-1}}{h^2} - q_n y_n = r_n & (n = 1, 2, \dots, N-1), \\ y_0 = \alpha, \quad y_N = \beta. \end{cases} \quad (5.7.10)$$

原书 PDF 页 151; 印刷页码 138。

[原页](#)

现在论证差分问题的可解性。由于式 (5.7.10) 是关于变量 y_n ($n = 0, 1, 2, \dots, N$) 的线性方程组, 要证明它的解存在唯一, 只要证明对应的齐次方程组只有零解。为此, 我们引进下述极值原理。

定理 5.2 对于一组不全相等的数 y_n ($n = 0, 1, \dots, N$), 记

$$l(y_n) = \frac{y_{n+1} - 2y_n + y_{n-1}}{h^2} - q_n y_n, \quad q_n \geq 0 \quad (n = 1, 2, \dots, N-1).$$

假定 $l(y_n) \geq 0$ ($n = 1, 2, \dots, N-1$), 则 y_n 的正的最大值只能是 y_0 或 y_N ; 如果 $l(y_n) < 0$ ($n = 1, 2, \dots, N-1$), 则 y_n 的负的最小值只能是 y_0 或 y_N 。

证明 用反证法。考察 $l(y_n) \geq 0$ 的情形, 设 y_m ($0 < m < N$) 是正的最大值, 即 $y_m = \max_{0 \leq n \leq N} y_n = M > 0$, 且 y_{m-1} 和 y_{m+1} 中至少有一个小于 M , 此时有

$$l(y_m) = \frac{y_{m+1} - 2y_m + y_{m-1}}{h^2} - q_m y_m < \frac{M - 2M + M}{h^2} - q_m M = -q_m M,$$

由于 $q_m \geq 0, M > 0$, 故由上式推出 $l(y_m) < 0$, 此与题设矛盾。

此外, $l(y_n) < 0$ 的情形可类似地进行讨论。证毕。

作为这一定理的推论有如下定理。

定理 5.3 差分问题式 (5.7.10) 的解存在并且是唯一的。

证明 只要证明对应的齐次方程组

$$\begin{cases} l(y_n) = \frac{y_{n+1} - 2y_n + y_{n-1}}{h^2} - q_n y_n = 0 & (n = 1, 2, \dots, N-1), \\ y_0 = y_N = 0 \end{cases}$$

只有零解。由于这里 $l(y_n) = 0$ ($n = 1, 2, \dots, N-1$), 由极值原理知, y_n 的正的最大值和负的最小值只能是 y_0 或 y_N , 而按边界条件 $y_0 = y_N = 0$, 故所有的 y_n ($n = 0, 1, \dots, N$) 全为零。证毕。

5.7.4 差分方法的收敛性

现在运用极值原理论证差分方法的收敛性并估计误差。

定理 5.4 设 y_n 是差分问题式 (5.7.10) 的解, 而 $y(x_n)$ 是边值问题 (5.7.9) 的解 $y(x)$ 在节点 x_n 的值, 则截断误差 $e_n = y(x_n) - y_n$ 有下列估计式:

$$|e_n| \leq \frac{M(b-a)^2}{96} h^2, \quad M = \max_{a \leq x \leq b} |y^{(4)}(x)|. \quad (5.7.11)$$

证明 由 Taylor 展开, 易得

$$\begin{cases} \frac{y(x_{n+1}) - 2y(x_n) + y(x_{n-1}))}{h^2} - q_n y(x_n) = r_n + \frac{h^2}{12} y^{(4)}(\xi_n), & x_{n-1} < \xi_n < x_{n+1}, \\ y(x_0) = \alpha, \quad y(x_N) = \beta. \end{cases} \quad (5.7.12)$$

校注 (agent 补充): 定理 5.2 的负最小值一侧, 原书确实写作严格条件 $l(y_n) < 0$; 定理 5.3 却在 $l(y_n) = 0$ 时同时引用正最大值、负最小值结论。此处原文均照录。补足该引用的方法是对 $-y_n$ 应用定理 5.2 的 $l \geq 0$ 一侧 (利用 l 的线性), 从而排除负最小值; 无需改动原书方程。

原书 PDF 页 152; 印刷页码 139。

[原页](#)

将式 (5.7.12) 与式 (5.7.10) 相减, 知截断误差 $e_n = y(x_n) - y_n$ 满足

$$\begin{cases} l(e_n) = \frac{e_{n+1} - 2e_n + e_{n-1}}{h^2} - q_n e_n = \frac{h^2}{12} y^{(4)}(\xi_n), \\ e_0 = e_N = 0, \end{cases}$$

其中 ξ_n 一般是不知道的。于是转向下列差分问题:

$$\begin{cases} l(\varepsilon_n) = \frac{\varepsilon_{n+1} - 2\varepsilon_n + \varepsilon_{n-1}}{h^2} - q_n \varepsilon_n = -\frac{h^2}{12} M, \\ \varepsilon_0 = \varepsilon_N = 0, \end{cases} \quad (5.7.13)$$

其中 $M = \max_{a \leq x \leq b} |y^{(4)}(x)|$ 。首先证明上述两个差分问题的解存在下列关系:

$$|e_n| \leq \varepsilon_n \quad (n = 0, 1, \dots, N). \quad (5.7.14)$$

事实上, 由于

$$l(\varepsilon_n) = -\frac{h^2}{12} M \leq -\frac{h^2}{12} |y^{(4)}(\xi_n)| = -|l(e_n)|,$$

故有

$$l(\varepsilon_n - e_n) \leq 0, \quad l(\varepsilon_n + e_n) \leq 0.$$

又

$$\varepsilon_0 - e_0 = \varepsilon_N - e_N = 0, \quad \varepsilon_0 + e_0 = \varepsilon_N + e_N = 0,$$

利用定理 5.2, 知 $\varepsilon_n - e_n \geq 0, \varepsilon_n + e_n \geq 0$, 即式 (5.7.14) 成立。

差分问题 (5.7.13) 的解仍不易直接求得, 于是进一步考察

$$\begin{cases} \tilde{l}(\rho_n) = \frac{\rho_{n+1} - 2\rho_n + \rho_{n-1}}{h^2} = -\frac{h^2}{12} M, \\ \rho_0 = \rho_N = 0. \end{cases} \quad (5.7.15)$$

这里 $\tilde{l}(\rho_n - \varepsilon_n) = -q_n \varepsilon_n \leq 0$, 又 $\rho_0 - \varepsilon_0 = \rho_N - \varepsilon_N = 0$, 故由定理 5.2 (注意, 当 $q_n = 0$ 时, $\tilde{l}(\tilde{y}_n)$ 就是 $l(y_n)$) 知 $\rho_n - \varepsilon_n \geq 0$, 即 $\varepsilon_n \leq \rho_n$, 于是有 $|e_n| \leq \varepsilon_n \leq \rho_n$ 。

然而 ρ_n 是容易求出的。求解差分问题 (5.7.15) 所对应的边值问题

$$\begin{cases} \rho'' = -\frac{h^2}{12}M, \\ \rho(a) = \rho(b) = 0, \end{cases}$$

解得

$$\rho(x) = \frac{h^2}{24}M(x-a)(b-x).$$

容易验证 $\rho_n = \rho(x_n)$ 是差分问题 (5.7.15) 的解, 注意到 $\rho(x)$ 在点 $x = \frac{a+b}{2}$ 达到最大值

$$\rho\left(\frac{a+b}{2}\right) = \frac{M(b-a)^2}{96}h^2,$$

因此有估计式 (5.7.11)。证毕。

据估计式 (5.7.11) 知, 当 $h \rightarrow 0$ 时有 $e_n \rightarrow 0$, 这表明差分方法 (5.7.10) 是收敛的。

校注 (agent 补充, ch05b-E006): 式 (5.7.15) 后括注的原图在 y 上方印波浪号, 即 $l(\tilde{y}_n)$, 这里照录。按该式定义与“当 $q_n = 0$ 时”的上下文, 此处疑将波浪号排错位置, 预期为 $\tilde{l}(y_n)$, 即去掉零阶项后的算子; 原文并未定义 $\tilde{y}_n$ 。

原书 PDF 页 153; 印刷页码 140。

[原页](#)

小结

本章研究求解常微分方程的差分方法。构造差分格式主要有两条途径: 基于数值积分的构造方法和基于 Taylor 展开的构造方法。后一种方法更灵活, 也更具有—般性。Taylor 展开方法还有一个优点, 它在构造差分公式的同时可以得到关于截断误差的估计。

但是, 直接用 Taylor 展开导出的 Taylor 级数法 (见 7.3.1 节) 不便于实际应用。基于 Taylor 展开构造出的四阶 Runge-Kutta 方法 (见 7.3.4 节) 则是电子计算机上的常用算法。四阶 Runge-Kutta 方法的优点是精度高, 程序简单, 计算过程稳定, 并且易于调节步长。

四阶 Runge-Kutta 方法也有不足之处, 它要求函数 f 具有较高的光滑性。如果 f 的光滑性差, 那么它的精度可能还不如 Euler 格式 (见 7.2.1 节) 或改进的 Euler 格式 (见 7.2.4 节)。

四阶 Runge-Kutta 方法的另一个缺点是计算量比较大, 需要耗费较多的机器时间 (每一步需四次计算函数 f 的值)。相比之下, Hamming 方法 (见 7.5.7 节) 可以节省计算量 (每一步只需两次计算函数 f 的值)。但 Hamming 方法是一种四步法, 它不是自开始的, 需要借助于四阶 Runge-Kutta 方法 (或四阶 Taylor 级数法) 提供开始值。

就差分方法来说, 常微分方程与偏微分方程有着紧密的联系, 读者可参看偏微分方程的数值解法的有关教材。

习题

1. 就初值问题 $y' = ax + b, y(0) = 0$ 分别导出 Euler 方法和改进的 Euler 方法的近似解的表达式, 并与准确解 $y = \frac{1}{2}ax^2 + bx$ 相比较。
2. 用改进的 Euler 方法解初值问题 $\begin{cases} y' = x + y, & 0 < x < 1, \\ y(0) = 1, \end{cases}$ 取步长 $h = 0.1$ 计算, 并与准确解 $y = -x - 1 + 2e^x$ 相比较。
3. 用改进的 Euler 方法解 $\begin{cases} y' = x^2 + x - y, \\ y(0) = 0, \end{cases}$ 取步长 $h = 0.1$ 计算 $y(0.5)$, 并与准确解 $y = -e^{-x} + x^2 - x + 1$ 相比较。
4. 用梯形方法解初值问题 $\begin{cases} y' + y = 0, \\ y(0) = 1, \end{cases}$ 证明其近似解为 $y_n = \left(\frac{2-h}{2+h}\right)^n$, 并证明当 $h \rightarrow 0$ 时, 它收敛于原初值问题的准确解 $y = e^{-x}$ 。
5. 利用 Euler 方法计算积分 $\int_0^x e^{t^2} dt$ 在点 $x = 0.5, 1, 1.5, 2$ 的近似值。

校注 (agent 补充, ch05b-E007): 小结中的五处节号按原印刷保留为 7.3.1、7.3.4、7.2.1、7.2.4、7.5.7; 这些节号与本版章节编排不符。对应主题在本书章节索引中分别为 5.3.1 (Taylor 级数法)、5.3.5 (四阶 Runge-Kutta 方法)、5.2.1 (Euler 格式)、5.2.4 (改进的 Euler 格式)、5.5.7 (Hamming 格式)。其中本章 PDF 140 的正文也明确将四阶 Runge-Kutta 格式指向 5.3.5, PDF 144 的实际标题为 5.5.7 Hamming 格式。

原书 PDF 页 154; 印刷页码 141。

[原页](#)

6. 取 $h = 0.2$, 用经典的四阶 Runge-Kutta 方法求解下列初值问题:

$$(1) \begin{cases} y' = x + y, & 0 < x < 1, \\ y(0) = 1; \end{cases}$$

$$(2) \begin{cases} y' = 3y/(1+x), & 0 < x < 1, \\ y(0) = 1. \end{cases}$$

7. 证明对任意参数 t , 下列 Runge-Kutta 格式是二阶的:

$$\begin{cases} y_{n+1} = y_n + \frac{h}{2}(K_2 + K_3), \\ K_1 = f(x_n, y_n), \\ K_2 = f(x_n + th, y_n + thK_1), \\ K_3 = f(x_n + (1-t)h, y_n + (1-t)hK_1). \end{cases}$$

8. 证明下列两种 Runge-Kutta 方法是三阶的:

(1)

$$\begin{cases} y_{n+1} = y_n + \frac{h}{4}(K_1 + 3K_3), \\ K_1 = f(x_n, y_n), \\ K_2 = f\left(x_n + \frac{h}{3}, y_n + \frac{h}{3}K_1\right), \\ K_3 = f\left(x_n + \frac{2}{3}h, y_n + \frac{2}{3}hK_2\right); \end{cases}$$

(2)

$$\begin{cases} y_{n+1} = y_n + \frac{h}{9}(2K_1 + 3K_2 + 4K_3), \\ K_1 = f(x_n, y_n), \\ K_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_1\right), \\ K_3 = f\left(x_n + \frac{3}{4}h, y_n + \frac{3}{4}hK_2\right). \end{cases}$$

9. 分别用二阶显式 Adams 方法和二阶隐式 Adams 方法解下列初值问题:

$$y' = 1 - y, \quad y(0) = 0,$$

取 $h = 0.2, y_0 = 0, y_1 = 0.181$, 计算 $y(1.0)$ 并与准确解 $y = 1 - e^{-x}$ 相比较。

10. 证明下列解 $y' = f(x, y)$ 的差分公式

$$y_{n+1} = \frac{1}{2}(y_n + y_{n-1}) + \frac{h}{4}(4y'_{n+1} - y'_n + 3y'_{n-1})$$

是二阶的，并求出截断误差的首项。

11. 导出具有下列形式的三阶方法：

$$y_{n+1} = a_0 y_n + a_1 y_{n-1} + a_2 y_{n-2} + h(b_0 y'_n + b_1 y'_{n-1} + b_2 y'_{n-2}).$$

12. 将下列方程化为一阶方程组：

$$(1) \quad y'' - 3y' + 2y = 0, \quad y(0) = 1, \quad y'(0) = 1;$$

$$(2) \quad y'' - 0.1(1 - y^2)y' + y = 0, \quad y(0) = 1, \quad y'(0) = 0;$$

$$(3) \quad x''(t) = -\frac{x}{r^3}, \quad y''(t) = -\frac{y}{r^3}, \quad r = \sqrt{x^2 + y^2}, \quad x(0) = 0.4, \quad x'(0) = 0, \quad y(0) = 0, \quad y'(0) = 2.$$

$$13. \text{ 取 } h = 0.25, \text{ 用差分法解边值问题 } \begin{cases} y'' + y = 0, \\ y(0) = 0, \quad y(1) = 1.68. \end{cases}$$

$$14. \text{ 对方程 } y'' = f(x, y) \text{ 可建立差分公式 } y_{n+1} = 2y_n - y_{n-1} + h^2 f(x_n, y_n), \text{ 试用这一公式求解初值问题 } \begin{cases} y'' = 1, \\ y(0) = y(1) = 0, \end{cases}$$

验证计算解恒等于准确解 $y(x) = \frac{x^2 - x}{2}$ 。

$$15. \text{ 取 } h = 0.2, \text{ 用差分方法解边值问题 } \begin{cases} (1 + x^2)y'' - xy' - 3y = 6x - 3, \\ y(0) - y'(0) = 1, \quad y(1) = 2. \end{cases}$$

校注 (agent 补充, ch05b-E008): 习题 14 原书写“初值问题”, 已照录; 但给定条件为两端的 $y(0) = y(1) = 0$, 按本章定义属于边值问题, 疑为称谓误植。此处只标记原书疑误, 未代做习题。

原书 PDF 页 155；印刷页码 142。

[原页](#)

第 6 章方程求根

6.1 根的搜索

数学、物理中的许多问题常常归结为求解函数方程 $f(x) = 0$ ，这里， $f(x)$ 可以是代数多项式，也可以是超越函数。方程 $f(x) = 0$ 的解 x^* 称为它的根，或称为 $f(x)$ 的零点。

设函数 $f(x)$ 在 $[a, b]$ 上连续，且 $f(a)f(b) < 0$ ，根据连续函数的性质可知方程 $f(x) = 0$ 在区间 (a, b) 内一定有实根，这时称 $[a, b]$ 为方程 $f(x) = 0$ 的有根区间。

6.1.1 逐步搜索法

为明确起见，不妨假定 $f(a) < 0, f(b) > 0$ 。从有根区间 $[a, b]$ 的左端点 $x_0 = a$ 出发，按某个预定的步长 h （譬如取 $h = \frac{b-a}{N}$ ， N 为正整数）一步一步地向右跨，每跨一步进行一次根的“搜索”，即检查节点 $x_k = a + kh$ 上的函数值 $f(x_k)$ 的符号，一旦发现节点 x_k 与端点 a 的函数值异号，即 $f(x_k) > 0$ ①，则可以确定一个缩小了的有根区间 $[x_{k-1}, x_k]$ ，其宽度等于预定的步长 h 。

例 6.1 考察方程 $f(x) = x^3 - x - 1 = 0$ 。注意到 $f(0) < 0, f(2) > 0$ ，知 $f(x)$ 在区间 $(0, 2)$ 内至少有一个实根。

解 设从 $x = 0$ 出发，取 $h = 0.5$ 为步长向右进行根的搜索，列表记录各个节点上函数值的符号（见表 6.1），我们发现区间 $(1.0, 1.5)$ 内必有一根。

表 6.1

x	0	0.5	1.0	1.5
$f(x)$ 的符号	—	—	—	+

在具体运用上述方法时，步长 h 的选择是个关键。很明显，只要步长 h 取得足够小，利用这种方法可以得到具有任意精度的近似根。不过当 h 缩小时，所要搜索的步数相应增多，从而使计算量增大。因此，如果精度要求比较高，单用这种逐步搜索方法是不合算的。下述二分法可以看作逐步搜索方法的一种改进。

6.1.2 二分法

再考察有根区间 $[a, b]$ ，取中点 $x_0 = (a + b)/2$ 将它分为两半，然后进行根的搜

①特别地，可能有 $f(x_k) = 0$ ，这时 x_k 即为所求的根。

原书 PDF 页 156; 印刷页码 143。

原页

索, 即检查 $f(x_0)$ 与 $f(a)$ 是否同号, 如果确系同号, 说明所求的根 x^* 在 x_0 的右侧, 这时令 $a_1 = x_0, b_1 = b$; 否则 x^* 必在 x_0 的左侧, 这时令 $a_1 = a, b_1 = x_0$ (见图 6.1)。不管出现哪一种情况, 新的有根区间 $[a_1, b_1]$ 的长度仅为 $[a, b]$ 的一半。

对压缩了的有根区间 $[a_1, b_1]$ 又可施行同样的手续, 即用中点 $x_1 = (a_1 + b_1)/2$ 将区间 $[a_1, b_1]$ 再分为两半, 然后通过根的搜索判定所求的根在 x_1 的哪一侧, 从而又确定一个新的有根区间 $[a_2, b_2]$, 其长度是 $[a_1, b_1]$ 的一半。

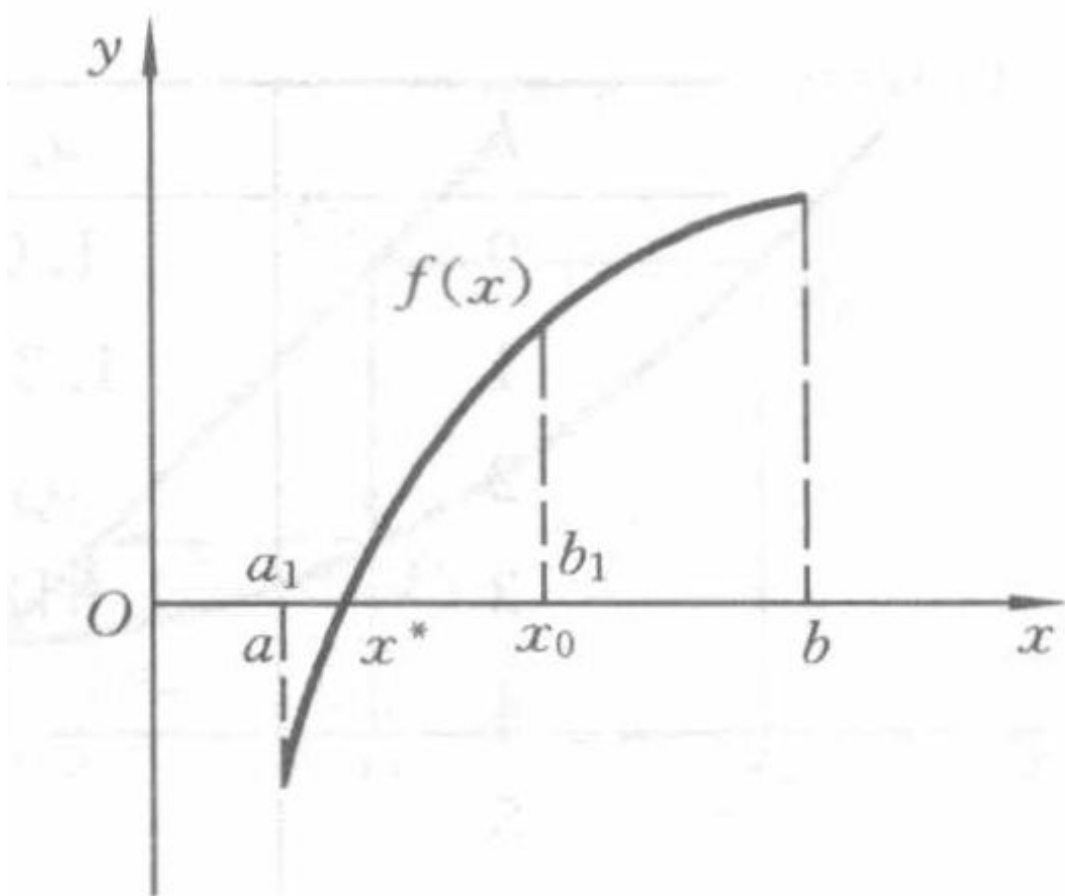


图 6.1

图 6.1

图意 (agent 整理): 原图显示二分法的有根区间压缩。曲线标为 $f(x)$, 坐标轴为 x, y , 原点 O ; 横轴标有 a, x^*, x_0, b , 新区间端点标有 a_1, b_1 , 其中 $a_1 = a, b_1 = x_0$ 。

如此反复二分下去, 即可得出一系列有根区间

$$[a, b] \supseteq [a_1, b_1] \supseteq [a_2, b_2] \supseteq \cdots \supseteq [a_k, b_k] \supseteq \cdots,$$

其中每个区间都是前一个区间的一半, 因此 $[a_k, b_k]$ 的长度 $b_k - a_k = (b - a)/2^k$ 当 $k \rightarrow \infty$ 时趋于零。就是说, 如果二分过程无限地继续下去, 这些区间最终必将收缩于一点 x^* , 该点显然就是所求的根。

每次二分后, 设取有根区间 $[a_k, b_k]$ 的中点 $x_k = (a_k + b_k)/2$ 作为根的近似值, 则在二分过程中可以获得一个近似根的序列 $x_0, x_1, x_2, \dots, x_k, \dots$, 该序列必以根 x^* 为极限。

不过在实际计算时, 不可能完成这个无限过程, 其实也没有这种必要, 因为数值分析的结果允许带有一定的误差。由于

$$|x^* - x_k| \leq (b_k - a_k)/2 = (b - a)/2^{k+1},$$

只要二分足够多次 (即 k 充分大), 便有 $|x^* - x_k| < \varepsilon$, 这里 ε 为预定的精度。

例 6.2 求方程 $f(x) = x^3 - x - 1 = 0$ 在区间 $(1.0, 1.5)$ 内的一个实根, 要求准确到小数点后的第二位。

解 这里 $a = 1.0, b = 1.5$, 而 $f(a) < 0, f(b) > 0$ 。取 (a, b) 的中点 $x_0 = 1.25$ 将区间二等分, 由于 $f(x_0) < 0$, 即 $f(x_0)$ 与 $f(a)$ 同号, 故所求的根 x^* 必在 x_0 右侧, 这时应令 $a_1 = x_0 = 1.25, b_1 = b = 1.5$, 而得到新的有根区间 $[a_1, b_1]$ 。

如此反复二分下去, 二分过程无需赘述。现在预估所要二分的次数, 按误差估计式 (6.1.1), 只要二分六次 ($k = 6$), 便能达到预定的精度

$$|x^* - x_6| \leq 0.005.$$

二分法的计算结果如表 6.2 所示。

二分法是电子计算机上一种常用算法, 下面列出计算步骤:

步 1 准备。计算 $f(x)$ 在有根区间 $[a, b]$ 端点处的值 $f(a), f(b)$ 。

步 2 二分。计算 $f(x)$ 在区间中点 $\frac{a+b}{2}$ 处的值 $f\left(\frac{a+b}{2}\right)$ 。

校注 (agent 补充): 原页例 6.2 引用了“误差估计式 (6.1.1)”, 但本页对应的误差估计式旁未见印刷编号。此处保留原文引用及无编号公式, 不补造公式标签; 见 [误差估计式原图裁切](#)。

原书 PDF 页 157；印刷页码 144。

[原页](#)

表 6.2

k	a_k	b_k	x_k	$f(x_k)$ 符号
0	1.0	1.5	1.25	—
1	1.25		1.375	+
2		1.375	1.3125	—
3	1.3125		1.3438	+
4		1.3438	1.3281	+
5		1.3281	1.3203	—
6	1.3203		1.3242	—

步 3 判断。若 $f\left(\frac{a+b}{2}\right) = 0$ 则 $\frac{a+b}{2}$ 即是根，计算过程结束。否则作如下检验：

若 $f\left(\frac{a+b}{2}\right)$ 与 $f(a)$ 异号，则根位于区间 $\left(a, \frac{a+b}{2}\right)$ 内，这时以 $\frac{a+b}{2}$ 代替 b ；

若 $f\left(\frac{a+b}{2}\right)$ 与 $f(a)$ 同号，则根位于区间 $\left(\frac{a+b}{2}, b\right)$ 内，这时以 $\frac{a+b}{2}$ 代替 a 。

反复执行步 2 和步 3，直到区间 $[a, b]$ 长度缩小到允许误差范围之内，此时区间中点 $\frac{a+b}{2}$ 即可作为所求的根。

二分法的优点是算法简单，而且收敛性总能得到保证。

6.2 迭代法

6.2.1 迭代过程的收敛性

考察下列形式的方程：

$$x = \varphi(x). \tag{6.2.1}$$

这种方程是隐式的，因而不能直接得出它的根。但如果给出根的某个猜测值 x_0 ，将它代入式 (6.2.1) 的右端，即可求得 $x_1 = \varphi(x_0)$ 。然后，又可取 x_1 作为猜测值，进一步得到 $x_2 = \varphi(x_1)$ 。如此反复迭代。如果按公式

$$x_{k+1} = \varphi(x_k), \quad k = 0, 1, 2, \dots \tag{6.2.2}$$

确定的数列 $\{x_k\}$ 有极限 $x^* = \lim_{k \rightarrow \infty} x_k$ ，则称迭代过程式 (6.2.2) 收敛，这时极限值 x^* 显然就是方程 (6.2.1) 的根。

上述迭代法是一种逐次逼近法，其基本思想是将隐式方程 (6.2.1) 归结为一组显式的计算公式 (6.2.2)，就是说，迭代过程实质上是一个逐步显式化的过程。

我们用几何图像来显示这一迭代过程。方程 $x = \varphi(x)$ 的求根问题在 Oxy 平面上

原书 PDF 页 158; 印刷页码 145。

[原页](#)

就是要确定曲线 $y = \varphi(x)$ 与直线 $y = x$ 的交点 P^* (见图 6.2)。对于 x^* 的某个近似值 x_0 , 在曲线 $y = \varphi(x)$ 上可确定一点 P_0 , 它以 x_0 为横坐标, 而纵坐标则等于 $\varphi(x_0) = x_1$ 。过 P_0 引平行 x 轴的直线, 设交直线 $y = x$ 于点 Q_1 , 然后过 Q_1 再作平行于 y 轴的直线, 它与曲线 $y = \varphi(x)$ 的交点记作 P_1 , 则点 P_1 的横坐标为 x_1 , 纵坐标等于 $\varphi(x_1) = x_2$ 。按图 6.2 中箭头所示的路径继续进行下去, 在曲线 $y = \varphi(x)$ 上得到点列 $P_1, P_2, \dots$, 其横坐标分别为依公式 $x_{k+1} = \varphi(x_k)$ 求得的迭代值 $x_1, x_2, \dots$ 。如果点列 $\{P_k\}$ 趋于点 P^* , 则相应的迭代值 x_k 收敛到所求的根 x^* 。

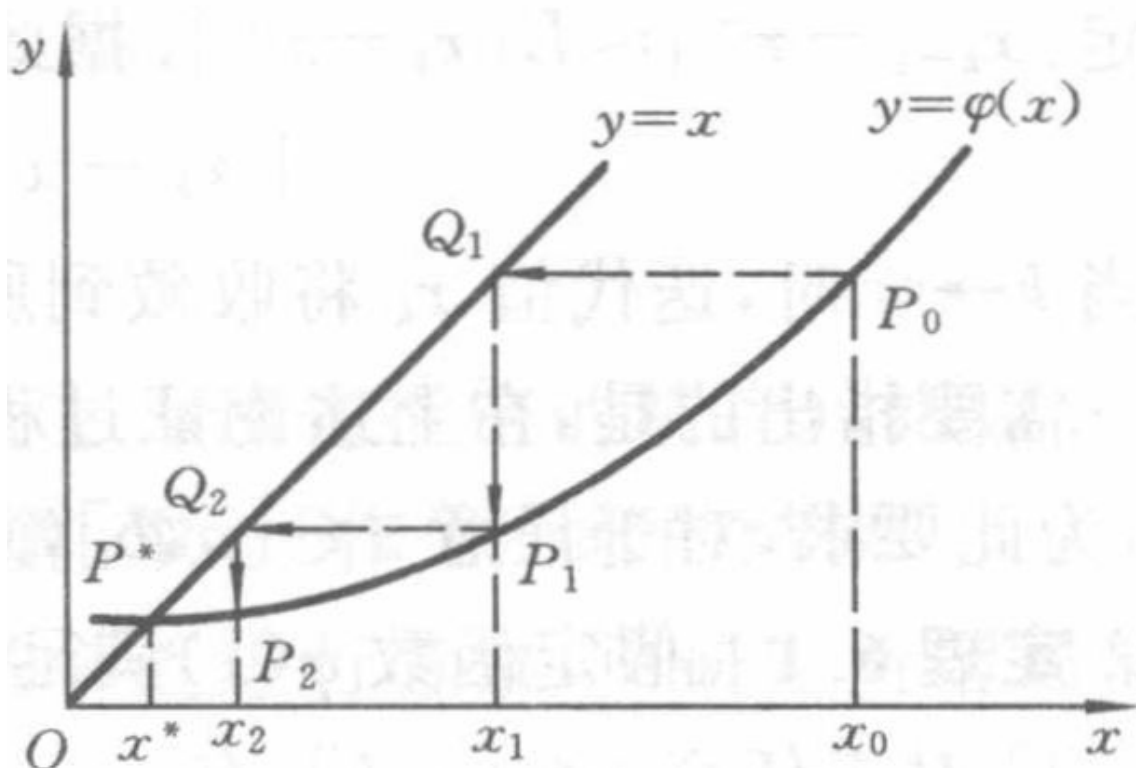


图 6.2

图 6.2

图意 (agent 整理): 原图用曲线 $y = \varphi(x)$ 、直线 $y = x$ 和带箭头的水平、竖直折线显示迭代过程。全部文字标签为 $x, y, O, x^*, x_2, x_1, x_0, P^*, P_2, P_1, P_0, Q_2, Q_1, y = x, y = \varphi(x)$ 。

例 6.3 求方程

$$f(x) = x^3 - x - 1 = 0 \quad (6.2.3)$$

在 $x_0 = 1.5$ 附近的根 x^* 。

解 设将方程 (6.2.3) 改写成 $x = \sqrt[3]{x+1}$ 的形式, 据此建立迭代公式

$$x_{k+1} = \sqrt[3]{x_k + 1}, \quad k = 0, 1, 2, \dots.$$

表 6.3 记录了各步迭代的结果。可以看到，如果仅取六位数字，那么结果 x_7 与 x_8 完全相同，这时可以认为 x_7 实际上已满足方程 (6.2.3)，即为所求的根。

表 6.3

k	x_k	k	x_k
0	1.5	5	1.324 76
1	1.357 21	6	1.324 73
2	1.330 86	7	1.324 72
3	1.325 88	8	1.324 72
4	1.324 94		

应当指出，迭代法的效果并不是总能令人满意的。譬如，设依方程 (6.2.3) 的另一种等价形式 $x = x^3 - 1$ ，建立迭代公式 $x_{k+1} = x_k^3 - 1$ ，迭代初值仍取 $x_0 = 1.5$ ，则有

$$x_1 = 2.375, \quad x_2 = 12.39.$$

继续迭代下去已经没有必要，因为结果显然会越来越大，不可能趋于某个极限。称这种不收敛的迭代过程是发散的。一个发散的迭代过程，纵使进行了上千百次迭代，其结果也是毫无价值的。

再考察一般情形。设方程 $x = \varphi(x)$ 在区间 (a, b) 内有根 x^* ，则保证迭代过程 $x_{k+1} = \varphi(x_k)$ 收敛的条件可表述为：存在定数 $0 < L < 1$ ，使对于任意 $x \in [a, b]$ 成立

$$|\varphi'(x)| \leq L. \quad (6.2.4)$$

事实上，由微分中值定理

$$x_{k+1} - x^* = \varphi(x_k) - \varphi(x^*) = \varphi'(\xi)(x_k - x^*),$$

其中 ξ 是 x^* 与 x_k 之间某一点，当 $x_k \in [a, b]$ 时 $\xi \in [a, b]$ 。因此利用条件 (6.2.4) 可以

原书 PDF 页 159; 印刷页码 146。

[原页](#)

断定 $|x_{k+1} - x^*| \leq L|x_k - x^*|$ 。据此反复递推, 有

$$|x_k - x^*| \leq L^k |x_0 - x^*|,$$

故当 $k \rightarrow \infty$ 时, 迭代值 x_k 将收敛到所求的根 x^* 。

需要指出的是, 在上述论证过程中, 应当保证一切迭代值 x_k 全落在区间 (a, b) 内, 为此要求, 对于任意 $x \in [a, b]$, 总有 $\varphi(x) \in [a, b]$ 。于是有下述论断。

定理 6.1 假定函数 $\varphi(x)$ 满足下列两项条件:

1° 对于任意 $x \in [a, b]$, 有

$$a \leq \varphi(x) \leq b, \quad (6.2.5)$$

2° 存在正数 $L < 1$, 使对于任意 $x \in [a, b]$ ①, 有

$$|\varphi'(x)| \leq L < 1, \quad (6.2.6)$$

则迭代过程 $x_{k+1} = \varphi(x_k)$ 对于任意初值 $x_0 \in [a, b]$ 均收敛于方程 $x = \varphi(x)$ 的根 x^* ②, 且有如下的误差估计式:

$$|x_k - x^*| \leq \frac{L^k}{1-L} |x_1 - x_0|. \quad (6.2.7)$$

证明 前已证明迭代过程的收敛性, 剩下的问题是导出估计式 (6.2.7)。按式 (6.2.6), 有

$$|x_{k+1} - x_k| = |\varphi(x_k) - \varphi(x_{k-1})| \leq L|x_k - x_{k-1}|. \quad (6.2.8)$$

据此反复递推得 $|x_{k+1} - x_k| \leq L^k |x_1 - x_0|$ 。于是对于任意正整数 p , 有

$$\begin{aligned} |x_{k+p} - x_k| &\leq |x_{k+p} - x_{k+p-1}| + |x_{k+p-1} - x_{k+p-2}| + \cdots + |x_{k+1} - x_k| \\ &\leq (L^{k+p-1} + L^{k+p-2} + \cdots + L^k) |x_1 - x_0| \leq \frac{L^k}{1-L} |x_1 - x_0|. \end{aligned}$$

在上式中令 $p \rightarrow \infty$, 注意到 $\lim_{p \rightarrow \infty} x_{k+p} = x^*$, 即得式 (6.2.7)。证毕。

迭代过程是个极限过程。在用迭代法进行实际计算时, 必须按精度要求控制迭代次数。误差估计式 (6.2.7) 原则上可用来确定迭代次数, 但它由于含有信息 L 而不便于实际应用。根据式 (6.2.8), 对于任意正整数 p , 有

$$|x_{k+p} - x_k| \leq (L^{p-1} + L^{p-2} + \cdots + 1) |x_{k+1} - x_k| \leq \frac{1}{1-L} |x_{k+1} - x_k|.$$

在上式中令 $p \rightarrow \infty$, 有

$$|x^* - x_k| \leq \frac{1}{1-L} |x_{k+1} - x_k|.$$

由此可见, 只要相邻两次计算结果的偏差 $|x_{k+1} - x_k|$ 足够小, 即可保证近似值 x_k 具有足够的精度, 因此可以通过检查 $|x_{k+1} - x_k|$ 来判断迭代过程应否终止。

①条件 2 可以放宽为, 存在正数 $L < 1$, 使对任意 $x, \bar{x} \in [a, b]$ 均满足 Lipschitz 条件

$$|\varphi(x) - \varphi(\bar{x})| \leq L|x - \bar{x}|.$$

②可以证明, 当式 (6.2.5)、(6.2.6) 成立时, 方程 $x = \varphi(x)$ 在 $[a, b]$ 内有且仅有一个根 x^* 。

原书 PDF 页 160；印刷页码 147。

[原页](#)

下面列出迭代法的计算步骤：

- 步 1 准备提供迭代初值 x_0 。
- 步 2 迭代计算迭代值 $x_1 = \varphi(x_0)$ 。
- 步 3 控制检查 $|x_1 - x_0|$ ：若 $|x_1 - x_0| > \varepsilon$ (ε 为预先指定的精度)，则以 x_1 替换 x_0 转步 2 继续迭代；当 $|x_1 - x_0| \leq \varepsilon$ 时终止计算，取 x_1 作为所求的结果。

在实际应用迭代法时，通常在所求的根 x^* 的邻近进行考察，而研究所谓局部收敛性。

定义 6.1 若存在 x^* 的某个邻域 $R: |x - x^*| \leq \delta$ ，使迭代过程 $x_{k+1} = \varphi(x_k)$ 对于任意初值 $x_0 \in R$ 均收敛，则称迭代过程 $x_{k+1} = \varphi(x_k)$ 在根 x^* 邻近具有局部收敛性。

定理 6.2 设 x^* 为方程 $x = \varphi(x)$ 的根， $\varphi'(x)$ 在 x^* 的邻近连续且 $|\varphi'(x^*)| < 1$ ，则迭代过程 $x_{k+1} = \varphi(x_k)$ 在 x^* 邻近具有局部收敛性。

证明由连续函数的性质，存在 x^* 的某个邻域 $R: |x - x^*| \leq \delta$ ，使对于任意 $x \in R$ 成立 $|\varphi'(x)| \leq L < 1$ 。此外，对于任意 $x \in R$ ，总有 $\varphi(x) \in R$ ，这是因为

$$|\varphi(x) - x^*| = |\varphi(x) - \varphi(x^*)| \leq L|x - x^*| \leq |x - x^*|,$$

于是，依据定理 6.1 可以断定，迭代过程 $x_{k+1} = \varphi(x_k)$ 对于任意初值 $x_0 \in R$ 均收敛。证毕。

例 6.4 求方程 $x = e^{-x}$ 在 $x = 0.5$ 附近的一个根，要求精度 $\varepsilon = 10^{-5}$ 。

解过 $x = 0.5$ 以 $h = 0.1$ 为步长搜索一次，即可发现，所求的根在区间 $(0.5, 0.6)$ 内。由于在根的邻近， $|(e^{-x})'| \approx 0.6$ ，这个值小于 1，因此迭代公式 $x_{k+1} = e^{-x_k}$ 对于初值 $x_0 = 0.5$ 是收敛的。

表 6.4 记录了迭代的结果。比较相邻的两次迭代值，迭代 18 次得所求的根 0.567 14。

表 6.4

k	x_k	k	x_k	k	x_k
0	0.5	7	0.568 438 0	14	0.567 118 8
1	0.606 530 6	8	0.566 409 4	15	0.567 157 1
2	0.545 239 2	9	0.567 559 6	16	0.567 135 4
3	0.579 703 1	10	0.566 907 2	17	0.567 147 7
4	0.560 064 6	11	0.567 277 2	18	0.567 140 7
5	0.571 172 1	12	0.567 067 3		
6	0.564 862 9	13	0.567 186 3		

6.2.2 迭代公式的加工

对于收敛的迭代过程，只要迭代足够多次，就可以使结果达到任意的精度，但有时迭代过程收敛缓慢，从而使计算量变得很大，因此迭代过程的加速是个重要的课题。

原书 PDF 页 161; 印刷页码 148。

[原页](#)

设 x_0 是根 x^* 的某个预测值, 用迭代公式校正一次得 $x_1 = \varphi(x_0)$, 而由微分中值定理有

$$x_1 - x^* = \varphi'(\xi)(x_0 - x^*),$$

其中 ξ 介于 x^* 与 x_0 之间。

假定 $\varphi'(x)$ 改变不大, 近似地取某个近似值 L , 则由

$$x_1 - x^* \approx L(x_0 - x^*),$$

得

$$x^* = \frac{1}{1-L}x_1 - \frac{L}{1-L}x_0. \quad (6.2.9)$$

可以期望, 按上式右端求得的

$$x_2 = \frac{1}{1-L}x_1 - \frac{L}{1-L}x_0 = x_1 + \frac{L}{1-L}(x_1 - x_0)$$

是比 x_1 更好的近似值。

将每得到一次改进值算作一步, 并用 $\bar{x}_k$ 和 x_k 分别表示第 k 步的校正值和改进值, 则加速迭代计算方案可表述如下:

$$\begin{aligned} \text{校正} \quad \bar{x}_{k+1} &= \varphi(x_k), \\ \text{改进} \quad x_{k+1} &= \bar{x}_{k+1} + \frac{L}{1-L}(\bar{x}_{k+1} - x_k). \end{aligned} \quad (6.2.10)$$

例 6.5 求解方程 $x = e^{-x}$ 。

解 由于在 $x_0 = 0.5$ 附近, $(e^{-x})' \approx -0.6$, 故上述计算公式的具体形式是

$$\begin{cases} \bar{x}_{k+1} = e^{-x_k}, \\ x_{k+1} = \bar{x}_{k+1} - \frac{0.6}{1.6}(\bar{x}_{k+1} - x_k). \end{cases}$$

表 6.5 列出了计算结果。

表 6.5

k	$\bar{x}_k$	x_k
0		0.5
1	0.606 53	0.566 58
2	0.567 46	0.567 13
3	0.567 15	0.567 14

例 6.4 迭代 18 次得到精度 10^{-5} 的结果 $x = 0.56714$ ，这里只要迭代 3 次同样得出这个结果。加速的效果是相当显著的。

然而上述加速方案有个缺点，由于其中含有导数 $\varphi'(x)$ 的有关信息 L ，实际使用不便。

仍设已知 x^* 的某个猜测值为 x_0 ，将校正值 $x_1 = \varphi(x_0)$ 再校正一次，又得

$$x_2 = \varphi(x_1),$$

由于 $x_2 - x^* \approx L(x_1 - x^*)$ 将它与式 (6.2.9) 联立，消去未知的 L ，有

$$\frac{x_1 - x^*}{x_2 - x^*} \approx \frac{x_0 - x^*}{x_1 - x^*}.$$

由此推知

$$x^* \approx \frac{x_0 x_2 - x_1^2}{x_0 - 2x_1 + x_2} = x_2 - \frac{(x_2 - x_1)^2}{x_0 - 2x_1 + x_2}.$$

这样构造出的改进公式确实不再含有关于导数的信息，但是它需要用两次迭代

校注 (agent 补充): 原页式 (6.2.9) 确印为等号，转录保留。由于上一步只是用近似常数 L 得到 $x_1 - x^* \approx L(x_0 - x^*)$ ，一般只能推出 $x^* \approx \frac{x_1 - Lx_0}{1 - L}$ ；除非该线性误差关系精确成立，式 (6.2.9) 的等号应作近似关系理解。此为原书等号精度的疑误，非 OCR 误识别。

原书 PDF 页 162; 印刷页码 149。

[原页](#)

值进行加工。如果将得到一次改进值作为一步, 则计算公式如下:

$$\begin{aligned} \text{校正} \quad \tilde{x}_{k+1} &= \varphi(x_k), \\ \text{再校正} \quad \bar{x}_{k+1} &= \varphi(\tilde{x}_{k+1}), \\ \text{改进} \quad x_{k+1} &= \bar{x}_{k+1} - \frac{(\bar{x}_{k+1} - \tilde{x}_{k+1})^2}{\bar{x}_{k+1} - 2\tilde{x}_{k+1} + x_k}. \end{aligned} \quad (6.2.11)$$

上述处理过程称为 Aitken 方法。

例 6.6 用 Aitken 方法求解方程 (6.2.3)。

解前面曾经指出, 求解这一方程的下述迭代公式是发散的:

$$x_{k+1} = x_k^3 - 1. \quad (6.2.12)$$

现在以这种迭代公式为基础形成 Aitken 算法, 即

$$\tilde{x}_{k+1} = x_k^3 - 1, \quad \bar{x}_{k+1} = \tilde{x}_{k+1}^3 - 1, \quad x_{k+1} = \bar{x}_{k+1} - \frac{(\bar{x}_{k+1} - \tilde{x}_{k+1})^2}{\bar{x}_{k+1} - 2\tilde{x}_{k+1} + x_k}.$$

仍然取 $x_0 = 1.5$, 计算结果如表 6.6 所示。

表 6.6

k	$\tilde{x}_k$	$\bar{x}_k$	x_k
0			1.5
1	2.375 00	12.396 5	1.416 29
2	1.840 92	5.238 88	1.355 65
3	1.491 40	2.317 28	1.328 95
4	1.347 10	1.444 35	1.324 80
5	1.325 18	1.327 14	1.324 72

可以看到, 将发散的迭代公式 (6.2.12) 通过 Aitken 方法处理后, 竟获得了相当好的收敛性。在 6.4 节将用几何图像解释这个有趣的现象。

6.3 Newton 法

6.3.1 Newton 公式

对于方程 $f(x) = 0$, 为要应用迭代法, 必须先将它改写成 $x = \varphi(x)$ 的形式, 即需要针对所给的函数 $f(x)$ 构造合适的迭代函数 $\varphi(x)$ 。

迭代函数 $\varphi(x)$ 可以是多种多样的。例如, 可令 $\varphi(x) = x + f(x)$, 这时相应的迭代公式是

原书 PDF 页 163; 印刷页码 150。

[原页](#)

$$x_{k+1} = x_k + f(x_k). \quad (6.3.1)$$

一般来说, 这种迭代公式不一定收敛, 或者收敛的速度缓慢。

运用前述加速技巧, 对于迭代过程 (6.3.1), 其加速公式 (6.2.10) 具有以下形式:

$$\begin{cases} \bar{x}_{k+1} = x_k + f(x_k), \\ x_{k+1} = \bar{x}_{k+1} + \frac{L}{1-L}(\bar{x}_{k+1} - x_k). \end{cases}$$

记 $M = L - 1$, 上面两个式子可以合并写成

$$x_{k+1} = x_k - \frac{f(x_k)}{M}.$$

这种迭代公式通常称为简化的 Newton 公式, 其相应的迭代函数是

$$\varphi(x) = x - \frac{f(x)}{M}. \quad (6.3.2)$$

需要注意的是, 由于 L 是 $\varphi'(x)$ 的估计值, 而 $\varphi(x) = x + f(x)$, 这里的 $M = L - 1$ 实际上是 $f'(x)$ 的估计值。如果用 $f'(x)$ 代替式 (6.3.2) 中的 M , 则得到如下形式的迭代函数:

$$\varphi(x) = x - \frac{f(x)}{f'(x)},$$

其相应的迭代公式

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \quad (6.3.3)$$

就是著名的 Newton 公式。

6.3.2 Newton 法的几何解释

对于方程 $f(x) = 0$, 如果 $f(x)$ 是线性函数, 则对它求根是容易的。Newton 法实质上是一种线性化方法, 其基本思想是将非线性方程 $f(x) = 0$ 逐步归结为某种线性方程来求解。

设已知方程 $f(x) = 0$ 有近似根 x_k , 将函数 $f(x)$ 在点 x_k 展开, 有

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k),$$

于是方程 $f(x) = 0$ 可近似地表示为

$$f(x_k) + f'(x_k)(x - x_k) = 0. \tag{6.3.4}$$

这是个线性方程，记其根为 x_{k+1} ，则 x_{k+1} 的计算公式就是 Newton 公式 (6.3.3)。

Newton 法有明显的几何解释。方程 $f(x) = 0$ 的根 x^* 可解释为曲线 $y = f(x)$ 与 x 轴的交点的横坐标（见图 6.3）。设 x_k 是根 x^* 的某个近似值，过曲

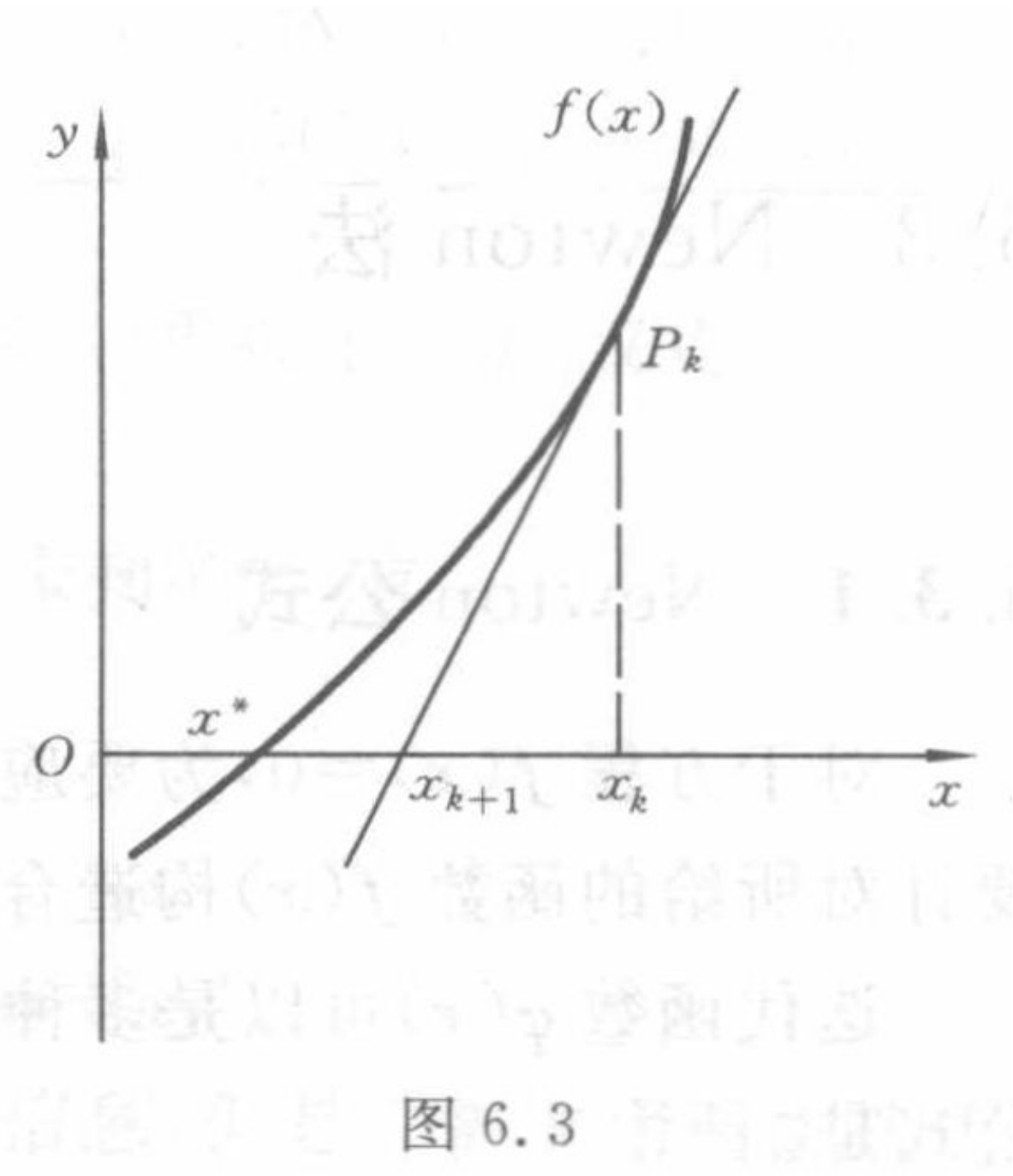


图 6.3

图意 (agent 整理)：曲线 $f(x)$ 在 P_k 处的切线与 x 轴相交，其交点横坐标是 x_{k+1} 。全部标签为 $x, y, O, x^*, x_{k+1}, x_k, P_k, f(x)$ 。

原书 PDF 页 164; 印刷页码 151。

[原页](#)

线 $y = f(x)$ 上横坐标为 x_k 的点 P_k 引切线, 并将该切线与 x 轴的交点的横坐标 x_{k+1} 作为 x^* 的新的近似值。注意到切线方程为

$$y = f(x_k) + f'(x_k)(x - x_k).$$

这样求得的值 x_{k+1} 必满足式 (6.3.4), 从而就是 Newton 公式 (6.3.3) 的计算结果。由于这种几何背景, Newton 法亦称切线法。

6.3.3 Newton 法的局部收敛性

对于一种迭代过程, 为了保证它是有效的, 需要肯定它的收敛性, 同时考察它的收敛速度。所谓迭代过程的收敛速度, 是指在接近收敛的过程中迭代误差的下降速度。

定义 6.2 设迭代过程 $x_{k+1} = \varphi(x_k)$ 收敛于方程 $x = \varphi(x)$ 的根 x^* , 如果迭代误差 $e_k = x_k - x^*$ 当 $k \rightarrow \infty$ 时成立下列渐近关系式:

$$\frac{e_{k+1}}{e_k^p} \rightarrow C \quad (C \neq 0 \text{ 为常数}),$$

则称该迭代过程是 p 阶收敛的。特别地, $p = 1$ 时称为线性收敛, $p > 1$ 时称为超线性收敛, $p = 2$ 时称为平方收敛。

定理 6.3 对于迭代过程 $x_{k+1} = \varphi(x_k)$, 如果 $\varphi^{(p)}(x)$ 在所求根 x^* 的邻近连续, 并且

$$\begin{cases} \varphi'(x^*) = \varphi''(x^*) = \cdots = \varphi^{(p-1)}(x^*) = 0, \\ \varphi^{(p)}(x^*) \neq 0, \end{cases} \quad (6.3.5)$$

则该迭代过程在点 x^* 邻近是 p 阶收敛的。

证明 由于 $\varphi'(x^*) = 0$, 据定理 6.2 立即可以断定迭代过程 $x_{k+1} = \varphi(x_k)$ 具有局部收敛性。

再将 $\varphi(x_k)$ 在根 x^* 处展开, 利用条件 (6.3.5), 则有

$$\varphi(x_k) = \varphi(x^*) + \frac{\varphi^{(p)}(\zeta)}{p!}(x_k - x^*)^p.$$

注意到

$$\varphi(x_k) = x_{k+1}, \quad \varphi(x^*) = x^*,$$

由上式得

$$x_{k+1} - x^* = \frac{\varphi^{(p)}(\zeta)}{p!}(x_k - x^*)^p,$$

因此对于迭代误差, 有 $\frac{e_{k+1}}{e_k^p} \rightarrow \frac{\varphi^{(p)}(x^*)}{p!}$ 。这表明迭代过程 $x_{k+1} = \varphi(x_k)$ 确实是 p 阶收敛的。证毕。

由上述定理可知, 迭代过程的收敛速度依赖于迭代函数 $\varphi(x)$ 的选取。如果当 $x \in [a, b]$ 时 $\varphi'(x) \neq 0$, 则该迭代过程只可能是线性收敛的。

对于 Newton 公式 (6.3.3), 其迭代函数为

原书 PDF 页 165; 印刷页码 152。

[原页](#)

$$\varphi(x) = x - \frac{f(x)}{f'(x)}, \quad \varphi'(x) = \frac{f(x)f''(x)}{[f'(x)]^2}.$$

假定 x^* 是 $f(x)$ 的一个单根, 即 $f(x^*) = 0, f'(x^*) \neq 0$, 则由上式知 $\varphi'(x^*) = 0$, 于是依据定理 6.3 可以断定, Newton 法在根 x^* 的邻近是平方收敛的。

例 6.7 用 Newton 法解方程

$$xe^x - 1 = 0. \quad (6.3.6)$$

解 这里 Newton 公式为 $x_{k+1} = x_k - \frac{x_k - e^{-x_k}}{1 + x_k}$, 取迭代初值 $x_0 = 0.5$, 迭代结果列于表 6.7 中。

表 6.7

k	x_k
0	0.5
1	0.571 02
2	0.567 16
3	0.567 14

所给方程 (6.3.6) 实际上是方程 $x = e^{-x}$ 的等价形式。比较例 6.7 与例 6.5 的计算结果可以看出, Newton 法的收敛速度是很快的。

下面列出 Newton 法的计算步骤:

步 1 准备。选定初始近似值 x_0 , 计算 $f_0 = f(x_0), f'_0 = f'(x_0)$ 。

步 2 迭代。按公式 $x_1 = x_0 - f_0/f'_0$ 迭代一次, 得新的近似值 x_1 , 计算

$$f_1 = f(x_1), \quad f'_1 = f'(x_1).$$

步 3 控制。如果 x_1 满足 $|\delta| < \varepsilon_1$ 或 $|f_1| < \varepsilon_2$, 则终止迭代, 以 x_1 作为所求的根; 否则转步 4。此处 $\varepsilon_1, \varepsilon_2$ 是允许误差, 而

$$\delta = \begin{cases} |x_1 - x_0|, & \text{当 } |x_1| < C \text{ 时,} \\ \frac{|x_1 - x_0|}{|x_1|}, & \text{当 } |x_1| \geqslant C \text{ 时,} \end{cases}$$

其中 C 是取绝对误差或相对误差的控制常数, 一般可取 $C = 1$ 。

步 4 修改。如果迭代次数达到预先指定的次数 N 或者 $f'_1 = 0$, 则方法失败, 否则以 (x_1, f_1, f'_1) 代替 (x_0, f_0, f'_0) 转步 2 继续迭代。

6.3.4 Newton 法应用举例

对于给定正数 a ，应用 Newton 法解二次方程 $x^2 - a = 0$ ，可导出求开方值 $\sqrt{a}$ 的计算程序

$$x_{k+1} = \frac{1}{2} \left(x_k + \frac{a}{x_k} \right). \quad (6.3.7)$$

下面证明这种迭代公式对于任意初值 $x_0 > 0$ 都是收敛的。

事实上，对式 (6.3.7) 施行配方手续，易知

$$x_{k+1} - \sqrt{a} = \frac{1}{2x_k}(x_k - \sqrt{a})^2, \quad x_{k+1} + \sqrt{a} = \frac{1}{2x_k}(x_k + \sqrt{a})^2.$$

将以上两式相除得

校注 (agent 补充)：原页步 4 的替换三元组印为 (x_1, f_1, f') ，第三项未印下标 1，转录保留原式。结合步 2 的定义及被替换的 f'_0 ，这里疑应为 f'_1 。这是原扫描上的疑似漏印，非 OCR 改写；局部原图见 [步 4 裁切](#)。

校注 (agent 补充)：本页首段“单根”推出“平方收敛”的原文亦予保留。若按前页定义 6.2 要求精确收敛阶的极限常数非零，则还需 $f''(x^*) \neq 0$ 才能断言恰为二阶；在通常光滑性条件下，单根保证的是局部至少二阶收敛。例如 $f(x) = x + x^3$ 在单根 0 附近的 Newton 迭代为 $x_{k+1} = 2x_k^3/(1 + 3x_k^2)$ ，实际为三阶。这是原文收敛阶表述的适用条件说明。

原书 PDF 页 166; 印刷页码 153。

[原页](#)

$$\frac{x_{k+1} - \sqrt{a}}{x_{k+1} + \sqrt{a}} = \left[\frac{x_k - \sqrt{a}}{x_k + \sqrt{a}} \right]^2.$$

据此反复递推, 有

$$\frac{x_k - \sqrt{a}}{x_k + \sqrt{a}} = \left[\frac{x_0 - \sqrt{a}}{x_0 + \sqrt{a}} \right]^{2^k}, \quad (6.3.8)$$

记 $q = \frac{x_0 - \sqrt{a}}{x_0 + \sqrt{a}}$, 整理式 (6.3.8), 得

$$x_k - \sqrt{a} = 2\sqrt{a} \frac{q^{2^k}}{1 - q^{2^k}}.$$

对于任意 $x_0 > 0$, 总有 $|q| < 1$, 故由上式推知, 当 $k \rightarrow \infty$ 时 $x_k \rightarrow \sqrt{a}$, 即迭代过程恒收敛。

例 6.8 求 $\sqrt{115}$ 。

解取初值 $x_0 = 10$, 对 $a = 115$ 按式 (6.3.7) 迭代 3 次便得到精度为 10^{-6} 的结果 (见表 6.8)。

表 6.8

k	x_k
0	10
1	10.750 000
2	10.723 837
3	10.723 805
4	10.723 805

由于式 (6.3.7) 对于任意初值 $x_0 > 0$ 均收敛, 并且收敛的速度很快, 因此, 可取确定的初值如 $x_0 = 1$ 编制通用程序。用这个通用程序求 $\sqrt{115}$, 也只要迭代 7 次便得到了上面的结果 10.723 805。

再举一个例子。

对于给定的正数 a , 应用 Newton 法于方程 $1/x - a = 0$, 可导出求 $1/a$ 而不用除法的计算程序, 即

$$x_{k+1} = x_k(2 - ax_k).$$

这个算法有实际意义, 早期设计电子计算机时, 为节省硬件设备, 曾运用这种技术避开除法操作的设置。

下面证明这一算法当初值 x_0 满足 $0 < x_0 < 2/a$ 时是收敛的。事实上, 由于

$$x_{k+1} - \frac{1}{a} = x_k(2 - ax_k) - \frac{1}{a} = -a \left(x_k - \frac{1}{a} \right)^2,$$

因而, 对 $r_k = 1 - ax_k$ 有递推公式 $r_{k+1} = r_k^2$ 。据此反复递推, 有

$$r_k = r_0^{2^k}.$$

若初值满足 $0 < x_0 < \frac{2}{a}$, 则对 $r_0 = 1 - ax_0$ 有 $|r_0| < 1$ 。这时将有 $r_k \rightarrow 0$, 从而迭代收敛。

6.3.5 Newton 下山法

前面已经讨论过 Newton 法的局部收敛性。一般来说, Newton 法的收敛性依赖

原书 PDF 页 167; 印刷页码 154。

[原页](#)

于初值 x_0 的选取, 如果 x_0 偏离所求的根 x^* 比较远, 则 Newton 法可能发散。

例如, 用 Newton 法求方程

$$x^3 - x - 1 = 0, \quad (6.3.9)$$

在 $x = 1.5$ 附近的一个根 x^* 。设取迭代初值 $x_0 = 1.5$, 用 Newton 公式

$$x_{k+1} = x_k - \frac{x_k^3 - x_k - 1}{3x_k^2 - 1} \quad (6.3.10)$$

计算, 得

$$x_1 = 1.34783, \quad x_2 = 1.32520, \quad x_3 = 1.32472.$$

迭代三次得到的结果 x_3 有六位有效数字。

但是, 如果改用 $x_0 = 0.6$ 作为迭代初值, 则依 Newton 公式 (6.3.10) 迭代一次得

$$x_1 = 17.9.$$

这个结果反而比 $x_0 = 0.6$ 更偏离了所求的根 $x^* = 1.32472$ 。

为了防止迭代发散, 对迭代过程再附加一项要求, 即具有单调性

$$|f(x_{k+1})| < |f(x_k)|. \quad (6.3.11)$$

满足这项要求的算法称为下山法。

将 Newton 法与下山法结合起来使用, 即可在下山法保证函数值稳定下降的前提下, 用 Newton 法加快收敛速度。为此, 将 Newton 法的计算结果 $\bar{x}_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$ 与前一步的近似值 x_k 适当加权平均作为新的改进值, 即

$$x_{k+1} = \lambda \bar{x}_{k+1} + (1 - \lambda)x_k, \quad (6.3.12)$$

其中 λ ($0 < \lambda \leq 1$) 称为下山因子。在希望挑选下山因子时, 希望使单调性条件 (6.3.11) 成立。

下山因子的选择是个逐步探索的过程。设从 $\lambda = 1$ 开始反复将 λ 减半进行试算, 如果能定出值 λ 使单调性条件 (6.3.11) 成立, 则称“下山成功”。与此相反, 如果在上述过程中找不到使条件 (6.3.11) 成立的下山因子 λ , 则称“下山失败”, 这时需另选初值 x_0 重算。

6.4 弦截法与抛物线法

上一节介绍了 Newton 法。在用 Newton 公式 (6.3.3) 求 x_{k+1} 时, 不但要求给出函数值 $f(x_k)$, 而且要求提供导数值 $f'(x_k)$ 。当函数 f 比较复杂时, 提供它的导数值往往是有困难的。现在设法利用迭代过程中的“老信息” $f(x_k), f(x_{k-1}), f(x_{k-2}), \dots$ 来回避导数值 $f'(x_k)$ 的计算, 导出这类求根方法的基础是插值原理。

设 $x_k, x_{k-1}, \dots, x_{k-r}$ 是 $f(x) = 0$ 的一组近似根, 利用函数值 $f(x_k), f(x_{k-1}), \dots, f(x_{k-r})$ 构造插值多项式 $P_r(x)$, 并适当选取 $P_r(x) = 0$ 的一个根作为 $f(x) = 0$ 的新

原书 PDF 页 168; 印刷页码 155。

[原页](#)

的近似根 x_{k+1} , 这就确定了一个迭代过程, 记迭代函数为 φ , 且

$$x_{k+1} = \varphi(x_k, x_{k-1}, \dots, x_{k-r}).$$

下面具体考察 $r = 1$ (弦截法) 和 $r = 2$ (抛物线法) 两种情形。

6.4.1 弦截法

设 x_k, x_{k-1} 是 $f(x) = 0$ 的近似根, 利用 $f(x_k), f(x_{k-1})$ 构造一次插值多项式 $P_1(x)$, 并用 $P_1(x) = 0$ 的根作为 $f(x) = 0$ 的新的近似根 x_{k+1} 。由于

$$P_1(x) = f(x_k) + \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}(x - x_k), \quad (6.4.1)$$

因此有

$$x_{k+1} = x_k - \frac{f(x_k)}{f(x_k) - f(x_{k-1})}(x_k - x_{k-1}). \quad (6.4.2)$$

这样导出的迭代公式 (6.4.2) 可以看作 Newton 公式 $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$ 中的导数 $f'(x)$ 用差商 $\frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$ 取代的结果。

现在解释这种迭代过程的几何意义。如图 6.4 所示, 曲线 $y = f(x)$ 上横坐标为 x_k, x_{k-1} 的点分别记作 P_k, P_{k-1} , 则弦线 $P_k P_{k-1}$ 的斜率等于差商值 $\frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$, 其方程是

$$f(x_k) + \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}(x - x_k) = 0.$$

因此, 按式 (6.4.2) 求得的 x_{k+1} 实际上是弦线 $P_k P_{k-1}$ 与 x 轴交点的横坐标。这种算法因此而称为弦截法。

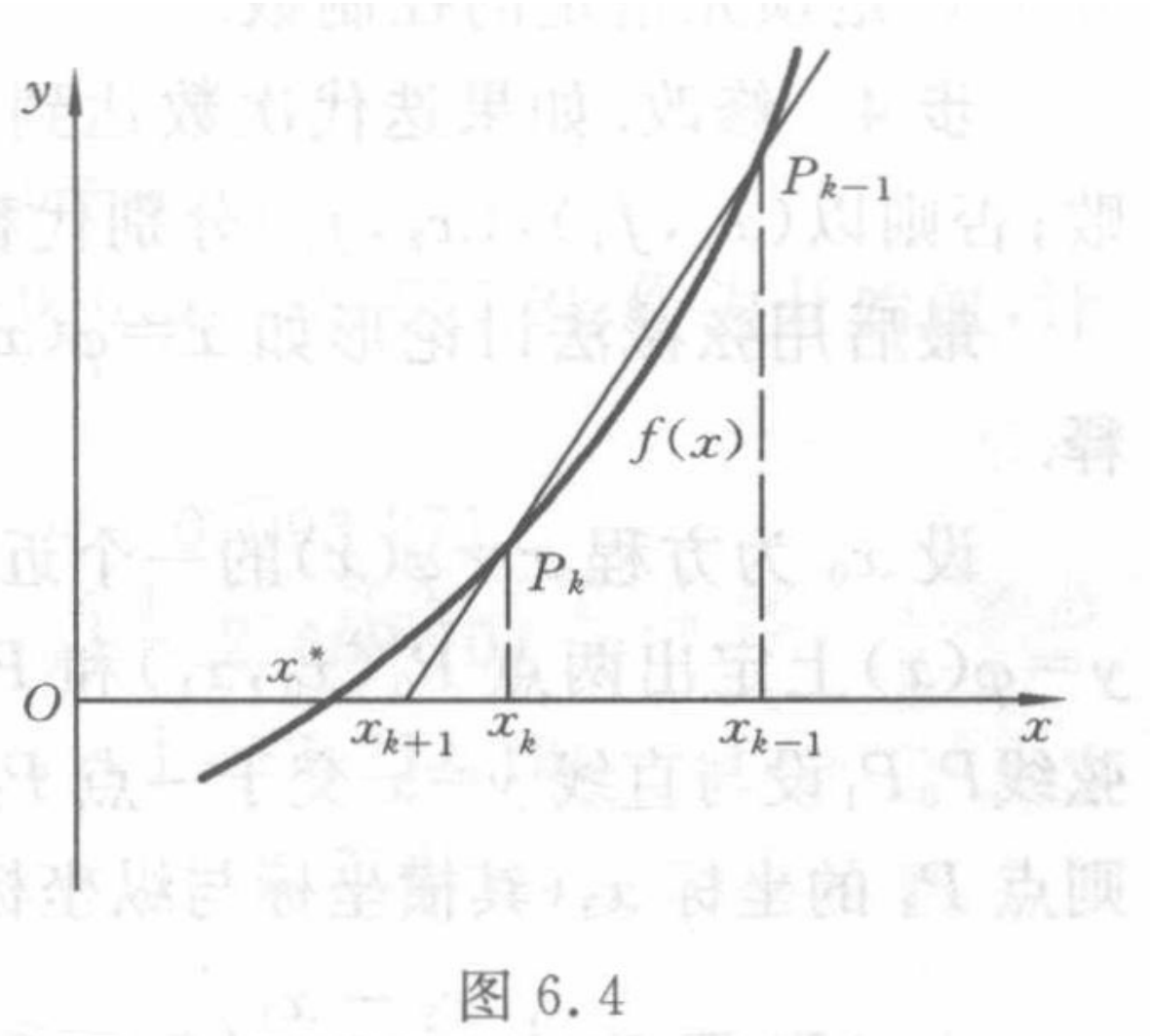


图 6.4

图 6.4

图意 (agent 整理): 曲线上的 P_k, P_{k-1} 所连弦线与横轴交于横坐标 x_{k+1} 的点。全部标签为 $x, y, O, x^*, x_{k+1}, x_k, x_{k-1}, P_k, P_{k-1}, f(x)$ 。

弦截法与切线法 (Newton 法) 都是线性化方法, 但二者有本质的区别。切线法在计算 x_{k+1} 时只用到前一步的值 x_k , 而弦截法 (见式 (6.4.2)) 在求 x_{k+1} 时要用到前面两步的结果 x_k, x_{k-1} , 因此使用这种方法必须先给出两个开始值 x_0, x_1 。

例 6.9 用弦截法解方程

$$f(x) = xe^x - 1 = 0.$$

解 设取 $x_0 = 0.5, x_1 = 0.6$ 作为开始值, 用弦截法求得的结果如表 6.9 所示, 比较例 6.7 Newton 法的计算结果可以看出, 弦截法的收敛速度也是相当快的。

表 6.9

k	x_k
0	0.5
1	0.6

k	x_k
2	0.565 32
3	0.567 09
4	0.567 14

下述定理断言，弦截法具有超线性的收敛性。

定理 6.4 假设 $f(x)$ 在根 x^* 的邻域 $\Delta: |x - x^*| \leq \delta$ 内具有二阶连续导数，且对于任意

校注 (agent 补充)：几何解释中原书称“其方程是”，随后印出的却是 $P_1(x) = 0$ ，已照录。弦线本身的方程为 $y = P_1(x)$ ；原式表达的是弦线与 x 轴交点的条件，与随后求 x_{k+1} 的用途一致。

原书 PDF 页 169; 印刷页码 156。

[原页](#)

$x \in \Delta$, 有 $f'(x) \neq 0$, 又初值 $x_0, x_1 \in \Delta$, 那么当邻域 Δ 充分小时, 弦截法 (6.4.2) 将按阶 $p = \frac{1+\sqrt{5}}{2} \approx 1.618$ 收敛到根 x^* 。

由于定理 4 的证明比较烦琐, 故略去不证。下面列出弦截法的计算步骤。

步 1 准备。选取初始近似值 x_0, x_1 , 计算相应的函数值 $f_0 = f(x_0), f_1 = f(x_1)$ 。

步 2 迭代。按公式

$$x_2 = x_1 - f_1 \left/ \frac{f_1 - f_0}{x_1 - x_0} \right.$$

迭代一次得新的近似值 x_2 , 计算 $f_2 = f(x_2)$ 。

步 3 控制。如果 x_2 满足 $|\delta| \leq \varepsilon_1$ 或 $|f_2| \leq \varepsilon_2$, 则认为过程收敛, 终止迭代而以 x_2 作为所求的根; 否则执行步 4。此处 $\varepsilon_1, \varepsilon_2$ 是允许误差, 而

$$\delta = \begin{cases} |x_2 - x_1|, & \text{当 } |x_2| < C \text{ 时,} \\ \frac{|x_2 - x_1|}{|x_2|}, & \text{当 } |x_2| \geq C \text{ 时,} \end{cases}$$

其中 C 是预先指定的控制数。

步 4 修改。如果迭代次数达到预先指定的次数 N , 则认为过程不收敛, 计算失败; 否则以 $(x_1, f_1), (x_2, f_2)$ 分别代替 $(x_0, f_0), (x_1, f_1)$, 而后转步 2 继续迭代。

最后用弦截法讨论形如 $x = \varphi(x)$ 的方程, 对 6.2.2 节的 Aitken 算法给出几何解释。

设 x_0 为方程 $x = \varphi(x)$ 的一个近似根, 依据迭代值 $x_1 = \varphi(x_0), x_2 = \varphi(x_1)$ 在曲线 $y = \varphi(x)$ 上定出两点 $P_0(x_0, x_1)$ 和 $P_1(x_1, x_2)$, 引弦线 P_0P_1 设与直线 $y = x$ 交于一点 P_3 (见图 6.5), 则点 P_3 的坐标 x_3 (其横坐标与纵坐标相等) 满足

$$x_3 = x_1 + \frac{x_2 - x_1}{x_1 - x_0}(x_3 - x_0).$$

由此解出

$$x_3 = \frac{x_0x_2 - x_1^2}{x_0 - 2x_1 + x_2},$$

此即 Aitken 加速公式 (6.2.2 节)。

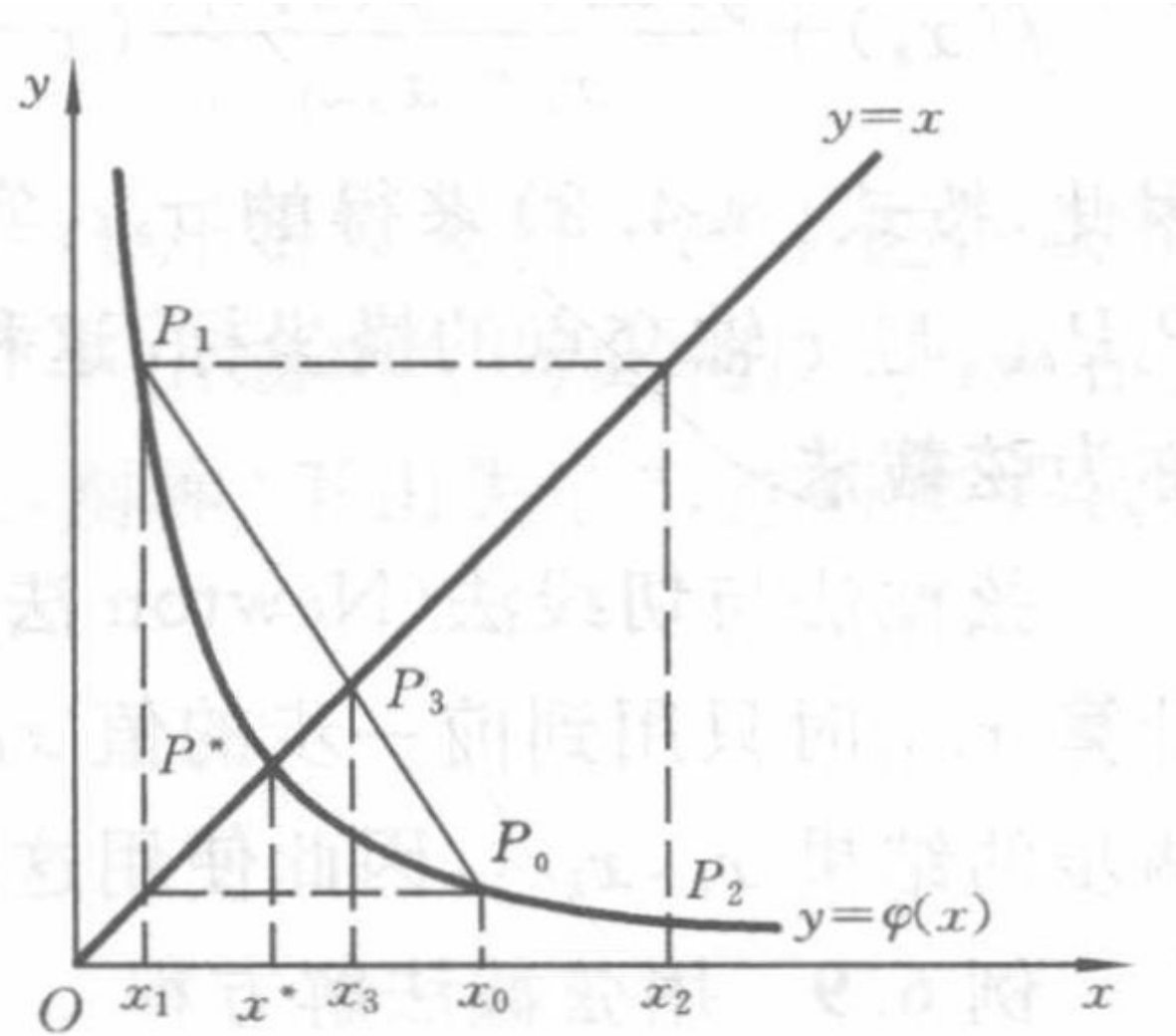


图 6.5

图 6.5

图意 (agent 整理): 图用递减曲线 $y = \varphi(x)$ 、直线 $y = x$ 、弦线 P_0P_1 及其与 $y = x$ 的交点 P_3 显示加速过程。全部标签为 $x, y, O, x_1, x^*, x_3, x_0, x_2, P_1, P^*, P_3, P_0, P_2, y = x, y = \varphi(x)$ 。

再考察图 6.5，所求的根 x^* 是曲线 $y = \varphi(x)$ 与 $y = x$ 交点 P^* 的横坐标，从图形上可以看出，尽管迭代值 x_2 比 x_0 和 x_1 更远地偏离了 x^* ，但按上式定出的 x_3 却明显地扭转了这种发散的趋势。

6.4.2 抛物线法

设已知方程 $f(x) = 0$ 的三个近似根 x_k, x_{k-1}, x_{k-2} ，以这三点为节点构造二次插值多项式 $P_2(x)$ ，并适当选取 $P_2(x)$ 的一个零点 x_{k+1} 作为新的近似根，这样确定的迭

校注 (agent 补充): 定理 6.4 的原文条件与 $p = (1 + \sqrt{5})/2$ 结论均已保留。若将此处“按阶”理解为定义 6.2 中极限常数非零的精确收敛阶，还需 $f''(x^*) \neq 0$ 等非退化条件，并排除有限步已得根的情形。例如一次函数满足原定理的光滑性与导数非零条件，但弦截法可一步得根，不能一概赋予上述精确阶。

原书 PDF 页 170; 印刷页码 157。

[原页](#)

代过程称为抛物线法。在几何图形上, 这种方法的基本思想是用抛物线 $y = P_2(x)$ 与 x 轴的交点 x_{k+1} 作为所求根 x^* 的近似位置 (见图 6.6)。

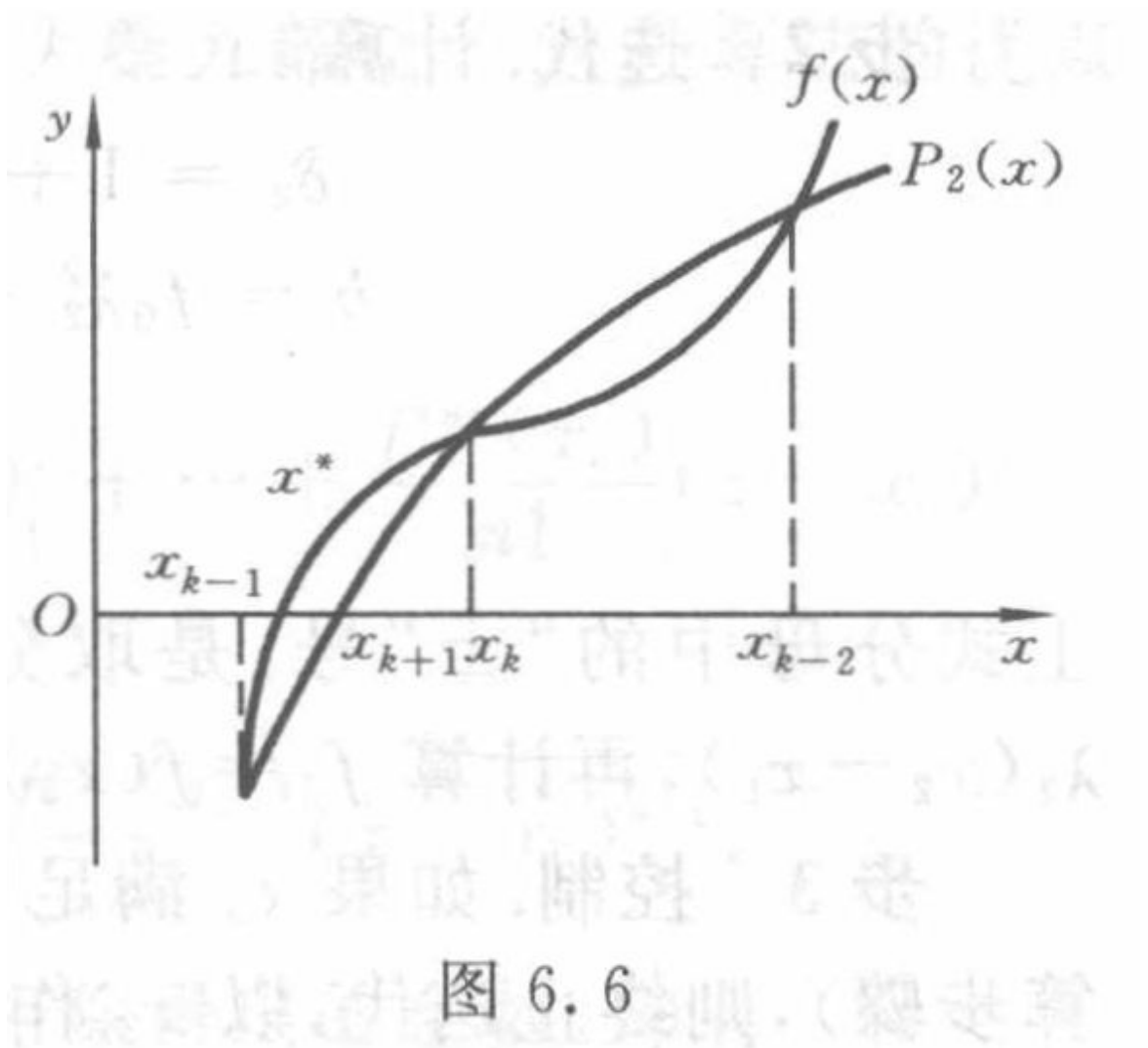


图 6.6

图 6.6

图意 (agent 整理): 原图以抛物线 $P_2(x)$ 近似曲线 $f(x)$, 用抛物线与 x 轴的交点作为新近似根。全部标签为 $x, y, O, x^*, x_{k-1}, x_{k+1}, x_k, x_{k-2}, f(x), P_2(x)$ 。

现在推导抛物线法的计算公式。插值多项式

$$P_2(x) = f(x_k) + f[x_k, x_{k-1}](x - x_k) + f[x_k, x_{k-1}, x_{k-2}](x - x_k)(x - x_{k-1})$$

有两个零点

$$x_{k+1} = x_k - \frac{2f(x_k)}{\omega \pm \sqrt{\omega^2 - 4f(x_k)f[x_k, x_{k-1}, x_{k-2}]}} \quad (6.4.3)$$

其中

$$\omega = f[x_k, x_{k-1}] + f[x_k, x_{k-1}, x_{k-2}](x_k - x_{k-1}).$$

为了从式 (6.4.3) 定出一个值 x_{k+1} , 需要讨论根式前正负号的取舍问题。

在 x_k, x_{k-1}, x_{k-2} 三个近似根中, 自然假定以 x_k 更接近所求的根 x^* , 这时, 为了保证精度, 可以选式 (6.4.3) 中较接近 x_k 的一个值作为新的近似根 x_{k+1} , 为此, 只要令根式前的符号与 ω 的符号相同。

例 6.10 用抛物线法求解方程 $f(x) = xe^x - 1 = 0$ 。

解 设用表 6.9 中的前三个值 $x_0 = 0.5, x_1 = 0.6, x_2 = 0.56532$ 作为开始值, 计算得

$$\begin{aligned} f(x_0) &= -0.175639, & f(x_1) &= -0.093271, \\ f(x_2) &= -0.005031, & f[x_1, x_0] &= 2.68910, \\ f[x_2, x_1] &= 2.83373, & f[x_2, x_1, x_0] &= 2.21418, \end{aligned}$$

故

$$\omega = f[x_2, x_1] + f[x_2, x_1, x_0](x_2 - x_1) = 2.75694.$$

代入式 (6.4.8) 求得

$$x_3 = x_2 - \frac{2f(x_2)}{\omega + \sqrt{\omega^2 - 4f(x_2)f[x_2, x_1, x_0]}} = 0.56714.$$

以上计算表明, 抛物线法比弦截法收敛得更快。

事实上, 在一定条件下可以证明, 对于抛物线法, 迭代误差有下列渐近关系式:

$$\frac{|e_{k+1}|}{|e_k|^{1.840}} \rightarrow \left| \frac{f'''(x^*)}{6f'(x^*)} \right|^{0.42},$$

可见抛物线法也是超线性收敛的, 其收敛的阶 $p = 1.840$, 收敛速度比弦截法更接近于 Newton 法。

下面列出抛物线法的计算步骤。

步 1 准备。选定初始近似值 x_0, x_1, x_2 , 并计算 $f(x)$ 相应的值 f_0, f_1, f_2 , 以及

$$\lambda_2 = \frac{x_2 - x_1}{x_1 - x_0}.$$

校注 (agent 补充): 原页例 6.10 印为 $f(x_1) = -0.093271$, 此处保留原文。由同页 $x_1 = 0.6$ 和 $f(x) = xe^x - 1$, 计算得 $f(0.6) = 0.093271280 \dots > 0$; 同时 $[f(0.6) - f(0.5)]/0.1 = 2.689106 \dots$ 与原页列出的差商一致, 因此原页该负号疑为误印。原页的“代入式 (6.4.8)”也予保留; 该段实际使用的零点公式编号是同页 (6.4.3)。

原书 PDF 页 171; 印刷页码 158。

[原页](#)

步 2 迭代。计算

$$\begin{aligned}\delta_2 &= 1 + \lambda_2, \quad a = f_0 \lambda_2^2 - f_1 \lambda_2 \delta_2 + f_2 \lambda_2, \\ b &= f_0 \lambda_2^2 - f_1 \delta_2^2 + f_2 (\lambda_2 + \delta_2), \quad c = f_2 \delta_2, \\ \lambda_3 &= \frac{-2c}{b \pm \sqrt{b^2 - 4ac}}.\end{aligned}$$

上式分母中的“ $\pm$ ”号，是取分母的模较大的一个。于是得新的近似值 $x_3 = x_2 + \lambda_3(x_2 - x_1)$ ，再计算 $f_3 = f(x_3)$ 。

步 3 控制。如果 x_3 满足 $|\delta| \leq \varepsilon_1$ 或 $|f_3| < \varepsilon_2$ ($\varepsilon_1, \varepsilon_2$ 和 δ 的意义同弦截法的计算步骤)，则终止迭代，以 x_3 作为所求的根；否则执行步 4。

步 4 修改。如果迭代次数达到预先指定的次数 N ，则认为过程不收敛，输出计算失效标志；否则以 $(x_1, x_2, x_3, f_1, f_2, f_3, \lambda_3)$ 分别代替 $(x_0, x_1, x_2, f_0, f_1, f_2, \lambda_2)$ 转步 2 继续迭代。

6.5 代数方程求根

如果 $f(x)$ 是多项式，则 $f(x) = 0$ 特别地称为代数方程。前面介绍的求根方法原则上也适用于解代数方程，但由于多项式的特殊性，可以针对其特点提供更为有效的算法。

6.5.1 多项式求值的秦九韶算法

多项式的一个重要特点是求值方便。

设给定多项式

$$f(x) = a_0 x^n + a_1 x^{n-1} + \cdots + a_{n-1} x + a_n,$$

其中系数 a_i ($0 \leq i \leq n$) 均为实数。用一次式 $x - x_0$ 除 $f(x)$ ，商记作 $P(x)$ ，余数显然等于 $f(x_0)$ ，即有

$$f(x) = f(x_0) + (x - x_0)P(x). \quad (6.5.1)$$

现在具体确定 $P(x)$ 与 $f(x_0)$ 。令

$$p(x) = b_0 x^{n-1} + b_1 x^{n-2} + \cdots + b_{n-2} x + b_{n-1},$$

代入式 (6.5.1)，并比较两端同次幂的系数，得

$$\begin{cases} a_0 = b_0, \\ a_i = b_i - x_0 b_{i-1}, \quad 1 \leq i \leq n-1, \\ a_n = f(x_0) - x_0 b_{n-1}. \end{cases}$$

从而有

$$\begin{cases} b_0 = a_0, \\ b_i = a_i + x_0 b_{i-1}, \quad 1 \leq i \leq n, \\ f(x_0) = b_n. \end{cases} \tag{6.5.2}$$

原书 PDF 页 172; 印刷页码 159。

[原页](#)

这里提供的一种计算函数值 $f(x_0)$ 的有效算法称为秦九韶法①。这种算法的优点是计算量小, 结构紧凑, 容易编制计算程序。

进一步考察 $f(x)$ 的 Taylor 展开式

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \cdots + \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n.$$

将它表示为式 (6.5.1) 的形式, 则式中

$$P(x) = f'(x_0) + \frac{f''(x_0)}{2!}(x - x_0) + \cdots + \frac{f^{(n)}(x_0)}{n!}(x - x_0)^{n-1}.$$

由此可见, 导数 $f'(x_0)$ 又可看作 $P(x)$ 用因式 $x - x_0$ 相除得出的余数, 即有

$$P(x) = f'(x_0) + (x - x_0)Q(x),$$

其中 $Q(x)$ 是 $n - 2$ 次多项式。设

$$Q(x) = c_0x^{n-2} + c_1x^{n-3} + \cdots + c_{n-3}x + c_{n-2},$$

那么, 再用秦九韶算法又可求出值 $f'(x_0)$, 对应于式 (6.5.2), 这里计算公式是

$$\begin{cases} c_0 = b_0, \\ c_i = b_i + x_0c_{i-1}, & 1 \leq i \leq n-1, \\ f'(x_0) = c_{n-1}. \end{cases} \quad (6.5.3)$$

继续这一过程, 可以依次求出 $f(x)$ 在点 x_0 的各阶导数。

6.5.2 代数方程的 Newton 法

再就多项式方程

$$f(x) = a_0x^n + a_1x^{n-1} + \cdots + a_{n-1}x + a_n = 0$$

考察 Newton 公式

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}. \quad (6.5.4)$$

据式 (6.5.2) 与式 (6.5.3), 式 (6.5.4) 中的函数值 $f(x_k)$ 和导数值 $f'(x_k)$ 均可方便地求出

$$\begin{cases} b_0 = a_0, \\ b_i = a_i + x_k b_{i-1}, & 1 \leq i \leq n, \\ f(x_k) = b_n, \end{cases} \quad (6.5.5)$$

$$\begin{cases} c_0 = b_0, \\ c_i = b_i + x_k c_{i-1}, & 1 \leq i \leq n-1, \\ f'(x_k) = c_{n-1}. \end{cases} \quad (6.5.6)$$

①这种算法是我国宋代的一位数学家秦九韶最先提出的。外国文献中通常称这种算法为 Horner 算法，其实 Horner 的工作比秦九韶晚了五六个世纪。

原书 PDF 页 173; 印刷页码 160。

[原页](#)

6.5.3 劈因子法

如果能从多项式 $f(x) = a_0x^n + a_1x^{n-1} + \cdots + a_{n-1}x + a_n$ 中分离出一个二次因式

$$\omega^*(x) = x^2 + u^*x + v^*,$$

就能获得它的一对共轭复根。劈因子法的基本思想是, 从某个近似的二次因子

$$\omega(x) = x^2 + ux + v$$

出发, 用某种迭代过程使之逐步精确化。

用二次式 $\omega(x)$ 除 $f(x)$, 商记作 $p(x)$, 它是个 $n-2$ 次多项式, 余式为一次式, 记作 $r_0x + r_1$, 即

$$f(x) = (x^2 + ux + v)p(x) + r_0x + r_1. \quad (6.5.7)$$

显然, r_0, r_1 均为 u, v 的函数 $\begin{cases} r_0 = r_0(u, v), \\ r_1 = r_1(u, v). \end{cases}$ 劈因子法的目的就是逐步修改 u, v 的值, 使余数 r_0, r_1 变得很小。考察方程

$$\begin{cases} r_0(u, v) = 0, \\ r_1(u, v) = 0. \end{cases} \quad (6.5.8)$$

这是关于 u, v 的非线性方程组, 设它有解 (u^*, v^*) , 将 $r_0(u^*, v^*) = 0, r_1(u^*, v^*) = 0$ 的左端在 (u, v) 展开到一阶项, 则有

$$\begin{cases} r_0 + \frac{\partial r_0}{\partial u}(u^* - u) + \frac{\partial r_0}{\partial v}(v^* - v) \approx 0, \\ r_1 + \frac{\partial r_1}{\partial u}(u^* - u) + \frac{\partial r_1}{\partial v}(v^* - v) \approx 0. \end{cases}$$

这样, 用 Newton 法的处理思想将非线性方程组 (6.5.8) 线性化, 归结得到下列线性方程组:

$$\begin{cases} r_0 + \frac{\partial r_0}{\partial u}\Delta u + \frac{\partial r_0}{\partial v}\Delta v = 0, \\ r_1 + \frac{\partial r_1}{\partial u}\Delta u + \frac{\partial r_1}{\partial v}\Delta v = 0. \end{cases} \quad (6.5.9)$$

从方程组 (6.5.9) 解出增量 $\Delta u, \Delta v$, 即可得到改进的二次因式

$$\omega(x) = x^2 + (u + \Delta u)x + v + \Delta v.$$

以下具体说明如何计算方程组 (6.5.9) 的各个系数。

(1) 计算 r_0 和 r_1 。将

$$P(x) = b_0x^{n-2} + b_1x^{n-3} + \cdots + b_{n-3}x + b_{n-2}$$

代入式 (6.5.7)，并比较各次幂的系数，易知

$$\begin{cases} a_0 = b_0, \\ a_1 = b_1 + ub_0, \\ a_i = b_i + ub_{i-1} + vb_{i-2}, & 2 \leq i \leq n-2, \\ a_{n-1} = ub_{n-2} + vb_{n-3} + r_0, \\ a_n = vb_{n-2} + r_1, \end{cases}$$

原书 PDF 页 174; 印刷页码 161。

[原页](#)

于是 r_0, r_1 的计算公式为

$$\begin{cases} b_0 = a_0, \\ b_1 = a_1 - ub_0, \\ b_i = a_i - ub_{i-1} - vb_{i-2}, & 2 \leq i \leq n, \\ r_0 = b_{n-1}, \\ r_1 = b_n + ub_{n-1}. \end{cases} \quad (6.5.10)$$

(2) 计算 $\frac{\partial r_0}{\partial v}, \frac{\partial r_1}{\partial v}$ 。对式 (6.5.7) 关于 v 求导, 得

$$P(x) = -(x^2 + ux + v) \frac{\partial P}{\partial v} + s_0 x + s_1, \quad (6.5.11)$$

其中

$$s_0 = -\frac{\partial r_0}{\partial v}, \quad s_1 = -\frac{\partial r_1}{\partial v}. \quad (6.5.12)$$

可见, 用 $x^2 + ux + v$ 除 $P(x)$, 作为余式可以得到 $s_0 x + s_1$ 。由于 $P(x)$ 是 $n-2$ 次多项式, 这里商 $\frac{\partial P}{\partial v}$ 是 $n-4$ 次多项式, 记

$$\frac{\partial P}{\partial v} = c_0 x^{n-4} + c_1 x^{n-5} + \cdots + c_{n-5} x + c_{n-4},$$

则相当于式 (6.5.10), 这里有

$$\begin{cases} c_0 = b_0, \\ c_1 = b_1 - ub_0, \\ c_i = b_i - uc_{i-1} - vc_{i-2}, & 2 \leq i \leq n-2, \\ s_0 = c_{n-3}, \\ s_1 = c_{n-2} + uc_{n-3}, \end{cases}$$

而按式 (6.5.12) 得

$$\frac{\partial r_0}{\partial v} = -s_0, \quad \frac{\partial r_1}{\partial v} = -s_1.$$

(3) 计算 $\frac{\partial r_0}{\partial u}, \frac{\partial r_1}{\partial u}$ 。对式 (6.5.7) 关于 u 求导, 得

$$xP(x) = -(x^2 + ux + v) \frac{\partial P}{\partial u} - \frac{\partial r_0}{\partial u} x - \frac{\partial r_1}{\partial u}.$$

另外, 由式 (6.5.11) 有

$$\begin{aligned} xP(x) &= -(x^2 + ux + v)x \frac{\partial P}{\partial v} + (s_0x + s_1)x \\ &= -(x^2 + ux + v) \left(x \frac{\partial P}{\partial v} - s_0 \right) - (us_0 - s_1)x - vs_0. \end{aligned}$$

比较以上两式可知

$$\frac{\partial r_0}{\partial u} = us_0 - s_1, \quad \frac{\partial r_1}{\partial u} = vs_0.$$

校注 (agent 补充): 本页原文把用 $x^2 + ux + v$ 除 $P(x)$ 的商记为 $\partial P / \partial v$, 并给出以 $c_0 = b_0$ 开始的递推。式 (6.5.11) 本身表明该商应为 $-\partial P / \partial v$, 因此商的叙述和紧随其后的展开式疑漏印负号。转录保留原文, 后续的余数 s_0, s_1 及其递推亦按原页保留。

原书 PDF 页 175; 印刷页码 162。

[原页](#)

小结

方程 $f(x) = 0$ 求根有多种方法, 本章重点介绍了 Newton 法。Newton 法是一种行之有效的迭代法, 在单根附近具有较高的收敛速度。应用 Newton 法的关键在于选取足够精确的初值。如果初值选取不当 (偏离所求的根比较远), 则 Newton 法可能发散。Newton 下山法 (见 6.3.5 节) 有时可用来克服这个局限性。

Newton 法的另一个局限性是要求计算导数值。如果函数 f 的形式复杂而不便于求导, 则可用导数的估值或差商代替导数, 而得出简化的 Newton 法 (见 6.3.1 节) 或近似的 Newton 法 (见 6.4.1 节)。不过这样处理势必会影响收敛速度。

应当指出, 对于代数方程 $a_0x^n + a_1x^{n-1} + \cdots + a_n = 0$, 其求根问题包含有许多深刻的理论和有效的算法, 关于这方面的知识, 请读者参看文献 [9] 的第 6 章《高次代数方程解法》。

习题

1. 用二分法求方程 $x^2 - x - 1 = 0$ 的正根, 要求误差小于 0.05。
2. 用比例求根法求 $f(x) = 1 - x \sin x = 0$ 在区间 $(0, 1)$ 内的一个根, 直到近似根 x_k 满足精度 $|f(x_k)| < 0.005$ 时终止计算。

3. 为求方程 $x^3 - x^2 - 1 = 0$ 在 $x_0 = 1.5$ 附近的一个根, 设将方程改写成下列等价形式, 并建立相应的迭代公式。

(1) $x = 1 + 1/x^2$, 迭代公式 $x_{k+1} = 1 + 1/x_k^2$;

(2) $x^3 = 1 + x^2$, 迭代公式 $x_{k+1} = \sqrt[3]{1 + x_k^2}$;

(3) $x^2 = \frac{1}{x-1}$, 迭代公式 $x_{k+1} = 1/\sqrt{x_k - 1}$ 。

试分析每种迭代公式的收敛性, 并选取一种公式求出具有四位有效数字的近似根。

4. 比较以下两种求 $e^x + 10x - 2 = 0$ 的根到三位小数所需的计算量:

(1) 在区间 $(0, 1)$ 内用二分法;

(2) 用迭代法 $x_{k+1} = (2 - e^{x_k})/10$, 取初值 $x_0 = 0$ 。

5. 给定函数 $f(x)$, 设对一切 x , $f'(x)$ 存在且 $0 < m \leq f'(x) \leq M$, 证明对于范围 $0 < \lambda < 2/M$ 内的任意定数 λ , 迭代过程 $x_{k+1} = x_k - \lambda f(x_k)$ 均收敛于 $f(x)$ 的根 x^* 。

6. 已知 $x = \varphi(x)$ 在区间 (a, b) 内只有一根, 且当 $a < x < b$ 时, $|\varphi'(x)| \geq k > 1$,

(1) 如何将 $x = \varphi(x)$ 化为适于迭代的形式?

(2) 将 $x = \tan x$ 化为适于迭代的形式, 并求 $x = 4.5$ rad 附近的根。

7. 用下列方法求 $f(x) = x^3 - 3x - 1 = 0$ 在 $x_0 = 2$ 附近的根。根的准确值 $x^* = 1.87938524 \cdots$, 要求计算结果准确到四位有效数字。

(1) 用 Newton 法;

(2) 用弦截法, 取 $x_0 = 2, x_1 = 1.9$;

(3) 用抛物线法, 取 $x_0 = 1, x_1 = 3, x_2 = 2$ 。

校注 (agent 补充): 习题 2 的区间在原页确为 $(0, 1)$, 转录保留。对 $0 < x < 1$ 有 $0 < x \sin x < 1$, 从而 $1 - x \sin x > 0$, 故该区间不含题述方程的根。此为原题条件的疑误; 未替教材指定修正后的区间, 也未代做该题。

原书 PDF 页 176; 印刷页码 163。

[原页](#)

8. 分别用二分法和 Newton 法求 $x - \tan x = 0$ 的最小正根。

9. 研究求 $\sqrt{a}$ 的 Newton 公式 $x_{k+1} = \frac{1}{2} \left(x_k + \frac{a}{x_k} \right)$, $x_0 > 0$, 证明: 对于一切 $k = 1, 2, \dots$, 有 $x_k \geq \sqrt{a}$, 且序列 $x_1, x_2, \dots$ 是单调递减的。

10. 对于 $f(x) = 0$ 的 Newton 公式 $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$, 证明

$$R_k = \frac{x_k - x_{k-1}}{(x_{k-1} - x_{k-2})^2}$$

收敛到 $-\frac{f''(x^*)}{2f'(x^*)}$, 这里 x^* 为 $f(x) = 0$ 的根。

11. 试就下列函数讨论 Newton 法的收敛性和收敛速度:

(1)

$$f(x) = \begin{cases} \sqrt{x}, & x \geq 0, \\ -\sqrt{-x}, & x < 0; \end{cases}$$

(2)

$$f(x) = \begin{cases} \sqrt[3]{x^2}, & x \geq 0, \\ -\sqrt[3]{x^2}, & x < 0. \end{cases}$$

12. 应用 Newton 法于方程 $x^3 - a = 0$, 导出求立方根 $\sqrt[3]{a}$ 的迭代公式, 并讨论其收敛性。

13. 应用 Newton 法于方程 $f(x) = 1 - \frac{a}{x^2} = 0$, 导出求 $\sqrt{a}$ 的迭代公式, 并求 $\sqrt{115}$ 的值。

14. 应用 Newton 法于方程 $f(x) = x^n - a = 0$ 和 $f(x) = 1 - \frac{a}{x^n} = 0$, 分别导出求 $\sqrt[n]{a}$ 的迭代公式, 并求

$$\lim_{k \rightarrow \infty} \frac{\sqrt[n]{a} - x_{k+1}}{(\sqrt[n]{a} - x_k)^2}.$$

15. 证明迭代公式 $x_{k+1} = \frac{x_k(x_k^2 + 3a)}{3x_k^2 + a}$ 是计算 $\sqrt{a}$ 的三阶方法。假定初值 x_0 充分靠近根 x^* , 求

$$\lim_{k \rightarrow \infty} \frac{\sqrt{a} - x_{k+1}}{(\sqrt{a} - x_k)^3}.$$

原书 PDF 页 177；印刷页码 164。

[原页扫描](#)

第 7 章解线性方程组的直接方法

7.1 引言

在自然科学和工程技术中，很多问题的解决常常归结为求解线性代数方程组，例如，电学中的网络问题，船体数学放样中建立三次样条函数问题，用最小二乘法求实验数据的曲线拟合问题，解非线性方程组问题，用差分法或者有限元方法解常微分方程、偏微分方程边值问题等，都导致求解线性代数方程组。而这些方程组的系数矩阵大致分为两种，一种是低阶稠密矩阵（例如，阶数上界大约为 150 的矩阵），另一种是大型稀疏矩阵（即阶数高且零元素较多的矩阵）。

关于线性方程组的数值解法一般有两类。

1. 直接法

直接法就是经过有限步算术运算即可求得方程组精确解的方法（若计算过程中没有舍入误差）。但实际计算中由于舍入误差的存在和影响，这种方法也只能求得线性方程组的近似解。本章将阐述这类算法中最基本的 Gauss 消去法及其某些变形。这类方法是解低阶稠密矩阵方程组的有效方法，近几十年来直接法在求解具有较大型稀疏矩阵方程组方面也取得了较大进展。

2. 迭代法

迭代法就是用某种极限过程去逐步逼近线性方程组精确解的方法。迭代法具有存储单元较少、程序设计简单、原始系数矩阵在计算过程中始终不变等优点，但存在收敛性及收敛速度方面的问题。迭代法是解大型稀疏矩阵方程组（尤其是由微分方程离散后得到的大型方程组）的重要方法（见第 8 章）。

7.2 Gauss 消去法

本节介绍 Gauss 消去法（逐次消去法）以及消去法和矩阵三角分解之间的关系。虽然 Gauss 消去法是一个古老的求解线性方程组的方法（早在公元前 250 年我国就掌握了解三元一次联立方程组的方法），但由它改进、变形得到的主元素消去法、三角分解法仍然是目前计算机上常用的有效方法。

原书 PDF 页 178; 印刷页码 165。

[原页扫描](#)

7.2.1 消元手续

设有线性方程组

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2, \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n, \end{cases} \quad (7.2.1)$$

或写成矩阵形式 $Ax = b$, 其中

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}, \quad x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}, \quad b = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix},$$

A 为非奇异矩阵。下面举一个简单的例子来说明消去法的基本思想。

例 7.1 用消去法解方程组

$$x_1 + x_2 + x_3 = 6, \quad (7.2.2)$$

$$4x_2 - x_3 = 5, \quad (7.2.3)$$

$$2x_1 - 2x_2 + x_3 = 1. \quad (7.2.4)$$

解 第一步, 将式 (7.2.2) 乘以 -2 加到式 (7.2.4) 上去, 消去式 (7.2.4) 中的未知数 x_1 , 得到

$$-4x_2 - x_3 = -11. \quad (7.2.5)$$

第二步, 将式 (7.2.3) 加到式 (7.2.5) 上, 消去式 (7.2.5) 中的未知数 x_2 , 得到与原方程组等价的三角方程组

$$\begin{cases} x_1 + x_2 + x_3 = 6, \\ 4x_2 - x_3 = 5, \\ -2x_3 = -6. \end{cases} \quad (7.2.6)$$

显然方程组 (7.2.6) 是容易求解的, 解为 $x^* = (1, 2, 3)^T$ 。上述过程相当于

$$(A:b) = \left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 4 & -1 & 5 \\ 2 & -2 & 1 & 1 \end{array} \right) \longrightarrow \left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 4 & -1 & 5 \\ 0 & -4 & -1 & -11 \end{array} \right) \longrightarrow \left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 4 & -1 & 5 \\ 0 & 0 & -2 & -6 \end{array} \right).$$

$$(-2) \times r_1^{①} + r_3 \rightarrow r_3, \quad r_2 + r_3 \rightarrow r_3.$$

由此看出，用消去法解方程组的基本思想是，用逐次消去未知数的方法把原来方程组 $Ax = b$ 化为与其等价的三角方程组，而求解三角方程组就容易了。换句话说，上

① r_i 表示矩阵的第 i 行。

原书 PDF 页 179; 印刷页码 166。

[原页扫描](#)

述过程就是用行的初等变换将原方程组系数矩阵化为简单形式, 从而将求解原方程组 (7.2.1) 的问题转化为求解简单方程组的问题。

下面来讨论一般的解 n 阶方程组的 Gauss 消去法。

将式 (7.2.1) 记作 $A^{(1)}x = b^{(1)}$, 其中 $A^{(1)} = (a_{ij}^{(1)}) = (a_{ij})$, $b^{(1)} = b$ 。

(1) 第一次消元。设 $a_{11}^{(1)} \neq 0$, 首先对行计算乘数 $m_{i1} = a_{i1}^{(1)}/a_{11}^{(1)}$ ($i = 2, 3, \dots, n$), 用 $-m_{i1}$ 乘式 (7.2.1) 的第 1 个方程, 加到第 i ($i = 2, 3, \dots, n$) 个方程上, 消去式 (7.2.1) 的第 2 个方程直到第 n 个方程中的未知数 x_1 , 得与式 (7.2.1) 等价的方程组

$$\begin{pmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1n}^{(1)} \\ 0 & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} \\ \vdots & \vdots & & \vdots \\ 0 & a_{n2}^{(2)} & \cdots & a_{nn}^{(2)} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1^{(1)} \\ b_2^{(2)} \\ \vdots \\ b_n^{(2)} \end{pmatrix}, \quad (7.2.7)$$

简记作 $A^{(2)}x = b^{(2)}$, 其中

$$a_{ij}^{(2)} = a_{ij}^{(1)} - m_{i1}a_{1j}^{(1)}, \quad b_i^{(2)} = b_i^{(1)} - m_{i1}b_1^{(1)} \quad (i, j = 2, 3, \dots, n).$$

(2) 一般第 k ($1 \leq k \leq n-1$) 次消元。设第 $k-1$ 步计算已经完成, 即已计算好与式 (7.2.1) 等价的方程组

$$A^{(k)}x = b^{(k)}, \quad (7.2.8)$$

且已消去未知数 $x_1, x_2, \dots, x_{k-1}$, 其中 $A^{(k)}$ 具有如下形式:

$$A^{(k)} = \begin{pmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & \cdots & \cdots & a_{1n}^{(1)} \\ & a_{22}^{(2)} & \cdots & \cdots & \cdots & a_{2n}^{(2)} \\ & & \ddots & & & \vdots \\ & & & a_{kk}^{(k)} & \cdots & a_{kn}^{(k)} \\ & & & \vdots & & \vdots \\ & & & a_{nk}^{(k)} & \cdots & a_{nn}^{(k)} \end{pmatrix}.$$

设 $a_{kk}^{(k)} \neq 0$, 计算乘数 $m_{ik} = a_{ik}^{(k)}/a_{kk}^{(k)}$ ($i = k+1, \dots, n$), 用 $-m_{ik}$ 乘式 (7.2.8) 的第 k 个方程加上第 i ($i = k+1, \dots, n$) 个方程, 消去第 $k+1$ 个方程直到第 n 个方程的未知数 x_k , 得到与式 (7.2.1) 等价的方程组 $A^{(k+1)}x = b^{(k+1)}$ 。

$A^{(k+1)}$ 元素的计算公式为

$$\begin{cases} a_{ij}^{(k+1)} = a_{ij}^{(k)} - m_{ik}a_{kj}^{(k)} & (i, j = k+1, \dots, n), \\ b_i^{(k+1)} = b_i^{(k)} - m_{ik}b_k^{(k)} & (i = k+1, \dots, n). \end{cases} \quad (7.2.9)$$

显然 $A^{(k+1)}$ 的第 1 行直到第 k 行与 $A^{(k)}$ 相同。

(3) 继续这一过程, 直到完成第 $n-1$ 次消元。最后得到与原方程组等价的三角方程组

$$A^{(n)}x = b^{(n)} \quad (7.2.10)$$

原书 PDF 页 180; 印刷页码 167。

[原页扫描](#)

或

$$\begin{pmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1n}^{(1)} \\ & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} \\ & & \ddots & \vdots \\ & & & a_{nn}^{(n)} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1^{(1)} \\ b_2^{(2)} \\ \vdots \\ b_n^{(n)} \end{pmatrix}.$$

由式 (7.2.1) 约化为式 (7.2.10) 的过程称为**消元过程**。

求解三角方程组 (7.2.10), 设 $a_{ii}^{(i)} \neq 0$ ($i = 1, 2, \dots, n-1$), 易得求解公式

$$\begin{cases} x_n = b_n^{(n)} / a_{nn}^{(n)}, \\ x_k = \left(b_k^{(k)} - \sum_{j=k+1}^n a_{kj}^{(k)} x_j \right) / a_{kk}^{(k)} \end{cases} \quad (k = n-1, n-2, \dots, 2, 1). \quad (7.2.11)$$

式 (7.2.10) 的求解过程称为**回代过程**。

如果 $a_{11}^{(1)} = 0$, 那么, 由于 A 为非奇异矩阵, 所以 A 的第 1 列一定有元素不等于零, 例如 $a_{i_1,1} \neq 0$, 于是可交换两行元素 ($r_1 \leftrightarrow r_{i_1}$), 将 $a_{i_1,1}$ 调到第 1 行第 1 列的位置, 然后进行消元计算, 这时 $A^{(2)}$ 右下角矩阵 ($n-1$ 阶) 亦为非奇异矩阵。继续这一过程, Gauss 消去法照样可进行计算。

总结上述讨论即有如下定理。

定理 7.1 如果 A 为 n 阶非奇异矩阵, 则可通过 Gauss 消去法 (及交换两行的初等变换) 将方程组 (7.2.1) 化为三角方程组 (7.2.10)。

A 在什么条件下才能保证 $a_{kk}^{(k)} \neq 0$ ($k = 1, 2, \dots, n$)? 下面的引理给出了这个条件。

引理 约化的主元素 $a_{ii}^{(i)} \neq 0$ ($i = 1, 2, \dots, k$) 的充要条件是矩阵 A 的顺序主子式 $D_i \neq 0$ ($i = 1, 2, \dots, k$), 即

$$D_1 = a_{11} \neq 0,$$

$$D_i = \begin{vmatrix} a_{11} & \cdots & a_{1i} \\ \vdots & & \vdots \\ a_{i1} & \cdots & a_{ii} \end{vmatrix} \neq 0 \quad (i = 2, 3, \dots, k). \quad (7.2.12)$$

证明 利用归纳法证明引理的充分性。显然, 当 $k = 1$ 时引理的充分性是成立的, 现假设引理对 $k-1$ 是成立的, 求证引理对 k 亦成立。由归纳法, 设 $a_{ii}^{(i)} \neq 0$ ($i = 1, 2, \dots, k-1$), 于是可用 Gauss 消去法将 $A^{(1)} = A$ 约化到 $A^{(k)}$ 中, 即

$$A^{(1)} \rightarrow A^{(k)} = \begin{pmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & \cdots & \cdots & a_{1n}^{(1)} \\ & a_{22}^{(2)} & \cdots & \cdots & \cdots & a_{2n}^{(2)} \\ & & \ddots & & & \vdots \\ & & & a_{kk}^{(k)} & \cdots & a_{kn}^{(k)} \\ & & & \vdots & & \vdots \\ & & & a_{nk}^{(k)} & \cdots & a_{nn}^{(k)} \end{pmatrix},$$

校注 (agent 补充): 式 (7.2.12) 原书矩阵右上角清楚印作 a_{1k} , 此处忠实保留。按同式左端 D_i 、左下角 a_{i1} 和右下角 a_{ii} , 第 i 阶顺序主子式的右上角应为 a_{1i} ; 这是疑似原书下标排印错误, 不是 OCR 改写。参见[原式放大裁图](#)。

原书 PDF 页 181; 印刷页码 168。

[原页扫描](#)

且有

$$D_2 = \begin{vmatrix} a_{11}^{(1)} & a_{12}^{(1)} \\ 0 & a_{22}^{(2)} \end{vmatrix} = a_{11}^{(1)} a_{22}^{(2)}, \quad D_3 = a_{11}^{(1)} a_{22}^{(2)} a_{33}^{(3)},$$

$$D_k = \begin{vmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1k}^{(1)} \\ & a_{22}^{(2)} & \cdots & a_{2k}^{(2)} \\ & & \ddots & \vdots \\ & & & a_{kk}^{(k)} \end{vmatrix} = a_{11}^{(1)} a_{22}^{(2)} \cdots a_{kk}^{(k)}. \quad (7.2.13)$$

由设 $D_i \neq 0$ ($i = 1, 2, \dots, k$) 及式 (7.2.13), 有 $a_{kk}^{(k)} \neq 0$, 即引理的充分性对 k 成立。

显然, 由假设 $a_{ii}^{(i)} \neq 0$ ($i = 1, 2, \dots, k$), 利用式 (7.2.13) 亦可推出 $D_i \neq 0$ ($i = 1, 2, \dots, k$)。

推论 如果 A 的顺序主子式 $D_k \neq 0$ ($k = 1, 2, \dots, n-1$), 则

$$\begin{cases} a_{11}^{(1)} = D_1, \\ a_{kk}^{(k)} = D_k / D_{k-1} \quad (k = 2, 3, \dots, n). \end{cases}$$

定理 7.2 如果 n 阶矩阵 A 的所有顺序主子式均不为零, 即 $D_i \neq 0$ ($i = 1, 2, \dots, n$), 则可通过 Gauss 消去法 (不进行交换两行的初等变换), 将方程组 (7.2.1) 约化为三角方程组 (7.2.10)。

计算公式如下:

(1) 消元计算 ($k = 1, 2, \dots, n-1$)。

$$m_{ik} = a_{ik}^{(k)} / a_{kk}^{(k)} \quad (i = k+1, \dots, n),$$

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - m_{ik} a_{kj}^{(k)} \quad (i, j = k+1, \dots, n),$$

$$b_i^{(k+1)} = b_i^{(k)} - m_{ik} b_k^{(k)} \quad (i = k+1, \dots, n).$$

(2) 回代计算。求解公式为式 (7.2.11)。

7.2.2 矩阵的三角分解

下面借助矩阵理论进一步对消去法作些分析, 从而建立 Gauss 消去法与矩阵因式分解的关系。

设式 (7.2.1) 中 A 的各顺序主子式均不为零。由于对 A 施行行的初等变换相当于用初等矩阵左乘 A , 于是对式 (7.2.1) 施行第一次消元后化为式 (7.2.7), 这时 $A^{(1)}$ 化为 $A^{(2)}$, $b^{(1)}$ 化为 $b^{(2)}$, 即

$$L_1A^{(1)} = A^{(2)}, \quad L_1b^{(1)} = b^{(2)},$$

其中

$$L_1 = \begin{pmatrix} 1 & & & \\ -m_{21} & 1 & & \\ -m_{31} & & 1 & \\ \vdots & & & \ddots \\ -m_{n1} & & & & 1 \end{pmatrix}.$$

原书 PDF 页 182; 印刷页码 169。

[原页扫描](#)

一般第 k 步消元, $A^{(k)}$ 化为 $A^{(k+1)}$, $b^{(k)}$ 化为 $b^{(k+1)}$, 相当于

$$L_k A^{(k)} = A^{(k+1)}, \quad L_k b^{(k)} = b^{(k+1)}.$$

重复这一过程, 最后得到

$$\begin{cases} L_{n-1} \cdots L_2 L_1 A^{(1)} = A^{(n)}, \\ L_{n-1} \cdots L_2 L_1 b^{(1)} = b^{(n)}, \end{cases} \quad (7.2.14)$$

其中

$$L_k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -m_{k+1,k} & 1 & \\ & & \vdots & & \ddots \\ & & -m_{nk} & & & 1 \end{pmatrix}.$$

将上三角矩阵 $A^{(n)}$ 记作 U , 由式 (7.2.14) 得到

$$A = L_1^{-1} L_2^{-1} \cdots L_{n-1}^{-1} U = LU,$$

其中

$$L = L_1^{-1} L_2^{-1} \cdots L_{n-1}^{-1} = \begin{pmatrix} 1 & & & & \\ m_{21} & 1 & & & \\ m_{31} & m_{32} & 1 & & \\ \vdots & \vdots & \ddots & \ddots & \\ m_{n1} & m_{n2} & \cdots & m_{n,n-1} & 1 \end{pmatrix}.$$

为单位下三角矩阵。

这就是说, Gauss 消去法实质上产生了一个将 A 分解为两个三角矩阵相乘的因式分解, 于是得到如下重要定理, 它在解方程组的直接法中起着重要作用。

定理 7.3 (矩阵的 LU 分解) 设 A 为 n 阶矩阵, 如果 A 的顺序主子式 $D_i \neq 0$ ($i = 1, 2, \dots, n-1$), 则 A 可分解为一个单位下三角矩阵 L 和一个上三角矩阵 U 的乘积, 且这种分解是唯一的。

证明 根据以上 Gauss 消去法的矩阵分析, $A = LU$ 的存在性已经得到证明, 现仅在 A 为非奇异矩阵的假定下来证明它的唯一性, 当 A 为奇异矩阵的情况留作课外练习。

设

$$A = LU = L_1 U_1,$$

其中 L, L_1 为单位下三角矩阵; U, U_1 为上三角矩阵。

由于 U_1^{-1} 存在, 故

$$L^{-1} L_1 = U U_1^{-1}.$$

上式右端为上三角矩阵, 左端为单位下三角矩阵, 从而上式两端都必须等于单位矩阵, 故 $U = U_1$, $L = L_1$ 。证毕。

例 7.2 对于例 7.1, 系数矩阵 $A = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 4 & -1 \\ 2 & -2 & 1 \end{pmatrix}$, 由 Gauss 消去法, 有

原书 PDF 页 183; 印刷页码 170。

[原页扫描](#)

$$m_{21} = 0, \quad m_{31} = 2, \quad m_{32} = -1,$$

故

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 2 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 \\ 0 & 4 & -1 \\ 0 & 0 & -2 \end{pmatrix} = LU.$$

7.2.3 计算量

下面考虑解式 (7.2.1) 的 Gauss 消去法的计算量。

(1) 消元过程的计算量。第一步计算乘数 m_{i1} ($i = 2, 3, \dots, n$) 需要 $n-1$ 次除法运算, 计算 $a_{ij}^{(2)}$ ($i, j = 2, 3, \dots, n$) 需要 $(n-1)^2$ 次乘法运算及 $(n-1)^2$ 次加、减法运算。一般可列表 (见表 7.1) 计算。

表 7.1

第 k 步	加、减法次数	乘法次数	除法次数
1	$(n-1)^2$	$(n-1)^2$	$n-1$
2	$(n-2)^2$	$(n-2)^2$	$n-2$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$n-1$	1	1	1
合计	$\frac{n(n-1)(2n-1)}{6}$	$\frac{n(n-1)(2n-1)}{6}$	$\frac{n(n-1)}{2}$

这里利用了求和公式

$$\sum_{i=1}^n i = n(n+1)/2, \quad \sum_{i=1}^n i^2 = n(n+1)(2n+1)/6 \quad (n \geq 1).$$

消元过程所需的乘、除法次数 MD 及加、减法次数 AS 分别为

$$MD = n(n^2 - 1)/3, \quad AS = n(n-1)(2n-1)/6.$$

(2) 计算 $b^{(n)}$ 的计算量。

$$MD = (n-1) + (n-2) + \dots + 2 + 1 = n(n-1)/2, \quad AS = n(n-1)/2.$$

(3) 解 $A^{(n)}x = b^{(n)}$ 所需的计算量。 $MD = n(n+1)/2$, $AS = n(n-1)/2$ 。解式 (7.2.1) 所需的总的乘除法次数及加减法次数分别为

$$MD = n^3/3 + n^2 - n/3 \approx n^3/3 \quad (\text{当 } n \text{ 比较大时}),$$

$$AS = n(n-1)(2n+5)/6 \approx n^3/3 \quad (\text{当 } n \text{ 比较大时}).$$

定理 7.4 如果 A 为 n 阶非奇异矩阵, 则用 Gauss 消去法解式 (7.2.1) 所需乘除法次数及加减法次数分别为

$$1^\circ MD = n^3/3 + n^2 - n/3;$$

$$2^\circ AS = n(n-1)(2n+5)/6.$$

如果用 Cramer 法则解式 (7.2.1), 就需要计算 $n+1$ 个 n 阶行列式, 若行列式计

原书 PDF 页 184; 印刷页码 171。

[原页扫描](#)

算是用子式展开, 总共需要 $(n+1)!$ 次乘法运算。例如 $n=10$ 时, Gauss 消去法需要 430 次乘、除法运算, 而 Cramer 法则则需要 39 916 800 次乘法运算。由此可见, 用 Cramer 法则解式 (7.2.1) 工作量太大, 不便使用。

7.3 Gauss 主元素消去法

由 Gauss 消去法知道, 在消元过程中可能出现 $a_{kk}^{(k)} = 0$ 的情况, 这时消去法将无法进行; 即使在主元素 $a_{kk}^{(k)} \neq 0$ 但很小时, 用其作除数, 也会导致其他元素数量级的严重增长和舍入误差的扩散, 最后会使得计算解不可靠。

例 7.3 求解方程组

$$\begin{pmatrix} 0.001 & 2.000 & 3.000 \\ -1.000 & 3.712 & 4.623 \\ -2.000 & 1.072 & 5.643 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1.000 \\ 2.000 \\ 3.000 \end{pmatrix}.$$

用四位浮点数进行计算。精确解舍入到四位有效数字为

$$x^* = (-0.490\,4, -0.051\,04, 0.367\,5)^T.$$

解 [方法 1] 用 Gauss 消去法求解。

$$(A, b) = \left(\begin{array}{ccc|c} 0.001 & 2.000 & 3.000 & 1.000 \\ -1.000 & 3.712 & 4.623 & 2.000 \\ -2.000 & 1.072 & 5.643 & 3.000 \end{array} \right)$$

$$m_{21} = -1.000/0.001 = -1000, \quad m_{31} = -2.000/0.001 = -2000$$

$$\rightarrow \left(\begin{array}{ccc|c} 0.001 & 2.000 & 3.000 & 1.000 \\ 0 & 2004 & 3005 & 1002 \\ 0 & 4001 & 6006 & 2003 \end{array} \right), \quad m_{32} = 4001/2004 = 1.997$$

$$\rightarrow \left(\begin{array}{ccc|c} 0.001 & 2.000 & 3.000 & 1.000 \\ 0 & 2004 & 3005 & 1002 \\ 0 & 0 & 5.000 & 2.000 \end{array} \right),$$

计算解为

$$\bar{x} = (-0.400\,0, -0.099\,80, 0.400\,0)^T.$$

显然, 计算解 $\bar{x}$ 是一个很坏的结果, 不能作为方程组的近似解。其原因是在消元计算时用了小主元 0.001, 使得约化后的方程组元素数量级大大增长, 经再舍入使得在计算第 3 行第 3 列的元素时发生了严重的相消情况 (第 3 行第 3 列的元素舍入到第四位数字的正确值是 5.922), 因此经消元后得到的三角方程组就不准确了。

[方法 2] 交换行，避免绝对值小的主元作除数。

$$(A, b) \xrightarrow{r_1 \leftrightarrow r_3} \left(\begin{array}{ccc|c} -2.000 & 1.072 & 5.643 & 3.000 \\ -1.000 & 3.712 & 4.623 & 2.000 \\ 0.001 & 2.000 & 3.000 & 1.000 \end{array} \right)$$
$$m_{21} = 0.500\,0, \qquad m_{31} = -0.000\,5$$

原书 PDF 页 185; 印刷页码 172。

[原页扫描](#)

$$\rightarrow \left(\begin{array}{ccc|c} -2.000 & 1.072 & 5.643 & 3.000 \\ 0 & 3.176 & 1.801 & 0.5000 \\ 0 & 2.001 & 3.003 & 1.002 \end{array} \right), \quad m_{32} = 0.6300$$

$$\rightarrow \left(\begin{array}{ccc|c} -2.000 & 1.072 & 5.643 & 3.000 \\ 0 & 3.176 & 1.801 & 0.5000 \\ 0 & 0 & 1.868 & 0.6870 \end{array} \right),$$

得计算解为

$$x = (-0.4900, -0.05113, 0.3678)^T \approx x^*.$$

这个例子告诉我们, 在采用 Gauss 消去法解方程组时, 小主元可能产生麻烦, 故应避免采用绝对值小的主元素 $a_{kk}^{(k)}$ 。对一般矩阵来说, 最好每一步都选取系数矩阵 (或消元后的低阶矩阵) 中绝对值最大的元素作为主元素, 以使 Gauss 消去法具有较好的数值稳定性。

下面介绍主元素消去法。本节总假定方程组 (7.2.1) 的 A 非奇异。

7.3.1 完全主元素消去法

设方程组 (7.2.1) 的增广矩阵为

$$B = \left(\begin{array}{cccccc|c} a_{11} & a_{12} & \cdots & a_{1j_1} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2j_1} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & & \vdots & & \vdots & \vdots \\ a_{i_1 1} & a_{i_1 2} & \cdots & a_{i_1 j_1} & \cdots & a_{i_1 n} & b_{i_1} \\ \vdots & \vdots & & \vdots & & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nj_1} & \cdots & a_{nn} & b_n \end{array} \right).$$

首先在 A 中选取绝对值最大的元素作为主元素, 例如 $|a_{i_1 j_1}| = \max_{\substack{1 \leq i \leq n \\ 1 \leq j \leq n}} |a_{ij}| \neq 0$, 然后交换 B 的第 1 行与第 i_1 行, 第 1 列与第 j_1 列, 经第一次消元计算得

$$(A, b) \rightarrow (A^{(2)}, b^{(2)}).$$

重复上述过程, 设已完成第 $k-1$ 步的选主元素, 交换两行及交换两列, 消元计算, (A, b) 约化为

$$(A^{(k)}, b^{(k)}) = \left[\begin{array}{cccccc|c} a_{11} & a_{12} & \cdots & \cdots & \cdots & a_{1n} & b_1 \\ & a_{22} & \cdots & \cdots & \cdots & a_{2n} & b_2 \\ & & \ddots & & & \vdots & \vdots \\ & & & a_{kk} & \cdots & a_{kn} & b_k \\ & & & \vdots & & \vdots & \vdots \\ & & & a_{nk} & \cdots & a_{nn} & b_n \end{array} \right],$$

其中 $A^{(k)}$ 元素仍记作 a_{ij} , $b^{(k)}$ 元素仍记作 b_i ($k = 1, 2, \cdots, n - 1$)。

原书 PDF 页 186; 印刷页码 173。

[原页扫描](#)

第 k 步选主元素 (在 $A^{(k)}$ 右下角方框内选), 即确定 i_k, j_k 使

$$|a_{i_k j_k}| = \max_{\substack{k \leq i \leq n \\ k \leq j \leq n}} |a_{ij}| \neq 0.$$

交换 $(A^{(k)}, b^{(k)})$ 第 k 行与 i_k 行元素, 交换 $(A^{(k)})$ 第 k 列与 j_k 列元素, 将 $a_{i_k j_k}$ 调到 (k, k) 位置, 再进行消元计算, 最后将原方程组化为

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ & a_{22} & \cdots & a_{2n} \\ & & \ddots & \vdots \\ & & & a_{nn} \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix},$$

其中 $y_1, y_2, \dots, y_n$ 的次序为未知数 $x_1, x_2, \dots, x_n$ 调换后的次序。回代求解得

$$\begin{cases} y_n = b_n / a_{nn}, \\ y_i = \left(b_i - \sum_{j=i+1}^n a_{ij} y_j \right) / a_{ii} \quad (i = n-1, \dots, 2, 1). \end{cases}$$

算法 1 完全主元素消去法, 其步骤如下:

设 $Ax = b$ 。本算法用 A 的带有行、列交换的 Gauss 消去法①, 消元结果冲掉 A , 乘数 m_{ij} 冲掉 a_{ij} , 计算解 x 冲掉常数项 b , 用 k 表示对 A 的消元次数。用一整型数组 $\text{Iz}(n)$ 开始记录未知数 $x_1, x_2, \dots, x_n$ 的次序 (即下标 $1, 2, \dots, n$), 最后记录调换后未知数的下标。

步 1 对于 $i = 1, 2, \dots, n$, 有 $\text{Iz}(i) \leftarrow i$; 对于 $k = 1, 2, \dots, n-1$, 做到步 6。

步 2 选主元素 $|a_{i_k j_k}| = \max_{\substack{k \leq i \leq n \\ k \leq j \leq n}} |a_{ij}|$ 。

步 3 如果 $a_{i_k j_k} = 0$, 则计算停止 (这时 $\det A = 0$)。

步 4 (1) 如果 $i_k = k$, 则转 (2), 否则换行: $a_{kj} \leftrightarrow a_{i_k j} \quad (j = k, k+1, \dots, n)$, $b_k \leftrightarrow b_{i_k}$; (2) 如果 $j_k = k$, 则转步 5, 否则换列: $a_{ik} \leftrightarrow a_{i j_k} \quad (i = 1, 2, \dots, n)$, $\text{Iz}(k) \leftrightarrow \text{Iz}(j_k)$ 。

步 5 计算乘数

$$a_{ik} \leftarrow m_{ik} = a_{ik} / a_{kk} \quad (i = k+1, \dots, n).$$

步 6 消元计算

$$a_{ij} \leftarrow a_{ij} - m_{ik} a_{kj} \quad (i = k+1, \dots, n; j = k+1, \dots, n);$$

$$b_i \leftarrow b_i - m_{ik} b_k \quad (i = k+1, \dots, n).$$

步 7 回代求解

(1) $b_n \leftarrow b_n/a_{n,n}$; (2) 对于 $i = n-1, n-2, \dots, 2, 1$, $b_i \leftarrow (b_i - \sum_{j=i+1}^n a_{ij}b_j)/a_{ii}$ 。

步 8 调整未知数的次序

(1) 对于 $i = 1, 2, \dots, n$; $a_i, \text{Iz}(i) \leftarrow b_i$; (2) 对于 $i = 1, 2, \dots, n$; $b_i \leftarrow a_{1i}$ 。

7.3.2 列主元素消去法

完全主元素消去法在选主元素时要花费较多机器时间。下面介绍另一种常用的

①在实际计算中可以考虑设计不进行行、列交换的算法。

校注 (agent 补充): 算法 1 步 8 (1) 原书印作 $a_i, \text{Iz}(i) \leftarrow b_i$ (逗号及 Iz 为基线排印), 本转录保留。按该步 (2) $b_i \leftarrow a_{1i}$ 及“调整未知数的次序”, (1) 应把 b_i 暂存到第一行第 $\text{Iz}(i)$ 列, 即 $a_{1, \text{Iz}(i)} \leftarrow b_i$; 疑似原书下标排印错误。参见[原步骤放大截图](#)。

原书 PDF 页 187; 印刷页码 174。

[原页扫描](#)

方法即列主元素消去法。它仅考虑依次按列选主元素, 然后换行使之变到主元位置上, 再进行消元计算。设用列主元素消去法解 $Ax = b$ 已完成 $k-1$ 步计算, 即有

$$(A, b) \rightarrow (A^{(k)}, b^{(k)}) = \left(\begin{array}{cccc|c} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & \cdots & a_{1n}^{(1)} & b_1^{(1)} \\ & a_{22}^{(2)} & \cdots & \cdots & a_{2n}^{(2)} & b_2^{(2)} \\ & & \ddots & & \vdots & \vdots \\ & & & a_{kk}^{(k)} & \cdots & a_{kn}^{(k)} & b_k^{(k)} \\ & & & \vdots & & \vdots \\ & & & a_{nk}^{(k)} & \cdots & a_{nn}^{(k)} & b_n^{(n)} \end{array} \right),$$

且 $A^{(k)}x = b^{(k)}$ 与 $Ax = b$ 等价, 第 k 步选主元素 (在 $A^{(k)}$ 第 k 列方框内选), 即确定 i_k 使

$$|a_{i_k, k}^{(k)}| = \max_{k \leq i \leq n} |a_{ik}^{(k)}|.$$

算法 2 列主元素消去法, 其步骤如下:

设 $Ax = b$ 。本算法用 A 的具有行交换的列主元素消去法①, 消元结果冲掉 A , 乘数 m_{ij} 冲掉 a_{ij} , 计算解 x 冲掉常数项 b , 行列式存放在 $\det A$ 。

步 1 $\det A \leftarrow 1$, 对于 $k = 1, 2, \dots, n-1$ 做到步 7。

步 2 按列选主元素 $|a_{i_k, k}| = \max_{k \leq i \leq n} |a_{ik}|$ 。

步 3 如果 $a_{i_k, k} = 0$, 则 $\det A \leftarrow 0$, 计算停止。

步 4 如果 $i_k = k$, 则转步 5, 否则换行:

$$a_{kj} \leftrightarrow a_{i_k, j} \quad (j = k, k+1, \dots, n), \quad b_k \leftrightarrow b_{i_k}, \quad \det A \leftarrow -\det A.$$

步 5 计算乘数 m_{ik}

$$a_{ik} \leftarrow m_{ik} = a_{ik}/a_{kk} \quad (i = k+1, \dots, n) \quad (|m_{ik}| \leq 1).$$

步 6 消元计算

$$a_{ij} \leftarrow a_{ij} - m_{ik}a_{kj} \quad (i, j = k+1, \dots, n), \quad b_i \leftarrow b_i - m_{ik}b_k \quad (i = k+1, \dots, n).$$

步 7 $\det A \leftarrow a_{kk} \det A$ 。

步 8 回代求解

$$b_n \leftarrow b_n/a_{nn}, \quad b_i \leftarrow \left(b_i - \sum_{j=i+1}^n a_{ij}b_j \right) / a_{ii} \quad (i = n-1, n-2, \dots, 1).$$

步 9 $\det A \leftarrow a_{nn} \det A$ 。

例 7.3 的方法 2 用的就是列主元素消去法。

下面用矩阵运算来描述解式 (7.2.1) 的列主元素消去法:

$$\begin{cases} L_1 I_{1i_1} A^{(1)} = A^{(2)}, & L_1 I_{1i_1} b^{(1)} = b^{(2)}, \\ L_k I_{ki_k} A^{(k)} = A^{(k+1)}, & L_k I_{ki_k} b^{(k)} = b^{(k+1)}, \end{cases} \quad (7.3.1)$$

其中 L_k 的元素满足 $|m_{ik}| \leq 1$ ($k = 1, 2, \dots, n-1$), I_{ki_k} 是初等排列矩阵 (由交换单位矩阵 I 的第 k 行与第 i_k 行得到)。

①在列主元素消去法中, 可考虑用一整型数组 $\text{Ip}(n)$ 来记录主行。

校注 (agent 补充): 本页首个增广矩阵右下角原书印作 $b_n^{(n)}$, 已保留。此处描述第 $k-1$ 步完成后的 $(A^{(k)}, b^{(k)})$, 同列第 k 行是 $b_k^{(k)}$, 所以末行相应应为 $b_n^{(k)}$; 疑似原书上标排印错误。参见[原矩阵放大裁图](#)。

原书 PDF 页 188; 印刷页码 175。

[原页扫描](#)

利用式 (7.3.1) 得到

$$L_{n-1}I_{n-1,i_{n-1}} \cdots L_2I_{2i_2}L_1I_{1i_1}A = A^{(n)} = U,$$

简记作

$$\tilde{P}A = U, \quad \tilde{P}b = b^{(n)},$$

其中

$$\tilde{P} = L_{n-1}I_{n-1,i_{n-1}} \cdots L_2I_{2i_2}L_1I_{1i_1}.$$

下面就 $n = 4$ 的情况来考察一下矩阵 $\tilde{P}$ 。

$$\begin{aligned} U = A^{(4)} &= L_3I_{3i_3}L_2I_{2i_2}L_1I_{1i_1}A \\ &= L_3(I_{3i_3}L_2I_{3i_3})(I_{3i_3}L_{2i_2}L_1I_{2i_2}I_{3i_3})(I_{3i_3}I_{2i_2}I_{1i_1})A \\ &\equiv \tilde{L}_3\tilde{L}_2\tilde{L}_1PA, \end{aligned} \tag{7.3.2}$$

其中

$$\tilde{L}_1 = I_{3i_3}I_{2i_2}L_1I_{2i_2}I_{3i_3}, \quad \tilde{L}_2 = I_{3i_3}L_2I_{3i_3}, \quad \tilde{L}_3 = L_3, \quad P = I_{3i_3}I_{2i_2}I_{1i_1}.$$

由本章的习题 8 知 $\tilde{L}_k$ ($k = 1, 2, 3$) 亦为单位下三角阵, 其元素的绝对值不大于 1。记 $L^{-1} = \tilde{L}_3\tilde{L}_2\tilde{L}_1$, 由式 (7.3.2) 得到 $PA = LU$, 其中 P 为排列矩阵, L 为单位下三角阵, U 为上三角阵。这说明对式 (7.2.1) 应用列主元素消去法, 相当于对 $(A | b)$ 先进行一系列行交换后再对 $PAx = Pb$ 应用 Gauss 消去法。在实际计算中只能在计算过程中进行行的交换。

总结以上的讨论可得如下定理。

定理 7.5 (列主元素的三角分解定理) 如果 A 为非奇异矩阵, 则存在排列矩阵 P , 使

$$PA = LU,$$

其中 L 为单位下三角阵, U 为上三角阵。

L 元素存放在数组 A 的下三角部分, U 元素存放在 A 上三角部分, 由整型数组 $\text{Ip}(n)$ 记录可知 P 的情况。

7.3.3 Gauss-Jordan 消去法

Gauss 消去法始终是消去对角线下方的元素, 现考虑 Gauss 消去法的一种修正, 即消去对角线下方和上方的元素, 这种方法称为 Gauss-Jordan 消去法。

设用 Gauss-Jordan 消去法已完成 $(k-1)$ 步, 于是 $Ax = b$ 化为等价方程组 $A^{(k)}x = b^{(k)}$, 其中

$$(A^{(k)}, b^{(k)}) = \left(\begin{array}{cccc|c} 1 & & & & b_1 \\ & 1 & & & \vdots \\ & & \ddots & & \vdots \\ & & & 1 & \vdots \\ & & & & a_{k-1,k} & \cdots & a_{k-1,n} & \vdots \\ & & & & a_{kk} & \cdots & a_{kn} & b_k \\ & & & & \vdots & & \vdots & \vdots \\ & & & & a_{nk} & \cdots & a_{nn} & b_n \end{array} \right), \quad k = 1, 2, \dots, n.$$

校注 (agent 补充): 式 (7.3.2) 第二行第二个括号内原书印作 $I_{3i_3} L_{2i_2} L_1 I_{2i_2} I_{3i_3}$, 其中 L_{2i_2} 在本段未定义。本页紧接着定义 $\tilde{L}_1 = I_{3i_3} I_{2i_2} L_1 I_{2i_2} I_{3i_3}$, 因而该处 L_{2i_2} 疑为 I_{2i_2} 的排印错误; 此处保留原式。参见[原式放大截图](#)。

原书 PDF 页 189; 印刷页码 176。

[原页扫描](#)

在第 k 步计算时, 考虑对上述矩阵的第 k 行上、下都进行消元计算。

步 1 按列选主元素, 即确定 i_k 使 $|a_{i_k k}| = \max_{k \leq i \leq n} |a_{ik}|$ 。

步 2 换行 (当 $i_k \neq k$) 交换 (A, b) 第 k 行与第 i_k 行元素。

步 3 计算乘数

$$m_{ik} = -a_{ik}/a_{kk} \quad (i = 1, 2, \dots, n; i \neq k), \quad m_{kk} = 1/a_{kk}.$$

(m_{ik} 可保存在存放 a_{ik} 的单元中。)

步 4 消元计算

$$a_{ij} \leftarrow a_{ij} + m_{ik}a_{kj} \quad \left(\begin{array}{l} i = 1, 2, \dots, n; i \neq k; \\ j = k+1, \dots, n \end{array} \right),$$

$$b_i \leftarrow b_i + m_{ik}b_k \quad (i = 1, 2, \dots, n; i \neq k).$$

步 5 计算主行 $a_{kj} \leftarrow a_{kj} \cdot m_{kk}$ ($j = k, k+1, \dots, n$), $b_k \leftarrow b_k \cdot m_{kk}$ 。

上述过程结束后, 有

$$(A, b) \rightarrow (A^{(k+1)}, b^{(k+1)}) = \left(\begin{array}{cccc|c} 1 & & & & \hat{b}_1 \\ & 1 & & & \hat{b}_2 \\ & & \ddots & & \vdots \\ & & & 1 & \hat{b}_n \end{array} \right)$$

这说明用 Gauss-Jordan 消去法将 A 约化为单位矩阵, 计算解就在常数项位置得到, 因此用不着回代求解。用 Gauss-Jordan 消去法解方程组的计算量大约需要 $n^3/2$ 次乘除法运算, 比 Gauss 消去法计算量大, 但用 Gauss-Jordan 消去法求一个矩阵的逆矩阵还是比较合适的。

定理 7.6 (Gauss-Jordan 消去法求逆矩阵) 设 A 为非奇异矩阵, 方程组 $AX = I_n$ 的增广矩阵为 $C = (A: I_n)$ 。如果对 C 应用 Gauss-Jordan 消去法化为 $(I_n: T)$, 则 $A^{-1} = T$ 。

事实上, 求 A 的逆矩阵 A^{-1} , 即求 n 阶矩阵 X , 使 $AX = I_n$, 其中 I_n 为单位矩阵。将 X 按列分块 $X = (x_1, x_2, \dots, x_n)$, $I = (e_1, e_2, \dots, e_n)$, 于是求解 $AX = I_n$ 等价于求解 n 个方程组 $Ax_j = e_j$ ($j = 1, 2, \dots, n$)。我们可用 Gauss-Jordan 消去法求解 $AX = I_n$ 。

例 7.4 用 Gauss-Jordan 消去法求 $A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 5 \\ 3 & 5 & 6 \end{pmatrix}$ 的逆矩阵 A^{-1} 。

解

$$C = \left(\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 2 & 4 & 5 & 0 & 1 & 0 \\ 3 & 5 & 6 & 0 & 0 & 1 \end{array} \right) \xrightarrow{r_1 \leftrightarrow r_3} \left(\begin{array}{ccc|ccc} \boxed{3} & 5 & 6 & 0 & 0 & 1 \\ 2 & 4 & 5 & 0 & 1 & 0 \\ 1 & 2 & 3 & 1 & 0 & 0 \end{array} \right)$$

$$\xrightarrow{\text{第一次消元}} \left(\begin{array}{ccc|ccc} 1 & 5/3 & 2 & 0 & 0 & 1/3 \\ 0 & 2/3 & 1 & 0 & 1 & -2/3 \\ 0 & 1/3 & 1 & 1 & 0 & -1/3 \end{array} \right)$$

原图标注：上述最后一列 $\begin{pmatrix} 1/3 \\ -2/3 \\ -1/3 \end{pmatrix}$ 以方框标出，标为 c_3 。

原书 PDF 页 190; 印刷页码 177。

[原页扫描](#)

$$\xrightarrow{\text{第二次消元}} \left(\begin{array}{ccc|ccc} 1 & 0 & -1/2 & 0 & -5/2 & 2 \\ 0 & 1 & 3/2 & 0 & 3/2 & -1 \\ 0 & 0 & \boxed{1/2} & 1 & -1/2 & 0 \end{array} \right)$$

原图标注: 上述右侧第二列 $\begin{pmatrix} -5/2 \\ 3/2 \\ -1/2 \end{pmatrix}$ 以方框标出, 标为 c_2 。

$$\xrightarrow{\text{第三次消元}} \left(\begin{array}{ccc|ccc} 1 & 0 & 0 & 1 & -3 & 2 \\ 0 & 1 & 0 & -3 & 3 & -1 \\ 0 & 0 & 1 & 2 & -1 & 0 \end{array} \right) = (I_n | A^{-1}).$$

原图标注: 上述右侧第一列 $\begin{pmatrix} 1 \\ -3 \\ 2 \end{pmatrix}$ 以方框标出, 标为 c_1 。

小方框内为每次按列所选的主元素, 且

$$m_1 = (m_{11}, m_{21}, m_{31})^T = c_3, \quad m_2 = (m_{12}, m_{22}, m_{32})^T = c_2, \quad m_3 = (m_{13}, m_{23}, m_{33})^T = c_1.$$

为了节省内存单元, 不必将单位矩阵存放起来, c_3 存放在 A 的第 1 列位置, c_2 存放在 A 的第 2 列位置, c_1 存放在 A 的第 3 列位置, 经消元计算, 最后再调整一下列就可在 A 的位置得到 A^{-1} 。注意第 k 步消元时, 由 A 的第 k 列

$$a_k = (a_{1k}, \dots, a_{kk}, \dots, a_{nk})^T$$

计算 $m_k = \left(-\frac{a_{1k}}{a_{kk}}, \dots, -1, \dots, -\frac{a_{nk}}{a_{kk}} \right)^T$ 且冲掉 a_k 。

最后, 在 A 位置如何调整列呢? 事实上, 在 A 位置最后得到矩阵 $PA \equiv A_1$ (其中 P 为排列矩阵) 的逆矩阵 A_1^{-1} , 于是 $A^{-1} = A_1^{-1}P$ 。

算法 3 Gauss-Jordan 列主元素方法求逆, 其步骤如下。

本算法是用列主元素的 Gauss-Jordan 方法求 A^{-1} , 计算结果存放在原矩阵 A 的数组中。用整型数组 $\text{Ip}(n)$ 记录主行, A 的行列式值存放在 $\det A$ 。

步 1 $\det A \leftarrow 1$; 对于 $k = 1, 2, \dots, n$ 做到步 8。

步 2 按列选主元素 $|a_{i_k k}| = \max_{k \leq i \leq n} |a_{ik}|$; $c_0 \leftarrow a_{i_k k}$, $\text{Ip}(k) \leftarrow i_k$ 。

步 3 如果 $c_0 = 0$, 则计算停止 (此时 A 为奇异矩阵)。

步 4 如果 $i_k = k$, 则转步 5, 否则换行: $a_{kj} \leftrightarrow a_{i_k j}$ ($j = 1, 2, \dots, n$), $\det A \leftarrow -\det A$ 。

步 5 $\det A \leftarrow \det A \cdot c_0$ 。

步 6 计算 $h \leftarrow a_{kk} \leftarrow 1/c_0$; $a_{ik} \leftarrow m_{ik} = -a_{ik} \cdot h$ ($i = 1, 2, \dots, n$; $i \neq k$)。

步 7 消元计算

$$a_{ij} \leftarrow a_{ij} + m_{ik}a_{kj} \quad \left(\begin{array}{l} i = 1, 2, \dots, n; i \neq k \\ j = 1, 2, \dots, n; j \neq k \end{array} \right).$$

步 8 计算主行 $a_{kj} \leftarrow a_{kj} \cdot h$ ($j = 1, 2, \dots, n; j \neq k$)。

步 9 交换列对于 $k = n - 1, n - 2, \dots, 2, 1$,

- (1) $t = \text{Ip}(k)$;
- (2) 如果 $t \leq k$, 则转 (3), 否则换列: $a_{ik} \leftrightarrow a_{it}$ ($i = 1, 2, \dots, n$);
- (3) 继续循环。

校注 (agent 补充): 解释段中 m_k 的第 k 个分量原书印作 -1 , 此处保留。按上一页所给 $m_{kk} = 1/a_{kk}$ 、本页算法 3 步 6 的 $a_{kk} \leftarrow 1/c_0$, 以及例 7.4 第一次消元的 $m_1 = c_3 = (1/3, -2/3, -1/3)^T$, 用于原地求逆时该位置应为 $1/a_{kk}$; 疑似原书公式错误。参见[原文放大截图](#)。

原书 PDF 页 191; 印刷页码 178。

[原页扫描](#)

7.4 Gauss 消去法的变形

Gauss 消去法有很多变形, 有的是 Gauss 消去法的改进、改写, 有的是用于某一类特殊性质矩阵的 Gauss 消去法的简化。

7.4.1 直接三角分解法

将 Gauss 消去法改写为紧凑形式, 可以直接从矩阵 A 的元素得到计算 L, U 元素的递推公式, 而不需任何中间步骤, 这就是所谓**直接三角分解法**。一旦实现了矩阵 A 的 LU 分解, 那么求解式 (7.2.1) 的问题就等价于求解以下两个三角方程组:

(1) $Ly = b$, 求 y ; (2) $Ux = y$, 求 x 。

1. 不选主元的三角分解法

设 A 为非奇异矩阵, 且有分解式 $A = LU$, 其中 L 为单位下三角阵, U 为上三角阵, 即

$$A = \begin{pmatrix} 1 & & & \\ l_{21} & 1 & & \\ \vdots & \ddots & \ddots & \\ l_{n1} & \cdots & l_{n,n-1} & 1 \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ & u_{22} & \cdots & u_{2n} \\ & & \ddots & \vdots \\ & & & u_{nn} \end{pmatrix}. \quad (7.4.1)$$

下面说明 L, U 的元素可以由 n 步直接计算定出, 其中第 r 步定出 U 的第 r 行和 L 的第 r 列元素。由式 (7.4.1), 有

$$a_{1i} = u_{1i} \quad (i = 1, 2, \dots, n),$$

于是得 U 的第 1 行元素;

$$a_{i1} = l_{i1}u_{11}, \quad l_{i1} = a_{i1}/u_{11} \quad (i = 2, \dots, n),$$

于是得 L 的第 1 列元素。

设已经定出 U 的第 1 行到第 $r-1$ 行元素与 L 的第 1 列到第 $r-1$ 列元素。由式 (7.4.1), 利用矩阵乘法, 有

$$a_{ri} = \sum_{k=1}^n l_{rk}u_{ki} = \sum_{k=1}^{r-1} l_{rk}u_{ki} + u_{ri} \quad (\text{当 } r < k, \quad l_{rk} = 0 \text{ 时}),$$

故

$$u_{ri} = a_{ri} - \sum_{k=1}^{r-1} l_{rk}u_{ki} \quad (i = r, r+1, \dots, n),$$

又由式 (7.4.1) 有

$$a_{ir} = \sum_{k=1}^n l_{ik} u_{kr} = \sum_{k=1}^{r-1} l_{ik} u_{kr} + l_{ir} u_{rr}.$$

总结上述讨论，得到用直接三角分解法解 $Ax = b$ （要求 A 所有顺序主子式都不为零）的计算公式，步骤如下。

原书 PDF 页 192; 印刷页码 179。

[原页扫描](#)

步 1 $u_{1i} = a_{1i}$ ($i = 1, 2, \dots, n$), $l_{i1} = a_{i1}/u_{11}$ ($i = 2, 3, \dots, n$), 计算 U 第 r 行, L 的第 r 列元素, $r = 2, 3, \dots, n$ 。

步 2

$$u_{ri} = a_{ri} - \sum_{k=1}^{r-1} l_{rk} u_{ki} \quad (i = r, r+1, \dots, n). \quad (7.4.2)$$

步 3

$$l_{ir} = \left(a_{ir} - \sum_{k=1}^{r-1} l_{ik} u_{kr} \right) / u_{rr} \quad (i = r+1, \dots, n; r \neq n), \quad (7.4.3)$$

求解 $Ly = b, Ux = y$ 计算公式。

步 4

$$\begin{cases} y_1 = b_1, \\ y_i = b_i - \sum_{k=1}^{i-1} l_{ik} y_k \quad (i = 2, 3, \dots, n). \end{cases} \quad (7.4.4)$$

步 5

$$\begin{cases} x_n = y_n / u_{nn}, \\ x_i = \left(y_i - \sum_{k=i+1}^n u_{ik} x_k \right) / u_{ii} \quad (i = n-1, n-2, \dots, 1). \end{cases} \quad (7.4.5)$$

例 7.5 用直接三角分解法解 $\begin{pmatrix} 1 & 2 & 3 \\ 2 & 5 & 2 \\ 3 & 1 & 5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 14 \\ 18 \\ 20 \end{pmatrix}$ 。

解用分解公式 (7.4.2)、(7.4.3) 计算, 得

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & -5 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & -4 \\ 0 & 0 & -24 \end{pmatrix} = LU.$$

求解

$$Ly = (14, 18, 20)^T,$$

得

$$y = (14, -10, -72)^T,$$

求解

$$Ux = (14, -10, -72)^T,$$

得

$$x = (1, 2, 3)^T.$$

在用计算机计算时, 由于计算好 u_{ri} 后 a_{ri} 就不用了, 因此计算好 L, U 的元素后就存放在 A 的相应位置。例如

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix} \rightarrow \begin{pmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ l_{21} & u_{22} & u_{23} & u_{24} \\ l_{31} & l_{32} & u_{33} & u_{34} \\ l_{41} & l_{42} & l_{43} & u_{44} \end{pmatrix}.$$

最后在存放 A 的数组中得到 L, U 的元素。

由直接三角分解计算公式, 需要计算形如 $\sum a_i b_i$ 的式子, 可采用“双精度累加”, 以提高精度。

直接分解法大约需要 $n^3/3$ 次乘、除法运算, 和 Gauss 消去法的计算量基本相同。

如果已经实现了 $A = LU$ 的分解计算, 且 L, U 保存在 A 的相应位置, 则用直接三

原书 PDF 页 193; 印刷页码 180。

[原页扫描](#)

角分解法解具有相同系数的方程组 $Ax = (b_1, b_2, \dots, b_m)$ 是相当方便的, 每解一个方程组 $Ax = b_j$ 仅需要增加 n^2 次乘除法运算。

矩阵 A 的分解公式 (7.4.2)、(7.4.3) 又称为 **Doolittle 分解公式**。

2. 选主元的三角分解法

从直接三角分解公式可看出, 当 $u_{rr} = 0$ 时计算将中断或者当 u_{rr} 绝对值很小时, 按分解公式计算可能引起舍入误差的累积。但如果 A 非奇异, 就可通过交换 A 的行实现矩阵 PA 的 LU 分解, 因此可采用与列主元素消去法类似的方法 (可以证明下述方法与列主元素消去法等价), 将直接三角分解法修改为 (部分) 选主元的三角分解法。

设第 $r-1$ 步分解已完成, 这时有

$$A \rightarrow \begin{pmatrix} u_{11} & u_{12} & \cdots & \cdots & \cdots & \cdots & u_{1n} \\ l_{21} & u_{22} & & & & & \vdots \\ l_{31} & l_{32} & \ddots & & & & \vdots \\ \vdots & & \ddots & u_{r-1,r-1} & \cdots & \cdots & u_{n-1,n} \\ \vdots & & & l_{r,r-1} & a_{rr} & \cdots & a_{rn} \\ \vdots & & & \vdots & \vdots & & \vdots \\ l_{n1} & l_{n2} & \cdots & l_{n,r-1} & a_{nr} & \cdots & a_{nn} \end{pmatrix}.$$

第 r 步分解需用到式 (7.4.2) 及式 (7.4.3), 为了避免用小的数 u_{rr} 作除数, 引进量

$$s_i = a_{ir} - \sum_{k=1}^{r-1} l_{ik} u_{kr} \quad (i = r, r+1, \dots, n),$$

于是有

$$u_{rr} = s_r, \quad l_{ir} = s_i / s_r \quad (i = r+1, \dots, n), \quad \max_{r \leq i \leq n} |s_i| = |s_{i_r}|.$$

用 s_{i_r} 作为 u_{rr} , 交换 A 的 r 行与 i_r 行元素 (将 (i, j) 位置的新元素仍记作 l_{ij} 及 a_{ij}), 于是有 $|l_{ir}| \leq 1$ ($i = r+1, \dots, n$)。由此再进行第 r 步分解计算。

算法 4 选主元的三角分解法, 其步骤如下:

设 $Ax = b$, 其中 A 为非奇异矩阵。本算法采用列主元的三角分解法, 用 $PA = I_{n-1, i_{n-1}} \cdots I_{1, i_1} A$ 的三角分解冲掉 A , 用整型数组 $\text{Ip}(n)$ 记录主行, 解 x 存放在 b 内。

对于 $r = 1, 2, \dots, n$, 做到步 4。

步 1 计算 s_i

$$a_{ir} \leftarrow s_i = a_{ir} - \sum_{k=1}^{r-1} l_{ik} u_{kr} \quad (i = r, r+1, \dots, n).$$

步 2 选主元 $|s_{i_r}| = \max_{r \leq i \leq n} |s_i|$, $\text{Ip}(r) \leftarrow i_r$ 。

步 3 交换 A 的 r 行与 i_r 行元素 $a_{ri} \leftrightarrow a_{i_r i}$ ($i = 1, 2, \dots, n$)。

步 4 计算 U 的第 r 行元素, L 的第 r 列元素

$$a_{rr} = u_{rr} = s_r,$$

校注 (agent 补充): 本页分解存储矩阵在对角元 $u_{r-1,r-1}$ 所在行的最右侧, 原书印作 $u_{n-1,n}$, 已保留; 按该行是第 $r-1$ 行, 应为 $u_{r-1,n}$ 。疑似原书下标排印错误, 参见[原矩阵放大裁图](#)。

原书 PDF 页 194; 印刷页码 181。

[原页扫描](#)

$$a_{ir} \leftarrow l_{ir} = s_i / u_{rr} = a_{ir} / a_{rr} \quad (i = r + 1, \dots, n),$$

$$a_{ri} \leftarrow u_{ri} = a_{ri} - \sum_{k=1}^{r-1} l_{rk} u_{ki} \quad (i = r + 1, \dots, n),$$

这时有 $|l_{ir}| \leq 1$ 。

上述计算过程完成后就实现了 PA 的 LU 分解, 且 U 保存在 A 上三角部分, L 保存在 A 的下三角部分, 排列阵 P 由 $\text{Ip}(n)$ 最后记录可知。

求解 $Ly = Pb$ 及 $Ux = y$ 。

步 5 $i = 1, 2, \dots, n - 1$ 。

(1) $t \leftarrow \text{Ip}(i)$; (2) 如果 $i = t$, 则转 (3), 否则 $b_i \leftrightarrow b_t$; (3) 继续循环。

步 6

$$b_i \leftarrow b_i - \sum_{k=1}^{i-1} l_{ik} b_k \quad (i = 2, 3, \dots, n).$$

步 7

$$b_n \leftarrow b_n / u_{nn}, \quad b_i \leftarrow \left(b_i - \sum_{k=i+1}^n u_{ik} b_k \right) / u_{ii} \quad (i = n - 1, \dots, 1).$$

利用算法 4 的结果 (实现 $PA = LU$ 三角分解), 则可以计算 A 的逆矩阵

$$A^{-1} = U^{-1} L^{-1} P.$$

利用 PA 的三角分解计算 A^{-1} 步骤:

- (1) 计算上三角阵的逆矩阵 U^{-1} ;
- (2) 计算 $U^{-1} L^{-1}$;
- (3) 交换 $U^{-1} L^{-1}$ 列 (利用 $\text{Ip}(n)$ 最后记录)。

上述方法求 A^{-1} 大约需要 n^3 次乘法运算。

7.4.2 平方根法

应用有限元法解结构力学问题, 最后归结为求解线性方程组, 这时系数矩阵大多具有对称正定性质。所谓平方根法, 就是利用对称正定矩阵的三角分解而得到的求解对称正定方程组的一种有效方法, 目前在计算机上广泛应用平方根法解此类方程组。

设 A 为对称阵, 且 A 的所有顺序主子式均不为零。由定理 7.3 知, A 可唯一分解为式 (7.4.1) 的形式。

为了利用 A 的对称性, 将 U 再分解, 即

$$U = \begin{pmatrix} u_{11} & & & \\ & u_{22} & & \\ & & \ddots & \\ & & & u_{nn} \end{pmatrix} \begin{pmatrix} 1 & \frac{u_{12}}{u_{11}} & \cdots & \frac{u_{1n}}{u_{11}} \\ & \ddots & \ddots & \vdots \\ & & \ddots & \frac{u_{n-1,n}}{u_{n-1,n-1}} \\ & & & 1 \end{pmatrix} = DU_0,$$

其中 D 为对角阵, U_0 为单位上三角阵。于是

$$A = LU = LDU_0. \quad (7.4.6)$$

原书 PDF 页 195; 印刷页码 182。

[原页扫描](#)

又

$$A = A^T = U_0^T(DL^T),$$

由分解的唯一性即得 $U_0^T = L$, 代入式 (7.4.6) 得到对称矩阵 A 的分解式 $A = LDL^T$ 。

总结上述讨论, 有以下定理。

定理 7.7 (对称阵的三角分解定理) 设 A 为 n 阶对称阵, 且 A 的所有顺序主子式均不为零, 则 A 可唯一分解为

$$A = LDL^T,$$

其中 L 为单位下三角阵, D 为对角阵。

现设 A 为对称正定矩阵。首先说明 A 的分解式 $A = LDL^T$ 中 D 的对角元素 d_i 均为正数。事实上, 由 A 的对称正定性, 7.2 节的推论成立, 即

$$d_1 = D_1 > 0, \quad d_i = D_i/D_{i-1} > 0 \quad (i = 2, 3, \dots, n).$$

于是

$$D = \begin{pmatrix} d_1 & & \\ & \ddots & \\ & & d_n \end{pmatrix} = \begin{pmatrix} \sqrt{d_1} & & \\ & \ddots & \\ & & \sqrt{d_n} \end{pmatrix} \begin{pmatrix} \sqrt{d_1} & & \\ & \ddots & \\ & & \sqrt{d_n} \end{pmatrix} = D^{\frac{1}{2}} D^{\frac{1}{2}},$$

由定理 7.7 得到

$$A = LDL^T = LD^{\frac{1}{2}} D^{\frac{1}{2}} L^T = (LD^{\frac{1}{2}})(LD^{\frac{1}{2}})^T = L_1 L_1^T,$$

其中 $L_1 = LD^{\frac{1}{2}}$ 为下三角阵。

定理 7.8 (对称正定矩阵的三角分解或 Cholesky 分解) 如果 A 为 n 阶对称正定矩阵, 则存在一个实的非奇异下三角阵 L 使 $A = LL^T$, 当限定 L 的对角元素为正时, 这种分解是唯一的。

下面用直接分解方法来确定计算 L 元素的递推公式。因为

$$A = \begin{pmatrix} l_{11} & & & \\ l_{21} & l_{22} & & \\ \vdots & \vdots & \ddots & \\ l_{n1} & l_{n2} & \cdots & l_{nn} \end{pmatrix} \begin{pmatrix} l_{11} & l_{21} & \cdots & l_{n1} \\ & l_{22} & \cdots & l_{n2} \\ & & \ddots & \vdots \\ & & & l_{nn} \end{pmatrix},$$

其中 $l_{ii} > 0$ ($i = 1, 2, \dots, n$)。由矩阵乘法及 $l_{jk} = 0$ (当 $j < k$ 时), 得

$$a_{ij} = \sum_{k=1}^n l_{ik} l_{jk} = \sum_{k=1}^{j-1} l_{ik} l_{jk} + l_{jj} l_{ij},$$

于是得到以下解对称正定方程组 $Ax = b$ 的平方根法计算公式。

对于 $j = 1, 2, \dots, n$,

步 1

$$l_{jj} = \left(a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2 \right)^{\frac{1}{2}}. \quad (7.4.7)$$

步 2

$$l_{ij} = \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right) / l_{jj} \quad (i = j+1, \dots, n),$$

原书 PDF 页 196; 印刷页码 183。

[原页图像](#)

(续 7.4.2 平方根法。)

求解 $Ax = b$, 即求解如下两个三角方程组:

(1) $Ly = b$, 求 y ; (2) $L^T x = y$, 求 x 。

$$\begin{aligned} \text{步 3 } y_i &= \left(b_i - \sum_{k=1}^{i-1} l_{ik} y_k \right) / l_{ii} \quad (i = 1, 2, \dots, n). \\ \text{步 4 } x_i &= \left(b_i - \sum_{k=i+1}^n l_{ki} x_k \right) / l_{ii} \quad (i = n, n-1, \dots, 1). \end{aligned} \quad (7.4.8)$$

由式 (7.4.7) 知 $a_{jj} = \sum_{k=1}^j l_{jk}^2$ ($j = 1, 2, \dots, n$), 所以

$$l_{jk}^2 \leq a_{jj} \leq \max_{1 \leq j \leq n} \{a_{jj}\},$$

$$\max_{j,k} \{l_{jk}^2\} \leq \max_{1 \leq j \leq n} \{a_{jj}\}.$$

上面分析说明, 分解过程中元素 l_{jk} 的数量级不会增长且对角元素 l_{jj} 恒为正数。于是不选主元素的平方根法是一个数值稳定的方法。

当求出 L 的第 j 列元素时, L^T 的第 j 行元素亦就算出, 所以平方根法约需 $n^3/6$ 次乘除法运算, 大约为一般直接 LU 分解法计算量的一半。

由于 A 为对称阵, 因此在用计算机计算时只需存储 A 的下三角部分, 共需要存储 $n(n+1)/2$ 个元素, 可用一维数组存放, 即 $\{a_{11}, a_{21}, a_{22}, \dots, a_{n1}, a_{n2}, \dots, a_{nn}\}$ 。

矩阵元素 a_{ij} 存放于一维数组的第 $(i \cdot \frac{i-1}{2} + j)$ 个元素, L 的元素存放在 A 的相应位置。

由式 (7.4.7) 可看出, 用平方根法解对称正定方程组时, 计算 L 的元素 l_{ii} 需要用到开方运算, 为了避免开方运算, 下面用定理 7.7 的分解式 $A = LDL^T$, 即

$$A = \begin{pmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ l_{31} & l_{32} & 1 & & \\ \vdots & \vdots & \ddots & \ddots & \\ l_{n1} & l_{n2} & \cdots & l_{n,n-1} & 1 \end{pmatrix} \begin{pmatrix} d_1 & & & & \\ & d_2 & & & \\ & & \ddots & & \\ & & & \ddots & \\ & & & & d_n \end{pmatrix} \begin{pmatrix} 1 & l_{21} & l_{31} & \cdots & l_{n1} \\ & 1 & l_{32} & \cdots & l_{n2} \\ & & \ddots & \ddots & \vdots \\ & & & \ddots & l_{n-1,n-1} \\ & & & & 1 \end{pmatrix},$$

按行计算 L 的元素 l_{ij} ($j = 1, 2, \dots, i-1$)。由矩阵乘法, 并注意 $l_{jj} = 1$, $l_{jk} = 0$ ($j < k$), 得

$$a_{ij} = \sum_{k=1}^n (LD)_{ik} (L^T)_{kj} = \sum_{k=1}^n l_{ik} d_k l_{jk} = \sum_{k=1}^{j-1} l_{ik} d_k l_{jk} + l_{ij} d_j l_{jj}.$$

于是得到以下计算 L 的元素及 D 的对角元素公式:

对于 $i = 1, 2, \dots, n$,

$$\begin{aligned} \text{步 1} \quad l_{ij} &= \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} d_k l_{jk} \right) / d_j \quad (j = 1, 2, \dots, i-1). \\ \text{步 2} \quad d_i &= a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2 d_k. \end{aligned} \tag{7.4.9}$$

校注 (agent 补充, 原书疑误): 式 (7.4.8) 步 4 原图确印 b_i , 这里忠实保留。由本页所列 $L^T x = y$, 回代右端应使用 y_i ; 否则与其前方三角方程组不一致。[原式局部图](#)。

校注 (agent 补充, 原书疑误): LDL^T 展开中最右矩阵次末行末项原图印作 $l_{n-1, n-1}$, 这里保留原文; 按左边 L 的转置, 该位置应为 $l_{n, n-1}$ 。[原矩阵局部图](#)。

原书 PDF 页 197; 印刷页码 184。

[原页图像](#)

(续 7.4.2 平方根法。)

为避免重复计算, 引进 $t_{ij} = l_{ij}d_j$, 由式 (7.4.9) 得到以下按行计算 L, T 元素的公式:

对于 $i = 1, 2, \dots, n$,

$$\begin{aligned} \text{步 1} \quad t_{ij} &= a_{ij} - \sum_{k=1}^{j-1} t_{ik}l_{jk} \quad (j = 1, 2, \dots, i-1). \\ \text{步 2} \quad l_{ij} &= t_{ij}/d_j \quad (j = 1, 2, \dots, i-1). \\ \text{步 3} \quad d_i &= a_{ii} - \sum_{k=1}^{i-1} t_{ik}l_{ik}. \end{aligned} \tag{7.4.10}$$

计算出 $T = LD$ 的第 i 行元素 t_{ij} ($j = 1, 2, \dots, i-1$) 后, 存放在 A 的第 i 行相应位置, 然后再计算 L 的第 i 行元素, 存放在 A 的第 i 行, D 的对角元素存放在 A 的相应位置。例如,

$$A = \begin{pmatrix} a_{11} & a_{21} & a_{31} & a_{41} \\ a_{21} & a_{22} & a_{32} & a_{42} \\ a_{31} & a_{32} & a_{33} & a_{43} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix} \rightarrow \begin{pmatrix} d_1 & & & \\ l_{21} & d_2 & & \\ l_{31} & l_{32} & d_3 & \\ t_{41} & t_{42} & t_{43} & t_{44} \end{pmatrix} \rightarrow \begin{pmatrix} d_1 & & & \\ l_{21} & d_2 & & \\ l_{31} & l_{32} & d_3 & \\ l_{41} & l_{42} & l_{43} & d_4 \end{pmatrix}.$$

对称正定矩阵 A 按 LDL^T 分解和按 LL^T 分解计算量差不多, 但 LDL^T 分解不需要开方计算。求解 $Ly = b, DL^Tx = y$ 的计算公式分别为步 4、步 5 所述公式。

$$\begin{aligned} \text{步 4} \quad & \begin{cases} y_1 = b_1, \\ y_i = b_i - \sum_{k=1}^{i-1} l_{ik}y_k \quad (i = 2, \dots, n). \end{cases} \\ \text{步 5} \quad & \begin{cases} x_n = y_n/d_n, \\ x_i = y_i/d_i - \sum_{k=i+1}^n l_{ki}x_k \quad (i = n-1, \dots, 2, 1). \end{cases} \end{aligned} \tag{7.4.11}$$

计算公式 (7.4.10)、(7.4.11) 称为**改进的平方根法**。

7.4.3 追赶法

在一些实际问题中, 例如解常微分方程边值问题, 解热传导方程以及船体数学放样中建立三次样条函数等中, 都会要求解系数矩阵为对角占优的三对角方程组

$$\begin{pmatrix} b_1 & c_1 & & & & \\ a_2 & b_2 & c_2 & & & \\ & \ddots & \ddots & \ddots & & \\ & & a_i & b_i & c_i & \\ & & & \ddots & \ddots & \ddots \\ & & & & a_{n-1} & b_{n-1} & c_{n-1} \\ & & & & & a_n & b_n \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_i \\ \vdots \\ x_{n-1} \\ x_n \end{pmatrix} = \begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_i \\ \vdots \\ f_{n-1} \\ f_n \end{pmatrix},$$

(方程组及说明续下页。)

校注 (agent 补充, 原书疑误): 式 (7.4.11) 步 4 的求和项原图印作 $l_{ik}y_h$, 这里保留; 求和指标为 k 且来源为 $Ly = b$, 相应项应为 $l_{ik}y_k$ 。原式局部图。

原书 PDF 页 198; 印刷页码 185。

[原页图像](#)

(续 7.4.3 追赶法。)

简记作

$$Ax = f, \quad (7.4.12)$$

其中 A 满足下列对角占优条件:

- (1) $|b_1| > |c_1| > 0$;
- (2) $|b_i| \geq |a_i| + |c_i|$, $a_i, c_i \neq 0$ ($i = 2, 3, \dots, n-1$);
- (3) $|b_n| > |a_n| > 0$ 。

下面利用矩阵的直接三角分解法来推导解三对角方程组 (7.4.12) 的计算公式。由系数阵 A 的特点, 可以将 A 分解为两个三角阵的乘积, 即

$$A = LU$$

其中 L 为下三角阵, U 为单位上三角阵。下面说明这种分解是可能的。设

$$A = \begin{pmatrix} b_1 & c_1 & & & \\ a_2 & b_2 & c_2 & & \\ & \ddots & \ddots & \ddots & \\ & & a_{n-1} & b_{n-1} & c_{n-1} \\ & & & a_n & b_n \end{pmatrix} = \begin{pmatrix} \alpha_1 & & & & \\ \gamma_2 & \alpha_2 & & & \\ & \gamma_3 & \alpha_3 & & \\ & & \ddots & \ddots & \\ & & & \gamma_n & \alpha_n \end{pmatrix} \begin{pmatrix} 1 & \beta_1 & & & \\ & 1 & \beta_2 & & \\ & & \ddots & \ddots & \\ & & & 1 & \beta_{n-1} \\ & & & & 1 \end{pmatrix}, \quad (7.4.13)$$

其中 $\alpha_i, \beta_i, \gamma_i$ 为待定系数。比较式 (7.4.13) 两端即得

$$\left. \begin{aligned} b_1 &= \alpha_1, & c_1 &= \alpha_1 \beta_1, \\ a_i &= \gamma_i, & b_i &= \gamma_i \beta_{i-1} + \alpha_i \quad (i = 2, \dots, n), \\ c_i &= \alpha_i \beta_i \quad (i = 2, 3, \dots, n-1). \end{aligned} \right\} \quad (7.4.14)$$

由 $\alpha_1 = b_1 \neq 0$, $|b_1| > |c_1| > 0$, $\beta_1 = c_1/b_1$, 得 $0 < |\beta_1| < 1$ 。下面用归纳法证明

$$|\alpha_i| > |c_i| \neq 0 \quad (i = 1, 2, \dots, n-1), \quad (7.4.15)$$

即

$$0 < |\beta_i| < 1,$$

从而由式 (7.4.14) 可求出 β_i 。

式 (7.4.15) 对 $i = 1$ 是成立的。现设式 (7.4.15) 对 $i - 1$ 成立, 求证对 i 亦成立。

由归纳法假设 $0 < |\beta_{i-1}| < 1$, 又由式 (7.4.15) 及 A 的假设条件, 有

$$|\alpha_i| = |b_i - a_i \beta_{i-1}| \geq |b_i| - |a_i \beta_{i-1}| > |b_i| - |a_i| \geq |c_i| \neq 0,$$

也就是 $0 < |\beta_i| < 1$ 。由式 (7.4.14) 得到

$$\alpha_i = b_i - a_i \beta_{i-1} \quad (i = 2, 3, \dots, n), \quad \beta_i = c_i / (b_i - a_i \beta_{i-1}) \quad (i = 2, 3, \dots, n-1).$$

这就是说, 由 A 的假设条件完全确定了 $\{\alpha_i\}, \{\beta_i\}, \{\gamma_i\}$, 实现了 A 的 LU 分解。

求解 $Ax = f$ 等价于解两个三角方程组 $Ly = f$ 与 $Ux = y$, 先后求 y 与 x , 从而得到以下解三对角方程组的**追赶法公式**:

步 1 计算 $\{\beta_i\}$ 的递推公式

$$\beta_1 = c_1 / b_1, \quad \beta_i = c_i / (b_i - a_i \beta_{i-1}) \quad (i = 2, 3, \dots, n-1);$$

步 2 解 $Ly = f$:

$$y_1 = f_1 / b_1, \quad y_i = (f_i - a_i y_{i-1}) / (b_i - a_i \beta_{i-1}) \quad (i = 2, 3, \dots, n);$$

步 3 解 $Ux = y$:

$$x_n = y_n, \quad x_i = y_i - \beta_i x_{i+1} \quad (i = n-1, n-2, \dots, 2, 1).$$

原书 PDF 页 199; 印刷页码 186。

[原页图像](#)

(续 7.4.3 追赶法。)

将计算系数 $\beta_1 \rightarrow \beta_2 \rightarrow \cdots \rightarrow \beta_{n-1}$ 及 $y_1 \rightarrow y_2 \rightarrow \cdots \rightarrow y_n$ 的过程称为**追的过程**, 将计算方程组的解 $x_n \rightarrow x_{n-1} \rightarrow \cdots \rightarrow x_1$ 的过程称为**赶的过程**。

总结上述讨论, 有以下定理。

定理 7.9 设有三对角方程组 $Ax = f$, 其中 A 满足对角占优条件, 则 A 为非奇异矩阵且由追赶法计算公式中 $\{\alpha_i\}, \{\beta_i\}$ 满足:

1° $0 < |\beta_i| < 1$ ($i = 1, 2, \cdots, n-1$);

2°

$$0 < |c_i| \leq |b_i| - |a_i| < |\alpha_i| < |b_i| + |a_i| \quad (i = 2, 3, \cdots, n-1),$$

$$0 < |b_n| - |a_n| < |\alpha_n| < |b_n| + |a_n|.$$

追赶法公式实际上就是把 Gauss 消去法用到求解三对角方程组上去的结果。这时由于 A 特别简单, 因此使得求解的计算公式也非常简单, 计算量仅为 $5n-4$ 次乘除法运算, 而另外增加解一个方程组 $Ax = f_2$ 仅需增加 $3n-2$ 次乘除运算。易见追赶法的计算量是比较小的。

定理 7.9 的 1°、2° 说明追赶法计算公式中不会出现中间结果数量级的巨大增长和舍入误差的严重累积。

在用计算机计算时, 只需用三个一维数组分别存储 A 的三条对角线元素 $\{a_i\}, \{b_i\}, \{c_i\}$, 此外还需要用两组存储单元保存 $\{\beta_i\}, \{y_i\}$ 或 $\{x_i\}$ 。

7.5 向量和矩阵的范数

为了研究线性方程组近似解的误差估计和迭代法的收敛性, 需要对 $\mathbf{R}^n$ (n 维向量空间) 中向量 (或 $\mathbf{R}^{n \times n}$ 中矩阵) 的“大小”引进某种度量——向量 (或矩阵) 范数的概念。向量范数概念是三维 Euclid 空间中向量长度概念的推广, 在数值分析中起着重要作用。

我们用 $\mathbf{R}^n$ 表示 n 维实向量空间, 用 $\mathbf{C}^n$ 表示 n 维复向量空间, 首先将向量长度概念推广到 $\mathbf{R}^n$ (或 $\mathbf{C}^n$) 中。

定义 7.1 设

$$x = (x_1, x_2, \cdots, x_n)^T, \quad y = (y_1, y_2, \cdots, y_n)^T \in \mathbf{R}^n (\text{或 } \mathbf{C}^n),$$

将实数 $(x, y) = y^T x = \sum_{i=1}^n x_i y_i$ (或复数 $(x, y) = y^H x = \sum_{i=1}^n x_i \overline{y_i}$, 其中 $y^H = \overline{y^T}$) 称为向量 x, y 的**数量积**。将非负实数

$$\|x\|_2 = (x, x)^{\frac{1}{2}} = \left(\sum_{i=1}^n x_i^2 \right)^{\frac{1}{2}}$$

或

$$\|x\|_2 = (x, x)^{\frac{1}{2}} = \left(\sum_{i=1}^n |x_i|^2\right)^{\frac{1}{2}}$$

(定义续下页。)

原书 PDF 页 200; 印刷页码 187。

[原页图像](#)

(续 7.5 定义 7.1。)

称为向量 x 的 **Euclid 范数**。

下述定理可在线性代数书中找到。

定理 7.10 设 $x, y \in \mathbf{R}^n$ (或 $\mathbf{C}^n$), 则

1° $(x, x) = 0$, 当且仅当 $x = 0$ 时成立;

2° $(\alpha x, y) = \alpha(x, y)$, α 为实数 (或 $(x, \alpha y) = \bar{\alpha}(x, y)$, α 为复数);

3° $(x, y) = (y, x)$ (或 $(x, y) = \overline{(y, x)}$);

4° $(x_1 + x_2, y) = (x_1, y) + (x_2, y)$;

5° (Cauchy-Schwarz 不等式) $|(x, y)| \leq \|x\|_2 \cdot \|y\|_2$, 等式当且仅当 x 与 y 线性相关时成立;

6° (三角不等式) $\|x + y\|_2 \leq \|x\|_2 + \|y\|_2$ 。

还可以用其他办法来度量 $\mathbf{R}^n$ 中向量的“大小”。例如 $x = (x_1, x_2)^T \in \mathbf{R}^2$, 用一个 x 的函数 $N(x) = \max_{i=1,2} |x_i|$ 求度量 x 的“大小”, 而且这种度量 x “大小”的方法计算起来比 Euclid 范数方便。在许多应用中, 对度量向量 x “大小”的函数 $N(x)$ 都要求是正定的、齐次的且满足三角不等式。下面给出向量范数的一般定义。

定义 7.2 (向量的范数) 如果向量 $x \in \mathbf{R}^n$ (或 $\mathbf{C}^n$) 的某个实值函数 $N(x) = \|x\|$, 满足条件

$$\begin{aligned} 1^\circ \quad & \|x\| \geq 0 \quad (\|x\| = 0 \text{ 当且仅当 } x = 0) \quad (\text{正定条件}); \\ 2^\circ \quad & \|\alpha x\| = |\alpha| \|x\|, \quad \forall \alpha \in \mathbf{R} (\text{或 } \alpha \in \mathbf{C}); \\ 3^\circ \quad & \|x + y\| \leq \|x\| + \|y\| \quad (\text{三角不等式}), \end{aligned} \tag{7.5.1}$$

则称 $N(x)$ 是 $\mathbf{R}^n$ (或 $\mathbf{C}^n$) 上的一个**向量范数** (或**模**)。

由条件 3° 可推出不等式

$$\| \|x\| - \|y\| \| \leq \|x - y\|. \tag{7.5.2}$$

下面给出几种常用的向量范数。

(1) 向量的 **∞ -范数 (最大范数)**: $\|x\|_\infty = \max_{1 \leq i \leq n} |x_i|$ 。

容易验证这样定义的向量 x 的函数 $N(x) = \|x\|_\infty$ 满足向量范数的三个条件。

(2) 向量的 **1-范数**: $\|x\|_1 = \sum_{i=1}^n |x_i|$ 。同样可证 $N(x) = \|x\|_1$ 是 $\mathbf{R}^n$ 上的一个向量范数。

(3) 向量的 **2-范数**: $\|x\|_2 = (x, x)^{\frac{1}{2}} = (\sum_{i=1}^n x_i^2)^{\frac{1}{2}}$ 。由定理 7.10 知, $N(x) = \|x\|_2$ 是 $\mathbf{R}^n$ 上一个向量范数, 称为向量 x 的 Euclid 范数。

(4) 向量的 **p -范数**: $\|x\|_p = (\sum_{i=1}^n |x_i|^p)^{1/p}$, 其中 $p \in [1, \infty)$ 。可以证明向量函数 $N(x) = \|x\|_p$ 是 $\mathbf{R}^n$ 上向量的范数, 且容易说明上述三种范数是 p 范数的特殊情况 ($\|x\|_\infty = \lim_{p \rightarrow \infty} \|x\|_p$)。

例 7.6 计算向量 $x = (1, -2, 3)^T$ 的各种范数。

(例 7.6 解续下页。)

原书 PDF 页 201; 印刷页码 188。

[原页图像](#)

(续 7.5 例 7.6。)

解 $\|x\|_1 = 6, \|x\|_\infty = 3, \|x\|_2 = \sqrt{14}$ 。

定义 7.3 设 $\{x^{(k)}\}$ 为 $\mathbf{R}^n$ 中一向量序列, $x^* \in \mathbf{R}^n$, 记 $x^{(k)} = (x_1^{(k)}, x_2^{(k)}, \dots, x_n^{(k)})^T$, $x^* = (x_1^*, x_2^*, \dots, x_n^*)^T$ 。如果 $\lim_{k \rightarrow \infty} x_i^{(k)} = x_i^*$ ($i = 1, 2, \dots, n$), 则称 $x^{(k)}$ **收敛**于向量 x^* , 记作

$$\lim_{k \rightarrow \infty} x^{(k)} = x^*.$$

定理 7.11 ($N(x)$ 的连续性) 设非负函数 $N(x) = \|x\|$ 为 $\mathbf{R}^n$ 上任一向量范数, 则 $N(x)$ 是 x 分量 $x_1, x_2, \dots, x_n$ 的连续函数。

证明 设 $x = \sum_{i=1}^n x_i e_i, y = \sum_{i=1}^n y_i e_i$, 其中 $e_i = (0, \dots, 1, 0, \dots, 0)^T$ (即第 i 个元素为 1)。

只需证明当 $x \rightarrow y$ 时 $N(x) \rightarrow N(y)$ 即可。事实上,

$$\begin{aligned} |N(x) - N(y)| &= \left| \|x\| - \|y\| \right| \leq \|x - y\| = \left\| \sum_{i=1}^n (x_i - y_i) e_i \right\| \\ &\leq \sum_{i=1}^n |x_i - y_i| \|e_i\| \leq \|x - y\|_\infty \sum_{i=1}^n \|e_i\|, \end{aligned}$$

即

$$|N(x) - N(y)| \leq c \|x - y\|_\infty \rightarrow 0 \quad (\text{当 } x \rightarrow y \text{ 时}),$$

其中

$$c = \sum_{i=1}^n \|e_i\|.$$

定理 7.12 (向量范数的等价性) 设 $\|x\|_s, \|x\|_t$ 为 $\mathbf{R}^n$ 上向量的任意两种范数, 则存在常数 $c_1, c_2 > 0$, 使得

$$c_1 \|x\|_s \leq \|x\|_t \leq c_2 \|x\|_s, \quad \text{对一切 } x \in \mathbf{R}^n.$$

证明 只要就 $\|x\|_s = \|x\|_\infty$ 证明上式成立即可, 即证明存在常数 $c_1, c_2 > 0$, 使

$$c_1 \leq \frac{\|x\|_t}{\|x\|_\infty} \leq c_2, \quad \text{对一切 } x \in \mathbf{R}^n \text{ 且 } x \neq 0.$$

考虑泛函 $f(x) = \|x\|_t \geq 0, \quad x \in \mathbf{R}^n$ 。

记 $S = \{x \mid \|x\|_\infty = 1, x \in \mathbf{R}^n\}$, 则 S 是一个有界闭集。由于 $f(x)$ 为 S 上的连续函数, 所以 $f(x)$ 于 S 上达到最大、最小值。设 $x \in \mathbf{R}^n$ 且 $x \neq 0$, 则 $\frac{x}{\|x\|_\infty} \in S$, 从而有

$$f(x') = c_1 \leq f\left(\frac{x}{\|x\|_\infty}\right) \leq c_2 = f(x''), \quad (7.5.3)$$

其中 $x', x'' \in S$ 。显然 $c_1, c_2 > 0$ ，上式为 $c_1 \leq \left\| \frac{x}{\|x\|_\infty} \right\| \leq c_2$ ，即

$$c_1 \|x\|_\infty \leq \|x\|_t \leq c_2 \|x\|_\infty, \quad \text{对一切 } x \in \mathbf{R}^n.$$

注意，定理 7.12 不能推广到无穷维空间。由定理 7.12 可得到结论：如果在一种范数意义下向量序列收敛，则在任何一种范数意义下该向量序列亦收敛。

原书 PDF 页 202; 印刷页码 189。

[原页图像](#)

(续 7.5 向量和矩阵的范数。)

定理 7.13 $\lim_{k \rightarrow \infty} x^{(k)} = x^* \iff \|x^{(k)} - x^*\| \rightarrow 0$ (当 $k \rightarrow \infty$ 时), 其中 $\|\cdot\|$ 为向量的任一种范数。

证明 显然, $\lim_{k \rightarrow \infty} x^{(k)} = x^* \iff \|x^{(k)} - x^*\|_\infty \rightarrow 0$ (当 $k \rightarrow \infty$ 时), 而对于 $\mathbf{R}^n$ 上任一种范数 $\|\cdot\|$, 由定理 7.11, 存在常数 $c_1, c_2 > 0$, 使

$$c_1 \|x^{(k)} - x^*\|_\infty \leq \|x^{(k)} - x^*\| \leq c_2 \|x^{(k)} - x^*\|_\infty,$$

于是又有

$$\|x^{(k)} - x^*\|_\infty \rightarrow 0 \iff \|x^{(k)} - x^*\| \rightarrow 0 \quad (\text{当 } k \rightarrow \infty \text{ 时}).$$

下面将向量范数概念推广到矩阵上去。用 $\mathbf{R}^{n \times n}$ 表示 $n \times n$ 矩阵的集合 (本质上是和 $\mathbf{R}^{n \times n}$ 一样的向量空间), 则由 $\mathbf{R}^{n \times n}$ 上 2-范数可以得到 $\mathbf{R}^{n \times n}$ 中矩阵的一种范数

$$F(A) = \|A\|_F = \left(\sum_{i,j=1}^n a_{ij}^2 \right)^{\frac{1}{2}}$$

称为 A 的 **Frobenius 范数**。 $\|A\|_F$ 显然满足正定性、齐次性及三角不等式。

下面给出矩阵范数的一般定义。

定义 7.4 (矩阵范数) 如果矩阵 $A \in \mathbf{R}^{n \times n}$ 的某个非负实值函数 $N(A) = \|A\|$, 满足条件

$$\begin{aligned} 1^\circ \quad & \|A\| \geq 0 \quad (\|A\| = 0 \iff A = 0) \quad (\text{正定条件}); \\ 2^\circ \quad & \|cA\| = |c| \|A\|, \quad c \text{ 为实数} \quad (\text{齐次条件}); \\ 3^\circ \quad & \|A + B\| \leq \|A\| + \|B\| \quad (\text{三角不等式}); \\ 4^\circ \quad & \|AB\| \leq \|A\| \|B\|, \end{aligned} \tag{7.5.4}$$

则称 $N(A)$ 是 $\mathbf{R}^{n \times n}$ 上的一个**矩阵范数** (或模)。

上面定义的 $F(A) = \|A\|_F$ 就是 $\mathbf{R}^{n \times n}$ 上的一个矩阵范数。

由于在大多数与估计有关的问题中, 矩阵和向量会同时参与讨论, 所以希望引进一种矩阵的范数, 它是和向量范数相联系而且和向量范数相容的, 即

$$\|Ax\| \leq \|A\| \|x\| \tag{7.5.5}$$

对于任意向量 $x \in \mathbf{R}^n$ 及 $A \in \mathbf{R}^{n \times n}$ 都成立。

为此再引进一种矩阵范数。

定义 7.5 (矩阵的算子范数) 设 $x \in \mathbf{R}^n, A \in \mathbf{R}^{n \times n}$, 给出一种向量范数 $\|x\|_v$ (如 $v = 1, 2$ 或 ∞), 相应地定义一个矩阵的非负函数

$$\|A\|_v = \max_{x \neq 0} \frac{\|Ax\|_v}{\|x\|_v}. \quad (7.5.6)$$

可验证 $\|A\|_v$ 满足定义 7.4 (见下面定理), 所以 $\|A\|_v$ 是 $\mathbf{R}^{n \times n}$ 上的一个矩阵范数, 称为 A 的**算子范数**。

定理 7.14 设 $\|x\|_v$ 是 $\mathbf{R}^n$ 上一个向量范数, 则 $\|A\|_v$ 是 $\mathbf{R}^{n \times n}$ 上的矩阵范数, 且满足相容条件

$$\|Ax\|_v \leq \|A\|_v \|x\|_v. \quad (7.5.7)$$

证明 由式 (7.5.6) 知, 相容性条件式 (7.5.7) 是显然的。现只验证定义 7.4 中条

(证明接下页。)

校注 (agent 补充, 原书疑误): 定理 7.13 证明中原文引用“定理 7.11”, 这里保留; 所用的两侧常数比较直接对应上一页的定理 7.12 (向量范数的等价性), 定理 7.11 是连续性定理。[原文局部图](#)。

原书 PDF 页 203; 印刷页码 190。

[原页图像](#)

(续 7.5 定理 7.14 证明。)

件 4°。

由式 (7.5.7), 有

$$\|ABx\|_v \leq \|A\|_v \|Bx\|_v \leq \|A\|_v \|B\|_v \|x\|_v.$$

当 $x \neq 0$ 时, 有

$$\frac{\|ABx\|_v}{\|x\|_v} \leq \|A\|_v \|B\|_v, \quad \|AB\|_v = \max_{x \neq 0} \frac{\|ABx\|_v}{\|x\|_v} \leq \|A\|_v \|B\|_v.$$

显然这种矩阵范数 $\|A\|_v$ 依赖于向量范数 $\|x\|_v$ 的具体含义。也就是说, 当给出一种具体的向量范数 $\|x\|_v$ 时, 相应地就得到了一种矩阵范数 $\|A\|_v$ 。

定理 7.15 设 $x \in \mathbf{R}^n, A \in \mathbf{R}^{n \times n}$, 则

1° $\|A\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{ij}|$ (称为 A 的**行范数**);

2° $\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}|$ (称为 A 的**列范数**);

3° $\|A\|_2 = \sqrt{\lambda_{\max}(A^T A)}$ (称为 A 的**2-范数**),

其中 $\lambda_{\max}(A^T A)$ 表示 $A^T A$ 的最大特征值。

证明 只就 1°、3° 给出证明, 2° 同理可证。

1° 设 $x = (x_1, x_2, \dots, x_n)^T \neq 0$, 不妨设 $A \neq 0$ 。记 $t = \max_i |x_i|$, $\mu = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{ij}|$, 则

$$\|Ax\|_\infty = \max_{1 \leq i \leq n} \left| \sum_{j=1}^n a_{ij} x_j \right| \leq \max_i \sum_{j=1}^n |a_{ij}| |x_j| \leq t \max_i \sum_{j=1}^n |a_{ij}|.$$

这说明, 对于任何非零 $x \in \mathbf{R}^n$, 有

$$\frac{\|Ax\|_\infty}{\|x\|_\infty} \leq \mu. \quad (7.5.8)$$

下面说明, 有一向量 $x_0 \neq 0$, 使 $\frac{\|Ax_0\|_\infty}{\|x_0\|_\infty} = \mu$ 。设 $\mu = \sum_{j=1}^n |a_{i_0 j}|$, 取向量

$$x_0 = (x_1, x_2, \dots, x_n)^T,$$

其中

$$x_j = \operatorname{sgn}(a_{i_0 j}) \quad (j = 1, 2, \dots, n).$$

显然 $\|x_0\|_\infty = 1$, 且 Ax_0 的第 i_0 个分量为 $\sum_{j=1}^n a_{i_0 j} x_j = \sum_{j=1}^n |a_{i_0 j}|$, 这说明

$$\|Ax_0\|_\infty = \max_i \sum_{j=1}^n |a_{ij} x_j| = \sum_{j=1}^n |a_{i_0 j}| = \mu.$$

3° 由于 $\|Ax\|_2^2 = (Ax, Ax) = (A^T Ax, x) \geq 0$, 对于一切 $x \in \mathbf{R}^n$, 从而 $A^T A$ 的特征值为非负实数, 设为

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n \geq 0. \quad (7.5.9)$$

$A^T A$ 为对称矩阵, 设 $u_1, u_2, \dots, u_n$ 为 $A^T A$ 的相应于式 (7.5.9) 的特征向量且

(证明续下页。)

校注 (agent 补充, 原书疑误): Ax_0 第 i_0 个分量表达式的第一个求和号下限原图为 $i = 1$, 这里保留; 该和式的项是 $a_{i_0 j} x_j$, 求和指标应为 j , 相应下限应为 $j = 1$ 。[原式局部图](#)。

原书 PDF 页 204; 印刷页码 191。

[原页图像](#)

(续 7.5 定理 7.15 证明。)

$(u_i, u_j) = \delta_{ij}$, 又设 $x \in \mathbf{R}^n$ 为任一非零向量, 于是有

$$x = \sum_{i=1}^n c_i u_i,$$

其中 c_i 为组合系数。

$$\frac{\|Ax\|_2^2}{\|x\|_2^2} = \frac{(A^T Ax, x)}{(x, x)} \leq \frac{\sum_{i=1}^n c_i^2 \lambda_1}{\sum_{i=1}^n c_i^2} = \lambda_1.$$

另一方面, 取 $x = u_1$, 则上式等号成立, 故

$$\|A\|_2 = \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2} = \sqrt{\lambda_1} = \sqrt{\lambda_{\max}(A^T A)}.$$

由定理 7.15 看出, 计算一个矩阵的 $\|A\|_\infty, \|A\|_1$ 还是比较容易的, 而矩阵的 2-范数 $\|A\|_2$ 在计算上不方便。但是矩阵的 2-范数具有许多好的性质, 它在理论上是有益的。

例 7.7 $A = \begin{pmatrix} 1 & -2 \\ -3 & 4 \end{pmatrix}$, 计算 A 的各种范数。

解 $\|A\|_1 = 6, \|A\|_\infty = 7, \|A\|_F \approx 5.477, \|A\|_2 = \sqrt{15 + \sqrt{221}} \approx 5.46$ 。

对于复矩阵 (即 $A \in \mathbf{C}^{n \times n}$), 定理 7.15 中的 1°、2° 显然成立; 3° 则应改为

$$\|A\|_2 = \max_{x \neq 0} \left(\frac{x^H A^H A x}{x^H x} \right) = \sqrt{\lambda_{\max}(A^H A)}.$$

其中 $A^H = \overline{A}^T$ (A 共轭转置)。

定义 7.6 设 $A \in \mathbf{R}^{n \times n}$ 的特征值为 λ_i ($i = 1, 2, \dots, n$), 称 $\rho(A) = \max_{1 \leq i \leq n} |\lambda_i|$ 为 A 的谱半径。

定理 7.16 (特征值上界) 设 $A \in \mathbf{R}^{n \times n}$, 则 $\rho(A) \leq \|A\|$, 即 A 的谱半径不超过 A 的任何一种算子范数 (对于 $\|A\|_F$ 亦对)。

证明 设 λ 是 A 的任一特征值, x 为相应的特征向量, 则 $Ax = \lambda x$, 由式 (7.5.7) 得

$$|\lambda| \|x\| = \|\lambda x\| = \|Ax\| \leq \|A\| \|x\|,$$

即

$$|\lambda| \leq \|A\|.$$

定理 7.17 如果 $A \in \mathbf{R}^{n \times n}$ 为对称矩阵, 则 $\|A\|_2 = \rho(A)$ 。

证明留作习题。

定理 7.18 如果 $\|B\| < 1$ ，则 $I \pm B$ 为非奇异矩阵，且 $\|(I \pm B)^{-1}\| \leq \frac{1}{1-\|B\|}$ ，其中 $\|\cdot\|$ 是指矩阵的算子范数。

证明用反证法。若 $\det(I \pm B) = 0$ ，则 $(I \pm B)x = 0$ 有非零解，即存在 $x_0 \neq 0$ 使

(证明续下页。)

校注 (agent 补充, 原书疑误): 复矩阵 2-范数公式的 Rayleigh 商外, 原图没有平方根或 1/2 次幂, 此处照录。
该最大商等于 $\lambda_{\max}(A^H A) = \|A\|_2^2$; 要与两侧 $\|A\|_2$ 和 $\sqrt{\lambda_{\max}(A^H A)}$ 一致, 中间项需开平方。[原式局部图](#)。

原书 PDF 页 205; 印刷页码 192。

[原页图像](#)

(续 7.5 定理 7.18 证明。)

$Bx_0 = x_0$, $\frac{\|Bx_0\|}{\|x_0\|} = 1$, 故 $\|B\| \geq 1$, 与假设矛盾。又由 $(I \pm B)(I \pm B)^{-1} = I$, 有

$$(I \pm B)^{-1} = I \mp B(I \pm B)^{-1},$$

从而

$$\|(I \pm B)^{-1}\| \leq \|I\| + \|B\|(I \pm B)^{-1}\|,$$

$$\|(I \pm B)^{-1}\| \leq \frac{1}{1 - \|B\|}.$$

7.6 误差分析

7.6.1 矩阵的条件数

考虑线性方程组 $Ax = b$, 其中设 A 为非奇异矩阵, x 为方程组的精确解。

由于 A (或 b) 元素是测量得到的, 或者是计算的结果, 所以 A (或 b) 常带有某些观测误差或者包含有舍入误差。因此, 在处理实际问题 $(A + \delta A)x = b + \delta b$ 时, 通常要考虑 A (或 b) 的微小误差对解的影响, 即考虑估计 $x - y$, 其中 y 是 $(A + \delta A)y = b + \delta b$ 的解。

首先考察一个例子。

例 7.8 设有方程组

$$\begin{pmatrix} 1 & 1 \\ 1 & 1.0001 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$$

或

$$Ax = b, \tag{7.6.1}$$

它的精确解为

$$x = (2, 0)^T.$$

现在考虑常数项的微小变化对方程组解的影响, 即考察方程组

$$\begin{pmatrix} 1 & 1 \\ 1 & 1.0001 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} 2 \\ 2.0001 \end{pmatrix}$$

或

$$A(x + \delta x) = b + \delta b, \quad (7.6.2)$$

其中 $\delta b = (0, 0.0001)^T$, $y = x + \delta x$, x 为式 (7.6.1) 的解。显然方程组 (7.6.2) 的解为

$$x + \delta x = (1, 1)^T.$$

可以看到, 式 (7.6.1) 的常数项 b 的第二个分量只有 $\frac{1}{10000}$ 的微小变化, 方程组的解却变化很大。这样的方程组称为病态方程组。

定义 7.7 如果矩阵 A 或常数项 b 的微小变化, 引起方程组 $Ax = b$ 解的巨大变化, 则称此方程组为**病态方程组**, 矩阵 A 称为**病态矩阵** (相对于方程组而言), 否则称方程组为**良态方程组**, A 称为**良态矩阵**。

应该注意, 矩阵的病态性质是矩阵本身的特性, 下面希望找出刻画矩阵病态性质

(正文续下页。)

校注 (agent 补充, 原书疑误): 本页定理 7.18 证明原文为 $Bx_0 = x_0$, 以及 $(I \pm B)^{-1} = I \mp B(I - B)^{-1}$, 均按印刷内容保留。若同时处理 $I + B$ 和 $I - B$ 两种情形, 前式应为 $Bx_0 = \mp x_0$, 后式右端的逆矩阵应为 $(I \pm B)^{-1}$; 原文写法只与减号分支一致。前式符号不影响随后所取范数等于 1 的结论。[原式局部图](#)。

原书 PDF 页 206; 印刷页码 193。

[原页图像](#)

(续 7.6.1 矩阵的条件数。)

的量。设有方程组

$$Ax = b, \quad (7.6.3)$$

其中 A 为非奇异阵, x 为式 (7.6.3) 的精确解。下面研究当方程组的系数矩阵 A (或 b) 有微小误差 (扰动) 时对解的影响。

现设 A 是精确的, b 有误差 δb , 解为 $x + \delta x$, 则

$$A(x + \delta x) = b + \delta b, \quad \delta x = A^{-1}\delta b, \quad \|\delta x\| \leq \|A^{-1}\|\|\delta b\|. \quad (7.6.4)$$

由式 (7.6.3), 有

$$\|b\| \leq \|A\|\|x\|,$$

即

$$\frac{1}{\|x\|} \leq \frac{\|A\|}{\|b\|} \quad (\text{设 } b \neq 0), \quad (7.6.5)$$

于是由式 (7.6.4) 式及式 (7.6.5), 有如下定理。

定理 7.19 设 A 是非奇异阵, $Ax = b \neq 0$, 且 $A(x + \delta x) = b + \delta b$, 则

$$\frac{\|\delta x\|}{\|x\|} \leq \|A^{-1}\|\|A\| \frac{\|\delta b\|}{\|b\|}.$$

上式给出了解的相对误差的上界, 常数项 b 的相对误差在解中可能放大 $\|A^{-1}\|\|A\|$ 倍。

现设 b 是精确的, A 有微小误差 (扰动) δA , 解为 $x + \delta x$, 则

$$(A + \delta A)(x + \delta x) = b,$$

$$(A + \delta A)\delta x = -(\delta A)x. \quad (7.6.6)$$

如果 δA 不受限制的话, $A + \delta A$ 可能奇异, 而

$$(A + \delta A) = A(I + A^{-1}\delta A).$$

由定理 7.18 知, 当 $\|A^{-1}\delta A\| < 1$ 时, $(I + A^{-1}\delta A)^{-1}$ 存在, 由式 (7.6.6), 有

$$\delta x = -(I + A^{-1}\delta A)^{-1}A^{-1}(\delta A)x, \quad \|\delta x\| \leq \frac{\|A^{-1}\|\|\delta A\|\|x\|}{1 - \|A^{-1}(\delta A)\|}.$$

设 $\|A^{-1}\|\|\delta A\| < 1$, 即得

$$\frac{\|\delta x\|}{\|x\|} \leq \frac{\|A^{-1}\|\|A\|\frac{\|\delta A\|}{\|A\|}}{1 - \|A^{-1}\|\|A\|\frac{\|\delta A\|}{\|A\|}}. \quad (7.6.7)$$

定理 7.20 设 A 为非奇异矩阵, $Ax = b \neq 0$, 且 $(A + \delta A)(x + \delta x) = b$, 如果 $\|A^{-1}\|\|\delta A\| < 1$, 则式 (7.6.7) 成立。

如果 δA 充分小, 且在 $\|A^{-1}\|\|\delta A\| < 1$ 条件下, 那么式 (7.6.7) 说明矩阵 A 的相对误差 $\frac{\|\delta A\|}{\|A\|}$ 在解中可能放大 $\|A^{-1}\|\|A\|$ 倍。

总之, 量 $\|A^{-1}\|\|A\|$ 愈小, 由 A (或 b) 的相对误差引起的解的相对误差就愈小; 量 $\|A^{-1}\|\|A\|$ 愈大, 该解的相对误差就可能愈大。所以量 $\|A^{-1}\|\|A\|$ 实际上刻画了解对原始数据变化的灵敏度, 即刻画了方程组的病态程度, 于是引进下述定

(定义续下页。)

原书 PDF 页 207; 印刷页码 194。

[原页图像](#)

(续 7.6.1 矩阵的条件数。)

义。

定义 7.8 设 A 为非奇异矩阵, 称数 $\text{cond}(A)_v = \|A^{-1}\|_v \|A\|_v$ ($v = 1, 2$ 或 ∞) 为矩阵 A 的**条件数**。

由此看出矩阵的条件数与范数有关。

矩阵的条件数是一个十分重要的概念。由上面讨论知, 当 A 的条件数相对地大时, 即 $\text{cond}(A) \gg 1$ 时, 式 (7.6.3) 是病态的 (即 A 是病态矩阵, 或者说 A 是坏条件的); 当 A 的条件数相对地小时, 式 (7.6.3) 是良态的 (或者说 A 是好条件的)。 A 的条件数愈大, 方程组的病态程度愈严重, 也就愈难得到方程组的比较准确的解。

通常使用的条件数有

$$(1) \text{cond}(A)_\infty = \|A^{-1}\|_\infty \|A\|_\infty;$$

(2) A 的**谱条件数**

$$\text{cond}(A)_2 = \|A\|_2 \|A^{-1}\|_2 = \sqrt{\frac{\lambda_{\max}(A^T A)}{\lambda_{\min}(A^T A)}}.$$

当 A 为对称阵时, $\text{cond}(A)_2 = \frac{|\lambda_1|}{|\lambda_n|}$, 其中 λ_1, λ_n 为 A 的绝对值最大和绝对值最小的特征值。

条件数有如下性质:

(1) 对任何非奇异矩阵 A , 都有 $\text{cond}(A)_v \geq 1$ 。事实上,

$$\text{cond}(A)_v = \|A^{-1}\|_v \|A\|_v \geq \|A^{-1}A\|_v = 1.$$

(2) 设 A 为非奇异矩阵, c 为不等于零的常数, 则 $\text{cond}(cA)_v = \text{cond}(A)_v$ 。

(3) 如果 A 为正交矩阵, 则 $\text{cond}(A)_2 = 1$; 如果 A 为非奇异矩阵, R 为正交矩阵, 则

$$\text{cond}(RA)_2 = \text{cond}(AR)_2 = \text{cond}(A)_2.$$

例 7.9 已知 Hilbert 矩阵

$$H_n = \begin{pmatrix} 1 & \frac{1}{2} & \cdots & \frac{1}{n} \\ \frac{1}{2} & \frac{1}{3} & \cdots & \frac{1}{n+1} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{1}{n} & \frac{1}{1+n} & \cdots & \frac{1}{2n-1} \end{pmatrix},$$

计算 H_3 的条件数。

解

$$H_3 = \begin{pmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{pmatrix}, \quad H_3^{-1} = \begin{pmatrix} 9 & -36 & 30 \\ -36 & 192 & -180 \\ 30 & -180 & 180 \end{pmatrix}.$$

(例 7.9 解续下页。)

原书 PDF 页 208; 印刷页码 195。

[原页图像](#)

(续 7.6.1 例 7.9。)

(1) 计算 H_3 条件数 $\text{cond}(H_3)_\infty$ 。

$$\|H_3\|_\infty = 11/6, \quad \|H_3^{-1}\|_\infty = 408,$$

所以 $\text{cond}(H_3)_\infty = 748$ 。同样可计算 $\text{cond}(H_6) = 2.9 \times 10^6$ 。一般当 n 愈大时, H_n 的病态愈严重。

(2) 考虑方程组

$$H_3 x = (11/6, 13/12, 47/60)^T = b.$$

设 H_3 及 b 有微小误差 (取三位有效数字), 有

$$\begin{pmatrix} 1.00 & 0.500 & 0.333 \\ 0.500 & 0.333 & 0.250 \\ 0.333 & 0.250 & 0.200 \end{pmatrix} \begin{pmatrix} x_1 + \delta x_1 \\ x_2 + \delta x_2 \\ x_3 + \delta x_3 \end{pmatrix} = \begin{pmatrix} 1.83 \\ 1.08 \\ 0.783 \end{pmatrix}, \quad (7.6.8)$$

简记作 $(H_3 + \delta H_3)(x + \delta x) = b + \delta b$ 。方程组 $H_3 x = b$ 与式 (7.6.8) 的精确解为

$$x = (1, 1, 1)^T, \quad x + \delta x = (1.089\,512\,538, 0.487\,967\,062, 1.491\,002\,798)^T,$$

于是

$$\delta x = (0.089\,5, -0.512\,0, 0.491\,0)^T, \quad \frac{\|\delta H_3\|_\infty}{\|H_3\|_\infty} \approx 0.18 \times 10^{-3} < 0.02\%,$$

$$\frac{\|\delta b\|_\infty}{\|b\|_\infty} \approx 0.182\%, \quad \frac{\|\delta x\|_\infty}{\|x\|_\infty} \approx 51.2\%.$$

这就是说, H_3 与 b 相对误差不超过 0.3%, 而引起解的相对误差超过 50%。

由上面讨论可知, 要判别一个矩阵是否病态, 需要计算条件数 $\text{cond}(A) = \|A^{-1}\| \|A\|$, 而计算 A^{-1} 是比较费劲的。那么在实际计算中如何发现病态情况呢?

(1) 如果在 A 的三角约化时 (尤其是用主元素消去法解方程组 (7.6.3) 时) 出现小主元, 那么对大多数矩阵来说, A 是病态矩阵。例如用选主元的直接三角分解法解方程组 (7.6.8) (用双精度累加计算 $\sum_i a_i b_i$, 结果舍入为三位浮点数), 则有

$$I_{23}(H_3 + \delta H_3) = \begin{pmatrix} 1 & & \\ 0.333 & 1 & \\ 0.500 & 0.994 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0.500\,0 & 0.333\,0 \\ & 0.083\,5 & 0.089\,1 \\ & & \boxed{-0.005\,07} \end{pmatrix} = LU.$$

(2) 如果 A 的最大特征值和最小特征值之比 (按绝对值) 是大的, 则 A 是病态的, 即当 $|\lambda_1|/|\lambda_n|$ 是大的, 其中 A 特征值次序为

$$|\lambda_1| \geq |\lambda_2| \geq \cdots \geq |\lambda_n| > 0.$$

显然

$$|\lambda_1| \leq \|A\|, \quad \frac{1}{|\lambda_n|} \leq \|A^{-1}\|,$$

因此

$$\text{cond}(A) \geq \frac{|\lambda_1|}{|\lambda_n|} \gg 1.$$

(3) 如果系数矩阵的行列式值相对来说很小，或系数矩阵某些行近似线性相关，则 A 可能是病态的。

(条目 (4) 续下页。)

校注 (agent 补充, 原书疑误): 原文印作 $\text{cond}(H_6) = 2.9 \times 10^6$, 这里保留。沿用本例的 ∞ -范数, 对 6×6 Hilbert 矩阵作有理数精确运算得到 $\|H_6\|_\infty \|H_6^{-1}\|_\infty = 29\,070\,279 \approx 2.9 \times 10^7$, 原书指数疑应为 7。原文局部图。

校注 (agent 补充, ch07b-E014): 本页称为“精确解”的小数向量按原书照录。对式 (7.6.8) 所列有限小数作精确有理数求解, 结果为 $(153\,971, 68\,960, 210\,710)^T / 141\,321 \approx (1.0895125282, 0.4879671103, 1.4910027526)^T$, 与所印三个分量的末几位不符。因此原书这些小数不能视作式 (7.6.8) 的精确解; 该差异不影响后文约 51.2% 的结论。

原书 PDF 页 209; 印刷页码 196。

[原页图像](#)

(续 7.6.1 矩阵的条件数。)

(4) 如果系数矩阵 A 元素间数量级相差很大, 并且无一定规则, 则 A 可能是病态的。

病态问题通常不能用选主元素的消去法来解决。对于此类问题一般采用高精度的算术运算 (采用双倍字长进行运算) 或者预处理方法。即将求解 $Ax = b$ 的问题转化为求解一等价方程组

$$\begin{cases} PAQy = Pb, \\ y = Q^{-1}x. \end{cases}$$

通过选择非奇异矩阵 P, Q , 使 $\text{cond}(PAQ) < \text{cond}(A)$ 。一般选择 P, Q 为对角阵或者三角阵。

当矩阵 A 的元素大小不均时, 在 A 的行 (或列) 中引进适当的比例因子 (使矩阵 A 的所有行或列按 ∞ -范数大体上有相同的长度, 使 A 的系数均衡), 对 A 的条件数是有影响的。但这种方法不能保证 A 的条件数一定得到改善。

例 7.10 设 $A = \begin{pmatrix} 1 & 10^4 \\ 1 & 1 \end{pmatrix}$, 且

$$\begin{pmatrix} 1 & 10^4 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 10^4 \\ 2 \end{pmatrix}, \quad (7.6.9)$$

计算 $\text{cond}(A)_\infty$ 。

解由

$$A = \begin{pmatrix} 1 & 10^4 \\ 1 & 1 \end{pmatrix},$$

有

$$A^{-1} = \frac{1}{10^4 - 1} \begin{pmatrix} -1 & 10^4 \\ 1 & -1 \end{pmatrix}, \quad \text{cond}(A)_\infty = \frac{(1 + 10^4)^2}{10^4 - 1} \approx 10^4.$$

现在 A 的第 1 行引进比例因子。如用 $s_1 = \max_{1 \leq i \leq 2} |a_{1i}| = 10^4$ 除第一个方程式, 得 $A'x = b'$, 即

$$\begin{pmatrix} 10^{-4} & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad (7.6.10)$$

$$(A')^{-1} = \frac{1}{1 - 10^{-4}} \begin{pmatrix} -1 & 1 \\ 1 & -10^{-4} \end{pmatrix}, \quad \text{cond}(A')_\infty = \frac{1}{1 - 10^{-4}} \approx 4.$$

当用列主元素消去法解式 (7.6.9) 时 (计算到三位数字), 得

$$(A, b) \longrightarrow \left(\begin{array}{cc|c} 1 & 10^4 & 10^4 \\ 0 & -10^4 & -10^4 \end{array} \right),$$

于是得到很坏的结果： $x_1 = 0, x_2 = 1$ 。

现用列主元素消去法解式 (7.6.10)，得

(例 7.10 解续下页。)

校注 (agent 补充, 原书疑误): $\text{cond}(A')_\infty$ 的分式分子原图为 1, 而后面印作 ≈ 4 , 这里保留。由本页两个矩阵, $\|A'\|_\infty = 2$, $\|(A')^{-1}\|_\infty = 2/(1 - 10^{-4})$, 故条件数应为 $4/(1 - 10^{-4}) = 40\,000/9\,999 \approx 4.0004$ 。[原式局部图](#)。

原书 PDF 页 210; 印刷页码 197。

[原页图像](#)

(续 7.6.1 例 7.10。)

$$(A', b') \rightarrow \left(\begin{array}{cc|c} 1 & 1 & 2 \\ 10^{-4} & 1 & 1 \end{array} \right) \rightarrow \left(\begin{array}{cc|c} 1 & 1 & 2 \\ 0 & 1 & 1 \end{array} \right),$$

从而得到较好的结果: $x_1 = 1, x_2 = 1$ 。

设 $\bar{x}$ 为方程组 $Ax = b$ 的近似解, 于是可计算 $\bar{x}$ 的剩余向量 $r = b - A\bar{x}$, 当 r 很小时, $\bar{x}$ 是否为 $Ax = b$ 一个较好的近似解呢? 下述定理给出了解答。

定理 7.21 (事后误差估计)

1° 设 A 为非奇异矩阵, x 是精确解, $Ax = b \neq 0$;

2° 设 $\bar{x}$ 是方程组的近似解, $r = b - A\bar{x}$, 则

$$\frac{\|x - \bar{x}\|}{\|x\|} \leq \text{cond}(A) \cdot \frac{\|r\|}{\|b\|}. \quad (7.6.11)$$

证明 由 $x - \bar{x} = A^{-1}r$, 得

$$\|x - \bar{x}\| \leq \|A^{-1}\| \|r\|, \quad (7.6.12)$$

又有

$$\|b\| = \|Ax\| \leq \|A\| \|x\|,$$

$$\frac{1}{\|x\|} \leq \frac{\|A\|}{\|b\|}, \quad (7.6.13)$$

由式 (7.6.12) 及式 (7.6.13) 即得到式 (7.6.11)。

7.6.2 舍入误差

在复杂的计算中, 由浮点运算而引进的舍入误差可能积累而影响答案, 因此对任何算法都需要进行舍入误差分析, 看其是否过度影响所得的结果。

下面叙述采用选主元素 Gauss 消去法解式 (7.6.3) 的计算过程中舍入误差对解的影响。在 Gauss 消去法的舍入误差的分析和研究方面, von Neumann (在 1947 年)、Givens (在 1954 年) 和 Wilkinson (在 1961 年、1963 年) 等人都作出了自己的贡献。

设 $\bar{x}$ 为用选主元素 Gauss 消去法解式 (7.6.3) 的计算解, x 为式 (7.6.3) 的精确解。若要直接计算每一步舍入误差对解的影响来获得界的估计 $\|x - \bar{x}\|$, 那将是非常困难的。Wilkinson 等人提出了“向后误差分析方法”, 其基本思想是把计算过程中舍入误差对解的影响归结为原始数据变化对解的影响, 即计算解 $\bar{x}$ 是下述扰动方程组的精确解 $(A + \delta A)x = b$, 其中 δA 为某个“小”矩阵。

下面给出一个定理来说明这个结果。

定理 7.22 (选主元素 Gauss 消去法误差分析) 如果

1° 设 A 为 n 阶非奇异矩阵;

2° 用列主元素消去法 (或完全主元素消去法) 解式 (7.6.3);

(定理条件与结论续下页。)

校注 (agent 补充, 原书疑误): 向后误差分析段中印刷方程为 $(A + \delta A)x = b$, 而紧邻文字明确指计算解 $\bar{x}$ 是其精确解。此处保留原式; 按本段符号约定, 应写 $(A + \delta A)\bar{x} = b$ 。[原文局部图](#)。

原书 PDF 页 211; 印刷页码 198。

[原页图像](#)

(续 7.6.2 定理 7.22。)

3° 记 $a_k = \max_{1 \leq i, j \leq n} |a_{ij}^{(k)}|$, $a = \max_{1 \leq i, j \leq n} |a_{ij}|$, $r = \max_{1 \leq k \leq n} a_k/a$ ——元素的增长因子, $A^{(k)} \equiv (a_{ij}^{(k)})_n$;

4° t 为计算机字长 (指尾数部分), 矩阵的阶数 n 满足 $n \cdot 2^{-t} \leq 0.01$,

则 (1) 用选主元素 Gauss 消去法计算的三角阵 L, U 满足 $LU = A + E$, 其中

$$|(E)_{ij}| \leq 2(n-1)ra2^{-t};$$

(2) 用选主元素 Gauss 消去法得到的计算解 $\bar{x}$ 精确满足 $(A + \delta A)x = b$, 其中

$$\|\delta A\|_{\infty} \leq 1.01(n^3 + 3n^2)ar2^{-t} \leq 1.01(n^3 + 3n^2)r\|A\|_{\infty}2^{-t};$$

(3) 计算解精度估计 (x 为 $Ax = b$ 精确解)

$$\frac{\|x - \bar{x}\|_{\infty}}{\|x\|_{\infty}} = \frac{\text{cond}(A)_{\infty}}{1 - \text{cond}(A)_{\infty} \frac{\|\delta A\|_{\infty}}{\|A\|_{\infty}}} \times 1.01(n^3 + 3n^2)r2^{-t}. \quad (7.6.14)$$

上述计算解 $\bar{x}$ 精度估计式说明, $\bar{x}$ 的相对误差限依赖于 $\text{cond}(A)_{\infty}$ 、元素的增长因子、方程组阶数、计算机字长等。

小结

在 Gauss 消去法中引进选主元素的技巧, 就得到了解方程组的完全主元素消去法和列主元素消去法, 对一般矩阵引进选主元素的技巧的根本作用是为了对增长因子进行控制。完全选主元素及列主元素方法都是数值稳定的算法 (即对舍入误差的增长得以控制的算法)。用完全主元素消去法解方程组 (至少对于良态方程组是这样) 具有较高的精确度, 但它需要花费较多的机器时间。列主元素消去法比完全主元素消去法更实用的算法, 一般使用较多。这两种方法都是计算机上解线性方程组的有效方法。用 Gauss-Jordan 方法求逆矩阵是比较方便的。

从代数角度看, 直接分解法和 Gauss 消去法本质上一致, 但如果采用“双精度累加”计算 $\sum_i a_i b_i$, 那么直接三角分解法的精度要比 Gauss 消去法的精度高, 如果不采用“双精度累加” ($\sum_i a_i b_i$ 中每一个乘积分别进行舍入), 那么两个方法将给出相同的结果。

对于对称正定矩阵 A , 采用不选主元素的平方根法 (或改进的平方根法) 求解式 (7.6.3) 比较适宜。理论分析指出, 解对称正定方程组的平方根法是一个稳定的算法, 在工程计算中使用比较广泛。

追赶法是解三对角方程组 (对角元占优) 的有效方法, 它具有计算量小、方法简单、算法稳定等优点。

关于矩阵的条件数, 病态方程组, 算法的稳定性, 这些概念都是计算数学中比较重要的概念, 这里只作了简单的介绍。本章的学习可参看文献 [10]、[11]、[12]。

校注 (agent 补充, 原书疑误): 定理 7.22 (2) 印作 “计算解 $\bar{x}$ 精确满足 $(A + \delta A)x = b$ ”, 此处按原文保留; 与上一页同样, 方程中的 x 应按符号约定写为 $\bar{x}$ 。[原式局部图](#)。

校注 (agent 补充, 原书疑误): 式 (7.6.14) 原图在左侧误差比与右侧分式之间未印关系符号, 此处保持两式并列; 按精度估计用途应为 $\leq$ 。另外, 由式 (7.6.7) 得到此界还须 $\text{cond}(A)_\infty \|\delta A\|_\infty / \|A\|_\infty < 1$, 保证分母为正; 这一适用条件未在本定理列出。[原式局部图](#)。

原书 PDF 页 212; 印刷页码 199。

[原页图像](#)

习题

1. 考虑方程组

$$\begin{cases} 0.4096x_1 + 0.1234x_2 + 0.3678x_3 + 0.2943x_4 = 0.4043, \\ 0.2246x_1 + 0.3872x_2 + 0.4015x_3 + 0.1129x_4 = 0.1550, \\ 0.3645x_1 + 0.1920x_2 + 0.3781x_3 + 0.0643x_4 = 0.4240, \\ 0.1784x_1 + 0.4002x_2 + 0.2786x_3 + 0.3927x_4 = -0.2557. \end{cases}$$

(1) 用 Gauss 消去法解所给方程组 (用四位小数计算);

(2) 用列主元素消去法解所给方程组并且与 (1) 比较结果。

2. (1) 设 A 是对称阵且 $a_{11} \neq 0$, 经过 Gauss 消去法一步后, A 约化为 $\begin{pmatrix} a_{11} & a_1^T \\ 0 & A_2 \end{pmatrix}$, 证明 A_2 是对称矩阵。

(2) 用 Gauss 消去法解对称方程组

$$\begin{cases} 0.6428x_1 + 0.3475x_2 - 0.8468x_3 = 0.4127, \\ 0.3475x_1 + 1.8423x_2 + 0.4759x_3 = 1.7321, \\ -0.8468x_1 + 0.4759x_2 + 1.2147x_3 = -0.8621. \end{cases}$$

3. (1) 用式 (7.2.9) 证明

$$a_{ij}^{(k)} = a_{ij}^{(1)} - m_{i1}a_{1j}^{(1)} - m_{i2}a_{2j}^{(2)} - \cdots - m_{i,k-1}a_{k-1,j}^{(k-1)} \quad (i, j \geq k), \quad a_{ij}^{(1)} = a_{ij}.$$

(2) Gauss 消去法中 $m_{ir}, a_{rj}^{(r)}$ 使

$$m_{ir} = l_{ir} \quad (i > r), \quad a_{rj}^{(r)} = u_{rj} \quad (j \geq r),$$

利用 (1) 证明

$$u_{rj} = a_{rj} - \sum_{k=1}^{r-1} l_{rk}u_{kj} \quad (j = r, r+1, \cdots, n),$$

$$l_{ir} = \left(a_{ir} - \sum_{k=1}^{r-1} l_{ik}u_{kr} \right) / u_{rr} \quad (i = r+1, \cdots, n).$$

4. 设 A 为 n 阶非奇异矩阵且有分解式 $A = LU$, 其中 L 为单位下三角阵, U 为上三角阵, 求证 A 的所有顺序主子式均不为零。

5. 用 Gauss 消去法说明当 $\Delta_i \neq 0$ ($i = 1, 2, \dots, n-1$) 时, $A = LU$, 其中 L 为单位下三角阵, U 为上三角阵。

6. 设 A 为 n 阶矩阵, 如果 $|a_{ii}| > \sum_{j \neq i}^n |a_{ij}|$ ($i = 1, 2, \dots, n$), 则称 A 为对角占优阵。证明: 若 A 是对角占优阵, 经过 Gauss 消去法一步后, A 具有形式

$$\begin{pmatrix} a_{11} & a_1^T \\ 0 & A_2 \end{pmatrix},$$

则 A_2 是对角占优阵。由此推断, 对于对称的对角占优阵来说, 用 Gauss 消去法和部分选主元素 Gauss 消去法可得到同样的结果。

7. 设 A 是对称正定矩阵, 经过 Gauss 消去法一步后, A 约化为 $\begin{pmatrix} a_{11} & a_1^T \\ 0 & A_2 \end{pmatrix}$, 其中

$$A = (a_{ij})_n, \quad A_2 = (a_{ij}^{(2)})_{n-1}.$$

证明: (1) A 的对角元素 $a_{ii} > 0$ ($i = 1, 2, \dots, n$);

(第 7 题其余小题续下页。)

原书 PDF 页 213; 印刷页码 200。

[原页图像](#)

(续第 7 章习题第 7 题。)

(2) A_2 是对称正定矩阵;

(3) $a_{ii}^{(2)} \leq a_{ii} \ (i = 2, 3, \dots, n)$;

(4) A 的绝对值最大的元素必在对角线上;

(5) $\max_{2 \leq i, j \leq n} |a_{ij}^{(2)}| \leq \max_{2 \leq i, j \leq n} |a_{ij}|$;

(6) 从 (2)、(3)、(5) 推出, 如果 $|a_{ij}| < 1$, 则对于所有 k , $|a_{ij}^{(k)}| < 1$ 。

8. 设 L_k 为指标为 k 的初等下三角阵, 即

$$L_k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & m_{k+1,k} & 1 & \\ & & \vdots & & \ddots \\ & & m_{n,k} & & & 1 \end{pmatrix}$$

(原矩阵上方标注“第 k 列”, 对应 $m_{k+1,k}, \dots, m_{n,k}$ 所在列。)

(除第 k 列对角元素之下的元素外, L_k 和单位阵 I 相同), 求证: 当 $i, j > k$ 时, $\tilde{L}_k = I_{ij} L_k I_{ij}$ 也是一个指标为 k 的初等下三角阵, 其中 I_{ij} 为初等排列阵。

9. 试推导矩阵 A 的 Crout 分解 $A = LU$ 的计算公式, 其中 L 为下三角阵, U 为单位上三角阵。

10. 设 $Ux = d$, 其中 U 为三角阵。

(1) 就 U 为上及下三角阵推导一般的求解公式, 并写出算法。

(2) 计算解三角方程组 $Ux = d$ 的乘除法次数。

(3) 设 U 为非奇异阵, 试推导求 U^{-1} 的计算公式。

11. 证明: (1) 如果 A 是对称正定矩阵, 则 A^{-1} 也是对称正定矩阵;

(2) 如果 A 是对称正定矩阵, 则 A 可唯一地写成 $A = L^T L$, 其中 L 是具有正对角元的下三角阵。

12. 用 Gauss-Jordan 方法求 A 的逆阵, 其中

$$A = \begin{pmatrix} 2 & 1 & -3 & -1 \\ 3 & 1 & 0 & 7 \\ -1 & 2 & 4 & -2 \\ 1 & 0 & -1 & 5 \end{pmatrix}.$$

13. 用追赶法解三对角方程组 $Ax = b$, 其中

$$A = \begin{pmatrix} 2 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & 2 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

14. 用改进的平方根法解方程组

$$\begin{pmatrix} 2 & -1 & 1 \\ -1 & -2 & 3 \\ 1 & 3 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix}.$$

15. 下述矩阵能否分解为 LU (其中 L 为单位下三角阵, U 为上三角阵)? 若能分解, 那么分解是否唯一?

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 1 \\ 4 & 6 & 7 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 1 & 1 \\ 2 & 2 & 1 \\ 3 & 3 & 1 \end{pmatrix}, \quad C = \begin{pmatrix} 1 & 2 & 6 \\ 2 & 5 & 15 \\ 6 & 15 & 46 \end{pmatrix}.$$

原书 PDF 页 214; 印刷页码 201。

[原页图像](#)

(续第 7 章习题。)

16. 试划出部分选主元素三角分解法框图, 并且用此法解方程组

$$\begin{pmatrix} 0 & 3 & 4 \\ 1 & -1 & 1 \\ 2 & 1 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}.$$

17. 如果方阵 A 有 $a_{ij} = 0$ ($|i - j| > t$), 则称 A 为带宽 $2t + 1$ 的带状矩阵。设 A 满足三角分解条件, 试推导 $A = LU$ 的以下计算公式:

对于 $r = 1, 2, \dots, n$, 有

(1)

$$u_{ri} = a_{ri} - \sum_{k=\max(1, i-t)}^{r-1} l_{rk} u_{ki} \quad (i = r, r+1, \dots, \min(n, r+t));$$

(2)

$$l_{ir} = \left(a_{ir} - \sum_{k=\max(1, i-t)}^{r-1} l_{ik} u_{kr} \right) / u_{rr} \quad (i = r+1, \dots, \min(n, r+t)).$$

18. 设 $A = \begin{pmatrix} 0.6 & 0.5 \\ 0.1 & 0.3 \end{pmatrix}$, 计算 A 的行范数、列范数、2-范数及 F -范数。

19. 求证: (1) $\|x\|_\infty \leq \|x\|_1 \leq n\|x\|_\infty$; (2) $\frac{1}{\sqrt{n}}\|A\|_F \leq \|A\|_2 \leq \|A\|_F$ 。

20. 设 $P \in \mathbf{R}^{n \times n}$ 且非奇异, 又设 $\|x\|$ 为 $\mathbf{R}^n$ 上一向量范数, 定义 $\|x\|_P = \|Px\|$ 。试证明 $\|x\|_P$ 是 $\mathbf{R}^n$ 上向量的一种范数。

21. 设 $A \in \mathbf{R}^{n \times n}$ 为对称正定矩阵, 定义 $\|x\|_A = (Ax, x)^{\frac{1}{2}}$, 试证明 $\|x\|_A$ 为 $\mathbf{R}^n$ 上向量的一种范数。

22. 证明: 当且仅当 x 和 y 线性相关且 $x^T y \geq 0$ 时, 才有 $\|x + y\|_2 = \|x\|_2 + \|y\|_2$ 。

23. 分别描述 $\mathbf{R}^2$ 中 (画图) $S_v = \{x \mid \|x\|_v = 1, x \in \mathbf{R}^2\}$ ($v = 1, 2, \infty$)。

24. 令 $\|\cdot\|$ 是 $\mathbf{R}^n$ (或 $\mathbf{C}^n$) 上的任意一种范数, 而 P 是任一非奇异实 (或复) 矩阵, 定义范数 $\|x\|' = \|Px\|$, 证明 $\|A\|' = \|PAP^{-1}\|$ 。

25. 设 $\|A\|_s, \|A\|_t$ 为 $\mathbf{R}^{n \times n}$ 上任意两种矩阵算子范数, 证明存在常数 $c_1, c_2 > 0$, 使对于一切 $A \in \mathbf{R}^{n \times n}$ 满足

$$c_1 \|A\|_s \leq \|A\|_t \leq c_2 \|A\|_s.$$

26. 设 $A \in \mathbf{R}^{n \times n}$, 求证 $A^T A$ 与 AA^T 特征值相等, 即求证 $\lambda(A^T A) = \lambda(AA^T)$ 。

27. 设 A 为非奇异矩阵, 求证

$$\frac{1}{\|A^{-1}\|_{\infty}} = \min_{y \neq 0} \frac{\|A\|_{y\infty}}{\|y\|_{\infty}}.$$

28. 设 A 为非奇异矩阵, 且 $\|A^{-1}\|\|\delta A\| < 1$, 求证 $(A + \delta A)^{-1}$ 存在且有估计

$$\frac{\|A^{-1} - (A + \delta A)^{-1}\|}{\|A^{-1}\|} \leq \frac{\text{cond}(A) \frac{\|\delta A\|}{\|A\|}}{1 - \text{cond}(A) \frac{\|\delta A\|}{\|A\|}}.$$

29. 矩阵第 1 行乘以一个数, 成为 $A = \begin{pmatrix} 2\lambda & \lambda \\ 1 & 1 \end{pmatrix}$, 证明当 $\lambda = \pm \frac{2}{3}$ 时, $\text{cond}(A)_{\infty}$ 有最小值。

30. 设 A 为对称正定矩阵, 且其分解为 $A = LDL^T = W^T W$, 其中 $W = D^{1/2} L^T$, 求证:

(1) $\text{cond}(A)_2 = [\text{cond}(W)_2]^2$; (2) $\text{cond}(A_2) = \text{cond}(W^T)_2 \text{cond}(W)_2$ 。

31. 设 $A = \begin{pmatrix} 100 & 99 \\ 99 & 98 \end{pmatrix}$, 计算 A 的条件数 $\text{cond}(A)_v$ ($v = 2, \infty$)。

32. 证明: 如果 A 是正交矩阵, 则 $\text{cond}(A)_2 = 1$ 。

33. 设 $A, B \in \mathbf{R}^{n \times n}$ 且 $\|\cdot\|$ 为 $\mathbf{R}^{n \times n}$ 上矩阵的算子范数, 证明 $\text{cond}(AB) \leq \text{cond}(A) \text{cond}(B)$ 。

校注 (agent 补充, 原书疑误): 第 27 题分子中的 y 在原图排到闭合范数竖线外的下标位置, 与 ∞ 并列, 此处以 $\|A\|_{y\infty}$ 保留这一印刷位置。按题意, 该分子应为 $\|Ay\|_{\infty}$ 。原式局部图。

校注 (agent 补充, 原书疑误): 第 30 题 (2) 左端原图印为 $\text{cond}(A_2)$, 此处保留; 问题只定义了矩阵 A , 并使用 2-范数条件数, 左端应为 $\text{cond}(A)_2$ 。原式局部图。

原书 PDF 页 215；印刷页码 202。

[原页图像](#)

第 8 章 解线性方程组的迭代法

8.1 引言

考虑线性方程组

$$Ax = b, \quad (8.1.1)$$

其中 A 为非奇异矩阵。当 A 为低阶稠密矩阵时，第 7 章所讨论的选主元素消去法是解式 (8.1.1) 的有效方法。但是，对于由工程技术中产生的大型稀疏矩阵方程组 (A 阶数 n 很大，但零元素较多，例如 $n \geq 10^4$ ，由某些偏微分方程数值解所产生的线性方程组)，利用迭代法求解式 (8.1.1) 是合适的。在计算机内存和运算两方面，迭代法通常都可利用 A 中有大量零元素的特点。

本章将介绍迭代法的一些基本理论及 Jacobi 迭代法、Gauss-Seidel 迭代法、超松弛迭代法。超松弛迭代法应用很广泛。

下面举简例，以便了解迭代法的思想。

例 8.1 求解方程组

$$\begin{cases} 8x_1 - 3x_2 + 2x_3 = 20, \\ 4x_1 + 11x_2 - x_3 = 33, \\ 6x_1 + 3x_2 + 12x_3 = 36, \end{cases} \quad (8.1.2)$$

记作

$$Ax = b,$$

其中

$$A = \begin{bmatrix} 8 & -3 & 2 \\ 4 & 11 & -1 \\ 6 & 3 & 12 \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \quad b = \begin{bmatrix} 20 \\ 33 \\ 36 \end{bmatrix}.$$

方程组的精确解是

$$x^* = (3, 2, 1)^T.$$

现将式 (8.1.2) 改写成

$$\begin{cases} x_1 = \frac{1}{8}(3x_2 - 2x_3 + 20), \\ x_2 = \frac{1}{11}(-4x_1 + x_3 + 33), \\ x_3 = \frac{1}{12}(-6x_1 - 3x_2 + 36) \end{cases} \quad (8.1.3)$$

原书 PDF 页 216; 印刷页码 203。

[原页图像](#)

或

$$x = B_0 x + f,$$

其中

$$B_0 = \begin{bmatrix} 0 & \frac{3}{8} & -\frac{2}{8} \\ -\frac{4}{11} & 0 & \frac{1}{11} \\ -\frac{6}{12} & -\frac{3}{12} & 0 \end{bmatrix} = I - D^{-1}A, \quad f = \begin{bmatrix} \frac{20}{8} \\ \frac{33}{11} \\ \frac{36}{12} \end{bmatrix} = D^{-1}b.$$

任取初始值, 例如取 $x^{(0)} = (0, 0, 0)^T$ 。将这些值代入式 (8.1.3) 右端 (若式 (8.1.3) 为等式即求得方程组的解, 但一般不满足), 得到新的值

$$x^{(1)} = (x_1^{(1)}, x_2^{(1)}, x_3^{(1)})^T = (2.5, 3, 3)^T,$$

再将 $x^{(1)}$ 分量代入式 (8.1.3) 右端, 得到 $x^{(2)}$ 。反复利用这个计算程序, 得到一向量序列和一般的计算公式 (迭代公式)

$$x^{(0)} = \begin{bmatrix} x_1^{(0)} \\ x_2^{(0)} \\ x_3^{(0)} \end{bmatrix}, \quad x^{(1)} = \begin{bmatrix} x_1^{(1)} \\ x_2^{(1)} \\ x_3^{(1)} \end{bmatrix}, \quad \dots, \quad x^{(k)} = \begin{bmatrix} x_1^{(k)} \\ x_2^{(k)} \\ x_3^{(k)} \end{bmatrix}, \quad \dots,$$

$$\begin{cases} x_1^{(k+1)} = (3x_2^{(k)} - 2x_3^{(k)} + 20)/8, \\ x_2^{(k+1)} = (-4x_1^{(k)} + x_3^{(k)} + 33)/11, \\ x_3^{(k+1)} = (-6x_1^{(k)} - 3x_2^{(k)} + 36)/12, \end{cases} \quad (8.1.4)$$

简写为

$$x^{(k+1)} = B_0 x^{(k)} + f,$$

其中 k ($k = 0, 1, 2, \dots$) 表示迭代次数。

迭代到第 10 次有

$$x^{(10)} = (3.000\,032, \, 1.999\,838, \, 0.999\,881\,3)^T,$$

$$\|\varepsilon^{(10)}\|_{\infty} = 0.000\,187 \quad (\varepsilon^{(10)} = x^{(10)} - x^*).$$

从此例看出, 由迭代法作出的向量序列 $x^{(k)}$ 逐步逼近方程组的精确解 x^* 。

对于任何一个方程组 $x = Bx + f$ (由 $Ax = b$ 变形得到的等价方程组), 按迭代法作出的向量序列 $x^{(k)}$ 是否一定逐步逼近方程组的解 x^* 呢? 回答是不一定。请读者考虑, 用迭代法解下述方程组

$$\begin{cases} x_1 = 2x_2 + 5, \\ x_2 = 3x_1 + 5. \end{cases}$$

对于给定方程组 $x = Bx + f$, 设有唯一解 x^* , 则

$$x^* = Bx^* + f. \quad (8.1.5)$$

又设 $x^{(0)}$ 为任取的初始向量, 按下述公式构造向量序列

$$\begin{cases} x^{(1)} = Bx^{(0)} + f, \\ x^{(2)} = Bx^{(1)} + f, \\ \vdots \\ x^{(k+1)} = Bx^{(k)} + f, \end{cases} \quad (8.1.6)$$

校注 (agent 补充): 原页将 $x^{(10)}$ 的第二分量印为 1.999 838, 误差范数印为 0.000 187, 均按原样保留。由式 (8.1.4) 从 $x^{(0)} = 0$ 作 10 次有理数迭代, 所得向量约为 $(3.0000318141, 1.9998740186, 0.9998812605)^T$, 实际误差的无穷范数约为 0.0001259814; 即使按本页印刷的三个分量计算, 误差无穷范数也应为 0.000162。第二分量及误差范数疑为原书数值错误, 并非 OCR 错误。

原书 PDF 页 217; 印刷页码 204。

[原页图像](#)

其中 k 表示迭代次数。

定义 8.1 1° 对于给定的方程组 $x = Bx + f$, 用式 (8.1.6) 逐步代入求近似解的方法称为**迭代法** (或称为一阶定常迭代法, 这里 B 与 k 无关)。

2° 如果 $\lim_{k \rightarrow \infty} x^{(k)}$ 存在 (记作 x^*), 称此迭代法**收敛**, 显然 x^* 就是方程组的解, 否则称此迭代法**发散**。

由上述讨论, 需要研究 $\{x^{(k)}\}$ 的收敛性。引进误差向量

$$\varepsilon^{(k+1)} = x^{(k+1)} - x^*,$$

由式 (8.1.6) 减去式 (8.1.5), 得

$$\varepsilon^{(k+1)} = B\varepsilon^{(k)} \quad (k = 0, 1, 2, \dots),$$

递推得

$$\varepsilon^{(k)} = B\varepsilon^{(k-1)} = \dots = B^k \varepsilon^{(0)}.$$

要考察 $\{x^{(k)}\}$ 的收敛性, 就要研究 B 在什么条件下有 $\varepsilon^{(k)} \rightarrow 0$ ($k \rightarrow \infty$), 亦即要研究 B 满足什么条件时有 $B^k \rightarrow O$ (零矩阵) ($k \rightarrow \infty$)。

8.2 Jacobi 迭代法与 Gauss-Seidel 迭代法

8.2.1 Jacobi 迭代法

设有方程组

$$\sum_{j=1}^n a_{ij}x_j = b_i \quad (i = 1, 2, \dots, n),$$

记作

$$Ax = b, \tag{8.2.1}$$

A 为非奇异阵且 $a_{ij} \neq 0$ ($i = 1, 2, \dots, n$)。将 A 分裂为 $A = D - L - U$, 其中

$$D = \begin{bmatrix} a_{11} & & & & \\ & a_{22} & & & \\ & & \ddots & & \\ & & & \ddots & \\ & & & & a_{nn} \end{bmatrix}, \quad L = - \begin{bmatrix} 0 & & & & \\ a_{21} & 0 & & & \\ a_{31} & a_{32} & 0 & & \\ \vdots & \vdots & \ddots & \ddots & \\ a_{n1} & a_{n2} & \cdots & a_{n,n-1} & 0 \end{bmatrix},$$

$$U = - \begin{bmatrix} 0 & a_{12} & a_{13} & \cdots & a_{1n} \\ & 0 & a_{23} & \cdots & a_{2n} \\ & & \ddots & \ddots & \vdots \\ & & & 0 & a_{n-1,n} \\ & & & & 0 \end{bmatrix}.$$

将式 (8.2.1) 第 i ($i = 1, 2, \dots, n$) 个方程用 a_{ii} 去除再移项, 得到等价方程组

校注 (agent 补充, ch08-E013): 本页式 (8.2.1) 下方原印 $a_{ij} \neq 0$ ($i = 1, 2, \dots, n$), 已按原样保留。这里没有给出 j 的范围; 后续 Jacobi 分裂以对角元 a_{ii} 作除数, 通常条件应为 $a_{ii} \neq 0$ 。此处疑为原书下标错误; 独立交叉复核指出后已回原图放大确认。

原书 PDF 页 218; 印刷页码 205。

[原页图像](#)

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j \right) \quad (i = 1, 2, \dots, n), \quad (8.2.2)$$

简记作

$$x = B_0 x + f,$$

其中

$$B_0 = I - D^{-1}A = D^{-1}(L + U), \quad f = D^{-1}b.$$

对方程组 (8.2.2) 应用迭代法, 得到解式 (8.2.1) 的 **Jacobi 迭代公式**

$$\begin{cases} x^{(0)} = (x_1^{(0)}, x_2^{(0)}, \dots, x_n^{(0)})^T \quad (\text{初始向量}), \\ x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(k)} \right), \end{cases} \quad (8.2.3)$$

其中 $x^{(k)} = (x_1^{(k)}, x_2^{(k)}, \dots, x_n^{(k)})^T$ 为第 k 次迭代向量。设 $x^{(k)}$ 已经算出, 由式 (8.2.3) 可计算下一次迭代向量 $x^{(k+1)}$ ($k = 0, 1, 2, \dots; i = 1, 2, \dots, n$)。

显然迭代公式 (8.2.3) 的矩阵形式为

$$\begin{cases} x^{(0)} \quad (\text{初始向量}), \\ x^{(k+1)} = B_0 x^{(k)} + f, \end{cases} \quad (8.2.4)$$

其中 B_0 称为 **Jacobi 方法迭代矩阵**。

由此看出, Jacobi 迭代法公式简单, 每迭代一次只需计算一次矩阵和向量乘法。在用计算机计算时, 需要两组工作单元, 以存储 $x^{(k)}$ 及 $x^{(k+1)}$ 。

8.2.2 Gauss-Seidel 迭代法

由 Jacobi 方法迭代公式 (8.2.4) 可知, 迭代的每一步计算过程, 都是用 $x^{(k)}$ 的全分量来计算 $x^{(k+1)}$ 的所有分量, 显然在计算第 i 个分量 $x_i^{(k+1)}$ 时, 已经计算出的最新分量 $x_1^{(k+1)}, x_2^{(k+1)}, \dots, x_{i-1}^{(k+1)}$ 没有被利用。从直观上看, 最新计算出的分量可能比旧的分量要好些。因此, 对这些最新计算出来的第 $k+1$ 次近似 $x^{(k+1)}$ 的分量 $x_j^{(k+1)}$ 加以利用, 就得到所谓解方程组的 **Gauss-Seidel 迭代法 (简称 G-S 方法)**:

$$\begin{aligned}
 x^{(0)} &= (x_1^{(0)}, x_2^{(0)}, \dots, x_n^{(0)})^T \quad (\text{初始向量}), \\
 x_i^{(k+1)} &= \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right) \quad (k=0, 1, 2, \dots; i=1, 2, \dots, n),
 \end{aligned} \tag{8.2.5}$$

或写为

$$\begin{cases} x_i^{(k+1)} = x_i^{(k)} + \Delta x_i & (k=0, 1, 2, \dots; i=1, 2, \dots, n), \\ \Delta x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i}^n a_{ij} x_j^{(k)} \right). \end{cases} \tag{8.2.5'}$$

上面第 2 个式子利用了最新计算出的分量 $x_1^{(k+1)}$, 第 i 个式子利用了计算出的最新分量 $x_j^{(k+1)}$ ($j=1, 2, \dots, i-1$)。式 (8.2.5) 还可写成矩阵形式

$$Dx^{(k+1)} = b + Lx^{(k+1)} + Ux^{(k)}, \quad (D-L)x^{(k+1)} = b + Ux^{(k)},$$

若设 $(D-L)^{-1}$ 存在, 则

校注 (agent 补充): 式 (8.2.3) 的求和下限原页明确印为 $i=1$, 下一行仍为 $j \neq i$, 已按原样保留。由本页式 (8.2.2) 可知, 此处求和指标应为 $j=1$, 属于疑似原书排印错误, 并非 OCR 错误。

原书 PDF 页 219; 印刷页码 206。

[原页图像](#)

$$x^{(k+1)} = (D - L)^{-1}Ux^{(k)} + (D - L)^{-1}b,$$

于是 Gauss-Seidel 迭代公式的矩阵形式为

$$x^{(k+1)} = Gx^{(k)} + f, \quad (8.2.6)$$

其中

$$G = (D - L)^{-1}U, \quad f = (D - L)^{-1}b.$$

由此可以看出, 应用 Gauss-Seidel 迭代法解式 (8.2.1), 就是对方程组 $x = Gx + f$ 应用迭代法。G 称为解式 (8.2.1) 的 Gauss-Seidel 迭代法的迭代矩阵。

Gauss-Seidel 迭代法的一个明显的优点是, 在用计算机计算时, 只需一组工作单元, 以便存放近似解。由式 (8.2.5) ' 可以看出, 每迭代一步只需计算一次矩阵与向量的乘法。

例 8.2 用 Gauss-Seidel 迭代法解例 8.1。取 $x^{(0)} = 0$, 迭代公式为

$$\begin{cases} x_1^{(k+1)} = (20 + 3x_2^{(k)} - 2x_3^{(k)})/8, \\ x_2^{(k+1)} = (33 - 4x_1^{(k+1)} + x_3^{(k)})/11, \\ x_3^{(k+1)} = (36 - 6x_1^{(k+1)} - 3x_2^{(k+1)})/12. \end{cases}$$

迭代到第 5 次时, 有

$$x^{(5)} = (2.999\,843, 2.000\,072, 1.000\,061)^T, \quad \|\varepsilon^{(5)}\|_\infty = 0.001\,57.$$

从此例看出, Gauss-Seidel 迭代法比 Jacobi 迭代法收敛快 (达到同样的精度所需的迭代次数较少), 但这个结论在一定条件下才是对的。甚至有这样的方程组, Jacobi 方法收敛, 而 Gauss-Seidel 迭代法却是发散的。

例 8.3 方程组

$$\begin{cases} x_1 + 2x_2 - 2x_3 = 1, \\ x_1 + x_2 + x_3 = 1, \\ 2x_1 + 2x_2 + x_3 = 1 \end{cases}$$

能够说明解此方程组的 Jacobi 迭代法收敛而 Gauss-Seidel 迭代法发散。

8.3 迭代法的收敛性

定义 8.2 设有矩阵序列 $A_k = (a_{ij}^{(k)})_{n \times n}$ ($k = 1, 2, \dots$) 及 $A = (a_{ij})_{n \times n}$, 如果

$$\lim_{k \rightarrow \infty} a_{ij}^{(k)} = a_{ij} \quad (i, j = 1, 2, \dots, n)$$

成立, 则称 $\{A_k\}$ **收敛于** A , 记作 $\lim_{k \rightarrow \infty} A_k = A$ 。

例 8.4 矩阵序列

$$A = \begin{pmatrix} \lambda & 1 \\ 0 & \lambda \end{pmatrix}, \quad A^2 = \begin{bmatrix} \lambda^2 & 2\lambda \\ 0 & \lambda^2 \end{bmatrix}, \quad \dots, \quad A^k = \begin{bmatrix} \lambda^k & k\lambda^{k-1} \\ 0 & \lambda^k \end{bmatrix}, \quad \dots,$$

当 $|\lambda| < 1$ 时, $A^k \rightarrow \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$ (当 $k \rightarrow \infty$ 时)。

矩阵序列极限的概念可以用任何矩阵范数来描述。

校注 (agent 补充): 例 8.2 的原页误差范数明确印为 0.00157, 已保留。由本页所印近似向量与精确解 $(3, 2, 1)^T$ 计算, 无穷范数为 0.000157; 不作中间舍入而以有理数迭代 5 次, 误差无穷范数约为 0.0001576134。所印误差范数疑为原书小数位错误, 并非 OCR 错误。

原书 PDF 页 220; 印刷页码 207。

[原页图像](#)

定理 8.1 $\lim_{k \rightarrow \infty} A_k = A$ 的充要条件是 $\|A_k - A\| \rightarrow 0$ ($k \rightarrow \infty$)。证明过程留给读者进行。

由 8.1 节知, 要考虑序列 $\{x^{(k)}\}$ 的收敛性, 就要研究 B 在什么条件下误差向量趋于零向量, 即

$$\varepsilon^{(k)} = x^{(k)} - x^* = B^k \varepsilon^{(0)} \rightarrow 0 \quad (k \rightarrow \infty).$$

定理 8.2 设 $B = (b_{ij})_{n \times n}$, 则 $B^k \rightarrow O$ ($k \rightarrow \infty$) 的充要条件是 $\rho(B) < 1$ 。

证明 由矩阵 B 的 Jordan 标准形, 存在非奇异矩阵 P , 使

$$P^{-1}BP = \begin{bmatrix} J_1 & & \\ & J_2 & \\ & & \ddots \\ & & & J_r \end{bmatrix} \equiv J, \quad \text{其中 } J_i = \begin{bmatrix} \lambda_i & 1 & & \\ & \lambda_i & 1 & \\ & & \ddots & \ddots \\ & & & \ddots & 1 \\ & & & & \lambda_i \end{bmatrix}_{n_i \times n_i},$$

其中 $\sum_{i=1}^r n_i = n$ 。显然 $B = PJP^{-1}$, 故

$$B^k = PJ^kP^{-1},$$

其中

$$J^k = \begin{bmatrix} J_1^k & & \\ & J_2^k & \\ & & \ddots \\ & & & J_r^k \end{bmatrix} \quad (k = 1, 2, \dots).$$

于是

$$B^k \rightarrow O \quad (k \rightarrow \infty) \iff J^k \rightarrow O \quad (k \rightarrow \infty),$$

$$J^k \rightarrow O \quad (k \rightarrow \infty) \iff J_i^k \rightarrow O \quad (k \rightarrow \infty) \quad (i = 1, 2, \dots, r).$$

引进记号

$$E_{tk} = \begin{bmatrix} \overbrace{0 \cdots 0}^{k\text{个}} & 1 & 0 & \cdots & 0 \\ & \ddots & \ddots & \ddots & \vdots \\ & & \ddots & \ddots & 0 \\ & & & \ddots & 1 \\ & & & & 0 \\ & & & & \ddots \\ & & & & & 0 \end{bmatrix}_{t \times t} \quad (\text{其中 } t = n_i),$$

$$E_{tk} = \begin{bmatrix} \overbrace{0 \cdots 0}^{k\text{个}} & 1 & 0 & \cdots & 0 \\ & \ddots & \ddots & \ddots & \vdots \\ & & \ddots & \ddots & 0 \\ & & & \ddots & 1 \\ & & & & 0 \\ & & & & \ddots \\ & & & & & 0 \end{bmatrix}_{t \times t} \quad (\text{其中 } t = n_i),$$

原图矩阵符号标签: E_{tk} , 首行前 k 个元素为零 (括注“ k 个”), 随后为 $1, 0, \dots, 0$; 第 k 条上副对角线为 1, 其余为零; 矩阵尺寸 $t \times t$, 其中 $t = n_i$ 。

一般有 $(E_{t1})^k = E_{tk}$, 当 $k \geq t$ 时 $E_{tk} = O$ 。由于

$$J_i = \begin{bmatrix} \lambda_i & & & \\ & \lambda_i & & \\ & & \ddots & \\ & & & \ddots \\ & & & & \lambda_i \end{bmatrix} + \begin{bmatrix} 0 & 1 & & \\ & 0 & 1 & \\ & & \ddots & \ddots \\ & & & \ddots & 1 \\ & & & & 0 \end{bmatrix} = \lambda_i I + E_{t1},$$

原书 PDF 页 221; 印刷页码 208。

[原页图像](#)

于是

$$J_i^k = (\lambda_i I + E_{t1})^k = \sum_{j=0}^k \binom{k}{j} \lambda_i^{k-j} (E_{t1})^j = \sum_{j=0}^k \binom{k}{j} \lambda_i^{k-j} E_{tj} = \sum_{j=0}^{t-1} \binom{k}{j} \lambda_i^{k-j} E_{tj}$$

$$= \begin{bmatrix} \lambda_i^k & \binom{k}{1} \lambda_i^{k-1} & \cdots & \cdots & \binom{k}{t-1} \lambda_i^{k-(t-1)} \\ & \lambda_i^k & \binom{k}{1} \lambda_i^{k-1} & \cdots & \binom{k}{t-2} \lambda_i^{k-(t-2)} \\ & & \ddots & \ddots & \vdots \\ & & & \ddots & \binom{k}{1} \lambda_i^{k-1} \\ & & & & \lambda_i^k \end{bmatrix}_{t \times t} \quad (i = 1, 2, \dots, r),$$

其中

$$E_{t0} = I, \quad \binom{k}{j} = \frac{k!}{j!(k-j)!} = \frac{k(k-1)\cdots(k-j+1)}{j!}.$$

利用极限 $\lim_{k \rightarrow \infty} k^r c^k = 0$ ($0 < c < 1, r \geq 0$), 得到

$$\binom{k}{j} \lambda^{k-j} \rightarrow 0 \quad (k \rightarrow \infty) \iff |\lambda| < 1,$$

所以 $J_i^k \rightarrow O$ ($k \rightarrow \infty$) 充要条件是 $|\lambda_i| < 1$ ($i = 1, 2, \dots, r$), 即 $\rho(B) < 1$ 。

定理 8.3 (迭代法基本定理) 设有方程组

$$x = Bx + f, \quad (8.3.1)$$

对于任意初始向量 $x^{(0)}$ 及任意 f , 解此方程组的迭代法 (即 $x^{(k+1)} = Bx^{(k)} + f$) 收敛的充要条件是 $\rho(B) < 1$ 。

证明 充分性。设 $\rho(B) < 1$, 易知 $Ax = f$ (其中 $A = I - B$) 有唯一解, 记作

$$x^* = Bx^* + f. \quad (8.3.2)$$

误差向量

$$\varepsilon^{(k)} = x^{(k)} - x^* = B^k \varepsilon^{(0)}, \quad \varepsilon^{(0)} = x^{(0)} - x^*.$$

由设 $\rho(B) < 1$, 应用定理 8.2, 有 $B^k \rightarrow O$ ($k \rightarrow \infty$)。于是对于任意 $x^{(0)}$ 及 f , 有 $\varepsilon^{(k)} \rightarrow 0$ ($k \rightarrow \infty$), 即 $x^{(k)} \rightarrow x^*$ ($k \rightarrow \infty$)。

必要性。设对于任意 $x^{(0)}$ 及任意 f , 皆有 $\lim_{k \rightarrow \infty} x^{(k)} = x^*$, 其中 $x^{(k+1)} = Bx^{(k)} + f$ 。显然,

(1) 极限 x^* 是方程组 (8.3.1) 的解,

(2) 对于任意 $x^{(0)}$ 及任意 f , 有

$$\varepsilon^{(k)} = x^{(k)} - x^* = B^k \varepsilon^{(0)} \rightarrow 0 \quad (k \rightarrow \infty).$$

显然

$$B^k \rightarrow O \quad (k \rightarrow \infty).$$

再由定理 8.2, 即得 $\rho(B) < 1$ 。

原书 PDF 页 222; 印刷页码 209。

[原页图像](#)

例 8.5 考察用 Jacobi 方法解例 8.1 方程组的收敛性。迭代矩阵 B_0 的特征方程为

$$\det(\lambda I - B_0) = \lambda^3 + 0.034\,090\,909\lambda + 0.039\,772\,727 = 0,$$

解得

$$\begin{aligned}\lambda_1 &= -0.308\,2, \\ \lambda_2 &= 0.154\,1 + \mathrm{i}0.324\,5, \\ \lambda_3 &= 0.154\,1 - \mathrm{i}0.324\,5, \\ |\lambda_2| &= |\lambda_3| = 0.359\,2 < 1, \quad |\lambda_1| < 1,\end{aligned}$$

即 $\rho(B_0) < 1$ 。所以用 Jacobi 迭代法解方程组 (8.3.1) 是收敛的。

例 8.6 考察用迭代过程的收敛性: $x^{(k+1)} = Bx^{(k)} + f$ ($k = 0, 1, 2, \dots$), 其中

$$B = \begin{pmatrix} 0 & 2 \\ 3 & 0 \end{pmatrix}, \quad f = \begin{pmatrix} 5 \\ 5 \end{pmatrix}.$$

特征方程为

$$\det(\lambda I - B) = \lambda^2 - 6 = 0,$$

特征根 $\lambda_{1,2} = \pm\sqrt{6}$, 即 $\rho(B) > 1$ 。这说明此迭代过程不收敛。

现讨论迭代法的收敛速度。考察误差向量 $\varepsilon^{(k)} = x^{(k)} - x^* = B^k \varepsilon^{(0)}$ 。设 B 有 n 个线性无关的特征向量 $u_1, u_2, \dots, u_n$, 相应的特征值为 $\lambda_1, \lambda_2, \dots, \lambda_n$ 。由 $\varepsilon^{(0)} = \sum_{i=1}^n a_i u_i$, 得

$$\varepsilon^{(k)} = B^k \varepsilon^{(0)} = \sum_{i=1}^n a_i B^k u_i = \sum_{i=1}^n a_i \lambda_i^k u_i.$$

可以看出, 当 $\rho(B) < 1$ 愈小时, $\lambda_i^k \rightarrow 0$ ($i = 1, 2, \dots, n; k \rightarrow \infty$) 愈快, 即 $\varepsilon^{(k)} \rightarrow 0$ 愈快, 故可用量 $\rho(B)$ 来刻画迭代法的收敛快慢。现在依据给定精度要求确定迭代次数 k , 即使

$$[\rho(B)]^k \leqslant 10^{-s}, \quad (8.3.3)$$

取对数得

$$k \geqslant \frac{s \ln 10}{-\ln \rho(B)}.$$

定义 8.3 称 $R(B) = -\ln \rho(B)$ 为迭代法的收敛速度。

可以看出, $\rho(B) < 1$ 愈小, $-\ln \rho(B)$ 就愈大, 式 (8.3.3) 成立所需迭代次数就愈少。

由于一般当 n 较大时, 矩阵特征值计算比较困难, 基本定理的条件比较难验证, 所以最好建立与矩阵元素直接有关的条件来判别迭代法的收敛性。由于 $\rho(B) \leq \|B\|_v$ ($v = 1, 2, \infty$ 或 F), 所以可用 $\|B\|_v$ 来作为 $\rho(B)$ 上界的一种估计。

定理 8.4 (迭代法收敛的充分条件) 如果方程组 (8.3.1) 的迭代公式为 $x^{(k+1)} = Bx^{(k)} + f$ ($x^{(0)}$ 为任意初始向量), 且迭代矩阵的某一种范数 $\|B\|_v = q < 1$, 则:

1° 迭代法收敛;

原书 PDF 页 223; 印刷页码 210。

[原页图像](#)

$$2^\circ \quad \|x^* - x^{(k)}\|_v \leq \frac{q}{1-q} \|x^{(k)} - x^{(k-1)}\|_v;$$

$$3^\circ \quad \|x^* - x^{(k)}\|_v \leq \frac{q^k}{1-q} \|x^{(1)} - x^{(0)}\|_v.$$

证明 利用定理 8.3, 结论 1° 是显然的。现证结论 2° 、 3° 。据题设显然有

$$x^* - x^{(k+1)} = B(x^* - x^{(k)}), \quad (8.3.4)$$

$$\|x^{(k+1)} - x^{(k)}\|_v \leq q \|x^{(k)} - x^{(k-1)}\|_v \quad (k = 1, 2, \dots), \quad (8.3.5)$$

$$\|x^* - x^{(k+1)}\|_v \leq q \|x^* - x^{(k)}\|_v. \quad (8.3.6)$$

于是

$$\begin{aligned} \|x^{(k+1)} - x^{(k)}\|_v &= \|x^* - x^{(k)} - (x^* - x^{(k+1)})\|_v \\ &\geq \|x^* - x^{(k)}\|_v - \|x^* - x^{(k+1)}\|_v \\ &\geq (1-q) \|x^* - x^{(k)}\|_v, \end{aligned}$$

即

$$\begin{aligned} \|x^* - x^{(k)}\|_v &\leq \frac{1}{1-q} \|x^{(k+1)} - x^{(k)}\|_v \\ &\leq \frac{q}{1-q} \|x^{(k)} - x^{(k-1)}\|_v \quad (k = 1, 2, \dots). \end{aligned}$$

反复利用式 (8.3.5) 即得到结论 3° 。

例 8.7 仍然考察例 8.1, Jacobi 方法迭代矩阵 B_0 的 ∞ -范数为

$$\|B_0\|_\infty = \max \left\{ \frac{5}{8}, \frac{5}{11}, \frac{9}{12} \right\} = \frac{9}{12} < 1,$$

所以对例 8.1 应用 Jacobi 迭代方法是收敛的。

例 8.8 设有迭代过程 $x^{(k+1)} = Bx^{(k)} + f$ ($k = 0, 1, 2, \dots$), 其中

$$B = \begin{pmatrix} 0.9 & 0 \\ 0.3 & 0.8 \end{pmatrix}, \quad f = \begin{pmatrix} 1 \\ 2 \end{pmatrix},$$

则

$$\|B\|_\infty = 1.1, \quad \|B\|_1 = 1.2, \quad \|B\|_2 = 1.021, \quad \|B\|_F = \sqrt{1.54}.$$

虽然 B 的这些范数都大于 1, 但 B 的特征值为 $\lambda_1 = 0.9, \lambda_2 = 0.8$, 由定理 8.3 知, 此迭代过程还是收敛的。

由定理 8.4 可知, $\|B\| = q < 1$ 愈小, 迭代法收敛愈快。

当 B 的某一种范数 $\|B\| < 1$ 时, 如果相邻两次迭代 $\|x^{(k)} - x^{(k-1)}\| < \varepsilon_0$ (ε_0 为给定的精度要求), 则 $\|x^* - x^{(k)}\| \leq \frac{q}{1-q} \varepsilon_0$, 所以在用计算机计算时通常利用 $\|x^{(k)} - x^{(k-1)}\| < \varepsilon_0$ 来作为控制迭代的终止条件。不过要注意, 当 $q \approx 1$ 时, $\frac{q}{1-q}$ 较大, 尽管 $\|x^{(k)} - x^{(k-1)}\|$ 已非常小, 但误差向量的模 $\|\varepsilon^{(k)}\| = \|x^* - x^{(k)}\|$ 可能较大, 迭代法收敛将是缓慢的。定理 8.4 中结论 3° 的估计还可以用来事先确定需要迭代多少次才能保证 $\|\varepsilon^{(k)}\| < \varepsilon_0$ 。

定理 8.5 解方程组 (8.2.1) 的 Gauss-Seidel 迭代法收敛的充要条件是 $\rho(G) < 1$,

原书 PDF 页 224; 印刷页码 211。

[原页图像](#)

其中 G 为 Gauss-Seidel 迭代法的迭代矩阵。

证明 由式 (8.2.6) 再应用定理 8.3 即得定理 8.5。

在实际应用中常遇到一些线性代数方程组, 其系数矩阵具有某些性质, 如系数矩阵的对角元素占优, 系数矩阵为对称正定等。充分利用这些性质往往可使判定迭代法收敛的问题变得简单。

定义 8.4 (对角占优阵) 设 $A = (a_{ij})_{n \times n} \in \mathbb{R}^{n \times n}$ (或 $\mathbb{C}^{n \times n}$),

1° 如果矩阵 A 满足条件

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| \quad (i = 1, 2, \dots, n), \quad (8.3.7)$$

即 A 的每一行对角元素的绝对值都严格大于同行其他元素绝对值之和, 则称 A 为**严格对角占优阵**。

2° 如果 $|a_{ii}| \geq \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|$ ($i = 1, 2, \dots, n$) 且至少有一个不等式严格成立, 称 A 为**弱对角占优阵**。

例 8.9 $A = \begin{bmatrix} -4 & 1 & 0 & 0 \\ 1 & -4 & 1 & 0 \\ 0 & 1 & -4 & 1 \\ 0 & 0 & 1 & -4 \end{bmatrix}$ 为严格对角占优阵, $B = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 2 \end{bmatrix}$ 为弱对角占优阵。

定义 8.5 (可约与不可约矩阵) 设 $A = (a_{ij})_n \in \mathbb{R}^{n \times n}$ (或 $\mathbb{C}^{n \times n}$), 当 $n \geq 2$ 时, 如果存在 n 阶置换阵 P 使

$$P^T A P = \begin{bmatrix} A_{11} & A_{12} \\ O & A_{22} \end{bmatrix} \quad (8.3.8)$$

成立, 其中 A_{11} 为 r 阶子矩阵, A_{22} 为 $n-r$ 阶子矩阵 ($1 \leq r \leq n$), 则称 A 是**可约矩阵**。如果不存在置换阵 P 使式 (8.3.8) 成立, 则称 A 是**不可约矩阵**。

A 是可约矩阵, 意味着 $Ax = b$ 可经过若干行列重排 (若 A 经过两行交换的同时进行相应的两列的交换, 称对 A 进行一次行列重排), 化为两个低阶方程组求解。

事实上, 由 $Ax = b$ 可化为 $P^T A P (P^T x) = P^T b$, 且记

$$y = P^T x = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix}, \quad P^T b = \begin{pmatrix} d_1 \\ d_2 \end{pmatrix},$$

其中 y_1, d_1 为 r 维向量。于是, 求解 $Ax = b$ 化为求解

$$\begin{cases} A_{11}y_1 + A_{12}y_2 = d_1, \\ A_{22}y_2 = d_2. \end{cases}$$

例 8.10 在例 8.9 中矩阵 B 是可约矩阵, 即存在置换阵 $P = I_{13}$, 使

校注 (agent 补充): 定义 8.5 的原页范围明确印为 $1 \leq r \leq n$, 已保留。可约分块需要两个非空低阶方阵, 通常应要求 $1 \leq r \leq n-1$; 若容许 $r = n$, 第二块阶数为零, 定义不能区分可约与不可约。此处疑为原书边界排印错误, 并非 OCR 错误。

原书 PDF 页 225; 印刷页码 212。

[原页图像](#)

$$P^T B P = \left[\begin{array}{c|cc} 2 & 1 & 0 \\ \hline 0 & 1 & 1 \\ 0 & 1 & 1 \end{array} \right].$$

定理 8.6 (对角占优定理) 如果 $A = (a_{ij})_n \in \mathbb{R}^{n \times n}$ (或 $\mathbb{C}^{n \times n}$) 为严格对角占优阵或为不可约弱对角占优阵, 则 A 是非奇异矩阵。

证明 设 A 为严格对角占优阵, 下面只就这种情况证明此定理。若 $\det A = 0$, 则 $Ax = 0$ 有非零解, 记作

$$x = (x_1, x_2, \dots, x_n)^T.$$

又记 $|x_k| = \max_{1 \leq i \leq n} |x_i| \neq 0$, 由齐次方程组的第 k 个方程

$$\sum_{j=1}^n a_{kj} x_j = 0,$$

得

$$\begin{aligned} |a_{kk} x_k| &= \left| \sum_{\substack{j=1 \\ j \neq k}}^n a_{kj} x_j \right| \leq \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}| |x_j| \leq |x_k| \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}|, \\ |a_{kk}| &\leq \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}|, \end{aligned} \tag{8.3.9}$$

与假设矛盾, 故 $\det A \neq 0$, 即 A 非奇异。

定理 8.7 如果 $A \in \mathbb{R}^{n \times n}$ 为严格对角占优阵或为不可约弱对角占优阵, 则对于任意的 $x^{(0)}$, 解方程组 (8.2.1) 的 Jacobi 迭代法、Gauss-Seidel 迭代法均收敛。

证明 设 A 为不可约弱对角占优阵, 现证明 Gauss-Seidel 迭代法收敛。其他证明留作习题。

由设知 $a_{ii} \neq 0$ ($i = 1, 2, \dots, n$), 方程组 (8.2.1) 的 Gauss-Seidel 迭代法的迭代矩阵为

$$G = (D - L)^{-1} U.$$

又

$$\begin{aligned} \det(\lambda I - G) &= \det(\lambda I - (D - L)^{-1} U) \\ &= \det(D - L)^{-1} \cdot \det(\lambda(D - L) - U) = 0, \end{aligned}$$

即

$$\det(\lambda(D - L) - U) = 0.$$

记

$$G = \lambda(D - L) - U = \begin{bmatrix} \lambda a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ \lambda a_{21} & \lambda a_{22} & a_{23} & \cdots & a_{2n} \\ \vdots & \lambda a_{32} & \ddots & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & a_{n-1,n} \\ \lambda a_{n1} & \lambda a_{n2} & \cdots & \lambda a_{n,n-1} & \lambda a_{nn} \end{bmatrix} = (c_{ij})_{n \times n}.$$

下面说明当 $|\lambda| \geq 1$ 时 $\det G \neq 0$ ，如果这个结论是正确的，那么 $\det G = 0$ 的根均满足 $|\lambda| < 1$ 。这说明解式 (8.2.1) 的 Gauss-Seidel 迭代法收敛。

事实上，由 A 为不可约矩阵，则 G 亦为不可约矩阵，且又由 A 为弱对角占优阵得

校注 (agent 补充): 本页先用 G 表示 Gauss-Seidel 迭代矩阵，随后在证明中又记 $G = \lambda(D - L) - U$ 。原页第二处字母无上横线或其他修饰，转录保留此符号重用；后面的 $\det G$ 指随后定义的矩阵。

原书 PDF 页 226; 印刷页码 213。

[原页图像](#)

到 (当 $|\lambda| \geq 1$ 时)

$$|c_{ii}| = |\lambda| |a_{ii}| \geq \sum_{j=1}^{i-1} |\lambda a_{ij}| + \sum_{j=i+1}^n |a_{ij}| = \sum_{j \neq i} |c_{ij}|,$$

且至少有一不等式严格成立, 也就是说当 $|\lambda| \geq 1$ 时, G 为不可约弱对角占优阵, 于是由定理 8.6 (当 $|\lambda| \geq 1$ 时) 有 $\det G \neq 0$, 故 $\rho(G) < 1$ 。

8.4 解线性方程组的超松弛迭代法

逐次超松弛迭代法 (successive over relaxation method, 简称 SOR 方法) 是 Gauss-Seidel 方法的一种加速方法, 是解大型稀疏矩阵方程组的有效方法之一, 它具有计算公式简单, 程序设计容易, 占用计算机内存较少等优点, 但需要选择好的加速因子 (即最佳松弛因子)。

设有方程组

$$Ax = b, \quad (8.4.1)$$

其中 $A \in \mathbb{R}^{n \times n}$ 为非奇异矩阵, 且设 $a_{ii} \neq 0$ ($i = 1, 2, \dots, n$), 分解 A 为

$$A = D - L - U. \quad (8.4.2)$$

设已知第 k 次迭代向量 $x^{(k)}$, 及第 $k+1$ 次迭代向量 $x^{(k+1)}$ 的分量 $x_j^{(k+1)}$ ($j = 1, 2, \dots, i-1$), 要求计算分量 $x_i^{(k+1)}$ 。

首先用 Gauss-Seidel 迭代法定义辅助量

$$\tilde{x}_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right) \quad (i = 1, 2, \dots, n), \quad (8.4.3)$$

再把 $x_i^{(k+1)}$ 取为 $x_i^{(k)}$ 与 $\tilde{x}_i^{(k+1)}$ 某个平均值 (即加权平均), 即

$$x_i^{(k+1)} = (1 - \omega) x_i^{(k)} + \omega \tilde{x}_i^{(k+1)} = x_i^{(k)} + \omega (\tilde{x}_i^{(k+1)} - x_i^{(k)}). \quad (8.4.4)$$

用式 (8.4.3) 代入式 (8.4.4) 即得到解方程组 $Ax = b$ 的逐次超松弛迭代公式

$$\begin{cases} x_i^{(k+1)} = x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i}^n a_{ij} x_j^{(k)} \right), \\ x^{(k)} = (x_1^{(k)}, x_2^{(k)}, \dots, x_n^{(k)})^T \quad (k = 0, 1, \dots; i = 1, 2, \dots, n). \end{cases} \quad (8.4.5)$$

其中 ω 称为**松弛因子**, 或写为

$$\begin{cases} x_i^{(k+1)} = x_i^{(k)} + \Delta x_i & (k = 0, 1, \dots; i = 1, 2, \dots, n), \\ \Delta x_i = \frac{\omega}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i}^n a_{ij} x_j^{(k)} \right). \end{cases} \quad (8.4.6)$$

显然, 当 $\omega = 1$ 时, 解式 (8.4.1) 的 SOR 方法就是 Gauss-Seidel 迭代法。

在 SOR 方法中, 迭代一次主要的运算量是计算一次矩阵与向量的乘法。由式 (8.4.5) 可知, 在计算机上应用 SOR 方法解方程组时只需一组工作单元, 以便存放近似

原书 PDF 页 227; 印刷页码 214。

[原页图像](#)

解。在用计算机计算时, 可用 $|p_0| = \max_i |\Delta x_i| = \max_{1 \leq i \leq n} |x_i^{(k+1)} - x_i^{(k)}| < \varepsilon$ 控制迭代终止。

当 $\omega < 1$ 时, 称式 (8.4.5) 为**低松弛法**; 当 $\omega > 1$ 时, 称式 (8.4.5) 为**超松弛法**。

例 8.11 用 SOR 方法解方程组

$$\begin{bmatrix} -4 & 1 & 1 & 1 \\ 1 & -4 & 1 & 1 \\ 1 & 1 & -4 & 1 \\ 1 & 1 & 1 & -4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix},$$

它的精确解为 $x^* = (-1, -1, -1, -1)^T$ 。

解 取 $x^{(0)} = 0$, 迭代公式为

$$\begin{cases} x_1^{(k+1)} = x_1^{(k)} - \omega(1 + 4x_1^{(k)} - x_2^{(k)} - x_3^{(k)} - x_4^{(k)})/4, \\ x_2^{(k+1)} = x_2^{(k)} - \omega(1 - x_1^{(k+1)} + 4x_2^{(k)} - x_3^{(k)} - x_4^{(k)})/4, \\ x_3^{(k+1)} = x_3^{(k)} - \omega(1 - x_1^{(k+1)} - x_2^{(k+1)} + 4x_3^{(k)} - x_4^{(k)})/4, \\ x_4^{(k+1)} = x_4^{(k)} - \omega(1 - x_1^{(k+1)} - x_2^{(k+1)} - x_3^{(k+1)} + 4x_4^{(k)})/4. \end{cases}$$

取 $\omega = 1.3$, 第 11 次迭代结果为

$$x^{(11)} = (-0.999\,996\,46, -1.000\,003\,10, -0.999\,999\,53, -0.999\,999\,12)^T,$$

$$\|\varepsilon^{(11)}\|_2 \leq 0.46 \times 10^{-5}.$$

对 ω 取其他值, 迭代次数如表 8.1 所示。从此例可以看到, 松弛因子选择得好, 会使 SOR 方法的收敛大大加速。本例中, $\omega = 1.3$ 是最佳松弛因子。

表 8.1

松弛因子 ω	满足误差 $\ x^{(k)} - x^*\ _2 < 10^{-5}$ 的迭代次数
1.0	22
1.1	17
1.2	12
<u>1.3</u>	11 (最少迭代次数)
1.4	14
1.5	17
1.6	23
1.7	33
1.8	53
1.9	109

下面写出 SOR 迭代公式的矩阵形式。迭代公式 (8.4.5) 亦可写为

$$a_{ii}x_i^{(k+1)} = (1-\omega)a_{ii}x_i^{(k)} + \omega \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right) \quad (i=1, 2, \dots, n). \quad (8.4.7)$$

用式 $A = D - L - U$, 则

$$Dx^{(k+1)} = \omega(b + Lx^{(k+1)} + Ux^{(k)}) + (1-\omega)Dx^{(k)},$$

即

$$(D - \omega L)x^{(k+1)} = ((1-\omega)D + \omega U)x^{(k)} + \omega b.$$

显然对于任何一个 ω 值, $D - \omega L$ 非奇异 (由设 $a_{ii} \neq 0$; $i=1, 2, \dots, n$), 于是

$$x^{(k+1)} = (D - \omega L)^{-1}[(1-\omega)D + \omega U]x^{(k)} + \omega(D - \omega L)^{-1}b.$$

这就是说, 解式 (8.2.1) (设 $a_{ii} \neq 0$; $i=1, 2, \dots, n$) 的 SOR 方法迭代公式为

$$x^{(k+1)} = L_\omega x^{(k)} + f. \quad (8.4.8)$$

校注 (agent 补充): 原页第 11 次迭代向量及误差界均按原样保留。按所印四个分量计算, $\|x^{(11)} - x^*\|_2 \approx 4.8100832 \times 10^{-6}$, 超过所印上界 0.46×10^{-5} 。以 $\omega = 13/10$ 从零向量作 11 次有理数迭代, 所得向量约为 $(-0.9999966722, -1.0000028727, -0.9999995354, -0.9999991925)^T$, 误差约为 4.4938646×10^{-6} ; 因此本页所印向量与误差界存在数值不一致, 并非 OCR 错误。

原书 PDF 页 228; 印刷页码 215。

[原页图像](#)

其中

$$L_{\omega} = (D - \omega L)^{-1}[(1 - \omega)D + \omega U], \quad f = \omega(D - \omega L)^{-1}b.$$

矩阵 L_{ω} 称为 SOR 方法的迭代矩阵。这说明对式 (8.2.1) 应用 SOR 方法相当于对方程组 $x = L_{\omega}x + f$ 应用一般迭代法。于是关于一般迭代法的理论可应用到式 (8.4.8), 得到下述定理。

定理 8.8 设有线性方程组 (8.4.1), 且 $a_{ii} \neq 0$ ($i = 1, 2, \dots, n$), 则解方程组的 SOR 方法收敛的充要条件是

$$\rho(L_{\omega}) < 1.$$

引进超松弛迭代法的想法是希望能选择松弛因子 ω 使得迭代过程式 (8.4.5) 收敛较快, 也就是应选择因子 ω 使 $\rho(L_{\omega}) = \min_{\omega}$ 。

下面研究, 对于一般方程组 (8.2.1) ($a_{ii} \neq 0; i = 1, 2, \dots, n$), 松弛因子 ω 在什么范围内取值, SOR 方法才可能收敛。现给出 SOR 方法收敛的必要条件。

定理 8.9 设解式 (8.2.1) ($a_{ii} \neq 0; i = 1, 2, \dots, n$) 的 SOR 方法收敛, 则

$$0 < \omega < 2.$$

证明 由设 SOR 方法收敛, 根据定理 8.8, $\rho(L_{\omega}) < 1$ 。

设 L_{ω} 的特征值为 $\lambda_1, \lambda_2, \dots, \lambda_n$, 则

$$|\det L_{\omega}| = |\lambda_1 \lambda_2 \cdots \lambda_n| \leq (\rho(L_{\omega}))^n, \quad |\det L_{\omega}|^{1/n} \leq \rho(L_{\omega}) < 1.$$

而

$$\det L_{\omega} = \det((D - \omega L)^{-1}) \cdot \det((1 - \omega)D + \omega U) = (1 - \omega)^n,$$

所以

$$|1 - \omega| < 1.$$

该定理说明对于解一般方程组 (8.2.1) ($a_{ii} \neq 0; i = 1, 2, \dots, n$), SOR 方法只有取松弛因子 ω 在 $(0, 2)$ 范围内才能收敛。而当 A 是对称正定矩阵时, 若 ω 满足 $0 < \omega < 2$, 则 SOR 方法一定收敛。

定理 8.10 如果 A 为对称正定矩阵, 且 $0 < \omega < 2$, 则解式 (8.4.1) 的 SOR 方法收敛。

证明 在上述假定下, 若能证明 $|\lambda| < 1$, 那么定理得证 (其中 λ 为 L_{ω} 的任一特征值)。

事实上, 设 y 为对应 λ 的 L_{ω} 的特征向量, 即

$$L_{\omega}y = \lambda y, \quad y = (y_1, y_2, \dots, y_n)^T \neq 0,$$

$$(D - \omega L)^{-1}[(1 - \omega)D + \omega U]y = \lambda y,$$

亦即

$$[(1 - \omega)D + \omega U]y = \lambda(D - \omega L)y.$$

为了找出 λ 的表达式, 考虑数量积

$$(((1 - \omega)D + \omega U)y, y) = \lambda((D - \omega L)y, y),$$

则

$$\lambda = \frac{(Dy, y) - \omega(Dy, y) + \omega(Uy, y)}{(Dy, y) - \omega(Ly, y)},$$

原书 PDF 页 229; 印刷页码 216。

[原页图像](#)

显然

$$(Dy, y) = \sum_{i=1}^n a_{ii}|y_i|^2 \equiv \sigma > 0, \quad (8.4.9)$$

记 $-(Ly, y) = \alpha + i\beta$, 由于 $A = A^T$, 所以 $U = L^T$ 。

$$-(Uy, y) = -(y, Ly) = -\overline{(Ly, y)} = \alpha - i\beta,$$

$$0 < (Ay, y) = ((D - L - U)y, y) = \sigma + 2\alpha, \quad (8.4.10)$$

所以

$$\lambda = \frac{(\sigma - \omega\sigma - \alpha\omega) + i\omega\beta}{(\sigma + \alpha\omega) + i\omega\beta},$$

从而

$$|\lambda|^2 = \frac{(\sigma - \omega\sigma - \alpha\omega)^2 + \omega^2\beta^2}{(\sigma + \alpha\omega)^2 + \omega^2\beta^2}.$$

当 $0 < \omega < 2$ 时, 利用式 (8.4.9)、式 (8.4.10), 有

$$(\sigma - \omega\sigma - \alpha\omega)^2 - (\sigma + \alpha\omega)^2 = \omega\sigma(\sigma + 2\sigma)(\omega - 2) < 0,$$

即 L_ω 的任一特征值满足 $|\lambda| < 1$, 故 SOR 方法收敛。注意, 当 $0 < \omega < 2$ 时, 可以证明

$$(\sigma + 2\omega)^2 + \omega^2\beta^2 \neq 0.$$

最佳松弛因子理论是由 Young (1950 年) 针对一类椭圆型微分方程数值解得到的代数方程组 $Ax = b$ (具有所谓性质 A 和相容次序) 所建立的理论。他给出了最佳松弛因子公式

$$\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - \rho^2(B_0)}},$$

其中 $\rho(B_0)$ 是 Jacobi 方法迭代矩阵 B_0 的谱半径。

一般来说, 在实际应用中计算 $\rho(B_0)$ 较困难, 对某些微分方程数值解问题可考虑用第 9 章求近似值的方法, 亦可由计算实践摸索出 (近似) 最佳松弛因子。

算法 本算法用 SOR 方法解式 (8.2.1), 其中 A 为对称正定矩阵。数组 x 为一组工作单元, 开始存放初始向量, 然后存放近似值解 $x^{(k)}$, 最后存放结果。用

$$|p_0| = \max_{1 \leq i \leq n} |\Delta x_i| = \max_{1 \leq i \leq n} |x_i^{(k+1)} - x_i^{(k)}| < \varepsilon \quad (\text{精度要求})$$

控制迭代终止。 k 表示迭代次数, 可以不用。

步 1 $k \leftarrow 0$ 。

步 2 $x_i \leftarrow 0 \quad (i = 1, 2, \dots, n)$ 。

步 3 $k \leftarrow k + 1$ 。

步 4 $p_0 \leftarrow 0$ 。

步 5 对于 $i = 1, 2, \dots, n$, 有

$$(1) \quad p \leftarrow \Delta x_i = \omega \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j - \sum_{j=i+1}^n a_{ij}x_j \right) / a_{ii};$$

(2) 如果 $|p| > |p_0|$, 则 $p_0 \leftarrow p$;

(3) $x_i \leftarrow x_i + p$ 。

步 6 输出 p_0 。

校注(agent 补充): 本页存在以下三处疑似原书排印错误, 均已放大核对并保留原式, 并非 OCR 错误。第一, 平方差的因子原印 $\sigma+2\sigma$, 代数展开应得 $\sigma+2\alpha$, 与式 (8.4.10) 一致。第二, 分母非零说明原印 $(\sigma+2\omega)^2+\omega^2\beta^2 \neq 0$, 由上文 $|\lambda|^2$ 表达式, 相应分母应为 $(\sigma+\alpha\omega)^2+\omega^2\beta^2$ 。第三, 算法步 5 (1) 的第二个求和下限原印 $j = i + 1$; 由于随后执行 $x_i \leftarrow x_i + p$, $p = \Delta x_i$ 应是校正量, 对照式 (8.4.6), 该下限应为 $j = i$ (或另减去 $a_{ii}x_i$), 否则漏掉对角项。

原书 PDF 页 230; 印刷页码 217。

[原页图像](#)

步 7 如果 $|p_0| > \varepsilon$, 则转步 3。

步 8 输出结果 x, k 。

小结

本章介绍了解线性方程组 $Ax = b$ 迭代法的一些基本理论及 Jacobi 迭代法、Gauss-Seidel 迭代法、SOR 迭代法, 这三种方法都是一阶定常迭代法。在应用中 SOR 方法较为重要, 它是解大型稀疏矩阵方程组的有效方法之一。

迭代法是一种逐次逼近方法, 在使用迭代法解方程组时, 其系数矩阵在计算过程中始终不变。

迭代法具有循环的计算公式, 方法简单, 适宜解大型稀疏矩阵方程组, 在用计算机计算时只需存储 A 的非零元素 (或可按一定公式形成系数, 这样 A 就不需要存储)。

在使用迭代法时, 要注意收敛性及收敛速度问题, 使用 SOR 方法要选择较佳松弛因子。

迭代法的进一步学习, 读者可参看文献 [11]、[15]、[16]。

习题

1. 设方程组

$$\begin{cases} 5x_1 + 2x_2 + x_3 = -12, \\ -x_1 + 4x_2 + 2x_3 = 20, \\ 2x_1 - 3x_2 + 10x_3 = 3. \end{cases}$$

(1) 考察用 Jacobi 迭代法、Gauss-Seidel 迭代法解此方程组的收敛性;

(2) 用 Jacobi 迭代法、Gauss-Seidel 迭代法解此方程组, 要求当 $\|x^{(k+1)} - x^{(k)}\|_\infty < 10^{-4}$ 时迭代终止。

2. 设 $A = \begin{pmatrix} 0 & 0 \\ 2 & 0 \end{pmatrix}$, 证明即使 $\|A\|_1 = \|A\|_\infty > 1$, 级数 $I + A + A^2 + \cdots + A^k + \cdots$ 也收敛。

3. 证明对于任意选择的 A , 序列 $I, A, \frac{1}{2}A^2, \frac{1}{3!}A^3, \frac{1}{4!}A^4, \dots$ 收敛于零。

4. 设方程组

$$\begin{cases} a_{11}x_1 + a_{12}x_2 = b_1, \\ a_{21}x_1 + a_{22}x_2 = b_2 \end{cases} \quad (a_{11} \cdot a_{22} \neq 0).$$

迭代公式为

$$\begin{cases} x_1^{(k)} = \frac{1}{a_{11}}(b_1 - a_{12}x_2^{(k-1)}), \\ x_2^{(k)} = \frac{1}{a_{22}}(b_2 - a_{21}x_1^{(k-1)}) \end{cases} \quad (k = 1, 2, \dots).$$

求证：由上述迭代公式产生的向量序列 $\{x^{(k)}\}$ 收敛的充要条件是 $r = \left| \frac{a_{12}a_{21}}{a_{11}a_{22}} \right| < 1$ 。

5. 设方程组

原书 PDF 页 231; 印刷页码 218。

[原页图像](#)

(1)

$$\begin{cases} x_1 + 0.4x_2 + 0.4x_3 = 1, \\ 0.4x_1 + x_2 + 0.8x_3 = 2, \\ 0.4x_1 + 0.8x_2 + x_3 = 3; \end{cases}$$

(2)

$$\begin{cases} x_1 + 2x_2 - 2x_3 = 1, \\ x_1 + x_2 + x_3 = 1, \\ 2x_1 + 2x_2 + x_3 = 1. \end{cases}$$

试考察解此方程组的 Jacobi 迭代法及 Gauss-Seidel 迭代法的收敛性。

6. 求证 $\lim_{k \rightarrow \infty} A_k = A$ 的充要条件是, 对于任何向量 x 都有 $\lim_{k \rightarrow \infty} A_k x = Ax$ 。

7. 设 $Ax = b$, 其中 A 为对称正定矩阵, 问解此方程组的 Jacobi 迭代法是否一定收敛。试考察习题 5 中方程组 (1)。

8. 设方程组

$$\begin{cases} x_1 - \frac{1}{4}x_3 - \frac{1}{4}x_4 = \frac{1}{2}, \\ x_2 - \frac{1}{4}x_3 - \frac{1}{4}x_4 = \frac{1}{2}, \\ -\frac{1}{4}x_1 - \frac{1}{4}x_2 + x_3 = \frac{1}{2}, \\ -\frac{1}{4}x_1 - \frac{1}{4}x_2 + x_4 = \frac{1}{2}. \end{cases}$$

(1) 求解此方程组的 Jacobi 迭代法的迭代矩阵 B_0 的谱半径;

(2) 求解此方程组的 Gauss-Seidel 迭代法的迭代矩阵 B_0 的谱半径;

(3) 考察解此方程组的 Jacobi 迭代法及 Gauss-Seidel 迭代法的收敛性。

9. 用 SOR 方法解方程组 (分别取松弛因子 $\omega = 1.03, \omega = 1, \omega = 1.1$)

$$\begin{cases} 4x_1 - x_2 = 1, \\ -x_1 + 4x_2 - x_3 = 4, \\ -x_2 + 4x_3 = -3. \end{cases}$$

精确解 $x^* = \left(\frac{1}{2}, 1, -\frac{1}{2}\right)^T$, 要求当 $\|x^* - x^{(k)}\|_\infty < 5 \times 10^{-6}$ 时迭代终止, 并且对每一个 ω 值确定迭代次数。

10. 用 SOR 方法解方程组 (取 $\omega = 0.9$)

$$\begin{cases} 5x_1 + 2x_2 + x_3 = -12, \\ -x_1 + 4x_2 + 2x_3 = 20, \\ 2x_1 - 3x_2 + 10x_3 = 3, \end{cases}$$

要求当 $\|x^{(k+1)} - x^{(k)}\|_\infty < 10^{-4}$ 时迭代终止。

11. 设有方程组 $Ax = b$, 其中 A 为对称正定矩阵, 迭代公式 $x^{(k+1)} = x^{(k)} + \omega(b - Ax^{(k)})$ ($k = 0, 1, 2, \dots$), 试证明当 $0 < \omega < \frac{2}{\beta}$ 时上述迭代法收敛 (其中 $0 < \alpha \leq \lambda(A) \leq \beta$)。

12. 用 Gauss-Seidel 方法解 $Ax = b$, 用 $x_i^{(k+1)}$ 记 $x^{(k+1)}$ 的第 i 个分量, 且

$$r_i^{(k+1)} = b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i}^n a_{ij}x_j^{(k)}.$$

(1) 证明 $x_i^{(k+1)} = x_i^{(k)} + \frac{r_i^{(k+1)}}{a_{ii}}$;

(2) 如果 $\varepsilon^{(k)} = x^{(k)} - \lambda^*$, 其中 λ^* 是方程组的精确解, 求证: $\varepsilon_i^{(k+1)} = \varepsilon_i^{(k)} - \frac{r_i^{(k+1)}}{a_{ii}}$, 其中

校注 (agent 补充): 习题 5 (1) 第二行原页确印 $0.4x_1 + x_2 + 0.8x_2 = 2$, 已保留。第 7 题将该方程组作为对称正定系数矩阵的考察对象; 若将最后一项改为 $0.8x_3$ 才与该对称结构一致, 故此处疑为原书下标错误。

校注 (agent 补充): 习题 8 (2) 的 Gauss-Seidel 迭代矩阵原页也记为 B_0 , 与 (1) 的 Jacobi 迭代矩阵符号相同, 转录保留。

校注 (agent 补充): 习题 12 有三处原书疑误, 均经放大目视后原样保留。残差定义第二个求和内确印 $x_i^{(k)}$, 对照 Gauss-Seidel 校正量应为 $x_j^{(k)}$; (2) 的误差定义确印 $x^{(k)} - \lambda^*$, 而紧随其后说明的是精确解 x^* ; 若误差定义按 $\varepsilon^{(k)} = x^{(k)} - x^*$ 理解, 则由 (1) 可知递推中的校正项应为加号, 原题所印减号与此前定义不一致。这里仅定位符号矛盾, 不代做习题证明。

原书 PDF 页 232; 印刷页码 219。

[原页图像](#)

$$r_i^{(k+1)} = \sum_{j=1}^{i-1} a_{ij} \varepsilon_j^{(k+1)} + \sum_{j=i}^n a_{ij} \varepsilon_j^{(k)};$$

(3) 设 A 是对称的, 二次型 $Q(\varepsilon^{(k)}) = (A\varepsilon^{(k)}, \varepsilon^{(k)})$, 证明

$$Q(\varepsilon^{(k+1)}) - Q(\varepsilon^{(k)}) = - \sum_{j=1}^n \frac{(r_j^{(k+1)})^2}{a_{jj}}.$$

(4) 由此推出, 如果 A 是具有正对角元素的非奇异矩阵, 且 Gauss-Seidel 方法对于任意初始向量 $x^{(0)}$ 是收敛的, 则 A 是正定矩阵。

13. 设 A 与 B 为 n 阶矩阵, A 为非奇异矩阵, 考虑解方程组 $Az_1 + Bz_2 = b_1$, $Bz_1 + Az_2 = b_2$, 其中 $z_1, z_2, b_1, b_2 \in \mathbb{R}^n$ 。

(1) 找出下述迭代方法收敛的充要条件

$$Az_1^{(m+1)} = b_1 - Bz_2^{(m)}, \quad Az_2^{(m+1)} = b_2 - Bz_1^{(m)} \quad (m \geq 0);$$

(2) 找出下述迭代方法收敛的充要条件

$$Az_1^{(m+1)} = b_1 - Bz_2^{(m)}, \quad Az_2^{(m+1)} = b_2 - Bz_1^{(m+1)} \quad (m \geq 0).$$

比较两个方法的收敛速度。

14. 证明矩阵 $A = \begin{bmatrix} 1 & a & a \\ a & 1 & a \\ a & a & 1 \end{bmatrix}$ 对于 $-\frac{1}{2} < a < 1$ 是正定的, 而 Jacobi 迭代只对 $-\frac{1}{2} < a < \frac{1}{2}$ 是收敛的。

15. 设 $A = \begin{bmatrix} 5 & 1 & 2 & 3 \\ 0 & 2 & 0 & 4 \\ 3 & -1 & 2 & -1 \\ 0 & 3 & 0 & 7 \end{bmatrix}$, 试说明 A 为可约矩阵。

16. 给定迭代过程 $x^{(k+1)} = Cx^{(k)} + g$, 其中 $C \in \mathbb{R}^{n \times n}$ ($k = 0, 1, 2, \dots$), 试证明: 如果 C 的特征值 $\lambda_i(C) = 0$ ($i = 1, 2, \dots, n$), 则此迭代过程最多迭代 n 次收敛于方程组的解。

17. 画出 SOR 方法的框图。

18. 设 A 为不可约弱对角占优阵且 $0 < \omega \leq 1$, 求证, 解 $Ax = b$ 的 SOR 方法收敛。

19. 设 $Ax = b$, 其中 A 为非奇异矩阵。

(1) 求证 $A^T A$ 为对称正定矩阵;

(2) 求证 $\text{cond}(A^T A)_2 = (\text{cond}(A)_2)^2$ 。

20. 设 A 为严格对角占优阵, 证明式 (2.8.23)。

校注 (agent 补充): 页首为习题 12 (2) 的续式。该残差表达式中的正号与上页所印误差定义 $\varepsilon^{(k)} = x^{(k)} - \lambda^*$ 不一致; 若误差定义取 $x^* - x^{(k)}$, 则可与此续式及上页的减号递推一致。两页原文均保留, 详见上页习题 12 校注。

原书 PDF 页 233; 印刷页码 220。

[原页图像](#)

第 9 章矩阵的特征值与特征向量计算

9.1 引言

物理、力学和工程技术中的很多问题在数学上都归结为求矩阵特征值的问题, 例如振动问题 (桥梁的振动、机械的振动、电磁振荡、地震引起的建筑物的振动等)、物理学中某些临界值的确定问题以及理论物理中的一些问题。这些实际问题可归结为如下数学问题。

(1) 已知 $A = (a_{ij})_{n \times n}$, 要求代数方程

$$\varphi(\lambda) = \det(\lambda I - A) = 0 \quad (9.1.1)$$

的根。 $\varphi(\lambda)$ 称为 A 的特征多项式。式 (9.1.1) 展开即有

$$\varphi(\lambda) = \lambda^n + c_1 \lambda^{n-1} + \cdots + c_n = 0.$$

一般 $\varphi(\lambda)$ 有 n 个零点, 称为 A 的特征值。

(2) 设 λ 为 A 的特征值, 要求相应的齐次方程组

$$(\lambda I - A)x = 0 \quad (9.1.2)$$

的非零解 (即求 $Ax = \lambda x$ 的非零解)。

式 (9.1.2) 的非零解 x 称为矩阵 A 的对应于 λ 的特征向量。下面叙述一些有关特征值问题的结论。

定理 9.1 如果 λ_i ($i = 1, 2, \dots, n$) 是矩阵 A 的特征值, 则有

$$1^\circ \quad \sum_{i=1}^n \lambda_i = \sum_{i=1}^n a_{ii} = \operatorname{tr} A;$$

$$2^\circ \quad \det A = \lambda_1 \lambda_2 \cdots \lambda_n.$$

定理 9.2 设 A 与 B 为相似矩阵 (即存在非奇异阵 T 使 $B = T^{-1}AT$), 则

1° A 与 B 有相同的特征值;

2° 若 x 是 B 的一个特征向量, 则 Tx 是 A 的特征向量。

定理 9.3 (Gerschgorin's 定理) 设 $A = (a_{ij})_{n \times n}$, 则 A 的每一个特征值必属于下述某个圆盘之中:

$$|\lambda - a_{ii}| \leq \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| \quad (i = 1, 2, \dots, n).$$

证明 设 λ 为 A 的任意一个特征值, x 为对应的特征向量, 即

$$(\lambda I - A)x = 0,$$

原书 PDF 页 234; 印刷页码 221。

[原页图像](#)

记 $x = (x_1, x_2, \dots, x_n)^T \neq 0$ 及 $|x_i| = \max_k |x_k|$, $x_i \neq 0$, 所以从式 (9.1.2) 的第 i 个方程

$$(\lambda - a_{ii})x_i = \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}x_j$$

以及 $|x_j/x_i| \leq 1$ ($j \neq i$), 有

$$|\lambda - a_{ii}| \leq \sum_{j \neq i} |a_{ij}|, \quad |x_j/x_i| \leq \sum_{j \neq i} |a_{ij}|.$$

这说明 λ 属于复平面上以 a_{ii} 为圆心、 $\sum_{j \neq i} |a_{ij}|$ 为半径的一个圆盘。

定理的证明, 不仅指出了 A 的每一个特征值必属于 A 的一个圆盘中, 而且指出, 若一个特征向量的第 i 个分量最大, 则对应的特征值一定属于第 i 个圆盘中。

定义 9.1 设 A 为 n 阶实对称矩阵, 对于任一非零向量 x , 称 $R(x) = \frac{(Ax, x)}{(x, x)}$ 为对应于向量 x 的 Rayleigh 商。

定理 9.4 设 $A \in \mathbf{R}^{n \times n}$ 为对称矩阵 (其特征值依次记作 $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$, 对应的特征向量 $x_1, x_2, \dots, x_n$ 组成规范化正交组, 即 $(x_i, x_j) = \delta_{ij}$), 则

$$1^\circ \quad \lambda_n \leq \frac{(Ax, x)}{(x, x)} \leq \lambda_1 \quad (\text{对于任何非零 } x \in \mathbf{R}^n);$$

$$2^\circ \quad \lambda_1 = \max_{\substack{x \in \mathbf{R}^n \\ x \neq 0}} \frac{(Ax, x)}{(x, x)};$$

$$3^\circ \quad \lambda_n = \min_{\substack{x \in \mathbf{R}^n \\ x \neq 0}} \frac{(Ax, x)}{(x, x)}.$$

证明 只证结论 1° , 结论 2° 、 3° 留作习题。

设 $x \neq 0$ 为 $\mathbf{R}^n$ 中任一向量, 则有展开式

$$x = \sum_{i=1}^n a_i x_i, \quad \|x\|_2 = \left(\sum_{i=1}^n a_i^2 \right)^{1/2} \neq 0,$$

于是

$$\frac{(Ax, x)}{(x, x)} = \frac{\sum_{i=1}^n a_i^2 \lambda_i}{\sum_{i=1}^n a_i^2},$$

从而结论 1° 成立。结论 1° 说明 Rayleigh 商必位于 λ_n 和 λ_1 之间。

关于计算矩阵 A 的特征值问题, 当 $n = 2, 3$ 时, 还可按行列式展开的办法求 $\varphi(\lambda) = 0$ 的根。但当 n 较大时, 如果按展开行列式的办法, 首先求出 $\varphi(\lambda)$ 的系数, 再求 $\varphi(\lambda)$ 的根, 工作量就非常大了。用这种办法求矩阵特征值是不切实际的, 由此需要研究求 A 的特征值及特征向量的数值方法。

本章将介绍计算机上常用的两类方法, 一类是幂法及反幂法 (迭代法), 另一类是正交相似变换的方法 (变换法)。

校注 (agent 补充, ch09a-E001) 原图中 Gerschgorin 证明的上述不等式在 $\sum_{j \neq i} |a_{ij}|$ 后印有逗号, 接着印 $|x_j/x_i| \leq \sum_{j \neq i} |a_{ij}|$, 已照录。由上一行除以 x_i 后取绝对值, 能推出的是 $|\lambda - a_{ii}| \leq \sum_{j \neq i} |a_{ij}| |x_j/x_i| \leq \sum_{j \neq i} |a_{ij}|$; 故原排式疑有标点或连乘排版错误。见[放大原式](#)。

校注 (agent 补充, ch09a-E002) 原图范数展开式的括号外指数印为 2, 已照录。由规范化正交性, $\|x\|_2^2 = \sum_i a_i^2$, 故该处指数应为 1/2; 这是原书疑误, 非转录时补成的指数。见[放大原式](#)。

原书 PDF 页 235; 印刷页码 222。

[原页图像](#)

9.2 幂法及反幂法

9.2.1 幂法

在一些工程、物理问题中, 通常只要求出矩阵的按模最大的特征值 (称为 A 的主特征值) 和相应的特征向量, 对于解这种特征值问题, 应用幂法是合适的。

幂法是一种计算实矩阵 A 的主特征值的一种迭代法, 它最大的优点是方法简单, 对于稀疏矩阵较合适, 但有时收敛速度很慢。

设实矩阵 $A = (a_{ij})_n$ 有一个完全的特征向量组, 其特征值为 $\lambda_1, \lambda_2, \dots, \lambda_n$, 相应的特征向量为 $x_1, x_2, \dots, x_n$ 。已知 A 的主特征值是实根, 且满足条件

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \dots \geq |\lambda_n|. \quad (9.2.1)$$

幂法的基本思想是任取一个非零的初始向量 v_0 , 由矩阵 A 构造一向量序列

$$\begin{cases} v_1 = Av_0, \\ v_2 = Av_1 = A^2v_0, \\ \vdots \\ v_{k+1} = Av_k = A^{k+1}v_0, \\ \vdots \end{cases} \quad (9.2.2)$$

称为迭代向量。由假设, v_0 可表示为

$$v_0 = a_1x_1 + a_2x_2 + \dots + a_nx_n \quad (\text{设 } a_1 \neq 0), \quad (9.2.3)$$

于是

$$\begin{aligned} v_k &= Av_{k-1} = A^k v_0 = a_1 \lambda_1^k x_1 + a_2 \lambda_2^k x_2 + \dots + a_n \lambda_n^k x_n \\ &= \lambda_1^k \left[a_1 x_1 + \sum_{i=2}^n a_i (\lambda_i / \lambda_1)^k x_i \right] = \lambda_1^k (a_1 x_1 + \varepsilon_k), \end{aligned}$$

其中 $\varepsilon_k = \sum_{i=2}^n a_i (\lambda_i / \lambda_1)^k x_i$ 。由假设 $|\lambda_i / \lambda_1| < 1$ ($i = 2, 3, \dots, n$), 故 $\varepsilon_k \rightarrow 0$ ($k \rightarrow \infty$), 从而

$$\lim_{k \rightarrow \infty} \frac{v_k}{\lambda_1^k} = a_1 x_1. \quad (9.2.4)$$

这说明序列 $\frac{v_k}{\lambda_1^k}$ 越来越接近 A 的对应于 λ_1 的特征向量, 或者说当 k 充分大时

$$v_k \approx a_1 \lambda_1^k x_1, \quad (9.2.5)$$

即迭代向量 v_k 为 λ_1 的特征向量的近似向量（除一个因子外）。

下面再考虑主特征值 λ_1 的计算。用 $(v_k)_i$ 表示 v_k 的第 i 个分量，则

$$\frac{(v_{k+1})_i}{(v_k)_i} = \lambda_1 \left\{ \frac{a_1(x_1)_i + (\varepsilon_{k+1})_i}{a_1(x_1)_i + (\varepsilon_k)_i} \right\}, \quad (9.2.6)$$

校注 (agent 补充, ch09a-E003) 本页 v_k 第二行求和项及 ε_k 定义中的系数下标，原图均印为 a_1 ，已照录；与前一行 $a_i \lambda_i^k x_i$ 的展开相比较，两处应为 a_i 才能保持恒等，属原书疑误。见[放大原式](#)。

原书 PDF 页 236; 印刷页码 223。

[原页图像](#)

故

$$\lim_{k \rightarrow \infty} \frac{(v_{k+1})_i}{(v_k)_i} = \lambda_1, \quad (9.2.7)$$

也就是说, 两相邻迭代向量分量的比值收敛到主特征值。

这种由已知非零向量 v_0 及矩阵 A 的乘幂 A^k 构造向量序列 $\{v_k\}$ 以计算 A 的主特征值 λ_1 (利用式 (9.2.7)) 及相应特征向量 (利用式 (9.2.5)) 的方法称为幂法。

由式 (9.2.6) 知, $(v_{k+1})_i/(v_k)_i \rightarrow \lambda_1$ 的收敛速度由比值 $r = \lambda_2/\lambda_1$ 来确定, r 越小收敛越快, 但当 $r = \lambda_2/\lambda_1 \approx 1$ 时收敛可能就很慢。

总结上述讨论, 有如下定理。

定理 9.5 设 $A \in \mathbf{R}^{n \times n}$ 有 n 个线性无关的特征向量, 主特征值 λ_1 满足

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|,$$

则对于任何非零初始向量 v_0 ($a_1 \neq 0$), 式 (9.2.4)、(9.2.7) 成立。

设 A 的主特征值为实重根, 即 $\lambda_1 = \lambda_2 = \cdots = \lambda_r$, 且 $|\lambda_r| > |\lambda_{r+1}| \geq \cdots \geq |\lambda_n|$, 又设 A 有 n 个线性无关的特征向量, λ_1 对应的 r 个线性无关特征向量为 $x_1, x_2, \cdots, x_r$, 则由式 (9.2.2), 有

$$v_k = A^k v_0 = \lambda_1^k \left\{ \sum_{i=1}^r a_i x_i + \sum_{i=r+1}^n a_i \left(\frac{\lambda_i}{\lambda_1} \right)^k x_i \right\}, \quad \lim_{k \rightarrow \infty} \frac{v_k}{\lambda_1^k} = \sum_{i=1}^r a_i x_i \quad \left(\text{设 } \sum_{i=1}^r a_i x_i \neq 0 \right).$$

这说明当 A 的主特征值是实的重根时, 定理 9.5 的结论还是正确的。

应用幂法计算 A 的主特征值 λ_1 及对应的特征向量时, 如果 $|\lambda_1| > 1$ (或 $|\lambda_1| < 1$), 迭代向量 v_k 的各个不等于零的分量将随 $k \rightarrow \infty$ 而趋于无穷 (或趋于零), 这样在用计算机计算时就可能“溢出”。为了克服这个缺点, 就需要将迭代向量加以规范化。

设有一向量 $v \neq 0$, 将其规范化得到向量 $u = \frac{v}{\max(v)}$, 其中 $\max(v)$ 表示向量 v 的绝对值最大的分量。

在定理 9.5 的条件下幂法可这样进行: 任取一初始向量 $v_0 \neq 0$ ($a_1 \neq 0$), 构造向量序列

$$\begin{cases} v_1 = Au_0 = Av_0, & u_1 = \frac{v_1}{\max(v_1)} = \frac{Av_0}{\max(Av_0)}, \\ v_2 = Au_1 = \frac{A^2 v_0}{\max(Av_0)}, & u_2 = \frac{v_2}{\max(v_2)} = \frac{A^2 v_0}{\max(A^2 v_0)}, \\ \vdots & \vdots \\ v_k = \frac{A^k v_0}{\max(A^{k-1} v_0)}, & u_k = \frac{A^k v_0}{\max(A^k v_0)}. \end{cases}$$

由式 (9.2.3), 有

$$Av_0 = \sum_{i=1}^n a_i \lambda_i^k x_i = \lambda_1^k \left[a_1 x_1 + \sum_{i=2}^n a_i \left(\frac{\lambda_i}{\lambda_1} \right)^k x_i \right], \quad (9.2.8)$$

校注 (agent 补充, ch09a-E004) 式 (9.2.8) 原图左端为 Av_0 , 右端方括号前为 λ_i^k , 均照录。由 $v_0 = \sum_i a_i x_i$ 和 $Ax_i = \lambda_i x_i$, 中间求和等于 $A^k v_0$, 而提取的公共因子应为 λ_1^k ; 故有漏印幂次及因子下标疑误。见[放大原式](#)。

原书 PDF 页 237; 印刷页码 224。

[原页图像](#)

$$\begin{aligned} u_k &= \frac{A^k v_0}{\max(A^k v_0)} = \frac{\lambda_1^k \left[a_1 x_1 + \sum_{i=2}^n a_i \left(\frac{\lambda_i}{\lambda_1} \right)^k x_i \right]}{\max \left[\lambda_1^k \left(a_1 x_1 + \sum_{i=2}^n a_i \left(\frac{\lambda_i}{\lambda_1} \right)^k x_i \right) \right]} \\ &= \frac{\left[a_1 x_1 + \sum_{i=2}^n a_i \left(\frac{\lambda_i}{\lambda_1} \right)^k x_i \right]}{\max \left[a_1 x_1 + \sum_{i=2}^n a_i \left(\frac{\lambda_i}{\lambda_1} \right)^k x_i \right]} \rightarrow \frac{x_1}{\max(x_1)} \quad (k \rightarrow \infty). \end{aligned}$$

这说明规范化向量序列收敛到主特征值对应的特征向量。

同理, 可得到

$$\begin{aligned} v_k &= \frac{\lambda_1^k \left[a_1 x_1 + \sum_{i=2}^n a_i \left(\frac{\lambda_i}{\lambda_1} \right)^k x_i \right]}{\max \left[\lambda_1^{k-1} a_1 x_1 + \sum_{i=2}^n a_i \left(\frac{\lambda_i}{\lambda_1} \right)^{k-1} x_i \right]}, \\ \max(v_k) &= \frac{\lambda_1 \max \left[a_1 x_1 + \sum_{i=2}^n a_i \left(\frac{\lambda_i}{\lambda_1} \right)^k x_i \right]}{\max \left[a_1 x_1 + \sum_{i=2}^n a_i \left(\frac{\lambda_i}{\lambda_1} \right)^{k-1} x_i \right]} \rightarrow \lambda_1 \quad (k \rightarrow \infty). \end{aligned}$$

收敛速度由比值 $r = \lambda_2/\lambda_1$ 确定。

总结上述讨论, 有如下定理。

定理 9.6 设 $A \in \mathbf{R}^{n \times n}$ 有 n 个线性无关的特征向量, 主特征值 λ_1 满足 $|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|$, 则对于任意非零初始向量 $v_0 = u_0$ ($a_1 \neq 0$), 按下述方法构造的向量序列

$$\begin{cases} v_0 = u_0 \neq 0, \\ v_k = A u_{k-1}, \\ u_k = \frac{v_k}{\max(v_k)} \end{cases} \quad (k = 1, 2, \cdots), \quad (9.2.9)$$

有

$$\lim_{k \rightarrow \infty} u_k = \frac{x_1}{\max(x_1)}, \quad \lim_{k \rightarrow \infty} \max(v_k) = \lambda_1.$$

例 9.1 用幂法计算 $A = \begin{pmatrix} 1.0 & 1.0 & 0.5 \\ 1.0 & 1.0 & 0.25 \\ 0.5 & 0.25 & 2.0 \end{pmatrix}$ 的主特征值和相应的特征向量。

解计算过程如表 9.1 所示。

下述结果是用 8 位浮点数字进行运算得到的, u_k 的分量值是舍入值。于是得到

$$\lambda_1 \approx 2.536\,532\,3$$

及相应特征向量 $(0.748\,2, 0.649\,7, 1)^T$ 。 λ_1 和相应的特征向量真值 (8 位数字) 为

$$\lambda_1 = 2.536\,525\,8, \quad \tilde{x}_1 = (0.748\,221\,16, 0.649\,661\,16, 1)^T.$$

校注 (agent 补充, ch09a-E005) 页首 u_k 第一行分子方括号前原印 λ_i^k , 分母对应因子为 λ_1^k , 已照录; 与 $A^k v_0$ 的展开及下一行相消过程对照, 分子因子应为 λ_1^k 。见[放大原式](#)。

校注 (agent 补充, ch09a-E006) 本页 v_k 式分母中, 原图 λ_1^{k-1} 仅直接乘 $a_1 x_1$, 随后为加号及求和项, 已照录。由上一页 $v_k = A^k v_0 / \max(A^{k-1} v_0)$, 分母应把 λ_1^{k-1} 乘在整个 $a_1 x_1 + \sum_{i=2}^n a_i (\lambda_i / \lambda_1)^{k-1} x_i$ 上, 故疑漏一层括号。见[放大原式](#)。

原书 PDF 页 238；印刷页码 225。

[原页图像](#)

表 9.1

k	u_k^T (规范化向量)	$\max(v_k)$
0	(1, 1, 1)	
1	(0.909 1, 0.818 2, 1)	2.750 000 0
5	(0.765 1, 0.667 4, 1)	2.558 791 8
10	(0.749 4, 0.650 8, 1)	2.538 002 9
15	(0.748 3, 0.649 7, 1)	2.536 625 6
16	(0.748 3, 0.649 7, 1)	2.536 584 0
17	(0.748 2, 0.649 7, 1)	2.536 559 8
18	(0.748 2, 0.649 7, 1)	2.536 545 6
19	(0.748 2, 0.649 7, 1)	2.536 537 4
20	(0.748 2, 0.649 7, 1)	2.536 532 3

9.2.2 加速方法

1. 原点平移法 由前面讨论知道，应用幂法计算 A 的主特征值时，其收敛速度主要由比值 $r = \frac{\lambda_1}{\lambda_2}$ 来决定，但当 r 接近于 1 时，收敛可能很慢。这时，一个补救的办法是采用加速收敛的方法。

引进矩阵 $B = A - pI$ ，其中 p 为选择参数。

设 A 的特征值为 $\lambda_1, \lambda_2, \cdots, \lambda_n$ ，则 B 的相应特征值为 $\lambda_1 - p, \lambda_2 - p, \cdots, \lambda_n - p$ ，而且 A, B 的特征向量相同。

如果需要计算 A 的主特征值 λ_1 ，就要选择适当的 p 使 $\lambda_1 - p$ 仍然是 B 的主特征值，且使

$$\left| \frac{\lambda_2 - p}{\lambda_1 - p} \right| < \left| \frac{\lambda_2}{\lambda_1} \right|.$$

对 B 应用幂法，使得在计算 B 的主特征值 $\lambda_1 - p$ 的过程中得到加速。这种方法通常称为原点平移法。对于 A 的特征值的某种分布，它是十分有效的。

例 9.2 设 $A = (a_{ij})_4$ 有特征值 $\lambda_j = 15 - j$ ($j = 1, 2, 3, 4$)，比值 $r = \lambda_2/\lambda_1 \approx 0.9$ 。

作变换

$$B = A - pI \quad (p = 12),$$

则 B 的特征值为 $\mu_1 = 2, \mu_2 = 1, \mu_3 = 0, \mu_4 = -1$ 。应用幂法计算 B 的主特征值 μ_1 的收敛速度的比值为

$$\left| \frac{\mu_2}{\mu_1} \right| = \left| \frac{\lambda_2 - p}{\lambda_1 - p} \right| = \frac{1}{2} < \left| \frac{\lambda_2}{\lambda_1} \right| \approx 0.9.$$

校注 (agent 补充, ch09a-E007) 本页“原点平移法”首段原印 $r = \lambda_1/\lambda_2$ ，已照录；前页及本页例 9.2 均用 $r = \lambda_2/\lambda_1$ ，与幂法误差项 $(\lambda_i/\lambda_1)^k$ 对照，此处疑将分子、分母颠倒。见[放大原式](#)。

原书 PDF 页 239; 印刷页码 226。

[原页图像](#)

虽然常常能够选择有利的 p 值, 使幂法得到加速, 但设计一个自动选择适当参数 p 的过程是困难的。

下面考虑当 A 的特征值是实数时, 怎样选择 p 使用幂法计算 λ_1 以得到加速。

设 A 的特征值满足

$$\lambda_1 > \lambda_2 \geq \cdots \geq \lambda_{n-1} > \lambda_n, \quad (9.2.10)$$

则不管 p 如何选择, $B = A - pI$ 的主特征值为 $\lambda_1 - p$ 或 $\lambda_n - p$ 。当希望计算 λ_1 及 x_1 时, 首先应选择 p 使 $|\lambda_1 - p| > |\lambda_n - p|$, 且使收敛速度的比值

$$\omega = \max \left\{ \frac{|\lambda_2 - p|}{|\lambda_1 - p|}, \frac{|\lambda_n - p|}{|\lambda_1 - p|} \right\} = \min,$$

显然, 当 $\lambda_2 - p = -(\lambda_n - p)$, $p = \frac{\lambda_2 + \lambda_n}{2} \equiv p^*$ 时 ω 为最小, 这时收敛速度的比值为

$$\frac{\lambda_2 - p^*}{\lambda_1 - p^*} = -\frac{\lambda_n - p^*}{\lambda_1 - p^*} \equiv \frac{\lambda_2 - \lambda_n}{2\lambda_1 - \lambda_2 - \lambda_n}.$$

当 A 的特征值满足式 (9.2.10) 且 λ_2, λ_n 能初步估计时, 就能确定 p^* 的近似值。

当希望计算 λ_n 时, 应选择 $p = \frac{\lambda_1 + \lambda_{n-1}}{2} = p^*$, 使得应用幂法计算 λ_n 得到加速。

例 9.3 计算例 9.1 矩阵 A 的主特征值。

解作变换 $B = A - pI$, 取 $p = 0.75$, 则

$$B = \begin{pmatrix} 0.25 & 1 & 0.5 \\ 1 & 0.25 & 0.25 \\ 0.5 & 0.25 & 1.25 \end{pmatrix}.$$

对 B 应用幂法, 计算结果如表 9.2 所示。

表 9.2

k	u_k^T (规范化向量)	$\max(v_k)$
0	(1, 1, 1)	
5	(0.751 6, 0.652 2, 1)	1.791 401 1
6	(0.749 1, 0.651 1, 1)	1.788 844 3
7	(0.748 8, 0.650 1, 1)	1.787 330 0
8	(0.748 4, 0.649 9, 1)	1.786 915 2
9	(0.748 3, 0.649 7, 1)	1.786 658 7
10	(0.748 2, 0.649 7, 1)	1.786 591 4

由此得 B 的主特征值为 $\mu_1 \approx 1.786 591 4$, A 的主特征值 λ_1 为

$$\lambda_1 \approx \mu_1 + 0.75 = 2.536\,591\,4.$$

这个结果比例 9.1 迭代 15 次得到的结果还要好。若迭代 15 次， $\mu_1 = 1.786\,525\,8$ （相应的 $\lambda_1 = 2.536\,525\,8$ ）。

原书 PDF 页 240; 印刷页码 227。

[原图图像](#)

原点位移的加速方法, 是一个矩阵变换方法。这种变换容易计算, 又不破坏矩阵 A 的稀疏性, 但 p 的选择依赖于对 A 的特征值分布的大致了解。

2. Rayleigh 商加速法 由定理 9.4 知, 对称矩阵 A 的 λ_1 及 λ_n 可用 Rayleigh 商的极值来表示。下面将把 Rayleigh 商应用到用幂法计算实对称矩阵 A 的主特征值的加速收敛上来。

定理 9.7 设 $A \in \mathbf{R}^{n \times n}$ 为对称矩阵, 特征值满足 $|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|$, 对应的特征向量满足 $(x_i, x_j) = \delta_{ij}$, 应用幂法 (式 (9.2.9)) 计算 A 的主特征值 λ_1 , 则规范化向量 u_k 的 Rayleigh 商给出 λ_1 的较好的近似, 即

$$\frac{(Au_k, u_k)}{(u_k, u_k)} = \lambda_1 + O\left(\left(\frac{\lambda_2}{\lambda_1}\right)^{2k}\right).$$

证明 由式 (9.2.8) 及 $u_k = \frac{A^k u_0}{\max(A^k u_0)}$, $v_{k+1} = Au_k = \frac{A^{k+1} u_0}{\max(A^k u_0)}$, 得

$$\frac{(Au_k, u_k)}{(u_k, u_k)} = \frac{(A^{k+1} u_0, A^k u_0)}{(A^k u_0, A^k u_0)} = \frac{\sum_{j=1}^n a_j^2 \lambda_j^{2k+1}}{\sum_{j=1}^n a_j^2 \lambda_j^{2k}} = \lambda_1 + O\left(\left(\frac{\lambda_2}{\lambda_1}\right)^{2k}\right). \quad (9.2.11)$$

9.2.3 反幂法

反幂法用来计算矩阵按模最小的特征值及其特征向量, 及计算对应于一个给定近似特征值的特征向量。

设 $A \in \mathbf{R}^{n \times n}$ 为非奇异矩阵, A 的特征值依次记作 $|\lambda_1| \geq |\lambda_2| \geq \cdots \geq |\lambda_n|$, 相应的特征向量为 $x_1, x_2, \cdots, x_n$, 则 A^{-1} 的特征值为 $\left|\frac{1}{\lambda_n}\right| \geq \left|\frac{1}{\lambda_{n-1}}\right| \geq \cdots \geq \left|\frac{1}{\lambda_1}\right|$, 对应的特征向量为 $x_n, x_{n-1}, \cdots, x_1$ 。

因此, 计算 A 的按模最小的特征值 λ_n 的问题就是计算 A^{-1} 的按模最大的特征值问题。

对 A^{-1} 应用幂法迭代法 (称为反幂法), 可求得矩阵 A^{-1} 的主特征值 $1/\lambda_n$, 从而求得 A 的按模最小的特征值 λ_n 。

反幂法迭代公式为任取初始向量 $v_0 = u_0 \neq 0$, 构造向量序列

$$\begin{cases} v_k = A^{-1}u_{k-1}, \\ u_k = \frac{v_k}{\max(v_k)} \end{cases} \quad (k = 1, 2, \cdots).$$

迭代向量 v_k 可以通过解方程组 $Av_k = u_{k-1}$ 求得。

定理 9.8 设

1° A 有 n 个线性无关的特征向量,

2° A 为非奇异矩阵且其特征值满足

原书 PDF 页 241; 印刷页码 228。

[原页图像](#)

$$|\lambda_1| \geq |\lambda_2| \geq \cdots \geq |\lambda_{n-1}| > |\lambda_n| > 0,$$

则对任何初始非零向量 $u_0 = v_0$ ($a_n \neq 0$), 由反幂法构造的向量序列 $\{v_k\}, \{u_k\}$ 满足

$$\lim_{k \rightarrow \infty} u_k = \frac{x_n}{\max(x_n)}, \quad \lim_{k \rightarrow \infty} \max(v_k) = \frac{1}{\lambda_n}.$$

收敛速度的比值为 $\left| \frac{\lambda_n}{\lambda_{n-1}} \right|$ 。

在反幂法中也可以用原点平移法来加速迭代过程或求其他特征值及特征向量。

如果矩阵 $(A - pI)^{-1}$ 存在, 显然其特征值为 $\frac{1}{\lambda_1 - p}, \frac{1}{\lambda_2 - p}, \cdots, \frac{1}{\lambda_n - p}$, 对应的特征向量仍然是 $x_1, x_2, \cdots, x_n$ 。现对矩阵 $(A - pI)^{-1}$ 应用幂法, 得到反幂法的迭代公式

$$\begin{cases} u_0 = v_0 \neq 0 \quad (\text{初始向量}), \\ v_k = (A - pI)^{-1} u_{k-1}, \\ u_k = \frac{v_k}{\max(v_k)} \end{cases} \quad (k = 1, 2, \cdots). \quad (9.2.12)$$

如果 p 是 A 的特征值 λ_j 的一个近似值, 且 $|\lambda_j - p| < |\lambda_i - p|$ ($i \neq j$), 就是说, $\frac{1}{\lambda_j - p}$ 是 $(A - pI)^{-1}$ 的主特征值, 可用反幂法式 (9.2.12) 计算其特征值及特征向量。

设 $A \in \mathbf{R}^{n \times n}$ 有 n 个线性无关的特征向量 $x_1, x_2, \cdots, x_n$, 则

$$u_0 = \sum_{i=1}^n a_i x_i \quad (a_i \neq 0), \quad v_k = \frac{(A - pI)^{-k} u_0}{\max((A - pI)^{-(k-1)} u_0)},$$

$$u_k = \frac{(A - pI)^{-k} u_0}{\max((A - pI)^{-k} u_0)},$$

其中

$$(A - pI)^{-k} u_0 = \sum_{i=1}^n a_i (\lambda_i - p)^{-k} x_i.$$

定理 9.9 设

1° $A \in \mathbf{R}^{n \times n}$ 有 n 个线性无关的特征向量, A 的特征值及对应的特征向量记作 λ_i 及 x_i ($i = 1, 2, \cdots, n$);

2° p 为 λ_j 的近似值, $(A - pI)^{-1}$ 存在, 且 $|\lambda_j - p| < |\lambda_i - p|$ ($i \neq j$);

3° $u_0 = \sum_{i=1}^n a_i x_i \neq 0$ 为给定的初始向量 ($a_i \neq 0$),

则由反幂法迭代公式 (9.2.12) 构造的向量序列 $\{v_k\}, \{u_k\}$ 满足

$$\lim_{k \rightarrow \infty} u_k = \frac{x_j}{\max(x_j)},$$

$$\lim_{k \rightarrow \infty} \max(v_k) = \frac{1}{\lambda_j - p}, \quad \text{即} \quad p + \frac{1}{\max(v_k)} \rightarrow \lambda_j \quad (\text{当 } k \rightarrow \infty),$$

且收敛速度由比值 $r = \max_{i \neq j} \left| \frac{\lambda_j - p}{\lambda_i - p} \right|$ 确定。

原书 PDF 页 242; 印刷页码 229。

原页图像

由定理 9.9 知, 对 $A - pI$ (其中 $p \approx \lambda_j$) 应用反幂法, 可计算特征向量 x_j 。只要选择的 p 是 λ_j 的一个较好的近似且特征值分离情况较好, 一般 r 很小, 常常只要迭代一两次就可完成特征向量的计算。

反幂法迭代公式中的 v_k 是通过解方程组 $(A - pI)v_k = u_{k-1}$ 求得的。为了节省工作量, 可以先将 $(A - pI)$ 进行三角分解, 即

$$P(A - pI) = LU,$$

其中 P 为某个置换矩阵, 于是求 v_k 相当于解两个三角形方程组 $Ly_k = Pu_{k-1}, Uv_k = y_k$ 。

反幂法迭代公式可写为

$$\begin{cases} Ly_k = Pu_{k-1}, \\ Uv_k = y_k, \\ u_k = \frac{v_k}{\max(v_k)} \end{cases} \quad (k = 1, 2, \dots). \quad (9.2.13)$$

实验表明, 按下述方法选择 $v_0 = u_0$ 是较好的: 选 u_0 使

$$Uv_1 = L^{-1}Pu_0 = (1, 1, \dots, 1)^T, \quad (9.2.14)$$

用回代求解式 (9.2.14) 即得 v_1 , 然后再按式 (9.2.13) 迭代。

例 9.4 用反幂法求

$$A = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 4 \end{pmatrix}$$

的对应计算特征值 $\lambda = 1.2679$ (精确特征值为 $\lambda_3 = 3 - \sqrt{3}$) 的特征向量 (用 5 位浮点数进行运算)。

解 用部分选主元的三角分解将 $A - pI$ (其中 $p = 1.2679$) 分解为

$$P(A - pI) = LU,$$

其中

$$P = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}, \quad L = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0.7321 & -0.26807 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 1 & 1.7321 & 1 \\ 0 & 1 & 2.7321 \\ 0 & 0 & 0.29405 \times 10^{-3} \end{pmatrix}.$$

由 $Uv_1 = (1, 1, 1)^T$, 得

$$v_1 = (12\,692, -9\,290.3, 3\,400.8)^T, \quad u_1 = (1, -0.731\,98, 0.267\,95)^T,$$

由 $LUv_2 = Pu_1$, 得

$$v_2 = (20\,404, -14\,937, 5\,467.4)^T, \quad u_2 = (1, -0.732\,06, 0.267\,96)^T.$$

λ_3 对应的特征向量是

$$x_3 = (1, 1 - \sqrt{3}, 2 - \sqrt{3})^T \approx (1, -0.732\,05, 0.267\,95)^T.$$

原书 PDF 页 243; 印刷页码 230。

[原页图像](#)

由此可以看出, u_2 是 x_3 的相当好的近似。

9.3 Householder 方法

9.3.1 引言

前面几节讨论的是求解矩阵最大(小)特征值及其对应特征向量的方法。若要求求出所有特征值及其特征向量, 应该用什么方法呢? 下面将讨论的以正交相似变换为基础的一类方法即是解决这类问题的方法。

首先, 讨论对于一般实矩阵 $A \in \mathbf{R}^{n \times n}$ 利用正交相似变换约化到什么程度的问题。由代数知识可知如下定理。

定理 9.10 设 $A \in \mathbf{R}^{n \times n}$, 则存在一个正交阵 R , 使

$$R^T A R = \begin{pmatrix} T_{11} & T_{12} & \cdots & T_{1s} \\ & T_{22} & \cdots & T_{2s} \\ & & \ddots & \vdots \\ & & & T_{ss} \end{pmatrix},$$

其中对角块为一阶或二阶矩阵, 每一个一阶对角块即为 A 的实特征值, 每一个二阶对角块的两个特征值是 A 的一对共轭复特征值。

定义 9.2 一方阵 B , 如果当 $i > j + 1$ 时有 $b_{ij} = 0$, 则称 B 为上 Hessenberg 阵, 即

$$B = \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ & \ddots & \ddots & \vdots \\ & & b_{n,n-1} & b_{nn} \end{pmatrix}.$$

本节讨论如下两个问题:

- (1) 用正交相似变换约化一般实矩阵为上 Hessenberg 阵;
- (2) 用正交相似变换约化对称阵为三对角阵。

这样, 求原矩阵特征值问题, 就转化为求上 Hessenberg 阵或对称三对角阵的特征值问题。

定义 9.3 设向量 w 满足 $\|w\|_2 = 1$, 矩阵 $H = I - 2ww^T$ 称为初等反射阵, 记作 $H(w)$, 即

$$H(w) = \begin{pmatrix} 1 - 2w_1^2 & -2w_1w_2 & \cdots & -2w_1w_n \\ -2w_2w_1 & 1 - 2w_2^2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & -2w_{n-1}w_n \\ -2w_nw_1 & \cdots & -2w_nw_{n-1} & 1 - 2w_n^2 \end{pmatrix},$$

原书 PDF 页 244; 印刷页码 231。

[原页图像](#)

其中

$$w = (w_1, w_2, \dots, w_n)^T.$$

定理 9.11 初等反射阵 H 是对称阵 ($H^T = H$)、正交阵 ($H^T H = I$) 和对合阵 ($H^2 = I$)。

证明 只证 H 的正交性, 其他显然。

$$H^T H = H^2 = (I - 2ww^T)(I - 2ww^T) = I - 4ww^T + 4w(w^T w)w^T = I.$$

设向量 $u \neq 0$, 则显然 $H = I - 2\frac{uu^T}{\|u\|_2^2}$ 是一个初等反射阵。

下面考察初等反射阵的几何意义。考虑以 w 为法向量过原点 O 的超平面

$$S : w^T x = 0.$$

设任意向量 $v \in \mathbf{R}^n$, 则 $v = x + y$, 其中 $x \in S, y \in S^\perp$ 。于是

$$Hx = (I - 2ww^T)x = x - 2ww^T x = x.$$

对于 $y \in S^\perp$, 易知 $Hy = -y$, 从而对于任意向量 $v \in \mathbf{R}^n$, 总有

$$Hv = x - y = v',$$

其中 v' 为 v 关于平面 S 的镜面反射 (见图 9.1)。

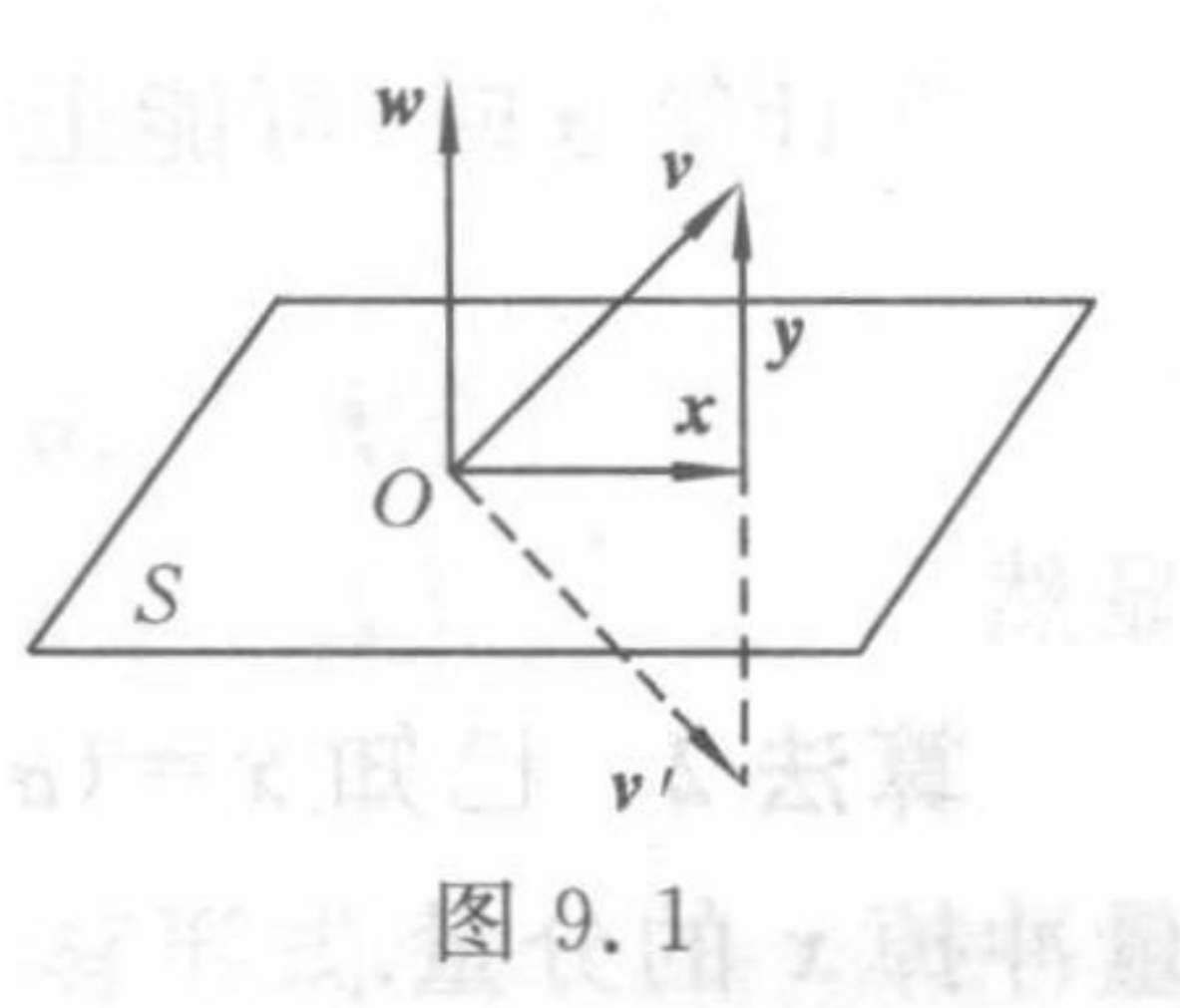


图 9.1

图 9.1 (原图裁切)。图中符号为 w, v, x, y, O, S, v' 。超平面 S 经过原点 O , w 为法向量; $v = x + y$ 分解为平面内分量 x 与垂直分量 y , v' 位于平面另一侧, 表示镜面反射所得 $x - y$ 。原图用虚线连接 O, v' 端点及垂直分量方向。

初等反射阵在计算上的意义是它能用来约化矩阵, 例如设向量 $a \neq 0$, 可选择一初等反射阵 H 使 $H_a = \sigma e_1$ 。这种约化矩阵的方法称为 Householder 方法。为此给出下面定理。

定理 9.12 设 x, y 为两个不相等的 n 维向量, $\|x\|_2 = \|y\|_2$, 则存在一个初等反射阵 H , 使 $Hx = y$ 。

证明 令 $w = \frac{x - y}{\|x - y\|_2}$, 则得到一个初等反射阵

$$H = I - 2ww^T = I - 2\frac{(x - y)(x - y)^T}{\|x - y\|_2^2},$$

而且

$$Hx = x - 2\frac{x - y}{\|x - y\|_2^2}(x^T - y^T)x = x - 2\frac{(x - y)(x^Tx - y^Tx)}{\|x - y\|_2^2},$$

因为

$$\|x - y\|_2^2 = (x - y)^T(x - y) = 2(x^Tx - y^Tx),$$

所以

$$Hx = x - (x - y) = y.$$

易知, $w = \frac{x-y}{\|x-y\|_2}$ 是使 $Hx = y$ 成立的唯一长度等于 1 的向量 (不计符号)。

推论 设向量 $x \in \mathbf{R}^n$ ($x \neq 0$), $\sigma = \pm\|x\|_2$, 且 $x \neq -\sigma e_1$, 则存在一个初等反射阵

$$H = I - 2 \frac{uu^T}{\|u\|_2^2} \equiv I - \rho^{-1}uu^T,$$

使 $Hx = -\sigma e_1$, 其中 $u = x + \sigma e_1$, $\rho = \|u\|_2^2/2$ 。

设

$$x = (\alpha_1, \alpha_2, \dots, \alpha_n)^T \neq 0, \quad u = (u_1, u_2, \dots, u_n)^T,$$

校注 (agent 补充, ch09a-E008) 在“可选择一初等反射阵 H 使”后, 原图把 a 印在 H 的下标位置, 故照录为 $H_a = \sigma e_1$ 。依本段语义及随后定理的矩阵作用, 应为乘积 $Ha = \sigma e_1$; 疑为原书排版错误。见[放大原式](#)。

原书 PDF 页 245; 印刷页码 232。

[原页图像](#)

则

$$u = (\alpha_1 + \sigma, \alpha_2, \dots, \alpha_n)^T,$$

$$\rho = \frac{1}{2} \|u\|_2^2 = \frac{1}{2} [(\alpha_1 + \sigma)^2 + \alpha_2^2 + \dots + \alpha_n^2] = \sigma(\sigma + \alpha_1).$$

如果 σ 和 α_1 异号, 那么计算 $\alpha_1 + \sigma$ 时有效数字可能损失, 取 σ 和 α_1 有相同的符号, 即取

$$\sigma = \text{sgn}(\alpha_1) \|x\|_2.$$

算法 1 已知向量 $x = (\alpha_1, \alpha_2, \dots, \alpha_n)^T \neq 0$, 本算法算出 σ, ρ 及 u , 使 $(I - \rho^{-1}uu^T)x = -\sigma e_1$, u 的分量冲掉 x 的分量。

步 1 计算 $\sigma = \text{sgn}(\alpha_1) \left(\sum_{i=1}^n \alpha_i^2 \right)^{\frac{1}{2}}$ 。

步 2 $\alpha_1 \rightarrow u_1 = \alpha_1 + \sigma$ 。

步 3 $\rho = \sigma u_1$ 。

在计算 σ 时, 可能上溢或下溢。为了避免溢出, 将 x 规范化

$$\eta = \max_i |\alpha_i|, \quad x' = \frac{x}{\eta},$$

显然

$$\sigma' = \sigma/\eta, \quad H' = H.$$

算法 2 已知 $x = (\alpha_1, \alpha_2, \dots, \alpha_n)^T \neq 0$, 本算法算出 H 及 σ 使 $Hx = -\sigma e_1$, u 的分量冲掉 x 的分量。

步 1 $\eta = \max_i |\alpha_i|$ 。

步 2 $\alpha_i \leftarrow u_i = \frac{\alpha_i}{\eta} \quad (i = 1, 2, \dots, n)$ 。

步 3 $\sigma = \text{sgn}(u_1) \left(\sum_{i=1}^n u_i^2 \right)^{\frac{1}{2}}$ 。

步 4 $u_1 \leftarrow u_1 + \sigma$ 。

步 5 $\rho = \sigma u_1$ 。

步 6 $\sigma \leftarrow \eta\sigma$ 。

关于 HA 的计算, 设 $A = (a_1, a_2, \dots, a_n)$, 其中 a_i 为 A 的第 i 列向量, 则

$$HA = (Ha_1, Ha_2, \dots, Ha_n),$$

因此计算 HA 就是要计算

$$Ha_i = (I - \rho^{-1}uu^T)a_i = a_i - (\rho^{-1}u^Ta_i)u \quad (i = 1, 2, \dots, n).$$

于是计算 Ha_i 只需要计算两向量的数量积和两向量的加法即可，且计算 HA 共需要 $2n^2$ 次乘法运算。

9.3.2 用正交相似变换约化矩阵

下面考虑用初等反射阵来正交相似约化一般矩阵和对称矩阵。设

原书 PDF 页 246; 印刷页码 233。

[原页图像](#)

$$A = \left(\begin{array}{c|ccc} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{array} \right) \equiv \begin{pmatrix} a_{11} & A_{12}^{(1)} \\ a_{21}^{(1)} & A_{22}^{(1)} \end{pmatrix},$$

步 1 不妨设 $a_{21}^{(1)} \neq 0$, 否则这一步不需约化, 选择初等反射阵 R_1 使 $R_1 a_{21}^{(1)} = -\sigma_1 e_1$, 其中

$$\begin{cases} \sigma_1 = \operatorname{sgn}(a_{21}) \left(\sum_{i=2}^n a_{i1}^2 \right)^{\frac{1}{2}}, \\ u_1 = a_{21}^{(1)} + \sigma_1 e_1, \\ \rho_1 = \frac{1}{2} \|u_1\|_2^2 = \sigma_1 (\sigma_1 + a_{21}), \\ R_1 = I - \rho_1^{-1} u_1 u_1^T. \end{cases} \quad (9.3.1)$$

令 $U_1 = \begin{pmatrix} I & 0 \\ 0 & R_1 \end{pmatrix}$, 则

$$A_2 = U_1 A_1 U_1 = \begin{pmatrix} a_{11} & A_{21}^{(1)} R_1 \\ R_1 a_{21}^{(1)} & R_1 A_{22}^{(1)} R_1 \end{pmatrix} \equiv \begin{pmatrix} A_{11}^{(2)} & a_{12}^{(2)} & A_{13}^{(2)} \\ O & a_{22}^{(2)} & A_{23}^{(2)} \end{pmatrix},$$

其中 $A_{11}^{(2)} \in \mathbf{R}^{2 \times 1}$, $a_{22}^{(2)} \in \mathbf{R}^{n-2}$, $A_{23}^{(2)} \in \mathbf{R}^{(n-2) \times (n-2)}$ 。

步 k 设对 A 已进行了第 $k-1$ 步正交相似约化, 即 A_k 有形式

$$\begin{aligned} A_k &= U_{k-1} A_{k-1} U_{k-1} \\ &= \left(\begin{array}{ccc|ccc} a_{11} & a_{12}^{(2)} & \cdots & a_{1k}^{(k)} & a_{1,k+1}^{(k)} & \cdots & a_{1n}^{(k)} \\ -\sigma_1 & a_{22}^{(2)} & \cdots & a_{2k}^{(k)} & a_{2,k+1}^{(k)} & \cdots & a_{2n}^{(k)} \\ & \ddots & \ddots & \vdots & \vdots & & \vdots \\ & & -\sigma_{k-1} & a_{kk}^{(k)} & a_{k,k+1}^{(k)} & \cdots & a_{k,n}^{(k)} \end{array} \right) \\ &\equiv \begin{pmatrix} A_{11}^{(k)} & a_{12}^{(k)} & A_{13}^{(k)} \\ O & a_{22}^{(k)} & A_{23}^{(k)} \end{pmatrix}, \end{aligned}$$

其中 $A_{11}^{(k)} \in \mathbf{R}^{k \times (k-1)}$, $a_{22}^{(k)} \in \mathbf{R}^{n-k}$, $A_{23}^{(k)} \in \mathbf{R}^{(n-k) \times (n-k)}$ 。

设 $a_{22}^{(k)} \neq 0$, 选择初等反射阵 R_k , 使 $R_k a_{22}^{(k)} = -\sigma_k e_1$, 其中

$$\begin{cases} \sigma_k = \operatorname{sgn}(a_{k+1,k}^{(k)}) \left(\sum_{i=k+1}^n a_{ik}^2 \right)^{\frac{1}{2}}, \\ u_k = a_{22}^{(k)} + \sigma_k e_1, \\ \rho_k = \frac{1}{2} \|u_k\|_2^2 = \sigma_k (\sigma_k + a_{k+1,n}^{(k)}), \\ R_k = I - \rho_k^{-1} u_k u_k^T. \end{cases} \quad (9.3.2)$$

校注 (agent 补充, ch09a-E009) 本页 A_2 的二阶分块表达式右上块原印 $A_{21}^{(1)} R_1$, 已照录。由页首定义的右上块 $A_{12}^{(1)}$ 和 U_1 的分块乘法, 此处应为 $A_{12}^{(1)} R_1$ 。见[页首分块](#)和[放大原式](#)。

校注 (agent 补充, ch09a-E010) 式 (9.3.2) 原图 σ_k 求和内为 a_{ik}^2 , 未印迭代上标; ρ_k 最后一个矩阵元为 $a_{k+1,n}^{(k)}$, 均照录。由本页 $a_{22}^{(k)} = (a_{k+1,k}^{(k)}, \dots, a_{nk}^{(k)})^T$ 和 $u_k = a_{22}^{(k)} + \sigma_k e_1$, 求和应使用当前列元素 $(a_{ik}^{(k)})^2$, 而范数平方恒等式给出 $\rho_k = \sigma_k (\sigma_k + a_{k+1,k}^{(k)})$ 。故前者疑漏迭代上标, 后者列下标疑将 k 印成 n 。见[放大原式](#)。

原书 PDF 页 247; 印刷页码 234。

原页

设 $U_k = \begin{pmatrix} I & O \\ O & R_k \end{pmatrix}$, 则

$$A_{k+1} = U_k A_k U_k = \begin{pmatrix} A_{11}^{(k)} & a_{12}^{(k)} & A_{13}^{(k)} R_k \\ O & R_k a_{22}^{(k)} & R_k A_{23}^{(k)} R_k \end{pmatrix} = \begin{pmatrix} A_{11}^{(k)} & a_{12}^{(k)} & A_{13}^{(k)} R_k \\ O & -\sigma_k e_1 & R_k A_{23}^{(k)} R_k \end{pmatrix}. \quad (9.3.3)$$

由式 (9.3.3) 知, A_{k+1} 的左上角 $k+1$ 阶子阵为上 Hessenberg 阵, 从而约化又进了一步, 重复这过程, 直到

$$U_{n-2} \cdots U_2 U_1 A U_1 U_2 \cdots U_{n-2} = \begin{pmatrix} a_{11} & \times & \times & \cdots & \times \\ -\sigma_1 & a_{22}^{(2)} & \times & \cdots & \times \\ & -\sigma_2 & a_{33}^{(3)} & \ddots & \vdots \\ & & \ddots & \ddots & \times \\ & & & -\sigma_{n-1} & a_{nn}^{(n-1)} \end{pmatrix} = A_{n-1}.$$

总结上述讨论, 有如下定理。

定理 9.13 如果 $A \in \mathbb{R}^{n \times n}$, 则存在初等反射阵 $U_1, U_2, \dots, U_{n-2}$, 使

$$U_{n-2} \cdots U_2 U_1 A U_1 U_2 \cdots U_{n-2} = C \quad (\text{上 Hessenberg 阵}).$$

在 $A_k \rightarrow A_{k+1} = U_k A_k U_k$ 的进一步约化中, 需要计算 R_k 和 $A_{13}^{(k)} R_k$, $R_k A_{23}^{(k)} R_k$ 。用初等反射阵正交相似约化 A 为上 Hessenberg 阵, 大约需要 $\frac{5}{3}n^3$ 次乘法运算。

由于 U_k 都是正交阵, 所以 $A_1 \sim A_2 \sim \cdots \sim A_{n-1}$ 。求 A 的特征值问题, 就转化为求上 Hessenberg 阵 C 的特征值问题。由定理 9.13, 记 $P = U_{n-2} \cdots U_2 U_1$, 则

$$P A P^T = C.$$

设 y 是 C 的对应特征值 λ 的特征向量, 则 $P^T y$ 为 A 的对应特征值 λ 的特征向量, 且

$$P^T y = U_1 U_2 \cdots U_{n-2} y = (I - \lambda_1^{-1} u_1 u_1^T) \cdots (I - \lambda_{n-2}^{-1} u_{n-2} u_{n-2}^T) y.$$

定理 9.14 如果 $A \in \mathbb{R}^{n \times n}$ 为对称阵, 则存在初等反射阵 $U_1, U_2, \dots, U_{n-2}$, 使

$$U_{n-2} \cdots U_2 U_1 A U_1 U_2 \cdots U_{n-2} = A_{n-1} = \begin{pmatrix} c_1 & b_1 & & & \\ b_1 & c_2 & b_2 & & \\ & \ddots & \ddots & \ddots & \\ & & b_{n-2} & c_{n-1} & b_{n-1} \\ & & & b_{n-1} & c_n \end{pmatrix} \equiv C.$$

证明 由定理 9.13, 存在初等反射阵 $U_1, U_2, \dots, U_{n-2}$, 使 A_{n-1} 为上 Hessenberg 阵, 但 A_{n-1} 又为对称阵, 因此 A_{n-1} 为对称三对角阵。

由上面讨论可知, 在由 $A_k \rightarrow A_{k+1} = U_k A_k U_k$ 一步计算过程中, 只需计算 R_k 和 $R_k A_{23}^{(k)} R_k$ 。由于 A 的对称性, 故只需计算 $R_k A_{23}^{(k)} R_k$ 的对角线下面的元素。注意到

$$R_k A_{23}^{(k)} R_k = (I - \rho_k^{-1} u_k u_k^T)(A_{23}^{(k)} - \rho_k^{-1} A_{23}^{(k)} u_k u_k^T),$$

校注 (agent 补充): 本页 $P^T y$ 展开式原印为 λ_1^{-1} 、 λ_{n-2}^{-1} , 而同页下方及次页初等反射阵的归一化因子使用 ρ_k^{-1} 。此处保留原印 λ , 记为疑似原书符号不一致。

原书 PDF 页 248; 印刷页码 235。

[原页](#)

引进记号

$$r_k = \rho_k^{-1} A_{23}^{(k)} u_k, \quad t_k = r_k - \frac{\rho_k^{-1}}{2} (u_k^T r_k) u_k,$$

则

$$R_k A_{23}^{(k)} R_k = A_{23}^{(k)} - u_k t_k^T - t_k u_k^T \quad (i = k+1, \dots, n; j = k+1, \dots, i).$$

算法 3 正交相似约化对称阵为对称三对角阵。设 $A \in \mathbb{R}^{n \times n}$ 是对称阵, 本算法确定初等反射阵 $U_1, U_2, \dots, U_{n-2}$, 使 $U_{n-2} \cdots U_1 A U_1 \cdots U_{n-2} = C$ (对称三对角阵), C 的对角元 c_i 存放在单元 $c_1, c_2, \dots, c_n$ 中, C 的非对角元 b_i 存放在单元 $b_1, b_2, \dots, b_{n-1}$ 中。单元 $b_1, b_2, \dots, b_n$ 最初可用来存放 r_k 及 t_k 的分量, 确定 U_k 的向量 u_k 的分量 $u_{k+1,k}, \dots, u_{nk}$, 存放在 A 的相应位置。 ρ_k 冲掉 a_{kk} 。约化 A 的结果冲掉 A , 数组 A 的上部元素不变。如果步 k 不需要变换, 则置 ρ_k 为零。

对于 $k = 1, 2, \dots, n-2$, 做到 L 。

步 1 $c_k = a_{kk}$ 。

步 2 确定变换:

(1) 计算 $\eta = \max_{k+1 \leq i \leq n} |a_{ik}|$;

(2) 如果 $\eta = 0$, 则

$$\begin{cases} a_{kk} \rightarrow \rho_k = 0, \\ b_k \rightarrow 0, \\ \text{转 } L, \text{ 否则继续;} \end{cases}$$

(3) 计算 $a_{ik} \leftarrow u_{ik} = a_{ik}/\eta \quad (i = k+1, \dots, n)$;

(4) $\sigma = \text{sgn}(u_{k+1,k}) \sqrt{u_{k+1,k}^2 + \dots + u_{nk}^2}$;

(5) $u_{k+1,k} \leftarrow u_{k+1,k} + \sigma$;

(6) $a_{kk} \leftarrow \rho_k = \sigma u_{k+1,k}$;

(7) $b_k \leftarrow -\sigma \eta$ 。

步 3 应用变换:

(1) $\sigma = 0$;

(2) 计算 $A_{23}^{(k)} u_k$ 及 $u_k^T r_k$, 对于 $i = k+1, \dots, n$, 做

$$\begin{cases} b_i \leftarrow s = \sum_{j=k+1}^n a_{ij} u_{jk} + \sum_{j=i+1}^n a_{ji} u_{jk}, \\ \sigma \leftarrow \sigma + s u_{ik}; \end{cases}$$

(3) 计算 t_k

$$b_i \leftarrow \rho_k^{-1} \left(b_i - \rho_k^{-1} \frac{\sigma}{2} u_{ik} \right) \quad (i = k+1, \dots, n);$$

(4) 计算 $R_k A_{23}^{(k)} R_k$

对于 $i = k+1, k+2, \dots, n; j = k+1, \dots, i$, 做 $a_{ij} \leftarrow a_{ij} - u_{ik} b_j - b_i u_{jk}$ 。

L : 继续循环。

对于 $k = n-1$, 有

$$c_{n-1} \leftarrow a_{n-1, n-1}, \quad c_n \leftarrow a_{nn}, \quad b_{n-1} \leftarrow a_{n, n-1}.$$

校注 (agent 补充): 算法 3 步 3 (2) 第一个求和上限原印为 n 。按只存储、更新下三角部分的算法结构, 该上限疑应为 i , 否则 $j > i$ 的项与第二个求和重叠, 且会读入未更新的上三角元素。正文保留原印, 列为疑似原书错误。

原书 PDF 页 249; 印刷页码 236。

原页

将对称阵 A 用初等反射阵正交相似约化为对称三对角阵约需做 $\frac{2}{3}n^3$ 次乘法运算。

用正交矩阵进行约化, 有一些特点, 如构造的 U_k 容易求逆, 且 U_k 的元素数量级不大, 因此这个算法是十分稳定的。

例 9.5 用 Householder 方法将下述矩阵化为上 Hessenberg 阵

$$A_1 = A = \begin{pmatrix} -4 & -3 & -7 \\ 2 & 3 & 2 \\ 4 & 2 & 7 \end{pmatrix}.$$

解 (1) 对于 $k=1$, 确定变换 $U_1 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & R_1 \\ 0 & \end{pmatrix}$, $a_{21}^{(1)} = \begin{pmatrix} 2 \\ 4 \end{pmatrix}$, 其中 R_1 为初等反射阵且使

$$R_1 a_{21}^{(1)} = -\sigma_1 \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \sigma_1 = \|a_{21}^{(1)}\|_2 = \sqrt{20} \approx 4.472136,$$

$$u_1 = a_{21}^{(1)} + \sigma_1 \rho_1 = \begin{pmatrix} 2 + \sqrt{20} \\ 4 \end{pmatrix} \approx \begin{pmatrix} 6.472136 \\ 4 \end{pmatrix},$$

$$\rho_1 = \sigma_1(\sigma_1 + a_{21}) = \sqrt{20}(\sqrt{20} + 2) \approx 28.94427, \quad R_1 = I - \rho_1^{-1} u_1 u_1^T.$$

(2) 计算 $R_1 A_{22}^{(1)}$ 。记

$$A_{22}^{(1)} = \begin{pmatrix} 3 & 2 \\ 2 & 7 \end{pmatrix} \equiv (a_1, a_2),$$

于是

$$R_1 A_{22}^{(1)} = (R_1 a_1, R_1 a_2) = \begin{pmatrix} -3.130496 & -7.155419 \\ -1.788855 & 1.341640 \end{pmatrix},$$

其中

$$R_1 a_i = (I - \rho_1^{-1} u_1 u_1^T) a_i = a_i - (\rho_1^{-1} u_1^T a_i) u_1 \quad (i=1, 2).$$

(3) 计算 $A_{12}^{(1)} R_1$ 及 $(R_1 A_{22}^{(1)}) R_1$, 即计算

$$\begin{pmatrix} A_{12}^{(1)} \\ (R_1 A_{22}^{(1)}) \end{pmatrix} R_1 \equiv \begin{pmatrix} b_1^T \\ b_2^T \\ b_3^T \end{pmatrix} R_1 = \begin{pmatrix} b_1^T R_1 \\ b_2^T R_1 \\ b_3^T R_1 \end{pmatrix} = \begin{pmatrix} 7.602634 & -0.447212 \\ 7.800003 & -0.399999 \\ -0.399999 & 2.200000 \end{pmatrix},$$

其中

$$b_i^T R_1 = b_i^T - (\rho_1^{-1} b_i^T u_1) u_1^T \quad (i = 1, 2, 3).$$

(4) 计算 $A_2 = U_1 A_1 U_1$ 。

$$A_2 = \begin{pmatrix} -4 & A_{12}^{(1)} R_1 \\ -\sigma_1 & R_1 A_{22}^{(1)} R_1 \\ 0 & \end{pmatrix} = \begin{pmatrix} -4 & 7.602\,634 & -0.447\,212 \\ -4.472\,136 & 7.800\,003 & -0.399\,999 \\ 0 & -0.399\,999 & 2.200\,000 \end{pmatrix}$$

为上 Hessenberg 阵。

校注 (agent 补充): 例 9.5 (1) 原印 $u_1 = a_{21}^{(1)} + \sigma_1 \rho_1$ 中的 ρ_1 已经由放大原图确认。结合紧随的列向量以及同段将 ρ_1 定义为标量的公式, 这里疑应为第一单位向量 e_1 ; 正文保留原印。

原书 PDF 页 250；印刷页码 237。

[原页](#)

9.4 QR 算法

9.4.1 引言

Rutishauser（在 1958 年）利用矩阵的三角分解提出了计算矩阵特征值的 LR 算法，Francis（在 1961 年、1962 年）利用矩阵的 QR 分解建立了计算矩阵特征值的 QR 方法。

QR 方法是一种变换方法，是计算一般矩阵（中小型矩阵）全部特征值问题的最有效的方法之一。目前，QR 方法主要用来计算：（1）上 Hessenberg 阵的全部特征值问题，（2）对称三对角阵的全部特征值问题。QR 方法具有收敛快、算法稳定等特点。

对于一般矩阵 $A \in \mathbb{R}^{n \times n}$ （或对称阵），首先用 Householder 方法将 A 化为上 Hessenberg 阵 B （或对称三对角阵），然后再用 QR 方法计算 B 的全部特征值问题。

事实上，除了可以用 Householder 法约化矩阵外，还可考虑用如下形式的平面旋转矩阵来约化：

$$P_{ij} = \begin{pmatrix} 1 & & & & & & & \\ & \ddots & & & & & & \\ & & 1 & & & & & \\ & & & c & & & s & \\ & & & & 1 & & & \\ & & & & & \ddots & & \\ & & & & & & 1 & \\ & & -s & & & & c & \\ & & & & & & & 1 \\ & & & & & & & & \ddots & \\ & & & & & & & & & 1 \end{pmatrix}, \quad (9.4.1)$$

矩阵原图标注： $c, -s$ 所在列为“第 i 列”， s, c 所在列为“第 j 列”； c, s 所在行为“第 i 行”， $-s, c$ 所在行为“第 j 行”。其中 $c = \cos \theta$, $s = \sin \theta$ 。

下面是用这种矩阵约化矩阵的一些结论。

引理 1 设 $x = (\alpha_1, \alpha_2, \dots, \alpha_i, \dots, \alpha_j, \dots, \alpha_n)^T$ ，其中 α_i, α_j 不全为零，则可选一平面旋转矩阵 P_{ij} 使 $P_{ij}x = y \equiv (\alpha_1, \alpha_2, \dots, \alpha_i^{(1)}, \dots, \alpha_j^{(1)}, \dots, \alpha_n)^T$ ，其中

$$\alpha_i^{(1)} = \sqrt{\alpha_i^2 + \alpha_j^2}, \quad (9.4.2)$$

$$\alpha_j^{(1)} = 0, \quad (9.4.3)$$

$$\begin{cases} c = \alpha_i / \sqrt{\alpha_i^2 + \alpha_j^2}, \\ s = \alpha_j / \sqrt{\alpha_i^2 + \alpha_j^2}. \end{cases} \quad (9.4.4)$$

原书 PDF 页 251; 印刷页码 238。

[原页](#)

证明 事实上, $P_{ij}x$ 只改变 x 的第 i 个及第 j 个元素, 且有

$$\alpha_i^{(1)} = c\alpha_i + s\alpha_j, \quad \alpha_j^{(1)} = -s\alpha_i + c\alpha_j,$$

于是可选 P_{ij} , 使 $\alpha_j^{(1)} = -s\alpha_i + c\alpha_j = 0$, 即按式 (9.4.4) 求 c, s , 且有式 (9.4.2) 及式 (9.4.3)。

用平面旋转阵进行左变换可产生一算法, 设给定 $x = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$, 计算 $c = \cos \theta$, $s = \sin \theta$, $v = \sqrt{\alpha^2 + \beta^2}$, 使 $P_{ij}x = \begin{pmatrix} c & s \\ -s & c \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} v \\ 0 \end{pmatrix}$, 为了防止溢出, 将 x 规范化, 有

$$\eta \equiv \|x\|_\infty = \max\{|\alpha|, |\beta|\} \neq 0, \quad x' = x/\eta = \begin{pmatrix} \alpha/\eta \\ \beta/\eta \end{pmatrix},$$

于是

$$\begin{cases} c' = c, & s' = s, \\ v' = v/\eta. \end{cases}$$

算法 1 设 $x = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$, 本算法产生 c, s 及 v 。

步 1 计算 $\eta = \max\{|\alpha|, |\beta|\}$ 。

步 2 如果 $\eta = 0$, 则 $c \leftarrow 1$, $s \leftarrow 0$, 转步 8。

步 3 $\alpha' = \alpha/\eta$ 。

步 4 $\beta' = \beta/\eta$ 。

步 5 $v' = \sqrt{\alpha'^2 + \beta'^2}$ 。

步 6 $c = \alpha'/v'$, $s = \beta'/v'$ 。

步 7 $v = \eta v'$ 。

步 8 计算终止。

定理 9.15 如果 A 为非奇异矩阵, 则存在正交矩阵 $P_1, P_2, \dots, P_{n-1}$ (即一系列平面旋转矩阵) 使

$$P_{n-1}, \dots, P_2 P_1 A = \begin{pmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ & r_{22} & \cdots & r_{2n} \\ & & \ddots & \vdots \\ & & & r_{nn} \end{pmatrix} \equiv R, \quad (9.4.5)$$

且 $r_{ii} > 0$ ($i = 1, 2, \dots, n-1$)。

证明 由于 A 的第 1 列一定存在 $a_{j1} \neq 0$, 于是, 如果 $a_{j1} \neq 0$ ($j = 2, 3, \dots, n$), 应用算法 1, 即存在平面旋转矩阵 $P_{12}, P_{13}, \dots, P_{1n}$, 使

$$P_{1n} \cdots P_{13} P_{12} A = \begin{pmatrix} r_{11} & a_{12}^{(2)} & \cdots & a_{1n}^{(2)} \\ 0 & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} \\ \vdots & \vdots & & \vdots \\ 0 & a_{n2}^{(2)} & \cdots & a_{nn}^{(2)} \end{pmatrix} \equiv A^{(2)},$$

且记 $P_{1n} \cdots P_{12} = P_1$ 。同理, 如果 $a_{2j}^{(2)} \neq 0$ ($j = 3, \dots, n$), 应用算法 1, 存在平面旋转矩阵

校注 (agent 补充): 引理 1 证明首式原印 $\alpha_j^{(1)} = -s\alpha_i + c\alpha_i$, 下一行则为 $-s\alpha_i + c\alpha_j$ 。按矩阵乘法, 首式末项疑应为 $c\alpha_j$; 正文保留原印。

校注 (agent 补充): 定理 9.15 证明末行原印 $a_{2j}^{(2)} \neq 0$ 。该步旋转消去第 2 列下方元素, 疑应为 $a_{j2}^{(2)} \neq 0$; 正文保留原印。

校注 (agent 补充): 算法 1 步 2 的零向量分支只指定 c, s 后即终止, 没有显式赋值 v 。按开头的 $v = \sqrt{\alpha^2 + \beta^2}$, 此分支应有 $v = 0$; 原算法步骤保留。

原书 PDF 页 252; 印刷页码 239。

[原页](#)

$P_{23}, \dots, P_{2n}$ (记 $P_2 = P_{2n} \cdots P_{23}$), 使

$$P_2 P_1 A = \begin{pmatrix} r_{11} & a_{12}^{(2)} & a_{13}^{(2)} & \cdots & a_{1n}^{(2)} \\ & r_{22} & a_{23}^{(3)} & \cdots & a_{2n}^{(3)} \\ & & a_{33}^{(3)} & \cdots & a_{3n}^{(3)} \\ & & \vdots & & \vdots \\ & & a_{n3}^{(3)} & \cdots & a_{nn}^{(3)} \end{pmatrix}.$$

重复上述过程, 最后得到: 存在正交阵 $P_1, P_2, \dots, P_{n-1}$, 使式 (9.4.5) 成立。

定理 9.16 (矩阵的 QR 分解) 如果 $A \in \mathbb{R}^{n \times n}$ 为非奇异矩阵, 则 A 可分解为一正交阵 Q 与上三角阵 R 的乘积, 即 $A = QR$, 且当 R 对角元素都为正数时分解唯一。

证明 由定理 9.12, 存在正交阵 $P_1, P_2, \dots, P_{n-1}$, 使

$$P_{n-1} \cdots P_2 P_1 A = R \quad (9.4.6)$$

为上三角阵, 记

$$Q^T = P_{n-1} \cdots P_2 P_1,$$

于是式 (9.4.6) 为 $Q^T A = R$, 即 $A = QR$, 其中 $Q = P_1^T P_2^T \cdots P_{n-1}^T$ 为正交阵。

现证唯一性: 设有 $A = Q_1 R_1 = Q_2 R_2$, 其中 R_1, R_2 为上三角阵 (显然为非奇异阵) 且对角元素都为正数, Q_1, Q_2 为正交阵。于是

$$Q_2^T Q_1 = R_2 R_1^{-1}, \quad (9.4.7)$$

由式 (9.4.7) 知上三角阵 $R_2 R_1^{-1}$ 为正交阵, 故 $R_2 R_1^{-1}$ 为对角阵, 即

$$R_2 R_1^{-1} = D = \text{diag}(d_1, d_2, \dots, d_n).$$

因为 $R_2 R_1^{-1}$ 是正交阵, 所以 $D^2 = I$, 又因 R_1, R_2 对角元素都为正数, 故 $d_i > 0$ ($i = 1, 2, \dots, n$), 即 $D = I$ 。于是 $R_2 = R_1$, 由式 (9.4.7) 得到 $Q_2 = Q_1$ 。

9.4.2 QR 算法

设 $A = A_1 = (a_{ij}) \in \mathbb{R}^{n \times n}$, 且对 A 进行 QR 分解, 即 $A = QR$, 其中 R 为上三角阵, Q 为正交阵, 于是可得到一新矩阵

$$B = RQ = Q^T A Q.$$

显然, B 是由 A 经过正交相似变换得到, 因此 B 与 A 特征值相同。再对 B 进行 QR 分解, 又可得一新的矩阵, 重复这过程可得到矩阵序列。

设 $A = A_1$, QR 分解, 得 $A_1 = Q_1 R_1$, 作矩阵 $A_2 = R_1 Q_1 = Q_1^T A_1 Q_1$, $\dots$, 求得 A_k 后将 A_k 进行 QR 分解, 得 $A_k = Q_k R_k$, 作矩阵 $A_{k+1} = R_k Q_k = Q_k^T A_k Q_k \dots$ 。

QR 算法, 就是利用矩阵的 QR 分解, 按上述递推法则构造矩阵序列 $\{A_k\}$ 的过程。只要 A 为非奇异矩阵, 则由 QR 算法就完全确定 $\{A_k\}$ 。

定理 9.17 (基本 QR 方法) 设 $A = A_1 \in \mathbb{R}^{n \times n}$, QR 算法为

$$\begin{cases} A_k = Q_k R_k & (Q_k^T Q_k = I, R_k \text{ 为上三角阵}), \\ A_{k+1} = R_k Q_k & (k = 1, 2, \dots), \end{cases} \quad (9.4.8)$$

校注 (agent 补充): 定理 9.16 证明首句原印“由定理 9.12”, 本段所需的平面旋转矩阵将 A 变成上三角阵的结论对应定理 9.15; 原引用保留, 列为疑似原书编号错误。

原书 PDF 页 253; 印刷页码 240。

原页

且记 $\tilde{Q}_k \equiv Q_1 Q_2 \cdots Q_k$, $\tilde{R}_k \equiv R_k \cdots R_2 R_1$, 则有

- 1° A_{k+1} 相似于 A_k , 即 $A_{k+1} = Q_k^T A_k Q_k$;
 2° $A_{k+1} = (Q_1 Q_2 \cdots Q_k)^T A_1 (Q_1 Q_2 \cdots Q_k) = \tilde{Q}_k^T A_1 \tilde{Q}_k$;
 3° A^k 的 QR 分解式为 $A^k = \tilde{Q}_k \tilde{R}_k$ 。

证明 结论 1°、2° 显然, 现用归纳法证结论 3°。显然, 当 $k=1$ 时有 $A_1 = \tilde{Q}_1 \tilde{R}_1 = Q_1 R_1$, 设 A^{k-1} 有分解式 $A^{k-1} = \tilde{Q}_{k-1} \tilde{R}_{k-1}$, 于是

$$\begin{aligned} \tilde{Q}_k \tilde{R}_k &= Q_1 Q_2 \cdots (Q_k R_k) \cdots R_1 = Q_1 Q_2 \cdots Q_{k-1} A_k R_{k-1} \cdots R_1 \\ &= \tilde{Q}_{k-1} A_k \tilde{R}_{k-1} = A \tilde{Q}_{k-1} \tilde{R}_{k-1} = A^k \quad (\text{因为 } A_k = \tilde{Q}_{k-1}^T A \tilde{Q}_{k-1}) . \end{aligned}$$

由定理 9.13 知, 将 A_k 进行 QR 分解, 即将 A_k 用正交变换 (左变换) 化为上三角阵。

$$Q_k^T A_k = R_k, \quad A_{k+1} = Q_k^T A_k Q_k = P_{n-1} \cdots P_2 P_1 A_k P_1^T P_2^T \cdots P_{n-1}^T,$$

其中

$$Q_k^T = P_{n-1} \cdots P_2 P_1.$$

这就是说 A_{k+1} 可由 A_k 按下述方法求得:

(1) 左变换 $P_{n-1} \cdots P_2 P_1 A_k = R_k$ (上三角阵); (2) 右变换 $R_k P_1^T P_2^T \cdots P_{n-1}^T = A_{k+1}$ 。

引理 2 设 $M_k = Q_k R_k$, 其中 Q_k 为正交阵, R_k 为具有正对角元素的上三角阵, 如果 $M_k \rightarrow I$ ($k \rightarrow \infty$), 则 $Q_k \rightarrow I$, 及 $R_k \rightarrow I$ ($k \rightarrow \infty$)。

证明 设 $R_k^T R_k = M_k^T M_k \rightarrow I$ ($k \rightarrow \infty$), 记 $R_k = (r_{ij}^{(k)})$, 矩阵 $R_k^T R_k$ 第 1 行是

$$r_{11}^{(k)} \cdot (r_{11}^{(k)}, r_{12}^{(k)}, \cdots, r_{1n}^{(k)}),$$

因此有

$$r_{11}^{(k)} \rightarrow 1, r_{12}^{(k)} \rightarrow 0, \cdots, r_{1n}^{(k)} \rightarrow 0 \quad (k \rightarrow \infty). \quad (9.4.9)$$

$R_k^T R_k$ 第 2 行是

$$r_{12}^{(k)} \cdot (r_{11}^{(k)}, r_{12}^{(k)}, \cdots, r_{1n}^{(k)}) + r_{22}^{(k)} \cdot (0, r_{22}^{(k)}, r_{23}^{(k)}, \cdots, r_{2n}^{(k)}),$$

利用式 (9.4.9) 的结果, 则有

$$r_{22}^{(k)} \rightarrow 1, r_{23}^{(k)} \rightarrow 0, \dots, r_{2n}^{(k)} \rightarrow 0 \quad (k \rightarrow \infty). \quad (9.4.10)$$

对于 $R_k^T R_k$ 其他行同理可得, 故 $R_k \rightarrow I \ (k \rightarrow \infty)$, 且易知有 $R_k^{-1} \rightarrow I \ (k \rightarrow \infty)$, 因此 $Q_k = M_k R_k^{-1} \rightarrow I \ (k \rightarrow \infty)$ 。

定理 9.18 (QR 方法的收敛性) 设 $A = (a_{ij}) \in \mathbb{R}^{n \times n}$,

1° 如果 A 的特征值满足: $|\lambda_1| > |\lambda_2| > \dots > |\lambda_n| > 0$;

2° A 有标准形 $A = XDX^{-1}$ 其中 $D = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$, 且设 X^{-1} 有三角分解 $X^{-1} = LU$ (L 为单位下三角阵, U 为上三角阵), 则由 QR 算法产生的 $\{A_k\}$ 本质上收敛于上三角阵, 即

$$A_k \xrightarrow{\text{本质上}} R = \begin{pmatrix} \lambda_1 & \times & \cdots & \times \\ & \lambda_2 & \ddots & \vdots \\ & & \ddots & \times \\ & & & \lambda_n \end{pmatrix} \quad (k \rightarrow \infty)$$

校注(agent 补充): 本页将 QR 分解引用为“定理 9.13”, 而 PDF247 上定理 9.13 的结论是约化为上 Hessenberg 阵; 本页所述左变换为上三角阵的结论见定理 9.15。保留原引用, 列为疑似原书编号错误。

原书 PDF 页 254; 印刷页码 241。

[原页](#)

或

$$a_{ii}^{(k)} \rightarrow \lambda_i \quad (k \rightarrow \infty), \quad (9.4.11)$$

当 $i > j$ 时,

$$a_{ij}^{(k)} \rightarrow 0 \quad (k \rightarrow \infty). \quad (9.4.12)$$

当 $i < j$ 时, $a_{ij}^{(k)}$ 极限不一定存在。

证明 由于 $A_{k+1} = \tilde{Q}_k^T A_1 \tilde{Q}_k$ 且 $\tilde{Q}_k$ 为 A^k 的 QR 分解中的正交矩阵。下面来确定 $\tilde{Q}_k$ 的表达式, 进而考虑 A_{k+1} 的极限情况。

由于 A 为非奇异矩阵, 所以存在非奇异矩阵 X 使 $X^{-1}AX = D$, 则

$$A^k = XD^kX^{-1}, \quad (9.4.13)$$

又有假设 $X^{-1} = LU$, 于是式 (9.4.13) 为

$$A^k = XD^kLU = X(D^kLD^{-k})D^kU.$$

显然

$$D^kLD^{-k} = I + E_k,$$

其中

$$E_k = \begin{pmatrix} 0 & & & & \\ \left(\frac{\lambda_2}{\lambda_1}\right)^k l_{21} & 0 & & & \\ \left(\frac{\lambda_3}{\lambda_1}\right)^k l_{31} & \left(\frac{\lambda_3}{\lambda_2}\right)^k l_{32} & 0 & & \\ \vdots & \vdots & \ddots & \ddots & \\ \left(\frac{\lambda_n}{\lambda_1}\right)^k l_{n1} & \left(\frac{\lambda_n}{\lambda_2}\right)^k l_{n2} & \cdots & \left(\frac{\lambda_n}{\lambda_{n-1}}\right)^k l_{n,n-1} & 0 \end{pmatrix}.$$

由假设条件 $|\lambda_i/\lambda_j| < 1$ (当 $i > j$ 时), 则 $E_k \rightarrow O$ ($k \rightarrow \infty$) 且

$$\|E_k\|_\infty \leq c \max_{1 \leq j \leq n-1} |\lambda_{j+1}/\lambda_j|^k \quad (c \text{ 为正的常数, } k \geq 1). \quad (9.4.14)$$

显然矩阵 X 有 QR 分解: $X = QR$, 其中 Q 为正交阵, R 为非奇异上三角阵。于是

$$A^k = QR(I + E_k)D^kU = Q(I + RE_kR^{-1})RD^kU. \quad (9.4.15)$$

由于 $R(I + E_k)$ (当 k 充分大时) 为非奇异, 则 $I + RE_kR^{-1}$ 亦非奇异, 于是 $(I + RE_kR^{-1})$ 有 QR 分解 (要求 R_k 对角元素均为正)

$$I + RE_kR^{-1} = Q_kR_k \quad \text{且} \quad Q_kR_k \rightarrow I \quad (\text{当 } k \rightarrow \infty \text{ 时}),$$

由引理 2 有 $Q_k \rightarrow I, R_k \rightarrow I$ (当 $k \rightarrow \infty$ 时), 由式 (4.15) 有

$$A^k = (QQ_k)(R_kRD^kU), \quad (9.4.16)$$

式 (9.4.16) 为 A^k 的 QR 分解式, 但 R_kRD^kU (为上三角阵) 对角元素不一定大于零, 现引入对角阵

$$D_k = \text{diag}(\pm 1, \pm 1, \dots, \pm 1),$$

以便保证 $D_k(R_kRD^kU)$ 对角元素都为正数。从而得到 A^k 的 QR 分解式

$$A^k = (QQ_kD_k)(D_kR_kRD^kU),$$

由 A^k 矩阵 QR 分解的唯一性得到

$$\begin{cases} \tilde{Q}_k = QQ_kD_k, \\ \tilde{R}_k = D_kR_kRD^kU, \end{cases} \quad (9.4.17)$$

校注 (agent 补充): 式 (9.4.16) 前原印 “由式 (4.15) 有”, 本页对应公式编号为 (9.4.15)。正文保留原印编号。

原书 PDF 页 255; 印刷页码 242。

[原页](#)

从而

$$A_{k+1} = \tilde{Q}_k^T A \tilde{Q}_k = D_k Q_k^T (RDR^{-1}) Q_k D_k \quad (\text{注意 } Q^T A Q = RDR^{-1}),$$

其中

$$R_0 \equiv RDR^{-1} = \begin{pmatrix} \lambda_1 & \times & \cdots & \times \\ & \lambda_2 & \ddots & \vdots \\ & & \ddots & \times \\ & & & \lambda_n \end{pmatrix},$$

于是

$$A_{k+1} = g_k^T R_0 g_k,$$

其中

$$\begin{cases} g_k = Q_k D_k, \\ R_0 = RDR^{-1} \quad (\text{上三角阵}), \\ Q_k \rightarrow I \quad (k \rightarrow \infty), \\ D_k \text{ 为对角阵, 其元素为 } +1 \text{ 或 } -1. \end{cases}$$

由此即证得式 (9.4.11)、式 (9.4.12)。且收敛速度依赖于 $Q_k \rightarrow I$ 收敛速度; 即依赖于式 (9.4.14) 的界。

定理 9.19 如果对称阵 A 满足定理 9.18 条件, 则由 QR 算法产生的 $\{A_k\}$ 收敛于对角阵。

证明 由定理 9.18 即知。下面提一下关于 QR 算法收敛性的另一结果。

设 $A \in \mathbb{R}^{n \times n}$, 如果 A 的等模特征值中只有实重特征值或多重复的共轭特征值, 则由 QR 算法产生 $\{A_k\}$ 本质收敛于分块上三角阵 (对角块为一阶和二阶子块) 且对角块每一个 2×2 子块给出 A 的一对共轭复特征值, 每一个对角子块给出 A 的实特征值, 即

$$A_k \rightarrow \begin{pmatrix} \lambda_1 & \times & \cdots & \times & \times \times & \cdots & \times \times \\ & \lambda_2 & \ddots & \vdots & \vdots & & \vdots \\ & & \ddots & \times & \vdots & & \vdots \\ & & & \lambda_m & \times \times & \cdots & \times \times \\ & & & & B_l & \ddots & \vdots \\ & & & & & \ddots & \times \times \\ & & & & & & B_l \end{pmatrix}.$$

其中 $m + 2l = n$, B_i 为 2×2 子块给出 A 一对共轭特征值。

9.4.3 带原点位移的 QR 方法

在定理 9.18 证明中进一步分析可知, $a_{nn}^{(k)} \rightarrow \lambda_n$ ($k \rightarrow \infty$) 速度依赖于比值 $r_n = |\lambda_n/\lambda_{n-1}|$, 当 r_n 很小时, 收敛较快, 如果 s 为 λ_n 一个估计, 且对 $A - sI$ 运用 QR 算法, 则 $(n, n-1)$ 元素将以收敛因子 $\left| \frac{\lambda_n - s}{\lambda_{n-1} - s} \right|$ 线性收敛于零, (n, n) 元素将比在基本算法中收敛更快。

校注 (agent 补充): 上方分块上三角阵中第一个二阶对角块原印为 B_l , 与末块同名; 放大字形确认后照录。
按 $m + 2l = n$ 及顺序编号, 第一个二阶块疑应为 B_1 。

原书 PDF 页 256; 印刷页码 243。

[原页](#)

为此, 为了加速收敛, 选择数列 $\{s_k\}$, 按下述方法构造矩阵序列 $\{A_k\}$, 称为带原点位移的 QR 算法。

步 1 设 $A = A_1 \in \mathbb{R}^{n \times n}$ 。

步 2 将 $A_k - s_k I$ 进行 QR 分解, 即 $A_k - s_k I = Q_k R_k$, $k = 1, 2, \dots$ 。

步 3 构造新矩阵 $A_{k+1} = R_k Q_k + s_k I = Q_k^T A_k Q_k$ 。

步 4 $A_{k+1} = \tilde{Q}_k^T A \tilde{Q}_k$, 其中 $\tilde{Q}_k = Q_1 Q_2 \cdots Q_k$, $\tilde{R}_k = R_k \cdots R_2 R_1$ 。

步 5 矩阵 $(A - s_1 I)(A - s_2 I) \cdots (A - s_k I) \equiv \varphi(A)$ 有 QR 分解式 $\varphi(A) = \tilde{Q}_k \tilde{R}_k$ 。

步 6 带位移 QR 方法变换一步的计算: 首先用正交变换 (左变换) 将 $A_k - s_k I$ 化为上三角阵, 即

$$P_{n-1} \cdots P_2 P_1 (A_k - s_k I) = R_k,$$

其中 $Q_k^T = P_{n-1} \cdots P_2 P_1$ 为一系列平面旋转矩阵的乘积。于是

$$A_{k+1} = P_{n-1} \cdots P_2 P_1 (A_k - s_k I) P_1^T P_2^T \cdots P_{n-1}^T + s_k I.$$

下面考虑用 QR 算法计算上 Hessenberg 阵特征值。

设

$$A = A_1 = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ & \ddots & \ddots & \vdots \\ & & a_{n,n-1} & a_{nn} \end{pmatrix}, \quad A \in \mathbb{R}^{n \times n}.$$

(1) 左变换计算。选择平面旋转阵 $P_{12}, P_{23}, \dots, P_{n-1,n}$, 使

$$P_{n-1,n} \cdots P_{23} P_{12} (A_1 - s_1 I) = R.$$

首先

$$a_{ii} \leftarrow a_{ii} - s_1 \quad (i = 1, 2, \dots, n).$$

第一次左变换, 选择平面旋转阵 P_{12} , 使第 2 行第 1 列元素为零。

$$P_{12}(A_1 - s_1 I) = \begin{pmatrix} v_1 & a_{12}^{(2)} & a_{13}^{(2)} & \cdots & a_{1n}^{(2)} \\ a_{22}^{(2)} & a_{23}^{(2)} & \cdots & a_{2n}^{(2)} \\ a_{32} & a_{33} & \cdots & a_{3n} \\ & \ddots & \ddots & \vdots \\ & & a_{n,n-1} & a_{nn} \end{pmatrix}.$$

设已完成第 $i-1$ 次左变换, 则

$$P_{i-1,i} \cdots P_{23} P_{12} (A_1 - s_1 I) = \begin{pmatrix} v_1 & a_{12}^{(2)} & \cdots & \cdots & \cdots & \cdots & \cdots & a_{1n}^{(2)} \\ & v_2 & a_{23}^{(2)} & \cdots & \cdots & \cdots & \cdots & a_{2n}^{(2)} \\ & & \ddots & \ddots & & & & \vdots \\ & & & v_{i-1} & a_{i-1,i}^{(i-1)} & \cdots & \cdots & a_{i-1,n}^{(i-1)} \\ & & & & \boxed{\begin{matrix} a_{ii}^{(i)} & \cdots & \cdots & a_{in}^{(i)} \\ a_{i+1,i} & \ddots & & a_{i+1,n} \\ & \ddots & \ddots & \vdots \\ & & a_{n,n-1} & a_{nn} \end{matrix}} & & & \end{pmatrix}.$$

原书 PDF 页 257; 印刷页码 244。

[原页](#)

现进行第 i 次左变换, 选择 $P_{i,i+1}$ (常数 c_i, s_i) 及 v_k 使 $(i+1, i)$ —元素为零, 则有

$$P_{i,i+1} \cdots P_{23} P_{12} (A_1 - s_1 I) = \begin{pmatrix} v_1 & \times & \cdots & \cdots & \cdots & \cdots & \times \\ & v_2 & \times & \cdots & \cdots & \cdots & \times \\ & & \ddots & \ddots & & & \vdots \\ & & & v_i & \times & \cdots & \times \\ & & & & \boxed{\begin{matrix} \times & \times & \cdots & \times \\ \times & \ddots & \ddots & \vdots \\ & \ddots & \ddots & \times \\ & & \times & \times \end{matrix}} & & & \end{pmatrix},$$

继续这过程, 最后

$$P_{n-1,n} \cdots P_{23} P_{12} (A_1 - s_1 I) = R \quad (\text{上三角阵}),$$

其中 $P_{i,i+1}$ ($i = 1, 2, \cdots, n+1$) 为平面旋转阵。

(2) 右变换计算。计算 $RP_{12}^T P_{23}^T \cdots P_{n-1,n}^T$, 其中上三角阵 R 元素仍记作 a_{ij} ($i \leq j$), 于是

$$RP_{12}^T = \begin{pmatrix} a_{11}^{(2)} & a_{12}^{(2)} & a_{13} & \cdots & a_{1n} \\ a_{21}^{(2)} & a_{22}^{(2)} & a_{23} & \cdots & a_{2n} \\ & & a_{33} & \cdots & a_{3n} \\ & & & \ddots & \vdots \\ & & & & a_{nn} \end{pmatrix},$$

$$RP_{12}^T P_{23}^T = \begin{pmatrix} a_{11}^{(2)} & a_{12}^{(3)} & a_{13}^{(3)} & a_{14} & \cdots \\ a_{21}^{(2)} & a_{22}^{(3)} & a_{23}^{(3)} & a_{24} & \cdots \\ & a_{32}^{(3)} & a_{33}^{(3)} & a_{34} & \cdots \\ & & a_{43}^{(3)} & a_{44} & \cdots \\ & & & \ddots & \ddots \end{pmatrix}.$$

继续这过程, 最后

$$RP_{12}^T P_{23}^T \cdots P_{n-1,n}^T = \begin{bmatrix} \times & \times & \cdots & \cdots & \times \\ \times & \times & \cdots & \cdots & \times \\ & \times & \ddots & & \vdots \\ & & \ddots & \ddots & \vdots \\ & & & \times & \times \end{bmatrix} \quad (\text{为上 Hessenberg 阵}),$$

故 $A_2 = RP_{12}^T P_{23}^T \cdots P_{n-1,n}^T + s_1 I$ 为上 Hessenberg 阵。

由上面的构造可知, 如果 A 为上 Hessenberg 阵, 则用 QR 算法产生的 $A_2, A_3, \dots, A_k, \dots$ 亦是上 Hessenberg 阵。显然, 每一次左变换仅改变矩阵的两行, 而每一次右变换仅改变矩阵的两列。为了节省存储量, 左变换和右变换可以同时进行, 例如

$$\begin{aligned} P_{23}P_{12}(A - s_1I) &\rightarrow P_{23}P_{12}(A - s_1I)P_{12}^T \rightarrow P_{34}P_{23}P_{12}(A - s_1I)P_{12}^T \\ &\rightarrow P_{34}P_{23}P_{12}(A - s_1I)P_{12}^TP_{23}^T \rightarrow \dots \end{aligned}$$

实际计算时, 用不同位移 $s_1, s_2, \dots, s_k, \dots$ 反复应用上述变换就产生一正交相似于上 Hessenberg 阵 A 的序列 $\{A_k\}$, 如果选取 $s_k = a_{nn}^{(k)}$, 那么当 $a_{n,n-1}^{(k)}$ 充分小, A_k 有形式

校注 (agent 补充): 本页开头第 i 次左变换原印“及 v_k ”, 对应矩阵对角位置为 v_i , 疑为下标不一致; 保留原印。

校注 (agent 补充): 左变换最后一式之后原印 $i = 1, 2, \dots, n+1$ 。因 $P_{i,i+1}$ 作用于 n 阶矩阵, 且乘积末项为 $P_{n-1,n}$, 上限疑应为 $n-1$; 保留原印。

原书 PDF 页 258; 印刷页码 245。

[原页](#)

$$\left(\begin{array}{cccc|c} \times & \times & \times & \cdots & \times \\ & & \times & \times & \cdots & \times \\ & & & \ddots & \ddots & \vdots \\ & & & & \times & \times \\ \hline & & & & & \lambda_n \end{array} \right)_{n \times n} \equiv \left(\begin{array}{c|c} B & \begin{smallmatrix} \times \\ \times \\ \vdots \\ \times \end{smallmatrix} \\ \hline O & \lambda_n \end{array} \right),$$

则数 $\lambda_n = a_{nn}^{(k)}$ 为 A 的近似特征值。采用收缩方法, 继续对 $B \in \mathbb{R}^{(n-1) \times (n-1)}$ 应用 QR 算法, 就可逐步求出 A 其余近似特征值。

判别 $a_{n,n-1}^{(k)}$ 充分小的准则如下:

- (1) $|a_{n,n-1}^{(k)}| \leq \varepsilon \|A\|_{\infty}$;
- (2) 或将 $a_{n,n-1}^{(k)}$ 与相邻元素进行比较 $|a_{n,n-1}^{(k)}| \leq \varepsilon \min(|a_{nn}^{(k)}|, |a_{n-1,n-1}^{(k)}|)$, 其中 $\varepsilon = 10^{-t}$, t 是计算中有效数字的个数。

上述应用带位移的 QR 算法, 可计算上 Hessenberg 阵 A 所有特征值, 但不能计算 A 复特征值, 因为上述 QR 算法是在实数中进行计算的, 位移 $s_k = a_{nn}$ 不能逼近一个复特征值。关于避免复数运算求上 Hessenberg 阵复特征值的 QR 算法——隐式位移的 QR 算法请参看有关文献。

一般来说位移 s_k 常用的选取方法有两种:

- (1) 选取 $s_k = a_{nn}^{(k)}$;
- (2) 选取 s_k 是 2×2 矩阵 $\begin{pmatrix} a_{n-1,n-1}^{(k)} & a_{n-1,n}^{(k)} \\ a_{n,n-1}^{(k)} & a_{nn}^{(k)} \end{pmatrix}$, 特征值 λ 且 $|a_{nn}^{(k)} - \lambda|$ 为最小 (记 $A_k = (a_{ij}^{(k)})$)。

例 9.6 用 QR 方法计算对称三对角阵的全部特征值

$$A = A_1 = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 4 \end{pmatrix}.$$

解 采用第一种选位移方法, 即选 $s_k = a_{nn}^{(k)}$ 。又 $s_1 = 4$ 。

$$P_{23}P_{12}(A_1 - s_1I) = R = \begin{pmatrix} 2.2361 & -1.342 & 0.4472 \\ & 1.0954 & -0.3651 \\ & & 0.81650 \end{pmatrix},$$

$$A_2 = RP_{12}^T P_{23}^T + s_1I = \begin{pmatrix} 1.4000 & 0.4899 & 0 \\ 0.4899 & 3.2667 & 0.7454 \\ 0 & 0.7454 & 4.3333 \end{pmatrix},$$

$$A_3 = \begin{pmatrix} 1.2915 & 0.2017 & 0 \\ 0.2017 & 3.0202 & 0.2724 \\ 0 & 0.2724 & 4.6884 \end{pmatrix}, \quad A_4 = \begin{pmatrix} 1.2737 & 0.0993 & 0 \\ 0.0993 & 2.9943 & 0.0072 \\ 0 & 0.0072 & 4.7320 \end{pmatrix},$$

原书 PDF 页 259; 印刷页码 246。

[原页](#)

$$A_5 = \begin{pmatrix} 1.2694 & 0.0498 & 0 \\ 0.0498 & 2.9986 & 0 \\ 0 & 0 & \boxed{4.7321} \end{pmatrix}, \quad \tilde{A}_5 = \begin{pmatrix} 1.2694 & 0.0498 \\ 0.0498 & 2.9986 \end{pmatrix}.$$

现在收缩, 继续对 A_5 的子矩阵 $\tilde{A}_5 \in \mathbb{R}^{2 \times 2}$ 进行变换, 得到

$$\tilde{A}_6 = P_{12}(\tilde{A}_5 - s_5 I)P_{12}^T + s_5 I = \begin{pmatrix} \boxed{1.2680} & -4 \times 10^{-5} \\ -4 \times 10^{-5} & \boxed{3.0000} \end{pmatrix},$$

故求得 A 近似特征值为

$$\lambda_1 \approx 4.7321, \quad \lambda_2 \approx 3.0000, \quad \lambda_3 \approx 1.2680,$$

且 A 的特征值是

$$\lambda_1 = 3 + \sqrt{3} \approx 4.7321, \quad \lambda_2 = 3.0, \quad \lambda_3 = 3 - \sqrt{3} \approx 1.2679.$$

小结

本章介绍了计算一般矩阵主特征值及对应特征向量的迭代法 (幂法), 这种方法在计算过程中原始矩阵 A 始终不变。因此, 这种方法适于求高阶稀疏矩阵的特征值问题。主特征值为复特征值情况可参看文献 [11]。

反幂法是计算矩阵特征向量的一个有效方法, 主要用来计算 Hessenberg 阵或三对角阵对应于一个给定的近似特征值的特征向量。

本章还介绍了正交相似变换的方法, 例如正交相似约化一般矩阵 A 为上 Hessenberg 阵的 Householder 方法, 计算一般矩阵 A 全部特征值问题的 QR 方法, 这些方法都是利用矩阵的正交相似变换约化矩阵 A 为某种简单形式, 以期求 A 的特征值的方法。这种方法将破坏原始矩阵。QR 方法 (与 Householder 方法结合使用) 收敛快, 精度高, 是求任意实矩阵 (中、小型) 特征值问题的最有效的方法之一。对计算矩阵特征值问题有兴趣的读者可参看文献 [10]、[13]、[14]。

习题

1. 用幂法计算下列矩阵的主特征值及对应的特征向量:

$$(1) \quad A_1 = \begin{pmatrix} 7 & 3 & -2 \\ 3 & 4 & -1 \\ -2 & -1 & 3 \end{pmatrix}; \quad (2) \quad A_2 = \begin{pmatrix} 3 & -4 & 3 \\ -4 & 6 & 3 \\ 3 & 3 & 1 \end{pmatrix},$$

当特征值有三位小数稳定时迭代终止。

2. 方阵 T 分块形式为

$$T = \begin{pmatrix} T_{11} & T_{12} & \cdots & T_{1n} \\ & T_{22} & \cdots & T_{2n} \\ & & \ddots & \vdots \\ & & & T_{nn} \end{pmatrix},$$

其中 T_{ii} ($i = 1, 2, \cdots, n$) 为方阵, T 称为块上

原书 PDF 页 260; 印刷页码 247。

原页

三角阵, 如果对角块的阶数至多不超过 2, 则称 T 为准三角形形式。用 $\sigma(T)$ 记矩阵 T 的特征值集合, 证明 $\sigma(T) = \bigcup_{i=1}^n \sigma(T_{ii})$ 。

3. 利用反幂法求矩阵 $\begin{pmatrix} 6 & 2 & 1 \\ 2 & 3 & 1 \\ 1 & 1 & 1 \end{pmatrix}$ 的最接近于 6 的特征值及对应的特征向量。

4. 求矩阵 $\begin{pmatrix} 4 & 0 & 0 \\ 0 & 3 & 1 \\ 0 & 1 & 3 \end{pmatrix}$ 与特征值 4 对应的特征向量。

5. (1) 设 A 是对称矩阵, λ 和 x ($\|x\|_2 = 1$) 是 A 的一个特征值及相应的特征向量, 又设 P 为一个正交阵, 使 $Px = e_1 = (1, 0, \dots, 0)^T$, 证明 $B = PAP^T$ 的第 1 行和第 1 列除了 λ 之外其余元素均为零。

(2) 对于矩阵 $A = \begin{pmatrix} 2 & 10 & 2 \\ 10 & 5 & -8 \\ 2 & -8 & 11 \end{pmatrix}$, $\lambda = 9$ 是其特征值, $x = \left(\frac{2}{3}, \frac{1}{3}, \frac{2}{3}\right)^T$ 是相应于 9 的特征向量, 试求一初等反射阵 P , 使 $Px = e_1$, 并计算 $B = PAP^T$ 。

6. 利用初等反射阵将 $A = \begin{pmatrix} 1 & 3 & 4 \\ 3 & 1 & 2 \\ 4 & 2 & 1 \end{pmatrix}$ 正交相似约化为对称三对角阵。

7. 设 $A \in \mathbb{R}^{n \times n}$, 且 a_{i1}, a_{j1} 不全为零, P_{ij} 为使 $a_{j1}^{(2)} = 0$ 的平面旋转阵, 试推导 $P_{ij}A$ 第 i 行、第 j 行元素的计算公式及 AP_{ij}^T 第 i 列、第 j 列元素的计算公式。

8. 设 A_{n-1} 是由 Householder 方法得到的矩阵, 又设 y 是 A_{n-1} 的一个特征向量。

(1) 证明矩阵 A 对应的特征向量是 $x = P_1 P_2 \cdots P_{n-2} y$; (2) 对于给出的 y 应如何计算 x ?

9. 用带位移的 QR 方法计算下列矩阵的全部特征值:

$$(1) \quad A = \begin{pmatrix} 1 & 2 & 0 \\ 2 & -1 & 1 \\ 0 & 1 & 3 \end{pmatrix}; \quad (2) \quad B = \begin{pmatrix} 3 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 1 \end{pmatrix}.$$

10. 试用初等反射阵将

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 2 & -1 & -1 \\ 2 & -4 & 5 \end{pmatrix}$$

分解为 QR , 其中 Q 为正交阵, R 为上三角阵。

原书 PDF 页 261；印刷页码 248。

[原页](#)

下篇 高效算法设计

* 第 10 章 快速算法设计：快速 Walsh 变换

10.1 美的 Walsh 函数

随着大规模集成电路技术的广泛应用，信号普遍采取数字脉冲波形的形式。三角函数（即简谐波）不便于描述这类信号。为满足实际需要，人们考虑选用阶跃函数类的基函数。Walsh 函数就是阶跃函数类中一个完备的正交函数系。

Walsh 函数的一个显著特点是取值简单。它们仅取 +1 和 -1 两个值，因而可以方便地利用开关元件产生和处理数字信号。以 Walsh 函数为基底的线性变换称作 Walsh 变换。

与 Fourier 变换比较，Walsh 变换有其特点与优势。快速 Walsh 变换仅涉及加减运算而不含乘除操作，因而比快速 Fourier 变换更为迅捷。

Walsh 分析有着深刻的内涵。20 世纪 70 年代初，著名应用数学家 Harmuth 曾惊人地预言：**Walsh 分析的研究将导致一场数学革命，就像 17、18 世纪的微积分那样。**

这是一场什么样的“数学革命”呢？

10.1.1 微积分的逼近法

经典数学的基础是微积分。从微积分的观点看，在一切函数中，以多项式最为简单。能否用简单的多项式来逼近一般函数呢？众所周知的 Taylor 分析（1715 年）肯定了这一事实。Taylor 级数

$$f(x) \sim \sum_{k=0}^{\infty} \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k$$

表明，一般的光滑函数 $f(x)$ 可用多项式来近似地刻画。Taylor 分析是 18 世纪初的一项重大的数学成就。

然而 Taylor 分析存在严重的缺陷：它的条件很苛刻，要求 $f(x)$ 足够光滑并提供出它的各阶导数值 $f^{(k)}(x_0)$ ；此外，Taylor 分析的整体逼近效果差，它仅能保证在展开点 x_0 的某个邻域内有效。

时移物换，百年之后 Fourier（1822 年）指出，“任何函数，无论怎样复杂，均可表示为三角级数的形式”：

原书 PDF 页 262；印刷页码 249。

[原页](#)

$$f(x) \sim \frac{a_0}{2} + \sum_{k=1}^{\infty} (a_k \cos 2\pi kx + b_k \sin 2\pi kx), \quad 0 \leq x < 1.$$

这就是今日被称作“Fourier 分析”的数学方法。著名数学家 M. Kline 评价这一数学成就是“19 世纪数学的第一大步，并且是真正极为重要的一步”。

Fourier 关于任意函数都可以表达为三角级数这一思想被誉为“数学史上最大胆，最辉煌的概念”。

Fourier 的成就使人们从 Taylor 分析的理想函数类中解放出来。Fourier 分析不仅放宽了光滑性的限制，还保证了整体的逼近效果。

从数学美的角度来看，Fourier 分析也比 Taylor 分析更美，其基函数系——三角函数系是个完备的正交函数系。尤其值得注意的是，这个函数系可以视作是由一个简单函数 $\cos x$ 经过简单的伸缩平移变换加工生成的。Fourier 分析表明，任何复杂函数都可以借助于简单函数 $\cos x$ 来刻画，即

$$\cos x \xrightarrow{\text{伸缩+平移}} \text{三角函数系} \xrightarrow{\text{组合}} \text{任意函数 } f(x).$$

这是一个惊人的事实。在这里，被逼近函数 $f(x)$ 的“繁”与逼近工具 $\cos x$ 的“简”两者反差很大，因此 Fourier 逼近很美。Fourier 分析在数学史上被誉“一首数学的诗”，Fourier 则有“数学诗人”的美称。

10.1.2 Walsh 函数的复杂性

1923 年，美国数学家 J. L. Walsh 又提出了一个完备的正交函数系，后人将其称作 Walsh 函数系。第 k 族 Walsh 函数含有 2^k 个函数，其中第 i 个函数 W_{ki} 有如下解析表达式：

$$W_{ki}(x) = \prod_{r=0}^{k-1} \text{sgn}[\cos i_r 2^r \pi x], \quad 0 \leq x < 1,$$

$$k = 0, 1, 2, \dots, \quad i = 0, 1, \dots, 2^k - 1,$$

式中， sgn 是**符号函数**，当 $x \geq 0$ 时 $\text{sgn}[x]$ 取值 $+1$ ，而 $x < 0$ 时取值 -1 。又 i_r 取值 0 或 1 是序数 i 的二进制码：

$$i = \sum_{r=0}^{k-1} i_r 2^r.$$

图 10.1 列出前面 16 个 Walsh 函数的波形。其中，第 1 个（标号 0）组成第 0 族，前两个（标号 0 与 1）组成第 1 族；前 4 个（标号 0、1、2、3）组成第 2 族，依此类推，前 16 个组成第 4 族 Walsh 函数。

Walsh 函数取值简单，它们仅取 ± 1 两个值，但其波形却很复杂，似乎比三角函数要复杂得多，以致依据定义很难作出它们的图形。

由于表达式中含有符号运算 sgn ，Walsh 函数的波形频繁起伏，甚至“几乎处处”不连续（见图 10.1），经典的微积分方法在这里难以施展身手，Walsh 函数系的形态

原书 PDF 页 263；印刷页码 250。

原页

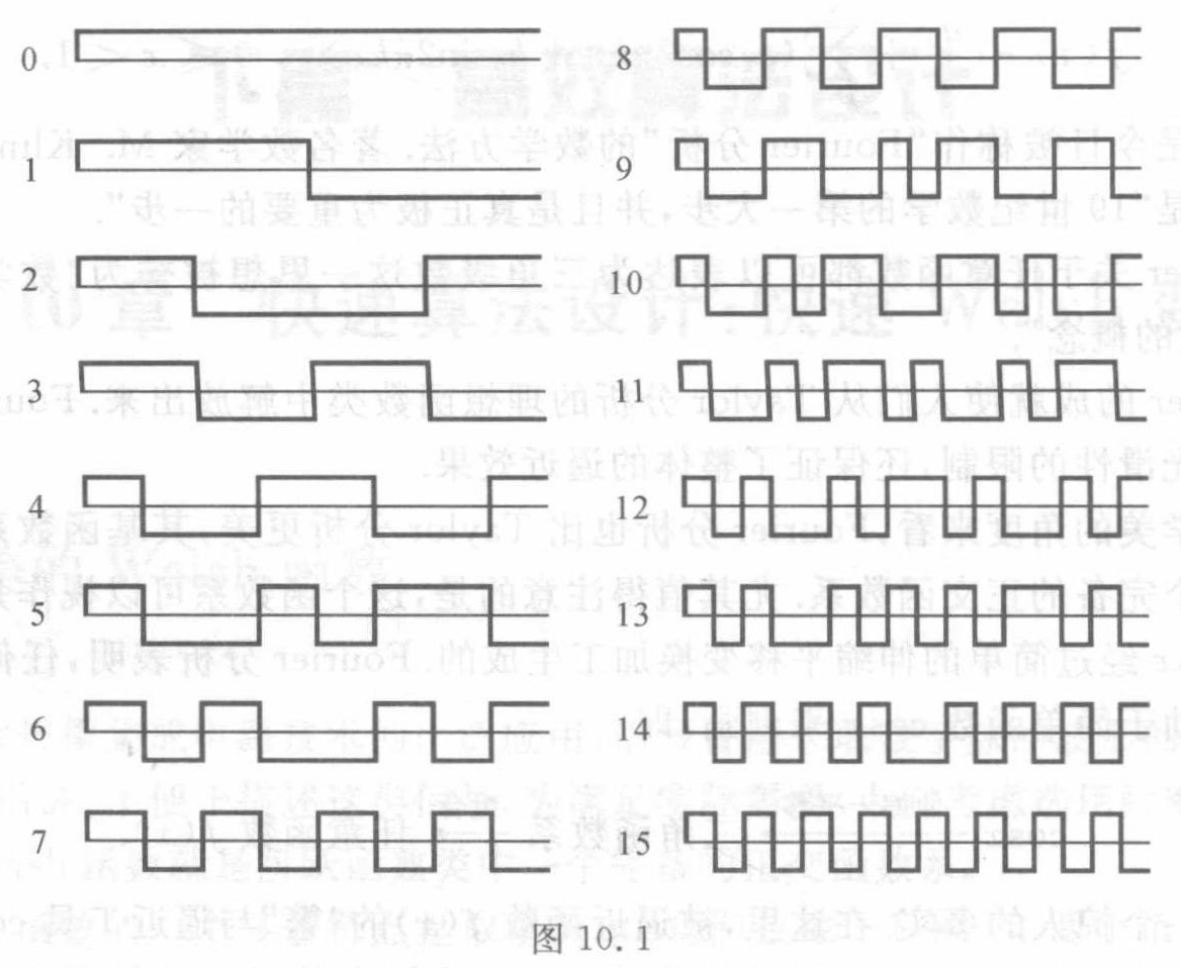


图 10.1

图中文字与图意（转录）：左列波形标号依次为 0、1、2、3、4、5、6、7，右列依次为 8、9、10、11、12、13、14、15。各条波形在水平基线上下取两个阶跃值；标号 0 的波形为方波。

怪异与表达式复杂使人们对它望而却步，在提出后的许多年里，它一直默默无闻，不被人们所重视。

直到 20 世纪 60 年代末，人们才惊异地发现，Walsh 函数可应用于信号处理的众多领域，诸如通信、声呐、雷达、图像处理、语音识别、遥感遥测遥控、仪表、医学、天文、地质等等。

“真”是“美”的反光。有着广泛应用的 Walsh 函数美在哪里呢？

10.1.3 Walsh 分析的数学美

后文将揭示出一个惊人的事实：表面看起来极其复杂的 Walsh 函数系，竟然是由一个简单得不能再简单的方波 $R(x) = 1$ 演化生成的。实际上，从方波 $R(x)$ 出发，经过伸缩、平移的二分手续，即可演化生成 Walsh 函数系。Walsh 函数系是个完备的正交函数系，它可以用来逼近一般的复杂函数。这样，Walsh 逼近有下述路线图：

$$R(x) = 1 \xrightarrow[\text{(二分手续)}]{\text{伸缩+平移}} \text{Walsh 函数系} \xrightarrow{\text{组合}} \text{复杂函数} f(x).$$

与 Fourier 分析相比, Walsh 分析更为简洁, 它表明: 在某种意义上, 任何复杂函数 $f(x)$ 都是简单的方波 $R(x) = 1$ 二分演化的结果。

综上所述, 数学史上近三个世纪提出的三种逼近方法, 即 18 世纪初 (1715 年) 的 Taylor 分析、19 世纪初 (1822 年) 的 Fourier 分析和 20 世纪初 (1923 年) 的 Walsh 分析, 它们都是数学美的光辉典范, 是“百年绝唱三首数学诗”。

这些逼近工具一个比一个更美。Fourier 分析具有深度的数学美, 而 Walsh 分析

原书 PDF 页 264；印刷页码 251。

[原页](#)

则具有极度的数学美。

问题在于，为了撩开 Walsh 函数玄妙而神秘的面纱，必须换一种思维方式进行考察。

10.2 Walsh 函数代数化

本节将限定在区间 $[0, 1)$ 上考察 Walsh 函数。由于自变量 x 在实际应用中通常代表时间，因此称区间 $[0, 1)$ 为**时基**。

10.2.1 时基上的二分集

由图 10.1 可以看出，Walsh 函数是时基上的阶跃函数，每个 Walsh 函数在给定分划的每个子段上取定值 $+1$ 或 -1 。怎样刻画 Walsh 函数所依赖的分划呢？

为便于刻画 Walsh 函数的跃变特征，首先引进二分集的概念。设将时基 $E_1 = [0, 1)$ 对半二分，其左右两个子段合并为集 E_2 ，即

$$E_2 = \left[0, \frac{1}{2}\right) \cup \left[\frac{1}{2}, 1\right).$$

再将 E_2 的每个子段对半二分，又得含有 4 个子段的区间集 E_4 ，即

$$E_4 = \left[0, \frac{1}{4}\right) \cup \left[\frac{1}{4}, \frac{1}{2}\right) \cup \left[\frac{1}{2}, \frac{3}{4}\right) \cup \left[\frac{3}{4}, 1\right).$$

如此二分下去，二分 n 次所得的区间集含有 $N = 2^n$ 个子段，即

$$E_N = \bigcup_{i=0}^{N-1} \left[\frac{i}{N}, \frac{i+1}{N}\right), \quad n = 0, 1, 2, \dots$$

这样得出的区间集 E_N ， $N = 1, 2, 4, \dots$ 称作时基上的**二分集**（见图 10.2）。

在二分集的每个子段上取定值的函数称作二分集上的**阶跃函数**。阶跃函数在某一子段上的函数值称作**阶跃值**。

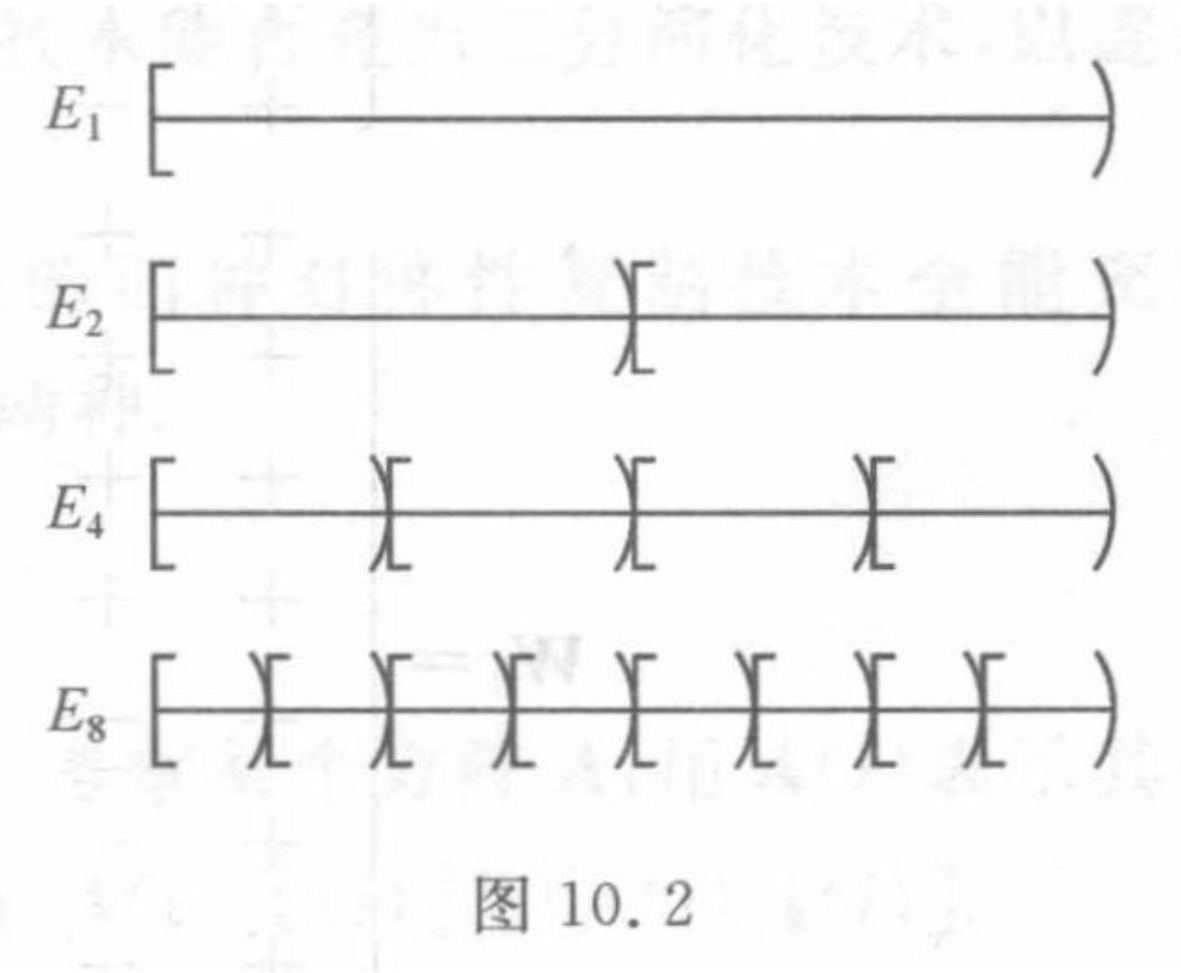


图 10.2

图 10.2

图中文字与图意（转录）：四行标号为 E_1 、 E_2 、 E_4 、 E_8 ，每行表示左闭右开的子段，按行依次二分。

现在的问题是，如何在二分集的各个子段上布值 $+1$ 与 -1 以设计出一个完备的正交函数系？实际上，这种函数系就是 Walsh 函数系。

为规范起见，约定 Walsh 函数第一个阶跃值（即最左侧的子段上的函数值）为 $+1$ ，如图 10.1 所示。

在形形色色的 Walsh 函数中，最简单的自然是**方波**

$$R(x) = 1, \quad 0 \leq x < 1.$$

然而这个函数过于平凡而显得“空虚”，其中似乎不含任何信息。“波”的含义是波动、起伏。按这种理解，时基上的方波似乎不能算作真正的“波”。具有波动性的最简单的

原书 PDF 页 265；印刷页码 252。

[原页](#)

波形是下列 **Haar 波**：

$$H(x) = \begin{cases} +1, & 0 \leq x < \frac{1}{2}, \\ -1, & \frac{1}{2} \leq x < 1. \end{cases}$$

由图 10.1 知，方波与 Haar 波是 Walsh 函数系的源头。

10.2.2 Walsh 函数的矩阵表示

Walsh 函数仅取 +1 与 -1 两个值。为简约起见，后文常将 +1 与 -1 简记为“+”与“-”。

由于 Walsh 函数在二分集的每个子段上取值 + 或 -，因而它们可表示为某个向量，而第 n 族 Walsh 函数的全体则可表达为一个 $N = 2^n$ 阶方阵，称作 **Walsh 方阵**。

据图 10.1 容易看出，前面几个 Walsh 方阵分别是

$$W_1 = [+],$$

$$W_2 = \begin{pmatrix} + & + \\ + & - \end{pmatrix},$$

$$W_4 = \begin{pmatrix} + & + & + & + \\ + & + & - & - \\ + & - & - & + \\ + & - & + & - \end{pmatrix},$$

$$W_8 = \begin{pmatrix} + & + & + & + & + & + & + & + \\ + & + & + & + & - & - & - & - \\ + & + & - & - & - & - & + & + \\ + & + & - & - & + & + & - & - \\ + & - & - & + & + & - & - & + \\ + & - & - & + & - & + & + & - \\ + & - & + & - & - & + & - & + \\ + & - & + & - & + & - & + & - \end{pmatrix}.$$

请读者据图 10.1 列出 Walsh 方阵 W_{16} 。

Walsh 方阵看上去是个复杂系统，这个复杂系统中究竟潜藏着怎样的规律性呢？

10.3 Walsh 阵的二分演化

现在的问题是，能否设计出某种简单的二分手续，以将方波 $W_1 = [+]$ 逐步演化

原书 PDF 页 266；印刷页码 253。

[原页](#)

生成各阶 Walsh 方阵，即

$$W_1 \Rightarrow W_2 \Rightarrow W_4 \Rightarrow W_8 \Rightarrow \cdots.$$

这里箭头“ $\Rightarrow$ ”表示所要设计的二分演化手续。

二分手续应当是简单而有效的。对于矩阵演化，什么样的演化手续最为简单呢？

10.3.1 矩阵的对称性复制

就矩阵演化来说，最为简单的演化手续是对称性复制。这种演化手续易于在计算机上实现，而且有丰富的文化内涵。

大自然的基本设计是美的，美意味着简单，美意味着对称。本节所考察的对称性分镜像对称与平移对称两种，它们在某种意义上互为反对称。镜像对称又分偶对称与奇对称，平移对称又分正对称与反对称。此外，矩阵的复制对象分矩阵行与矩阵块两种情况，这样，Walsh 方阵的对称性复制可考虑表 10.1 所列的四种方案。

表 10.1

对称性	复制对象：矩阵行	复制对象：矩阵块
镜像对称	镜像行复制	镜像块复制
平移对称	平移行复制	平移块复制

人们自然关心，矩阵的上述几种对称性复制技术能否充当二分演化技术，以逐步演化生成各种 Walsh 方阵呢？

答案是令人振奋的。事实上，表 10.1 所列的四种对称性复制技术全能充当 Walsh 演化的二分手续。后文将着重考察其中的两种。

10.3.2 Walsh 阵的演化生成

首先考察表 10.1 中镜像行复制的演化方式。考察某个方阵 A ，用 $A(i)$ 表示其第 i 行，对 $A(i)$ 施行偶复制与奇复制，分别生成向量 $[A(i) : \ddot{A}(i)]$ 与 $[A(i) : \dot{A}(i)]$ 。

例如，设 $A(i) = [+ \ -]$ ，则

$$[A(i) : \ddot{A}(i)] = [+ \ - : - \ +],$$

$$[A(i) : \dot{A}(i)] = [+ \ - : + \ -].$$

进一步，若 $A(i) = [+ \ - : + \ -]$ ，则

$$[A(i) : \ddot{A}(i)] = [+ \ - \ + \ - : - \ + \ - \ +],$$

$$[A(i) : \dot{A}(i)] = [+ \ - \ + \ - : + \ - \ + \ -].$$

如果对方阵 A 的每一行先后施行偶复制与奇复制两种复制手续，即可生成一个阶数倍增的方阵 B ，这种演化手续称作**镜像行复制**，即

原书 PDF 页 267；印刷页码 254。

[原页](#)

$$A = \begin{pmatrix} \vdots \\ A(i) \\ \vdots \end{pmatrix} \rightarrow B = \begin{pmatrix} \vdots \\ A(i) : \ddot{A}(i) \\ A(i) : \dot{A}(i) \\ \vdots \end{pmatrix}.$$

人们自然会问，如果对方波 $[+]$ 反复施行镜像行复制的演化手续，使其阶数逐步倍增，将会生成什么样的方阵序列呢？

1 阶方阵 $[+]$ 仅有一行（一列），对它施行偶复制与奇复制，分别生成 $[+ : +]$ 与 $[+ : -]$ ，两者合成在一起，结果生成一个 2 阶方阵

$$[+] \rightarrow \begin{pmatrix} + & + \\ + & - \end{pmatrix}.$$

对所生成的 2 阶方阵的两行 $[+ +]$ 与 $[+ -]$ 分别施行镜像复制的偶复制与奇复制，进一步生成一个 4 阶方阵

$$\begin{pmatrix} + & + \\ + & - \end{pmatrix} \rightarrow \begin{pmatrix} + & + & + & + \\ + & + & - & - \\ + & - & - & + \\ + & - & + & - \end{pmatrix}.$$

继续对所生成的 4 阶方阵施行镜像行复制，获得如下 8 阶方阵，即

$$\begin{pmatrix} + & + & + & + \\ + & + & - & - \\ + & - & - & + \\ + & - & + & - \end{pmatrix} \rightarrow \begin{pmatrix} + & + & + & + & + & + & + & + \\ + & + & + & + & - & - & - & - \\ + & + & - & - & - & - & + & + \\ + & + & - & - & + & + & - & - \\ + & - & - & + & + & - & - & + \\ + & - & - & + & - & + & + & - \\ + & - & + & - & - & + & - & + \\ + & - & + & - & + & - & + & - \end{pmatrix}.$$

上述方阵与 10.2.2 小节列出的 Walsh 方阵相比较，两者完全一致。可以证明，从方波 $[+]$ 出发，运用镜像行复制的演化技术可以生成 Walsh 方阵序列。这个 Walsh 方阵等价于 10.1.2 小节的原始定义。

Walsh 方阵有多种排序方式。镜像行复制生成的 Walsh 方阵特别称作 **Walsh 阵**。

10.3.3 Walsh 阵的演化机制

Walsh 阵的演化方式从属于二分演化模式。事实上，这里从初态 $W_1 = [+]$ 出发，将 $W_{N/2}$ 加工成 W_N 的二分手续如下。

(1) 分裂手续。

原书 PDF 页 268；印刷页码 255。

[原页](#)

对 $W_{N/2}$ 的每一行 $W_{N/2}(i)$ 分别施行偶复制与奇复制，生成两个 N 维向量 $[W_{N/2}(i) : \ddot{W}_{N/2}(i)]$ 与 $[W_{N/2}(i) : \dot{W}_{N/2}(i)]$ 。

(2) 合成手续。

将 $W_{N/2}$ 的每一行按上述分裂手续扩展为相邻的两个 N 维向量，从而将 $W_{N/2}$ 扩展成为一个 N 阶方阵

$$W_N = \begin{pmatrix} \vdots & \vdots \\ W_{N/2}(i) : \ddot{W}_{N/2}(i) \\ W_{N/2}(i) : \dot{W}_{N/2}(i) \\ \vdots & \vdots \end{pmatrix},$$

如此反复地做下去。这种二分演化机制如图 10.3 所示。如 10.1.2 小节所看到的，Walsh 函数的表达式很复杂，直接利用表达式生成 Walsh 函数很困难。然而依据上述镜像行复制的演化方式，一蹴而就地派生出一个又一个 Walsh 方阵，从而得到一族又一族 Walsh 函数。Walsh 函数的数目是逐族倍增的，这是一种快速生成算法。

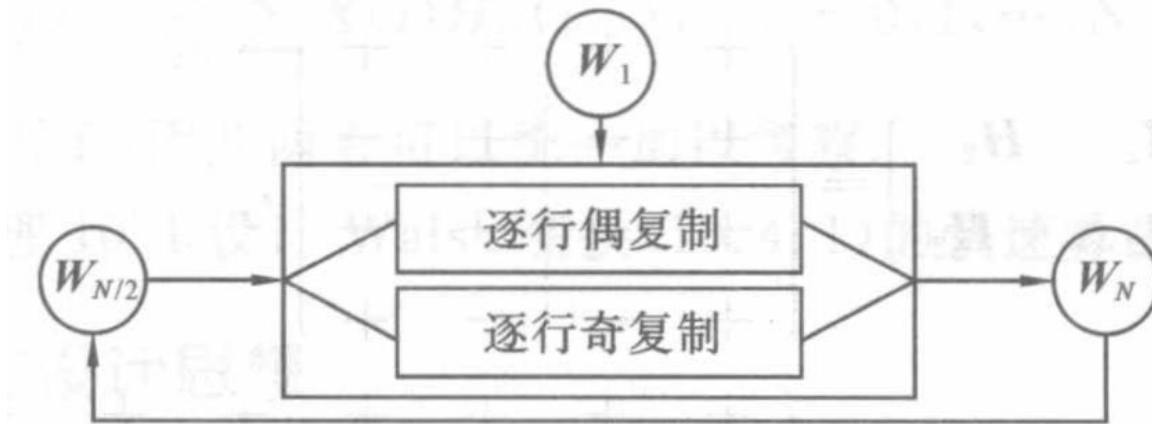


图 10.3

图 10.3

图中文字与图意（转录）：初态 W_1 从上方进入； $W_{N/2}$ 分成“逐行偶复制”和“逐行奇复制”两路，合成为 W_N ； W_N 经下方反馈线返回 $W_{N/2}$ 。

10.3.4 Hadamard 阵的演化生成

上述镜像行复制的演化方式能否进一步简化呢？

从研究者的角度来说，平移对称比镜像对称更易于接受，而矩阵块比矩阵行更易于把握，现在进一步考察表 10.1 所列的平移块复制的演化方式。

考察某个方阵 A ，直接对它施行平移正复制与平移反复制，分别生成 $[A : A]$ 与 $[A : -A]$ ，两者合成在一起，得阶数倍增的方阵

$$B = \begin{pmatrix} A & A \\ A & -A \end{pmatrix}.$$

这种演化方式称作**平移块复制**。

仍然从方波 $[+]$ 出发，反复施行平移块复制的演化方式，所生成的一系列方阵称作 **Hadamard 阵**。 N 阶 Hadamard 阵记作 H_N 。特别地， $H_1 = [+]$ 。

显然，Hadamard 阵的演化机制同样从属于二分演化模式，如图 10.4 所示。

按平移块复制的演化方式，如果将 H_N 对分为 4 块，则其左上、右上与左下三块均为 $H_{N/2}$ ，而其右下则为 $-H_{N/2}$ ，即 Hadamard 阵有形式简单的递推表达式

原书 PDF 页 269；印刷页码 256。

[原页](#)

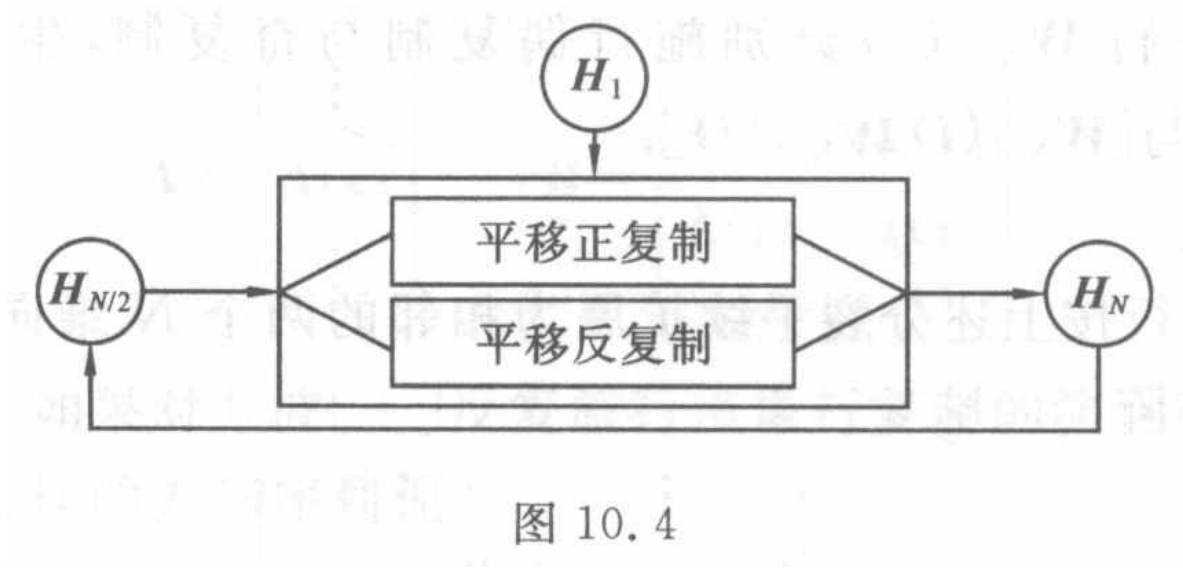


图 10.4

图中文字与图意（转录）：初态 H_1 从上方进入； $H_{N/2}$ 分成“平移正复制”和“平移反复制”两路，合成为 H_N ； H_N 经下方反馈线返回 $H_{N/2}$ 。

$$H_N = \begin{pmatrix} H_{N/2} & H_{N/2} \\ H_{N/2} & -H_{N/2} \end{pmatrix}, \quad N = 2, 4, 8, \dots \quad (10.3.1)$$

可见，Hadamard 阵的演化过程是简单的。事实上，从方波 $[+]$ 出发，按式 (10.3.1) 反复施行演化手续，有

$$H_1 = [+],$$

$$H_2 = \begin{pmatrix} H_1 & H_1 \\ H_1 & -H_1 \end{pmatrix} = \begin{pmatrix} + & + \\ + & - \end{pmatrix},$$

$$H_4 = \begin{pmatrix} H_2 & H_2 \\ H_2 & -H_2 \end{pmatrix} = \begin{pmatrix} + & + & + & + \\ + & - & + & - \\ + & + & - & - \\ + & - & - & + \end{pmatrix},$$

$$H_8 = \begin{pmatrix} H_4 & H_4 \\ H_4 & -H_4 \end{pmatrix} = \begin{pmatrix} + & + & + & + & + & + & + & + \\ + & - & + & - & + & - & + & - \\ + & + & - & - & + & + & - & - \\ + & - & - & + & + & - & - & + \\ + & + & + & + & - & - & - & - \\ + & - & + & - & - & + & - & + \\ + & + & - & - & - & - & + & + \\ + & - & - & + & - & + & + & - \end{pmatrix}.$$

如此继续下去，可以证明，这样演化生成的 Hadamard 阵同样是一种 Walsh 方阵。这里的 Hadamard 阵同 Walsh 阵相比较，两者只是行（列）的排序方式不同而已。

进一步考察矩阵元素的递推关系。前已指出，如果将矩阵 H_N 对分为 4 块，则其左上、右上与左下 3 块均为 $H_{N/2}$ ，而右下块则为 $-H_{N/2}$ 。记 $H_N(i, j)$ 为矩阵 H_N 第 i 行第 j 列的元素，则上下两组**平移对** (i, j) 、 $(i, N/2 + j)$ 与 $(N/2 + i, j)$ 、 $(N/2 + i, N/2 + j)$ 的矩阵元素有定理 10.1 所述的关系。

定理 10.1 对于 $0 \leq i, j \leq N/2 - 1$ ，有

$$\begin{aligned} H_N(i, j) &= H_N(i, N/2 + j) = H_N(N/2 + i, j) \\ &= -H_N(N/2 + i, N/2 + j) = H_{N/2}(i, j). \end{aligned}$$

原书 PDF 页 270；印刷页码 257。

[原页](#)

现在基于 Hadamard 阵的上述表达式设计 Walsh 变换的快速算法 FWT。FWT 的设计同样从属于二分演化模式。

10.4 快速变换 FWT

不同排序方式的 Walsh 变换，其快速算法的设计方法彼此类同。本节将着重考察 Hadamard 序的 Walsh 变换 N -WT

$$X(i) = \sum_{j=0}^{N-1} x(j) H_N(i, j), \quad i = 0, 1, \dots, N-1, \quad (10.4.1)$$

式中， H_N 为 N 阶 Hadamard 阵， $\{x(j)\}_0^{N-1}$ 为输入数据，输出数据 $\{X(i)\}_0^{N-1}$ 待求。这里仍然假定 $N = 2^n$ ， n 为正整数。

由于 Hadamard 阵是对称正交阵，Walsh 变换 (10.4.1) 同它的逆变换

$$x(j) = \frac{1}{N} \sum_{i=0}^{N-1} X(i) H_N(i, j), \quad j = 0, 1, \dots, N-1$$

仅仅相差一个常数因子，因此两者可以统一加以考察。

本节将基于定理 10.1 设计 Walsh 变换 (10.4.1) 的快速算法 FWT。

校注 (agent 补充, CH10-E002): 原文“对称正交阵”照录。依本章未归一化的 ± 1 元素定义， $H_N^T H_N = NI$ ， $N > 1$ 时并非通常定义下满足 $Q^T Q = I$ 的正交矩阵；准确地说其行列两两正交， $H_N/\sqrt{N}$ 才是正交矩阵。原页逆变换中的 $1/N$ 因子与该定义一致。本项是原书术语疑误，不是 OCR 错误。

10.4.1 FWT 的设计思想

在具体设计快速算法 FWT 之前，首先考察两种简单情形。由于 1 阶和 2 阶 Hadamard 阵为

$$H_1 = [+],$$

$$H_2 = \begin{pmatrix} + & + \\ + & - \end{pmatrix},$$

因而 1-WT 具有极其简单的形式

$$X(0) = x(0).$$

这里输入数据即为所求结果，因而不需要做任何计算。此外，2-WT 为

$$\begin{cases} X(0) = x(0) + x(1), \\ X(1) = x(0) - x(1). \end{cases}$$

这项计算也很平凡，不存在算法设计问题。

可见，1-WT 与 2-WT 都是极为简单的。

快速算法 FWT 的设计思想是，基于规模减半的二分手续，通过 2-WT 的反复计算，将所给 N -WT 逐步加工成 1-WT，从而得出所求的结果。

快速算法 FWT 是优秀算法的一朵奇葩，它鲜明地展现了“简单的重复生成复杂”这一算法设计的基本理念。此外，它可以充当一个样板，示范运用二分演化机制设

原书 PDF 页 271；印刷页码 258。

[原页](#)

计快速变换的全过程。

10.4.2 FWT 的演化机制

前已反复指出，二分技术是快速算法设计的基本技术。二分技术的基本点是运用某种二分手续，将所给计算问题化归为规模减半的同类问题。

对于 N 点 Walsh 变换 N -WT (10.4.1)，即

$$X(i) = \sum_{j=0}^{N-1} x(j)H_N(i, j), \quad i = 0, 1, \dots, N-1.$$

将其右端的和式对半拆开，有

$$\begin{aligned} X(i) &= \sum_{j=0}^{N/2-1} x(j)H_N(i, j) + \sum_{j=N/2}^{N-1} x(j)H_N(i, j) \\ &= \sum_{j=0}^{N/2-1} [x(j)H_N(i, j) + x(N/2 + j)H_N(i, N/2 + j)], \quad i = 0, 1, \dots, N-1. \end{aligned}$$

然后再将这组算式对半分为两组算式，有

$$\begin{cases} X(i) = \sum_{j=0}^{N/2-1} [x(j)H_N(i, j) + x(N/2 + j)H_N(i, N/2 + j)], \\ X(N/2 + i) = \sum_{j=0}^{N/2-1} [x(j)H_N(N/2 + i, j) + x(N/2 + j)H_N(N/2 + i, N/2 + j)], \end{cases}$$

$$i = 0, 1, \dots, N/2 - 1.$$

利用定理 1 的递推关系将上述算式化简，得

$$\begin{cases} X(i) = \sum_{j=0}^{N/2-1} [x(j) + x(N/2 + j)]H_{N/2}(i, j), \\ X(N/2 + i) = \sum_{j=0}^{N/2-1} [x(j) - x(N/2 + j)]H_{N/2}(N/2 + i, j), \end{cases} \quad i = 0, 1, \dots, N/2 - 1.$$

这样，所给 N -WT (10.4.1) 被加工成下列两个 $N/2$ -WT:

$$\begin{cases} X(i) = \sum_{j=0}^{N/2-1} x_1(j)H_{N/2}(i, j), \\ X(N/2 + i) = \sum_{j=0}^{N/2-1} x_1(N/2 + j)H_{N/2}(N/2 + i, j), \end{cases} \quad i = 0, 1, \dots, N/2 - 1.$$

为此所要施行的二分手续是

$$\begin{cases} x_1(j) = x(j) + x(N/2 + j), \\ x_1(N/2 + j) = x(j) - x(N/2 + j), \end{cases} \quad j = 0, 1, \dots, N/2 - 1 \quad (10.4.2)$$

上述二分手续将所给 N -WT 加工成 2 个 $N/2$ -WT。每个 $N/2$ -WT 通过二分手续可进一步加工成 2 个 $N/4$ -WT。如此反复二分，使问题的规模逐次减半，最终可将

校注 (agent 补充, CH10-E001): 本页化简式及其后两个 $N/2$ -WT 的第二式, 原图均写作 $H_{N/2}(N/2 + i, j)$, 上文已照录。按前页定理 10.1 和本章对 $N/2$ 阶矩阵的索引范围, 这里的行指标 $N/2 + i$ 超出 $0, \dots, N/2 - 1$; 由 $H_N(N/2 + i, j) = H_{N/2}(i, j)$, 两处应为 $H_{N/2}(i, j)$ 。这是原书下标疑误, 不是 OCR 错误。[原式放大裁片](#)。此外, 本页“定理 1”照录, 所用递推关系对应前页“定理 10.1”。

原书 PDF 页 272；印刷页码 259。

原页

N -WT 加工成 N 个 1-WT，从而得出所求的结果。这种演化过程

$$\underset{\text{(计算模型)}}{N\text{-WT}} \Rightarrow 2 \text{ 个 } N/2\text{-WT} \Rightarrow 4 \text{ 个 } N/4\text{-WT} \Rightarrow \cdots \Rightarrow \underset{\text{(计算结果)}}{N \text{ 个 } 1\text{-WT}}$$

称作**快速 Walsh 变换**。这里箭头“ $\Rightarrow$ ”表示二分手续 (10.4.2)。

进一步剖析二分手续 (10.4.2) 的内涵，计算模型 N -WT 所要加工的数据 $\{x(j)\}$ 是个 N 维向量，将它对半二分，得 $N/2$ 个**平移对** $(x(j), x(N/2+j))$ 。可见二分手续 (10.4.2) 的含义是，将平移对的两个数据相加减，因而 FWT 从属于图 10.5 的二分演化模式。

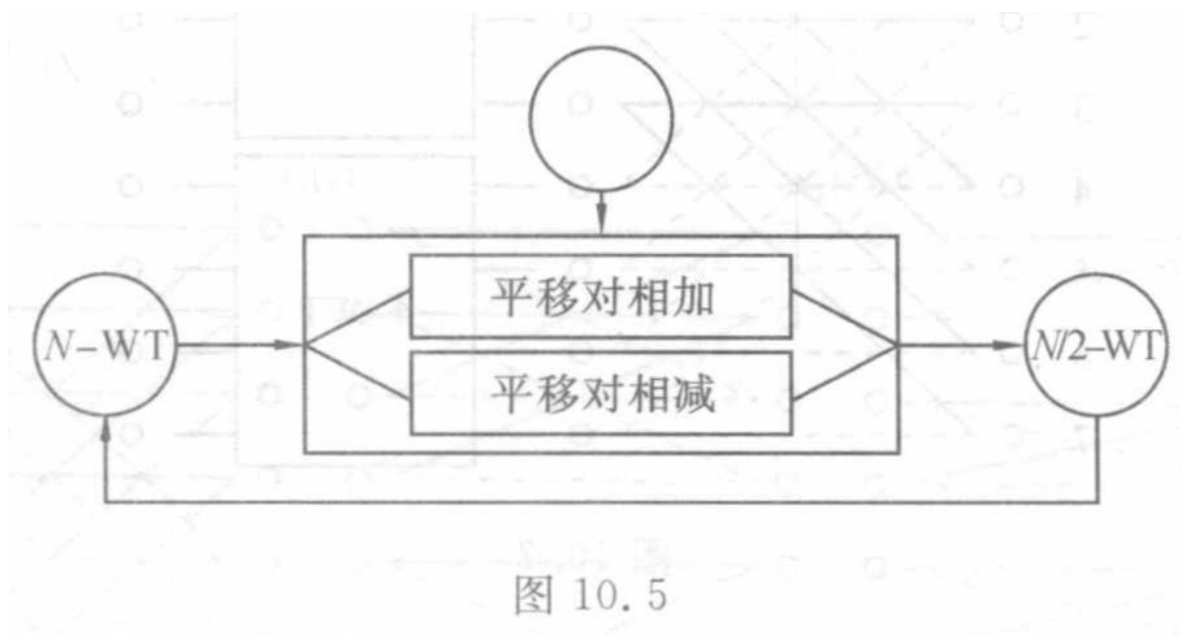


图 10.5

图 10.5

图中文字与图意 (转录)：左端 N -WT 进入“平移对相加”和“平移对相减”两路，合成右端 $N/2$ -WT，底部有返回左端的反馈线；顶部有一个向下接入的空圆，原图未印文字标签。

最后统计 FWT 的运算量。由于 FWT 的每一步使问题的规模减半，欲将所给 N -WT， $N = 2^n$ 加工成 N 个 1-WT，二分演化需做 $n = \log_2 N$ 步，又形如式 (10.4.2) 的二分手续的每一步要做 N 次加减操作，因而 FWT 的总运算量为 $N \log_2 N$ 次加减操作。另一方面，如果直接计算 N -WT (10.4.1) 要做 N^2 次加减操作，故 FWT 是快速算法，其加速比

$$\frac{N^2}{N \log_2 N} \rightarrow \infty \quad (\text{当 } N \rightarrow \infty \text{ 时}).$$

10.4.3 FWT 的计算流程

二分手续 (10.4.2) 采取两两加工的处理方式，即将一对数据 $(x(j), x(N/2+j))$ 加工成一对新的数据 $(x_1(j), x_1(N/2+j))$ ，其计算格式如图 10.6 所示。这里分别用实线与虚线区分数据的相加与相减两种运算。

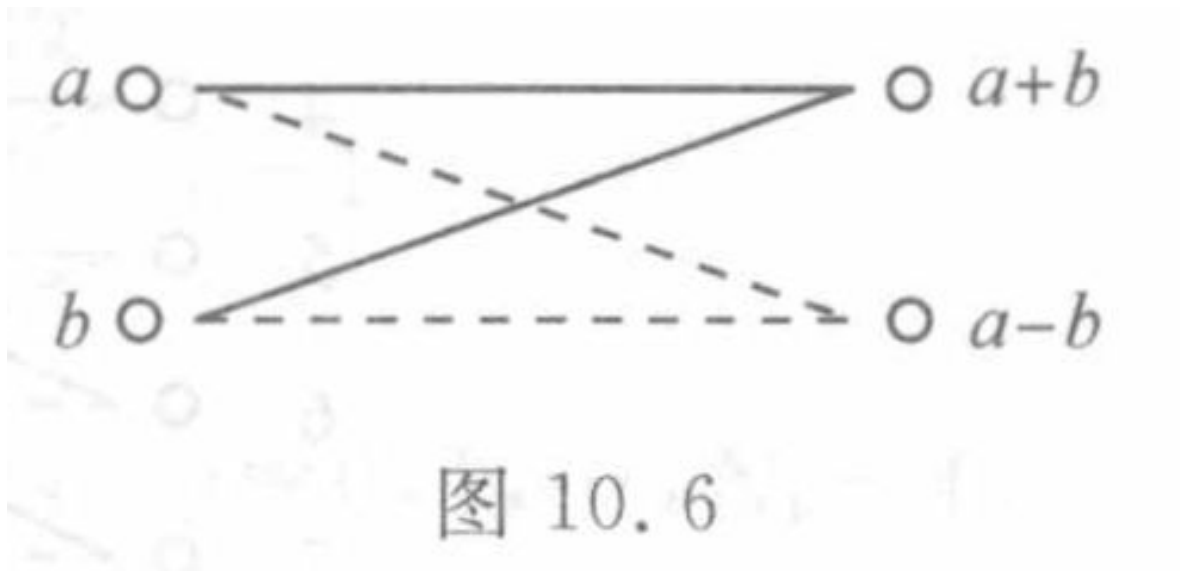


图 10.6

图中文字与图意（转录）：左端输入从上到下为 a 、 b ，右端输出从上到下为 $a+b$ 、 $a-b$ 。输入至上方输出的两条线为实线，输入至下方输出的两条线为虚线，形成交叉的两两计算格式。

现在运用二分技术针对 8-WT：

$$X(i) = \sum_{j=0}^7 x(j)H_8(i, j), \quad i = 0, 1, \dots, 7. \quad (10.4.3)$$

具体显示前述 FWT 的计算流程。

步 1 施行 $N = 8$ 的二分手续 (10.4.2)，即

$$x_1(0) = x(0) + x(4), \quad x_1(4) = x(0) - x(4),$$

校注 (agent 补充)：原页“使问题的的规模减半”重复“的”，上文照录。

原书 PDF 页 273；印刷页码 260。

[原页](#)

$$\begin{aligned}x_1(1) &= x(1) + x(5), & x_1(5) &= x(1) - x(5), \\x_1(2) &= x(2) + x(6), & x_1(6) &= x(2) - x(6), \\x_1(3) &= x(3) + x(7), & x_1(7) &= x(3) - x(7),\end{aligned}$$

将所给 8-WT 加工成 2 个 4-WT。借助于图 10.6 的计算格式，这一演化步骤如图 10.7 所示。

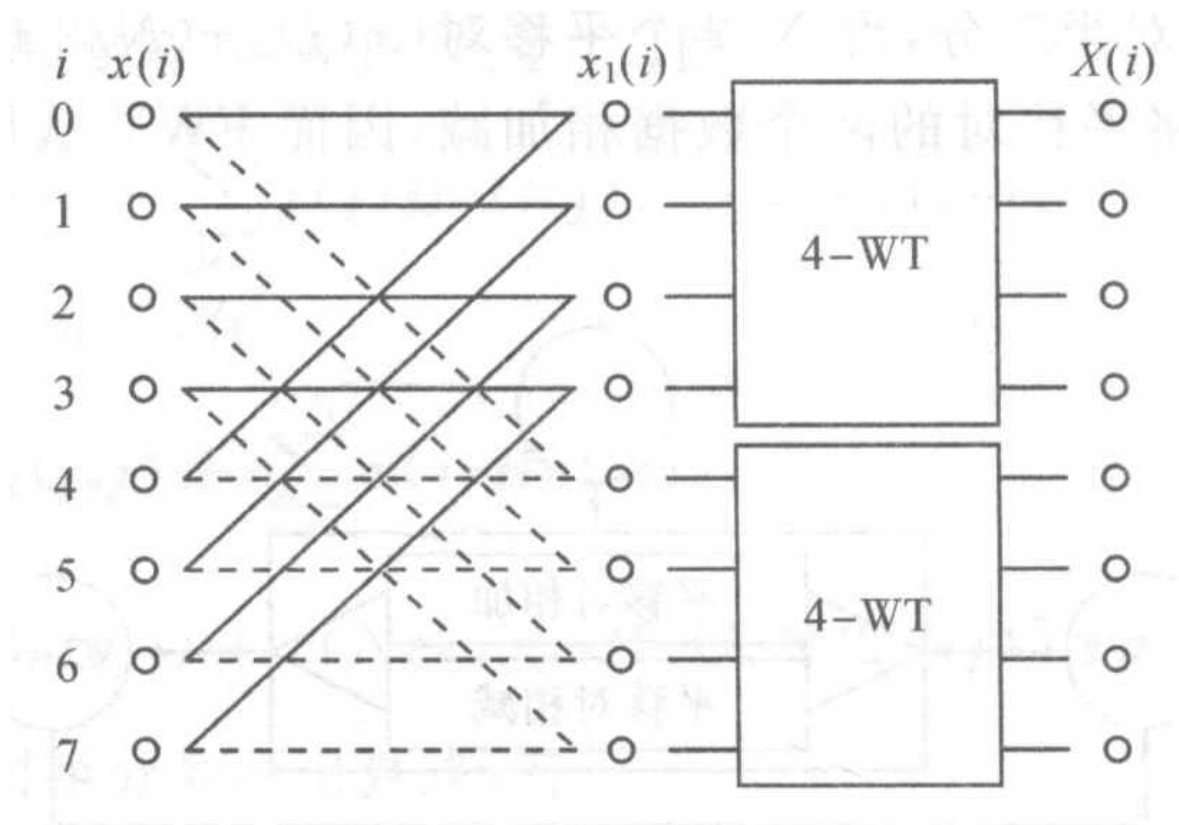


图 10.7

图 10.7

图中文字与图意（转录）：从左到右的列标为 i 、 $x(i)$ 、 $x_1(i)$ 、 $X(i)$ 。 i 的行标为 0、1、2、3、4、5、6、7；两组模块均标为 4-WT。第一个加工阶段将行对 (0,4)、(1,5)、(2,6)、(3,7) 相加减，所得 $x_1(i)$ 的前四项、后四项分别接入一个 4-WT，右端输出 $X(i)$ 。

步 2 对 2 个 4-WT 分别施行 $N = 4$ 的二分手续 (10.4.2)，即

$$\begin{aligned}x_2(0) &= x_1(0) + x_1(2), & x_2(2) &= x_1(0) - x_1(2), \\x_2(1) &= x_1(1) + x_1(3), & x_2(3) &= x_1(1) - x_1(3)\end{aligned}$$

与

$$\begin{aligned}x_2(4) &= x_1(4) + x_1(6), & x_2(6) &= x_1(4) - x_1(6), \\x_2(5) &= x_1(5) + x_1(7), & x_2(7) &= x_1(5) - x_1(7),\end{aligned}$$

进一步加工出关于数据 $\{x_2(j)\}$ 的 4 个 2-WT。这一演化步如图 10.8 所示。

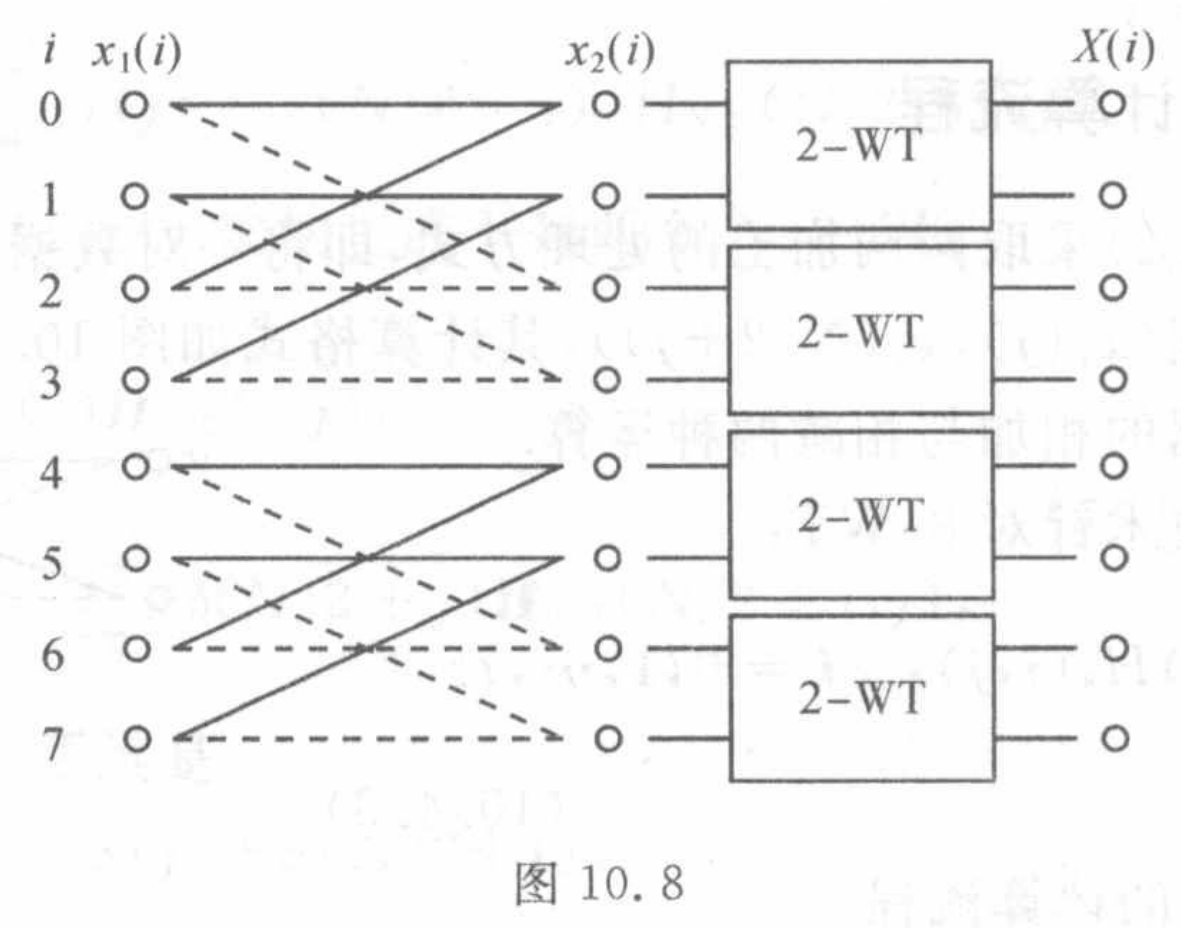


图 10.8

图中文字与图意（转录）：从左到右的列标为 i 、 $x_1(i)$ 、 $x_2(i)$ 、 $X(i)$ 。 i 的行标为 0、1、2、3、4、5、6、7；四组模块均标为 2-WT。该阶段将行对 (0,2)、(1,3)、(4,6)、(5,7) 相加减，所得 $x_2(i)$ 每两项接入一个 2-WT，右端输出 $X(i)$ 。

步 3 再对每个 2-WT 分别施行二分手续，即

原书 PDF 页 274；印刷页码 261。

原页

$$\begin{aligned}x_3(0) &= x_2(0) + x_2(1), & x_3(1) &= x_2(0) - x_2(1), \\x_3(2) &= x_2(2) + x_2(3), & x_3(3) &= x_2(2) - x_2(3), \\x_3(4) &= x_2(4) + x_2(5), & x_3(5) &= x_2(4) - x_2(5), \\x_3(6) &= x_2(6) + x_2(7), & x_3(7) &= x_2(6) - x_2(7).\end{aligned}$$

加工得出关于数据 $\{x_3(i)\}$ 的 8 个 1-WT，即得所求结果：

$$X(i) = x_3(i), \quad i = 0, 1, \dots, 7.$$

上述算法 FWT，其计算模型与输入数据同步进行加工，在将计算模型从 8-WT 加工成 1-WT 的同时，输入数据被加工成输出结果 $\{X(i)\}$ 。综合上述各步即得 FWT 的数据加工流程图 10.9。

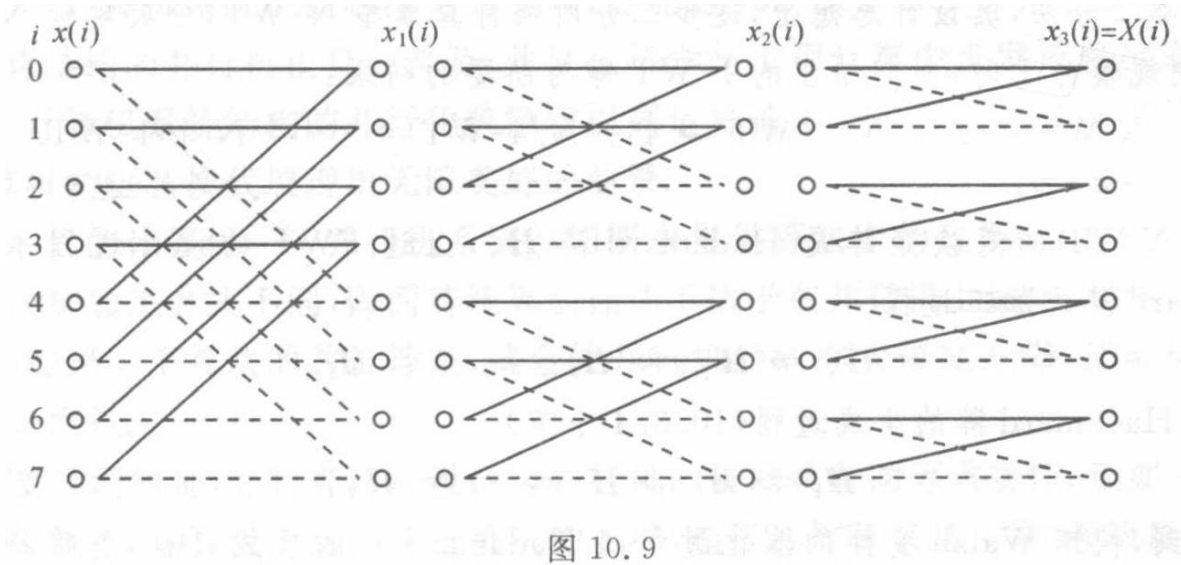


图 10.9

图中文字与图意（转录）：列标依次为 $i, x(i), x_1(i), x_2(i), x_3(i) = X(i)$ ，行标为 0、1、2、3、4、5、6、7。第一阶段配对 (0,4)、(1,5)、(2,6)、(3,7)；第二阶段配对 (0,2)、(1,3)、(4,6)、(5,7)；第三阶段配对 (0,1)、(2,3)、(4,5)、(6,7)。每对按图 10.6 的实线相加、虚线相减计算，三步后由 $x_3(i)$ 得到 $X(i)$ 。

10.4.4 FWT 的算法实现

回头考察一般形式的 Walsh 变换 N -WT (10.4.1)。仍设 $N = 2^n$ ， n 为正整数。前已指出，其快速算法设计分 $n = \log_2 N$ 步，每一步将计算问题的规模减半。记 $N_k = N/2^k$ 。快速 Walsh 变换 FWT 的第 k 步将所给 N -WT 化归为 2^k 个 N_k -WT，其输入数据 $x_k(i)$ 被分割成 2^k 段，每段含有 N_k 个数据，具体地说，其 l ($l = 1, 2, \dots, 2^k$) 段数据为

$$x_k((l-1)N_k + j), \quad j = 0, 1, \dots, N_k - 1.$$

这样，二分过程的第 k 步是先将 $k-1$ 步生成的数据段

$$x_{k-1}((l-1)N_{k-1} + j), \quad j = 0, 1, \dots, N_{k-1} - 1$$

再对半切成两部分，其前半部分与后半部分分别是

$$x_{k-1}((l-1)N_{k-1} + j); \quad x_{k-1}((l-1)N_{k-1} + N_k + j), \quad j = 0, 1, \dots, N_k - 1.$$

然后将这两组数据按式 (10.4.2) 进行加工，结果有

$$\begin{cases} x_k((l-1)N_{k-1} + j) = x_{k-1}((l-1)N_{k-1} + j) + x_{k-1}((l-1)N_{k-1} + N_k + j), \\ x_k((l-1)N_{k-1} + N_k + j) = x_{k-1}((l-1)N_{k-1} + j) - x_{k-1}((l-1)N_{k-1} + N_k + j), \end{cases} \quad (10.4.4)$$

$$j = 0, 1, \dots, N_k - 1; \quad l = 1, 2, \dots, 2^k.$$

校注 (agent 补充, CH10-E004): 式 (10.4.4) 原印 $l = 1, 2, \dots, 2^k$, 上文照录。此式用 $(l-1)N_{k-1}$ 定位第 $k-1$ 步的数据段, 该步只有 2^{k-1} 段, 因此上限应为 2^{k-1} 。例如 $N = 8, k = 1$ 时, 按原印取 $l = 2$ 会访问从索引 8 开始的数据, 越过 $0, \dots, 7$ 。本页前文用于描述第 k 步结果分段数的 2^k 则正确。下一页算法 10.1 引用此式, 也受该范围疑误影响。[原式放大裁片](#)。

原书 PDF 页 275；印刷页码 262。

[原页](#)

这就是第 k 步所要施行的二分手续。反复施行这种二分手续即得所求的结果。于是有下列快速 Walsh 变换 FWT（算法 10.1）。

算法 10.1 令 $x_0(i) = x(i)$, $i = 0, 1, \dots, N-1$, 对 $k = 1, 2, \dots$ 直到 $n = \log_2 N$ 执行算式 (10.4.4), 结果有

$$X(i) = x_n(i), \quad i = 0, 1, \dots, N-1.$$

小结

本章阐述快速 Walsh 变换 FWT 的设计机理与设计方法。可以看到, FWT 本质上是一类二分法, 其设计思想是, 逐步二分所给计算模型 N -WT, 令其规模 N 逐次减半, 直到规模为 1 时, 所归结出的 1-WT 即为所要的结果:

$$\begin{array}{c} N\text{-WT} \\ \text{(计算模型)} \end{array} \Rightarrow 2 \text{ 个 } N/2\text{-WT} \Rightarrow 4 \text{ 个 } N/4\text{-WT} \Rightarrow \dots \Rightarrow \begin{array}{c} N \text{ 个 } 1\text{-WT} \\ \text{(所求结果)} \end{array}.$$

注意到 N -WT 的变换矩阵是 Hadamard 阵 H_N , 上述 FWT 的设计过程本质上是 Hadamard 阵的加工过程

$$H_N \Rightarrow H_{N/2} \Rightarrow H_{N/4} \Rightarrow \dots \Rightarrow H_1.$$

再对比 Hadamard 阵的生成过程 (10.3.4 小节)

$$H_1 \Rightarrow H_2 \Rightarrow H_4 \Rightarrow \dots \Rightarrow H_N,$$

可以看到, 快速 Walsh 变换的演化过程同 Hadamard 阵的生成过程, 它们两者互为反过程。如果后者视为**进化过程** (矩阵阶数逐步倍增), 那么前者则是**退化过程** (矩阵阶数逐次减半)。在适当定义“规模”的前提下, 它们两者全都从属于二分演化模式。

Walsh 分析处处都渗透了对立统一的辩证思维。

正因为 Walsh 函数具有极度的数学美, 正由于 Walsh 分析展现了一种新的思维方式, 因而在 Walsh 分析的基础上可以开展许多重要的研究。

快速 Walsh 变换是快速变换的一个重要的组成部分。运用变异技术, 基于 Walsh 变换可以派生出其他种种快速变换, 诸如 Haar 变换、斜变换、Hartly 变换等等, 从而实现快速变换方法的大统一。

Walsh 分析有着广泛的应用前景。然而更为重要的是, 它展现了一种新的数学方法——**演化数学方法**。

宇宙是演化的, 生物是演化的。时至今日, 辩证法关于发展变化的观点, 即事物从低级到高级不断演化的观点, 已经被科学界认为是无须论证的常识了。

Walsh 函数的演化分析用数学语言表述了这种“常识”。

Walsh 函数的演化分析无疑是新的数学革命即将爆发的先兆。还是 Harmuth 有远见: **Walsh 分析的研究将导致一场数学革命, 就像 17、18 世纪的微积分那样。**

原书 PDF 页 276；印刷页码 263。

[原页图像](#)

* 第 11 章 并行算法设计：递推计算并行化

从世界上第一台电子计算机的问世至今仅有六七十年，在这短短的时间里，计算机的基本元件从电子管、晶体管发展到了大规模集成电路，计算机的运算速度以指数形式迅速增长。然而人们对高性能计算的需求是永无止境的，在诸如能源、气象、军事、人工智能和生命科学等许多领域，都迫切要求提供性能更高、速度更快的新型计算机系统。

今天，计算机系统正面临深刻的变革，传统的 von Neumann 格局已经被突破，采用并行化结构的并行机正日益普及，并且在科学与工程计算中正发挥着越来越重要的作用。计算机系统结构的并行化蕴涵着提高运算速度和增加信息存储量的巨大潜力，计算机的更新换代展现出无限美好的前景。

新一代的计算机——并行机系统迫切要求提供算法上的支持。并行机与传统计算机的数据加工方式不同，因而传统算法往往不适于在并行机上运用。科学计算的实践表明，如果一个算法的并行性差，就会使并行机的效率大幅度下降，甚至从亿次机降为百万次机。

需要强调的是，并行算法的设计与并行机的研制具有同等重要性。正如一位著名学者所尖锐指出的：没有好的并行算法的支撑，超级计算机只是一堆“超级废铁”。

计算机发展的并行化趋势，必然会促使算法设计的并行化。随着并行机系统的日益普及，学习和研究适应并行机系统的并行算法，已是科学计算工作者的当务之急。

11.1 什么是并行计算

11.1.1 一则寓言故事

20 世纪 80 年代初国产银河巨型机问世，国内掀起一股并行算法热。究竟什么是并行计算呢？这里先讲一个生动的寓言故事。

相传很久很久以前，有一个年轻的国王名叫川行，是个数学天才。川行爱上了邻国聪颖美丽且爱好数学的公主邱比郑南。

川行差人前往邻国求婚。公主答应了这桩婚事，但提出了一项先决条件，她要亲自考核一下川行的数学才能。公主的考题是，针对一个 15 位数求出它的真因子。

接到试题之后，川行立即忙碌起来，一个数接一个数地试算。川行有数学才能，算得很快，然而由于 15 位数的真因子可能是个 8 位数，找出全部真因子要花费上亿次

原书 PDF 页 277；印刷页码 264。

原页图像

整数除法，总的计算量大得惊人。

川行感到很为难。这是一道“大数分解”的数学难题，如何才能尽快地找出它的答案呢？

川行有个足智多谋的宰相名叫孔幻士。孔幻士提出了一个计谋：将全国老百姓按军、师、团、营、连、排、班、兵 8 个等级编号，每 10 个兵组成 1 班，10 个班为 1 排，10 个排为 1 连……10 个师为 1 军，10 个军全归川行统帅。这样，在编的每个老百姓都有一个 8 位十进制的编号。完成这种编制以后，通知全国老百姓用自己的编号去除公主给出的 15 位数，能除尽的立即上报，给予重奖。这样很快找出了所有的真因子，而川行则依靠全国老百姓的帮助赢得了公主的爱情。

这则寓言浅显易懂，但意味深长。公主“邱比郑南”是“求比证难”的谐音。大数分解问题的可解性不言而喻，但具体求解却很困难。国王“川行”是“串行”的谐音。串行计算的效率很低，往往不能承担大规模的计算工程。宰相“孔幻士”则是“空换时”的谐音，其含义是，并行计算的设计思想是用扩大空间、增加处理机台数为代价来换取计算时间的节省。“空换时”是并行计算的基本策略。

这则故事所涉及的大数分解问题有重大的学术价值，求解这类问题的计算量随着“大数”的增大而急剧增长。譬如，计算一个 155 位数的真因子，如果用串行算法进行计算，即使用每秒亿次的巨型机去承担，也得要连续工作上万年。当然这是没有实际意义的。1990 年 6 月 20 日美国报道了一则消息：贝尔实验室用 1 000 台处理机并行计算，仅仅花费了几个月的时间，就成功地找出一个 155 位数的 3 个真因子，它们分别是 7 位数、49 位数和 99 位数。这是科学计算的一项重大突破。当年我国《科技日报》评价这项成就为“1990 年世界十大科技成就之一”。

11.1.2 同步并行算法的设计策略

采取并行处理方式运行的计算机系统称作并行机系统，简称并行机。

并行机出现于 20 世纪的 70 年代初，至今仅有近 50 年的历史。1972 年，美国研制成功阵列机 Illiac IV，此后于 1976 年又进一步研制出向量机 Cray-1。并行机的更新换代和商业化开发强有力地推动了并行计算的蓬勃发展。

并行机的体系结构各不相同，但大致可分为两类：一类是单指令流多数据流 SIMD (single instruction stream, multiple data stream) 型，如阵列机、向量机；另一类是多指令流多数据流 MIMD (multiple instruction stream, multiple data stream) 型，称作多处理机。

针对 SIMD 与 MIMD 两类并行机系统，并行算法大致分为同步与异步两类。本章仅研究同步并行算法。

所谓同步性，是指不同处理机在同一时刻针对不同数据执行同一种操作。同步并行计算的典型例子是向量计算。

原书 PDF 页 278；印刷页码 265。

[原页图像](#)

同步并行计算的基本策略是“分而治之”。所谓分而治之，就是将所考察的计算问题分裂成若干较小的子问题，并将这些子问题映射到多台处理机上去各自完成，然后再将分散的结果拼装成所求的解。

值得指出的是，在同步并行算法设计时，“分而治之”的设计原则往往被误解为整体上的先分后治，而将“分”与“治”两个环节截然分开。这种算法设计技术就是所谓的倍增技术。

其实，在并行计算过程中，“分”与“治”是矛盾的两个方面，它们既是对立的，又是统一的。基于这种理解我们推荐了同步并行算法设计的二分技术。

需要强调的是，为避免局限于具体的机器特征而束缚了并行算法的研究，人们提出了理想计算机的概念。“理想化”的假设包括：任何时刻可以使用任意多台处理机，任何时刻有任意多个主存单元可供使用，处理机同主存间的数据通信时间可以忽略不计。

11.2 叠加计算

叠加计算是一类最简单、最基本的计算模型。本章所研究的叠加计算包括数列求和

$$S = \sum_{i=0}^{N-1} a_i \quad (11.2.1)$$

与多项式求值

$$P = \sum_{i=0}^{N-1} a_i x^i. \quad (11.2.2)$$

上述两种叠加计算模型之间有着紧密的联系。事实上，和式 (11.2.1) 是式 (11.2.2) 取 $x = 1$ 的具体情形。

在着手具体设计算法之前，首先引进问题的规模的概念。所谓规模是用来刻画问题“大小”的某个正整数。譬如，上述叠加计算问题的规模均可定为它们的项数 N 。

不言而喻，并行计算所要求解的问题，其重要特点是规模很大，即为大规模或超大规模的科学计算。为简化叙述，今后将假定计算问题的规模 N 为 2 的幂，即

$$N = 2^n,$$

式中， $n = \log_2 N$ 是正整数。这种限制通常是非实质性的，譬如，对于上述两种叠加计算，只要适当地补充几个零系数 a_i ，总可以将规模 N 扩充为 $N = 2^n$ 的形式。后文将会看到，这种扩充对于算法运行时间的影响几乎可以忽略不计。本章所考察的其他计算问题也可作类似的处理。

11.2.1 倍增技术

有些学者认为，倍增技术是设计同步并行算法的一项基本技术。这项设计技术反

原书 PDF 页 279；印刷页码 266。

原页图像

复地将计算问题分裂成具有同等规模的两个子问题。在问题逐步分裂的过程中，子问题的个数是逐步倍增的，倍增法因此而得名。

倍增法的设计基于这样的考虑，如果将各个子问题适当地映射到多台处理机上，即可实现计算过程的并行化。

现在就用简单的数列求和问题（11.2.1）考察倍增技术的设计原理和设计方法。为此，引进和式

$$S(i,j)=\sum_{k=j}^i a_k.$$

显然，问题（11.2.1）的已给数据与所求结果均可用这种和式来表达：

$$\begin{aligned} a_i &= S(i,i), \quad i=0,1,\cdots,N-1; \\ S &= S(N-1,0). \end{aligned}$$

倍增法的设计过程含分裂与合成两个环节。分裂过程将所给和式 $S(N-1,0)$ 逐步“一分为二”，从而拆成若干个子 and 式。这种分裂过程的特点是，子和式的个数是逐步倍增的。

$$\begin{aligned} S(N-1,0) &= S\left(N-1,\frac{N}{2}\right)+S\left(\frac{N}{2}-1,0\right) \\ &= S\left(N-1,\frac{3}{4}N\right)+S\left(\frac{3}{4}N-1,\frac{N}{2}\right)+S\left(\frac{N}{2}-1,\frac{N}{4}\right)+S\left(\frac{N}{4}-1,0\right) \\ &= \cdots. \end{aligned}$$

由于在和式二分的上述过程中，每个子和式的项数逐次减半，因而最终可拆成每段仅含两项的最简形式

$$S(N-1,0)=S(N-1,N-2)+S(N-3,N-4)+\cdots+S(1,0).$$

图 11.1 取 $N=8$ 自顶向下地描述了倍增法的分裂过程。

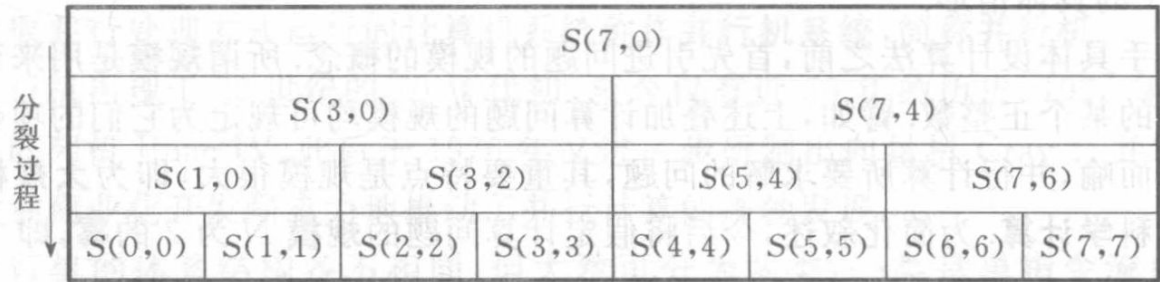


图 11.1

图 11.1 原图的“分裂过程”箭头自上向下。各层由左至右的符号标签为：

层次（由上至下）	原图中的子和式
1	$S(7, 0)$
2	$S(3, 0), S(7, 4)$
3	$S(1, 0), S(3, 2), S(5, 4), S(7, 6)$
4	$S(0, 0), S(1, 1), S(2, 2), S(3, 3), S(4, 4), S(5, 5), S(6, 6), S(7, 7)$

每个上层区间对应下层相邻的两个子区间。（本段及上表为原图标签与结构的文字转录。）

将所给和式拆成若干个子 and 式后，可将这些子和式分配给各台处理机去并行计算。问题在于，基于这些子和式的计算，如何得出所求的结果呢？

倍增法的合成过程是将所拆出的各个子和式的值再逐步“合二为一”，最后归并出所给和式的值。这种归并过程的特点是数据量逐次减半。图 11.2 自底向上地描述了倍增法的合成过程。

原书 PDF 页 280；印刷页码 267。

[原页图像](#)

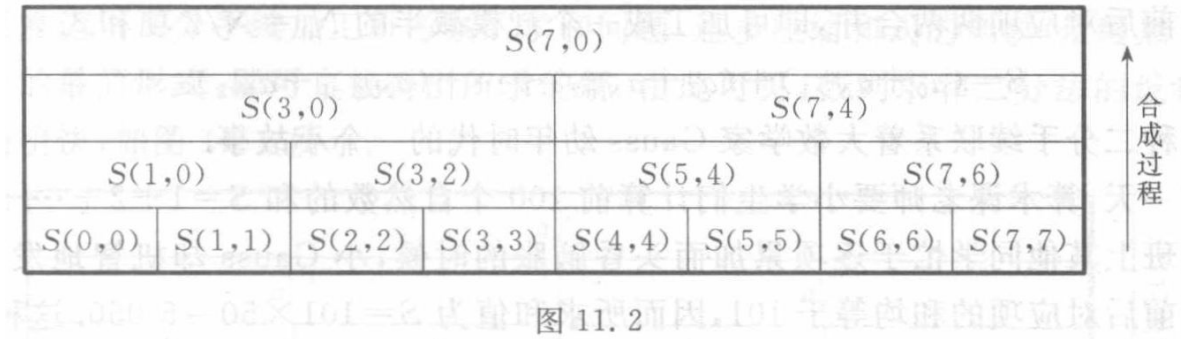


图 11.2

图 11.2 原图“合成过程”箭头自下向上。各层从左至右的符号标签为：

层次（由上至下）	原图中的子和式
1	$S(7,0)$
2	$S(3,0), S(7,4)$
3	$S(1,0), S(3,2), S(5,4), S(7,6)$
4	$S(0,0), S(1,1), S(2,2), S(3,3), S(4,4), S(5,5), S(6,6), S(7,7)$

每层中相邻的两个子区间向上合成为一个区间。（本段及上表为原图标签与结构的文字转录。）

现在列出倍增法的算法步骤（图 11.2）。

倍增法的第 1 步是利用所给的 N 个数据（它们均可视为一项和式） $a_i = S(i, i)$ 求出 2 项和式的值，而得出 $N_1 = N/2$ 个中间结果：

$$S(2i+1, 2i) = S(2i+1, 2i+1) + S(2i, 2i), \quad i = 0, 1, \dots, N_1 - 1.$$

第 2 步再用两项和式求出 4 项和式的值，而有 $N_2 = N/4$ 个中间结果：

$$S(4i+3, 4i) = S(4i+3, 4i+2) + S(4i+1, 4i), \quad i = 0, 1, \dots, N_2 - 1.$$

保持和式项数逐步倍增，而数据量则为逐次减半这个特征，其第 k 步所承担的工作是，用 2^{k-1} 项和式求出 2^k 项和式的值，从而得出 $N_k = N/2^k$ 个中间结果：

$$S(2^k i + 2^k - 1, 2^k i) = S(2^k i + 2^k - 1, 2^k i + 2^{k-1}) + S(2^k i + 2^{k-1} - 1, 2^k i), \quad i = 0, 1, \dots, N_k - 1. \quad (11.2.3)$$

如此做 $n = \log_2 N$ 步即可得出所求的和值：

$$S(N-1, 0) = S(N-1, N_1) + S(N_1-1, 0).$$

综上所述，数列求和 (11.2.1) 的倍增法可表述为算法 11.1。

算法 11.1 对 $k = 1, 2, \dots$ 直到 $n = \log_2 N$ 执行算式 (11.2.3)，则所求的和值为 $S = S(N - 1, 0)$ 。

倍增法的分裂过程是反复将和式一分为二，在这一过程中，子和式的个数是逐步倍增的；与此相反，其合成过程是反复将数据合二为一，在合成过程中，数据量则为逐次减半。正是由于倍增法的分裂过程与合成过程相对峙，这种技术不便于实际运用。

11.2.2 二分手续

为使并行算法设计的原理与方法简单而和谐，这里推荐一种设计技术——二分技术。

二分技术的设计原理是，反复地将所给计算问题加工成规模减半的同类问题，直到规模足够小（通常当规模为 1）时直接得出问题的解。

需要强调的是，与倍增技术不同，二分技术不是着眼于问题的分裂，而是立足于问题的加工。今后所说的二分手续，是指将问题规模减半的加工手续。

譬如，对于 N 项和式

$$S = \sum_{i=0}^{N-1} a_i,$$

原书 PDF 页 281；印刷页码 268。

[原页图像](#)

若将其前后对应项两两合并，即可加工成一个规模减半的 $N_1 = N/2$ 项和式

$$S = (a_0 + a_{N-1}) + (a_1 + a_{N-2}) + \cdots + (a_{N/2-1} + a_{N/2}).$$

这种二分手续联系着大数学家 Gauss 幼年时代的一个小故事。

有一天，算术课老师要小学生们计算前 100 个自然数的和 $S = 1 + 2 + \cdots + 99 + 100$ 。当班上其他同学忙于逐项累加而头昏脑胀的时候，小 Gauss 却机智地发现，所给和式前后对应项的和均等于 101，因而所求和值为 $S = 101 \times 50 = 5050$ 。这种简捷的快速算法可以视作二分手续的巧妙应用。

11.2.3 数列求和的二分法

再考察所给和式 (10.2.1)，容易看出，如果将其奇偶项两两合并，即可使其规模变成 $N_1 = N/2$ ，即

$$S = \sum_{i=0}^{N_1-1} (a_{2i} + a_{2i+1}) = \sum_{i=0}^{N_1-1} a_i^{(1)}.$$

为此，所要施行的运算手续是

$$a_i^{(1)} = a_{2i} + a_{2i+1}, \quad i = 0, 1, \dots, N_1 - 1.$$

注意到这样加工出的求和问题

$$S = \sum_{i=0}^{N_1-1} a_i^{(1)}$$

与所给问题 (10.2.1) 属于同一类型，所不同的只是规模缩减了一半，因此上述加工手续是一种二分手续。

反复施行这种二分手续，二分 k 次后和式的项数压缩成 $N_k = N/2^k$ ，即

$$S = \sum_{i=0}^{N_k-1} a_i^{(k)},$$

式中

$$a_i^{(k)} = a_{2i}^{(k-1)} + a_{2i+1}^{(k-1)}, \quad i = 0, 1, \dots, N_k - 1. \quad (11.2.4)$$

这样二分 $n = \log_2 N$ 次后，所给和式最终退化为一项，从而直接得出所求的和值 S 。于是有数列求和的二分算法 11.2。

算法 11.2 对 $k = 1, 2, \dots$ 直到 $n = \log_2 N$ 执行算式 (11.2.4)，结果有

$$S = a_0^{(n)}.$$

这一算法显然可以向量化。事实上，算式 (11.2.4) 可以表为向量形式：

$$\begin{pmatrix} a_0^{(k)} \\ a_1^{(k)} \\ \vdots \\ a_{N_k-1}^{(k)} \end{pmatrix} = \begin{pmatrix} a_0^{(k-1)} \\ a_2^{(k-1)} \\ \vdots \\ a_{N_{k-1}-2}^{(k-1)} \end{pmatrix} + \begin{pmatrix} a_1^{(k-1)} \\ a_3^{(k-1)} \\ \vdots \\ a_{N_{k-1}-1}^{(k-1)} \end{pmatrix}.$$

校注 (agent 补充, CH11-E001): 本页两处回引均印作 “(10.2.1)”, 已照录; 从本节的数列求和定义及上下文判断, 应回引本章的式 (11.2.1)。这是原书交叉引用疑误, 不是 OCR 改写。

原书 PDF 页 282；印刷页码 269。

原页图像

反复运用二分手续加工所考察的求和问题，逐步压缩和式的规模，最终加工成规模为 1 的最简形式，即可直接得出所求的解。由此可见，数列求和二分法的设计过程简洁而明快，如图 11.3 所示。

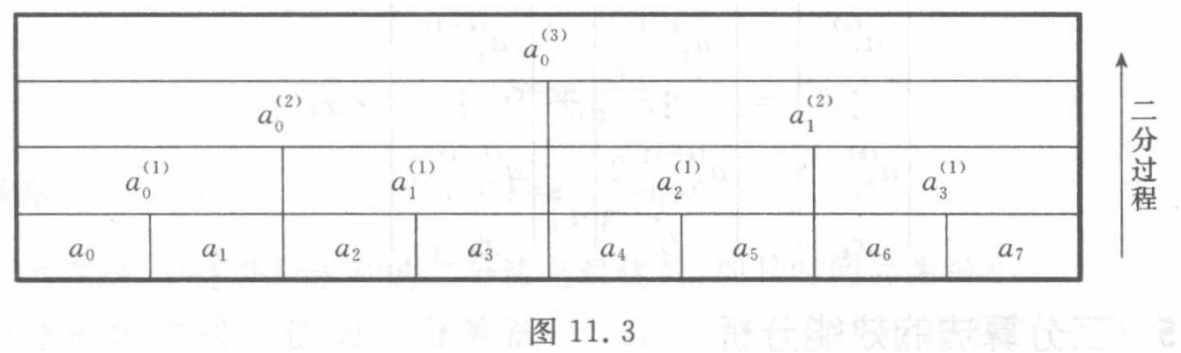


图 11.3 原图“二分过程”箭头自下向上。各层从左至右的符号标签为：

层次（由下至上）	原图中的数据
初始层	$a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7$
第 1 次二分	$a_0^{(1)}, a_1^{(1)}, a_2^{(1)}, a_3^{(1)}$
第 2 次二分	$a_0^{(2)}, a_1^{(2)}$
第 3 次二分	$a_0^{(3)}$

每次将相邻两格合为上层的一格。（本段及上表为原图标签与结构的文字转录。）

应当指出的是，上面提供的两个算法——算法 11.1 和算法 11.2，尽管思路不同，繁简互异，但却是殊途同归。事实上，式（11.2.4）与式（11.2.3）得出的是同样的结果：

$$a_i^{(k)} = S(2^k i + 2^k - 1, 2^k i).$$

11.2.4 多项式求值的二分法

进一步讨论多项式求值问题。仿照数列求和的做法，将所给多项式（11.2.2）的奇偶项两两合并，得

$$P = \sum_{i=0}^{N_1-1} (a_{2i} + a_{2i+1}x)x^{2i}.$$

若令

$$\begin{cases} a_i^{(1)} = a_{2i} + a_{2i+1}x, & i = 0, 1, \cdots, N_1 - 1, \\ x_1 = x^2, \end{cases}$$

则有

$$P = \sum_{i=0}^{N_1-1} a_i^{(1)} x_1^i.$$

这样加工得出的是一个以 x_1 为变元的多项式，它与所给多项式 (11.2.2) 的类型相同，只是规模压缩了一半，因此上述手续是一项二分手续。

重复这种手续，二分 k 次后所给多项式被加工成

$$P = \sum_{i=0}^{N_k-1} a_i^{(k)} x_k^i,$$

这里

$$\begin{cases} a_i^{(k)} = a_{2i}^{(k-1)} + a_{2i+1}^{(k-1)} x_{k-1}, & i = 0, 1, \dots, N_k - 1, \\ x_k = x_{k-1}^2. \end{cases} \quad (11.2.5)$$

这样二分 $n = \log_2 N$ 次，最终得出的系数 $a_0^{(n)}$ 即为所求多项式的值 P 。于是，多项式求值问题 (11.2.2) 有多项式求值的二分算法 11.3。

算法 11.3 对 $k = 1, 2, \dots$ 直到 $n = \log_2 N$ 执行算式 (11.2.5)，结果有

原书 PDF 页 283；印刷页码 270。

[原页图像](#)

$$P = a_0^{(n)}.$$

上述算法同样可以向量化，事实上，算式 (11.2.5) 可表为向量化形式：

$$\begin{pmatrix} a_0^{(k)} \\ a_1^{(k)} \\ \vdots \\ a_{N_k-1}^{(k)} \\ x_k \end{pmatrix} = \begin{pmatrix} a_0^{(k-1)} \\ a_2^{(k-1)} \\ \vdots \\ a_{N_{k-1}-2}^{(k-1)} \\ 0 \end{pmatrix} + \begin{pmatrix} a_1^{(k-1)} \\ a_3^{(k-1)} \\ \vdots \\ a_{N_{k-1}-1}^{(k-1)} \\ x_{k-1} \end{pmatrix} \cdot x_{k-1}.$$

11.2.5 二分算法的效能分析

评价一种并行算法，人们首先关心的是它的算法复杂性，即算法的运行时间（时间复杂性）与所要提供的处理机台数（空间复杂性）。并行算法设计的基本思想是用增加处理机台数的办法来换取算法运行时间的节省。处理机台数充分多时的最少运行时间称作算法的时间界，而算法的运行时间达到时间界时所提供的（最少的）处理机台数则称作处理机台数界。

为简化分析，今后将假定每台处理机的算术运算（无论是加减还是乘除）的操作时间相同，均取单位时间。这样，在估算算法的运行时间时，只要针对各并行步骤统计运算次数即可。

对于所考察的某个并行算法，记 T^* 为算法的时间界， P^* 为处理机台数界，另记 T_1 为串行算法的运行时间，将

$$S = \frac{T_1}{T^*}$$

称作该并行算法的加速比，而将

$$E = \frac{T_1}{P^* T^*}$$

称作其效率。

加速比与效率是评估一种并行算法的“得”与“失”的两项重要指标。加速比 S 表示该并行算法在运行时间方面的节省；注意到 $P^* T^*$ 表示并行算法的总计算量，而 T_1 则表示串行算法的计算量，因而效率 E 刻画了该并行算法在计算量方面的损耗。

现在分析前述几种二分算法的时间界与处理机台数界。

首先考察数列求和的二分算法——算法 11.2。令式 (11.2.4) 的各个系数 $a_i^{(k)}$ 并行计算，则其每一步含一次运算（加法），故其时间界为

$$T^* = n = \log_2 N.$$

然而，为使每个 $a_i^{(k)}$ 能并行计算，第 k 步按式 (11.2.4) 需提供 $N_k = N/2^k$ 台处理机，因此算法 11.2 的处理机台数界为

原书 PDF 页 284；印刷页码 271。

[原页图像](#)

$$P^* = \max_{1 \leq k \leq n} \frac{N}{2^k} = \frac{N}{2}.$$

注意到数列求和问题 (11.2.1) 的串行算法的运行时间 $T_1 = N - 1$ ，算法 11.2 的加速比

$$S \approx \frac{N}{\log_2 N},$$

而其效率

$$E \approx \frac{2}{\log_2 N}.$$

不难看出，上述并行求和的二分法是最优的，即其时间界为最小。

再分析多项式求值的二分算法——算法 11.3。首先注意一个事实：算式 (11.2.5) 中的 x_k 可与 $a_i^{(k)}$ 并行计算，为此只要将式 $x_k = x_{k-1}^2$ 改写成

$$x_k = 0 + x_{k-1} \cdot x_{k-1}$$

的形式。这样，算法 11.3 的每一步需做 2 次运算（一次乘法与一次加法），因而其时间界为

$$T^* = 2 \log_2 N.$$

此外，为使 $a_i^{(k)}$ 与 x_k 按式 (11.2.5) 并行计算，第 k 步需处理机 $N/2^k$ 台，因此算法 11.3 的处理机台数界为

$$P^* = \max_{1 \leq k \leq n} \frac{N}{2^k} = \frac{N}{2}.$$

注意到多项式求值的串行算法（秦九韶-Horner 算法）需做 $T_1 = 2(N - 1)$ 次运算，易知算法 11.3 的加速比及效率均与算法 11.2 相同，仍为

$$S \approx \frac{N}{\log_2 N}, \quad E \approx \frac{2}{\log_2 N}.$$

11.2.6 二分算法的基本特征

本小节从最简单的计算模型——叠加计算入手，考察并行计算的二分算法的基本特征。

在设计原理上，并行的二分算法与串行的递推算法，两者的设计过程都是计算模型不断演化的过程，其区别在于，串行递推算法的规模逐次减 1，而并行二分算法的规模则为逐次减半。例如，累加求和算法的加工过程为

N 项和式 $\rightarrow N - 1$ 项和式 $\rightarrow N - 2$ 项和式 $\rightarrow \cdots \rightarrow 1$ 项和式，

而二分求和算法的加工过程是

N 项和式 $\Rightarrow N/2$ 项和式 $\Rightarrow N/4$ 项和式 $\Rightarrow \dots \Rightarrow 1$ 项和式。

可见，串行算法与并行算法的设计思想是一脉相承的，后者可以视作是前者的延伸与发展。

从算法的结构来看，并行的二分算法与串行递推算法均具有递归结构，即将复杂

校注 (agent 补充, CH11-E002): 多项式算法的处理机计数按原书照录。上一页的向量形式同时更新 N_k 个系数及 x_k , 共有 $N_k + 1$ 个分量; 在原书所述每个分量完全同步执行一次乘法和一次加法的安排下, 处理机数应计入计算 x_k 的额外一台。原书此处写 $N/2^k$, 可能省略了常数项或依赖未说明的调度, 属处理机台数界的疑误; 未据此改动原式。

原书 PDF 页 285；印刷页码 272。

[原页图像](#)

计算归结为简单计算的重复。譬如数列求和计算，是将多项求和归结为简单的二项求和的重复，而多项式求值是将高次式求值归结为简单的一次式求值的重复，等等。对串行算法这里所谓“重复”意味着循环，而并行算法所指的“重复”则综合采取串行与并行两种处理方式。

从算法效能的角度来看，串行算法与并行算法各有所长，前者拥有高效率，而后者则具有高速度。不过，并行算法的高速度是以处理机台数的增加为代价的。

11.3 一阶线性递推

设计串行算法的一项基本技术是递推化。递推计算采取逐步推进的方式，其每一步计算都要用到前面几步的信息。正是由于这种时序性，递推计算的并行化似乎存在实质性的困难。

设计递推计算问题的并行算法，有些学者采用了倍增技术，这种设计技术从展开式入手展示了递推问题内在的并行性。

与倍增技术不同，本节所推荐的二分技术将直接开发递推算式本身的并行性，因而设计思想更简明，使用方法更简便。这种算法设计技术可广泛应用于众多类型的递推问题。

本小节着重研究一阶线性递推问题，即寻求数列 x_i , $i = 0, 1, \dots, N-1$ ，使之满足

$$\begin{cases} x_0 = b_0, \\ x_i = a_i x_{i-1} + b_i, \quad i = 1, 2, \dots, N-1, \end{cases} \quad (11.3.1)$$

式中，系数 a_i, b_i 为已给。

值得指出的是，只要引进矩阵和向量的记号，总可以将高阶线性递推归结为上述一阶线性递推的情形。

11.3.1 相关链的二分手续

为了便于刻画二分法的设计思想，首先引进相关链的概念。由于递推关系式反映了变元之间的相关性和时序性，一组有序的相关变元可抽象地表述为如下形式的相关链：

$$\cdots \rightarrow x_{i-j} \rightarrow x_i \rightarrow \cdots,$$

其中相邻两元素 x_i 与 x_{i-j} 的下标之差 j 称作间距。如果相关链各节的间距为定值，则将其称作步长。

对应于递推问题 (11.3.1) 的相关链有 N 节，即

$$x_0 \rightarrow x_1 \rightarrow \cdots \rightarrow x_{N-1},$$

原书 PDF 页 286；印刷页码 273。

[原页图像](#)

这里步长等于 1。

设将上述相关链按其下标的奇偶拆成两条，则每条子链含 $N_1 = N/2$ 节，即

$$\begin{cases} x_0 \rightarrow x_2 \rightarrow \cdots \rightarrow x_{N-2}, \\ x_1 \rightarrow x_3 \rightarrow \cdots \rightarrow x_{N-1}. \end{cases}$$

(11.3.2)

这样加工得出的递推问题有两个结果 x_0, x_1 ，且其步长等于 2。这是一种二分手续。

在保持结果数逐步倍增及步长逐步倍增两项基本特征的前提下反复施行这一手续，则二分 k 次后得出 2^k 个结果 $x_i, i = 0, 1, \cdots, 2^k - 1$ ，且步长增至 2^k ，相应地，所给相关链被加工成 2^k 条子链，每条子链含 $N_k = N/2^k$ 节，即

$$\begin{cases} x_0 \rightarrow x_{2^k} \rightarrow \cdots \rightarrow x_{N-2^k}, \\ x_1 \rightarrow x_{2^k+1} \rightarrow \cdots \rightarrow x_{N-2^k+1}, \\ \vdots \\ x_{2^k-1} \rightarrow x_{2^{k+1}-1} \rightarrow \cdots \rightarrow x_{N-1}. \end{cases}$$

(11.3.3)

如此二分 $n = \log_2 N$ 次后，所给相关链最终退化为每条仅含一节的最简形式

$$x_0, x_1, \cdots, x_{N-1},$$

从而得出所求的解。

这种以下标的奇偶分离为特征的二分手续称作奇偶二分。这是最基本的一种二分手续。

对于 $N = 8$ 的具体情形，相关链的奇偶二分过程如图 11.4 所示，图中用波纹线标出每一步的新结果。

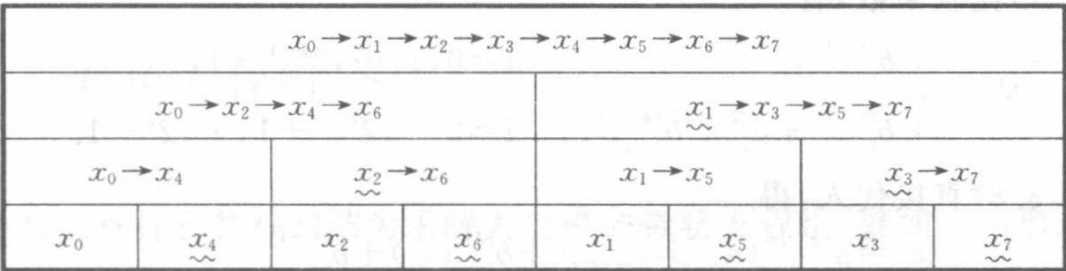


图 11.4

图 11.4 原图右侧箭头向下，逐层由左至右的全部相关链与波纹线标记如下：

层次（由上至下）	相关链或单节	波纹线标出的新结果
初始层	$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4 \rightarrow x_5 \rightarrow x_6 \rightarrow x_7$	无
第 1 次二分	$x_0 \rightarrow x_2 \rightarrow x_4 \rightarrow x_6;$ $x_1 \rightarrow x_3 \rightarrow x_5 \rightarrow x_7$	x_1

层次（由上至下）	相关链或单节	波纹线标出的新结果
第 2 次二分	$x_0 \rightarrow x_4; \ x_2 \rightarrow x_6; \ x_1 \rightarrow x_5;$ $x_3 \rightarrow x_7$	x_2, x_3
第 3 次二分	$x_0; \ x_4; \ x_2; \ x_6; \ x_1; \ x_5; \ x_3; \ x_7$	x_4, x_6, x_5, x_7

（上表为原图标签与结构的文字转录。）

11.3.2 算式的建立

现在运用消元手续具体建立上述奇偶二分法的算式。回到递推问题（11.3.1），利用它的第 $i-1$ 式从其第 i 式中消去 x_{i-1} ，得

$$\begin{cases} x_i = b_i^{(1)}, & i = 0, 1, \\ x_i = a_i^{(1)}x_{i-2} + b_i^{(1)}, & i = 2, 3, \cdots, N-1, \end{cases}$$

(11.3.4)

式中

$$a_i^{(1)} = a_i a_{i-1}, \quad i = 2, 3, \cdots, N-1,$$

(11.3.5)

原书 PDF 页 287；印刷页码 274。

[原页图像](#)

$$b_i^{(1)} = \begin{cases} b_i, & i = 0, \\ b_i + a_i b_{i-1}, & i = 1, 2, \dots, N-1. \end{cases} \quad (11.3.6)$$

容易看出，式 (11.3.4) 可按下标的奇偶拆成两个规模减半的子问题，即

$$\begin{cases} x_0 = b_0^{(1)}, \\ x_{2i} = a_{2i}^{(1)} x_{2i-2} + b_{2i}^{(1)}, & i = 1, 2, \dots, N_1 - 1; \\ \\ x_1 = b_1^{(1)}, \\ x_{2i+1} = a_{2i+1}^{(1)} x_{2i-1} + b_{2i+1}^{(1)}, & i = 1, 2, \dots, N_1 - 1. \end{cases}$$

它们分别对应于形如式 (11.3.2) 的奇偶相关链。

前已指出，二分 k 步后加工得出的相关链式 (11.3.3) 含有 2^k 个结果 x_i , $i = 0, 1, \dots, 2^k - 1$, 且其步长增至 2^k , 因此其相应的递推问题具有如下形式:

$$\begin{cases} x_i = b_i^{(k)}, & i = 0, 1, \dots, 2^k - 1, \\ x_i = a_i^{(k)} x_{i-2^k} + b_i^{(k)}, & i = 2^k, 2^k + 1, \dots, N-1. \end{cases} \quad (11.3.7)$$

为了导出系数 $a_i^{(k)}, b_i^{(k)}$ 的计算公式，回到前一步加工得出的递推问题:

$$\begin{cases} x_i = b_i^{(k-1)}, & i = 0, 1, \dots, 2^{k-1} - 1, \\ x_i = a_i^{(k-1)} x_{i-2^{k-1}} + b_i^{(k-1)}, & i = 2^{k-1}, 2^{k-1} + 1, \dots, N-1. \end{cases} \quad (11.3.8)$$

显然，据此可得出 2^{k-1} 个新结果

$$x_i = a_i^{(k-1)} b_{i-2^{k-1}}^{(k-1)} + b_i^{(k-1)}, \quad i = 2^{k-1}, 2^{k-1} + 1, \dots, 2^k - 1,$$

与式 (11.3.7) 比较系数，有

$$b_i^{(k)} = \begin{cases} b_i^{(k-1)}, & i = 0, 1, \dots, 2^{k-1} - 1, \\ b_i^{(k-1)} + a_i^{(k-1)} b_{i-2^{k-1}}^{(k-1)}, & i = 2^{k-1}, 2^{k-1} + 1, \dots, 2^k - 1. \end{cases} \quad (11.3.9)$$

又将式 (11.3.8) 直接代入，得

$$\begin{aligned} x_i &= a_i^{(k-1)} (a_{i-2^{k-1}}^{(k-1)} x_{i-2^k} + b_{i-2^{k-1}}^{(k-1)}) + b_i^{(k-1)} \\ &= (a_i^{(k-1)} a_{i-2^{k-1}}^{(k-1)}) x_{i-2^k} + (a_i^{(k-1)} b_{i-2^{k-1}}^{(k-1)} + b_i^{(k-1)}). \end{aligned}$$

再与式 (11.3.7) 比较系数，知

$$\begin{cases} a_i^{(k)} = a_i^{(k-1)} a_{i-2^{k-1}}^{(k-1)}, \\ b_i^{(k)} = b_i^{(k-1)} + a_i^{(k-1)} b_{i-2^{k-1}}^{(k-1)}, \end{cases} \quad i = 2^k, 2^k + 1, \dots, N-1.$$

将计算 $b_i^{(k)}$ 的上述两组算式归并在一起，即可归纳出求解递推问题的奇偶二分算法 11.4。

算法 11.4 对 $k = 1, 2, \dots$ 直到 $n = \log_2 N$ 执行算式

$$a_i^{(k)} = a_i^{(k-1)} a_{i-2^{k-1}}^{(k-1)}, \quad i = 2^k, 2^{k+1}, \dots, N-1; \quad (11.3.10)$$

$$b_i^{(k)} = \begin{cases} b_i^{(k-1)}, & i = 0, 1, \dots, 2^{k-1} - 1, \\ b_i^{(k-1)} + a_i^{(k-1)} b_{i-2^{k-1}}^{(k-1)}, & i = 2^{k-1}, 2^{k-1} + 1, \dots, N-1, \end{cases} \quad (11.3.11)$$

校注 (agent 补充, CH11-E003): 式 (11.3.10) 中的下标序列确实印为 $i = 2^k, 2^{k+1}, \dots, N-1$, 已按放大原图照录。与本页上方推导及式 (11.3.7) 的逐个下标范围比较, 此处第二项疑应为 $2^k + 1$, 而不是 2^{k+1} ; 属于原书排印疑误。

原书 PDF 页 288；印刷页码 275。

[原页图像](#)

则 $x_i = b_i^{(n)}$, $i = 0, 1, \dots, N-1$ 即为所求。

11.3.3 二分算法的效能分析

考察奇偶二分法的算法 11.4。按式 (11.3.10)、式 (11.3.11)，每一步需做两次运算（乘法、加法各一次），共做 $n = \log_2 N$ 步，故其时间界为

$$T^* = 2 \log_2 N.$$

再考察该算法的第 k 步。为使每个系数 $a_i^{(k)}, b_i^{(k)}$ 各有一台处理机去独立计算，则按式 (11.3.10)、式 (11.3.11) 计算所需的处理机台数为

$$P_k = (N - 2^k) + (N - 2^{k-1}) = 2N - 3 \times 2^{k-1}.$$

因此，该算法的处理机台数界为

$$P^* = \max_{1 \leq k \leq n} P_k \approx 2N.$$

注意到一阶线性递推问题串行计算的运行时间 $T_1 = 2(N-1)$ ，可知奇偶二分法的加速比

$$S = \frac{T_1}{T^*} \approx \frac{N}{\log_2 N},$$

而其效率

$$E = \frac{S}{P^*} \approx \frac{1}{2 \log_2 N}.$$

这一算法同叠加计算的二分法 (11.2.5 小节) 具有相同的加速比，只是效率降低了。

11.4 三对角方程组

科学与工程计算往往归结为求解大规模的带状方程组，譬如下列形式的三对角方程组：

$$\begin{cases} b_0 x_0 + c_0 x_1 = f_0, \\ a_i x_{i-1} + b_i x_i + c_i x_{i+1} = f_i, \quad i = 1, 2, \dots, N-2, \\ a_{N-1} x_{N-2} + b_{N-1} x_{N-1} = f_{N-1}. \end{cases} \quad (11.4.1)$$

由于其具有实用背景，这类方程组的求解一直是并行算法研究的热门课题，受到人们的广泛关注。

众所周知，求解三对角方程组的一种行之有效的方法是追赶法。早期并行算法研究的热点是直接将追赶法并行化，发掘出隐含在递推算式中的内在并行性，然而这样设计出的算法稳定性差，效果不理想。

以下运用二分技术设计求解三对角方程 (11.4.1) 的并行算法。为便于描述算法，仍然假定 $N = 2^n$ ， n 为正整数。

原书 PDF 页 289；印刷页码 276。

[原页图像](#)

11.4.1 相关链的二分手续

对于所给方程组 (11.4.1)，其每个变元 x_i 与“左邻” x_{i-1} 、“右舍” x_{i+1} 相关联，设将其相关性

$$a_i x_{i-1} + b_i x_i + c_i x_{i+1} = f_i$$

抽象地表达为如下形式的相关链：

$$x_{i-1} \leftrightarrow x_i \leftrightarrow x_{i+1}.$$

这样，所给方程组 (11.4.1) 可抽象地表达为

$$x_0 \leftrightarrow x_1 \leftrightarrow \cdots \leftrightarrow x_{N-1},$$

其步长等于 1。

假设通过某种手续，可将上列相关链按下标的奇偶拆成两条，即

$$\begin{aligned} x_0 &\leftrightarrow x_2 \leftrightarrow \cdots \leftrightarrow x_{N-2}, \\ x_1 &\leftrightarrow x_3 \leftrightarrow \cdots \leftrightarrow x_{N-1}, \end{aligned}$$

则这样加工出的两条子链，其步长均等于 2。在保持链数逐步倍增及步长逐步倍增两项基本特征的前提下反复施行这一手续，则二分 k 步后加工出 2^k 条子链，其步长均等于 2^k ，即

$$\left\{ \begin{array}{l} x_0 \leftrightarrow x_{2^k} \leftrightarrow \cdots \leftrightarrow x_{N-2^k}, \\ x_0 \leftrightarrow x_{2^k+1} \leftrightarrow \cdots \leftrightarrow x_{N-2^k+1}, \\ \vdots \\ x_{2^k-1} \leftrightarrow x_{2^k+1-1} \leftrightarrow \cdots \leftrightarrow x_{N-1}. \end{array} \right. \quad (11.4.2)$$

如此二分 $n = \log_2 N$ 次，所给相关链最终被加工成每条仅含一节的最简形式

$$x_0, x_1, \cdots, x_{N-1},$$

从而得出所求的解。

下面再就 $N = 8$ 的情形具体描述上述加工方案。对于方程组

$$\left\{ \begin{array}{l} b_0 x_0 + c_0 x_1 = f_0, \\ a_i x_{i-1} + b_i x_i + c_i x_{i+1} = f_i, \quad i = 1, 2, \cdots, 6, \\ a_7 x_6 + b_7 x_7 = f_7, \end{array} \right. \quad (11.4.3)$$

其相关链的加工过程如图 11.5 所示。

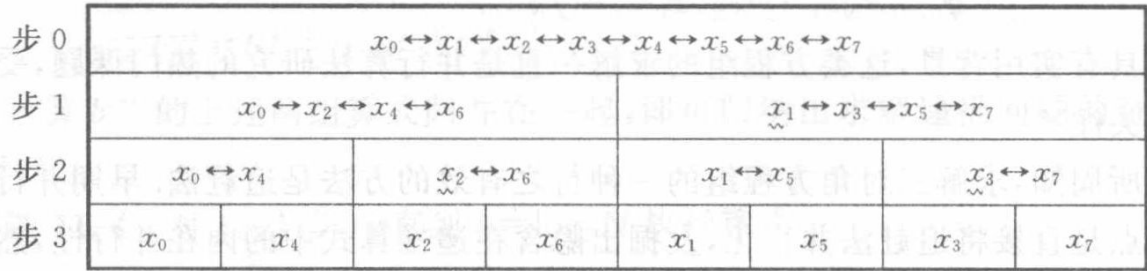


图 11.5

图 11.5 原图逐层的步数标签、相关链与波纹线符号如下：

原图步数标签	从左至右的相关链或单节	波纹线标记
步 0	$x_0 \leftrightarrow x_1 \leftrightarrow x_2 \leftrightarrow x_3 \leftrightarrow x_4 \leftrightarrow x_5 \leftrightarrow x_6 \leftrightarrow x_7$	无
步 1	$x_0 \leftrightarrow x_2 \leftrightarrow x_4 \leftrightarrow x_6$; $x_1 \leftrightarrow x_3 \leftrightarrow x_5 \leftrightarrow x_7$	x_1
步 2	$x_0 \leftrightarrow x_4$; $x_2 \leftrightarrow x_6$; $x_1 \leftrightarrow x_5$; $x_3 \leftrightarrow x_7$	x_2, x_3
步 3	x_0 ; x_4 ; x_2 ; x_6 ; x_1 ; x_5 ; x_3 ; x_7	无

(上表为原图标签与结构的文字转录。)

具体地说，其第 1 步将所给方程组 (11.4.3) 加工成两个子系统：

校注 (agent 补充, CH11-E004)：式 (11.4.2) 第二行的首项在原图中印为 x_0 ，已照录。按照前文奇偶拆分、该行后续下标以及式 (11.3.3) 的相同链结构，此首项疑应为 x_1 ；属于原书排印疑误。

原书 PDF 页 290；印刷页码 277。

[原页图像](#)

$$\begin{cases} b_0^{(1)}x_0 + c_0^{(1)}x_2 = f_0^{(1)}, \\ a_2^{(1)}x_0 + b_2^{(1)}x_2 + c_2^{(1)}x_4 = f_2^{(1)}, \\ a_4^{(1)}x_2 + b_4^{(1)}x_4 + c_4^{(1)}x_6 = f_4^{(1)}, \\ a_6^{(1)}x_4 + b_6^{(1)}x_6 = f_6^{(1)}; \end{cases}$$

$$\begin{cases} b_1^{(1)}x_1 + c_1^{(1)}x_3 = f_1^{(1)}, \\ a_3^{(1)}x_0 + b_3^{(1)}x_3 + c_3^{(1)}x_4 = f_3^{(1)}, \\ a_5^{(1)}x_2 + b_5^{(1)}x_5 + c_5^{(1)}x_6 = f_5^{(1)}, \\ a_7^{(1)}x_4 + b_7^{(1)}x_7 = f_7^{(1)}. \end{cases}$$

它们分别对应于相关链 $x_0 \leftrightarrow x_2 \leftrightarrow x_4 \leftrightarrow x_6$ 与 $x_1 \leftrightarrow x_3 \leftrightarrow x_5 \leftrightarrow x_7$ (见图 11.5)。重复这种二分手续，第 2 步进一步加工出 4 个子系统：

$$\begin{cases} b_0^{(2)}x_0 + c_0^{(2)}x_4 = f_0^{(2)}, \\ a_4^{(2)}x_0 + b_4^{(2)}x_4 = f_4^{(2)}; \end{cases} \quad \begin{cases} b_2^{(2)}x_2 + c_2^{(2)}x_6 = f_2^{(2)}, \\ a_6^{(2)}x_2 + b_6^{(2)}x_6 = f_6^{(2)}; \end{cases}$$

$$\begin{cases} b_1^{(2)}x_1 + c_1^{(2)}x_5 = f_1^{(2)}, \\ a_5^{(2)}x_1 + b_5^{(2)}x_5 = f_5^{(2)}; \end{cases} \quad \begin{cases} b_3^{(2)}x_3 + c_3^{(2)}x_7 = f_3^{(2)}, \\ a_7^{(2)}x_3 + b_7^{(2)}x_7 = f_7^{(2)}. \end{cases}$$

它们分别对应于相关链 $x_0 \leftrightarrow x_4$, $x_2 \leftrightarrow x_6$, $x_1 \leftrightarrow x_5$, $x_3 \leftrightarrow x_7$ 。最后，第 3 步将所给方程组 (11.4.3) 加工成如下形式：

$$b_i^{(3)}x_i = f_i^{(3)}, \quad i = 0, 1, \dots, 7.$$

它对应于 8 条仅含一节的子链

$$x_0, x_1, \dots, x_7,$$

从而立即得出所求的解

$$x_i = f_i^{(3)} / b_i^{(3)}, \quad i = 0, 1, \dots, 7.$$

11.4.2 算式的建立

现在运用消元手续具体建立上述奇偶二分法的算式。

回到一般形式的三对角方程组 (11.4.3)，利用它的第 $i-1$ 个方程和第 $i+1$ 个方程从其第 i 个方程中消去变元 x_{i-1} 和 x_{i+1} (不言而喻，其首尾两个方程需做特殊处理)，结果加工得出

$$\begin{cases} b_i^{(1)}x_i + c_i^{(1)}x_{i+2} = f_i^{(1)}, & i = 0, 1, \\ a_i^{(1)}x_{i-2} + b_i^{(1)}x_i + c_i^{(1)}x_{i+2} = f_i^{(1)}, & i = 2, 3, \dots, N-3, \\ a_i^{(1)}x_{i-2} + b_i^{(1)}x_i = f_i^{(1)}, & i = N-2, N-1, \end{cases}$$

式中

$$a_i^{(1)} = -a_i a_{i-1} / b_{i-1}, \quad i = 2, 3, \dots, N-1,$$

校注 (agent 补充, CH11-E005): 本页第 1 步的第二个 (奇数下标) 子系统中, 原书依次印有 $a_3^{(1)}x_0$ 、 $c_3^{(1)}x_4$ 、 $a_5^{(1)}x_2$ 、 $c_5^{(1)}x_6$ 、 $a_7^{(1)}x_4$, 已据放大原图照录。按该页明确给出的奇数相关链, 其变元依次应为 x_1, x_5, x_3, x_7, x_5 ; 这里五个偶数下标与奇数子系统不相容, 属原书下标疑误。

校注 (agent 补充, CH11-E006): 11.4.2 开头将“一般形式的三对角方程组”回引为 (11.4.3), 已照录。式 (11.4.3) 是 $N = 8$ 的示例, 一般形式在式 (11.4.1); 此处疑为原书交叉引用错误。

原书 PDF 页 291；印刷页码 278。

[原页图像](#)

$$b_i^{(1)} = \begin{cases} b_i - c_i a_{i+1}/b_{i+1}, & i = 0, 1, \\ b_i - a_i c_{i-1}/b_{i-1} - c_i a_{i+1}/b_{i+1}, & i = 2, 3, \dots, N-3, \\ b_i - a_i c_{i-1}/b_{i-1}, & i = N-2, N-1, \end{cases}$$

$$c_i^{(1)} = -c_i c_{i+1}/b_{i+1}, \quad i = 0, 1, \dots, N-3,$$

$$f_i^{(1)} = \begin{cases} f_i - c_i f_{i+1}/b_{i+1}, & i = 0, 1, \\ f_i - a_i f_{i-1}/b_{i-1} - c_i f_{i+1}/b_{i+1}, & i = 2, 3, \dots, N-3, \\ f_i - a_i f_{i-1}/b_{i-1}, & i = N-2, N-1. \end{cases}$$

可以看出，上述消元手续的特点在于，它从奇（偶）数编号的方程中消去偶（奇）数编号的变元，从而将下标为奇、偶的变元相互分离开来。也就是说，将所给方程组 (11.4.3) 加工成下标分别为奇、偶的两个子系统：

$$\begin{cases} b_0^{(1)} x_0 + c_0^{(1)} x_2 = f_0^{(1)}, \\ a_{2i}^{(1)} x_{2i-2} + b_{2i}^{(1)} x_{2i} + c_{2i}^{(1)} x_{2i+2} = f_{2i}^{(1)}, \quad i = 1, 2, \dots, N/2 - 1, \\ a_{N-2}^{(1)} x_{N-4} + b_{N-2}^{(1)} x_{N-2} = f_{N-2}^{(1)}; \end{cases}$$

$$\begin{cases} b_1^{(1)} x_1 + c_1^{(1)} x_3 = f_1^{(1)}, \\ a_{2i+1}^{(1)} x_{2i-1} + b_{2i+1}^{(1)} x_{2i+1} + c_{2i+1}^{(1)} x_{2i+3} = f_{2i+1}^{(1)}, \quad i = 1, 2, \dots, N/2 - 1, \\ a_{N-1}^{(1)} x_{N-3} + b_{N-1}^{(1)} x_{N-1} = f_{N-1}^{(1)}. \end{cases}$$

可见，上述消元手续是一项奇偶二分手续。反复施行这种二分手续加工 k 步，其相关链如式 (11.4.2) 所示，相应地，所给方程组 (11.4.3) 被加工成如下形式：

$$\begin{cases} b_i^{(k)} x_0 + c_i^{(k)} x_{i+2^k} = f_i^{(k)}, & i = 0, 1, \dots, 2^k - 1, \\ a_i^{(k)} x_{i-2^k} + b_i^{(k)} x_i + c_i^{(k)} x_{i+2^k} = f_i^{(k)}, & i = 2^k, 2^k + 1, \dots, N - 2^k - 1, \\ a_i^{(k)} x_{i-2^k} + b_i^{(k)} x_i = f_i^{(k)}, & i = N - 2^k, N - 2^k + 1, \dots, N - 1. \end{cases}$$

仿照 10.3.2 小节关于一阶线性递推的处理方法，不难导出如下算式：

$$a_i^{(k)} = -a_i^{(k-1)} a_{i-2^{k-1}}^{(k-1)} / b_{i-2^{k-1}}^{(k-1)}, \quad i = 2^k, 2^k + 1, \dots, N - 1,$$

$$b_i^{(k)} = \begin{cases} b_i^{(k-1)} - c_i^{(k-1)} a_{i+2^{k-1}}^{(k-1)} / b_{i+2^{k-1}}^{(k-1)}, & i = 0, 1, \dots, 2^k - 1, \\ b_i^{(k-1)} - a_i^{(k-1)} c_{i-2^{k-1}}^{(k-1)} / b_{i-2^{k-1}}^{(k-1)} - c_i^{(k-1)} a_{i+2^{k-1}}^{(k-1)} / b_{i+2^{k-1}}^{(k-1)}, & i = 2^k, 2^k + 1, \dots, N - 2^k - 1, \\ b_i^{(k-1)} - a_i^{(k-1)} c_{i-2^{k-1}}^{(k-1)} / b_{i-2^{k-1}}^{(k-1)}, & i = N - 2^k, N - 2^k + 1, \dots, N - 1, \end{cases}$$

$$c_i^{(k)} = -c_i^{(k-1)} c_{i+2^{k-1}}^{(k-1)} / b_{i+2^{k-1}}^{(k-1)}, \quad i = 0, 1, \dots, N - 2^k - 1,$$

校注 (agent 补充, CH11-E006): 本页关于一般 N 阶方程组的两处回引仍印为 (11.4.3), 已照录, 一般形式实际见式 (11.4.1); “10.3.2 小节关于一阶线性递推” 亦按原书照录, 该主题与本章 11.3.2 相符。这些属于原书交叉引用疑误。

校注 (agent 补充, CH11-E007): 本页 $b_i^{(1)}, f_i^{(1)}$ 的边界分段及后续 $b_i^{(k)}$ (下一页还有 $f_i^{(k)}$) 按原书照录, 但与所述同时消去左右邻元的手续不一致。例如在分母非零的前提下, 消去第 1 行方程中的 x_0, x_2 应给出 $b_1^{(1)} = b_1 - a_1 c_0 / b_0 - c_1 a_2 / b_2$ 和 $f_1^{(1)} = f_1 - a_1 f_0 / b_0 - c_1 f_2 / b_2$; 原书将 $i = 1$ 列入仅含右邻消元项的第一分支, 漏去了左邻贡献。 $i = N - 2$ 同样漏去右邻贡献。一般 k 步分段采用新步长 2^k 作为消元边界, 亦需复核; 当 $k = n$ 时首尾分支还发生重叠并出现越界下标。这是原书边界公式疑误, 未据此修订正文。

校注 (agent 补充, CH11-E008): 两个奇偶子系统的中间行范围均印为 $i = 1, 2, \dots, N/2 - 1$, 已照录。取上界时分别出现 x_N, x_{N+1} , 并与单列的末行重复; 按该分段写法, 中间行的上界疑应为 $N/2 - 2$ 。

校注 (agent 补充, CH11-E009): 二分 k 步后的分段方程首行印为 $b_i^{(k)} x_0$, 已据放大原图照录。其范围包含多个 i , 与本节各行的中心变元应为 x_i 不一致, 疑应为 $b_i^{(k)} x_i$ 。

原书 PDF 页 292；印刷页码 279。

[原页图像](#)

$$f_i^{(k)} = \begin{cases} f_i^{(k-1)} - c_i^{(k-1)} f_{i+2^{k-1}}^{(k-1)} / b_{i+2^{k-1}}^{(k-1)}, & i = 0, 1, \dots, 2^k - 1, \\ f_i^{(k-1)} - a_i^{(k-1)} f_{i-2^{k-1}}^{(k-1)} / b_{i-2^{k-1}}^{(k-1)} - c_i^{(k-1)} f_{i+2^{k-1}}^{(k-1)} / b_{i+2^{k-1}}^{(k-1)}, & i = 2^k, 2^k + 1, \dots, N - 2^k - 1, \\ f_i^{(k-1)} - a_i^{(k-1)} f_{i-2^{k-1}}^{(k-1)} / b_{i-2^{k-1}}^{(k-1)}, & i = N - 2^k, N - 2^k + 1, \dots, N - 1. \end{cases}$$

容易看出，按照上述二分消元手续加工 $n = \log_2 N$ 步，所给方程组最终退化为下列形式：

$$b_i^{(n)} x_i = f_i^{(n)}, \quad i = 0, 1, \dots, N - 1,$$

因此有算法 10.5。

算法 10.5 对 $k = 1, 2, \dots$ 直到 $n = \log_2 N$ 执行上述算式，则

$$x_i = f_i^{(n)} / b_i^{(n)}, \quad i = 0, 1, \dots, N - 1,$$

即为方程组 (11.4.1) 的解。

小结

我们生活在一个巨变的时代。并行机的出现，使科学计算领域发生了翻天覆地的变化。一个并行机系统可能拥有成千上万台处理机。面对这样一个全新的计算平台，人们感到迷茫。国外一些权威专家强调，并行算法是一门“全新”的算法，在设计过程中必须彻底摆脱传统算法设计思想的“束缚”。“串行”与“并行”难道是水火不容、不可调和的吗？

本章侧重于同步并行算法的研究。所展示的研究成果表明，串行算法与并行算法是一脉相承的。

在众多形形色色的计算模型中，数列求和无疑是最简单的。本章以这种简单模型作为并行算法设计的源头，透过它阐述并行算法设计的基本策略与基本特征。

数列求和是一个累加过程，它具有时序性与递推性。事实上，累加求和

$$\begin{cases} S_0 = a_0, \\ S_i = S_{i-1} + a_i, \quad i = 1, 2, \dots, N - 1, \end{cases}$$

本质上是个递推过程

$$S_0 \rightarrow S_1 \rightarrow \dots \rightarrow S_{N-1}.$$

我们看到，数列求和的二分算法将这种递推过程加工成如图 11.3 所示的递推结构。由此可见，递推不是串行算法的专利，并行算法进一步强化了递推结构。

然而无可非议的是，递推计算是并行算法设计的困难所在。前人提出的倍增技术试图绕开这个难点，而将递推关系式转化为某种累算形式的展开式来处理。这种“节外生枝”的处理方法增加了问题的复杂性。

与此不同，本章所推荐的二分技术直接开发递推算式内在的并行性。借助于形象

校注 (agent 补充, CH11-E006)：本页“因此有算法 10.5”及算法标题均印为 10.5，已照录。按本章算法 11.1—11.4 的连续编号，此算法疑应编号 11.5；属于原书编号疑误。

校注 (agent 补充, CH11-E007)：本页 $f_i^{(k)}$ 的分段定义延续上一页系数更新的边界问题，原式已完整保留。尤其 $k = n$ 时首、尾范围重叠，公式还可能引用越界的邻元系数；相关直接消元对照见 PDF 291 的校注。

原书 PDF 页 293；印刷页码 280。

[原页图像](#)

直观的相关链，二分法着眼于二分手续的设计。

需要指出的是，在介绍并行算法时，本章尽量回避算法的简单罗列，力图通过一些典型算法揭示同步并行算法的二分技术的有效性。运用二分技术可以设计出一系列优秀的并行算法。笔者所在的华中科技大学并行计算研究所，在 20 世纪 80 年代中期曾针对国产银河巨型机研制成功了一个“线性代数二分法软件包”。

原书 PDF 页 294；印刷页码 281。

[原页图像](#)

* 第 12 章 加速算法设计：重差加速技术

12.1 千古疑案

12.1.1 阿基米德的“穷竭法”

在数学史上，圆周率这个奇妙的数字牵动着一代又一代数学家的心，不少人为之耗费了毕生的精力。据文献记载，在这方面做出过突出贡献的，当首推古希腊的阿基米德。在公元前 3 世纪，阿基米德求出了 π 的近似值 3.14，突破了古率 $\pi = 3$ 的传统观念。阿基米德开创了圆周率科学计算的新纪元。

被誉为“古代数学之神”的阿基米德，其重大数学成就之一，是用所谓“穷竭法”计算一些曲边图形的面积，例如用内接与外切正多边形的周长来“穷竭”圆周，当多边形边数足够多时，作为正多边形的周长获得圆周率的近似值。他从正 6 边形做起，割到 12 边形、24 边形、48 边形，一直割到内接与外切正 96 边形，得到 π 的弱近似值 $\frac{223}{71}$ 与强近似值 $\frac{22}{7}$ ，据此得知 π 约等于 3.14。

此后一直到 17 世纪，在长达两千年的漫长岁月中，许多数学家都效仿阿基米德的做法，用内接与外切多边形的周长逼近圆周长，令边数逐步增多，从而获得越来越准确的圆周率。

譬如，在 15 世纪，阿拉伯数学家阿尔·卡西从正 6 边形、正 12 边形一直割到正 6×2^{27} （约 8 亿）边形，求得准确到小数点后 17 位的圆周率

$$\pi = 3.141\,592\,653\,589\,792\,23.$$

17 世纪初，德国人鲁道夫从圆的内接与外切正方形做起，用正 2^{64} （约 1800 亿亿）边形的周长逼近圆周长，得出准确到小数点后 35 位的圆周率。鲁道夫为此感到自豪，要求在他的墓碑上铭刻这项成就。因此德国人常称 π 为“鲁道夫数”。

总之，为了计算圆周率，从阿基米德到鲁道夫，西方人孜孜不倦地探索了两千年，除了阿基米德的穷竭法以外，再没有找到更好的方法和途径。然而这种方法由于收敛速度缓慢，在近代科学计算中已被人们摒弃了。

12.1.2 祖冲之“缀术”之谜

祖冲之是公元 5 世纪南北朝时期的数学家。

校注 (agent 补充)：原页印作“作为正多边形的周长获得圆周率的近似值”，语句疑有排印问题，转录保留原文。

校注 (agent 补充)：本页阿尔·卡西圆周率末段印作 ... 792 23，PDF302（书页 289）同一结果印作 ... 793 23。独立高精度计算所得 $\pi = 3.141\,592\,653\,589\,793\,238 \dots$ ，支持本页“792”为排印错误；此处仍保留原数值。

原书 PDF 页 295；印刷页码 282。

原页图像

祖冲之在数学方面的重大成就，当首推关于圆周率的计算。据《隋书·律历志》记载：

“祖冲之更造密法，以圆径一亿为一丈，圆周盈数三丈一尺四寸一分五厘九毫二秒七忽，朒数三丈一尺四寸一分五厘九毫二秒六忽，正数在盈朒二限之间。”

这就是说，祖冲之定出了圆周率的取值范围为

$$3.141\,592\,6 < \pi < 3.141\,592\,7$$

这个惊人的纪录领先世界一千多年。

此后一直到 15 世纪，再没有出现比这更好的结果。这项辉煌的数学成就，在千年漫长岁月中一直处于世界领先的地位。

祖冲之的 π 值究竟是怎样求出来的呢？据考证祖冲之的算法载于他的《缀术》一书中，可惜该书久已失传，致使这一奇妙算法成了数学史上一桩千古疑案。

后人提出了诸多猜测和假说。

华罗庚先生在《高等数学引论》一书中，以“祖冲之计算圆周率的方法”为题提出了自己的见解，他说：“祖冲之证明了圆周率 π 落在

$$3.141\,592\,6 < \pi < 3.141\,592\,7$$

之间，这是数学史上的一个光辉成就。他的算法也是极限的最好说明！他从单位圆的内接的和外切的正六边形出发，显然圆夹在这两个六边形之间，再作内接的和外切的正 12 边形、正 24 边形、……、正 $6 \cdot 2^{n-1}$ 边形等等，边数愈多，内接的和外切的正 $6 \cdot 2^{n-1}$ 边形的面积就愈接近圆的面积，由此可以逐步地精确地算出圆周的长度。”

钱宝琮先生主编的《中国数学史》指出，“缀术失传，祖冲之推算圆周率的方法难以详考。”不过，如果用多边形逼近实现如此高的精度，需要割到 24 576 边形。在筹算的古代，完成如此繁浩的计算量是极为困难的。

一个无可置疑的事实是，祖冲之计算圆周率肯定使用了某种“绝技”。

祖冲之称他的计算技术为“缀术”。据史书记载，缀术“时人称之精妙”，赞扬它“指要精密，算氏之最”。《隋书·律历志》说，祖冲之所著之书名《缀术》，“学官莫能究其深奥”。传说唐代指定《缀术》一书为朝廷钦定的数学教材。

“缀术”是什么？

祖冲之的原著《缀术》已千年失传，无从查察，人们还能还原其“真面目”吗？

12.2 神来之笔

12.2.1 数学史上一篇千古奇文

成书于汉代的《九章算术》是我国古代最重要的数学经典。该书“圆田术”给出了

校注 (agent 补充)：《隋书·律历志》引文中“朒数”“盈朒”的“朒”按原页字形保留。

原书 PDF 页 296；印刷页码 283。

[原页图像](#)

圆面积的计算公式：

“半周半径相乘得积步。”

即圆面积等于半圆周长与半径的乘积。

这一事实是人们所熟知的，然而由于圆是曲边图形，对古人来说，计算圆面积是个数学难题。

刘徽在注《九章算术》时撰写了《圆田术注》，约 1 800 字，后人称之为《割圆术》。割圆术证明了圆面积公式，并且提供了一个计算圆周率的优秀算法。

刘徽的《割圆术》是一篇千古奇文。书中有许多亮点，譬如，早在 1800 年前，刘徽就在人类数学史上首次提出了极限观念。

刘徽从圆的内接正六边形做起，令边数逐步倍增，计算圆内接正 n 边形面积 S_n ($n = 6, 12, 24, \dots$)，并建立了圆面积 S^* 的一个逼近序列

$$S_6 \rightarrow S_{12} \rightarrow S_{24} \rightarrow S_{48} \rightarrow \dots \rightarrow S^*.$$

这种加工手续开创了极限计算的先河。

相比之下，古希腊人畏惧无穷，阿基米德的穷竭法同极限思想毫不相干，因而与刘徽的“割圆术”不可同日而语。

其次，刘徽的割圆术中提出了一种高明的逼近策略，建立了下列**双侧逼近公式**

$$S_{2n} < S^* < S_{2n} + (S_{2n} - S_n).$$

在逼近过程中，刘徽直接用数据的偏差 $S_{2n} - S_n$ 作为校正量，生成圆面积 S^* 的强近似值，从而舍弃了圆的外切多边形的计算，相比阿基米德的穷竭法显著地节省了计算量。

最后，特别值得指出的是，刘徽的割圆术提出了一种逼近加速技术，在《割圆术》末尾，刘徽突然发力给出了一个神奇的精加工算式——我们称之为“刘徽神算”。

本章推荐的逼近加速技术正是从刘徽神算中感悟并提炼出来的。

12.2.2 “一飞冲天”的“刘徽神算”

前已指出，阿基米德用穷竭法割到内接与外切正 96 边形，获得圆周率 $\pi = 3.14$ ，这项成就开创了圆周率科学计算的新纪元。

人们自然会问，阿基米德为什么割到正 96 边形就终止了呢？他为什么不再继续割下去？显然更高精度的圆周率是诱人的。

这个问题的原因很简单，实际计算就会明白，在割圆过程中，每二分一次都要耗费相当大的计算量（对于古人来说，这种计算量是很大的），而且，少数几次割圆对改善精度意义不大。面对这种现实，阿基米德终止于正 96 边形得出圆周率 3.14，这种做法是明智的。

然而刘徽却不满足这个现状。他取圆半径 $r = 10$ 寸（即 1 尺）进行计算，发现正 96 边形二分割圆前后的两个结果

原书 PDF 页 297；印刷页码 284。

[原页图像](#)

$$S_{96} = 313\frac{584}{625}, \quad S_{192} = 314\frac{64}{625}$$

都相当于 $\pi = 3.14$ ，它们太粗糙了。面对这种情况，刘徽突发奇想：在几乎不耗费计算量的前提下，能否通过某种简单的加工手续，将两个粗糙的近似值 S_{96} ， S_{192} 加工成高精度的结果呢？

又想化粗为精，又不愿耗费计算量，这似乎有点异想天开。

《割圆术》下篇一开头，刘徽突然“发力”，他将偏差值

$$\Delta = S_{192} - S_{96} = \frac{105}{625}$$

乘以因子

$$\omega = \frac{36}{105} \quad (12.2.1)$$

作为 S_{192} 的校正量，求得

$$\begin{aligned} \widehat{S} &= S_{192} + \frac{36}{105}(S_{192} - S_{96}) \\ &= 314\frac{64}{625} + \frac{36}{105} \left(314\frac{64}{625} - 313\frac{584}{625} \right) = 314\frac{4}{25}. \end{aligned} \quad (12.2.2)$$

刘徽指出，这样加工的结果 $314\frac{4}{25}$ 相当于正 3 072 边形的面积。

这样得出的结果 $S_{3072} = 314\frac{4}{25}$ 相当于圆周率 $\pi = 3.1416$ ，比 $\pi = 3.14$ 一下子提高了两个数量级。真是一跃千里，一飞冲天。

我们称这个数学案例为“刘徽神算”。

刘徽神算化粗为精的加工效果太神奇了。它的设计机理已大大地超出了人们想象力的限度，因而虽历经千年，至今尚未获得人们的理解和接纳，而一直被禁锢在数学古籍之中。

该怎样破解刘徽神算的玄机呢？

“探赜索隐，钩深致远。”（《周易·系辞》）本章试图直面这样的课题：探究刘徽神算的隐秘事理，钩取深沉的规律和法则，以破解长期困扰数学界的逼近加速问题。

探索将针对刘徽神算围绕下列三个问题展开：

- (1) 刘徽神算的校正因子 $\omega = \frac{36}{105}$ 是从哪里来的？
- (2) 感悟刘徽神算，校正因子应当具有怎样的特质？
- (3) 推广刘徽神算，归纳出一类普适性的逼近加速法则。

12.3 奇光异彩

我们看到，尽管二分割圆生成的多边形面积 S_n 逼近于圆面积 S^* ，但逼近过程收

原书 PDF 页 298；印刷页码 285。

[原页图像](#)

敛缓慢，为了获得高精度的圆周率，所要耗费的计算量可能变得很大。譬如，需要割到 24 576 边形，才能得出祖冲之的“密率”3.141 592 6。在古代用算筹一类简单的计算工具，实现如此浩大的计算工程是难以想象的。

12.3.1 刘徽的新视野

面对这个现实，刘徽独具慧眼地提出了这样一个挑战性的课题：设法将已经获得的数据进行“再加工”，希望以尽量少的计算量为代价获得高精度的结果。

刘徽用一个具体的算例

$$\widehat{S} = S_{192} + \frac{36}{105}(S_{192} - S_{96}) \approx S_{3072}$$

演示了这种设计方案的可行性。

被称为“刘徽神算”的这个数学案例，实际上表达了一种数据精加工的方法。推广刘徽神算，自然可提出下述**校正技术**：

设法寻求某个**松弛因子** ω ，将偏差 $\Delta_n = S_{2n} - S_n$ 的 ω 倍作为数据 S_{2n} 的**校正量**，而使**校正值**

$$\widehat{S} = S_{2n} + \omega(S_{2n} - S_n), \quad 0 < \omega < 1 \quad (12.3.1)$$

比 S_n 和 S_{2n} 具有更高的精度。

自然会问，作为校正技术的式 (12.3.1)，为什么要选取形如 $\omega(S_{2n} - S_n)$ 的校正项呢？

前已指出刘徽在《割圆术》中导出了圆面积的双侧挤压公式

$$S_{2n} < S^* < S_{2n} + (S_{2n} - S_n),$$

借助于偏差 $\Delta_n = S_{2n} - S_n$ 这个公式可表达为

$$0 \cdot \Delta_n < S^* - S_{2n} < 1 \cdot \Delta_n,$$

即误差 $S^* - S_{2n}$ 被夹在系数分别为 0 和 1 的左右两极之间。

中华文化崇尚“中庸之道”，无论处理什么事情都要避免走极端，尽量不左不右，不偏不倚，执中致和。因此，依据上述挤压公式，应令

$$S^* - S_{2n} \approx \omega \Delta_n,$$

式中 $0 < \omega < 1$ ，即取式 (12.3.1) 作为校正公式。

另一方面，校正公式 (12.3.1) 亦可改写成两个数据 S_n 与 S_{2n} 的组合形式：

$$\widehat{S} = (1 + \omega)S_{2n} - \omega S_n.$$

按这种理解，所述精加工方法也是一种**组合技术**。

问题在于，这里组合系数为什么采取一正一负的逆反形式呢？

事实上，比较 S_n 和 S_{2n} 两个近似值，虽然它们都很粗糙，但 S_{2n} 总比 S_n 更为精确，即 S_{2n} 为“优”值而 S_n 则为“劣”值，所以加权平均应采取“激浊扬清”的态势，充分激发 S_{2n} 的“优”势而抑制 S_n 的“劣”势，为此令 S_n 的权系数 $-\omega < 0$ ，而令 S_{2n} 的权系数 $1 + \omega > 1$ 。

原书 PDF 页 299；印刷页码 286。

[原页图像](#)

激浊扬清、优劣互补的态势，标志着这种技术从属于高效算法设计的二分演化模式。

总之，作为刘徽神算推广的校正公式 (12.3.1)，是校正技术与组合技术两种技术的综合。为使这类技术真正保证提高精度的要求，该怎样具体地选取松弛因子 ω 呢？

12.3.2 偏差比中传出好“消息”

再换一种视角考察前述数据校正技术。针对无穷逼近过程 $\{S_n\}$ ，能否设计更“好”的逼近过程 $\{\widehat{S}_n\}$ ，使之具有更快的收敛速度呢？这就是逼近加速问题。

欲使校正公式 (12.3.1) 成为逼近加速公式，校正因子 ω 应当是驾驭逼近过程的某个数学不变量，即与逼近过程相关的某个普适常数，称之为加速因子。前已指出，这个常数 ω 应当在 0 与 1 之间，即 $0 < \omega < 1$ 。

数学常数通常采取比率的形式，刘徽指出，比率的本意是相关事物的比例关系。

在二分割圆过程中，需要考虑什么样的“相关量”呢？

《割圆术》称偏差 $\Delta_n = S_{2n} - S_n$ 为差幂，刘徽特别关注幂率 $\delta_n = \Delta_n / \Delta_{2n}$ 。在割圆计算过程中，利用差幂 Δ_n 计算幂率 δ_n ，割圆术实际上已造出下列数据表 12.1。

表 12.1

n	S_n	Δ_n	δ_n
12	300	$10\frac{364}{625}$	3.95
24	$310\frac{364}{625}$	$2\frac{425}{625}$	3.99
48	$313\frac{164}{625}$	$\frac{420}{625}$	4.00
96	$313\frac{584}{625}$	$\frac{105}{625}$	
192	$314\frac{64}{625}$		

从这些数据中能获得什么样的信息呢？无需用高深的数学知识进行理论分析，直接观察幂率 δ_n 的数据表（见表 12.1）即可发现，幂率几乎为定值 4。

刘徽由此发现了一个奇妙的事实：表面上杂乱无章的数据 S_n 中竟蕴藏着极其鲜明的规律性。

12.3.3 只要做一次“俯冲”

偏差比几乎是个定值，基于这个奇妙的规律，分析误差只是一蹴而就的事，据此只要做一次“俯冲”便能捕捉到所要的校正公式。

事实上，由于在割圆计算过程中偏差比近似等于 4，从而得出一系列近似关系

原书 PDF 页 300；印刷页码 287。

[原页图像](#)

式：

$$\begin{aligned} S_{2n} - S_n &\approx 4(S_{4n} - S_{2n}), \\ S_{4n} - S_{2n} &\approx 4(S_{8n} - S_{4n}), \\ S_{8n} - S_{4n} &\approx 4(S_{16n} - S_{8n}), \\ &\vdots \\ S_{2N} - S_N &\approx 4(S_{4N} - S_{2N}), \end{aligned}$$

式中， N 是某个远大于 n 的正整数。

将这些式子累加在一起，其中间项相互抵消，得

$$S_{2N} - S_n \approx 4(S_{4N} - S_{2n}).$$

这样，若取 S_{4N} 和 S_{2N} 作为 S_{2n} 的校正值 $\widehat{S}$ ，则有

$$\widehat{S} - S_n = 4(\widehat{S} - S_{2n}), \quad (12.3.2)$$

即有

$$\frac{\widehat{S} - S_n}{\widehat{S} - S_{2n}} \approx 4.$$

如果称近似值 S_n 与校正值 $\widehat{S}$ 两者之差为残差，则上式说明，若偏差比几乎为定值 4，那么残差比也近似等于 4。

这样一来，精加工方法的设计就水到渠成了。事实上，从式 (12.3.2) 解出未知的 $\widehat{S}$ ，有

$$\widehat{S} = S_{2n} + \frac{1}{3}(S_{2n} - S_n). \quad (12.3.3)$$

这个式子的含义是，在偏差比“几乎”为定值 4 的情况下，近似值的残差比也近似等于 4，因而残差 $\widehat{S} - S_{2n}$ 几乎等于偏差 $S_{2n} - S_n$ 的 $\frac{1}{3}$ ，从而立即导出所求的加速公式 (12.3.3)。

值得指出的是，刘徽神算 (12.2.2) 即

$$\widehat{S} = S_{192} + \frac{36}{105}(S_{192} - S_{96})$$

中的校正因子 $\omega = \frac{36}{105}$ 非常接近实际精加工公式 (12.3.3) 中的校正因子 $\omega = \frac{1}{3} = \frac{35}{105}$ 。至于刘徽为什么这样处理，一个明显的理由是，刘徽过于偏爱计算公式与计算结果的简洁美。

12.3.4 差之毫厘，失之千里

加速因子究竟怎样选取才算合适呢？循着刘徽割圆术的思路，精确地计算直

原书 PDF 页 301；印刷页码 288。

[原页图像](#)

到正 3 072 边形的面积，直接选取加速因子 $\omega = \frac{1}{3}$ ，再按式 (12.3.3) 计算，计算结果列于表 12.2 中，表中括弧 $\langle \cdot \rangle$ 标明数据准确到小数点后第几位。 π 的真值为 3.141 592 653 5…。

表 12.2

n	S_n	$\widehat{S}_n$
12	3.000 000 000 $\langle 0 \rangle$	
24	3.105 828 541 $\langle 1 \rangle$	3.141 104 722 $\langle 3 \rangle$
48	3.132 628 613 $\langle 1 \rangle$	3.141 561 971 $\langle 4 \rangle$
96	3.139 350 203 $\langle 1 \rangle$	3.141 590 733 $\langle 5 \rangle$
192	3.141 031 951 $\langle 3 \rangle$	3.141 592 534 $\langle 6 \rangle$
384	3.141 452 472 $\langle 3 \rangle$	3.141 592 646 $\langle 7 \rangle$
768	3.141 557 608 $\langle 4 \rangle$	3.141 592 653 $\langle 8 \rangle$
1 536	3.141 583 892 $\langle 4 \rangle$	3.141 592 654 $\langle 8 \rangle$
3 072	3.141 590 463 $\langle 5 \rangle$	3.141 592 654 $\langle 8 \rangle$

在割圆计算中，刘徽已获知直到正 3 072 边形的数据。以上数据表显示，利用这些数据按照刘徽加速技术进行精加工，即可获得祖冲之的“密率”3.141 592 65。

刘徽如果这样做，那么中华数学史乃至世界数学史上有关圆周率科学计算部分就要彻底改写了。

差之毫厘，失之千里。刘徽的校正技术极为精彩，只是由于校正因子的处理稍欠精细，从而失之交臂，把一项千年称雄的数学成就留给了两百年后的祖冲之。

12.3.5 “缀术”再剖析

本章一开头指出，南北朝数学家祖冲之给出了高精度的圆周率，这项成就领先世界一千多年。

祖冲之的圆周率是怎样得出来的？如果直接用内接正多边形来逼近，要一直算到正 $24\,576 = 6 \times 2^{12}$ 边形。耗费如此巨大的计算量，在筹算的古代是难以想象的。

中华民族是个智慧的民族，是个善于创新的民族。刘徽的加速技术是中华文明前瞻性思维的一个明证。

据隋唐古书记载，祖冲之的算法设计技术称为“缀术”。史书盛赞缀术“指要精密，

校注 (agent 补充)：表 12.2 的 S_n 数值从 3.000 000 000 开始，按原表保留；此前半径 $r = 10$ 寸的表 12.1 数值从 300 开始。本页未明说此处改用单位圆或作了归一化，转录未擅自改变尺度。

原书 PDF 页 302；印刷页码 289。

[原页图像](#)

算氏之最”。但关于缀术的具体内容已无从考证，仅仅靠“缀术”这个名词能够窥探出其中的奥秘吗？

查看汉语词典，“缀”字有两个含义：一是“缀合”，即组合，因此“缀术”就是组合技术；另一个含义是“缀补”，即修补和校正，在这个意义上，“缀术”亦可理解为校正技术。

这样，所谓“缀术”其实就是组合技术与校正技术。本节一开头就指出，刘徽的精加工技术正是这种技术。也许，据此可以断定，祖冲之的“缀术”正是继承了刘徽的“衣钵”——特别是刘徽的割圆术，归纳总结出来的。

数学史先辈钱宝琮先生早就猜测过祖冲之与刘徽之间的传承关系。我们深信，祖冲之将自己的算法设计技术命名为“缀术”，目的也是试图向世人表白：自己的数学成就，只是前人特别是刘徽的研究工作的缀补和修正。因此，“缀术”实质上只是《九章算术》刘徽注的祖冲之注。

也许这就是历史的真相。

12.3.6 平庸的新纪录

前已指出，祖冲之于公元 5 世纪所获得的准确到小数点后 7 位的圆周率 3.141 592 6...千年称雄于世界。这项纪录直到 15 世纪才被打破。阿拉伯人阿尔·卡西于 1424 年写成《圆周论》，发表了当时世界上最为精确的圆周率。

阿尔·卡西的割圆过程袭用阿基米德的做法：从正六边形做起，逐步计算圆的内接与外切多边形的周长。每分割一次，令多边形的边数倍增。到了 15 世纪，阿拉伯数字和十进小数记数法的使用，给实际计算提供了很大方便。阿尔·卡西反复割圆，一直算到

$$6 \times 2^{27} = 805\,306\,368$$

即正 8 亿多多边形，得出准确到小数点后 17 位的圆周率

$$\pi = 3.141\,592\,653\,589\,793\,23,$$

从而打破了祖冲之千年称雄的世界纪录。

阿尔·卡西的这项计算其实是平庸的。其一，他所使用的其实就是刘徽公式；其二，后文将会看到，运用刘徽的加速技术，用正 24 576 边形的数据就能加工出阿尔·卡西正 8 亿多多边形的结果。

这里再现阿尔·卡西所获得的数据。令直径为 1，则内接正 n 边形的边长

$$L_n = n \cdot \sin \frac{\pi}{n}.$$

设取 $n = 6, 12, 24, \dots$ 反复进行计算，计算结果列于表 12.3 中。表中数据是借助于数学软件 Mathematica 获得的，计算过程避开了舍入误差的积累。

校注 (agent 补充)：本页圆周率末段印作 ...793 23，PDF294 (书页 281) 同一阿尔·卡西结果印作 ...792 23，两处原文不一致，均按原页保留。

校注 (agent 补充)：原文称 $L_n = n \sin(\pi/n)$ 为“边长”，该式实际为直径 1 的圆内接正 n 边形的周长；单边长应为 $\sin(\pi/n)$ 。这是原书文字与所列公式的不一致，正文未作替换。

原书 PDF 页 303；印刷页码 290。

[原页图像](#)

表 12.3

n	L_n	δ_n
6	3.000 000 000 000 000 000	$\langle 0 \rangle$ 3.948 815 549 036 689 1
12	3.105 828 541 230 249 148	$\langle 1 \rangle$ 3.987 162 707 851 017 4
24	3.132 628 613 281 238 197	$\langle 1 \rangle$ 3.996 788 098 191 334 7
48	3.139 350 203 046 867 207	$\langle 1 \rangle$ 3.999 196 863 295 489 2
96	3.141 031 950 890 509 638	$\langle 3 \rangle$ 3.999 799 205 744 363 9
192	3.141 452 472 285 462 075	$\langle 3 \rangle$ 3.999 949 800 806 102 4
384	3.141 557 607 911 857 645	$\langle 4 \rangle$ 3.999 987 450 162 151 0
768	3.141 583 892 148 318 408	$\langle 4 \rangle$ 3.999 996 862 538 076 8
1 536	3.141 590 463 228 050 095	$\langle 5 \rangle$ 3.999 999 215 634 365 4
3 072	3.141 592 105 999 271 550	$\langle 6 \rangle$ 3.999 999 803 908 581 7
6 144	3.141 592 516 692 157 447	$\langle 6 \rangle$ 3.999 999 950 977 144 8
12 288	3.141 592 619 365 383 955	$\langle 7 \rangle$ 3.999 999 987 744 286 1
24 576	3.141 592 645 033 690 896	$\langle 7 \rangle$ 3.999 999 996 936 071 5
49 152	3.141 592 651 450 767 651	$\langle 8 \rangle$ 3.999 999 999 234 017 8
98 304	3.141 592 653 055 036 841	$\langle 9 \rangle$ 3.999 999 999 808 504 4
196 608	3.141 592 653 456 104 139	$\langle 9 \rangle$ 3.999 999 999 952 126 1
393 216	3.141 592 653 556 370 963	$\langle 10 \rangle$ 3.999 999 999 988 031 5
786 432	3.141 592 653 581 437 669	$\langle 11 \rangle$ 3.999 999 999 997 007 8
1 572 864	3.141 592 653 587 704 346	$\langle 11 \rangle$ 3.999 999 999 999 251 9
3 145 728	3.141 592 653 589 271 015	$\langle 12 \rangle$ 3.999 999 999 999 812 9
6 291 456	3.141 592 653 589 662 682	$\langle 12 \rangle$ 3.999 999 999 999 953 2
12 582 912	3.141 592 653 589 760 599	$\langle 13 \rangle$ 3.999 999 999 999 988 3
25 165 824	3.141 592 653 589 785 078	$\langle 13 \rangle$ 3.999 999 999 999 997 0
50 331 648	3.141 592 653 589 791 198	$\langle 14 \rangle$ 3.999 999 999 999 999 2
100 663 296	3.141 592 653 589 792 728	$\langle 14 \rangle$ 3.999 999 999 999 999 8
201 326 592	3.141 592 653 589 793 110	$\langle 15 \rangle$ 3.999 999 999 999 999 9
402 653 184	3.141 592 653 589 793 206	$\langle 16 \rangle$
805 306 368	3.141 592 653 589 793 230	$\langle 17 \rangle$

原书 PDF 页 304；印刷页码 291。

[原页图像](#)

表 12.3 所列的计算结果表明，在二分逼近过程中，反复计算内接正 n 边形的周长 L_n ，发现阿尔·卡西的“新纪录”果然是对的。

为了运用刘徽方法进行精加工，首先需要澄清偏差比是否“几乎”为定值。利用逼近数据 L_n 计算偏差 $\Delta_n = L_{2n} - L_n$ ，进而求出偏差比 $\delta_n = \Delta_n/\Delta_{2n}$ ，计算结果 δ_n 列于表 12.3 的右侧。我们看到，这里偏差比 δ_n 越来越逼近定值 4，因此加速公式仍具有式 (12.3.3) 的形式：

$$\widehat{L}_n = L_{2n} + \frac{1}{3}(L_{2n} - L_n).$$

依这一公式加工表 12.3 的数据 L_n ，加工结果 $\widehat{L}_n$ 列于表 12.4 中。数据尾部依然标明准确到小数第几位。

表 12.4

n	$\widehat{L}_n$	n	$\widehat{L}_n$
6	3.141 104 721 640 332 197 <3>	768	3.141 592 653 587 960 658 <11>
12	3.141 561 970 631 567 880 <4>	1 536	3.141 592 653 589 678 702 <12>
24	3.141 590 732 968 743 543 <5>	3 072	3.141 592 653 589 786 079 <13>
48	3.141 592 533 505 057 115 <6>	6 144	3.141 592 653 589 792 791 <14>
96	3.141 592 646 083 779 554 <7>	12 288	3.141 592 653 589 793 210 <16>
192	3.141 592 653 120 656 168 <9>	24 576	3.141 592 653 589 793 236 <17>
384	3.141 592 653 560 471 996 <10>		

对照表 12.3 和表 12.4 可以看出，为了获得阿尔·卡西割到正 8 亿多多边形所创造的新纪录，运用刘徽的精加工方法只要割到正 24 576 边形，后者相当于祖冲之已经掌握的逼近数据。

刘徽加速技术的效果简直令人难以置信！

12.4 万能引擎

数学研究中常常碰到复杂的数学对象。面对复杂，数学家往往不是进行正面的“攻击”，而是设法用某种简单去逼近它。例如用正多边形逼近圆周计算圆周长与圆面积。通常这种化繁为简、以简御繁的逼近过程是个无穷过程。

无穷逼近过程的收敛速度决定逼近方法的成败和优劣，一个过于缓慢的逼近过程是没有实用价值的，因此人们特别关注逼近过程的加速技术。

校注 (agent 补充)：按本页原式，表中 $\widehat{L}_{24\,576}$ 需要 $L_{24\,576}$ 和 $L_{49\,152}$ 两个数据。正文“只要割到正 24 576 边形”与公式及表下标所表示的数据需求不一致；原文和数值均予保留。

原书 PDF 页 305；印刷页码 292。

[原页图像](#)

12.4.1 逼近加速的重差公设

逼近加速是个难题，如果没有一个深邃的思想作为指导是很难圆满解决的。重差术是刘徽学说又一亮点。限于篇幅，这里避开重差术的历史渊源以及它在测高望远领域中的广泛应用，而仅仅着眼于它同逼近加速技术的联系。

讨论一般形式的逼近过程

$$\{a_n\} : a_0, a_1, a_2, \dots$$

设逼近数列 $\{a_n\}$ 单调有界，则它必有极限 a^* 。记其**偏差** $\Delta_n = a_{n+1} - a_n$ ，相邻偏差的比值

$$\delta_n = \frac{\Delta_n}{\Delta_{n+1}} = \frac{a_{n+1} - a_n}{a_{n+2} - a_{n+1}}$$

称为**重差**。

顾名思义，重差就是差数的重叠，即偏差比。

如果逼近过程的重差 δ_n “几乎”是个定值 δ ，则称它满足**重差公设**。容易想象，重差公设是逼近数列的一个本质属性。我们将会看到，重差公设是保证逼近加速的一台万能引擎。

12.4.2 重差加速法则

考察收敛的逼近数列 $\{a_n\}$ ：

$$a_1, a_2, \dots, a_n \rightarrow a^*.$$

假设它满足重差公设，即其偏差比 $\delta_n = \Delta_n / \Delta_{n+1}$ ， $\Delta_n = a_{n+1} - a_n$ “几乎”为定值 δ ，亦即

$$\frac{a_{n+1} - a_n}{a_{n+2} - a_{n+1}} \approx \delta, \quad \delta > 1, \quad (12.4.1)$$

则所生成的数列

$$\begin{aligned} \hat{a}_n &= a_{n+1} + \frac{1}{\delta - 1}(a_{n+1} - a_n) \\ &= \left(1 + \frac{1}{\delta - 1}\right)a_{n+1} - \frac{1}{\delta - 1}a_n \end{aligned} \quad (12.4.2)$$

远比原先的数列 $\{a_n\}$ 收敛得更快，即 $\hat{a}_n$ 比 a_n 具有更高的精度。

这一加速方法称作**重差加速法则**。式 (12.4.2) 称作**重差加速公式**，或称精加工公式。 $\hat{a}_n$ 称作 a_n 的**改进值**。

重差加速的提出显然是前述刘徽加速的推广。仿照上一节的讨论不难证明重差加速的合理性。这里无需赘述。

值得强调指出的是，刘徽重差加速技术是极度并行的。因为松弛因子 δ 是整个逼近过程 $\{a_n\}$ 的普适常数，所以精加工公式 (12.4.2) 对任何数据 $\{a_n\}$ 均有效。

原书 PDF 页 306；印刷页码 293。

[原页图像](#)

12.4.3 重差加速的逻辑推理

假定所给逼近数列 $\{a_n\}$ 由迭代公式

$$a_{n+1} = \Phi(a_n)$$

所生成，则有

$$\begin{aligned} a^* - a_{n+1} &= \Phi(a^*) - \Phi(a_n) \\ &= \Phi'(\xi_1)(a^* - a_n). \end{aligned}$$

又

$$\begin{aligned} a_{n+2} - a_{n+1} &= \Phi(a_{n+1}) - \Phi(a_n) \\ &= \Phi'(\xi_2)(a_{n+1} - a_n), \end{aligned}$$

假定导数 $\Phi'(\xi)$ 在所考察的范围内改变不大，则据上式得知

$$\frac{a^* - a_{n+1}}{a^* - a_n} \approx \frac{a_{n+2} - a_{n+1}}{a_{n+1} - a_n}.$$

这样，利用重差公设（12.4.1）可以断定

$$\frac{a^* - a_{n+1}}{a^* - a_n} \approx \frac{1}{\delta},$$

即有

$$a^* \approx \frac{\delta}{\delta - 1} a_{n+1} - \frac{1}{\delta - 1} a_n,$$

因而有加速公式（12.4.2），重差加速法则得证。

上述推理过程自然会令人联想起经典数值分析中的松弛技术（参看第 4 章 4.3 节），两种加速公式的形式相同。

不过，更要区分传统的松弛技术与刘徽的重差技术两者的差异：

其一，松弛技术的前提是给出了具体的迭代函数 Φ ，并且要求函数 Φ 具有一定的光滑性，而重差加速的理论分析所针对的迭代函数 Φ 是虚构的，实际问题中并不存在。前述理论分析只是“借题发挥”而已。

其二，传统的松弛技术是近代学者的研究成果，而重差加速是中华先贤刘徽在两千年前提出的大智慧，两者不可同日而语。

其三，传统的松弛技术仅仅解决一些简单的数值分析例题，而刘徽的重差加速普遍适用于形形色色的大规模的计算工程，两者相比，真是“小巫见大巫”了。

值得指出的是，这里关于刘徽加速的讨论其实不能算作证明，充其量只是一种“说明”而已。逼近过程的收敛性与逼近加速的关系能否给出严格的逻辑证明，有待人们进一步深入研究。

校注 (agent 补充)：原文“参看第 4 章 4.3 节”已保留。回看 PDF101 (书页 88)，该节实际标题为“4.3 Romberg 算法”，此处对“松弛技术”的章节指引疑有误；未推定或代填作者意指的章节号。

原书 PDF 页 307；印刷页码 294。

[原页图像](#)

* 第 13 章总览

13.1 算法重在设计

电子计算机的问世开创了现代科学的新时代。随着计算机的广泛应用，科学计算正成为一种新的科学方法，它与科学实验、科学理论并列，构成科学方法论的三大组成部分。

今天，随着科学技术的蓬勃发展，实际课题的规模空前扩大，大型乃至超大型科学计算日益为人们所重视。与此相适应，巨型计算机在科学计算中正扮演着越来越重要的角色。计算机的更新换代强有力地推动着算法研究的深入，科学计算正面临蓬勃发展的新机遇。

13.1.1 算法设计关系到科学计算的成败

计算机是一种功能很强的计算工具。现代超级计算机的运算速度已高达每秒亿亿次。计算机运算速度如此之快，是否意味着计算机上的算法可以随意选择呢？

举个简单的例子。

众所周知，Cramer 法则原则上可用来求解线性方程组。用这种方法求解一个 n 阶方程组，要计算 $n+1$ 个 n 阶行列式的值，总共要做 $(n+1)n!(n-1)$ 次乘除操作。当 n 充分大时，这个计算量是惊人的。譬如一个不算太大的 20 阶线性方程组，大约要做 10^{21} 次乘除操作，这项计算即使用每秒 30 万亿次的超级计算机来承担，也得要连续工作

$$\frac{10^{21}}{3 \times 10^{11} \times 60 \times 60 \times 24 \times 365} \approx 1(\text{年})$$

才能完成。当然这是完全没有实际意义的。

其实，求解线性方程组有许多实用解法。譬如，运用人们熟悉的消元技术，一个 20 阶的线性方程组即使用普通的计算器也能很快地解出来。这个简单的例子说明，**能否合理地选择算法是科学计算成败的关键。**

随着计算机的广泛应用与日益普及，算法设计的重要性正越来越为人们所认识。《计算机大百科全书》在其“算法学”词条中指出：“凡与计算机打交道，无不研究各种类型的算法。”“**算法学是计算机科学最重要的内容，有的计算机学者甚至称，计算机科学就是算法的科学。**”

校注 (agent 补充, CH13-E001): 原页正文印“每秒 30 万亿次”，上式分母却印 3×10^{11} ，且计算结果印为“ ≈ 1 (年)”。此处按原式保留。 $30 \text{ 万亿} = 3 \times 10^{13}$ ，用正文速度计算约为 1.057 年；用原式分母计算约为 105.7 年。疑为原书分母指数排印错误，非 OCR 错误。已对照放大原图核对指数为 11。

原书 PDF 页 308；印刷页码 295。

[原页图像](#)

13.1.2 算法设计追求简单与统一

在知识“大爆炸”的今天，算法的数量也正以“大爆炸”的速度与日俱增，所涉及的文献著作数以千万计，形成浩繁的卷帙。面对这知识的汪洋大海，该如何进行有效的学习呢？许多有志于从事科学计算的青年工作者正为这门学科的知识庞杂所困扰。

学习和研究算法，应当从最简单的做起。每学一个专题，首先剖析一两个最简单、最初等的范例。基于这些极其简单的范例可以提炼出一般性的设计技术，这是个“点石成金”的过程。

要学好算法，关键在于将各种各样的具体算法进行归纳分类，并触类旁通。据我国最古老的一部算书《周髀算经》记载，上古先贤陈子教导后人，学习算法要有“智类之明”，“问一类而以万事达”。陈子这种“问一知万”的大智慧，是一服解读各种算法的灵丹妙药。

1976 年，英国著名数学家、菲尔兹奖获得者 Atiyah 在题为“数学的统一性”的演讲中，突出地强调了数学的简单性和统一性。他说：“数学的目的，就是用简单而基本的词汇去尽可能地解释世界。……如果我们积累起来的经验要一代一代传下去的话，我们就必须不断地努力把它们加以简化和统一。”

算法设计追求简单和统一。后文将基于几个有趣的范例，提炼出算法设计的一条基本原理，进而概括出算法设计的几种基本技术。

13.2 直接法的缩减技术

所谓直接法是这样一类算法，它通过有限步计算可以直接得出问题的精确解（如果不考虑舍入误差的话）。

13.2.1 数列求和的累加算法

下述数列求和问题是人们所熟知的：

$$S = a_0 + a_1 + \cdots + a_n. \quad (13.2.1)$$

这个计算模型有两个简单的特例。当 $n = 0$ 即为一项和式 $S = a_0$ 时，所给计算模型就是它的解，这时不需要做任何计算。这表明，对于数列求和问题，它的解是计算模型退化的情形。又当 $n = 1$ 即计算两项和式 $S = a_0 + a_1$ 时，计算过程是平凡的，这时不存在算法设计问题。

现在基于这两种简单情形考察所给和式 (13.2.1) 的累加求和算法。设 b_k 表示前 $k+1$ 项的部分和 $a_0 + a_1 + \cdots + a_k$ ，则有

原书 PDF 页 309; 印刷页码 296。

[原页图像](#)

$$\begin{cases} b_0 = a_0, \\ b_k = b_{k-1} + a_k, \quad k = 1, 2, \dots, n, \end{cases} \quad (13.2.2)$$

而计算结果 b_n 即为所求的和值

$$S = b_n. \quad (13.2.3)$$

上述数列求和的累加算法, 其设计思想是将多项求和 (式 (13.2.1)) 化归为两项求和 (式 (13.2.2)) 的重复。而依式 (13.2.2) 重复加工若干次, 最终即可将所给和式 (13.2.1) 加工成一项和式 (13.2.3) 的退化情形, 从而得出和值 S 。

再剖析计算模型自身的演变过程。按式 (13.2.2) 每加工一次, 所给和式 (13.2.1) 便减少一项, 而所生成的计算模型依然是数列求和。反复施行这种加工手续, 计算模型不断变形为

$$\begin{array}{c} n+1 \text{ 项和式} \Rightarrow n \text{ 项和式} \Rightarrow n-1 \text{ 项和式} \Rightarrow \dots \Rightarrow 1 \text{ 项和式}, \\ \text{(计算模型)} \qquad \qquad \qquad \text{(所求结果)} \end{array}$$

这里, 符号 “ $\Rightarrow$ ” 表示重复施行两项求和的加工手续。

这样, 如果定义和式的项数为数列求和问题的**规模**, 则所求和值可以视作规模为 1 的退化情形。因此, 只要令和式的规模 (项数) 逐次减 1, 最终当规模为 1 时即可直接得出所求的和值。这样设计出的算法就是累加求和算法。

上述累加求和算法可以视作规模缩减技术的一个范例。

13.2.2 缩减技术的设计机理

许多数值计算问题可以引进某个实数——所谓问题的**规模**来刻画其“大小”, 而问题的解则是其规模为足够小, 譬如规模为 1 或 0 的退化情形。求解这类问题, 一种行之有效的办法是通过某种简单的运算手续逐步缩减问题的规模, 直到加工得出所求的解。算法设计的这种技术称作**规模缩减技术**, 简称**缩减技术**。

缩减技术适用的一类问题是, 求解这类问题的困难在于它的规模 (适当定义) 比较大。针对这类问题运用缩减技术, 就是设法逐步缩减计算问题的规模, 直到规模变得足够小时直接生成或方便地求出问题的解。

缩减技术的设计机理可用“**大事化小, 小事化了**”这句俗话来概括。

所谓“大事化小”意即逐步压缩问题的规模。在运用缩减技术时, “大事”是如何“化小”的呢? 这个处理过程具有如下两项基本特征。

(1) **结构递归**: “大事化小”是逐步完成的, 其每一步将所考察的计算模型加工成同样类型的计算模型, 因而这类算法具有明晰的递归结构。

(2) **规模递减**: 每一步加工前后的计算模型虽然从属于同一类型, 但其规模已被压缩了。压缩系数愈小, 算法的效率愈高。

再考察“小事化了”的处理过程。**所谓“小事化了”, 是指当问题的规模变得足够小时即可直接或方便地得出问题的解。**

原书 PDF 页 310；印刷页码 297。

[原页图像](#)

“小事”是如何“化了”的呢？

对于某些计算模型，如前面讨论过的数列求和问题，它们的规模为正整数，而其解则是规模为 0 或 1 的退化情形。这时只要设法使规模逐次减 1，加工若干步后即可直接得出所求的解。这里“小事化了”是直截了当的。

这样设计出一类算法统称**直接法**。前述数列求和的累加算法以及下面将要讲到的多项式求值的秦九韶算法都是直接法。

13.2.3 多项式求值的秦九韶算法

微积分方法的核心是逼近法。多项式是微积分学中最为基本的一种逼近工具，因而多项式求值算法在微积分计算中具有重要意义。

设要对给定的 x 计算下列多项式的值：

$$P = a_0x^n + a_1x^{n-1} + \cdots + a_{n-1}x + a_n = \sum_{k=0}^n a_kx^{n-k}. \quad (13.2.4)$$

由于计算每一项 a_kx^{n-k} 需做 $n-k$ 次乘法，如果先逐项计算 a_kx^{n-k} ，然后再累加求和得出多项式的值 P ，这种**逐项生成算法**所要耗费的乘法次数为

$$Q = \sum_{k=0}^n (n-k) \approx \frac{n^2}{2}.$$

当 n 充分大时，这个计算量是相当大的。

现在设法改进这一算法。类似于数列求和计算，首先考察两个特例：当 $n=0$ 时，所给计算模型即为所求的解

$$P = a_0,$$

这时不需要做任何计算；又当 $n=1$ 时，计算模型

$$P = a_0x + a_1$$

为简单的一次式，这时虽然需要进行计算，但不存在算法设计问题。

注意，当 $x=1$ 时多项式 (13.2.4) 便退化为和式 (13.2.1)，可以类比数列求和算法的设计过程讨论多项式求值算法的设计问题。

设将多项式的次数规定为多项式求值问题的规模，如果从式 (13.2.4) 的前两项中提出公因子 x^{n-1} ，则有

$$P = (a_0x + a_1)x^{n-1} + \sum_{k=2}^n a_kx^{n-k}.$$

这样，如果算出一次式

$$v_1 = a_0x + a_1$$

的值, 则所给 n 次式的计算模型 (13.2.4) 便化归为 $n-1$ 次式

$$P = v_1x^{n-1} + \sum_{k=2}^n a_kx^{n-k}$$

原书 PDF 页 311；印刷页码 298。

原页图像

的计算，从而使问题的规模减少了 1 次。不断地重复这种加工手续，使计算问题的规模逐次减 1，则经过 n 步即可将所给多项式的次数降为 0，从而获得所求的解：

算法 13.1（秦九韶算法） 令 $v_0 = a_0$ ，对 $k = 1, 2, \dots, n$ 计算

$$v_k = xv_{k-1} + a_k, \quad (13.2.5)$$

则结果 $P = v_n$ 即为所给多项式 (13.2.4) 的值。

容易看出，按递推算式 (13.2.5) 计算多项式 (13.2.4) 的值，总共只要做 n 次乘法，其计算量远比前述逐项生成算法的计算量小。这是一种优秀算法。

这一优秀算法称作**秦九韶算法**。它是公元 13 世纪我国南宋大数学家秦九韶 (1208—1261) 最先提出来的。需要注意的是，国外文献常称这一算法为 Horner 算法，其实 Horner 的工作比秦九韶晚了五六百年。

秦九韶算法说明， n 次式 (13.2.4) 的求值问题可化归为一次式 (13.2.5) 求值计算的重复。设以符号 “ $\Rightarrow$ ” 表示一次式的求值手续，则秦九韶算法的模型加工流程如下：

$$\begin{array}{ccccccc} n \text{ 次式求值} & \Rightarrow & n-1 \text{ 次式求值} & \Rightarrow & n-2 \text{ 次式求值} & \Rightarrow & \cdots \Rightarrow 0 \text{ 次式求值} \\ \text{(计算模型)} & & & & & & \text{(计算结果)} \end{array}$$

13.3 迭代法的校正技术

上一节介绍了设计直接法的缩减技术。缩减技术针对这样的问题，它的规模为正整数，而解则是规模足够小（通常规模为 0 或 1）的退化情形。这样，只要设法令规模逐次减 1，即可将计算模型逐步加工成解的形式。这种加工过程可用“大事化小，小事化了”这句俗话来概括。

有些问题的“大事化小”过程似乎无法了结，这是一类无限的逼近过程，计算问题的规模通常是实数。如果逼近过程的规模（适当定义）按某个比例常数一致地缩减，那么适当提供某个精度即可控制计算过程的终止。这样设计出的算法通常称作**迭代法**。

13.3.1 开方算法

四千多年以前，在亚洲西南部的古巴比伦地区（现伊拉克境内）就已经萌发出数学智慧的幼芽。古巴比伦数学取得了一系列重要成就，譬如制成了有关开方值的数表。古巴比伦人制造开方表的方法难以考证，不过可以想象其计算方法必定相当简单。

现代电子计算机上又是怎样计算开方值的呢？

相对于加减乘除四则运算来说，开方运算无疑是复杂的。人们自然希望将复杂的开方运算归结为四则运算的重复，为此需要设计某种算法。

原书 PDF 页 312; 印刷页码 299。

[原页图像](#)

给定 $a > 0$, 求开方根 $\sqrt{a}$ 的问题就是要解方程

$$x^2 - a = 0. \quad (13.3.1)$$

这是个非线性的二次方程, 从初等数学的角度来看, 它的求解有难度。该如何化难为易呢?

设给定某个预报值 x_0 , 我们希望借助于某种简单方法确定校正量 Δx , 使校正值 $x_1 = x_0 + \Delta x$ 能够比较准确地满足所给方程 (13.3.1), 即有

$$(x_0 + \Delta x)^2 \approx a.$$

假设校正量 Δx 是个小量, 为简化计算, 舍弃上式中的高阶小量 $(\Delta x)^2$, 而令

$$x_0^2 + 2x_0\Delta x = a.$$

这是关于 Δx 的一次方程, 据此定出 Δx , 从而对校正值 $x_1 = x_0 + \Delta x$ 有

$$x_1 = \frac{1}{2} \left(x_0 + \frac{a}{x_0} \right).$$

反复施行这种预报校正手续, 即可导出开方公式

$$x_{k+1} = \frac{1}{2} \left(x_k + \frac{a}{x_k} \right), \quad k = 0, 1, 2, \dots \quad (13.3.2)$$

从给定的某个初值 $x_0 > 0$ 出发, 利用上式反复迭代, 即可获得满足精度要求的开方值 $\sqrt{a}$ 。

算法 13.2 (开方算法) 任给 $x_0 > 0$, 对 $k = 0, 1, 2, \dots$ 执行算式 (13.3.2), 直到偏差 $|x_{k+1} - x_k| < \varepsilon$ (ε 为给定精度) 为止, 最终获得的近似值 x_k 即为所求。

13.3.2 校正技术的设计机理

开方算法虽然结构简单, 但它深刻地揭示了校正技术的设计思想。前已指出, **算法设计的基本原则是将复杂计算化归为一系列简单计算的重复**。迭代法突出地体现了这一原则, 其设计机理可概括为“以简御繁, 逐步求精”。

所谓“以简御繁”是指构造某个简化方程近似替代原先比较复杂的方程, 以确定所给预报值的校正量。这种用于计算校正量的简化方程称作**校正方程**。关于校正方程有以下两项基本要求。

- (1) **逼近性**: 它与所给方程是近似的。逼近程度越高, 所获得的校正量越准确。
- (2) **简单性**: 校正方程越简单, 所需计算越小。求校正量通常采取显式计算。

应当指出的是, 在设计校正方程时, 上述逼近性与简单性两项要求往往会顾此失彼。逼近性高会导致校正方程的复杂化, 使计算量显著增加。在具体设计校正方程时需要权衡得失。

如何利用简单的校正方程获得原方程的解呢? 为使简单转化为复杂, 一种行之有效的方法是递推。对于给定的某个预报值 x_0 , 利用校正方程计算校正量, 从而得出校正值 x_1 , 这就完成了迭代过程的一步。是否需要继续迭代取决于校正量是否满足

原书 PDF 页 313；印刷页码 300。

[原页图像](#)

精度要求。如果不满足精度要求，则用老的校正值充当新的预报值重复上述步骤。如此继续下去，直到获得的校正值满足精度要求为止。由此可见，迭代过程是个“逐步求精”的递归过程。

13.4 迭代优化的超松弛技术

13.4.1 超松弛技术的设计机理

在实际计算中常常可以获得与目标值 F^* 相伴随的两个近似值 F_0 与 F_1 ，如何将它们加工成更高精度的结果呢？改善精度的一种简便而有效的办法是，取两者的某种加权平均作为近似值，即令

$$\begin{aligned}\widehat{F} &= (1 - \omega)F_0 + \omega F_1 \\ &= F_0 + \omega(F_1 - F_0).\end{aligned}$$

也就是说，适当选取权系数 ω 来调整校正量 $\omega(F_1 - F_0)$ ，以将近似值 F_0, F_1 加工成更高精度的结果 $\widehat{F}$ 。正是由于基于校正量的调整与松动，故这种方法称作**松弛技术**。权系数 ω 称作**松弛因子**。

有一种情况特别引人注目。如果所提供的一对近似值有优劣之分，譬如 F_1 为优而 F_0 为劣，这时往往采取如下松弛方式：

$$\widehat{F} = (1 + \omega)F_1 - \omega F_0, \quad \omega > 0.$$

这种设计策略称作**超松弛**。

超松弛技术在提高精度方面的效果是奇妙的。数据的加权平均几乎不需要耗费计算量，然而超松弛的结果往往能显著地提高精度。这种方法在将优劣互异（精度不同）的两类近似值进行松弛时，能最大限度地张扬优值而抑制劣值，从而获得高精度的松弛值，其设计机理可概括为“优劣互补，激浊扬清”。

需要指出的是，使超松弛技术真正实现提高精度的效果的关键在于松弛因子 ω 的选择，而这往往是极其困难的。不可思议的是，早在两千年以前，智慧的中华先贤已掌握了这门精湛的算法设计技术。

13.4.2 刘徽的“割圆术”

在数学史上，圆周率这个奇妙的数字牵动着一代又一代数学家的心，不少人为之耗费了毕生精力。据现存文献记载，在这方面作出过突出贡献的，当首推古希腊的 Archimedes。公元前 3 世纪，他用圆内接与外切正 96 边形逼近圆周，得出 π 的近似值

原书 PDF 页 314；印刷页码 301。

[原页图像](#)

为 3.14。这是公元前最好的结果。

关于圆周率的计算问题，我国古代数学家也作出过杰出的贡献。魏晋大数学家刘徽（公元 3 世纪）曾提出过“割圆术”，他从内接正 6 边形割到正 12 边形，再割到正 24 边形，如此一直割到内接正 3072 边形，得出 π 的近似值 3.1416。这是当时最好的结果。

刘徽不但提供了一种圆周率计算的迭代算法，而且还给出了一个加速逼近公式。这是刘徽割圆术中最为精彩的部分。

刘徽用内接正 n 边形的面积 S_n 来逼近面积 S ，并取半径 $r = 10$ 进行实际计算。这时 $S = 100\pi$ 。他利用 $S_{96} = 313\frac{584}{625}$ 与 $S_{192} = 314\frac{64}{625}$ 两个粗糙的数据进行加工，并提供了如下加工过程：

$$\begin{aligned} S_{192} + \frac{36}{105}(S_{192} - S_{96}) &= 314\frac{64}{625} + \frac{36}{105} \times \frac{105}{625} \\ &= 314\frac{100}{625} = 314\frac{4}{25} \approx S_{3072}. \end{aligned}$$

据此获知 $\pi = 3.1416$ 。

就这样，刘徽利用两个粗糙的近似值 S_{96}, S_{192} 进行松弛，结果获得了高精度的近似值 S_{3072} ，从而实现了圆周率计算的一次革命性飞跃。

刘徽的“割圆术”在数学史上占有重要的地位，它开创了加速算法设计的先河。直到 1700 多年后的 20 世纪，西方数学家才基于余项展开式获知逼近过程的加速方法。

13.5 递推加速的二分技术

13.5.1 “结绳记数”的快速算法

考古发现，上古没有文字，中华先民靠在绳索上打结的办法来记录数字，这就是所谓的“结绳记数”。

结绳记数有许多优点：绳结容易制作，造价低廉；打结方法简单，无需附加工具；绳结可随意变形，保管携带都很方便。绳结是个理想的“数字存储器”。

绳结可以用来记数，这是古人早就认识到的。问题在于，如果一条绳结上存有大量结点，逐个累加不但枯燥乏味，而且效率低下，那么该怎样快速地求出结点总数呢？熟知“刚柔相推，变在其中”（《周易·系辞》）的中华先贤自然早就知道，“软硬兼施”的绳结有一种快速计数方法。

设绳结上的结点是等距排列的。试将它对折为两半，并令其首尾两个结点对齐

原书 PDF 页 315；印刷页码 302。

原页图像

(见图 13.1)。设用阴阳二分点来刻画结点数是偶数还是奇数：若结点数为偶数，则二分点是个虚的结点，记作“●”；若结点数为奇数，则二分点是个实的结点，记作“○”。这种二分手续取得了“大事化小”的效果。

对二分后的子段又可继续施行上述二分手续，从而产生一个新的二分点●或○。这样反复做下去，直到子段上仅含一个结点为止。最终留下的一个结点自然视为二分点○。这样，二分过程最终实现了“小事化了”的目标。

绳结的这种二分演化过程如图 13.1 所示。

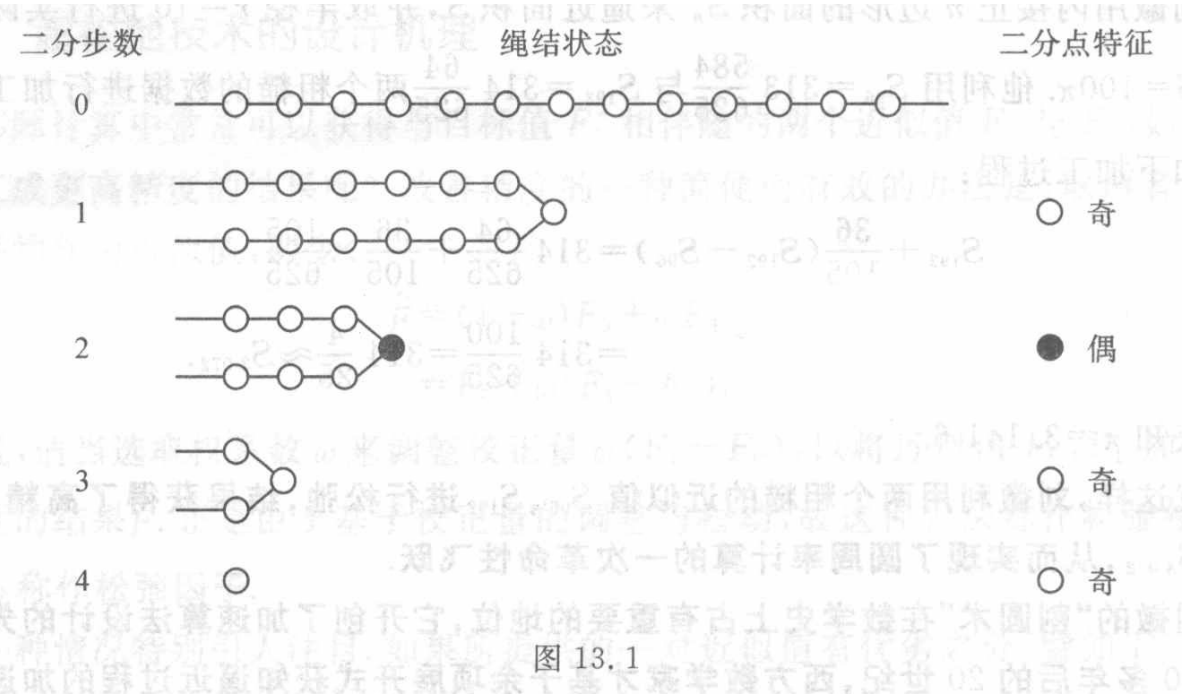


图 13.1

图意及标签转录（据原图）：原图三列为“二分步数”“绳结状态”“二分点特征”。第 0 步为包含 13 个空心结点的直绳；第 1 步将绳对折，两侧各 6 个空心结点，对折处为一个空心结点；第 2 步的两侧各 3 个空心结点，对折处标为实心圆；第 3 步的两侧各 1 个空心结点，对折处为一个空心结点；第 4 步仅余一个空心结点。二分点特征栏的全部标记为：

二分步数	二分点特征
0	(原图留空)
1	○奇
2	●偶
3	○奇
4	○奇

在上述二分演化过程中，逐步生成了一个二分点的序列。设将先后生成的二分点从右往左顺序排列，即可生成一个有方向、有层次的虚拟绳结（见图 13.2）。这个虚拟绳结同原先的绳结（见图 13.1）的含义一致，但信息量被大大地压缩了。

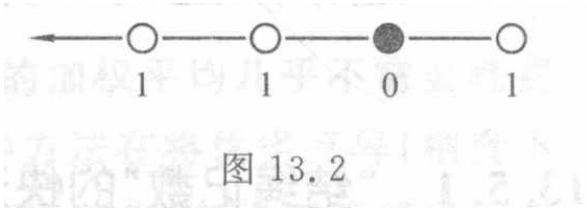


图 13.2

图意及标签转录（据原图）：箭头指向左方；沿水平绳从左到右的四个结点依次为○、○、●、○，其下数字依次为 1,1,0,1。

设原先的绳结数为 N ，则二分过程的信息压缩比

$$\frac{N}{\log_2 N} \rightarrow +\infty.$$

一个出人意料的结论是，如果将虚拟绳结的虚实结点●与○分别赋值 0 与 1，即可得到所求绳结数的二进制数 $(1101)_2 = 13$ 。

摆弄绳结可以孕育出二进制，这是一个奇妙的事实，这一事实表现了中华古算的博大精深。

13.5.2 二分技术的设计机理

二分技术是缩减技术的延伸。

原书 PDF 页 316；印刷页码 303。

原页图像

前已看到，在逐步推进过程中，缩减技术将所给计算问题加工成同类问题，每做一步其规模减 1，即其规模按等差级数（公差为 1）递减，直到规模为 1 或 0 结束。

二分技术所面对的是规模 $N = 2^n$ （ n 为正整数）的大规模计算问题。对于较大的 n ，数值 $N = 2^n$ 之“大”往往是人们无法想象的。譬如，1 s 只是短短的一瞬间，而 2^{64} s 等于 5849 亿年，这个数字远远超过宇宙的“年龄”200 亿年。对于这些“可怕的”大数 $N = 2^n$ ，运用每做一步规模减 1 的缩减技术实在太慢了。

作为缩减技术的优化，二分技术仍然是将计算问题加工成同类问题，不过，**二分技术的每一步使问题的规模减半，即其规模按等比级数（公比为 1/2）递减，直到规模变为 1 时终止计算**。这样，对于规模为 $N = 2^n$ （ n 为正整数）的大规模计算问题，只要二分 $n = \log_2 N$ 次即可使其规模变为 1，从而得出所求的解。与缩减技术相比，二分技术的加速比为

$$\frac{N}{\log_2 N} \rightarrow \infty.$$

可见，二分技术是一类高效算法的设计技术。

二分演化机制所涵盖的领域极为宽泛。不言而喻，近似值的优与劣自然是一对矛盾，因此，前述超松弛技术同样从属于二分演化机制的范畴。事实上，超松弛技术与二分技术是高效算法设计包括加速算法设计、快速算法设计及并行算法设计的基本技术。

小结

学习计算机上的数值算法，要领悟一条基本原理，区分两类基本方法，掌握四种基本技术。

计算机上数值算法设计技术大致有四种：缩减技术、校正技术、超松弛技术和二分技术，其设计机理与设计思想如表 13.1 所述。

表 13.1

设计技术	设计机理		设计思想
缩减技术	大事化小	小事化了	化大为小
校正技术	以简御繁	逐步求精	化难为易
超松弛技术	优劣互补	激浊扬清	变粗为精
二分技术	刚柔相推	变在其中	变慢为快

这四种技术并不是孤立的，它们彼此有着深刻的联系。二分技术是缩减技术的加速，而超松弛技术则是校正技术的优化。它们分别是直接法与迭代法的设计技术。

值得指出的是，直接法与迭代法这两类方法是相通的。如前所述，这两类算法本

原书 PDF 页 317；印刷页码 304。

[原页图像](#)

质上都是按照规模缩减的原则演化的，不过，直接法的规模是正整数，其规模缩减是个有限过程；而迭代法的规模（某种事先定义的误差）是实数，其规模缩减本质上是个无穷过程。前面已讲过，对于某些计算问题，所设计出的迭代法与直接法互为反方法。

计算机上的算法形形色色，但万变不离其宗。不管哪一种数值算法，其设计原理都是将复杂转化为简单的重复，或者说，通过简单的重复生成复杂。在算法设计与算法实现过程中，重复就是力量！

原书 PDF 页 318; 印刷页码 305。

[原页图像](#)

部分习题答案

第 1 章

1. δ 。
2. $0.02n$ 。
4. 1.05×10^{-3} , 0.215 , 10^{-5} 。
6. $\frac{1}{2} \times 10^{-3}$ 。
9. 边长误差小于 0.005 cm 。
11. 不稳定, $\varepsilon^*(y_{10}) \approx \frac{1}{2} \times 10^8$ 。
13. 0.3×10^{-2} , 0.834×10^{-6} 。

第 2 章

2. $5x^2/6 + 3x/2 - 7/3$ 。
3. $-0.620\,219$, $-0.616\,707$ 。
4. 1.06×10^{-8} 。
5. $(10 + 7\sqrt{7})/27$ 。
8. $h \leq 0.006$ 。
9. $\Delta^4 y^n = 2^n$, $\delta^4 y_n = 2^{n-2}$ 。
16. 1 , 0 。
18. $P(x) = x^2(x-1)^2/4$ 。
21. $|R_1(x)| \leq h^2/4$ 。
22. $|R_4(x)| \leq h^4/16$ 。

第 3 章

3. $P_6(x) = 0$ 。
4. $P(x) = (M+m)/2$ 。
5. $a = 3/4$ 。

6. $P_1(x) = \frac{2}{\pi}x + 0.105\,256\,8。$

8. $r = -1/2。$

9. $P_3(x) = 5x^3 - \frac{5}{4}x^2 + \frac{1}{4}x - \frac{129}{128}。$

13. $a = 0.664\,438\,9, b = 0.114\,770\,7。$

15. $0.196\,116\,1$, 用中值定理 $\frac{1}{14} < \int_0^1 \frac{x^6}{1+x} dx < \frac{1}{7}。$

16. $a = 0, |a| \leqslant 1。$

18. $S(x) = 0.117\,187\,5 + 1.640\,625x^2 - 0.820\,312\,5x^4。$

第 4 章

1. (1) $A_{-1} = A_1 = h/3, A_0 = 4h/3$, 三次代数精度;

(2) $A_{-1} = A_1 = 8h/3, A_0 = -4h/3$, 三次代数精度;

(3) $x_1 = -0.289\,90, x_2 = 0.626\,60$ 或 $x_1 = 0.689\,90, x_2 = -0.126\,60$, 二次代数精度;

(4) $a = 1/12$, 三次代数精度。

2. (1) $T_8 = 0.111\,40, S_4 = 0.111\,57$; (2) $T_1 = 1.391\,48, S_5 = 1.454\,71$;

(3) $T_4 = 17.227\,74, S_2 = 17.322\,22$; (4) $T_6 = 1.035\,62, S_3 = 1.035\,77。$

4. $S_1 = 0.632\,33$, 误差 $0.000\,35。$

7. $n > \sqrt{\frac{(b-a)^3 M}{12\varepsilon}}, M = \max_{a \leqslant x \leqslant b} |f''(x)|。$

8. $0.713\,27。$

9. $48\,708\text{ km}。$

11. (1) $1.098\,63$; (2) $1.089\,40, 1.098\,62$; (3) $1.098\,54。$

校注 (agent 补充, BACK-S001): 第 2 章第 9 题第一式原扫描排为 $\Delta^4 y^n = 2^n$, 其中 y 后的 n 位于上标; 第二式为 $\delta^4 y_n = 2^{n-2}$ 。原题 (PDF 56, 书页 43) 明确给出 $y_n = 2^n$, 所求为 $\Delta^4 y_n$ 及 $\delta^4 y_n$, 因此答案第一式的索引与题目不一致。四阶前向差分为 $2^n(2-1)^4 = 2^n$; 四阶中心差分为 $2^n(4-8+6-2+1/4) = 2^{n-2}$ 。据此, 第一式应使用下标 y_n ; 此处转录仍保留原书上标。

校注 (agent 补充, BACK-S002): 第 4 章第 2 题 (2) 原扫描清楚排为 T_1 。原题 (PDF 117, 书页 104) 明确指定 $n = 10$, 故梯形公式的下标与题目不一致, 疑应为 T_{10} 。此处保留原书 T_1 。本校注只核定下标问题, 未验证所列 $1.391\,48$ 与 $1.454\,71$ 的数值正确性。

原书 PDF 页 319; 印刷页码 306。

[原页图像](#)

12. 用三点公式求得一阶导数值 -0.247 , -0.217 , -0.189 , 用五点公式得 -0.2483 , -0.2163 , -0.1883 。

第 5 章

2. 1.11, 1.242 05, 1.398 47, 1.581 81, 1.794 90, 2.040 86, 2.323 15, 2.645 58, 3.012 37, 3.428 17。

3. 0.145。

5. 0.500, 1.142, 2.501, 7.245。

6. (1) 1.242 8, 1.583 6, 2.044 2, 2.651 0, 3.436 5;

(2) 1.727 6, 2.743 0, 4.094 2, 5.829 2, 7.996 0。

9. 显式 0.626, 隐式 0.633, 真值 0.632 1。

11.

$$\begin{aligned}a_0 + a_1 + a_2 &= 1, \\-a_1 - 2a_2 + b_0 + b_1 + b_2 &= 1, \\a_1 + 4a_2 - 2b_1 - 4b_2 &= 1, \\-a_1 - 8a_2 + 3b_1 + 12b_2 &= 1.\end{aligned}$$

14. 0.49, 0.96, 1.36。

15. 1.014 87, 1.017 85, 1.070 10, 1.210 30, 1.513 29。

第 6 章

2. 0.512。

3. (1) 和 (2) 收敛; (3) 发散。

4. (1) 二分十四次得 0.090 545 6; (2) 迭代五次得 0.090 526 4。

6. (2) 4.493 42。

11. (1) 发散; (2) 一阶收敛。

14. $-\frac{n-1}{2\sqrt[n]{a}}, \frac{n+1}{2\sqrt[n]{a}}$ 。

15. $\frac{1}{4a}$ 。

第 7 章

1. 计算中每步 4 位小数。结果为

$$(1) \ x = (-0.167\,0, -1.650\,4, 2.196\,7, -0.446\,8)^T;$$

$$(2) \ x = (-0.183\,3, -1.663\,8, 2.218\,5, -0.446\,3)^T.$$

计算机上用列主元素消去法计算结果为

$$x = (-0.181\,918\,7, -1.663\,031, 2.217\,229, -0.446\,704\,1)^T.$$

9.

$$\begin{cases} l_{i1} = a_{i1} & (i = 1, 2, \dots, n), \\ u_{1j} = a_{1j}/l_{11} & (j = 2, 3, \dots, n), \\ l_{ik} = a_{ik} - \sum_{r=1}^{k-1} l_{ir}u_{rk} & (i = k, k+1, \dots, n), \\ u_{kj} = \left(a_{kj} - \sum_{r=1}^{k-1} l_{kr}u_{rj} \right) / l_{kk} & (j = k+1, k+2, \dots, n). \end{cases}$$

10. (1) 设 U 为上三角阵

$$x_n = b_n/u_{nn}, \quad x_i = \left(b_i - \sum_{j=i+1}^n u_{ij}x_j \right) / u_{ii} \quad (i = n-1, n-2, \dots, 1);$$

原书 PDF 页 320; 印刷页码 307。

[原页图像](#)

(2) $n(n+1)/2$; (3) 记 U^{-1} 的元素为 s_{ij} , U 的元素记作 u_{ij} , 有

$$\begin{cases} s_{ii} = 1/u_{ii} & (i = 1, 2, \dots, n), \\ s_{ij} = -\sum_{k=i+1}^j u_{ik}s_{kj}/u_{ii} & (i = n-1, n-2, \dots, 1; j = i+1, i+2, \dots, n). \end{cases}$$

11.

$$A^{-1} = \begin{pmatrix} -0.047\,058\,85 & 0.588\,235\,3 & -0.270\,588\,2 & -0.941\,176\,4 \\ 0.388\,235\,3 & -0.352\,941\,2 & 0.482\,352\,9 & 0.764\,705\,8 \\ -0.223\,529\,4 & 0.294\,117\,7 & -0.035\,294\,12 & -0.470\,588\,2 \\ -0.035\,294\,12 & -0.058\,823\,53 & 0.047\,058\,82 & 0.294\,117\,6 \end{pmatrix}.$$

13. (1) $\beta_1 = -1/2$, $\beta_2 = -2/3$, $\beta_3 = -3/4$, $\beta_4 = -4/5$;

(2) 解 $Ly = f$, $y = (1/2, 1/3, 1/4, 1/5, 1/6)^T$;

(3) 解 $Ux = y$, $x = (5/6, 2/3, 1/2, 1/3, 1/6)^T$ 。

14. $x = (1.111\,11, 0.777\,78, 2.555\,56)^T$ 。

15. (1) A 不能分解为三角阵的乘积, 但换行后可以; (2) B 可以但不唯一, C 可以且唯一。

18. $\|A\|_\infty = 1.1$, $\|A\|_1 = 0.8$, $\|A\|_2 = 0.825$, $\|A\|_F = 0.842\,6$ 。

31. $\text{cond}(A)_\infty = 39\,601$, $\text{cond}(A)_2 = 39\,206$ 。

第 8 章

1. (1) 两种方法均收敛;

(2) 用 Jacobi 迭代法迭代 18 次, $x^{(18)} = (-3.999\,996\,4, 2.999\,973\,9, 1.999\,999\,9)^T$, 用 Gauss-Seidel 迭代法迭代 8 次, $x^{(8)} = (-4.000\,036, 2.999\,985, 2.000\,003)^T$ 。

5. (1) Jacobi 迭代法不收敛, Gauss-Seidel 迭代法收敛;

(2) Jacobi 迭代法收敛, Gauss-Seidel 迭代法不收敛。

8. (1) $\rho(B_0) = 0.5$; (2) $\rho(G) = 0.25$; (3) 两种方法均收敛。

9. $\omega = 1.03$ 时迭代五次达到精度要求

$$x^{(5)} = (0.500\,004\,3, 0.100\,000\,1, -0.499\,999\,9)^T,$$

$\omega = 1$ 时迭代 6 次达到精度要求

$$x^{(6)} = (0.500\,003\,8, 0.100\,000\,2, -0.499\,999\,5)^T,$$

$\omega = 1.1$ 时迭代 6 次达到精度要求,

$$x^{(6)} = (0.500\,003\,5, 0.999\,998\,9, -0.500\,000\,3)^T.$$

10. $\omega = 0.9$, 迭代 8 次时达到精度要求,

$$x^{(8)} = (-4.000\,027, 0.299\,998\,9, 0.200\,000\,3)^T.$$

13. (1) $\rho(A^{-1}B) < 1$; (2) $\rho((A^{-1}B)^2) < 1$; (3) 迭代法收敛速度是 a 的 2 倍。

第 9 章

1. (1) 取 $v_0 = (1, 1, 1)^T$, $\lambda_1 \approx 9.605\,8$, $x_1 \approx (1, 0.605\,6, -0.394\,5)^T$;

(2) 取 $v_0 = (1, 1, 1)^T$, $\lambda_1 \approx 8.869\,51$, $x_1 \approx (-0.604\,22, 1, 0.150\,94)^T$ 。

9. (2) A 的特征值为 $\lambda_1 = 2 + \sqrt{3}$, $\lambda_2 = 2$, $\lambda_3 = 2 - \sqrt{3}$ 。选取位移 $s_k = a_{33}^{(k)}$,

$$A_s = \begin{pmatrix} 3.731\,692\,597\,4 & 0.024\,906\,021\,0 & 0.0 \\ 0.024\,906\,021\,0 & 2.000\,358\,210\,2 & \varepsilon \\ 0.0 & \varepsilon & 0.267\,949\,192\,4 \end{pmatrix},$$

其中 $|\varepsilon| < 5 \times 10^{-11}$ 。

校注(agent 补充, BACK-S003): 第 8 章第 9 题原题(PDF 231, 书页 218)明确给出精确解 $x^* = (1/2, 1, -1/2)^T$, 并要求无穷范数误差小于 5×10^{-6} 。答案中 $\omega = 1.03$ 、 $\omega = 1$ 两组的第二分量分别印为 0.100 000 1、0.100 000 2, 两组原印向量的无穷范数误差分别为 0.899 999 9、0.899 999 8, 不满足题意, 疑似小数点排印错误; 原值在正文中保留。 $\omega = 1.1$ 原印向量的无穷范数误差为 3.5×10^{-6} , 符合原题要求, 其第二分量 0.999 998 9 不能列为此处疑误。本校注直接比较原印向量与题给精确解, 未声称复现原书所有迭代尾数。

校注(agent 补充, BACK-S004): 第 8 章第 10 题原题(PDF 231, 书页 218)的方程组为 $5x_1 + 2x_2 + x_3 = -12$ 、 $-x_1 + 4x_2 + 2x_3 = 20$ 、 $2x_1 - 3x_2 + 10x_3 = 3$, 其精确解为 $(-4, 3, 2)^T$ 。答案原页的第二、第三个分量分别清楚排为 0.299 998 9 和 0.200 000 3, 量级与该解不符, 疑似小数点排印错误。此处保留原值, 不据此推定应替换的全部尾数。

原书 PDF 页 321；印刷页码 308。

[原页图像](#)

参考文献

- [1] WILKINSON J H. Rounding Errors in Algebraic Process[M]. London: H M Stationery Office, 1963.
- [2] MOORE R E. Interval Analysis[M]. New Jersey: Prentice-Hall, 1966.
- [3] 李岳生, 齐东旭. 样条函数方法 [M]. 北京: 科学出版社, 1979.
- [4] 纳唐松 И И. 函数构造论: 上册 [M]. 徐家福, 郑维行, 译. 北京: 科学出版社, 1958.
- [5] 纳唐松 И И. 函数构造论: 下册 [M]. 何旭初, 唐述钊, 译. 北京: 科学出版社, 1959.
- [6] 王仁宏. 数值有理逼近 [M]. 上海: 上海科技出版社, 1980.
- [7] 赵访熊, 李庆扬. 傅里叶变换滤波在地震勘探数字处理中的应用 [J]. 清华大学学报, 1978 (4): 1-14.
- [8] 黄友谦, 李岳生. 数值逼近 [M]. 2 版. 北京: 高等教育出版社, 1987.
- [9] 徐利治, 周蕴时. 高维数值积分 [M]. 北京: 科学出版社, 1980.
- [10] 清华大学、北京大学计算方法编写组. 计算方法 [M]. 北京: 科学出版社, 1980.
- [11] 斯图尔特 G W. 矩阵计算引论 [M]. 王国荣, 黄丽萍, 译. 上海: 上海科学技术出版社, 1980.
- [12] 冯康, 等. 数值计算方法 [M]. 北京: 国防工业出版社, 1978.
- [13] STOER J, BULIRSCH R. Introduction to Numerical Analysis[M]. New York: Springer-verlag, 1980.
- [14] 拉尔斯登 A, 维尔夫 H S. 数字计算机上用的数学方法: 第二卷 [M]. 本书翻译组, 译. 上海: 上海人民出版社, 1976.
- [15] 高尔腊依 A R, 瓦特桑 G A. 矩阵特征问题的计算方法 [M]. 唐焕文, 等, 译. 上海: 上海科学技术出版社, 1980.
- [16] 奥特加 J M. 数值分析 [M]. 张丽君, 张乃玲, 译. 北京: 高等教育出版社, 1983.
- [17] 瓦格 R S. 矩阵迭代分析 [M]. 蒋尔雄, 游兆永, 张玉德, 译. 上海: 上海科学技术出版社, 1966.
- [18] 王能超. 千古绝技“割圆术”——刘徽的大智慧 [M]. 2 版. 武汉: 华中科技大学出版社, 2003.
- [19] 王能超. 算法演化论 [M]. 北京: 高等教育出版社, 2008.
- [20] 王能超, 王学东. 简易数值分析 [M]. 武汉: 华中科技大学出版社, 2017.

原书 PDF 页 322；无印刷页码。

[原页图像](#)

普通高等教育“十三五”规划教材

普通高等院校数学精品教材



标志文字：十三五。书目区域背景另有淡色装饰文字“十三五”。

左栏	右栏
◇数值分析（第 5 版）	◇简易数值分析
◇高等数学基础与应用	◇概率论与数理统计
◇复变函数与积分变换（第二版）	◇线性代数简明教程
◇微积分 =Calculus. I: 英文	◇微积分 =Calculus. II: 英文

- ◇线性代数 =Linear Algebra：英文
- ◇概率论与数理统计 =Probability and Statistics：英文
- ◇高等数学解题方法技巧归纳（上册）（同济·七版配套）
- ◇高等数学解题方法技巧归纳（下册）（同济·七版配套）
- ◇线性代数解题方法技巧归纳（同济·六版配套）
- ◇概率论与数理统计解题方法技巧归纳（浙大·四版配套）
- ◇经济数学（微积分）解题方法技巧归纳（人大版赵树嫄·三版配套）
- ◇经济数学（线性代数）解题方法技巧归纳（人大版赵树嫄·四版配套）
- ◇经济数学（概率论与数理统计）解题方法技巧归纳（人大版袁荫棠·修订本配套）

策划编辑 王汉江

责任编辑 王汉江

ISBN 978-7-5680-3946-8

条码数字：9 787568 039468

定价：39.80 元

